RStatsbook

Andreas Busjahn

2026-07-13

Table of contents

Preface 9

1 Introduction 10
- 1.1 Installation of R . . . 10
- 1.2 Installation of RStudio Desktop (UI) . . . 10
- 1.3 Installation of Quarto (Reportgenerator) . . . 10
- 1.4 Installation of git: . . . 10
- 1.5 Setup RStudio . . . 11
- 1.6 Projects on your computer . . . 12
  - 1.6.1 Option 1: Create manually . . . 12
  - 1.6.2 Option 2: Create from a github repository . . . 13

2 Some useful resources 14
- 2.1 Books . . . 14
- 2.2 Ressorces . . . 14

3 Syntax rules / basic things to know about R 15
- 3.1 Script preparation / basic setup . . . 15
- 3.2 Numeric operations . . . 16
- 3.3 Variables . . . 17
  - 3.3.1 Variable names . . . 17
  - 3.3.2 Basic classes of data . . . 17
  - 3.3.3 Exercise: Variables / Assignments . . . 21
- 3.4 Factors . . . 22
  - 3.4.1 Exercise: Factors . . . 24
- 3.5 Forcing / casting classes . . . 26
- 3.6 Indexing variables . . . 28
- 3.7 Usage of variables . . . 29
- 3.8 Functions . . . 30
  - 3.8.1 Function usage . . . 30
  - 3.8.2 Functions combined . . . 32
  - 3.8.3 Writing functions . . . 34
- 3.9 More complex data types, created by functions . . . 36
  - 3.9.1 Matrix . . . 36
  - 3.9.2 Data frame . . . 40

3.9.3 Tibble . . . 43
3.9.4 List . . . 56
3.10 Control structures . . . 59
3.10.1 Loops . . . 59
3.10.2 Conditions . . . 63
3.10.3 Exercise: `for` and `while` loops, and `if`/`ifelse`/`case_xxx` . . . 68

4 Regular expressions **70**
4.1 Intro . . . 70
4.2 Some basic examples: . . . 71
4.2.1 Finding something . . . 71
4.2.2 Replacing something . . . 71
4.3 An example for their use in renaming variables: . . . 73
4.4 Exercise . . . 74

5 Importing data **75**
5.1 Import from text files (.txt, .csv) . . . 75
5.2 Import from Excel . . . 76
5.2.1 Tidy Excel files . . . 76
5.2.2 Excel file with units row . . . 76
5.3 ODS files . . . 78
5.4 Import from SPSS/SAS . . . 78

6 Changing structure wide <–> long **81**
6.1 Example 1: single repeated measure . . . 82
6.2 Example 2: several repeated measures . . . 82
6.3 Example 3: long to wide . . . 85
6.4 More examples . . . 85
6.4.1 Step 1: Create example data: . . . 85
6.4.2 Step 2: Transform that data to a long form: . . . 86
6.4.3 Step 3 Transform long to wide . . . 87
6.5 Exercise: Reshaping Data . . . 89
6.5.1 Part 1: From Long to Wide (`pivot_wider()`) . . . 89
6.5.2 Part 2: From Wide to Long (`pivot_longer()`) . . . 90

7 Visualize data with ggplot **91**
7.1 Example data . . . 91
7.2 Basic structure of a ggplot call . . . 92
7.3 fill vs. color . . . 97
7.4 Color systems . . . 99
7.4.1 External color definitions from ggsci . . . 102
7.5 Exporting ggplots . . . 105
7.6 Other geoms . . . 105

7.7 Exercise 1 . . . 118
7.8 Combining and finetuning aesthetics . . . 118
7.9 Positioning elements . . . 125
7.10 Order of layers . . . 131
7.11 Local aesthetics for layers . . . 136
7.12 Faceting (splitting) plots . . . 138
7.12.1 facet_grid . . . 138
7.12.2 facet_wrap . . . 142
7.12.3 Controlling scales in facets (default: scales = "fixed") . . . 145
7.13 Exercise 2 . . . 150
7.14 Showing summaries . . . 150
7.15 Indicating significances . . . 156
7.16 Theme definitions / changes . . . 161
7.17 Combining figures with patchwork . . . 167
7.18 ggplots in loops . . . 169

8 Grouping of variables by type / distribution / use **175**
8.1 Test for Normal distribution . . . 175
8.1.1 Testing a single variable . . . 175
8.1.2 Testing several variables . . . 182
8.2 Exercise: Distribution of penguin measures . . . 189
8.3 Picking column names and positions . . . 189

9 Descriptive statistics **191**
9.1 Typical descriptives . . . 191
9.2 Reading in data . . . 191
9.3 Graphical exploration should start before descriptive statistics . . . 192
9.4 Gaussian variables . . . 192
9.5 A little theory . . . 192
9.5.1 Simple function calls . . . 193
9.5.2 Combined reporting . . . 194
9.6 Ordinal variables . . . 195
9.7 Categorical variables . . . 197
9.8 Summarize data . . . 200
9.9 Exercise . . . 206

10 Summarize / across **207**

11 Simple test statistics **217**
11.1 Tests require hypotheses . . . 218
11.1.1 Null hypothesis ? . . . 219
11.2 Quantitative measures with Gaussian distribution . . . 219
11.3 Ordinal data . . . 230

11.4 Categorial data . . . 233
11.5 Exercise . . . 238

12 Covariance / Correlation **239**
12.1 Covariance . . . 239
12.2 Correlation . . . 239
12.3 Vizualizations . . . 242

13 Intro to linear models **247**
13.1 Setup . . . 247
13.2 Import / Preparation . . . 247
13.3 Graphical exploration . . . 248
13.4 Linear Models . . . 253
13.4.1 Linear regression . . . 253
13.4.2 ANOVA . . . 259
13.4.3 LM with continuous AND categorical IV . . . 264
13.5 compare_n_numvars() as reporting option . . . 271
13.6 Model exploration with package performance . . . 273
13.7 Exercise . . . 278

14 Interaction in linear models **279**
14.1 No age effect, no treatment effect, interaction treatment with agegroup . . . 279
14.2 Age effect, no treatment effect, interaction treatment with agegroup . . . 283
14.3 Age effect, treatment effect, interaction treatment with agegroup . . . 289
14.4 Interaction in regression-like lm . . . 296
14.5 How to specify interaction in multivariable models . . . 299

15 Logistic regression **303**
15.1 Odds vs. probability . . . 303
15.2 Data preparation . . . 304
15.3 Exploration . . . 306
15.4 Build model . . . 309
15.5 Univariable model . . . 309
15.6 Multivariable model . . . 311
15.7 Create structure for ggplot . . . 314
15.8 Create forest plot . . . 315
15.9 Create predictions . . . 316
15.10Regression tree as alternative to glm . . . 321

16 Linear Mixed Models **327**
16.1 Import / Preparation . . . 327
16.2 Random intercept model . . . 329
16.2.1 Package nlme . . . 329

16.2.2 Package lme4 . . . 329
16.2.3 Visualization of fixed and random effects . . . 331
16.3 Random slope model . . . 334
16.3.1 Visualization of fixed and random effects . . . 335
16.4 Exercise: . . . 338

17 Machine Learning with R: Basic concepts 339
17.1 Literature links . . . 339
17.2 structured vs. unstructured data . . . 339
17.3 supervised vs. unsupervised methods . . . 339
17.4 Resampling methods . . . 339
17.4.1 Permutation tests . . . 340
17.4.2 Bootstrapping . . . 343
17.4.3 Cross validation . . . 348
17.4.4 Jacknife . . . 350

18 Markov Chain Monte Carlo: Metropolis algorithm simulation 352
18.1 Rule definition . . . 353
18.2 Data structures for simulation . . . 354
18.3 Simulation . . . 354
18.4 Results . . . 355

19 k nearest neighbors knn 371
19.1 Data preparation . . . 371
19.2 Exploratory plots . . . 372
19.3 Scaling . . . 374
19.3.1 Center + scale (z-score normalization) . . . 374
19.3.2 Range transformation . . . 374
19.3.3 Percentile transformation . . . 374
19.3.4 Visual comparison of the scaled data . . . 375
19.4 Modelling . . . 378
19.4.1 Definition of predictor variables (IV) . . . 378
19.4.2 Data splitting . . . 378
19.4.3 Model fitting . . . 378
19.4.4 Model evaluation . . . 380
19.5 More complex model . . . 383

20 Regression and classification trees 388
20.1 Graphical exploration . . . 388
20.2 Modelling . . . 389
20.2.1 Direct call to rpart . . . 390
20.2.2 Tidy-models approach . . . 391
20.2.3 Visualize the tuning results . . . 393

20.3 Model evaluation . . . 397

21 Random forests **401**
21.1 Modelling . . . 402
21.2 See the top performing models . . . 402
21.3 Variable importance . . . 404
21.4 Model evaluation . . . 405

22 Boosted regression trees **407**
22.1 Direct modelling . . . 407
22.1.1 Preprocessing . . . 407
22.1.2 Modelling . . . 408
22.2 Tidy model approach . . . 409
22.2.1 Fit the model to the data . . . 410
22.2.2 See the top performing models . . . 410
22.2.3 Select the best parameters based on your preferred metric . . . 411
22.2.4 Visualize the tuning results . . . 411
22.2.5 Model evaluation . . . 413

23 Linear discriminant analysis LDA **417**

24 tidymodels framework **424**
24.1 Step 1: Pre-processing . . . 425
24.2 Step 2: Defining the model using Parsnip package . . . 425
24.2.1 1. Multinomial Regression (`multinom_reg`) . . . 425
24.2.2 2. Decision Trees (`decision_tree`) . . . 426
24.2.3 3. Random Forests (`rand_forest`) . . . 427
24.2.4 4. K-Nearest Neighbors (`nearest_neighbor`) . . . 428
24.2.5 5. Boosted Trees (`boost_tree` via XGBoost) . . . 429
24.2.6 6. C5.0 Decision Trees (`boost_tree` via C5.0) . . . 430
24.2.7 7. Linear Discriminant Analysis (`discrim_linear`) . . . 431
24.2.8 8. Support Vector Machines (`svm_poly` / `svm_rbf`) . . . 431
24.2.9 9. Naive Bayes (`naive_Bayes`) . . . 432
24.2.10 10. Multilayer Perceptron / Neural Networks (`mlp`) . . . 432
24.2.11 11. Logistic Regression with Elastic Net (`logistic_reg`) . . . 433
24.3 Step 3: Competitive Model Evaluation (Using `workflow_set`) . . . 433
24.4 Step 4: Cross-Validation . . . 434
24.5 Step 5: Ranking the models . . . 436
24.6 Step 6: Best model selection . . . 439

25 Principal Components Analysis **441**
25.1 Exploration of correlations between predictor variables . . . 441
25.2 Two variable example . . . 445

25.3 tidymodels approach 452
25.4 base approach 455

26 Cluster analysis **468**
26.1 kmeans 468
26.2 HClust 475
26.3 Unified from easystats 482

Preface

This booklet is meant as companion to my R / statistics seminars. It is NOT a complete guide to either R or statistical data analysis.

1 Introduction

Chapters will follow my usual schedule, starting from R basics, introducing ggplot, data import, data preparation and cleaning, descriptive statistics, simple test statistics, then progression to linear models (regression/ANOVA), generalized linear models with logistic regression as example, linear mixed effect models, and machine learning.

But first things first, installation of necessary and useful tools and setup of our environment.

1.1 Installation of R

The Comprehensive R Archive Network (r-project.org)

If using Windows, install Rtools as well (3 GB)

If on linux, R2U is useful:

CRAN as Ubuntu Binaries - r2u (eddelbuettel.github.io)

1.2 Installation of RStudio Desktop (UI)

RStudio Desktop - Posit

1.3 Installation of Quarto (Reportgenerator)

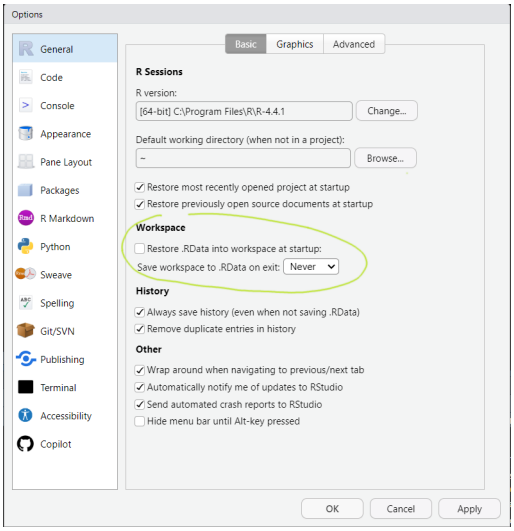
Quarto - Get Started

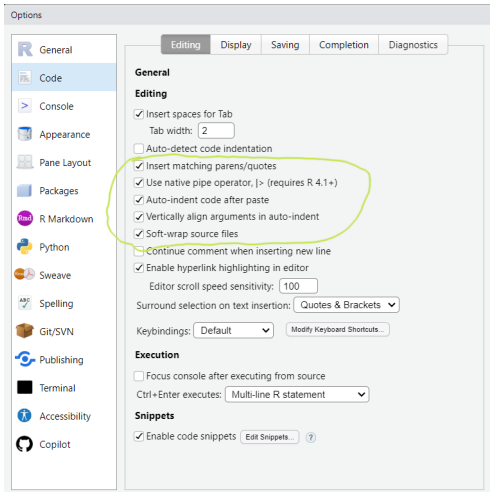
1.4 Installation of git:

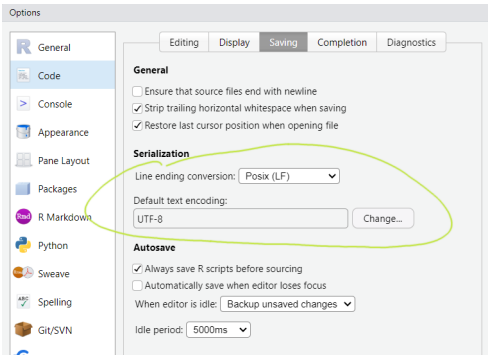
Git - Downloads (git-scm.com)

1.5 Setup RStudio

There are many options available to adjust the UI to your liking, some default settings should be changed. There are global and project specific options, both can be found under Menu /Tools. Some useful (IMHO) changes are highlighted:







There are MANY more settings to adjust RStudios appearance and behavior to your liking.

1.6 Projects on your computer

It is terribly important (or at least helpful) to be organized ! Different analyses for e.g. new experiments or new data sets should be separated, demo analyses and exercises as well. Project structures should be consistent to make re-use of scripts easier.

As a starting point, I suggest creating a folder for everything R-related, e.g. ***Rstuff***

1.6.1 Option 1: Create manually

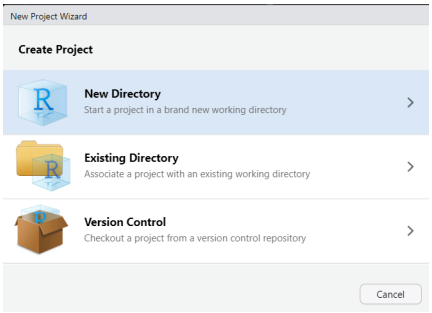
- Define location for project folder (often somewhere under C:/Users/SomeName/Documents),
  for seminar projects this should be the folder you just created, Rstuff or whatever name you used.
- In that folder, create a new folder, e.g. Rexercises
- Create useful sub-folders, I recommend /RScripts and /Data as minimal structure; be consistent in naming!!

You should have a folder structure something like this:

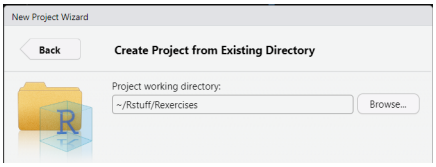
- Rstuff
  - Rexercises
    - RScripts
    - Data
  - SomeProject

* RScripts
* Data

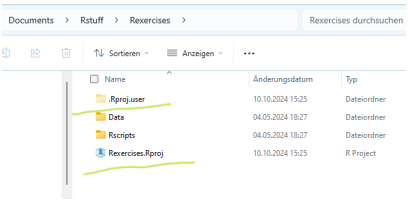
Now you are ready to create a project in RStudio:



As there is an existing directory, this is what you select.



In this new window you browse to the root folder of your project and create your project. This will create new (possibly hidden) entries:



1.6.2 Option 2: Create from a github repository

e.g. https://github.com/abusjahn/RStatsbook

Here you have to define source and new location.

2 Some useful resources

2.1 Books

There are plenty of books available, check out e.g.

R for Data Science

Modern Statistics for Modern Biology

Modern R with the tidyverse

ggplot2: Elegant Graphics for Data Analysis

The big book of R

Fundamentals of Data Visualization (clauswilke.com)

Statistical Inference via Data Science (moderndive.com)

Data Science Live Book (datascienceheroes.com)

Biostatistics for Biomedical Research (hbiostat.org)

Welcome | Data Science at the Command Line, 2e (jeroenjanssens.com)

2.2 Ressorces

Cheatsheets - Posit

Contributed Cheatsheets (rstudio.github.io)

R-bloggers

Bioconductor - Home

3 Syntax rules / basic things to know about R

This is going to be the boring technical stuff... We'll get to the more interesting topics in the next chapters.

As a general structure, I'll try to make a distinction between more basic solutions to a task and rather 'cosmetic' touches.

3.1 Script preparation / basic setup

At the beginning of (almost) every script we define packages to be used. This could be done by either

- checking if packages needed are installed and otherwise do so, followed by function `library(packagename)`

OR

- simplifying this using function `p_load()` from package `pacman`; if you want to create fool-proof scripts, check for `pacman` and install if needed.

```
# ↑ this is the head of a code chunk
#| output: false
#| message: false
Sys.setenv(LANG = "en_EN.UTF-8") # to get errors/warnings in English
if (!requireNamespace("pacman", quietly = TRUE)) {
  install.packages("pacman")
}
# library("pacman") adds all functions from package to namespace
# use("pacman", "p_load") # starting with R 4.5.0, selected functions can be extracted / put
pacman::p_load(
  conflicted, # tests/solutions for name conflicts
  tidyverse, # metapackage
  wrappedtools, # my own tools package
  randomNames # used to create pseudo names
)
conflict_scout()
```

```
3 conflicts

* `filter()`: dplyr and stats

* `lag()`: dplyr and stats

* `mean_cl_boot()`: wrappedtools and ggplot2
```

```
conflicts_prefer(
  dplyr::filter,
  stats::lag
)
```

```
[conflicted] Will prefer dplyr::filter over any other package.

[conflicted] Will prefer stats::lag over any other package.
```

3.2 Numeric operations

```
2 + 5
```

```
[1] 7
```

```
3 * 5
```

```
[1] 15
```

```
15 / 3 # not 15:3!!, would create vector 15,14,13 ... 3
```

```
[1] 5
```

```
3^2
```

```
[1] 9
```

```
9^0.5
```

```
[1] 3
```

```
10 %% 3 # modulo
```

```
[1] 1
```

3.3 Variables

3.3.1 Variable names

Naming things is harder than you may expect. Try to be verbose and consistent in language and style. Commonly used are snake_case_style, CamelCaseStyle, and kebab-case-style.

Decide about computer-friendly (syntactical) or human-friendly names, illegal names can be used inside backticks: ‘measure [unit]‘. My preference is syntactical for script variables and humane for data variables, e.g. column names, print labels etc.

There are rules for valid syntactical names:

- UPPERCASE and lowercase are distinguished
- start with letter or symbols as `.__` , but not with a number
- no mathematical symbols or brackets allowed

To store some value into a variable, use the assignment operator `<-` ; while it possible to use ** = ** or `->` , this is rather unusual. Assignments are silent, so either a call of the variable, or `print()` / `cat()` function are needed to inspect. Alternatively, put brackets around assignment: (varname <- content).

```
test <- 1 # putting spaces around symbols like <- is good style
test1 <- 1
# 1test <- 2 # wrong, would result in error
`1test` <- 2 # this would be possible
test_1 <- 5
test.1 <- 2
`test-1` <- 6
`test(1)` <- 5
Test <- "bla"
HereAreFilteredData <- "" # CamelCase
here_are_filtered_data <- "test" # snake_case
`Weight [kg]` <- 67
```

3.3.2 Basic classes of data

R is ‘guessing’ the suitable type of data from input. This should be checked after e.g. importing data! If elements of different classes are found, the more inclusive is used. There are functions to change / force a type if needed.

The **class()** function returns the class of an object, which determines how it behaves with respect to functions like **print()**. The class of an object can be changed by using generic functions and methods.

The **typeof()** function returns the basic data type of an object, which determines how it is stored in memory. The basic data type of an object cannot be changed.

The **str()** function shows class and examples of an object.

3.3.2.1 Guessed classes

```
float_num <- 123.456
class(float_num)
```

```
[1] "numeric"
```

```
typeof(float_num)
```

```
[1] "double"
```

```
str(float_num)
```

```
 num 123
```

```
int_num <- 123L # L specifies integer, guessing requires more values
class(int_num)
```

```
[1] "integer"
```

```
typeof(int_num)
```

```
[1] "integer"
```

```
str(int_num)
```

```
 int 123
```

```
integer(length = 3)
```

```
[1] 0 0 0
```

```r
result <- 9^(1 / 2)
class(result)
```

```
[1] "numeric"
```

```r
result
```

```
[1] 3
```

```r
print(result)
```

```
[1] 3
```

```r
cat(result)
```

```
3
```

```r
char_var <- "some words"
class(char_var)
```

```
[1] "character"
```

```r
typeof(char_var)
```

```
[1] "character"
```

```r
character(length = 5)
```

```
[1] "" "" "" "" ""
```

```r
logical_var <- TRUE # can be abbreviated to T
logical_var2 <- FALSE # or F, but seen as bad style
class(logical_var)
```

```
[1] "logical"
```

```r
typeof(logical_var)
```

```
[1] "logical"
```

```r
logical(length = 3)
```

```
[1] FALSE FALSE FALSE
```

```
# logicals usualy are defined by conditions:
int_num < float_num
```

```
[1] TRUE
```

```
# all numbers are true but 0
as.logical(c(0, 1, 5, -7.45678)) # c() combines values into a vector
```

```
[1] FALSE  TRUE  TRUE  TRUE
```

3.3.3 Exercise: Variables / Assignments

(Please do exercises in scripts in a different project!)

Scenario: Imagine you've just received a new sample in the lab, and you need to record some basic information about it in R. This will help you get familiar with how R stores different kinds of data.

Learning Objectives:

- Learn how to create variables (objects) in R using the assignment operator (`<-`).
- Understand different basic data types (classes) in R: `numeric`, `character`, `logical`.
- Practice viewing the content and class of your variables.

Instructions:

1. **Assign a Sample ID:**
    - Create a variable called `sample_id`.
    - Assign it a unique identifier for your sample. Since it's text, make sure to put it in quotes, e.g., `"ExpA_S001"`.
2. **Record the Organism:**
    - Create a variable called `organism_name`.
    - Assign the scientific name of the organism your sample came from. Again, use quotes, e.g., `"Saccharomyces cerevisiae"`.
3. **Record a Measurement:**
    - You've measured the length of a cell from this sample in micrometers.
    - Create a variable called `cell_length_um`.
    - Assign a numerical value (without quotes) for the length, e.g., `7.5`.
4. **Record a Binary Condition:**
    - Was this sample grown in the presence of a specific nutrient (e.g., glucose)?
    - Create a variable called `grown_on_glucose`.
    - Assign a logical value: `TRUE` if yes, `FALSE` if no (these are special keywords in R, no quotes needed). E.g., `TRUE`.
5. **Check Your Work (Content and Class):**

- After creating each variable, type the variable name on a new line and press Enter to see its content.
- Use the `class()` function to check what type of data R thinks each variable holds. For example:
  - `class(sample_id)`
  - `class(organism_name)`
  - `class(cell_length_um)`
  - `class(grown_on_glucose)`

3.4 Factors

Factors are categorical variables with limited sets of distinct values (called levels).
The "Integer with a Name Tag" Concept: Think of it this way: R stores factors internally as numbers (integers), but it "tags" these numbers with meaningful labels (the "name tags"). For example, R might store "Male" as 1 and "Female" as 2, but it always displays "Male" and "Female" to you. This saves memory and makes computations efficient, while keeping your data interpretable. Everything intended to group subjects or representing categories (like species, tissue type, treatment) should be stored as factor for efficient storage and proper statistical handling in R. In Python, Pandas Categorical and dictionaries are related constructs. Factors can be ordered, but beware: this changes behavior in analyses!

Package forcats provides nice tools for factors!

```
factor_var <- factor(c("m", "m", "f", "m", "f", "f", "?"))
factor_var
```

```
[1] m m f m f f ?
Levels: ? f m
```

```
class(factor_var)
```

```
[1] "factor"
```

```
typeof(factor_var) # that is why factors can be called enumerated type
```

```
[1] "integer"
```

```
levels(factor_var)
```

```
[1] "?" "f" "m"
```

```
# factor definition can reorder, rename, and drop levels:
factor_var2 <- factor(c("m", "m", "f", "m", "f", "f", "?"),
  levels = c("m", "f"),
  labels = c("male", "female")
)
factor_var2
```

```
[1] male   male   female male   female female <NA>
Levels: male female
```

3.4.1 Exercise: Factors

Scenario: Beyond just individual measurements, biologists often need to categorize samples based on different experimental conditions, genetic backgrounds, or developmental stages. R uses a special data type called **factors** for this. Factors are crucial because they tell R that your data belongs to discrete groups, which is very important for statistical analysis!

Learning Objectives:

- Understand factors as a way to store **categorical data**.
- Learn how to create factors from character data.
- See how to inspect the **levels** (the "name tags") of a factor.
- Discover how R internally represents factors as **integers**.
- Learn how to create **ordered factors** when the categories have a natural sequence.

Instructions:

1. **Categorizing by Treatment Group (Unordered Factor):**
    - Imagine your samples are from different treatments: "Control", "Drug A", "Drug B".
    - First, create a *character variable* for the treatment of one sample

      What is its class?
    - Now, convert `sample_treatment` into a **factor**. This tells R it's a category.

      What is its class now? Notice how R also lists "Levels" when you print it.
    - To see all the possible "name tags" (categories) R knows for this factor, use `levels()`

      Why do you think R automatically inferred "Control", "Drug A", "Drug B" as levels, even though you only assigned "Drug A"? (Hint: It hasn't; it only knows "Drug A" for now. We'll fix this in the next step!)
    - **The "Integer Tag" Reveal:** To see the hidden integer that R uses for "Drug A", convert the factor to a numeric type:

      What number did you get? This number corresponds to the alphabetical position of "Drug A" among the levels R currently sees (`Drug A`).
2. **Defining All Levels Explicitly:**
    - When you create a factor, it's good practice to tell R *all* the possible levels upfront, even if a particular sample only has one of them. This ensures consistency.

- Re-create `sample_treatment_factor`, but this time specifying all the `levels`:

  Now, what number do you get from `as.numeric()`? How does it relate to the `levels` you explicitly defined? This demonstrates how the integer tag links to its "name tag" position in the defined levels.

3. **Categorizing by Developmental Stage (Ordered Factor):**

   - Sometimes, categories have a natural order. For example, developmental stages: "Larva", "Pupa", "Adult". "Adult" is definitely "later" than "Larva".
   - Let's create a factor for a sample's developmental stage, making sure R understands the order:

     How does the `class()` output differ from `sample_treatment_factor_full`? What integer did "Pupa" get, and why?

Key Takeaways:

- Use **`character`** for free text (like comments or unique IDs).
- Use **`factor`** for **categorical data** (like experimental groups, sexes, genotypes, species names in a fixed list).
- Factors are essential for statistical models as they correctly identify groups for comparisons.
- **`levels()`** shows you the "name tags."
- **`as.numeric()`** shows you the underlying "integer tags."

Dates / Time:

```
(date_var <- Sys.Date())
```

```
[1] "2026-10-13"
```

```
class(date_var)
```

```
[1] "Date"
```

```
typeof(date_var)
```

```
[1] "double"
```

```
class(Sys.time())
```

```
[1] "POSIXct" "POSIXt"
```

```
typeof(Sys.time())
```

```
[1] "double"
```

Mixed classes:

If the data that goes into a variable has more than 1 class, the more general class is selected.

```
test2 <- c(1, 2, "a", "b")
class(test2)
```

```
[1] "character"
```

```
test2
```

```
[1] "1" "2" "a" "b"
```

3.5 Forcing / casting classes

If the assigned class is not correct or appropriate, it can be changed. Casting functions usually start with `as.` When creating variables filled with NA, use casting functions or specific variants of NA to force type!

```
(test <- c(1, 2, 3, "a", "b", "c"))
```

```
[1] "1" "2" "3" "a" "b" "c"
```

```
(test_n <- as.numeric(test))
```

```
Warning: NAs introduced by coercion
```

```
[1]  1  2  3 NA NA NA
```

```
as.numeric(factor_var)
```

```
[1] 3 3 2 3 2 2 1
```

```
as.character(10:19)
```

```
 [1] "10" "11" "12" "13" "14" "15" "16" "17" "18" "19"
```

```r
# NAs
(test_NA1 <- rep(NA, 10))
```

```
 [1] NA NA NA NA NA NA NA NA NA NA
```

```r
class((test_NA1))
```

```
[1] "logical"
```

```r
class(NA_real_)
```

```
[1] "numeric"
```

```r
(test_NA2 <- rep(NA_real_, 10))
```

```
 [1] NA NA NA NA NA NA NA NA NA NA
```

```r
class((test_NA2))
```

```
[1] "numeric"
```

```r
class(NA_integer_)
```

```
[1] "integer"
```

```r
class(NA_character_)
```

```
[1] "character"
```

```r
class(NA_Date_) # from package lubridate
```

```
[1] "Date"
```

3.6 Indexing variables

The most general kind of indexing is by position, starting with **1**. Negative numbers result in exclusion of position(s). Position indices are provided within square brackets. The index can (and usually will) be a variable instead of hard coded numbers.

```
(numbers1 <- c(5, 3, 6, 8, 2, 1))
```

```
[1] 5 3 6 8 2 1
```

```
numbers1[1]
```

```
[1] 5
```

```
numbers1[1:3]
```

```
[1] 5 3 6
```

```
numbers1[-c(1, 3)]
```

```
[1] 3 8 2 1
```

```
numbers2 <- 1:3
numbers1[numbers2]
```

```
[1] 5 3 6
```

```
# numbers1[1,2] #Error: incorrect number of dimensions
```

To get first or last entries, head() and tail() can be used. By default 6 entries are returned.

```
tail(x = numbers1, n = 1)
```

```
[1] 1
```

```
head(x = numbers1, n = 3)
```

```
[1] 5 3 6
```

```
nth(x = numbers1, n = -2) # 2nd to last
```

```
[1] 2
```

3.7 Usage of variables

Variables are like placeholders for their content, so that you don't have to remember where you left things. Operations on variables are operations on their content. Changing the content of a variable does not automatically save those changes back to the variable, this needs to be done explicitly!

```
numbers1 + 100 # not stored anywhere, just printed
```

```
[1] 105 103 106 108 102 101
```

```
numbers1 + numbers2 # why does this even work?
```

```
[1] 6 5 9 9 4 4
```

When combining variables of different length, the short one is recycled, so the numbers2 is added to the first 3 elements of numbers2, then is reused and added to the remaining 3 elements. If the length of the longer is not a multiple of the shorter, there will be a warning.

```
c(2, 4, 6, 8) + 1
```

```
[1] 3 5 7 9
```

```
c(2, 4, 6, 8) + c(1, 2)
```

```
[1]  3  6  7 10
```

```
c(2, 4, 6, 8) + c(1, 2, 3)
```

```
Warning in c(2, 4, 6, 8) + c(1, 2, 3): longer object length is not a multiple
of shorter object length
```

```
[1] 3 6 9 9
```

3.8 Functions

3.8.1 Function usage

Functions have the same naming rules as variables, but the name is always followed by opening/closing round brackets, within those brackets function parameters/arguments can be specified to provide input or control behavior:

FunctionName(parameter1 = x1,parameter2 = x2,x3,...)

Most functions have named arguments, those argument names may be omitted as long as parameter values are supplied in the defined order. Arguments may have predefined default values, see help! Some functions like `c()` use unnamed arguments.

```
c("my", "name") # unnamed
```

```
[1] "my"   "name"
```

```
# ?mean
mean(x = c(3, 5, 7, NA)) # using default parameters
```

```
[1] NA
```

```
mean(x = c(3, 5, 7, NA), na.rm = TRUE) # overriding default parameter
```

```
[1] 5
```

```
mean(na.rm = TRUE, x = c(3, 5, 7, NA)) # changed order of arguments
```

```
[1] 5
```

```
mean(c(3, 5, 7, NA), na.rm = TRUE) # name of 1st argument omitted
```

```
[1] 5
```

```
sd(c(3, 5, 7, NA), na.rm = TRUE)
```

```
[1] 2
```

```
# same logic as mean, partially the same arguments
median(1:100, TRUE)
```

```
[1] 50.5
```

```
# omitting arguments influences readability of a function, careful!
t <- c(1:10, 100)
quantile(x = t, probs = c(.2, .8))
```

```
20% 80%
  3   9
```

```
# putting text elements together
paste("some text", 1:3)
```

```
[1] "some text 1" "some text 2" "some text 3"
```

```
paste0("some text", 1:3)
```

```
[1] "some text1" "some text2" "some text3"
```

```
paste("some text", 1:3, sep = ": ")
```

```
[1] "some text: 1" "some text: 2" "some text: 3"
```

```
paste("some text", 1:3, sep = ": ", collapse = "; ")
```

```
[1] "some text: 1; some text: 2; some text: 3"
```

```
paste("some text", 1:3, sep = ": ", collapse = "\n") |> cat("\n")
```

```
some text: 1
some text: 2
some text: 3
```

```
paste("mean", "SD", sep = " \u00b1 ")
```

```
[1] "mean ± SD"
```

3.8.2 Functions combined

Functions often just solve one problem or task, so usually we need to combine them to, for instance, filter our data, then calculate some statistics, and finally tabulize the results. This can be done by nesting or piping. Piping (creation of pipelines or production belts) makes reading/understanding scripts easier, as it shows order of information flowing from one function to the next, often visualized with a special symbol |>

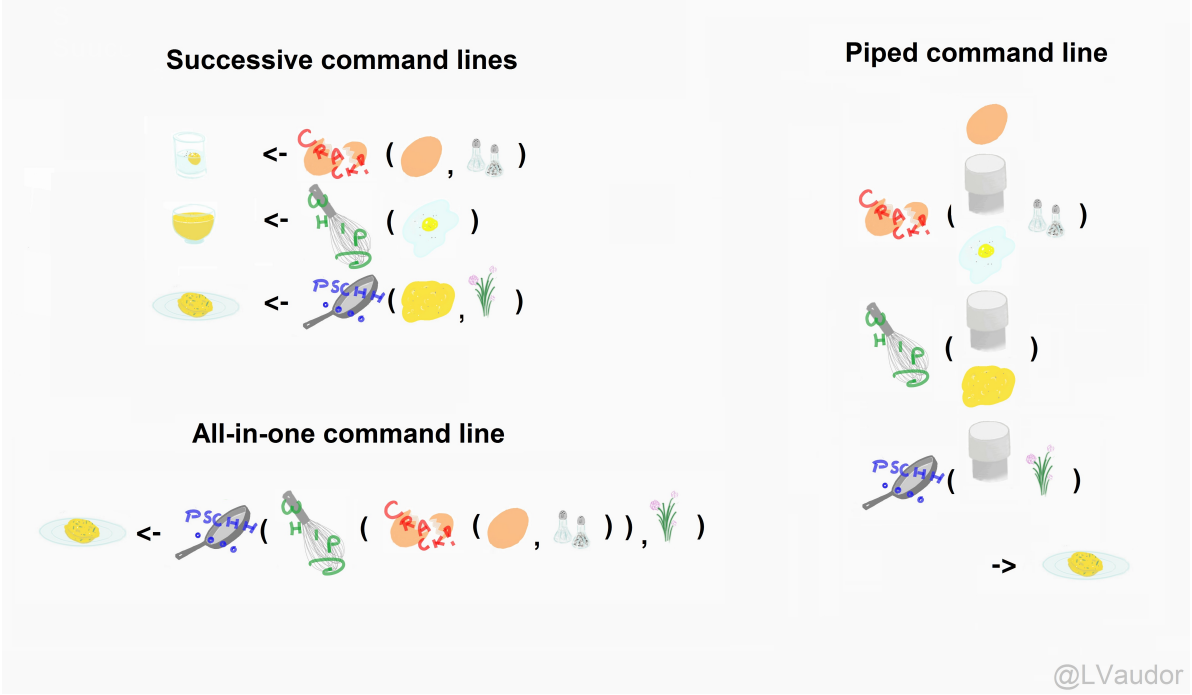


Figure 3.1: *Piping functions*

```
# functions may be nested:
floor(
  as.numeric(
    Sys.Date() -
      as.Date("1985/12/10")
  ) /
    365.25
)
```

```
[1] 40
```

```
# or (usually better) piped:
mtcars |> # inbuild example data, use F1!
```

```
  mutate(am = factor(am,
    levels = c(0, 1),
    labels = c(
      "automatic",
      "manual"
    )
  )) |> # #change into better class
  filter(vs == 1) |> # filter out V-shaped
  group_by(am) |> # ask for grouped analysis
  summarise(across(
    .cols = c(wt, mpg, qsec, disp),
    .fns = meansd
  )) |>
  pivot_longer(cols = -am,
               names_to = "Measure") |> # put variables in rows
  pivot_wider(
    id_cols = Measure,
    names_from = am, # put groups in cols
    values_from = value
  )
```

```
# A tibble: 4 x 3
  Measure automatic manual
  <chr>   <chr>     <chr>
1 wt      3.2 ± 0.3 2.0 ± 0.4
2 mpg     21 ± 2    28 ± 5
3 qsec    20 ± 1    19 ± 1
4 disp    175 ± 49  90 ± 19
```

If a sequence of functions is used often, combining them into a new function is advisable, e.g. this function is a combination of descriptive and test statistics:

```
# can be combined into higher order functions:
compare2numvars(
  data = mtcars,
  dep_vars = c("mpg", "wt", "qsec"),
  indep_var = "am",
  add_n = TRUE,
  gaussian = TRUE
)
```

```
# A tibble: 3 x 5
```

```
  Variable desc_all            `am 0`              `am 1`              p
  <chr>    <chr>               <chr>               <chr>               <chr>
1 mpg      20 ± 6  [n = 32]    17 ± 4  [n = 19]    24 ± 6  [n = 13]    0.001
2 wt       3.2 ± 1.0  [n = 32] 3.8 ± 0.8  [n = 19] 2.4 ± 0.6  [n = 13] 0.001
3 qsec     18 ± 2  [n = 32]    18 ± 2  [n = 19]    17 ± 2  [n = 13]    0.206
```

3.8.3 Writing functions

Functions can be thought of as blocks/chunks of code with defined in- and output. Functions intended for general use (e.g. published in a package) should be enhanced by error prevention / handling and documentation.

```
# FunctionName<-function(parameters...){definition}
division <- function(y, x) {
  return(x / y)
}
(Sys.Date() - as.Date("1958/12/10")) |>
  as.numeric() |>
  division(y = 365.25, x = _) |>
  floor()
```

```
[1] 67
```

```
mean <- function(values) {
  return(base::mean(values, na.rm = TRUE))
}

mean(c(1, 2, 3, NA))
```

```
[1] 2
```

```
rm(mean) # to revert to original function base::mean

mark_sign <- function(SignIn) {
  SignIn <- as.numeric(SignIn)
  if (is.na(SignIn)) {
    SignOut <- "wrong input, stupido!"
  } else {
    # if (!is.na(SignIn)) {
    SignOut <- "n.s."
    if (SignIn <= 0.1) {
      SignOut <- "+"
```

```
    }
    if (SignIn <= 0.05) {
      SignOut <- "*"
    }
    if (SignIn <= 0.01) {
      SignOut <- "**"
    }
    if (SignIn <= 0.001) {
      SignOut <- "***"
    }
  }
  return(SignOut)
}

mark_sign(SignIn = 0.035)
```

```
[1] "*"
```

```
mark_sign(SignIn = "0.35")
```

```
[1] "n.s."
```

```
mark_sign(SignIn = "p=3,5%") # wrong input
```

```
Warning in mark_sign(SignIn = "p=3,5%"): NAs introduced by coercion
```

```
[1] "wrong input, stupido!"
```

different implementation

```
markSign0 <- function(SignIn, plabel = c("n.s.", "+", "*", "**", "***")) {
  SignIn <- suppressWarnings(
    as.numeric(SignIn)
  )
  SignOut <- cut(SignIn,
    breaks = c(-Inf, .001, .01, .05, .1, 1),
    labels = rev(plabel)
  )
  return(SignOut)
}

markSign0(SignIn = c(0.035, 0.00002, .234))
```

```
[1] *    ***  n.s.
Levels: *** ** * + n.s.
```

```
markSign0(SignIn = "0.35")
```

```
[1] n.s.
Levels: *** ** * + n.s.
```

```
markSign0(SignIn = "p=3,5%") # wrong input
```

```
[1] <NA>
Levels: *** ** * + n.s.
```

```
# source("somepath/R/myfunctions.R")
```

3.9 More complex data types, created by functions

3.9.1 Matrix

A matrix is a 2-dimensional data structure, where all elements are of the same class.

3.9.1.1 Creation

```
my1.Matrix <-
  matrix(
    data = 1:12,
    # nrow = 4, # this is not needed, can be derived from data
    ncol = 3,
    byrow = TRUE, # data are put into row 1 first
    dimnames = list(
      paste0("row", 1:4),
      paste0("col", 1:3)
    )
  )
print(my1.Matrix)
```

```
     col1 col2 col3
row1    1    2    3
row2    4    5    6
row3    7    8    9
row4   10   11   12
```

```r
data <- seq(from = 1, to = 100, by = 1) # 1:100
nrow <- 20
matrix(
  data = data,
  nrow = nrow,
  byrow = FALSE, # data are put into column 1 first
  dimnames = list(
    paste0("row", 1:nrow),
    paste0("col", 1:(length(data) / nrow))
  )
) |>
  head()
```

```
     col1 col2 col3 col4 col5
row1    1   21   41   61   81
row2    2   22   42   62   82
row3    3   23   43   63   83
row4    4   24   44   64   84
row5    5   25   45   65   85
row6    6   26   46   66   86
```

```r
my2.Matrix <- matrix(c(1, 2, 3, 11, 12, 13),
  nrow = 2, ncol = 3
) # byrow = FALSE, specified but default
my2.Matrix
```

```
     [,1] [,2] [,3]
[1,]    1    3   12
[2,]    2   11   13
```

3.9.1.2 Indexing

Addressing a matrix is done with [row_index, column_index]

```r
my1.Matrix[2, 3] # Index:[row,column]
```

```
[1] 6
```

```r
my1.Matrix[2, ] # all columns
```

```
col1 col2 col3
   4    5    6
```

```r
my1.Matrix[, 2] # all rows
```

```
row1 row2 row3 row4
   2    5    8   11
```

```r
my1.Matrix[c(1, 3), -2] # exclude column 2
```

```
     col1 col3
row1    1    3
row3    7    9
```

```r
my1.Matrix[1:3,"col3"]
```

```
row1 row2 row3
   3    6    9
```

```r
my1.Matrix[1, 1] <- NA # Index can be used for writing as well
```

3.9.1.3 Exercise: Matrix

Scenario: Imagine you are a field biologist studying plant distribution. You've set up several quadrants (square frames) in your study area and counted the number of individuals for three different plant species in each quadrant.

Instructions:

1. **Create Your Data:**
   - You have the following counts:
     - **Species A:** Quadrant 1: 12, Quadrant 2: 8, Quadrant 3: 15
     - **Species B:** Quadrant 1: 5, Quadrant 2: 10, Quadrant 3: 7
     - **Species C:** Quadrant 1: 20, Quadrant 2: 14, Quadrant 3: 18
   - Create a numeric vector for each species' counts (e.g., `species_A_counts <- c(12, 8, 15)`).
2. **Create a Matrix:**
   - Combine these three vectors into a single matrix. Name this matrix `quadrant_data`.
   - Make sure each row represents a species and each column represents a quadrant.
   - Set the number of rows (`nrow`) to 3 and the number of columns (`ncol`) to 3.
3. **Name Rows and Columns:**
   - Assign row names to your matrix: `"SpeciesA"`, `"SpeciesB"`, `"SpeciesC"`.
   - Assign column names to your matrix: `"Quadrant1"`, `"Quadrant2"`, `"Quadrant3"`.
4. **Index Your Matrix (Access Data):**
   - **Access a single element:** Get the count of `Species B` in `Quadrant 2`.
   - **Access a full row:** Get all counts for `Species A`.
   - **Access a full column:** Get all counts from `Quadrant 3`.
   - **Access multiple elements/rows/columns:**
     - Get the counts for `Species A` and `Species C` in `Quadrant 1` and `Quadrant 2`.

3.9.2 Data frame

A data frame has 2 dimensions, it can handle various data types (1 per columns). This structure is rather superseded by tibbles (see below).

3.9.2.1 Creation

Data frames are defined by creating and filling columns, functions can be used (and piped) to create content.

```
patientN <- 15
(myTable <- data.frame(
  patientCode = paste0("pat", 1:patientN),
  Var1 = 1, # gets recycled
  Var2 = NA_Date_
)) |> head()
```

```
  patientCode Var1 Var2
1        pat1    1 <NA>
2        pat2    1 <NA>
3        pat3    1 <NA>
4        pat4    1 <NA>
5        pat5    1 <NA>
6        pat6    1 <NA>
```

```
str(myTable)
```

```
'data.frame':   15 obs. of  3 variables:
 $ patientCode: chr  "pat1" "pat2" "pat3" "pat4" ...
 $ Var1       : num  1 1 1 1 1 1 1 1 1 1 ...
 $ Var2       : Date, format: NA NA ...
```

```
set.seed(101)
myTable <- data.frame(
  patientCode = paste0("pat", 1:patientN),
  Age = runif(n = patientN, min = 18, max = 65) |> floor(),
  Sex = factor(rep(x = NA, times = patientN),
    levels = c("m", "f")
  ),
  `sysRR (mmHg)` = round(rnorm(n = patientN, mean = 140, sd = 10)),
  check.names = FALSE
)
head(myTable)
```

```
  patientCode Age  Sex sysRR (mmHg)
1        pat1  35 <NA>          142
2        pat2  20 <NA>          132
3        pat3  51 <NA>          122
4        pat4  48 <NA>          157
5        pat5  29 <NA>          144
6        pat6  32 <NA>          148
```

3.9.2.2 Indexing

Beside the numeric index, columns can be addressed by name. This can be done by either dfname$colname (for the content of a single column) or dfname[,"colname"] for 1 or more columns.

```
myTable[1:5, 1]
```

```
[1] "pat1" "pat2" "pat3" "pat4" "pat5"
```

```
myTable[1:5, ]
```

```
  patientCode Age  Sex sysRR (mmHg)
1        pat1  35 <NA>          142
2        pat2  20 <NA>          132
3        pat3  51 <NA>          122
4        pat4  48 <NA>          157
5        pat5  29 <NA>          144
```

```
myTable[, 1:2]
```

```
   patientCode Age
1         pat1  35
2         pat2  20
3         pat3  51
4         pat4  48
5         pat5  29
6         pat6  32
7         pat7  45
8         pat8  33
9         pat9  47
10       pat10  43
11       pat11  59
12       pat12  51
```

```
13       pat13  52
14       pat14  61
15       pat15  39
```

```
myTable$patientCode[1:5]
```

```
[1] "pat1" "pat2" "pat3" "pat4" "pat5"
```

```
myTable[1:5, "patientCode"]
```

```
[1] "pat1" "pat2" "pat3" "pat4" "pat5"
```

```
# returns vector of values for a single column, data.frame otherwise
myTable["patientCode"] # returns df
```

```
   patientCode
1         pat1
2         pat2
3         pat3
4         pat4
5         pat5
6         pat6
7         pat7
8         pat8
9         pat9
10       pat10
11       pat11
12       pat12
13       pat13
14       pat14
15       pat15
```

```
columns <- c("Sex", "Age")
myTable[1:5, columns]
```

```
   Sex Age
1 <NA>  35
2 <NA>  20
3 <NA>  51
4 <NA>  48
5 <NA>  29
```

```
myTable[1:5, c("patientCode", "Age")]
```

```
  patientCode Age
1        pat1  35
2        pat2  20
3        pat3  51
4        pat4  48
5        pat5  29
```

```
myTable[, 1] <- paste0("Code", 1:patientN)
```

3.9.3 Tibble

Tibbles are a modern and efficient data structure that extends data frames, providing enhanced features and performance for data manipulation and analysis.

3.9.3.1 Creation

```
patientN <- 25
set.seed(2901)
rawdata <- tibble(
  PatID = paste("P", 1:patientN), # as in  data.frame
  Sex = sample(
    x = c("male", "female"), # random generator
    size = patientN, replace = TRUE,
    prob = c(.7, .3)
  ),
  Ethnicity = sample(
    x = 1:6,
    size = patientN,
    replace = TRUE,
    prob = c(.01, .01, .05, .03, .75, .15)
  ),
  # random assignments
  `Given name` = randomNames(
    n = patientN,
    gender = Sex,
    # this is a reference to column Sex
    ethnicity = Ethnicity,
    which.names = "first"
  ),
  `Family name` = randomNames(
    n = patientN,
```

```
    ethnicity = Ethnicity,
    which.names = "last"
  ),
  Treatment = sample(
    x = c("Placebo", "Verum"),
    size = patientN,
    replace = TRUE
  ),
  `sysRR (mmHg)` = round(rnorm(n = patientN, mean = 140, sd = 10)) -
    (Treatment == "Verum") * 15,
  `diaRR (mmHg)` = round(rnorm(n = patientN, mean = 80, sd = 10)) -
    (Treatment == "Verum") * 10,
  HR = round(rnorm(n = patientN, mean = 90, sd = 7))
)
rawdata
```

```
# A tibble: 25 x 9
   PatID Sex    Ethnicity `Given name` `Family name` Treatment `sysRR (mmHg)`
   <chr> <chr>      <int> <chr>        <chr>         <chr>                <dbl>
 1 P 1   male           6 Shaddaad     el-Minhas     Placebo                127
 2 P 2   male           5 Jacob        Roberge       Verum                  152
 3 P 3   female         5 Savannah     Williams      Placebo                113
 4 P 4   male           5 Russell      Miltenberger  Placebo                133
 5 P 5   male           5 Greg         Huter         Placebo                126
 6 P 6   male           5 Sean         Mandel        Placebo                138
 7 P 7   male           5 Hans         Cardinal      Placebo                142
 8 P 8   male           4 Vince        Duran         Verum                  132
 9 P 9   male           2 Roger        Dhindsa       Verum                  126
10 P 10  male           4 Omar         Marley        Verum                  136
# i 15 more rows
# i 2 more variables: `diaRR (mmHg)` <dbl>, HR <dbl>
```

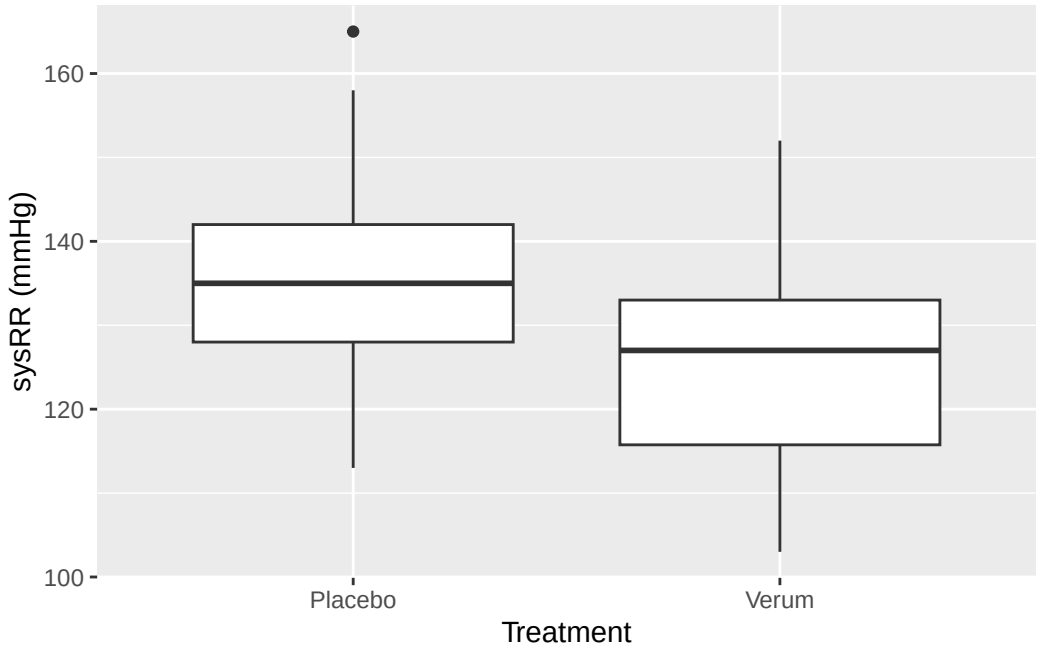
```
colnames(rawdata)
```

```
[1] "PatID"        "Sex"          "Ethnicity"    "Given name"   "Family name"
[6] "Treatment"    "sysRR (mmHg)" "diaRR (mmHg)" "HR"
```

```
cn() # shortcut from wrappedtools
```

```
[1] "PatID"        "Sex"          "Ethnicity"    "Given name"   "Family name"
[6] "Treatment"    "sysRR (mmHg)" "diaRR (mmHg)" "HR"
```

```r
# example of data management for a tibble, recoding ethnicity:
rawdata <- rawdata |>
  mutate(
    Ethnicity = factor(
      Ethnicity,
      levels = 1:6,
      labels = c(
        "American Indian or Native Alaskan",
        "Asian or Pacific Islander",
        "Black (not Hispanic)",
        "Hispanic",
        "White (not Hispanic)",
        "Middle-Eastern, Arabic"
      )
    )
  )
# quick visual inspection
ggplot(rawdata, aes(x = Treatment, y = `sysRR (mmHg)`)) +
  geom_boxplot()
```



3.9.3.2 Indexing

The same rules as for the data frame, but more consistent behavior.

```
rawdata[1:5, 1:2]
```

```
# A tibble: 5 x 2
  PatID Sex
  <chr> <chr>
1 P 1   male
2 P 2   male
3 P 3   female
4 P 4   male
5 P 5   male
```

```
rawdata[, 6]
```

```
# A tibble: 25 x 1
   Treatment
   <chr>
 1 Placebo
 2 Verum
 3 Placebo
 4 Placebo
 5 Placebo
 6 Placebo
 7 Placebo
 8 Verum
 9 Verum
10 Verum
# i 15 more rows
```

```
rawdata[6]
```

```
# A tibble: 25 x 1
   Treatment
   <chr>
 1 Placebo
 2 Verum
 3 Placebo
 4 Placebo
 5 Placebo
```

```
 6 Placebo
 7 Placebo
 8 Verum
 9 Verum
10 Verum
# i 15 more rows
```

```
rawdata[[6]]
```

```
 [1] "Placebo" "Verum"   "Placebo" "Placebo" "Placebo" "Placebo" "Placebo"
 [8] "Verum"   "Verum"   "Verum"   "Verum"   "Verum"   "Verum"   "Verum"
[15] "Placebo" "Placebo" "Verum"   "Verum"   "Placebo" "Verum"   "Placebo"
[22] "Placebo" "Verum"   "Placebo" "Placebo"
```

```
rawdata$`Family name`
```

```
 [1] "el-Minhas"    "Roberge"      "Williams"     "Miltenberger" "Huter"
 [6] "Mandel"       "Cardinal"     "Duran"        "Dhindsa"      "Marley"
[11] "Heller"       "Haynes"       "Rabideau"     "Rafferty"     "al-Azimi"
[16] "Suits"        "Guptill"      "Steinman"     "Donato"       "Olson"
[21] "Heller"       "Anderson"     "al-Mohammed"  "Chavez"       "Biggs"
```

Differences in addressing data frames and tibbles:

- tibble and `[` always returns tibble
- tibble and `[[` always returns vector
- data.frame and `[` may return data.frame (if >1 column) or vector
- data.frame and `[[` always returns vector

```
rawdata_df <- as.data.frame(rawdata)
rawdata[2] # returns Tibble with 1 column
```

```
# A tibble: 25 x 1
   Sex
   <chr>
 1 male
 2 male
 3 female
 4 male
 5 male
 6 male
```

```
 7 male
 8 male
 9 male
10 male
# i 15 more rows
```

```
rawdata[[2]] # returns vector
```

```
 [1] "male"   "male"   "female" "male"   "male"   "male"   "male"   "male"
 [9] "male"   "male"   "male"   "male"   "male"   "female" "male"   "female"
[17] "male"   "female" "male"   "female" "male"   "female" "female" "male"
[25] "male"
```

```
rawdata[, 2] # returns Tibble with 1 column
```

```
# A tibble: 25 x 1
   Sex
   <chr>
 1 male
 2 male
 3 female
 4 male
 5 male
 6 male
 7 male
 8 male
 9 male
10 male
# i 15 more rows
```

```
rawdata[, 2:3] # returns tibble with 2 columns
```

```
# A tibble: 25 x 2
   Sex    Ethnicity
   <chr>  <fct>
 1 male   Middle-Eastern, Arabic
 2 male   White (not Hispanic)
 3 female White (not Hispanic)
 4 male   White (not Hispanic)
 5 male   White (not Hispanic)
 6 male   White (not Hispanic)
 7 male   White (not Hispanic)
```

```
 8 male   Hispanic
 9 male   Asian or Pacific Islander
10 male   Hispanic
# i 15 more rows
```

```
rawdata_df[2] # returns DF with 1 column
```

```
      Sex
1    male
2    male
3  female
4    male
5    male
6    male
7    male
8    male
9    male
10   male
11   male
12   male
13   male
14 female
15   male
16 female
17   male
18 female
19   male
20 female
21   male
22 female
23 female
24   male
25   male
```

```
rawdata_df[[2]] # returns vector
```

```
 [1] "male"   "male"   "female" "male"   "male"   "male"   "male"   "male"
 [9] "male"   "male"   "male"   "male"   "male"   "female" "male"   "female"
[17] "male"   "female" "male"   "female" "male"   "female" "female" "male"
[25] "male"
```

```
rawdata_df[, 2] # returns vector
```

```
 [1] "male"   "male"   "female" "male"   "male"   "male"   "male"   "male"
 [9] "male"   "male"   "male"   "male"   "male"   "female" "male"   "female"
[17] "male"   "female" "male"   "female" "male"   "female" "female" "male"
[25] "male"
```

```
rawdata_df[, 2:3] # returns DF with 2 columns
```

```
      Sex                 Ethnicity
1    male    Middle-Eastern, Arabic
2    male      White (not Hispanic)
3  female      White (not Hispanic)
4    male      White (not Hispanic)
5    male      White (not Hispanic)
6    male      White (not Hispanic)
7    male      White (not Hispanic)
8    male                  Hispanic
9    male Asian or Pacific Islander
10   male                  Hispanic
11   male      White (not Hispanic)
12   male      White (not Hispanic)
13   male      White (not Hispanic)
14 female      White (not Hispanic)
15   male    Middle-Eastern, Arabic
16 female      Black (not Hispanic)
17   male      White (not Hispanic)
18 female      White (not Hispanic)
19   male      White (not Hispanic)
20 female      White (not Hispanic)
21   male      White (not Hispanic)
22 female      White (not Hispanic)
23 female    Middle-Eastern, Arabic
24   male      White (not Hispanic)
25   male      White (not Hispanic)
```

There are specific functions for 'picking' columns or rows, especially useful in pipes.

```
rawdata |> select(PatID:Ethnicity, `sysRR (mmHg)`:HR)
```

```
# A tibble: 25 x 6
   PatID Sex    Ethnicity                 `sysRR (mmHg)` `diaRR (mmHg)`    HR
   <chr> <chr>  <fct>                              <dbl>          <dbl> <dbl>
 1 P 1   male   Middle-Eastern, Arabic                127             80    95
```

```
 2 P 2   male   White (not Hispanic)                152              78    83
 3 P 3   female White (not Hispanic)                113              89    80
 4 P 4   male   White (not Hispanic)                133              80    89
 5 P 5   male   White (not Hispanic)                126              92    96
 6 P 6   male   White (not Hispanic)                138              68    99
 7 P 7   male   White (not Hispanic)                142              80    91
 8 P 8   male   Hispanic                           132              66    98
 9 P 9   male   Asian or Pacific Islander          126              53    87
10 P 10  male   Hispanic                           136              60    92
# i 15 more rows
```

```
rawdata |>
  select(PatID:Ethnicity, `sysRR (mmHg)`:HR) |>
  slice(1:5)
```

```
# A tibble: 5 x 6
  PatID Sex    Ethnicity              `sysRR (mmHg)` `diaRR (mmHg)`    HR
  <chr> <chr>  <fct>                           <dbl>          <dbl> <dbl>
1 P 1   male   Middle-Eastern, Arabic             127             80    95
2 P 2   male   White (not Hispanic)               152             78    83
3 P 3   female White (not Hispanic)               113             89    80
4 P 4   male   White (not Hispanic)               133             80    89
5 P 5   male   White (not Hispanic)               126             92    96
```

```
rawdata |> select(contains("RR", ignore.case = FALSE))
```

```
# A tibble: 25 x 2
   `sysRR (mmHg)` `diaRR (mmHg)`
            <dbl>          <dbl>
 1            127             80
 2            152             78
 3            113             89
 4            133             80
 5            126             92
 6            138             68
 7            142             80
 8            132             66
 9            126             53
10            136             60
# i 15 more rows
```

```
rawdata |> select(ends_with("r"))
```

```
# A tibble: 25 x 1
      HR
   <dbl>
 1    95
 2    83
 3    80
 4    89
 5    96
 6    99
 7    91
 8    98
 9    87
10    92
# i 15 more rows
```

```
rawdata |> select(-contains("name"))
```

```
# A tibble: 25 x 7
   PatID Sex    Ethnicity          Treatment `sysRR (mmHg)` `diaRR (mmHg)`    HR
   <chr> <chr>  <fct>              <chr>              <dbl>          <dbl> <dbl>
 1 P 1   male   Middle-Eastern, A~ Placebo              127             80    95
 2 P 2   male   White (not Hispan~ Verum                152             78    83
 3 P 3   female White (not Hispan~ Placebo              113             89    80
 4 P 4   male   White (not Hispan~ Placebo              133             80    89
 5 P 5   male   White (not Hispan~ Placebo              126             92    96
 6 P 6   male   White (not Hispan~ Placebo              138             68    99
 7 P 7   male   White (not Hispan~ Placebo              142             80    91
 8 P 8   male   Hispanic           Verum                132             66    98
 9 P 9   male   Asian or Pacific ~ Verum                126             53    87
10 P 10  male   Hispanic           Verum                136             60    92
# i 15 more rows
```

```
rawdata |> select(where(is.numeric))
```

```
# A tibble: 25 x 3
   `sysRR (mmHg)` `diaRR (mmHg)`    HR
            <dbl>          <dbl> <dbl>
 1            127             80    95
 2            152             78    83
 3            113             89    80
 4            133             80    89
 5            126             92    96
```

```
 6            138             68    99
 7            142             80    91
 8            132             66    98
 9            126             53    87
10            136             60    92
# i 15 more rows
```

```
rawdata |> select(`sysRR (mmHg)`)
```

```
# A tibble: 25 x 1
   `sysRR (mmHg)`
            <dbl>
 1            127
 2            152
 3            113
 4            133
 5            126
 6            138
 7            142
 8            132
 9            126
10            136
# i 15 more rows
```

```
rawdata |> select(contains("r"), -contains("rr"))
```

```
# A tibble: 25 x 2
   Treatment    HR
   <chr>     <dbl>
 1 Placebo      95
 2 Verum        83
 3 Placebo      80
 4 Placebo      89
 5 Placebo      96
 6 Placebo      99
 7 Placebo      91
 8 Verum        98
 9 Verum        87
10 Verum        92
# i 15 more rows
```

```
rawdata |> pull(`sysRR (mmHg)`)
```

```
 [1] 127 152 113 133 126 138 142 132 126 136 139 119 108 117 158 165 128 130 135
[20] 112 151 128 103 139 133
```

```
rawdata |> slice(1:5)
```

```
# A tibble: 5 x 9
  PatID Sex    Ethnicity     `Given name` `Family name` Treatment `sysRR (mmHg)`
  <chr> <chr>  <fct>         <chr>        <chr>         <chr>               <dbl>
1 P 1   male   Middle-Easte~ Shaddaad     el-Minhas     Placebo               127
2 P 2   male   White (not H~ Jacob        Roberge       Verum                 152
3 P 3   female White (not H~ Savannah     Williams      Placebo               113
4 P 4   male   White (not H~ Russell      Miltenberger  Placebo               133
5 P 5   male   White (not H~ Greg         Huter         Placebo               126
# i 2 more variables: `diaRR (mmHg)` <dbl>, HR <dbl>
```

```
rawdata |> slice_tail()
```

```
# A tibble: 1 x 9
  PatID Sex   Ethnicity      `Given name` `Family name` Treatment `sysRR (mmHg)`
  <chr> <chr> <fct>          <chr>        <chr>         <chr>               <dbl>
1 P 25  male  White (not Hi~ Eric         Biggs         Placebo               133
# i 2 more variables: `diaRR (mmHg)` <dbl>, HR <dbl>
```

```
rawdata |> slice_max(order_by = `sysRR (mmHg)`, n = 5)
```

```
# A tibble: 5 x 9
  PatID Sex    Ethnicity     `Given name` `Family name` Treatment `sysRR (mmHg)`
  <chr> <chr>  <fct>         <chr>        <chr>         <chr>               <dbl>
1 P 16  female Black (not H~ Alyxandra    Suits         Placebo               165
2 P 15  male   Middle-Easte~ Anas         al-Azimi      Placebo               158
3 P 2   male   White (not H~ Jacob        Roberge       Verum                 152
4 P 21  male   White (not H~ Brandon      Heller        Placebo               151
5 P 7   male   White (not H~ Hans         Cardinal      Placebo               142
# i 2 more variables: `diaRR (mmHg)` <dbl>, HR <dbl>
```

```
rawdata |> slice_sample(n = 5)
```

```
# A tibble: 5 x 9
  PatID Sex   Ethnicity      `Given name` `Family name` Treatment `sysRR (mmHg)`
  <chr> <chr> <fct>          <chr>        <chr>         <chr>               <dbl>
1 P 17  male  White (not Hi~ James        Guptill       Verum                 128
2 P 9   male  Asian or Paci~ Roger        Dhindsa       Verum                 126
```

```
3 P 11  male  White (not Hi~ Andrew       Heller        Verum                139
4 P 19  male  White (not Hi~ Gadge        Donato        Placebo              135
5 P 13  male  White (not Hi~ Tyler        Rabideau      Verum                108
# i 2 more variables: `diaRR (mmHg)` <dbl>, HR <dbl>
```

```
rawdata |> filter(Sex == "male", `sysRR (mmHg)` > 140) #drops both NA and FALSE
```

```
# A tibble: 4 x 9
  PatID Sex   Ethnicity      `Given name` `Family name` Treatment `sysRR (mmHg)`
  <chr> <chr> <fct>          <chr>        <chr>         <chr>                <dbl>
1 P 2   male  White (not Hi~ Jacob        Roberge       Verum                  152
2 P 7   male  White (not Hi~ Hans         Cardinal      Placebo                142
3 P 15  male  Middle-Easter~ Anas         al-Azimi      Placebo                158
4 P 21  male  White (not Hi~ Brandon      Heller        Placebo                151
# i 2 more variables: `diaRR (mmHg)` <dbl>, HR <dbl>
```

```
rawdata |> filter_out(Sex == "male", `sysRR (mmHg)` > 140) #keeps both NA and FALSE
```

```
# A tibble: 21 x 9
   PatID Sex    Ethnicity    `Given name` `Family name` Treatment `sysRR (mmHg)`
   <chr> <chr>  <fct>        <chr>        <chr>         <chr>                <dbl>
 1 P 1   male   Middle-East~ Shaddaad     el-Minhas     Placebo                127
 2 P 3   female White (not ~ Savannah     Williams      Placebo                113
 3 P 4   male   White (not ~ Russell      Miltenberger  Placebo                133
 4 P 5   male   White (not ~ Greg         Huter         Placebo                126
 5 P 6   male   White (not ~ Sean         Mandel        Placebo                138
 6 P 8   male   Hispanic     Vince        Duran         Verum                  132
 7 P 9   male   Asian or Pa~ Roger        Dhindsa       Verum                  126
 8 P 10  male   Hispanic     Omar         Marley        Verum                  136
 9 P 11  male   White (not ~ Andrew       Heller        Verum                  139
10 P 12  male   White (not ~ Nathaniel    Haynes        Verum                  119
# i 11 more rows
# i 2 more variables: `diaRR (mmHg)` <dbl>, HR <dbl>
```

3.9.3.3 Exercise: tibble

Think of a cruet_stand / Gewürzmenage

- define n_elements <- 5*10^3
- create a tibble "menage" with columns saltshaker, peppercaster and n_elements each for saltgrain and pepperflake
- print saltshaker (tibble with 1 columns)
- print salt (content of column, all saltgrains)
- print 100 saltgrains



3.9.4 List

While matrix, data.frames, and tibbles always have the same number of rows for each column, sometimes different lengths are required. A list can handle all kinds of data with different number of elements for each sublist. This is a typical output format for statistical functions and is useful for collecting e.g. result tables or figures. Package rlist provides useful tools.

3.9.4.1 Creation

```
shopping <- list(
  beverages = c(
    "beer", "water",
    "gin(not Gordons!!)", "tonic"
  ),
  snacks = c("chips", "pretzels"),
  nonfood = c("DVDs", "Akku"),
  mengen = 1:10,
  volumen = rnorm(50, 100, 2)
)
shopping
```

```
$beverages
[1] "beer"               "water"              "gin(not Gordons!!)"
[4] "tonic"

$snacks
[1] "chips"    "pretzels"

$nonfood
[1] "DVDs" "Akku"

$mengen
 [1]  1  2  3  4  5  6  7  8  9 10

$volumen
 [1] 100.20427 101.57779  95.20183 100.05557 101.11299 103.04200 103.44314
 [8]  96.59841  99.28831  99.46499 101.35992  99.92323  95.05061  98.06574
[15] 103.15413  99.22843  98.80153  99.37971 104.23233 100.57505 103.19573
[22] 100.57648  97.68568 100.87445  98.07713  98.70284 101.20433 100.02141
[29]  96.73696  98.48913 100.21616 100.94879  95.41324 100.74757  97.28826
[36]  98.87935  96.96170 102.15186 100.25006 102.60369 101.19669 101.62505
[43]  99.75774 101.18467  98.74629 102.89916  98.51234 101.68294 100.58268
[50]  97.81400
```

3.9.4.2 Indexing

```
shopping$snacks
```

```
[1] "chips"    "pretzels"
```

```
shopping[1] # returns a list
```

```
$beverages
[1] "beer"                "water"               "gin(not Gordons!!)"
[4] "tonic"
```

```
shopping[[1]] # returns a vector
```

```
[1] "beer"                "water"               "gin(not Gordons!!)"
[4] "tonic"
```

```
str(shopping[1])
```

```
List of 1
 $ beverages: chr [1:4] "beer" "water" "gin(not Gordons!!)" "tonic"
```

```
str(shopping[[1]])
```

```
 chr [1:4] "beer" "water" "gin(not Gordons!!)" "tonic"
```

```
str(shopping$beverages)
```

```
 chr [1:4] "beer" "water" "gin(not Gordons!!)" "tonic"
```

```
shopping[1][2]
```

```
$<NA>
NULL
```

```
shopping[[1]][2]
```

```
[1] "water"
```

```
shopping$beverages[2]
```

```
[1] "water"
```

```
t_out <- t.test(
  x = rnorm(n = 20, mean = 10, sd = 1),
  y = rnorm(20, 12, 1)
)
str(t_out)
```

```
List of 10
 $ statistic  : Named num -3.52
  ..- attr(*, "names")= chr "t"
 $ parameter  : Named num 32.7
  ..- attr(*, "names")= chr "df"
 $ p.value    : num 0.00129
 $ conf.int   : num [1:2] -1.53 -0.41
  ..- attr(*, "conf.level")= num 0.95
 $ estimate   : Named num [1:2] 10.6 11.6
  ..- attr(*, "names")= chr [1:2] "mean of x" "mean of y"
 $ null.value : Named num 0
  ..- attr(*, "names")= chr "difference in means"
 $ stderr     : num 0.276
 $ alternative: chr "two.sided"
 $ method     : chr "Welch Two Sample t-test"
 $ data.name  : chr "rnorm(n = 20, mean = 10, sd = 1) and rnorm(20, 12, 1)"
 - attr(*, "class")= chr "htest"
```

```
t_out$p.value
```

```
[1] 0.001293516
```

```
t_out |> pluck("p.value")
```

```
[1] 0.001293516
```

3.10 Control structures

3.10.1 Loops

Repetitive tasks like computation of descriptive statistics over many variables or repeated simulations of data can be declared inside of a loop. There are functions (like summarise(across(…))) that create those repetitions internally, but often doing this explicitly improves readability or helps solving various tasks like describing AND plotting data.

3.10.1.1 for-loop

In a for-loop, we can define the number of runs in advance, e.g. by the number of variables to describe. There are 2 ways/styles, how to define this number:

1. by creating an index variable with an integer vector 1,2,3, … number of runs/variables

2. by creating an index containing e.g. colnames

```
# integer index
print("### Game of Loops ###")
```

```
[1] "### Game of Loops ###"
```

```
for (season_i in 1:3) {
  cat(paste("GoL Season", season_i, "\n"))
  for (episode_i in 1:5) {
    cat(paste0(
      "   GoL S.", season_i,
      " Episode ", episode_i, "\n"
    ))
  }
  cat("\n")
}
```

```
GoL Season 1
   GoL S.1 Episode 1
   GoL S.1 Episode 2
   GoL S.1 Episode 3
   GoL S.1 Episode 4
   GoL S.1 Episode 5

GoL Season 2
   GoL S.2 Episode 1
   GoL S.2 Episode 2
   GoL S.2 Episode 3
   GoL S.2 Episode 4
   GoL S.2 Episode 5

GoL Season 3
   GoL S.3 Episode 1
   GoL S.3 Episode 2
   GoL S.3 Episode 3
   GoL S.3 Episode 4
   GoL S.3 Episode 5
```

```
# content index
## names of elements
for (col_i in colnames(rawdata)) {
  cat(col_i, "\n")
```

```
  print(str(rawdata |> pull(col_i)))
}
```

```
PatID
 chr [1:25] "P 1" "P 2" "P 3" "P 4" "P 5" "P 6" "P 7" "P 8" "P 9" "P 10" ...
NULL
Sex
 chr [1:25] "male" "male" "female" "male" "male" "male" "male" "male" ...
NULL
Ethnicity
 Factor w/ 6 levels "American Indian or Native Alaskan",..: 6 5 5 5 5 5 5 4 2 4 ...
NULL
Given name
 chr [1:25] "Shaddaad" "Jacob" "Savannah" "Russell" "Greg" "Sean" "Hans" ...
NULL
Family name
 chr [1:25] "el-Minhas" "Roberge" "Williams" "Miltenberger" "Huter" ...
NULL
Treatment
 chr [1:25] "Placebo" "Verum" "Placebo" "Placebo" "Placebo" "Placebo" ...
NULL
sysRR (mmHg)
 num [1:25] 127 152 113 133 126 138 142 132 126 136 ...
NULL
diaRR (mmHg)
 num [1:25] 80 78 89 80 92 68 80 66 53 60 ...
NULL
HR
 num [1:25] 95 83 80 89 96 99 91 98 87 92 ...
NULL
```

```
# automatic creation of integer index from elements
# for(col_i in 1:ncol(rawdata))){
for (col_i in seq_along(colnames(rawdata))) {
  print(colnames(rawdata)[col_i])
}
```

```
[1] "PatID"
[1] "Sex"
[1] "Ethnicity"
[1] "Given name"
[1] "Family name"
```

```
[1] "Treatment"
[1] "sysRR (mmHg)"
[1] "diaRR (mmHg)"
[1] "HR"
```

```
for (col_i in seq_len(length(colnames(rawdata)))) {
  print(colnames(rawdata)[col_i])
}
```

```
[1] "PatID"
[1] "Sex"
[1] "Ethnicity"
[1] "Given name"
[1] "Family name"
[1] "Treatment"
[1] "sysRR (mmHg)"
[1] "diaRR (mmHg)"
[1] "HR"
```

```
for (col_i in 1:length(colnames(rawdata))) {
  print(colnames(rawdata)[col_i])
}
```

```
[1] "PatID"
[1] "Sex"
[1] "Ethnicity"
[1] "Given name"
[1] "Family name"
[1] "Treatment"
[1] "sysRR (mmHg)"
[1] "diaRR (mmHg)"
[1] "HR"
```

```
# edge-case of 0 elements -> 0 runs
n_gaussvars <- 0
for (col_i in seq_len(n_gaussvars)) {
  print(colnames(rawdata)[col_i])
}

for (col_i in 1:n_gaussvars) {
  print(colnames(rawdata)[col_i])
}
```

```
[1] "PatID"
character(0)
```

3.10.1.2 while-loops

If not number of repetitions is know, but a condition.

```
test <- 0
while (test < 10) {
  print(test)
  test <- test + 1
}
```

```
[1] 0
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
```

3.10.2 Conditions

3.10.2.1 if else

We can run code if condition(s) are true:

```
sex <- "female"
if (sex == "male") {
  print("Male")
} else {
  print("Female")
}
```

```
[1] "Female"
```

```r
if (sex == "male") {
  print("Male")
}

if (sex != "male") {
  print("Female")
}
```

```
[1] "Female"
```

```r
testvar <- 4
if (testvar %in% c(1, 3, 5)) {
  print("uneven")
} else {
  print("probably even")
}
```

```
[1] "probably even"
```

```r
TRUE & FALSE # AND
```

```
[1] FALSE
```

```r
all(TRUE, FALSE)
```

```
[1] FALSE
```

```r
(1 < 10) & (sex == "male")
```

```
[1] FALSE
```

```r
all(1 < 10, sex == "male")
```

```
[1] FALSE
```

```r
TRUE | FALSE # OR
```

```
[1] TRUE
```

```r
any(TRUE, FALSE)
```

```
[1] TRUE
```

```
(1 > 10) | (1 < 5)
```

```
[1] TRUE
```

```
age <- 5
(sex == "female" & age <= 12) | (sex == "male" & age <= 14)
```

```
[1] TRUE
```

```
if (
  any(
    all(sex == "female", age <= 12),
    all(sex == "male", age <= 14)
  )) {
  cat("kid is still growing\n")
}
```

```
kid is still growing
```

3.10.2.2 ifelse

We can get text conditionally:

```
sex <- "female"
print(ifelse(test = sex == "male",
  yes = "is male",
  no = "is female"
))
```

```
[1] "is female"
```

```
p <- .012
paste0(
  "That is ",
  ifelse(test = p <= .05, yes = "", no = "not "),
  "significant"
)
```

```
[1] "That is significant"
```

```r
if (p > .05) {
  sign_out <- "not "
} else {
  sign_out <- ""
}
paste0(
  "That is ",
  sign_out,
  "significant"
)
```

```
[1] "That is significant"
```

3.10.2.3 case_when / replace_when / recode_values / replace_values

When there are many tests to do, `case_when`/`case_replace` or `recode_values`/`replace_values` (`case_match` is deprecated) are nice replacements for `ifelse`. While case_when allows complex conditions, recode_values is used for simple comparisons:

```r
rawdata <- mutate(
  .data = rawdata,
  Hypertension = case_when(
    `sysRR (mmHg)` < 120 & `diaRR (mmHg)` < 70 ~ "normotensive",
    `sysRR (mmHg)` < 160 & `diaRR (mmHg)` <= 80 ~ "borderline",
    .default = "hypertensive"
  ),
  # `prescribe something?` = recode_values( #case_match(
  #   Hypertension,
  #   "hypertensive" ~ "yes",  # lookup value ~ new value
  #   "borderline" ~ "possibly",
  #   "normotensive" ~ "no"
  # )
  `prescribe something?` = recode_values(
    Hypertension,
    from = c("hypertensive", "borderline", "normotensive"), # lookup-tables
    to = c("yes", "possibly", "no")
  )
)
rawdata |>
  select(contains("RR"), Hypertension, contains("pres"))
```

```
# A tibble: 25 x 4
```

```
   `sysRR (mmHg)` `diaRR (mmHg)` Hypertension `prescribe something?`
            <dbl>          <dbl> <chr>        <chr>
 1            127             80 borderline   possibly
 2            152             78 borderline   possibly
 3            113             89 hypertensive yes
 4            133             80 borderline   possibly
 5            126             92 hypertensive yes
 6            138             68 borderline   possibly
 7            142             80 borderline   possibly
 8            132             66 borderline   possibly
 9            126             53 borderline   possibly
10            136             60 borderline   possibly
# i 15 more rows
```

```
p <- 0.07
paste0(
  "That is ",
  case_when(p <= .05 ~ "", p <= .1 ~ "borderline ", .default = "not "),
  "significant"
)
```

```
[1] "That is borderline significant"
```

3.10.3 Exercise: `for` and `while` loops, and `if/ifelse/case_xxx`

3.10.3.1 1. Analyzing Gene Expression Data (Using `for` loop and `if/else`)

Scenario: You have a data-set of gene expression levels (e.g., RNA-seq counts) for several genes across different samples. You want to identify genes that are either highly expressed or lowly expressed based on a threshold.

Task:

- **Create Sample Data:** Generate a numeric vector representing expression levels for 20 genes (mean = 100, SD = 30).
- **Classify Genes by their Expression Level:**
  - Using a `for` loop, iterate through each gene's expression level.
  - Inside the loop, use an `if/else if/else` statement to categorize each gene:
    - If expression is above 120, print `paste("Gene", i, "is highly expressed:", gene_expression[i])`.
    - If expression is below 70, print `paste("Gene", i, "is lowly expressed:", gene_expression[i])`.
    - Otherwise, print `paste("Gene", i, "is moderately expressed:", gene_expression[i])`.
  - Create a similar loop using `case_when` to simplify the rules.

3.10.3.2 2. Simulating Population Growth (Using `while` loop)

Scenario: You want to simulate the growth of a bacterial population over time until it reaches a certain threshold.

Task:

- **Set Initial Conditions:**
  - Start with `population_size <- 100`.
  - Set a `growth_rate <- 1.1` (meaning 10% increase per generation).
  - Set a `carrying_capacity <- 1000`.
  - Initialize `generation <- 0`.
- **Simulate Growth:**

- Use a `while` loop that continues as long as `population_size < carrying_capacity`.
- Inside the loop:
- Update `population_size <- population_size * growth_rate`.
- Increment `generation <- generation + 1`.
- Print the `population_size` and `generation` at each step.

- **Add a Condition:** When at the last run of the loop, print a message indicating how many generations it took to reach the carrying capacity.

3.10.3.3 3. Classifying Species Based on Traits (Using `for` loop and `ifelse`)

Scenario: You have a dataset of different plant species and two of their traits: leaf length (cm) and number of petals. You want to classify them into broad groups.

Task:

- **Create Sample Data:**
  - `species <- c("Oak", "Maple", "Cherry", "Rose", "Lily", "Daisy", "Sunflower")`
  - `leaf_length <- c(15, 12, 8, 4, 18, 5, 25)`
  - `petal_count <- c(0, 0, 5, 20, 6, 30, 0)` (0 for trees, flowers have petals)
- **Categorize Species:**
- Create an empty vector `species_type <- character(length(species))` to store the results.
- Use a `for` loop to iterate through each species.
- Inside the loop, use nested `ifelse` statements to determine `species_type` based on these rules:
  - If `petal_count[i] == 0`, it's a "Tree".
  - If `petal_count[i] > 10`, it's a "Many-petaled Flower".
  - If `petal_count[i] > 0`, it's a "Few-petaled Flower".
- Assign the result to `species_type[i]`.
- **Display Results:** After the loop, create a data frame or tibble with `species`, `leaf_length`, `petal_count`, and the new `species_type` column.

4 Regular expressions

4.1 Intro

Regex cheatsheet

stringr cheatsheet

Regular expressions are a ***powerful*** tool for searching and manipulating text data. They allow you to define specific patterns within sequences, which is particularly useful when analyzing biological data such as DNA or protein sequences. But more mundane, they are terribly useful for practical tasks like finding or renaming variables, correcting common typos, and checking input patterns.

Basic functions like `grep()` or `gsub()` are difficult to use in pipelines and not very intuitive, tidyverse functions from stringr like `str_detect()` or `str_replace()` are more verbose and easier to use.

Key symbols and their meanings:

- **.** (period): Matches any single character except a newline. For example, "A.T" would match "AAT", "AGT", "ACT", etc.
- + means 1 or more occurrences (.+), ? means zero or 1 ocurrence
- [] (square brackets): Defines a character class. Any character within the brackets will match. For example, "[ATGC]" matches any DNA nucleotide.
- **{}** (curly braces): Specify the number of occurrences of the preceding element. For example, "A{3}" matches exactly three consecutive "A"s, like in "AAA".
- **\d**: Matches any digit (0-9). For example, "\d+" matches one or more digits, which could be useful for finding numerical identifiers in protein databases.
- **\D**: Matches any non-digit character.

4.2 Some basic examples:

4.2.1 Finding something

```
pacman::p_load(tidyverse)
starttext <- "Did grandma eat all the pizza?"
str_detect(string = starttext, pattern = "pizza")
```

```
[1] TRUE
```

```
str_detect(string = starttext, pattern = "pasta")
```

```
[1] FALSE
```

```
str_detect(string = starttext, pattern = "e.+a\\?$")
```

```
[1] TRUE
```

```
str_view(string = starttext, pattern = "e.+a\\?$")
```

```
[1] | Did grandma <eat all the pizza?>
```

```
str_detect(string = starttext, pattern = "grand[mp]a")
```

```
[1] TRUE
```

```
str_view(string = starttext, pattern = "grand[mp]a")
```

```
[1] | Did <grandma> eat all the pizza?
```

4.2.2 Replacing something

```
str_replace(
  string = starttext,
  pattern = "ma",
  replacement = "pa"
)
```

```
[1] "Did grandpa eat all the pizza?"
```

```
str_replace(
  string = starttext,
  pattern = " all ",
  replacement = " half "
)
```

```
[1] "Did grandma eat half the pizza?"
```

```
str_replace(
  string = starttext,
  pattern = "^(\\w+) (\\w+) (.+)",
  replacement = "\\2 \\1 \\3"
) |>
  str_to_sentence()
```

```
[1] "Grandma did eat all the pizza?"
```

```
str_replace(
  string = starttext,
  pattern = "^(\\w+) (\\w*) (.*)\\?",
  replacement = "\\2 \\1 \\3!"
) |>
  str_to_sentence()
```

```
[1] "Grandma did eat all the pizza!"
```

```
str_replace_all(
  string = starttext,
  pattern = c(
    "ma" = "pa",
    "all" = "half",
    "izz" = "ast"
  )
)
```

```
[1] "Did grandpa eat half the pasta?"
```

4.3 An example for their use in renaming variables:

```
temptibble <- tibble(
  cup_weigh = seq(10, 20, .5),
  CAPSIZE_cm = 5,
  height_of_cup_cm = rnorm(21, 10, .01)
)
colnames(temptibble)
```

```
[1] "cup_weigh"        "CAPSIZE_cm"       "height_of_cup_cm"
```

```
rename_with(
  .data = temptibble,
  .fn = \(x) str_replace_all(
    x,
    c(
      "CAP" = "CUP",
      "_(cm)" = " [\\1]",
      "(.*)_.+_(.+)( .*)" = "\\2\\1\\3",
      "_" = ""
    )
  ) |>
    str_to_sentence()
) |>
  colnames()
```

```
[1] "Cupweigh"       "Cupsize [cm]"   "Cupheight [cm]"
```

4.4 Exercise

```r
testset1 <- c("weight_mm", "height_cm", "age_yr", "temp_c")
# replace _ with space
# replace _ with space and add unit in brackets

testset2 <- c(
  "Meier", "Mayer", "Maier", "Meyer", "Mayr", "Hans Meier",
  "Maya", "Mayor", "Faltermeyer", "Meierhoven"
)
# find all variations of the name "Meier" (not Maya or Mayor etc given in 2nd row)

testset3 <- c("1980_12_30", "13.04.2005", "2005/04/25", "24121990")
# transform into YYYY-MM-DD

# advanced!!
testset4 <- c("pw2000", "That1sb3tt3r", "M@kesSense?", "NoDigits@this1")
# test pwd strength, rules: Upper, lower, special char, number, min 8 char long
```

5 Importing data

```
pacman::p_load(
  conflicted, tidyverse, wrappedtools, here,
  readxl, readODS, foreign, haven
)
```

5.1 Import from text files (.txt, .csv)

There are base functions like read.csv() and tidyverse-based updated ones like read_csv(). Different versions like read_csv2() or read_delim() have different settings for deliminators, number formats etc.

```
rawdata <- read_csv2(here("data/Medtest_e.csv"))
```

```
i Using "',' as decimal and "'.'" as grouping mark. Use `read_delim()` for more control.
```

```
Rows: 28 Columns: 25
-- Column specification --------------------------------------------------------
Delimiter: ";"
chr  (1): sex
dbl (24): randomcode, included, finalized, testmedication, size, weight, sys...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
Medtest_e <- read_delim(here("data/Medtest_e.csv"),
  delim = ";", escape_double = FALSE, locale = locale(
    date_names = "de",
    decimal_mark = ",", grouping_mark = "."
  ),
  trim_ws = TRUE
)
```

```
Rows: 28 Columns: 25
-- Column specification --------------------------------------------------------
Delimiter: ";"
chr  (1): sex
dbl (24): randomcode, included, finalized, testmedication, size, weight, sys...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
View(Medtest_e)
```

5.2 Import from Excel

5.2.1 Tidy Excel files

```
rawdata <- read_excel(path = here("data/Medtest_e.xlsx")) |> # cosmetic changes beyond impor
  rename_with(.fn = \(x) str_replace_all(x,
    pattern = "_",
    replacement = " "
  )) |>
  rename_with(
    .fn = str_to_title,
    .cols = !contains(c("BP", "BMI", "NY"))
  ) |>
  rename(
    `Size (cm)` = Size, # newname = oldname
    `Weight (kg)` = Weight
  ) |>
  select(-`Sex M`)
saveRDS(rawdata, file = here("data/rawdata.rds"))
```

5.2.2 Excel file with units row

1. Import names section and data section separately
2. Loop over all columns
    1. test for existence of unit, if not NA
    2. paste 1st row, " [", 2nd row,"]"
3. Use 1st row as colnames for data

```r
cn_temp <- read_excel(
  path = here("data/Medtest_e.xlsx"),
  range = "A1:Y2", col_names = FALSE,
  sheet = 2
)
```

```
New names:
* `` -> `...1`
* `` -> `...2`
* `` -> `...3`
* `` -> `...4`
* `` -> `...5`
* `` -> `...6`
* `` -> `...7`
* `` -> `...8`
* `` -> `...9`
* `` -> `...10`
* `` -> `...11`
* `` -> `...12`
* `` -> `...13`
* `` -> `...14`
* `` -> `...15`
* `` -> `...16`
* `` -> `...17`
* `` -> `...18`
* `` -> `...19`
* `` -> `...20`
* `` -> `...21`
* `` -> `...22`
* `` -> `...23`
* `` -> `...24`
* `` -> `...25`
```

```r
for (col_i in colnames(cn_temp)) {
  if (!is.na(cn_temp |> slice(2) |> pull(col_i))) {
    cn_temp[1, col_i] <-
      paste0(cn_temp[1, col_i], " [", cn_temp[2, col_i], "]")
  }
}

rawdata <- read_excel(
  path = here("data/Medtest_e.xlsx"),
  skip = 2, col_names = FALSE,
```

```
  sheet = 2
)
```

```
New names:
* `` -> `...1`
* `` -> `...2`
* `` -> `...3`
* `` -> `...4`
* `` -> `...5`
* `` -> `...6`
* `` -> `...7`
* `` -> `...8`
* `` -> `...9`
* `` -> `...10`
* `` -> `...11`
* `` -> `...12`
* `` -> `...13`
* `` -> `...14`
* `` -> `...15`
* `` -> `...16`
* `` -> `...17`
* `` -> `...18`
* `` -> `...19`
* `` -> `...20`
* `` -> `...21`
* `` -> `...22`
* `` -> `...23`
* `` -> `...24`
* `` -> `...25`
```

```
colnames(rawdata) <- cn_temp[1, ]
```

5.3 ODS files

Package readODS provides similar functionality for OpenOffice/LibreOffice files.

5.4 Import from SPSS/SAS

Import from SPSS generic files is implemented in various packages:

- foreign::read.spss is a more base approach,
  - on the positive side it has an option to read in value labels
  - on the other hand it returns lists or data frames, so casting into tibble is advised
- haven::read_sav comes from tidyverse
  - variable- and value-labels are imported into attributes
  - as_factor uses value labels

```
import1 <- read.spss(
  file = here("data/Zellbeads.sav"),
  to.data.frame = TRUE,
  use.value.labels = TRUE
)
```

```
re-encoding from CP1252
```

```
str(import1)
```

```
'data.frame':   360 obs. of  5 variables:
 $ bead_nr   : num  1 2 3 4 5 6 7 8 9 10 ...
 $ ZahlZellen: num  NA NA NA NA NA NA NA NA NA NA ...
 $ Wachstum  : num  135 111 101 115 135 ...
 $ Passage   : num  1 2 3 4 5 6 1 2 3 4 ...
 $ Bedingung : Factor w/ 3 levels "Kontrolle","AngII",..: 1 1 1 1 1 1 1 1 1 1 ...
 - attr(*, "variable.labels")= Named chr(0)
  ..- attr(*, "names")= chr(0)
 - attr(*, "codepage")= int 1252
```

```
as_tibble(import1)
```

```
# A tibble: 360 x 5
   bead_nr ZahlZellen Wachstum Passage Bedingung
     <dbl>      <dbl>    <dbl>   <dbl> <fct>
 1       1         NA     135.       1 Kontrolle
 2       2         NA     111.       2 Kontrolle
 3       3         NA     101.       3 Kontrolle
 4       4         NA     115.       4 Kontrolle
 5       5         NA     135.       5 Kontrolle
 6       6         NA     114.       6 Kontrolle
 7       7         NA     134.       1 Kontrolle
```

```
 8       8         NA     119.       2 Kontrolle
 9       9         NA     127.       3 Kontrolle
10      10         NA     119.       4 Kontrolle
# i 350 more rows
```

```
import2 <- read_sav(file = here("data/Zellbeads.sav"))
str(import2$Bedingung)
```

```
 dbl+lbl [1:360] 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
 @ format.spss: chr "F8.0"
 @ labels     : Named num [1:3] 1 2 3
  ..- attr(*, "names")= chr [1:3] "Kontrolle" "AngII" "Whatever"
```

```
attr(import2$Bedingung, "labels")
```

```
Kontrolle     AngII  Whatever
        1         2         3
```

```
import2 <- mutate(import2,
  Bedingung = as_factor(Bedingung)
)
str(import2$Bedingung)
```

```
 Factor w/ 3 levels "Kontrolle","AngII",..: 1 1 1 1 1 1 1 1 1 1 ...
```

6 Changing structure wide <–> long

There are many examples and explanations in the tidyR cheatsheet

https://rstudio.github.io/cheatsheets/html/tidyr.html

and the vignette:

https://tidyr.tidyverse.org/articles/pivot.html

When working with repeated measures (e.g. follow-ups or changes over shorter periods of time) there are two typical formats:

1. wide data:

ID	Var1Time1	Var1Time2	Var2Time1	Var2Time2
P1				
P2				
P3				

2. long data

ID	Time	Var1	Var2
P1	1		
P1	2		
P2	1		
P2	2		
P3	1		
P3	2		

While long data can be seen as the tidier version and is necessary for many statistical procedures, the wide format makes computation of differences and procedures as the t-test for dependent samples easier. Package tidyr provides the functions pivot_wider and pivot_longer for conversations between those forms. Another use-case is the plotting of several variables into ggplot facets, which can be achieved by combining those variables into a single column. Summarizing several variables and grouped data may result in a wide table with groups as rows and variables as columns, contrary to the common opposite form, another use-case.

```
pacman::p_load(conflicted, tidyverse, wrappedtools)
```

6.1 Example 1: single repeated measure

```
n <- 3
wide_data <- tibble(
  ID = paste("P", 1:n),
  Var1 = LETTERS[1:n],
  Var2Time1 = rnorm(n = n, mean = 100, sd = 15),
  Var2Time2 = Var2Time1 + rnorm(n, 10, 5)
)
wide_data
```

```
# A tibble: 3 x 4
  ID    Var1  Var2Time1 Var2Time2
  <chr> <chr>     <dbl>     <dbl>
1 P 1   A          107.      108.
2 P 2   B          121.      131.
3 P 3   C          113.      120.
```

```
long_data <- pivot_longer(
  data = wide_data,
  cols = contains("Time")
)
long_data
```

```
# A tibble: 6 x 4
  ID    Var1  name      value
  <chr> <chr> <chr>     <dbl>
1 P 1   A     Var2Time1  107.
2 P 1   A     Var2Time2  108.
3 P 2   B     Var2Time1  121.
4 P 2   B     Var2Time2  131.
5 P 3   C     Var2Time1  113.
6 P 3   C     Var2Time2  120.
```

6.2 Example 2: several repeated measures

```
set.seed(42)
wide_data <- tibble(
```

```
  ID = paste("P", 1:n),
  Var1Time1 = rnorm(n = n, mean = 100, sd = 15),
  Var1Time2 = Var1Time1 + rnorm(n, 10, 5),
  Var2Time1 = rnorm(n = n, mean = 10, sd = 2),
  Var2Time2 = Var2Time1 + rnorm(n, 0, 1),
  Var3 = LETTERS[1:n]
)
wide_data
```

```
# A tibble: 3 x 6
  ID    Var1Time1 Var1Time2 Var2Time1 Var2Time2 Var3
  <chr>     <dbl>     <dbl>     <dbl>     <dbl> <chr>
1 P 1       121.       134.     13.0       13.0 A
2 P 2        91.5      104.      9.81      11.1 B
3 P 3       105.       115.     14.0       16.3 C
```

```
# version with intermediate step:
very_long_data <- pivot_longer(
  data = wide_data,
  cols = contains("Time"),
  names_to = c("Variable", "Time"),
  names_pattern = "(Var\\d+)(Time[12])",
  values_to = "Value"
)
very_long_data
```

```
# A tibble: 12 x 5
   ID    Var3  Variable Time   Value
   <chr> <chr> <chr>    <chr>  <dbl>
 1 P 1   A     Var1     Time1 121.
 2 P 1   A     Var1     Time2 134.
 3 P 1   A     Var2     Time1  13.0
 4 P 1   A     Var2     Time2  13.0
 5 P 2   B     Var1     Time1  91.5
 6 P 2   B     Var1     Time2 104.
 7 P 2   B     Var2     Time1   9.81
 8 P 2   B     Var2     Time2  11.1
 9 P 3   C     Var1     Time1 105.
10 P 3   C     Var1     Time2 115.
11 P 3   C     Var2     Time1  14.0
12 P 3   C     Var2     Time2  16.3
```

```r
long_data <- pivot_wider(very_long_data,
  names_from = Variable,
  values_from = Value
)
long_data
```

```
# A tibble: 6 x 5
  ID    Var3  Time   Var1  Var2
  <chr> <chr> <chr> <dbl> <dbl>
1 P 1   A     Time1 121.  13.0
2 P 1   A     Time2 134.  13.0
3 P 2   B     Time1  91.5  9.81
4 P 2   B     Time2 104.  11.1
5 P 3   C     Time1 105.  14.0
6 P 3   C     Time2 115.  16.3
```

As a better alternative with 1 step, value column names will be extracted from parts of the wide colnames. This requires either a separator (often something like "_") or a name pattern as regex with "(" and ")" indicating extraction parts:

```r
long_data <- pivot_longer(
  data = wide_data,
  cols = contains("Time"),
  names_to = c(".value", "Time"), # .value will be replaced dynamically
  # names_sep = "Time"
  names_pattern = "(Var\\d+)Time(\\d+)"
)
long_data
```

```
# A tibble: 6 x 5
  ID    Var3  Time   Var1  Var2
  <chr> <chr> <chr> <dbl> <dbl>
1 P 1   A     1     121.  13.0
2 P 1   A     2     134.  13.0
3 P 2   B     1      91.5  9.81
4 P 2   B     2     104.  11.1
5 P 3   C     1     105.  14.0
6 P 3   C     2     115.  16.3
```

In this example, (Var\d+) finds Var1, Var2, and Var3; they go into .value and will be used as the new colnames. Times serves as a separator, (\d+) finds 1 and 2, they go into column Time.

6.3 Example 3: long to wide

```
wide_again_data <- pivot_wider(
  data = long_data,
  names_from = Time,
  values_from = Var1:Var2,
  names_glue = "{.value}@{Time}" #cosmetic change from Var1_1 to Var1@Time1
)
wide_again_data
```

```
# A tibble: 3 x 6
  ID    Var3  `Var1@1` `Var1@2` `Var2@1` `Var2@2`
  <chr> <chr>    <dbl>    <dbl>    <dbl>    <dbl>
1 P 1   A        121.      134.    13.0      13.0
2 P 2   B         91.5     104.     9.81     11.1
3 P 3   C        105.      115.    14.0      16.3
```

6.4 More examples

6.4.1 Step 1: Create example data:

- 5 subjects per group, 2 groups A/B
- 3 measurements weight (V1, V2, V3) with random numbers,
  - means 46, 50,51
  - SDs 2
- 2 measurements length (V1, V3) #no visit 2!
  - means 120, 135
  - SDs 3

```
set.seed(42)
n <- 10
rawdata <-
  tibble(
    ID = paste("Pat", seq_len(n), sep = "#"),
    groups = rep(c("A", "B"), each = n / 2),
    weight_V1 = rnorm(n = n, mean = 46, sd = 2),
    weight_V2 = rnorm(n = n, mean = 50, sd = 2),
```

```
    weight_V3 = rnorm(n = n, mean = 51, sd = 2),
    size_V1 = rnorm(n = n, mean = 120, sd = 3),
    size_V3 = rnorm(n = n, mean = 135, sd = 3)
  )

head(rawdata)
```

```
# A tibble: 6 x 7
  ID    groups weight_V1 weight_V2 weight_V3 size_V1 size_V3
  <chr> <chr>      <dbl>     <dbl>     <dbl>   <dbl>   <dbl>
1 Pat#1 A           48.7      52.6      50.4    121.    136.
2 Pat#2 A           44.9      54.6      47.4    122.    134.
3 Pat#3 A           46.7      47.2      50.7    123.    137.
4 Pat#4 A           47.3      49.4      53.4    118.    133.
5 Pat#5 A           46.8      49.7      54.8    122.    131.
6 Pat#6 B           45.8      51.3      50.1    115.    136.
```

6.4.2 Step 2: Transform that data to a long form:

- 1 column for weight
- 1 column for length
- 1 column for measurement time named "Visit"

```
# with intermediate super_long step
rawdata_long <-
  # to superlong
  pivot_longer(
    data = rawdata,
    cols = contains("V"),
    names_to = c(          # splits weight_V1 into weight and V1
      "what_was_measured",
      "Visit"
    ),
    names_sep = "_",
    values_to = "weight_or_size"
  ) |>
  # from superlong to long
  pivot_wider(
    names_from = what_was_measured,
    values_from = weight_or_size
  )
head(rawdata_long)
```

```
# A tibble: 6 x 5
  ID    groups Visit weight  size
  <chr> <chr>  <chr>  <dbl> <dbl>
1 Pat#1 A      V1      48.7  121.
2 Pat#1 A      V2      52.6   NA
3 Pat#1 A      V3      50.4  136.
4 Pat#2 A      V1      44.9  122.
5 Pat#2 A      V2      54.6   NA
6 Pat#2 A      V3      47.4  134.
```

```
# single step approach
rawdata_long2 <-
  pivot_longer(
    data = rawdata,
    cols = contains("V"),
    names_to = c(".value", "Visit"),
    # .value will be replaced by weigh or size
    names_sep = "_"
  )
head(rawdata_long2)
```

```
# A tibble: 6 x 5
  ID    groups Visit weight  size
  <chr> <chr>  <chr>  <dbl> <dbl>
1 Pat#1 A      V1      48.7  121.
2 Pat#1 A      V2      52.6   NA
3 Pat#1 A      V3      50.4  136.
4 Pat#2 A      V1      44.9  122.
5 Pat#2 A      V2      54.6   NA
6 Pat#2 A      V3      47.4  134.
```

6.4.3 Step 3 Transform long to wide

```
# 2-steps
rawdata_wide <-
  pivot_longer(rawdata_long,
    cols = c(weight, size),
    names_to = "what_was_measured",
    values_to = "weight_or_size"
  ) |>
  pivot_wider(
    names_from = c(what_was_measured, Visit),
```

```
    # names created from 2 sources
    values_from = weight_or_size,
    names_sep = "_"
  )
head(rawdata_wide)
```

```
# A tibble: 6 x 8
  ID    groups weight_V1 size_V1 weight_V2 size_V2 weight_V3 size_V3
  <chr> <chr>      <dbl>   <dbl>     <dbl>   <dbl>     <dbl>   <dbl>
1 Pat#1 A           48.7    121.      52.6      NA      50.4    136.
2 Pat#2 A           44.9    122.      54.6      NA      47.4    134.
3 Pat#3 A           46.7    123.      47.2      NA      50.7    137.
4 Pat#4 A           47.3    118.      49.4      NA      53.4    133.
5 Pat#5 A           46.8    122.      49.7      NA      54.8    131.
6 Pat#6 B           45.8    115.      51.3      NA      50.1    136.
```

```
# 1step option
rawdata_wide2 <-
  pivot_wider(rawdata_long,
    values_from = c(weight, size),
    # values come from 2 sources, names will used in names_glue
    names_from = Visit,
    names_glue = "{.value}_{Visit}"
  )
head(rawdata_wide2)
```

```
# A tibble: 6 x 8
  ID    groups weight_V1 weight_V2 weight_V3 size_V1 size_V2 size_V3
  <chr> <chr>      <dbl>     <dbl>     <dbl>   <dbl>   <dbl>   <dbl>
1 Pat#1 A           48.7      52.6      50.4    121.      NA    136.
2 Pat#2 A           44.9      54.6      47.4    122.      NA    134.
3 Pat#3 A           46.7      47.2      50.7    123.      NA    137.
4 Pat#4 A           47.3      49.4      53.4    118.      NA    133.
5 Pat#5 A           46.8      49.7      54.8    122.      NA    131.
6 Pat#6 B           45.8      51.3      50.1    115.      NA    136.
```

6.5 Exercise: Reshaping Data

Scenario: You are a biologist studying the growth of chicks under different diets. The `ChickWeight` dataset in R contains observations on the weight of chicks over time, grouped by individual chick and diet. Understanding how to reshape data is crucial for different types of analyses and visualizations.

Goal: Practice transforming data between "long" and "wide" formats using `pivot_wider()` and `pivot_longer()`.

6.5.1 Part 1: From Long to Wide (`pivot_wider()`)

Sometimes, you might want to see all the measurements for a single individual (e.g., a chick) laid out in one row, with each time point becoming a separate column. This "wide" format can be useful for quick visual comparison of an individual's trajectory or for certain statistical tests that expect a specific column structure.

Instructions:

1. **Inspect the original data:**
    - Type `head(ChickWeight)` and `str(ChickWeight)` to understand its current "long" format (each row is a single observation of weight at a specific time for a specific chick).
    - Notice the `Time` and `weight` columns.
2. **Transform to Wide Format:**
    - Use `pivot_wider()` to transform the `ChickWeight` dataset.
    - You want `Time` values to become new column names.
    - You want the `weight` values to fill these new columns.
    - Each row should represent a unique `Chick` and `Diet`.
    - **Hint:** Think about `id_cols`, `names_from`, and `values_from`.
3. **Reflect:**
    - What are the new column names?
    - What does `NA` mean in this new `chick_weight_wide` dataset? (Hint: Not all chicks were measured at all time points, or some started later).

6.5.2 Part 2: From Wide to Long (`pivot_longer()`)

Imagine you received data from a collaborator where each time point (e.g., Day 0, Day 2, Day 4, etc.) is its own column. For many biological analyses, especially for plotting time series with `ggplot2` or running mixed-effects models, you need this data in a "long" format, where all the measurements are in a single `weight` column, and there's a separate `Time` column indicating when that measurement was taken.

Instructions:

1. **Start with the wide data:** We will use the `chick_weight_wide` tibble you created in Part 1.
2. **Transform to Long Format:**
    - Use `pivot_longer()` on `chick_weight_wide` (or `chick_weight_wide_example`).
    - You want to gather all the columns that represent `Time` points (e.g., `0`, `2`, `4`, ..., `21`) into two new columns: one for the `Time` itself and one for the `weight` measurement.
    - **Hint:** Think about `cols`, `names_to`, and `values_to`. You might also need `names_transform` to convert the `Time` column back to a numeric type.
3. **Reflect:**
    - Compare `chick_weight_long` to the original `ChickWeight` dataset. Are they identical in structure (ignoring the order of columns/rows which might vary slightly)?

7 Visualize data with ggplot

While there are various packages providing visualizations, here we are focusing on ggplot2 (grammar of graphics) as a very flexible and versatile approach with many extensions implemented in additional packages. See e.g. https://exts.ggplot2.tidyverse.org/.

```
pacman::p_load(
  conflicted, tidyverse, here,
  grid, gridExtra, car,
  ggsci, ggsignif, ggthemes, ggridges,
  # gganimate,
  ggforce,
  ggbeeswarm,
  wrappedtools,
  # emojifont,
  patchwork,
  GGally,
  ggprism,
  ggh4x # some ggplot hacks
)
conflicts_prefer(
  dplyr::filter,
  ggplot2::mean_cl_boot
) # solves name conflict
```

7.1 Example data

The typical examples use either diamonds from `ggplot2` or mtcars from `datasets`. There are help files for both.

```
head(diamonds)
```

```
# A tibble: 6 x 10
  carat cut       color clarity depth table price     x     y     z
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
```

```
2  0.21 Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
3  0.23 Good      E     VS1      56.9    65   327  4.05  4.07  2.31
4  0.29 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
5  0.31 Good      J     SI2      63.3    58   335  4.34  4.35  2.75
6  0.24 Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
```

```
head(mtcars)
```

```
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

Exercises will be using the penguins data set from package palmerpenguins. Beware that the same data is now part of package datasets, but with different column names!

7.2 Basic structure of a ggplot call

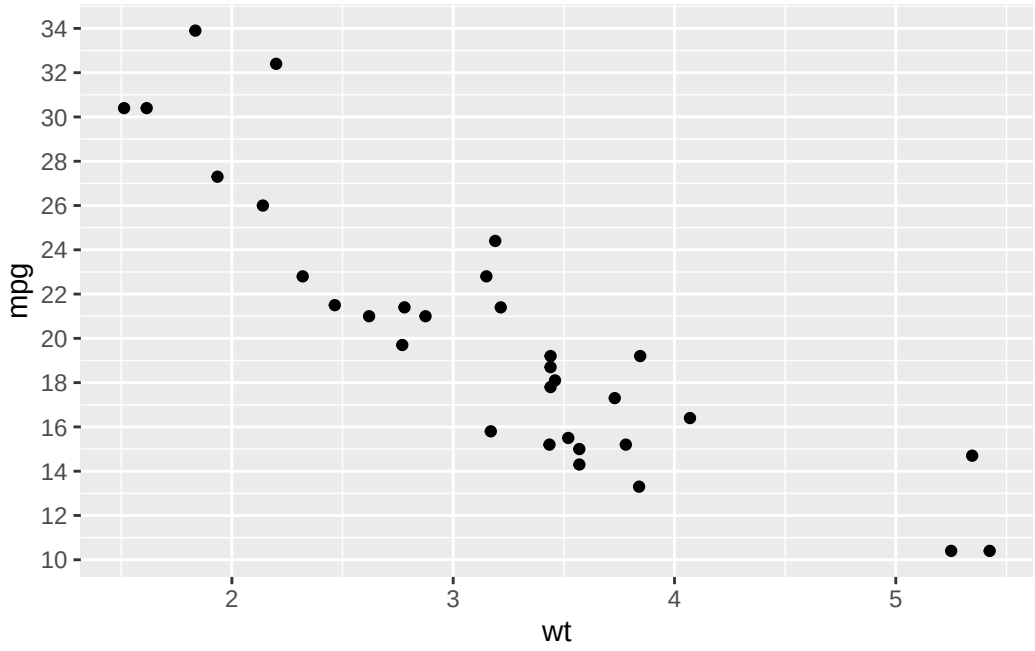
To create a figure, we at least use 2 function calls:

1. ggplot(data = my_data, mapping = aes(x = ..., y = ..., color = ..., shape = ...)) to define data and inside aes() some global defaults for aesthetic mappings, this (sort of) creates the canvas to draw on
2. geom_xxx() to define the geometry to be used to show the data, e.g. bar, boxplot, point

When mapping data to aesthetics, the class of data matters: Numerical data are interpreted as continuous, so a color heatmap is mapped rather than discrete colors, grouping of data requires factors / characters.

Extensive example:

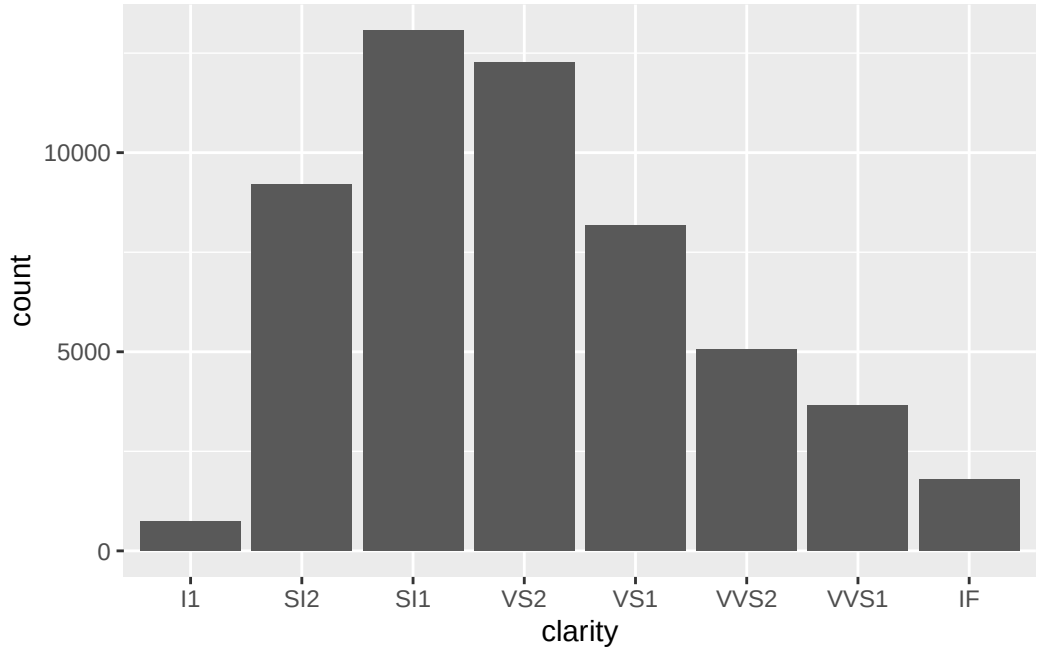
```
ggplot(
  data = mtcars,
  aes(x = wt, y = mpg)
) +
  geom_point() +
  scale_y_continuous(breaks = seq(0, 50, 2)) +
  theme(legend.position = "bottom")
```



ggplot(#base definition of plot
data = mtcars, # what data set to use
aes(x = **wt**, y = **mpg**)) + # which **variables** will be bound to ***aes****thetics*
geom_point() + # which **geom**etric form to show the data (bar, boxplot …)
scale_y_continuous(breaks = seq(0,50,2)) + # tuning of translation of data into aesthetics
theme(legend.position = "bottom") # all non-data aspects of a plot

Minimal example:

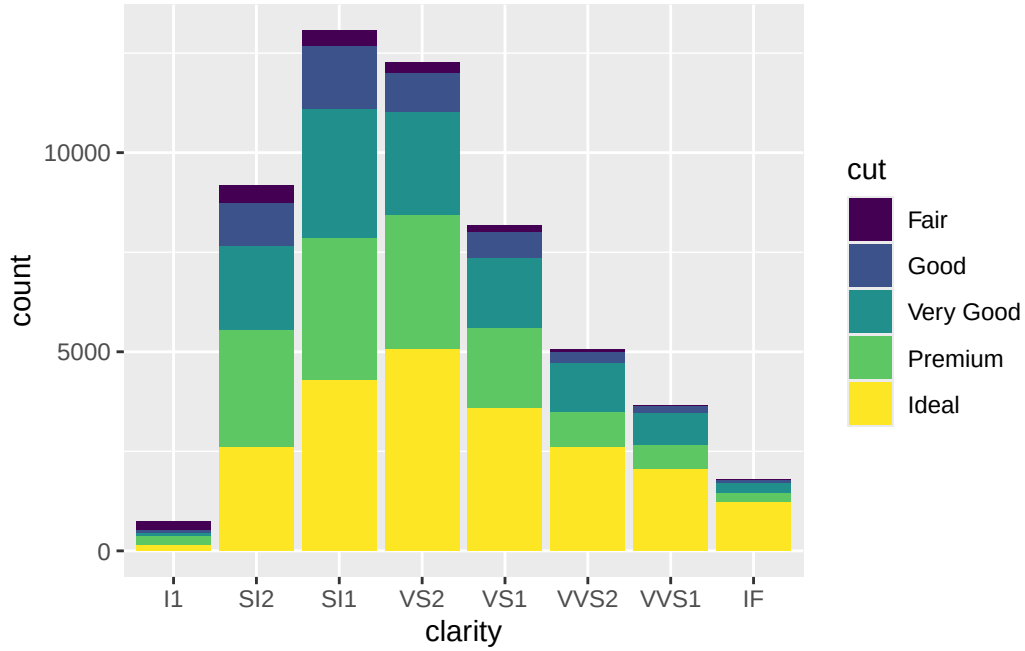
```
ggplot(data = diamonds, mapping = aes(x = clarity)) +
  geom_bar()
```



geom_bar() inherits the global aestetic x (*build a x-axis based on values in column clarity*) and does not need a y-axis definition, as it uses some in-build statistics ("count") and defines y.

We can add additional aesthetic parameters like fill-color. This automatically creates sub-groups for counting:

```
ggplot(data = diamonds, aes(x = clarity, fill = cut)) +
  geom_bar()
```



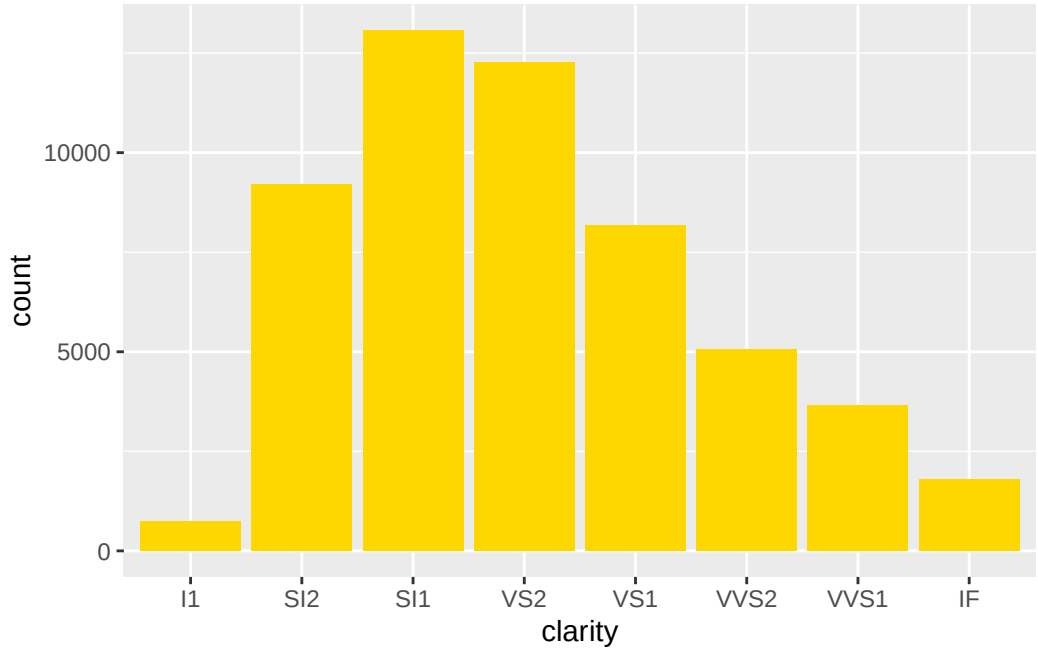
This graph represents this count table:

```
diamonds |>
  group_by(clarity, cut) |>
  count() |>
  pivot_wider(names_from = clarity, values_from = n)
```

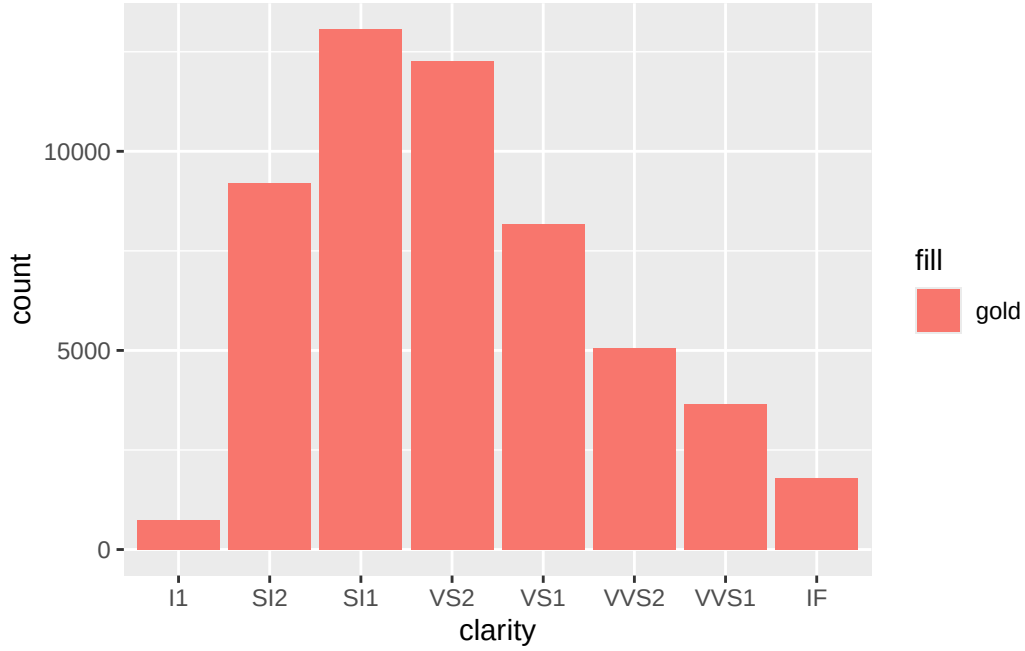
```
# A tibble: 5 x 9
# Groups:   cut [5]
  cut          I1   SI2   SI1   VS2   VS1  VVS2  VVS1    IF
  <ord>     <int> <int> <int> <int> <int> <int> <int> <int>
1 Fair        210   466   408   261   170    69    17     9
2 Good         96  1081  1560   978   648   286   186    71
3 Very Good    84  2100  3240  2591  1775  1235   789   268
4 Premium     205  2949  3575  3357  1989   870   616   230
5 Ideal       146  2598  4282  5071  3589  2606  2047  1212
```

Aesthetic parameters can represent data / have some meaning (as cut quality), but they can be defined to reflect your taste rather than data. In that case, you define them outside of `aes()`. Careful, as this may lead to confusion:

```
# aesthetics outside aes
ggplot(data = diamonds, aes(x = clarity)) +
  geom_bar(fill = "gold")
```

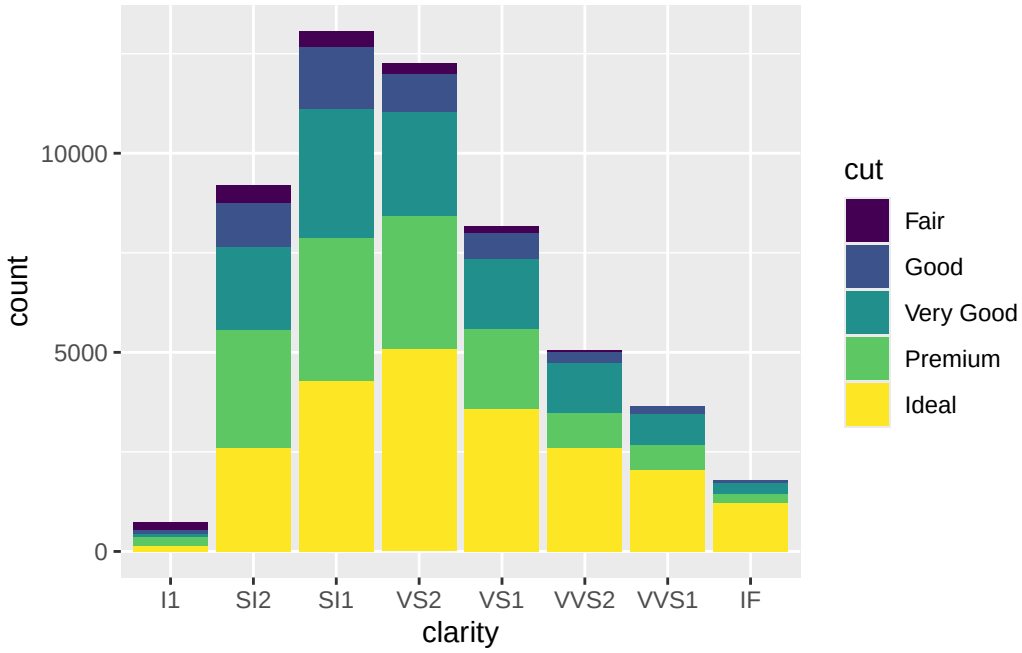



```
ggplot(data = diamonds, aes(x = clarity)) +
  geom_bar(aes(fill = "gold")) # should be outside aes!
```




```
ggplot(data = diamonds, aes(x = clarity)) +
```

```
geom_bar(aes(fill = cut)) # may be defined locally as well globally
```



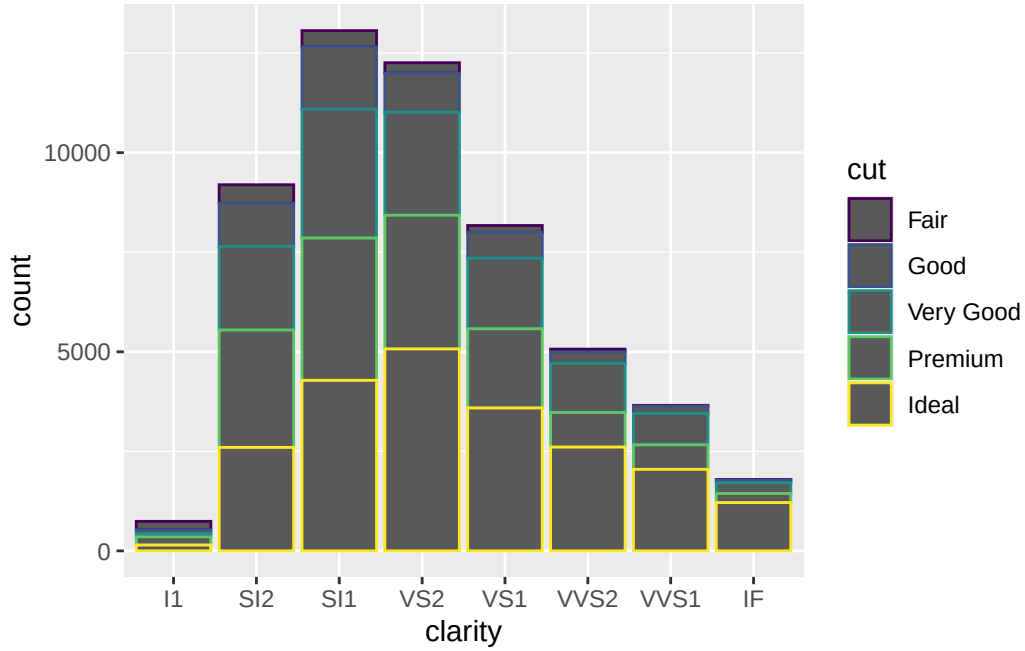
7.3 fill vs. color

Some elements (as e.g. the bar or boxplot) know 2 color elements:

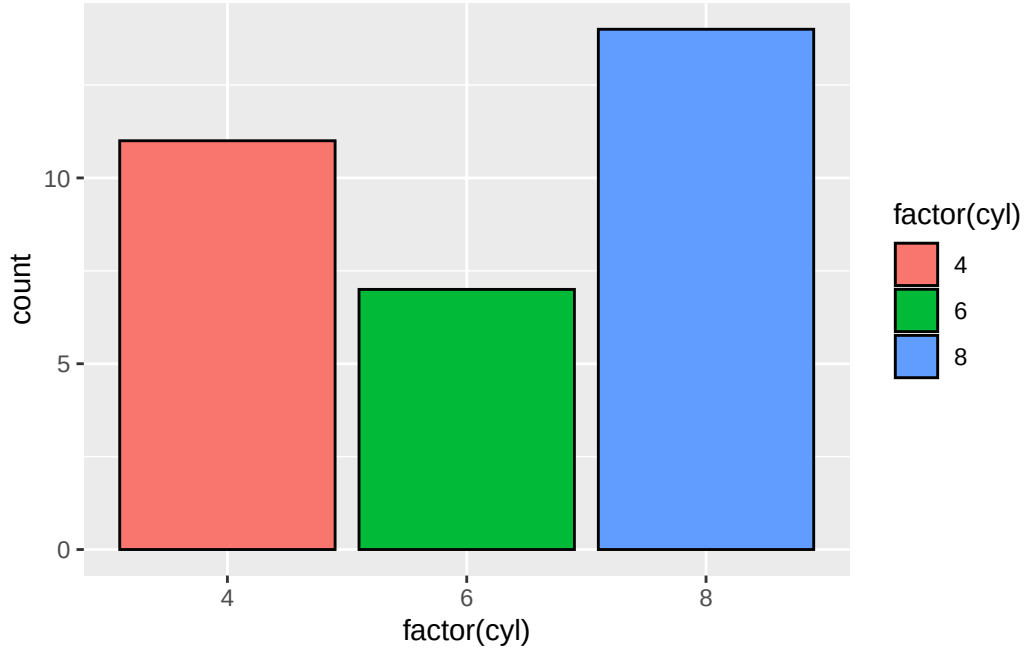
- inner color, defined by `fill`
- outer frame color, defined by `color`

Other elements (as e.g. the line) only have a single color definition, specified by color. And for some elements (as e.g. dots), it depends. See help for points for examples.

```
ggplot(data = diamonds, aes(x = clarity, color = cut)) +
  geom_bar()
```

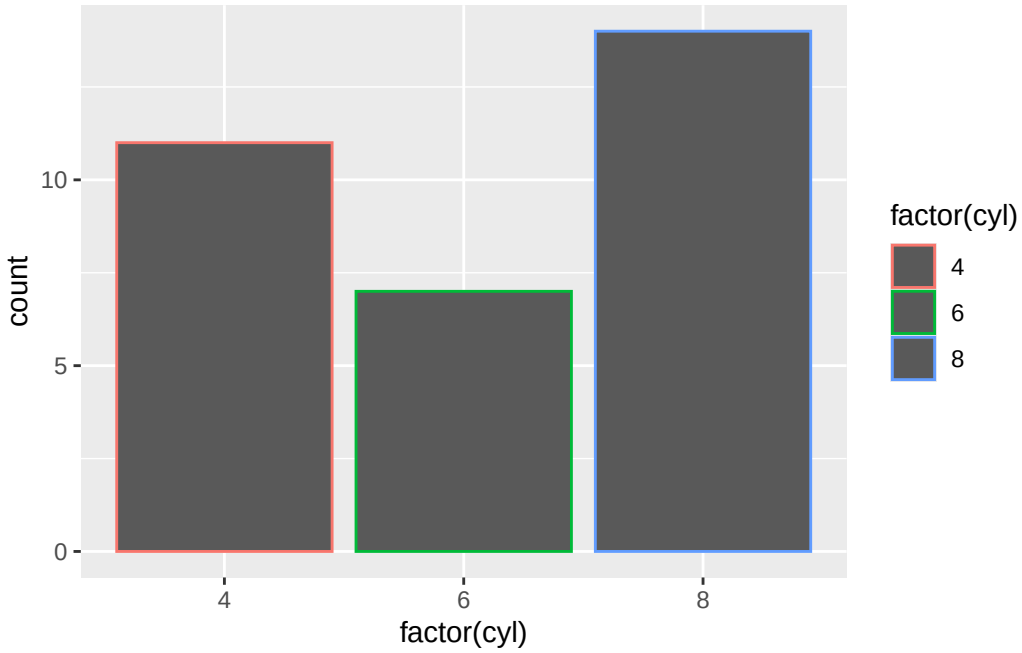



```
ggplot(data = mtcars, aes(factor(cyl), fill = factor(cyl))) +
  geom_bar(color = "black")
```




```
ggplot(data = mtcars, aes(factor(cyl), color = factor(cyl))) +
```

```
geom_bar()
```



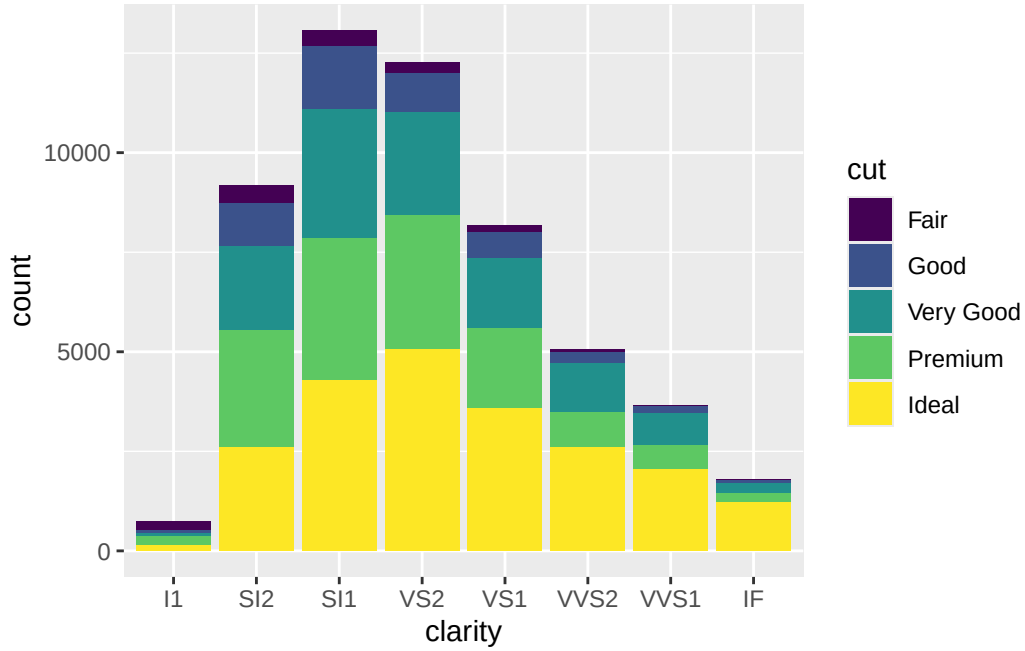
7.4 Color systems

ggplot2 comes with various color definitions, many external packages extend that. Manual definition of colors is possible as well. Redefining the mapping between data and aesthetics can be done with scale_... functions. Manual definitions of colors may use predefined color names ("red"), hexadecimal RGB definitions ("+FF0000"), or different numerical RGBs like rgb(1,0,0) or rgb(255,0,0, maxColorValue = 255)

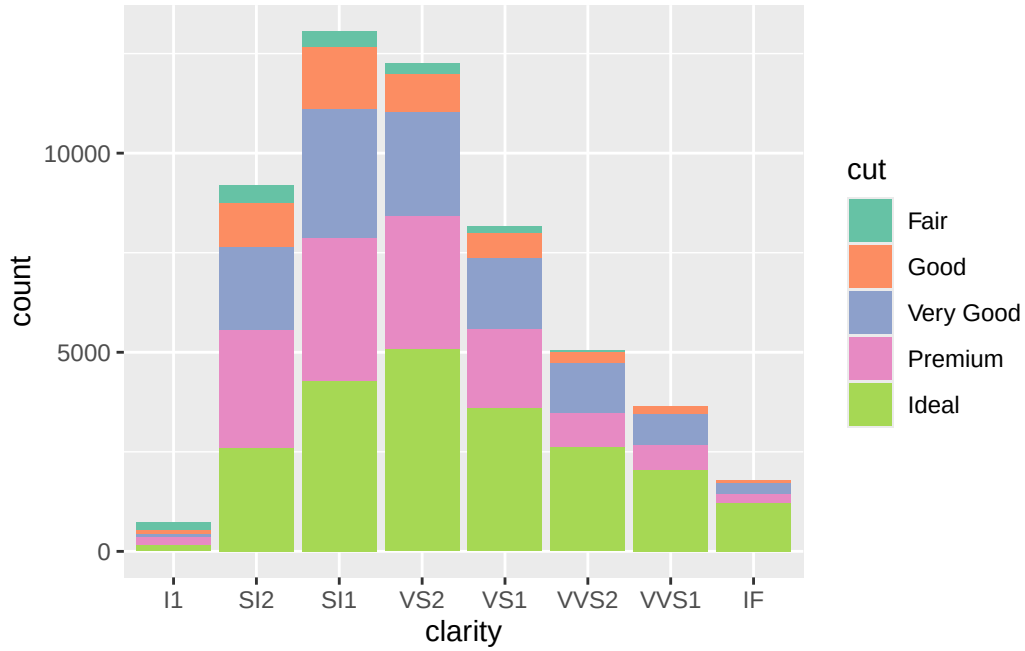
comprehensive overview for color palettes

For the demonstration, plot gets stored into a variable, this includes all data and plot definitions, nothing like jpg!

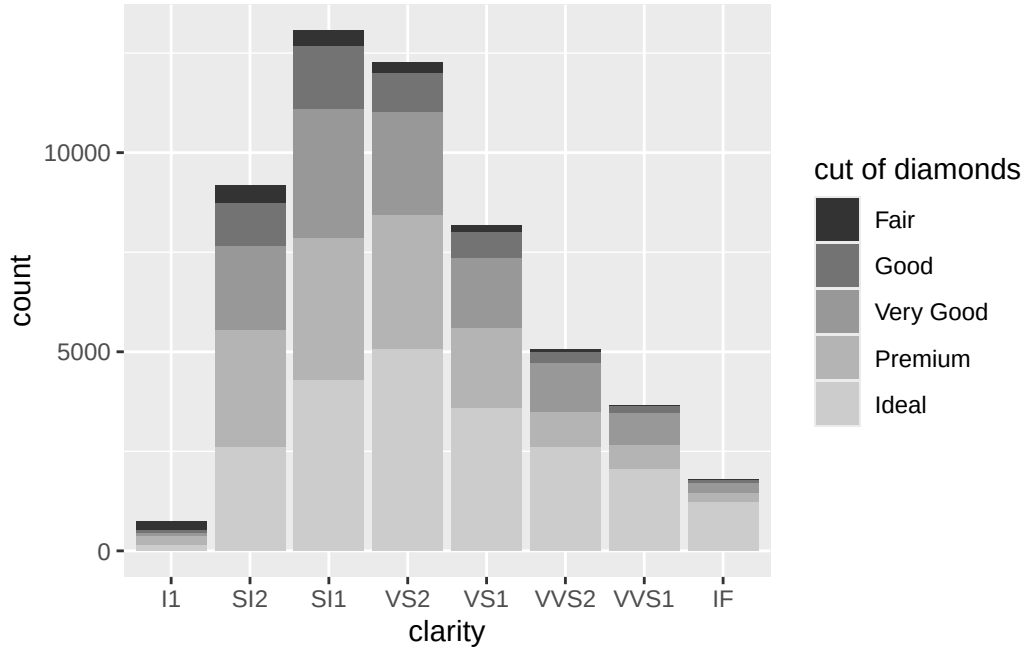
```
plottemp <- ggplot(data = diamonds, aes(x = clarity, fill = cut)) +
  geom_bar()
plottemp
```



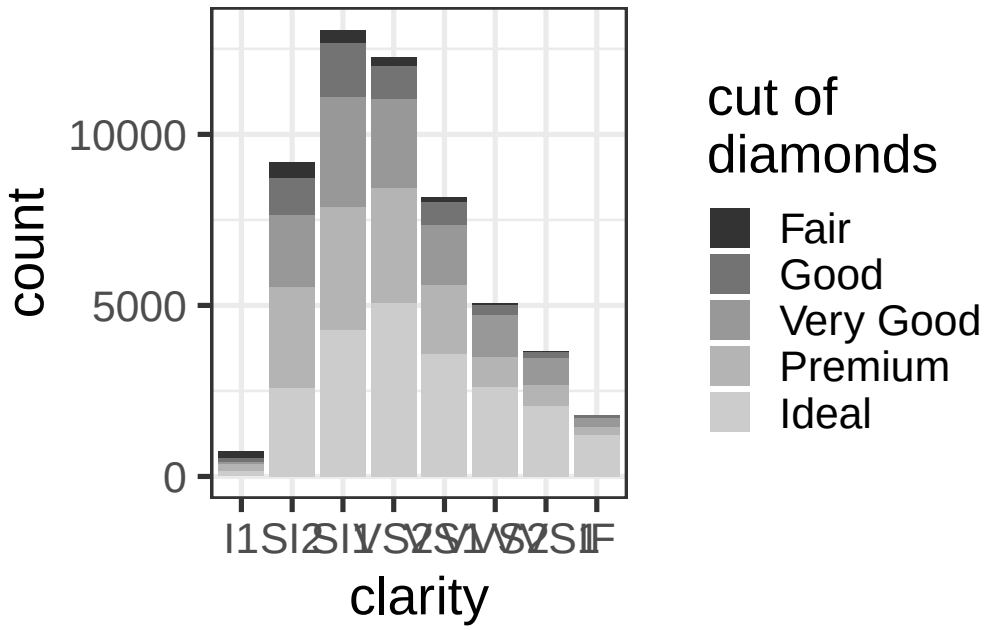

```
plottemp + scale_fill_brewer(palette = "Set2") # in-built "scale" for fill
```




```
plottemp + scale_fill_grey(name = "cut of diamonds")
```

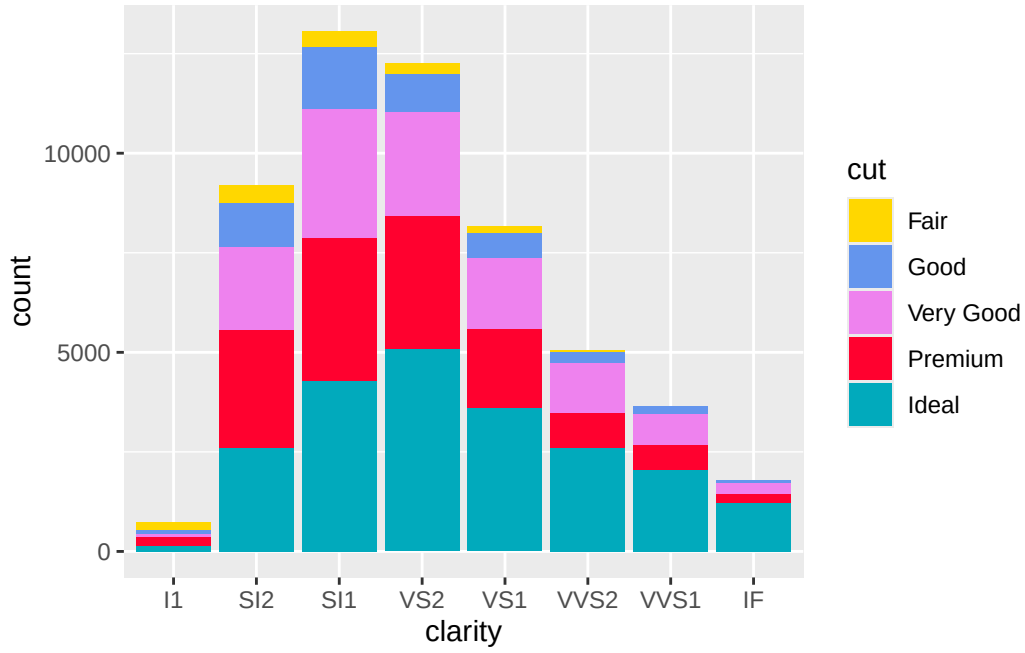



```
plottemp + scale_fill_grey(name = "cut of\ndiamonds") +
  theme_bw(base_size = 20) # avoids grey background
```



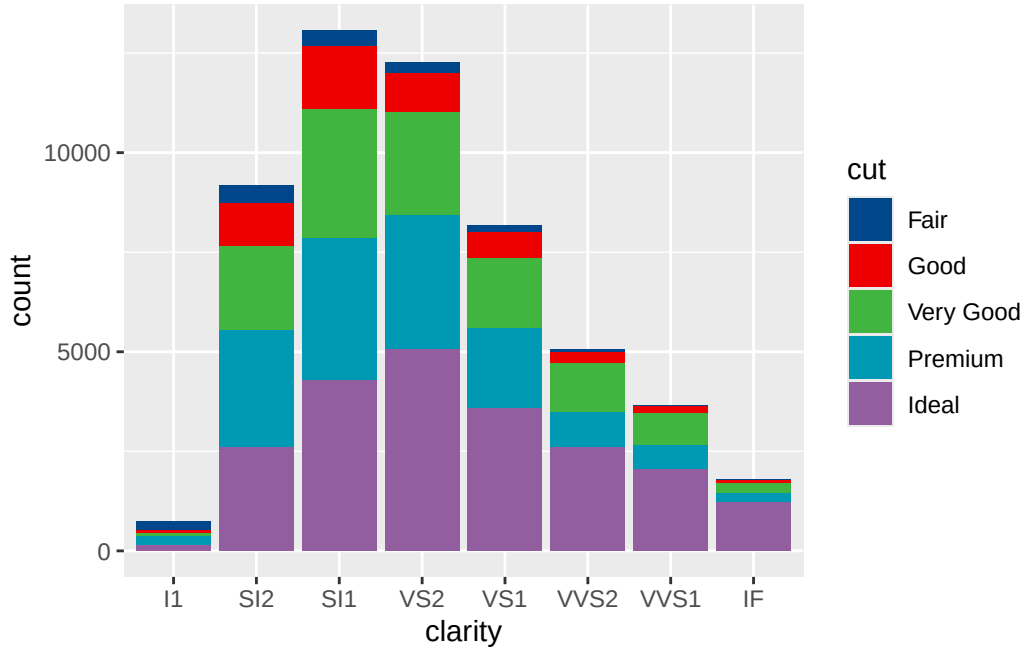

```
my_colors <- c("gold", "cornflowerblue", "violet", "#FF012F", "#01AABC")
```

```
plottemp + scale_fill_manual(values = my_colors)
```

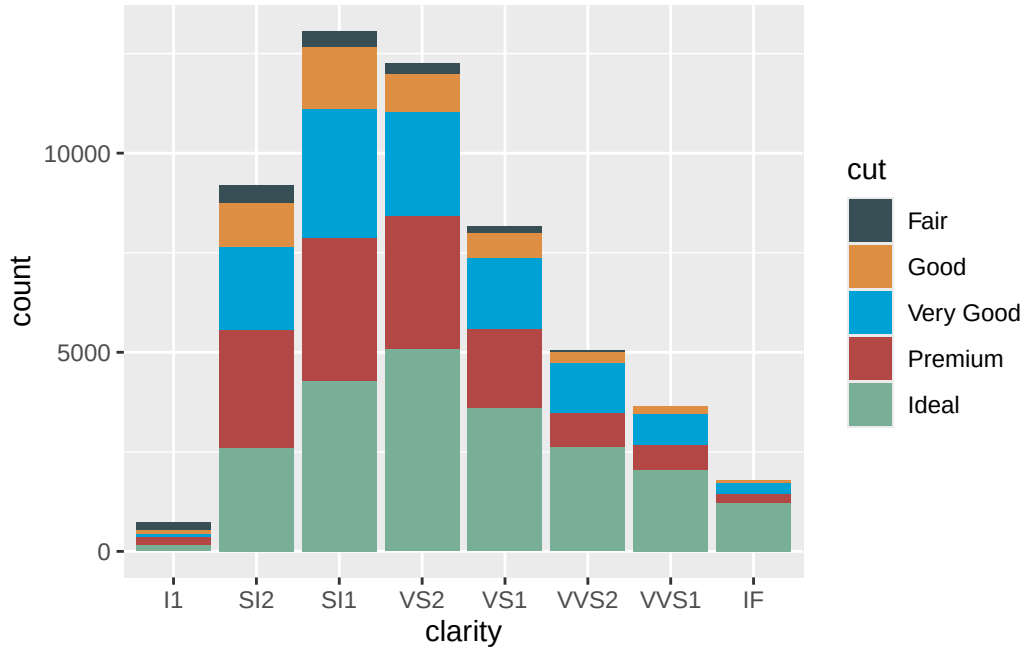


7.4.1 External color definitions from ggsci

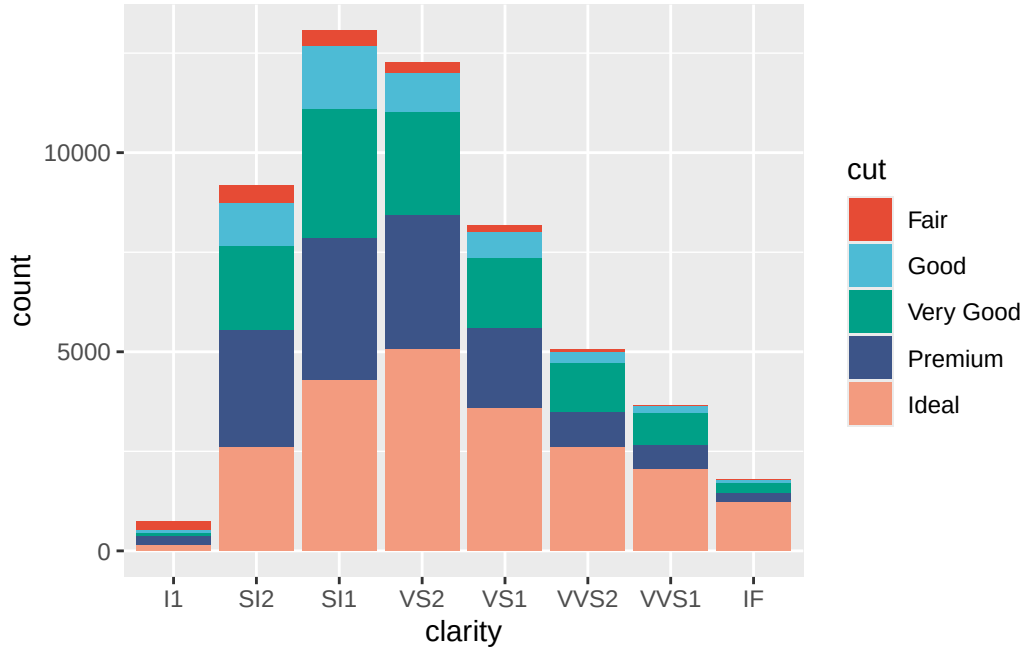
```
plottemp + scale_fill_lancet()
```



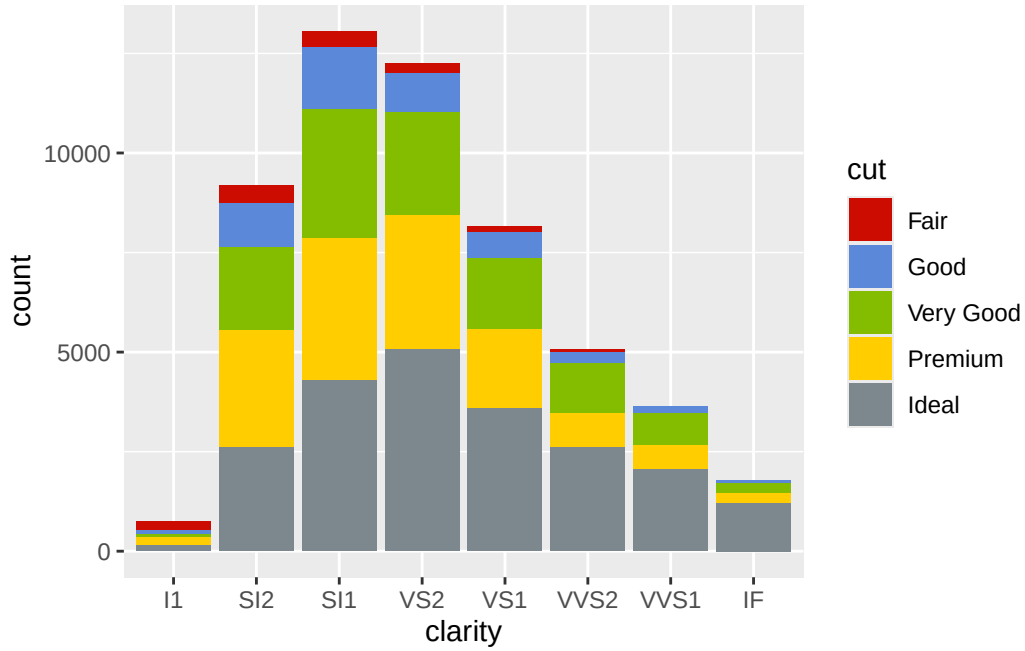

```
plottemp + scale_fill_jama()
```




```
plottemp + scale_fill_npg()
```




```
(printplot <- plottemp + scale_fill_startrek())
```



7.5 Exporting ggplots

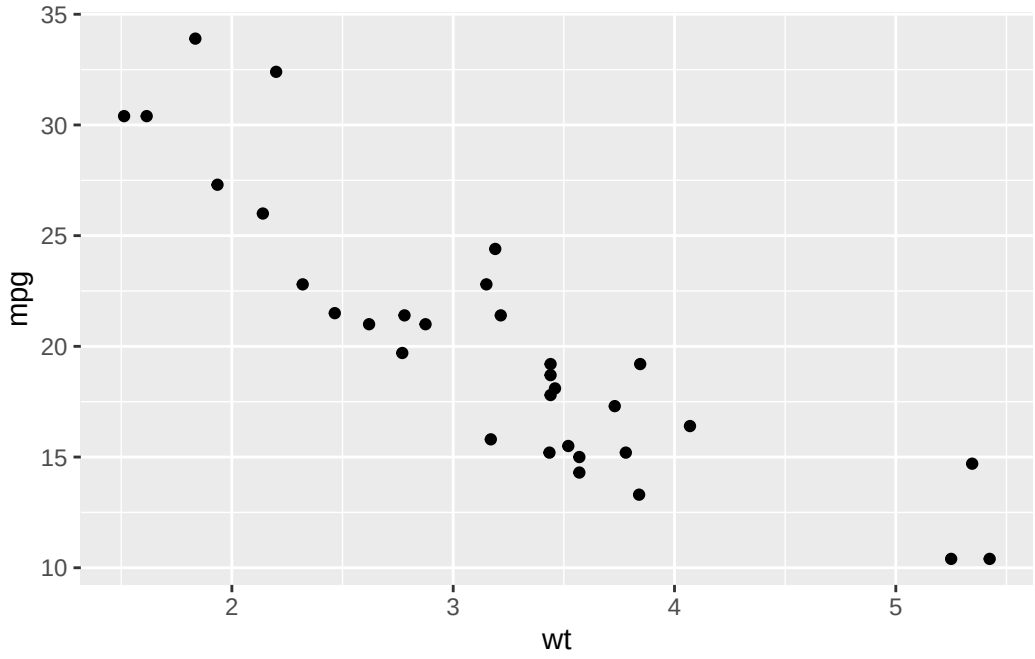
There are two distinct ways, using ggsave() or more generally creating an external graphic device with e.g. png() / tiff() / pdf():

```
ggsave(
  filename = here("Graphs/ggtestplot.png"),
  plot = printplot,
  width = 20, height = 20,
  units = "cm", dpi = 150
)
ggsave(
  filename = here("Graphs/ggtestplot.tiff"),
  plot = printplot,
  width = 20, height = 20,
  units = "cm", dpi = 600
)
ggsave(
  filename = here("Graphs/ggtestplot_c.tiff"),
  plot = printplot,
  width = 20, height = 20, compression = "lzw",
  units = "cm", dpi = 600
)
# alternative:
png(
  filename = here("Graphs/ggtestplot2.png"),
  width = 20, height = 20, units = "cm", res = 150
)
plottemp
dev.off()
```

7.6 Other geoms

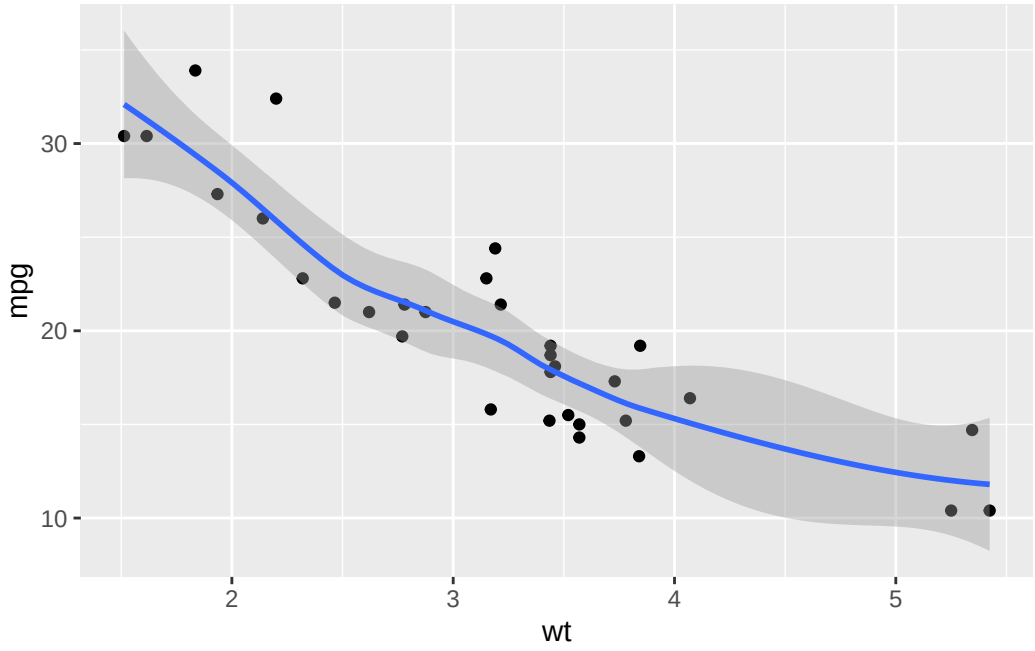
Common forms of plots are barplots, boxplots, scatterplots (possibly with regression line), and density-plots. For plotting dots for groups, there are various options to avoid over-plotting of repeated data, with the beeswarm as my preference.

```
ggplot(data = mtcars, aes(x = wt, y = mpg)) +
  geom_point()
```



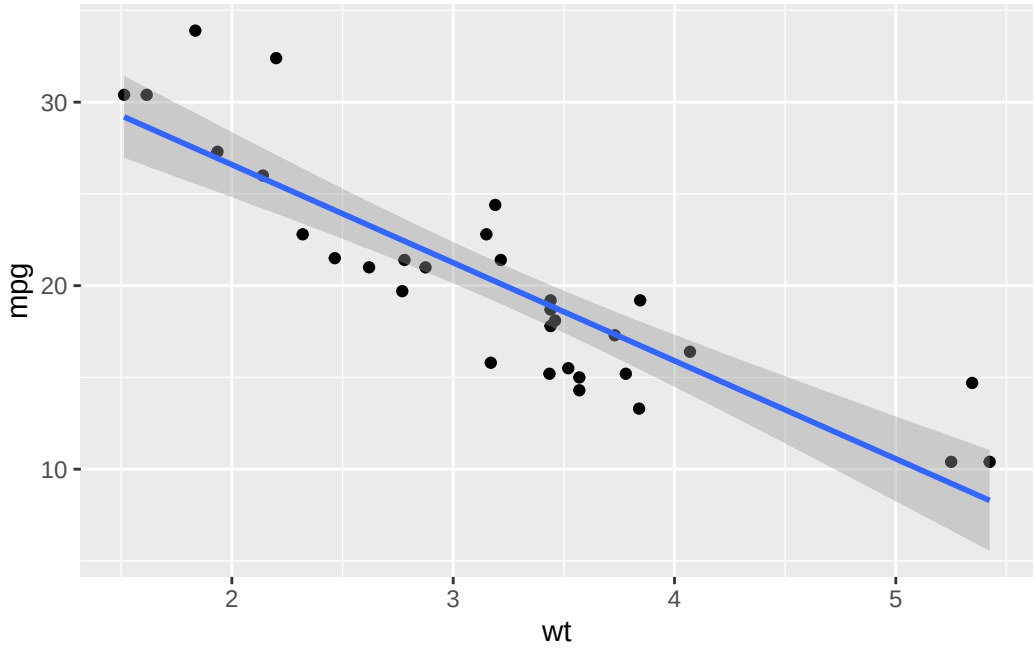

```
ggplot(data = mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

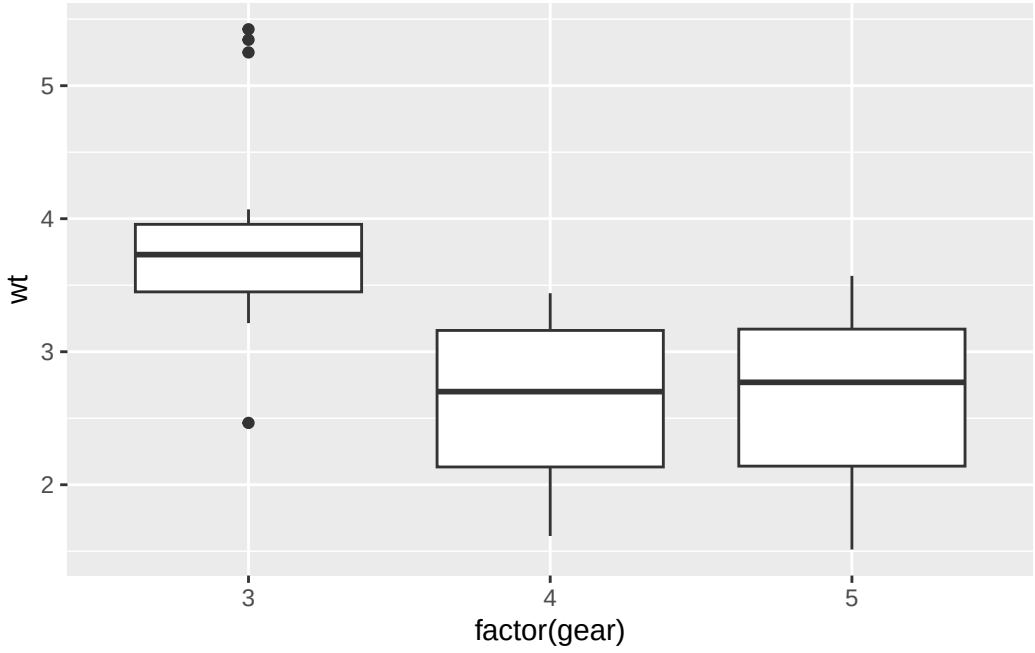



```
ggplot(data = mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_smooth(method = "lm")
```

```
`geom_smooth()` using formula = 'y ~ x'
```

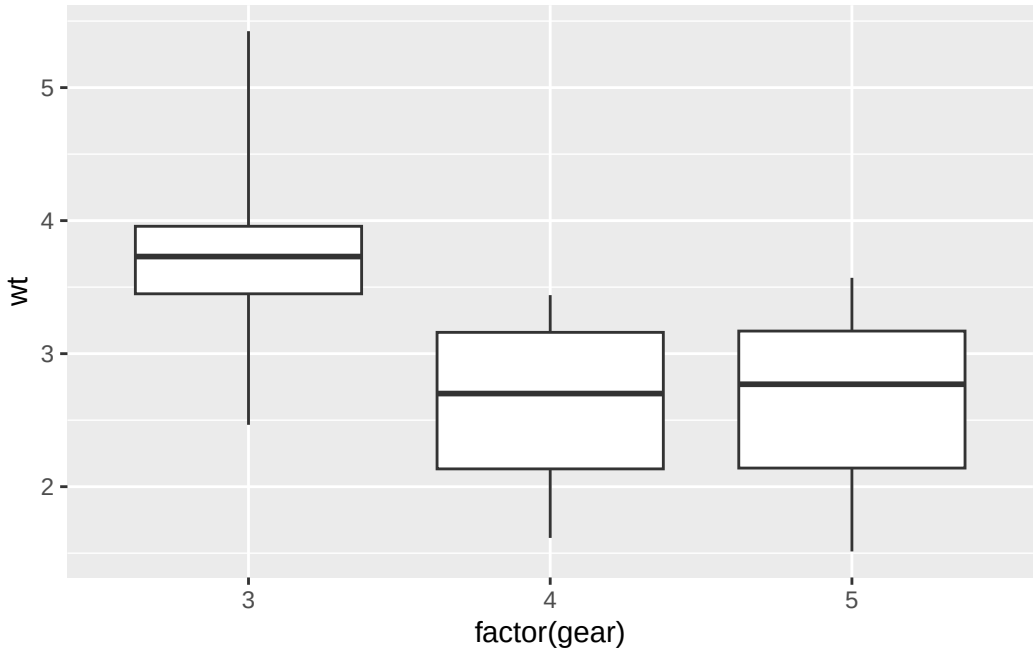



```
ggplot(mtcars, aes(x = factor(gear), y = wt)) +
  geom_boxplot() # default 1.5 IQR
```

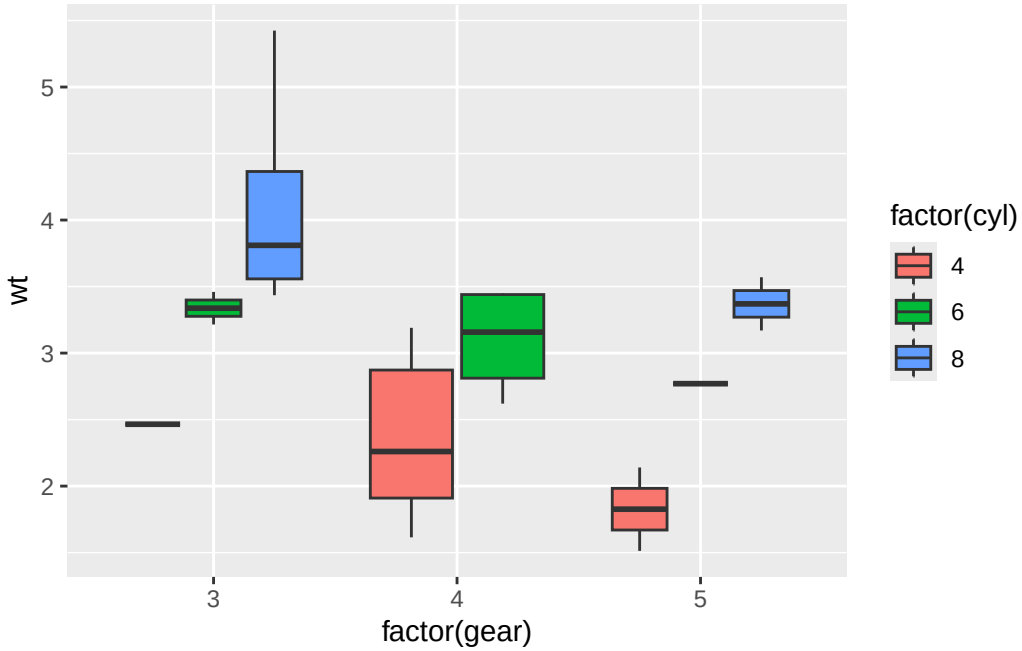



```
ggplot(mtcars, aes(x = factor(gear), y = wt)) +
```

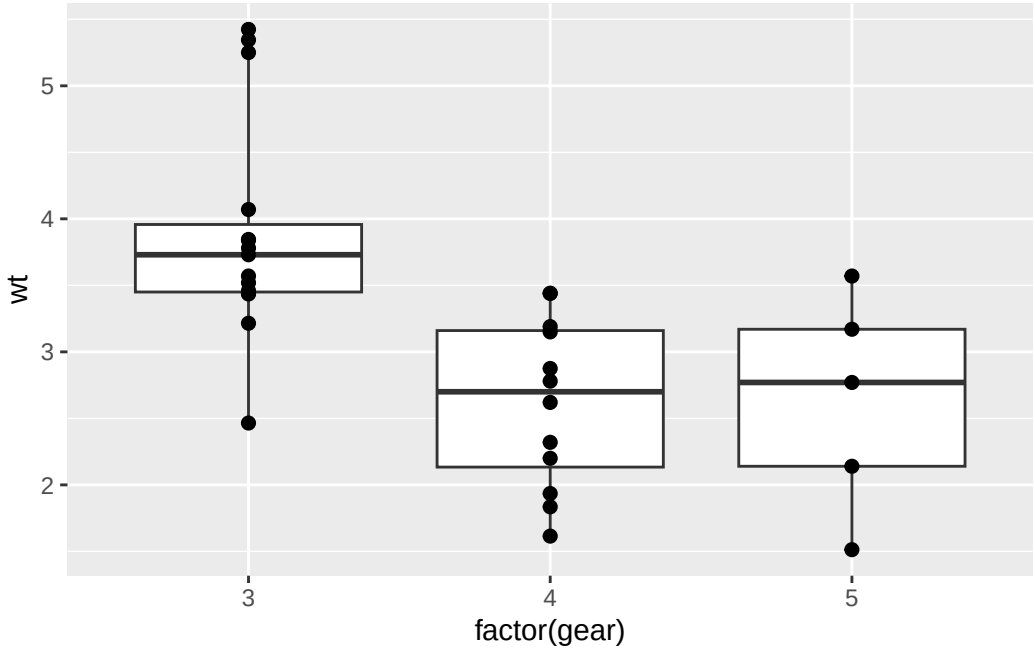
```
  geom_boxplot(coef = 3) # this extends range of expected values / outliers
```



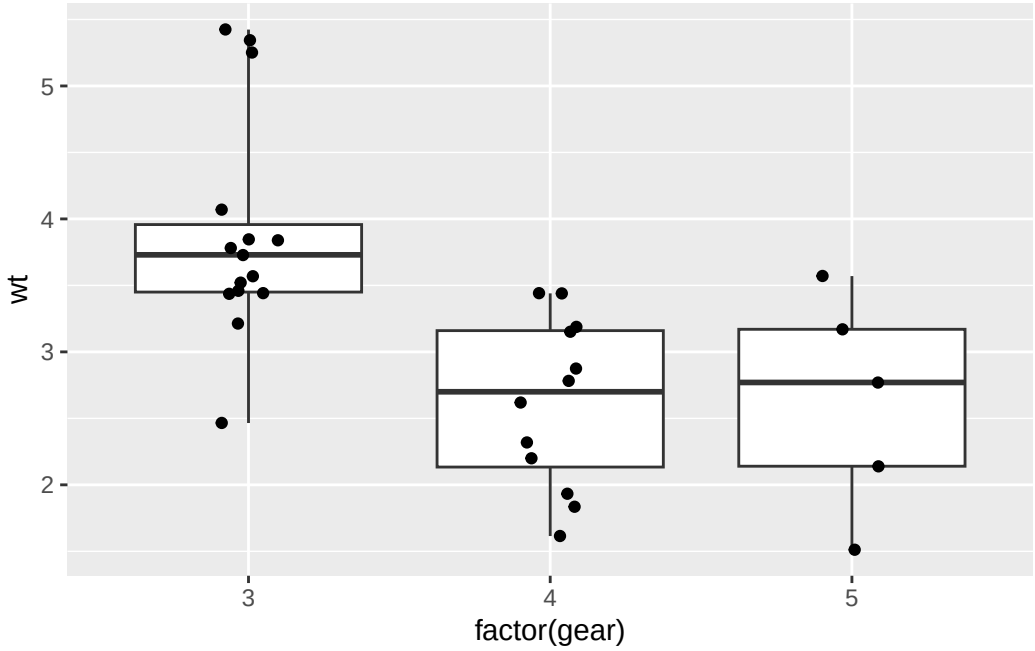

```
ggplot(mtcars, aes(
  x = factor(gear), y = wt,
  fill = factor(cyl)
)) +
  geom_boxplot(coef = 3) # group by cyl, as it is mapped to fill
```




```
ggplot(mtcars, aes(x = factor(gear), y = wt)) +
  geom_boxplot(coef = 3) +
  geom_point(size = 2) # may contain overlapping points
```

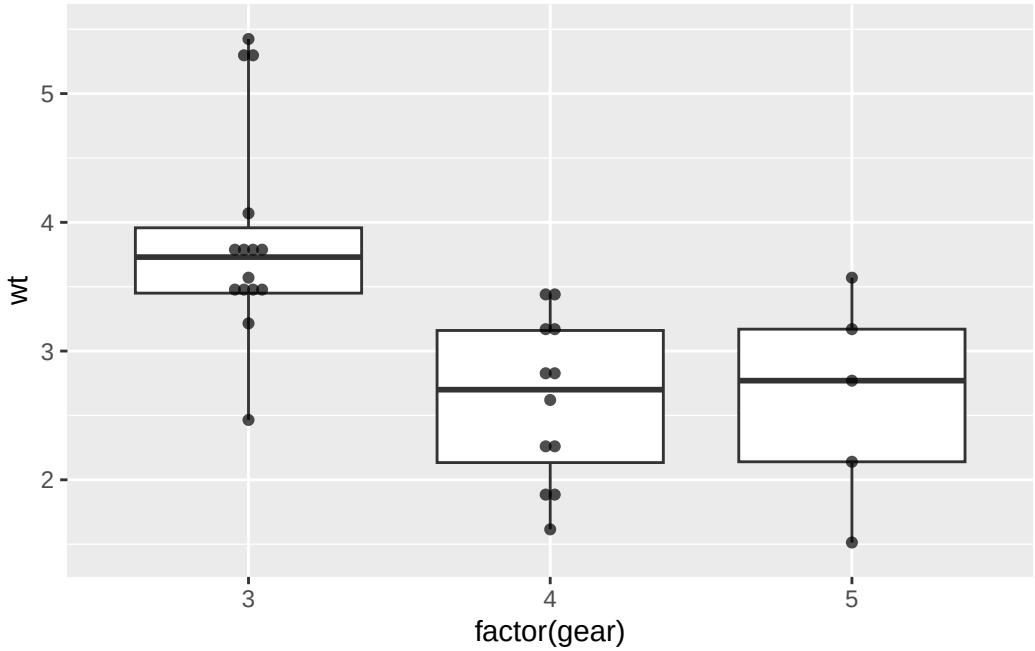


```
ggplot(mtcars, aes(x = factor(gear), y = wt)) +
  geom_boxplot(coef = 3) +
  geom_point(position = position_jitter(width = .1))
```

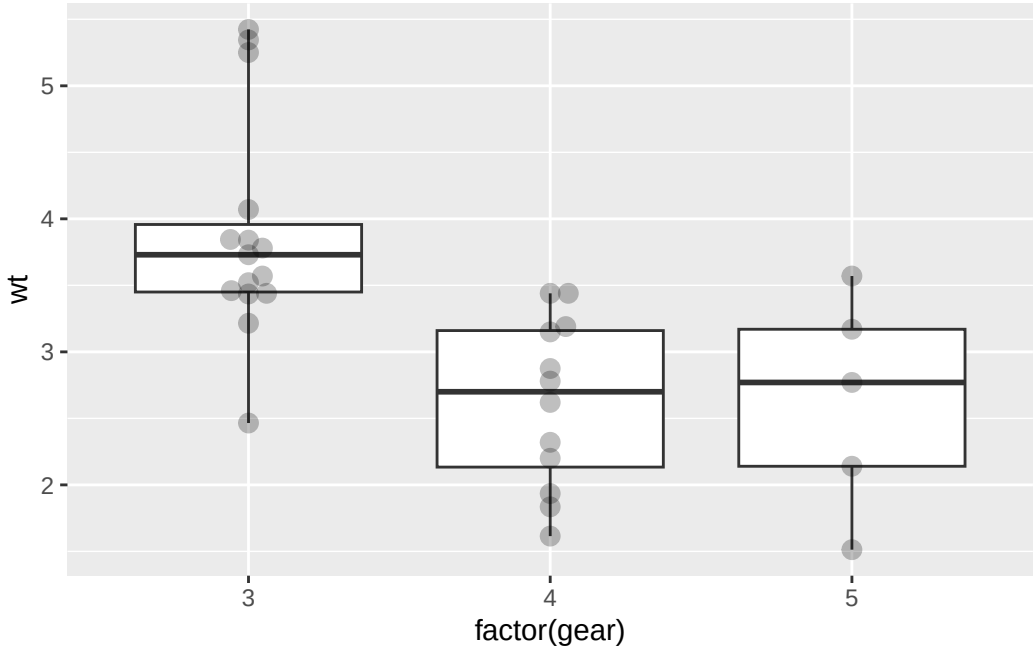



```
ggplot(mtcars, aes(x = factor(gear), y = wt)) +
  geom_boxplot(coef = 3) +
  geom_dotplot(
    alpha = .7, # group similar(ish) data on a line
    binaxis = "y", stackdir = "center",
    stackratio = .9, dotsize = .6
  )
```

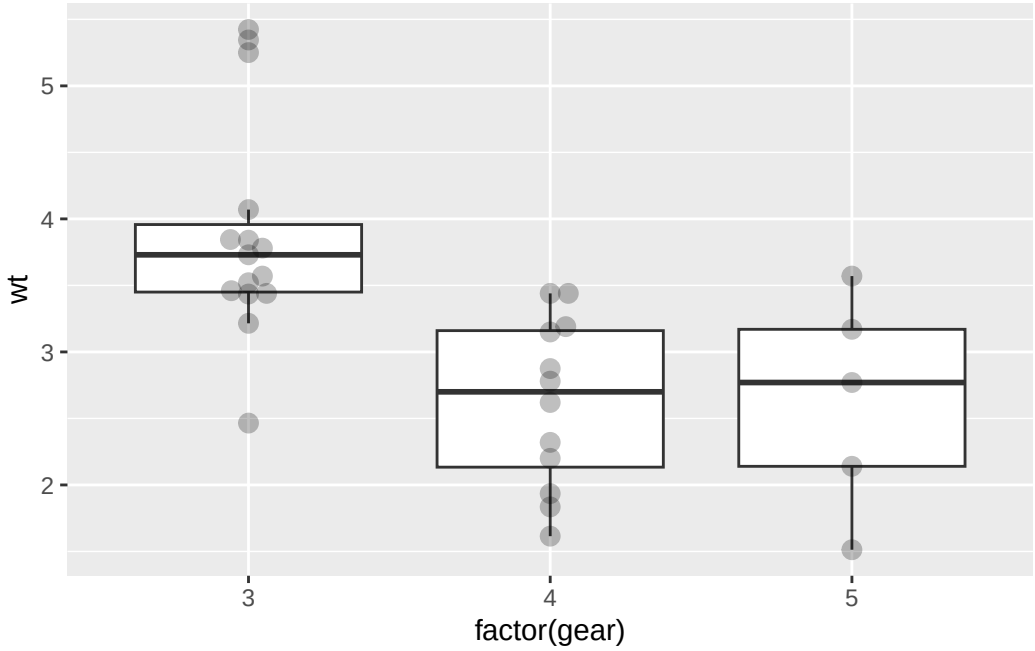
```
Bin width defaults to 1/30 of the range of the data. Pick better value with
`binwidth`.
```



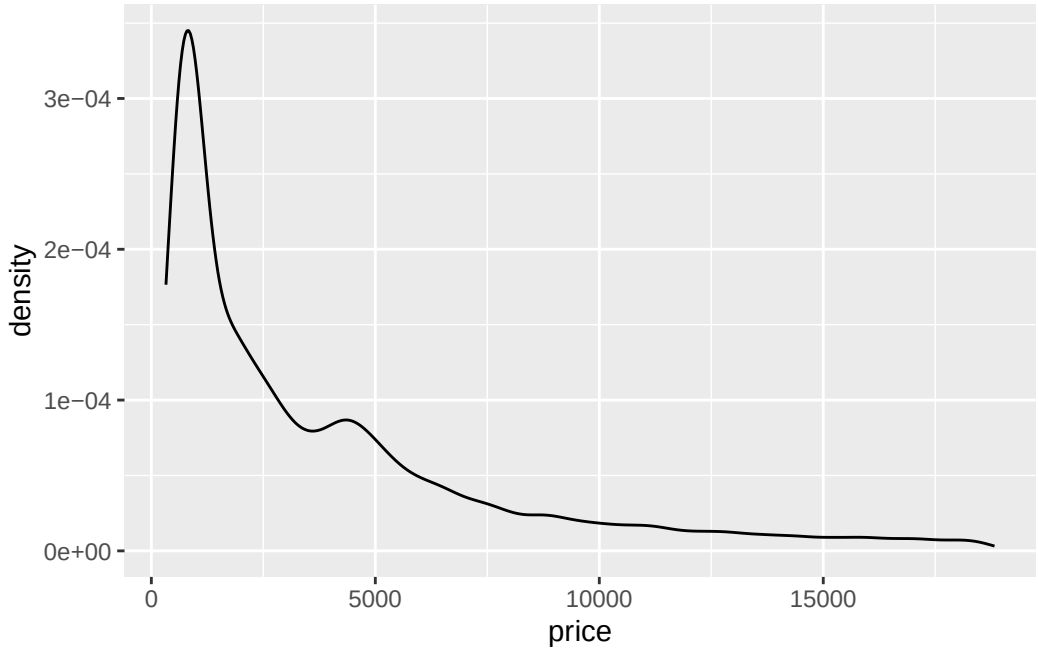

```
ggplot(mtcars, aes(x = factor(gear), y = wt)) +
  geom_boxplot(coef = 3) +
  ggbeeswarm::geom_beeswarm(cex = 2, size = 3, alpha = .25)
```



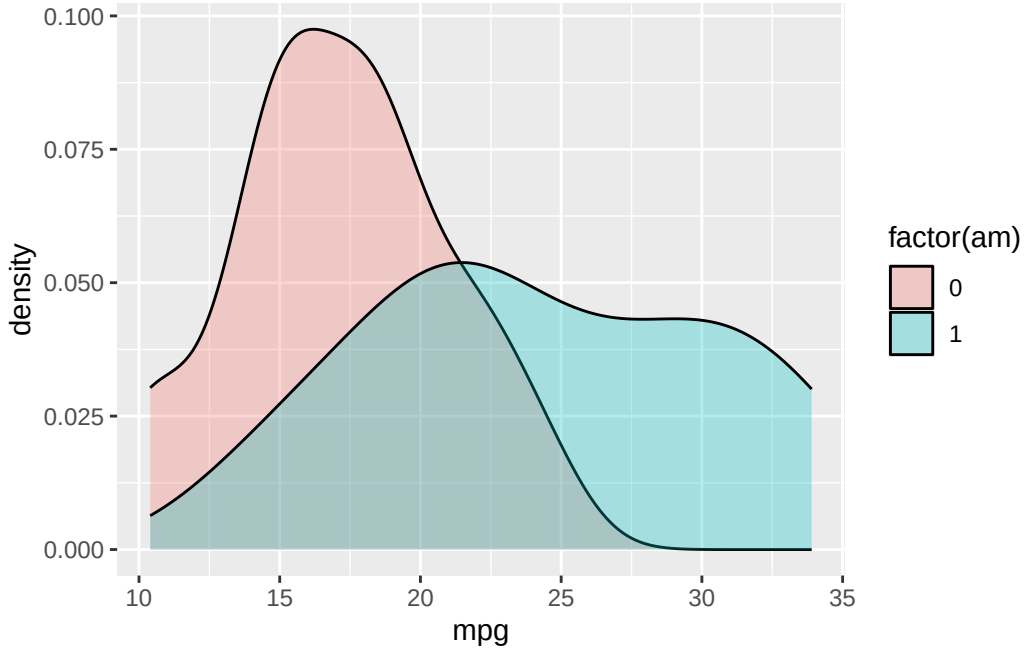
```r
ggplot(mtcars, aes(x = factor(gear), y = wt)) +
  geom_boxplot(outlier.alpha = 0) + # to avoid plotting outliers twice
  geom_beeswarm(cex = 2, size = 3, alpha = .25)
```




```r
# density plot
ggplot(diamonds, aes(price)) +
  geom_density()
```



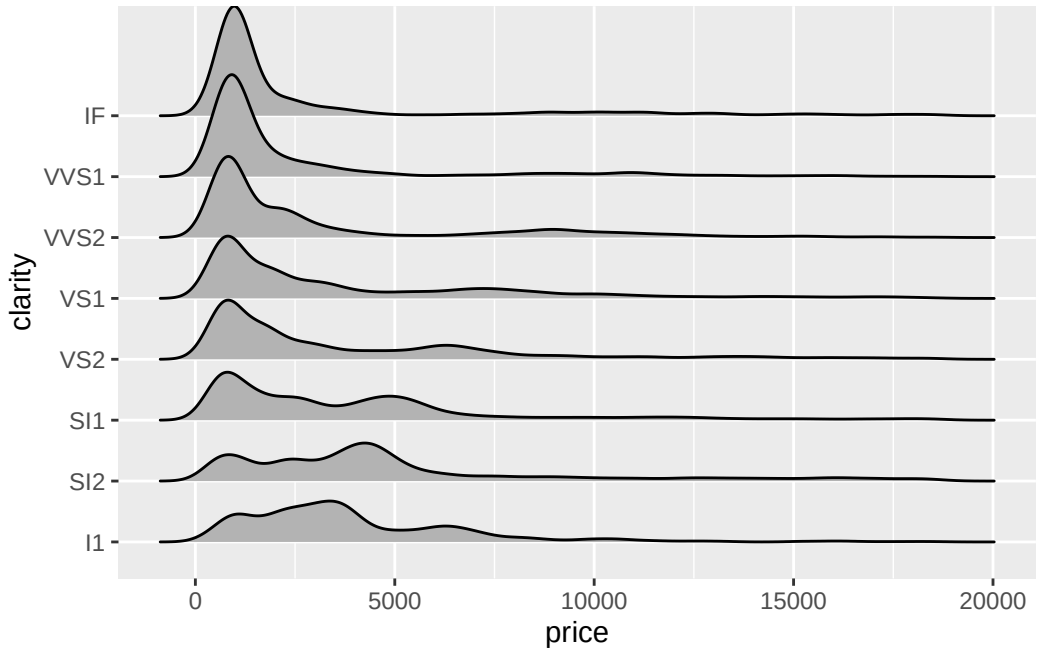

```
ggplot(mtcars, aes(mpg, fill = factor(am))) +
  geom_density(alpha = .3)
```



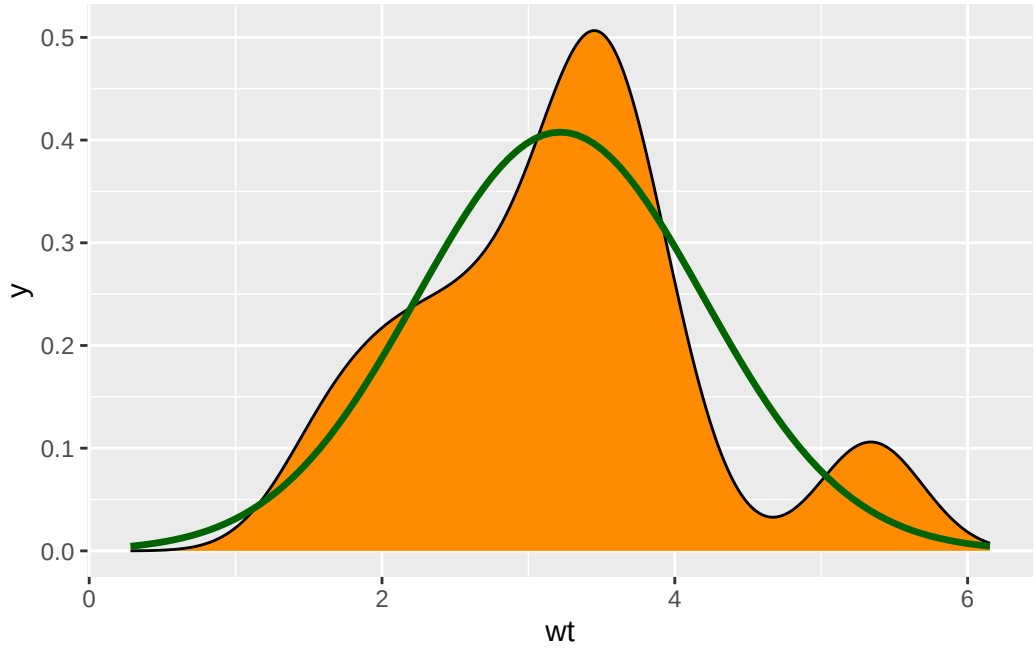

```
ggplot(diamonds, aes(price, y = clarity)) +
```

```
  geom_density_ridges()
```

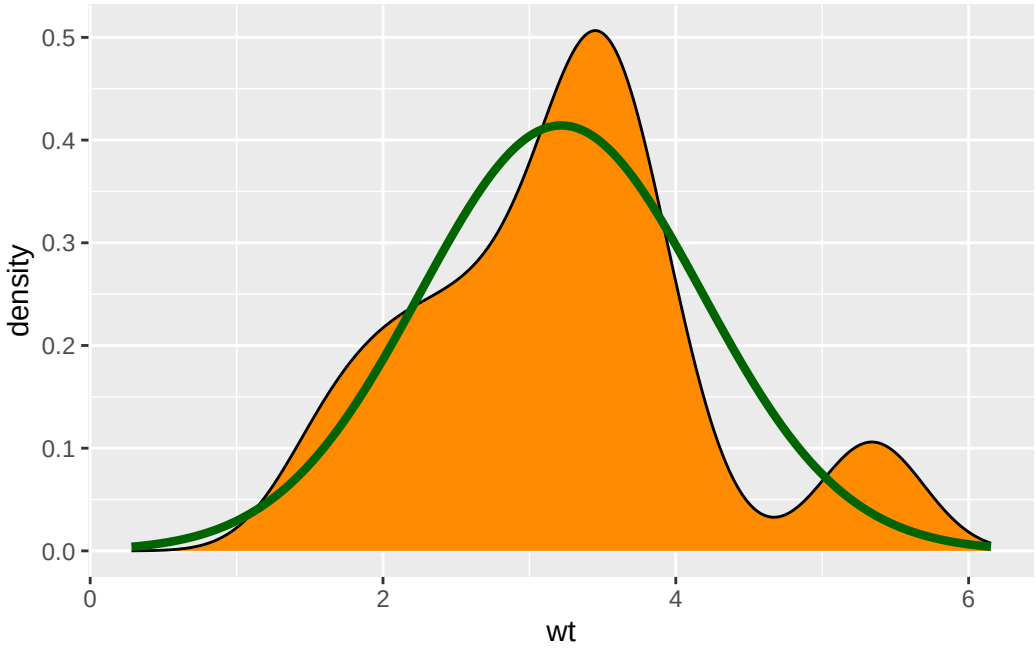
```
Picking joint bandwidth of 403
```




```
# empirical vs. theoretical distribution
ggplot(mtcars, aes(wt)) +
  geom_density(fill = "darkorange") +
  stat_function(
    fun = dnorm,
    args = list(
      mean = mean(mtcars$wt),
      sd = sd(mtcars$wt)
    ),
    color = "darkgreen",
    linewidth = 1.2
  ) +
  scale_x_continuous(
    limits = c(
      mean(mtcars$wt) - 3 * sd(mtcars$wt),
      mean(mtcars$wt) + 3 * sd(mtcars$wt)
    )
  )
```




```
# simplified by package ggh4x
ggplot(mtcars, aes(wt)) +
  geom_density(fill = "darkorange") +
  stat_theodensity(distri = "norm", linewidth = 1.5, color = "darkgreen") +
  scale_x_continuous(
    limits = c(
      mean(mtcars$wt) - 3 * sd(mtcars$wt),
      mean(mtcars$wt) + 3 * sd(mtcars$wt)
    )
  )
```


0.5
0.4
0.3
0.2
0.1
0.0
density
0
2
4
6
wt

7.7 Exercise 1

In this exercise, you will practice building layers and mapping aesthetics in ggplot2 using the penguins dataset.

Note: In the real world, the questions stakeholders ask rarely map 1:1 to column names. Inspect the dataset first to find the right variables!

Task A: Counts & Variations Bar Chart: Visualize the total count of penguins broken down by sex within each species.

Variability (2 Variables): Create a hybrid plot combining a boxplot and a beeswarm plot (ggbeeswarm::geom_beeswarm()) to show how penguin weight is distributed across different species.

Variability (3 Variables): Expand the previous plot to show the weight distribution across both species and sex.

Challenge: Pay attention to how the points sit on top of the boxplots. How can you map the aesthetics so that both variables remain visually distinct and readable?

Task B: Relationships & Regression Basic Scatterplot: Create a scatterplot showing the relationship between a penguin's flipper length (x-axis) and its body mass (y-axis).

Add a Trendline: Add a linear regression line to the scatterplot to visualize the overall trend.

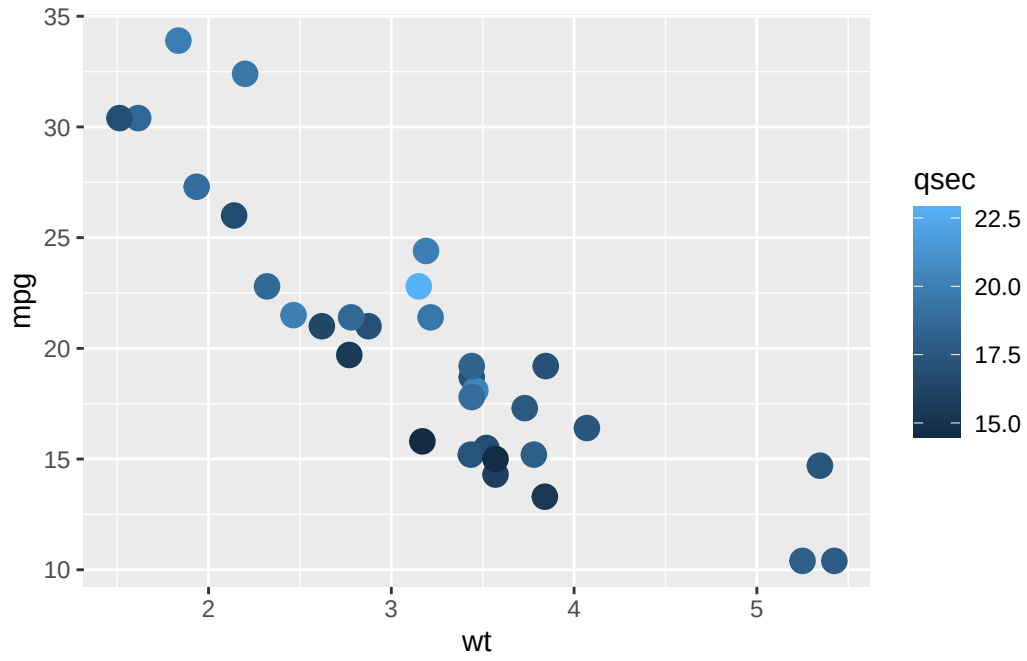
Subgroup Analysis: Modify the scatterplot and regression lines so the data is grouped and visualized by both species and sex without using facets.

Challenge: You have two grouping variables but only one scatterplot. Combine different aesthetics creatively so the viewer can instantly distinguish between an Adélie male, an Adélie female, a Chinstrap male, etc., while keeping the trendlines readable.

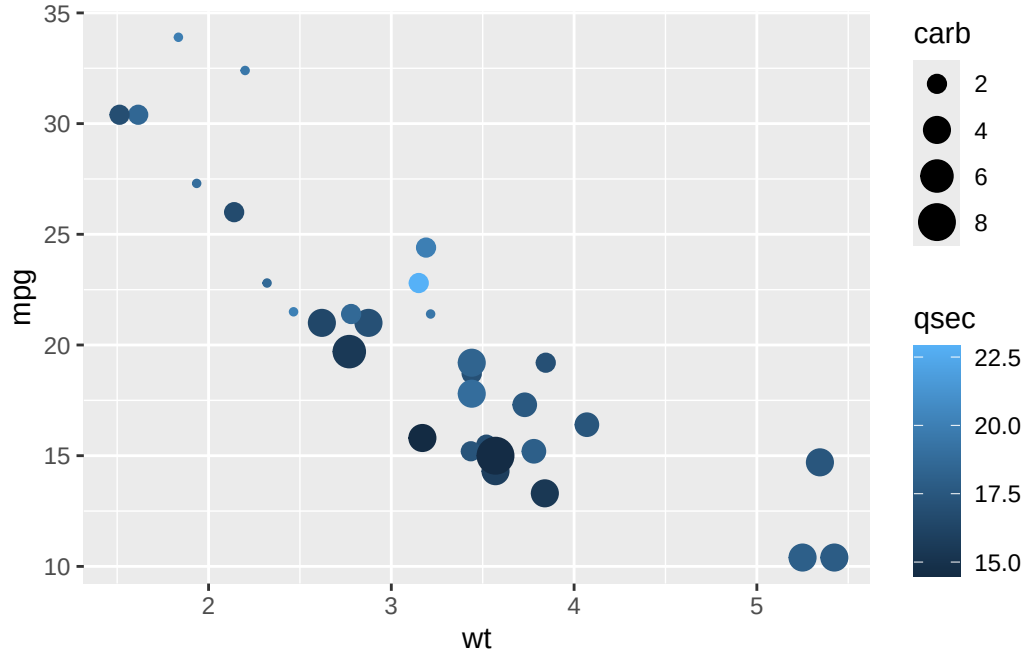
Task C: Styling (Optional Challenge) Custom Palette: Manually define your own color palette for the species using custom color names or hex codes.

7.8 Combining and finetuning aesthetics

```
ggplot(data = mtcars, aes(wt, mpg, color = qsec)) +
  geom_point(size = 4) # outside aes!
```

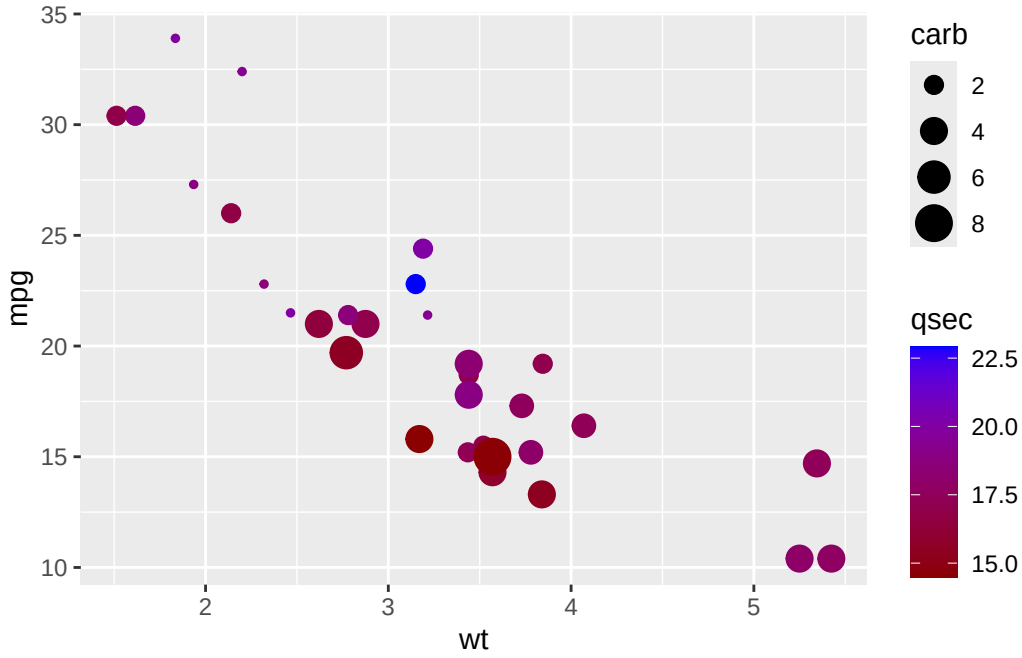



```
ggplot(data = mtcars, aes(wt, mpg, color = qsec, size = carb)) +
  geom_point()
```

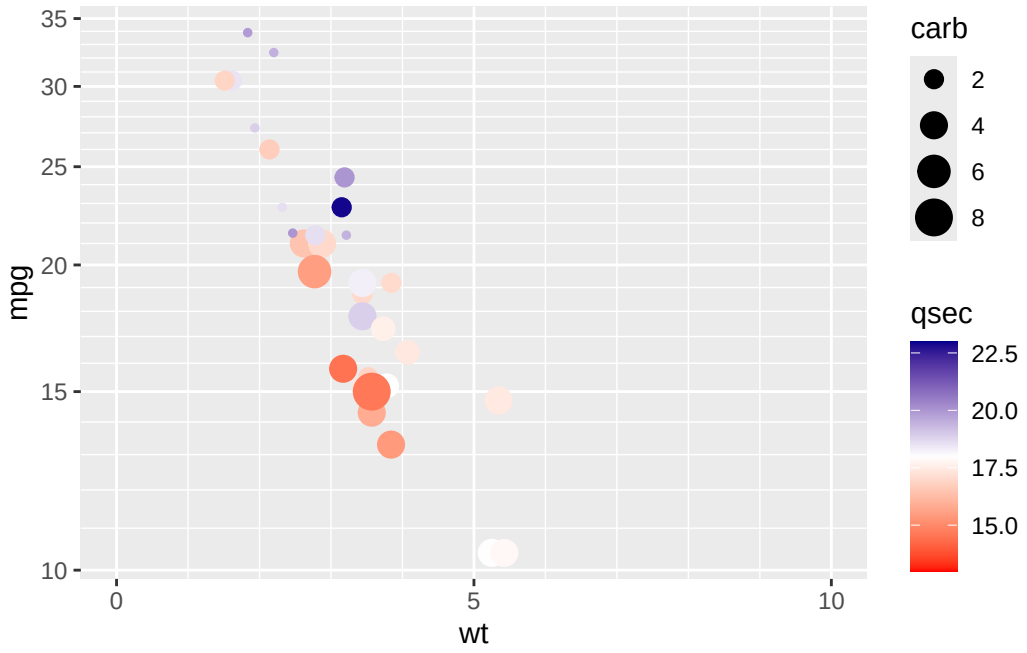



```
ggplot(data = mtcars, aes(wt, mpg, color = qsec, size = carb)) +
```

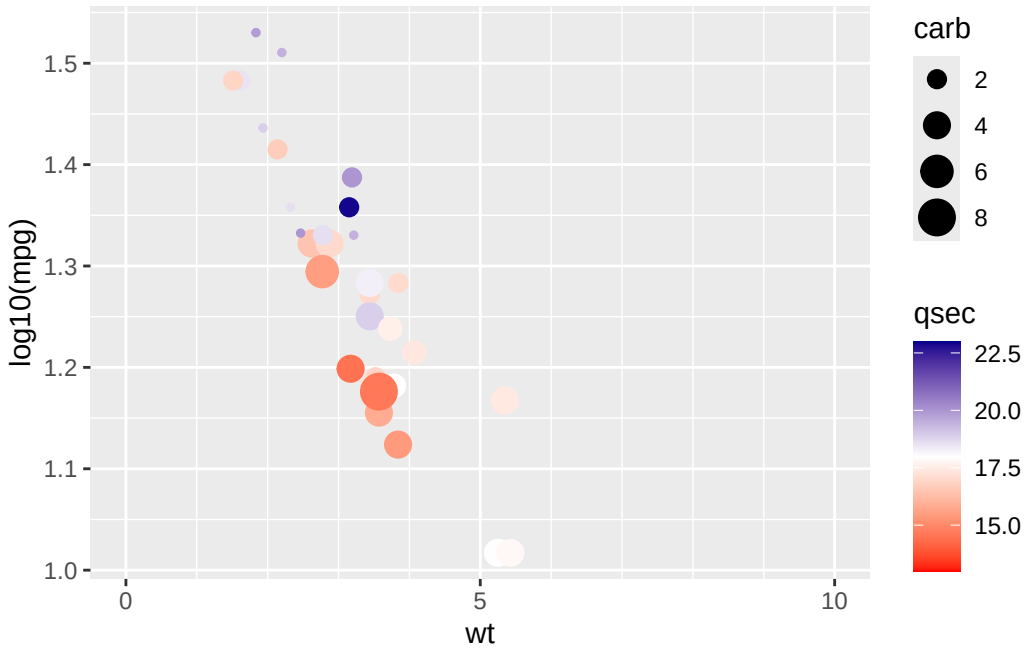
```
  scale_color_gradient(low = "darkred", high = "blue") +
  geom_point()
```




```
ggplot(data = mtcars, aes(wt, mpg, color = qsec, size = carb)) +
  scale_color_gradient2(
    low = "red", high = "darkblue",
    mid = "white",
    limits = c(13, 23), midpoint = 18
  ) +
  geom_point() +
  scale_x_continuous(
    breaks = seq(-10^3, 10^3, 5),
    minor_breaks = seq(-10^3, 10^3, 1),
    limits = c(0, 10)
  ) +
  scale_y_log10(
    breaks = seq(0, 100, 5),
    minor_breaks = seq(0, 100, 1)
  )
```

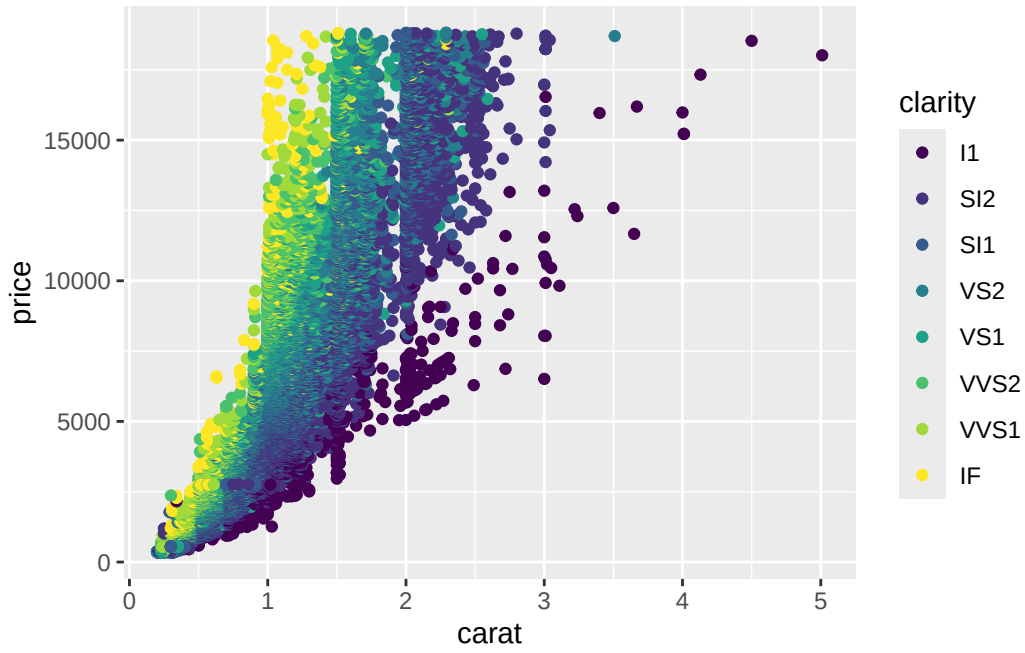



```
ggplot(data = mtcars, aes(wt, log10(mpg), color = qsec, size = carb)) +
  scale_color_gradient2(
    low = "red", high = "darkblue",
    mid = "white",
    limits = c(13, 23), midpoint = 18
  ) +
  geom_point() +
  scale_x_continuous(
    breaks = seq(0, 100, 5),
    minor_breaks = seq(0, 100, 1),
    limits = c(0, 10)
  ) #+
```

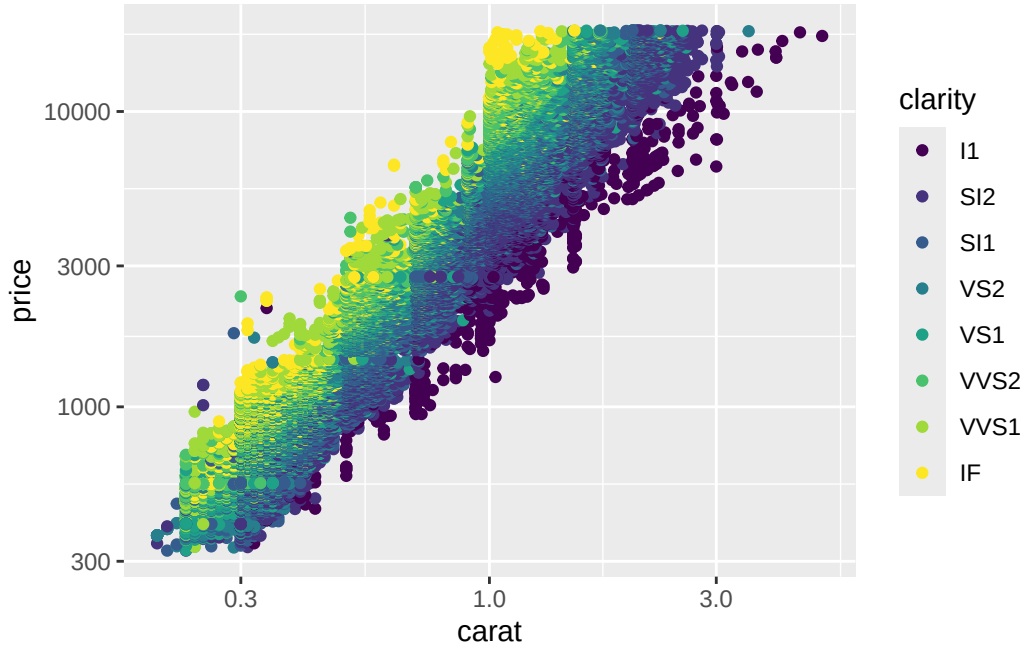



```
# scale_y_log10(minor_breaks = seq(0,100,1))
```

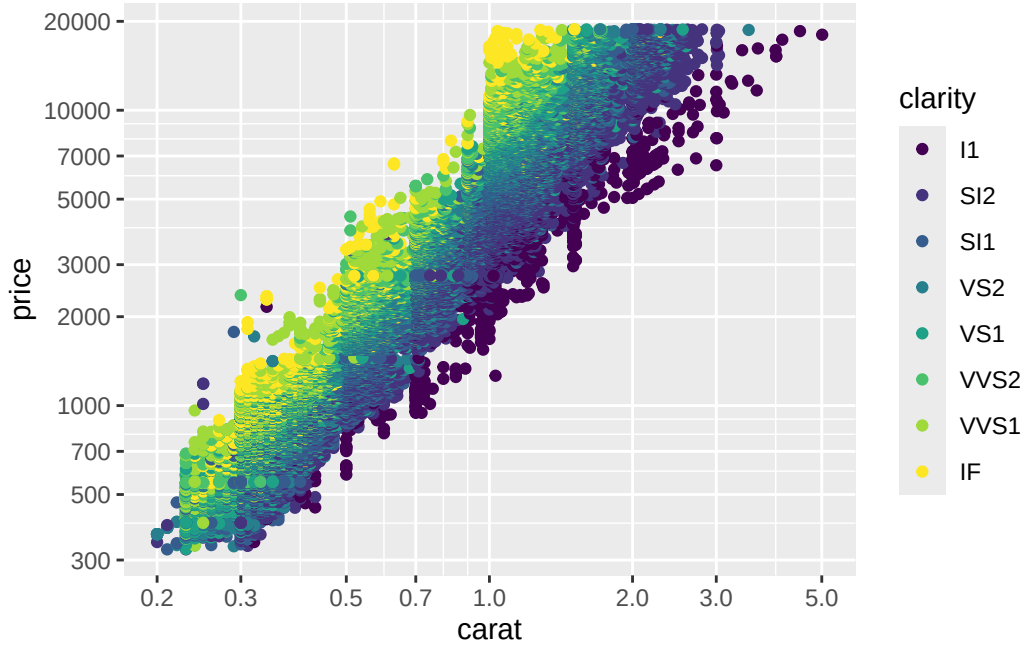
```
ggplot(diamonds, aes(carat, price, color = clarity)) +
  geom_point()
```



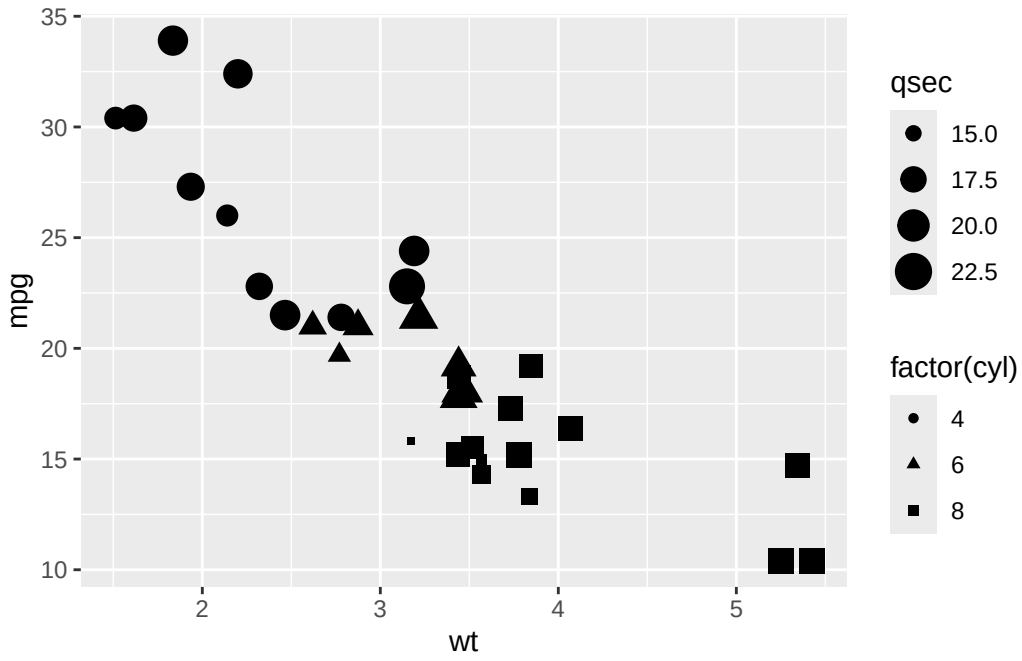

```
ggplot(diamonds, aes(carat, price, color = clarity)) +
  geom_point() +
  scale_x_log10() +
  scale_y_log10()
```



```
ggplot(diamonds, aes(carat, price, color = clarity)) +
  geom_point() +
  scale_x_log10(
    breaks = lograngе_12357,
    minor_breaks = lograngе_123456789
  ) +
  scale_y_log10(
    breaks = lograngе_12357,
    minor_breaks = lograngе_123456789
  )
```



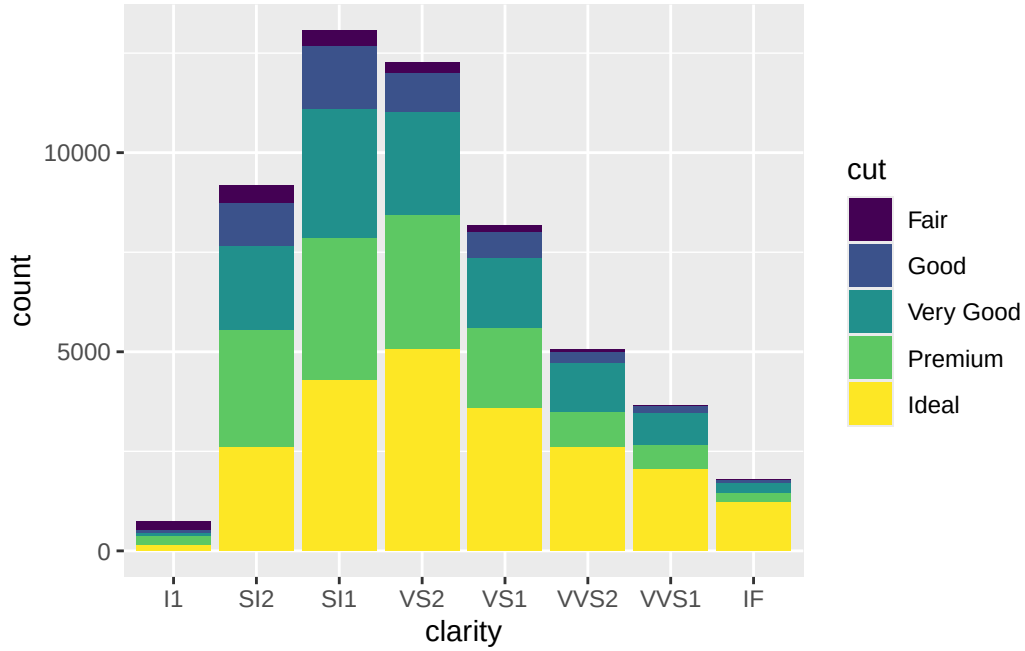

```
# use different aesthetic mappings
ggplot(
  data = mtcars,
  aes(wt, mpg, size = qsec, shape = factor(cyl))
) +
  geom_point()
```



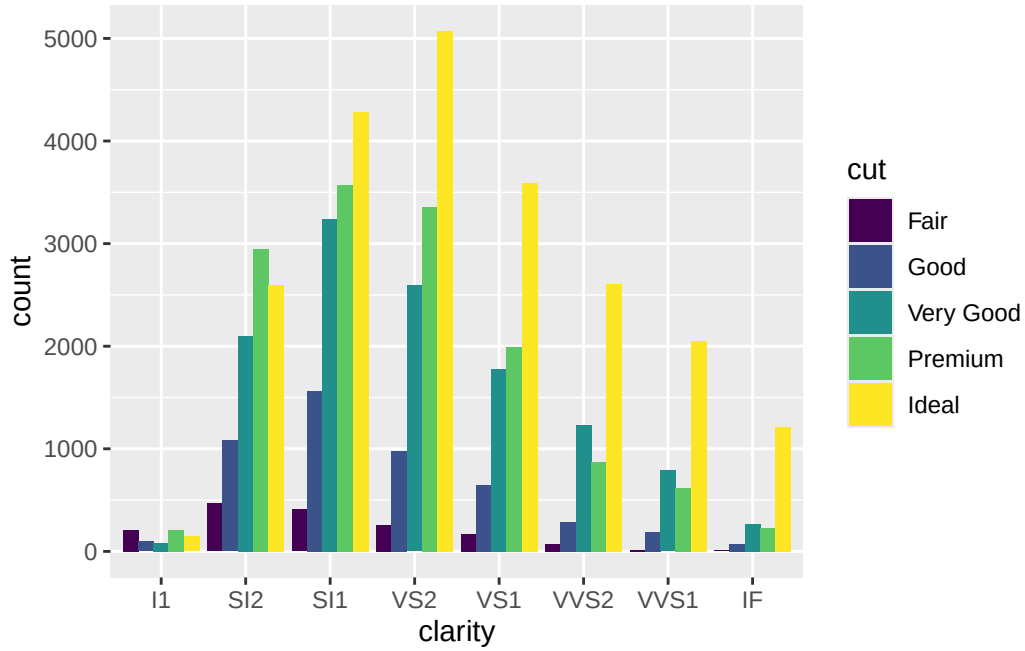
7.9 Positioning elements

The position arguments allows stacking, dodging, jittering and exact positioning of elements. Positioning is an essential part of storytelling, the same data can be presented with different focus.

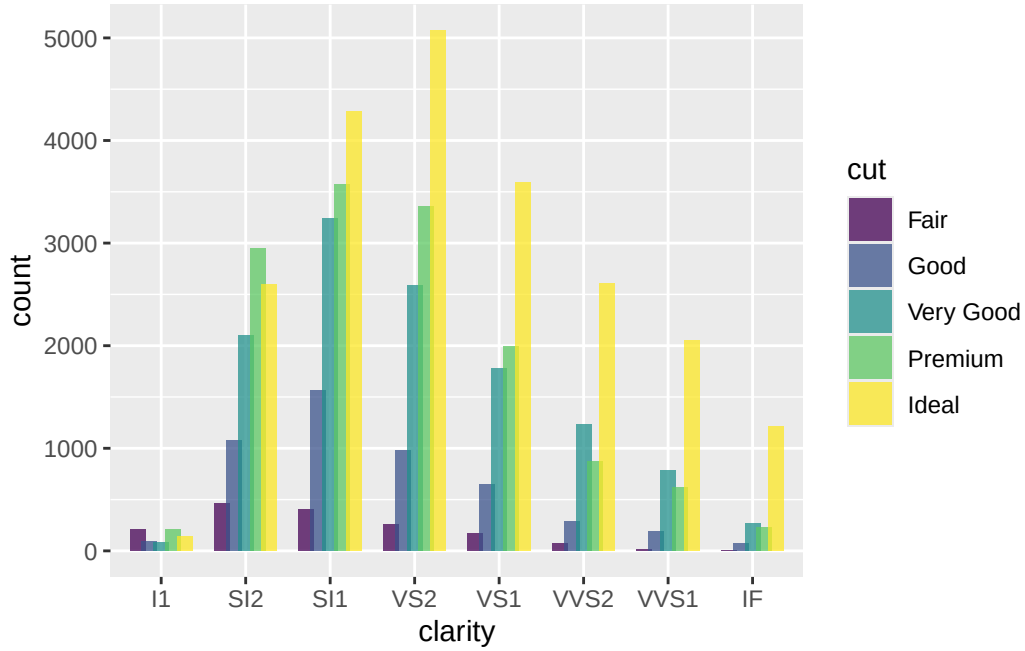
```
p <- ggplot(data = diamonds, aes(clarity, fill = cut))
p + geom_bar(position = "stack") # default for bar
```



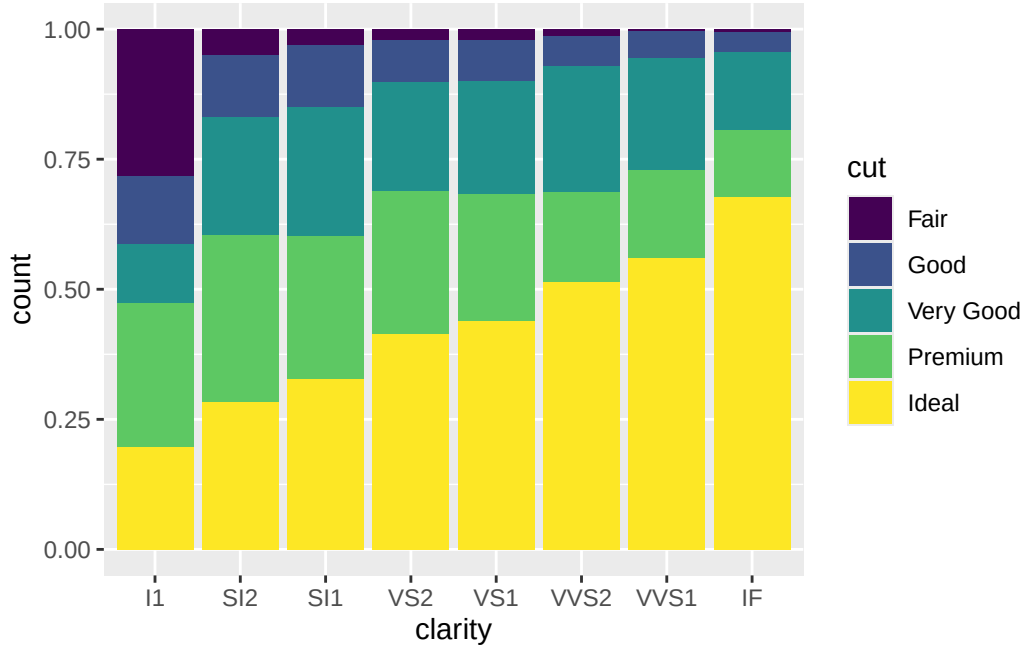

```
p + geom_bar(position = "dodge")
```




```
p + geom_bar(position = position_dodge(width = 0.7), alpha = .75)
```

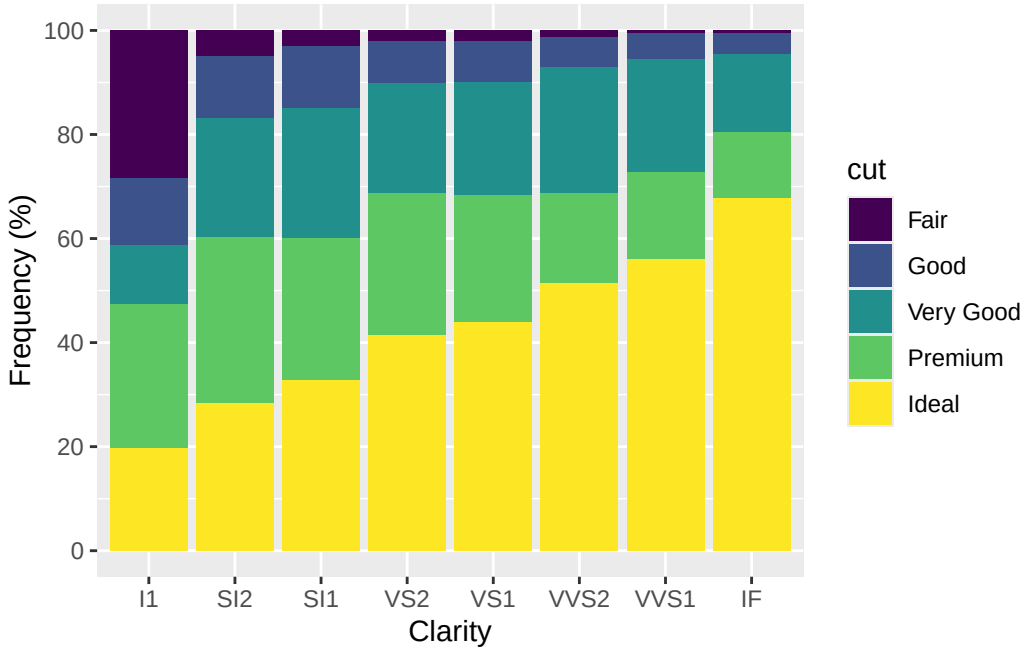



```
p + geom_bar(position = "fill") # y-axis labeling needs tuning
```

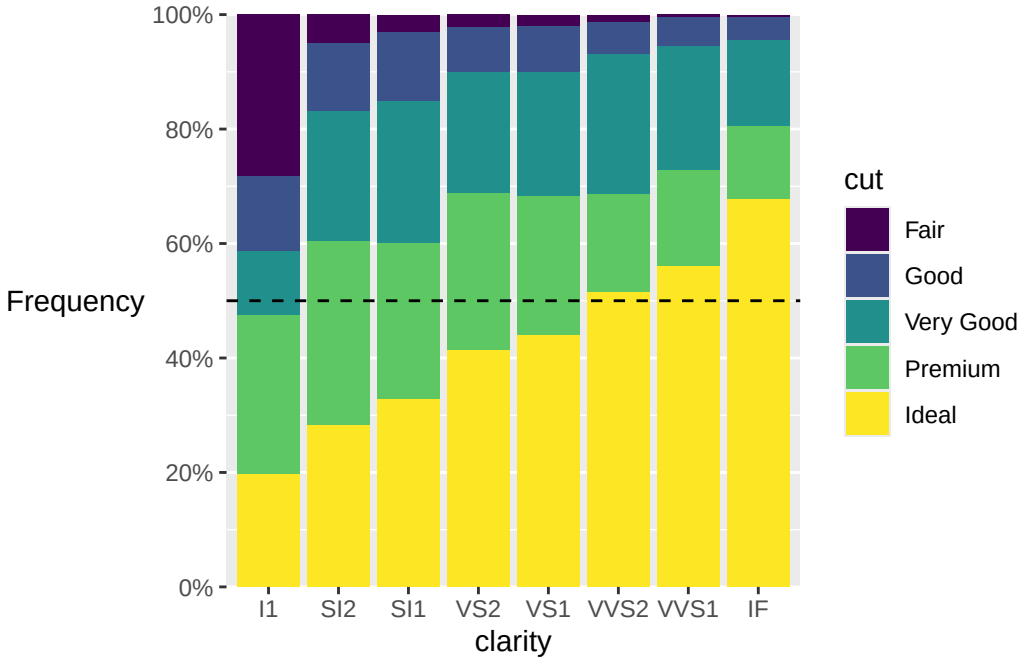



```
p + geom_bar(position = "fill") +
  xlab("Clarity") +
```

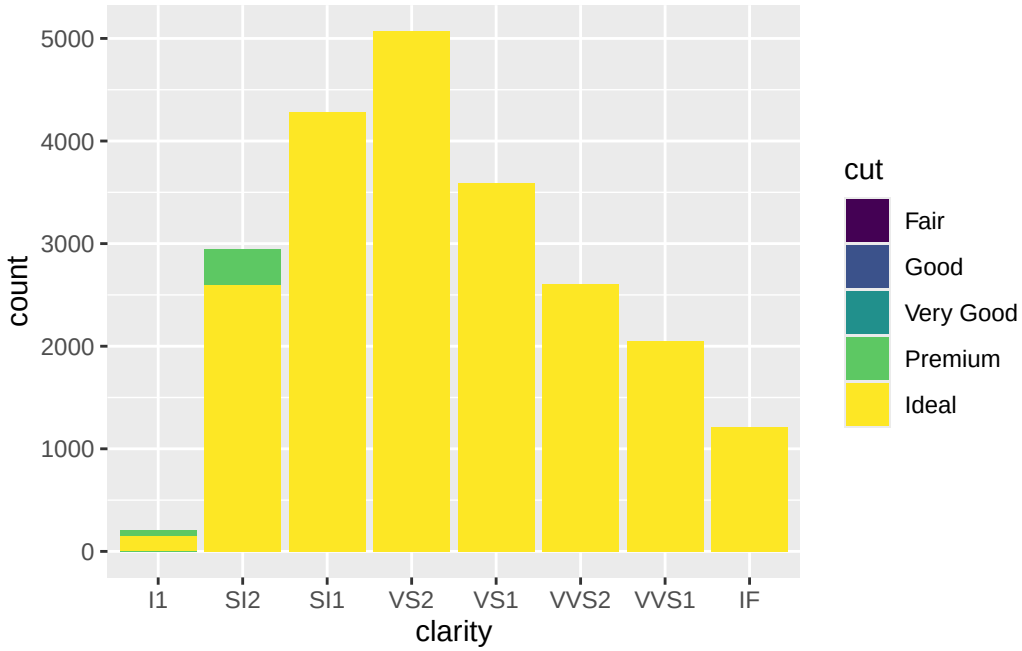
```
  scale_y_continuous(
    name = "Frequency (%)",
    breaks = seq(from = 0, to = 1, by = .2), # steps
    labels = seq(0, 100, 20)
  )
```



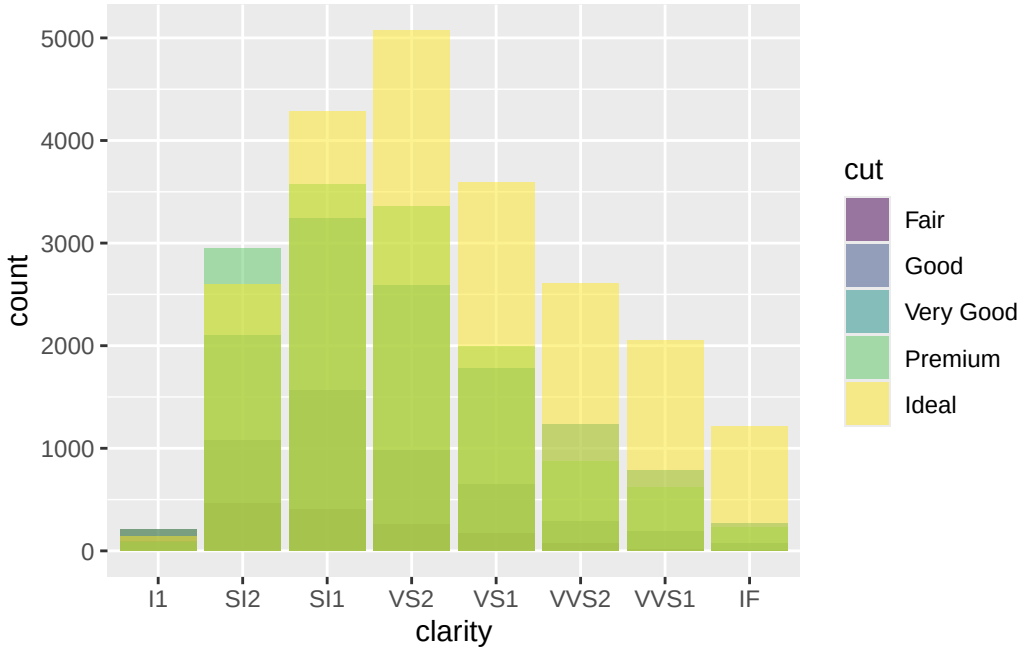

```
p + geom_bar(position = "fill") +
  scale_y_continuous("Frequency",
    breaks = seq(0, 1, .2),
    labels = scales::percent,
    expand = expansion(mult = c(0, 0))
  ) +
  theme(axis.title.y = element_text(
    angle = 0,
    vjust = .5
  )) +
  geom_hline(yintercept = .5, linetype = 2) # e.g. for reference lines
```



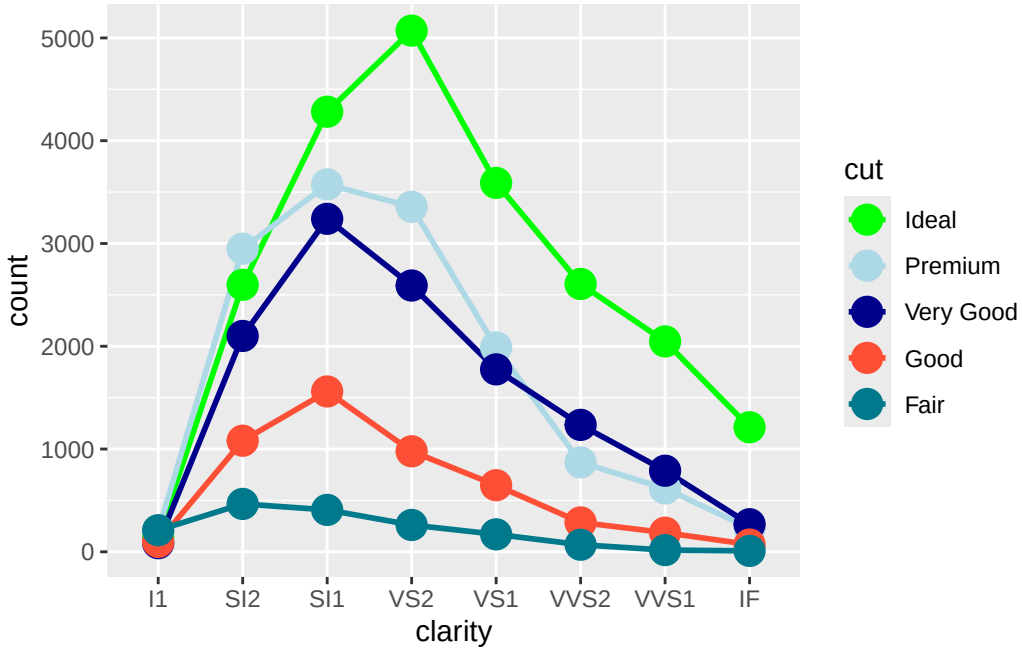

```
p + geom_bar(position = "identity") # bad idea!
```




```
p + geom_bar(position = "identity", alpha = .5) # even worse!!
```




```
diamonds |>
  mutate(cut = fct_rev(cut)) |>
  ggplot(aes(clarity, color = cut, group = cut)) +
  geom_line(stat = "count", position = "identity", lwd = 1) +
  geom_point(stat = "count", size = 5) +
  scale_color_manual(values = c(
    "green", "lightblue",
    "darkblue",
    rgb(253, 79, 54,
      maxColorValue = 255
    ),
    "#00798d"
  ))
```

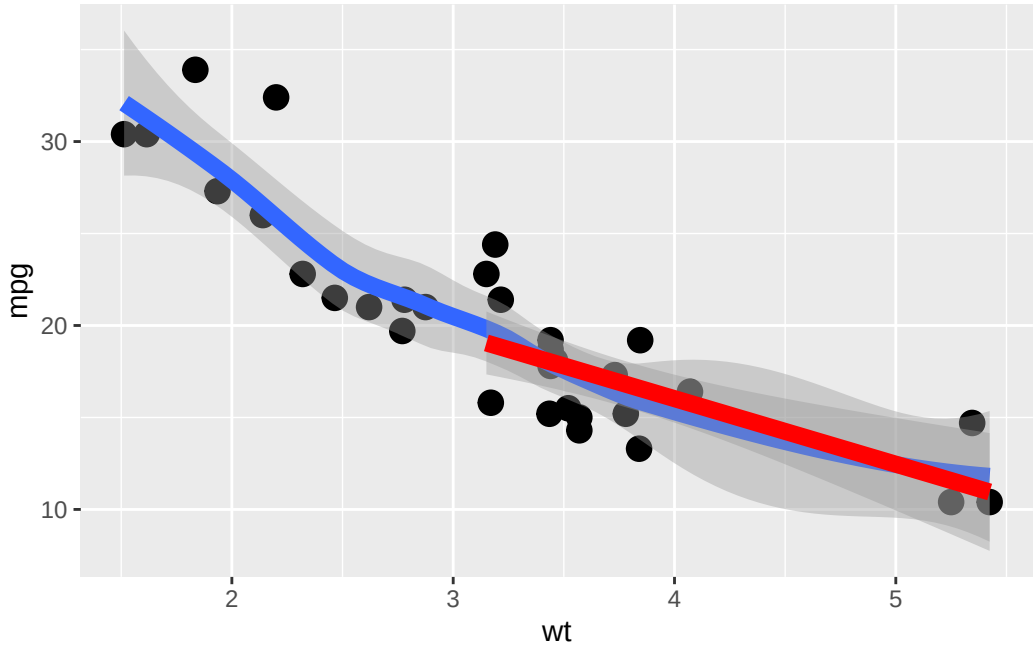


7.10 Order of layers

When combining various geoms, the order is important, as elements are not transparent by default.

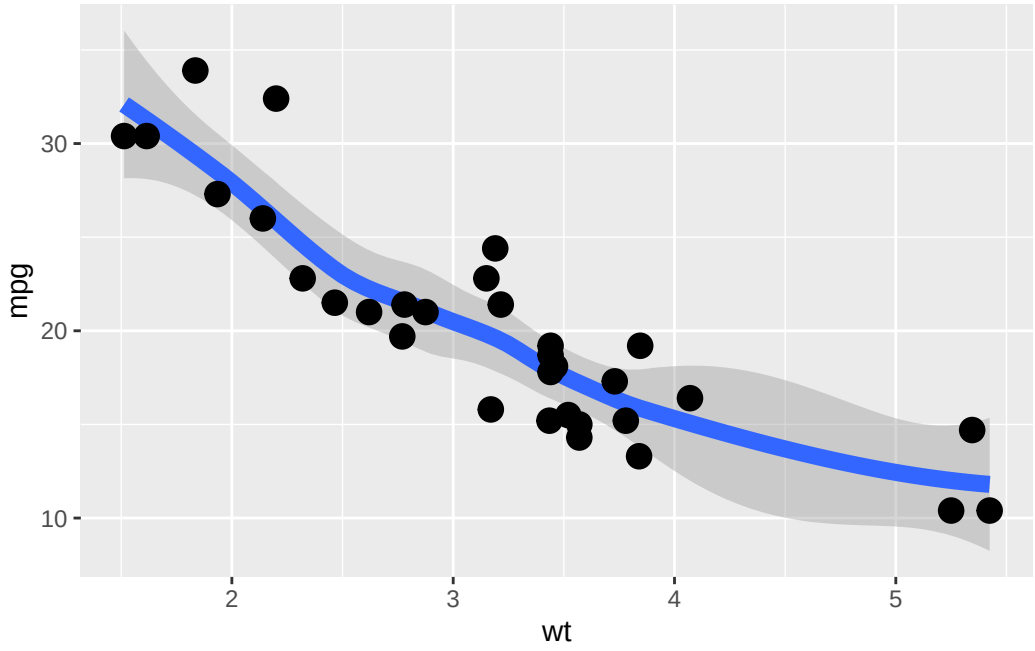
```
ggplot(data = mtcars, aes(wt, mpg)) +
  geom_point(size = 4) +
  geom_smooth(linewidth = 3) + # line overlaps points
  geom_smooth(
    data = mtcars |> filter(wt > 3), # picks a sub-sample
    method = "lm", linewidth = 3, color = "red"
  )
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
```



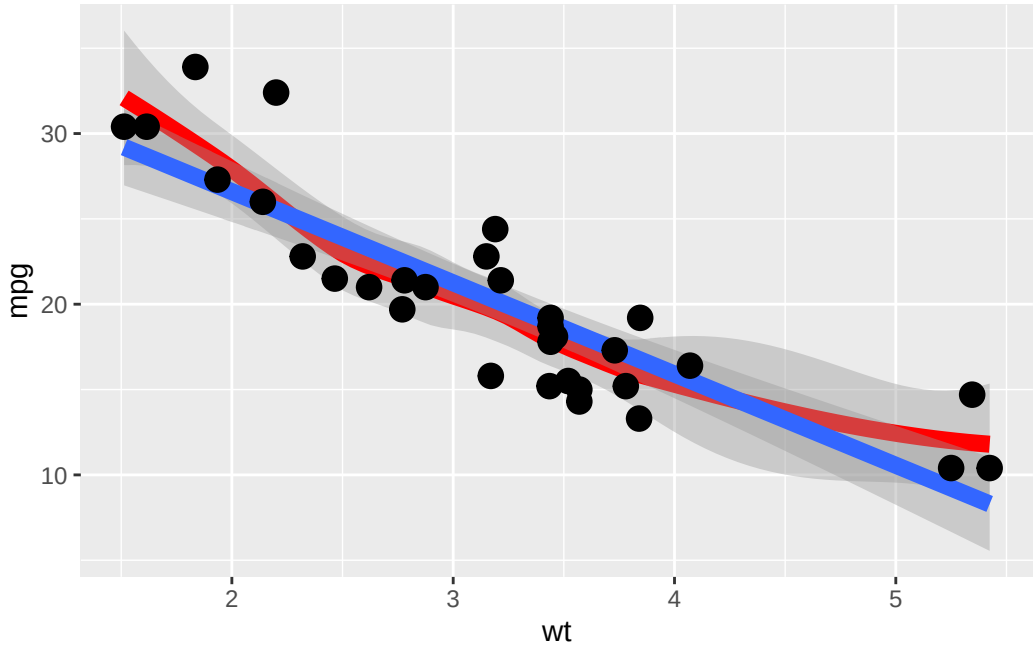

```
ggplot(data = mtcars, aes(wt, mpg)) +
  geom_smooth(linewidth = 3) +
  geom_point(size = 4)
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



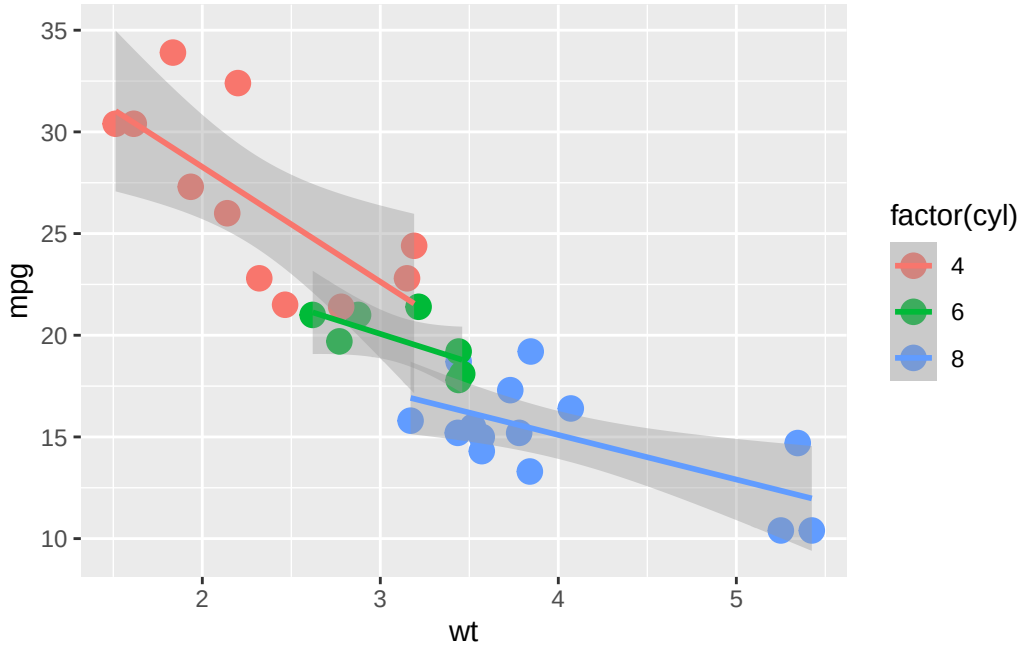

```
ggplot(data = mtcars, aes(wt, mpg)) +
  geom_smooth(linewidth = 3, color = "red") +
  geom_smooth(method = "lm", linewidth = 3) +
  geom_point(size = 4)
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
```



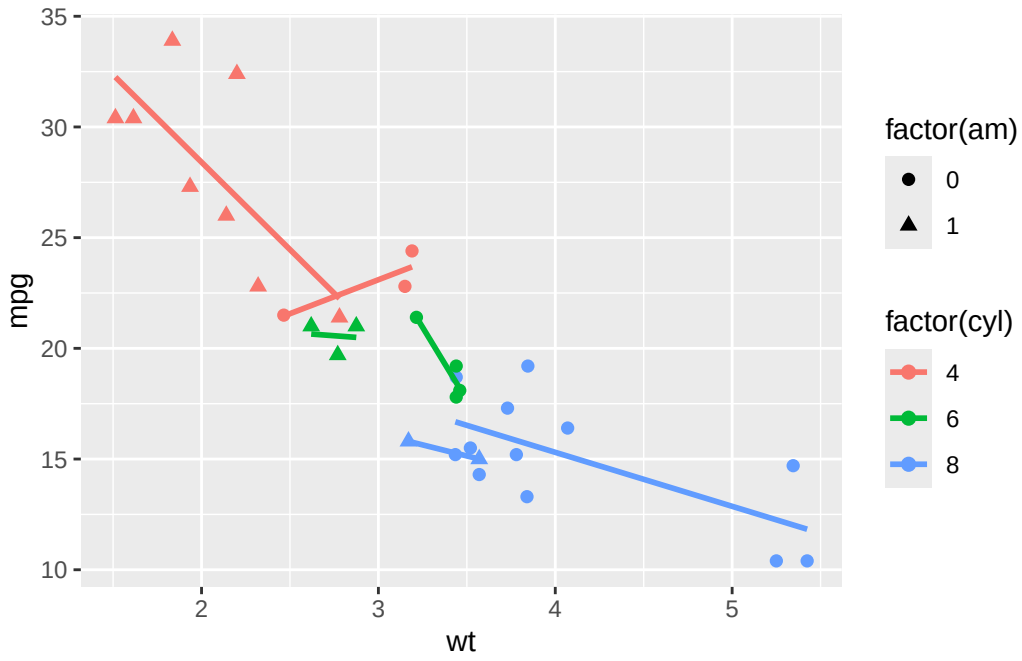

```
ggplot(data = mtcars, aes(wt, mpg,
  color = factor(cyl)
)) +
  geom_point(size = 4) +
  geom_smooth(method = "lm", linewidth = 1)
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
ggplot(data = mtcars, aes(wt, mpg,
  color = factor(cyl),
  shape = factor(am)
)) +
  geom_point(size = 2) +
  geom_smooth(method = "lm", linewidth = 1, se = FALSE)
```

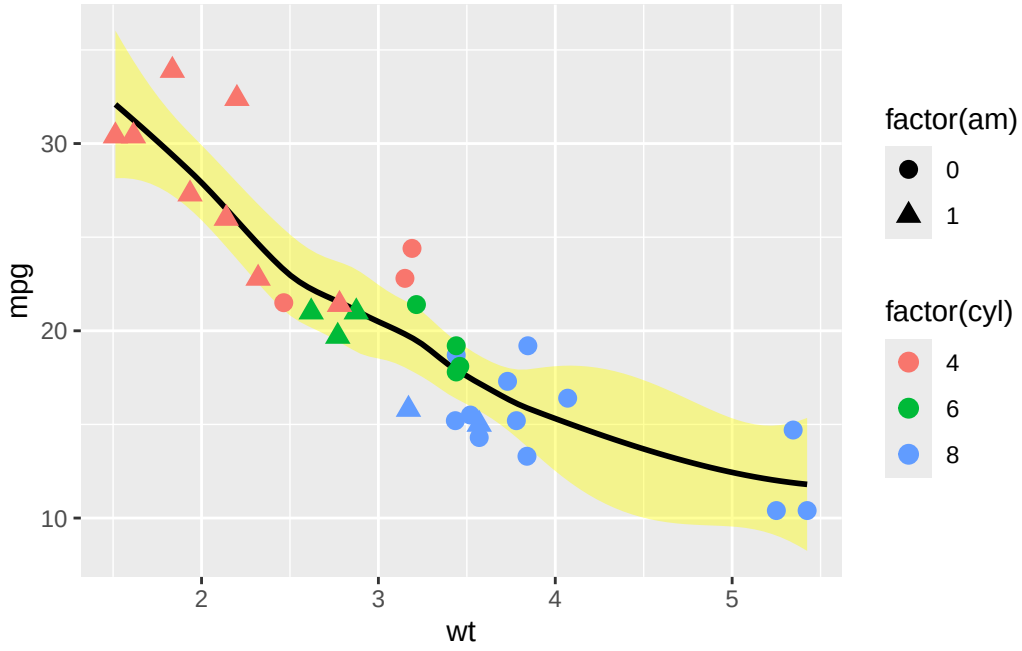
```
`geom_smooth()` using formula = 'y ~ x'
```



7.11 Local aesthetics for layers

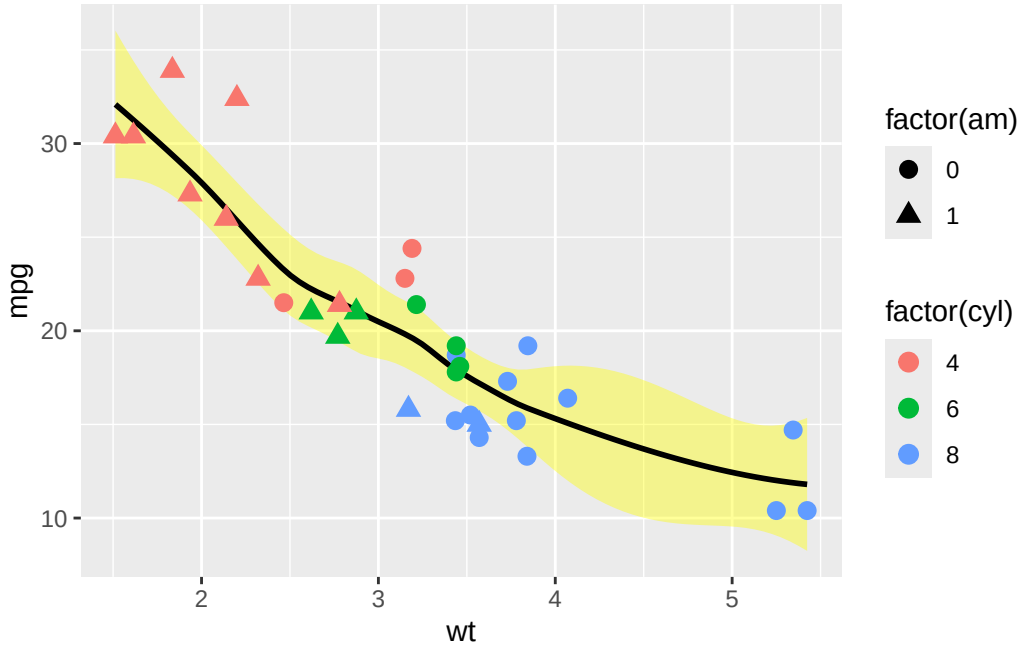
```
# ? lm for all?
ggplot(data = mtcars, aes(wt, mpg)) +
  geom_smooth(linewidth = 1, color = "black", fill = "yellow") +
  geom_point(size = 3, aes(color = factor(cyl), shape = factor(am))) # aes for geom only
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
ggplot(data = mtcars, aes(wt, mpg, color = factor(cyl))) +
  geom_smooth(linewidth = 1, color = "black", fill = "yellow") + # global color overwritten
  geom_point(size = 3, aes(shape = factor(am)))
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



7.12 Faceting (splitting) plots

Visualizing many groups can lead to confusing / too-busy plots, splitting is often an alternative. Visualizing many variables at the same time can be achieved with facets as well (after pivot_longer).

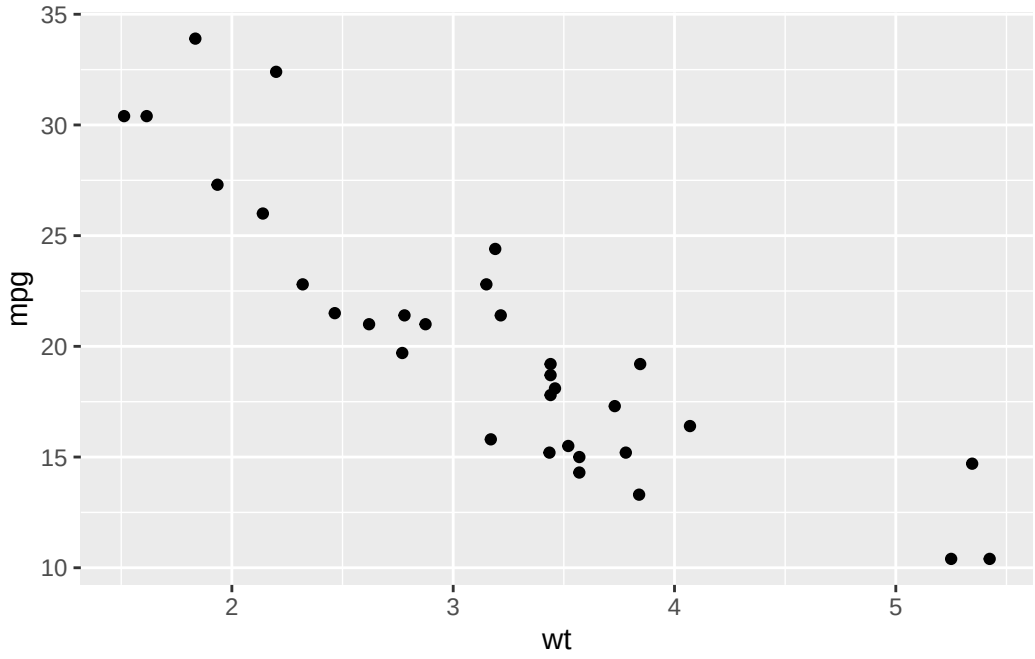
7.12.1 facet_grid

Grids are specified by defining variables for rows and/or columns, empty combinations still are shown.

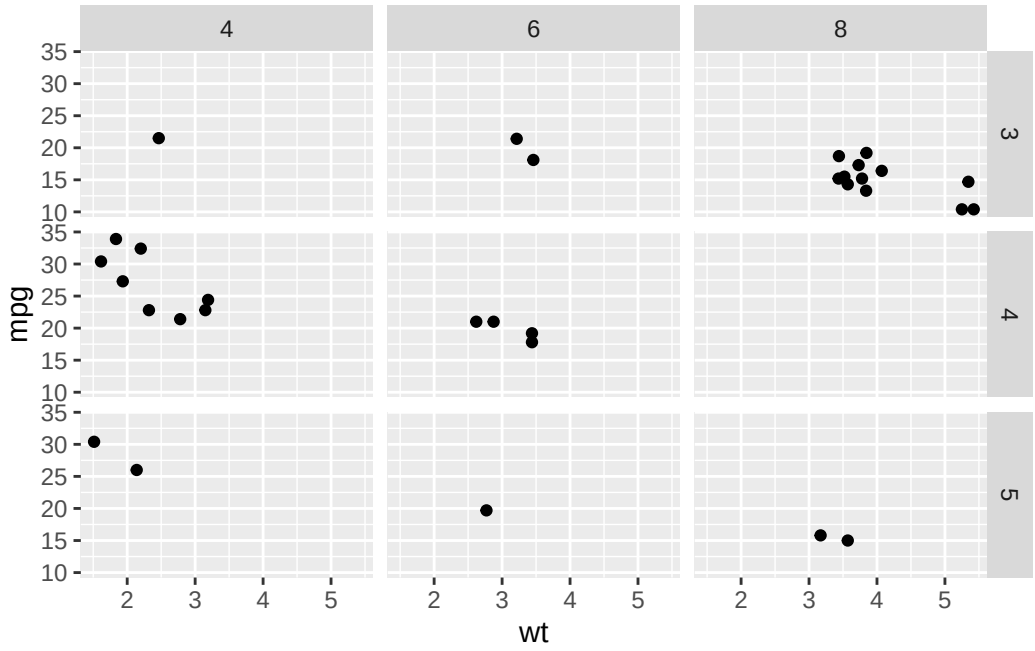
Labeling of facets often requires name and content to be informative.

Margins (taking all elements together) can be shown for rows and/or columns.

```
(plot_tmp <- ggplot(mtcars, aes(wt, mpg)) +
  geom_point())
```



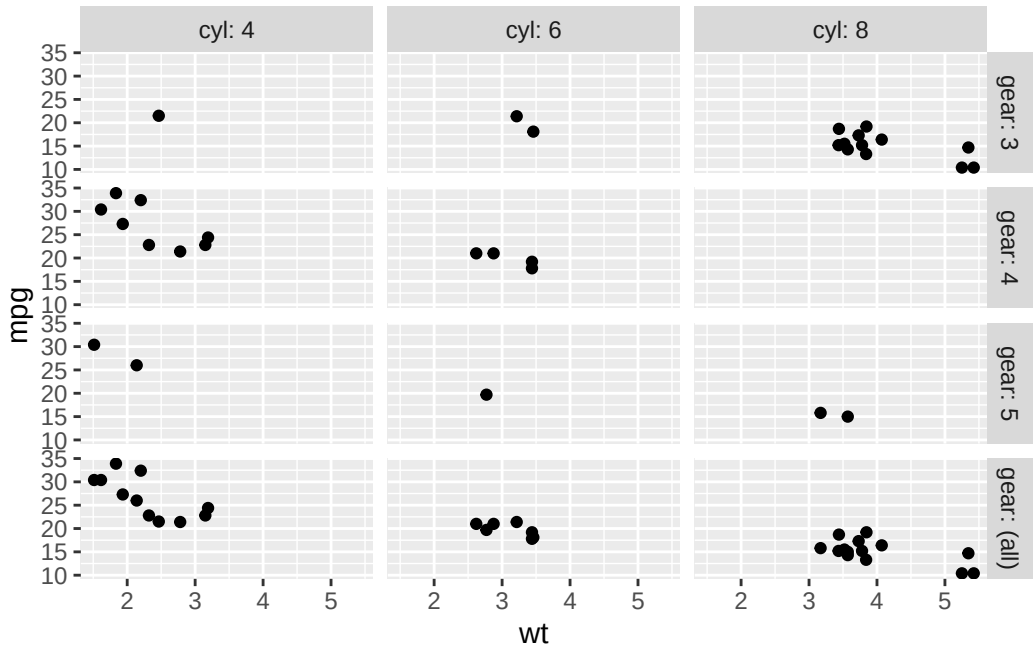

```
plot_tmp + facet_grid(
  rows = vars(gear),
  cols = vars(cyl)
)
```



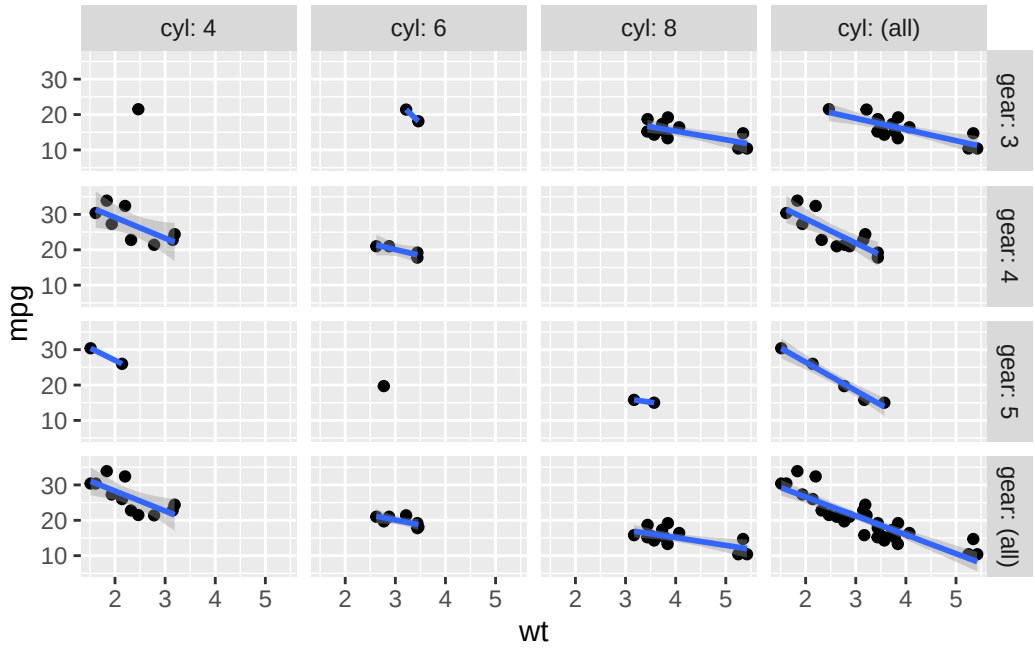
```
cat("facet labeling improved:\n")
```

```
facet labeling improved:
```

```
plot_tmp + facet_grid(
  rows = vars(gear),
  cols = vars(cyl),
  labeller = label_both, margins = "gear"
)
```

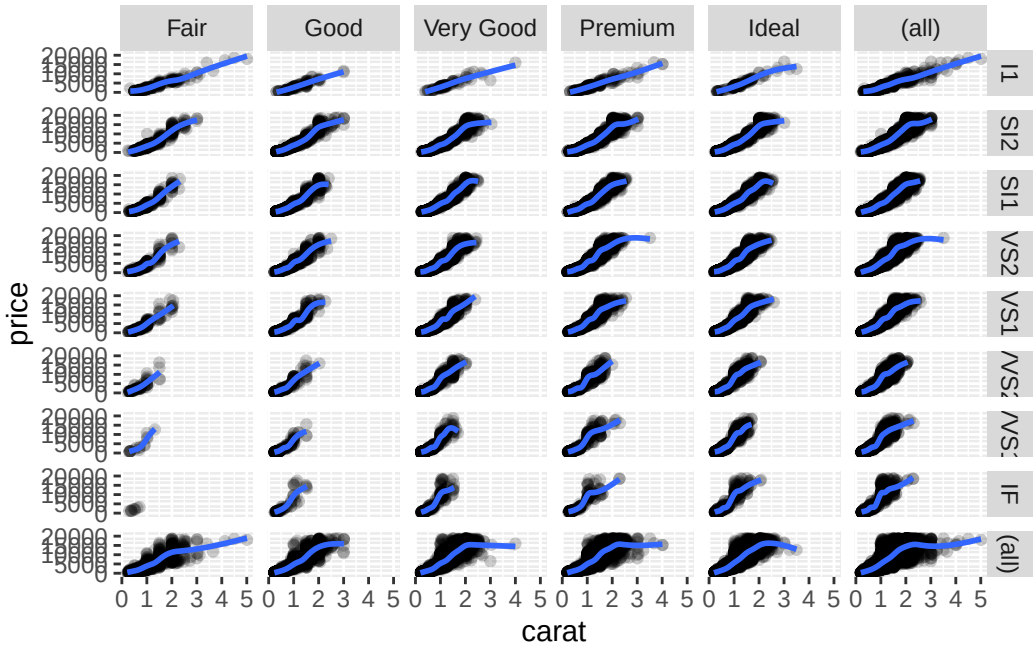



```
# options(warn = -1)
plot_tmp + geom_smooth(method = "lm") +
  facet_grid(
    rows = vars(gear),
    cols = vars(cyl),
    labeller = label_both, margins = TRUE
  )
```




```
# options(warn = 0)

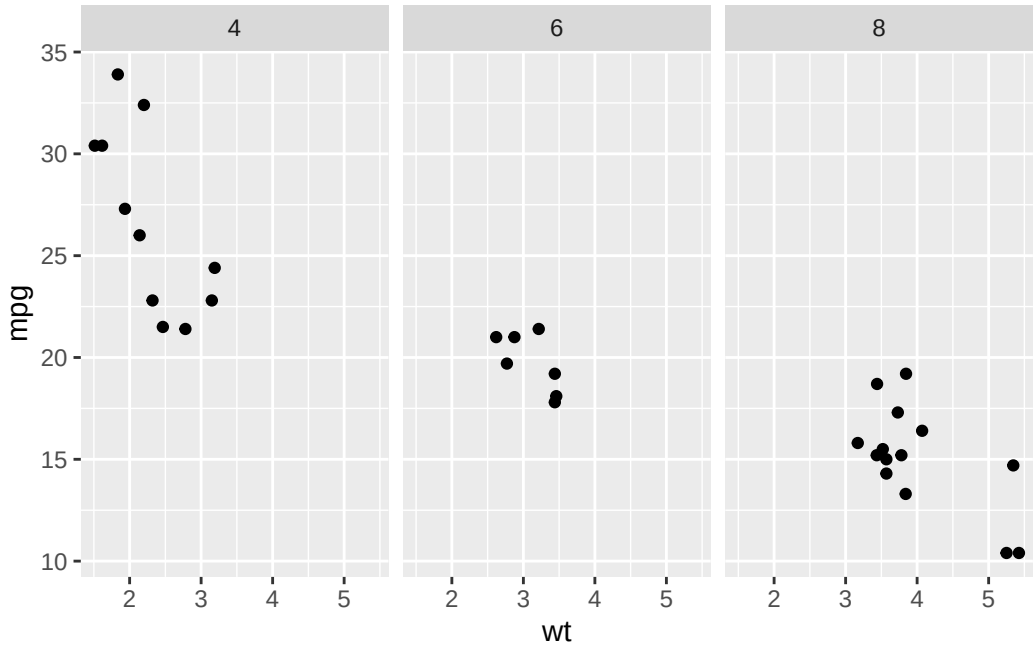
ggplot(diamonds, aes(carat, price)) +
  geom_point(alpha = .2) +
  geom_smooth() +
  facet_grid(
    rows = vars(clarity),
    cols = vars(cut),
    margins = TRUE
  )
```



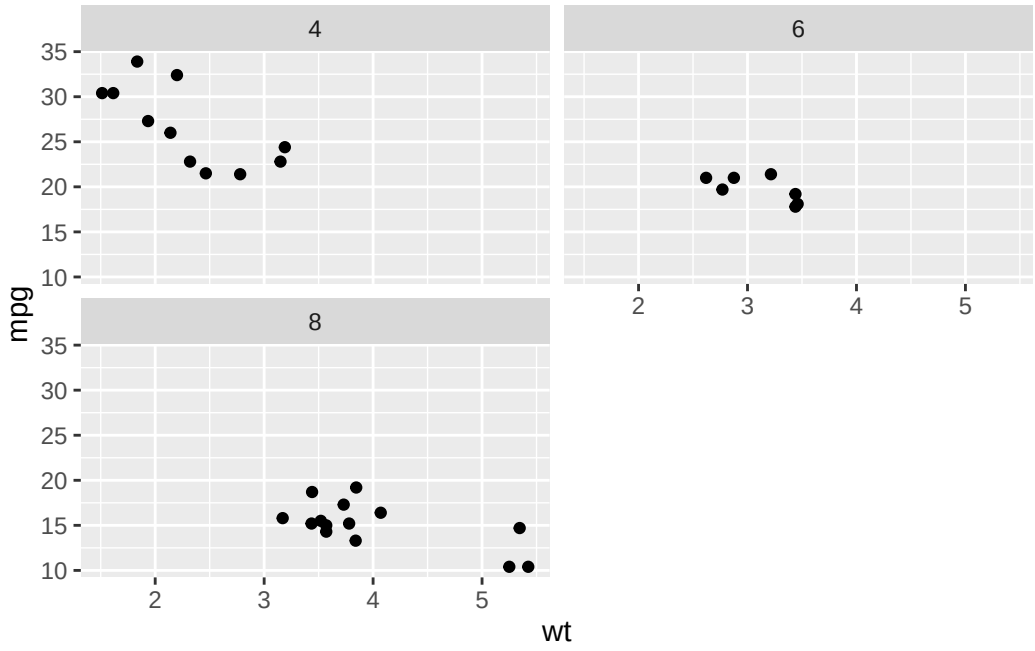
7.12.2 facet_wrap

When showing many facets, wrapping around after some is useful, less systematic than grid.

```
plot_tmp + facet_wrap(facets = vars(cyl))
```

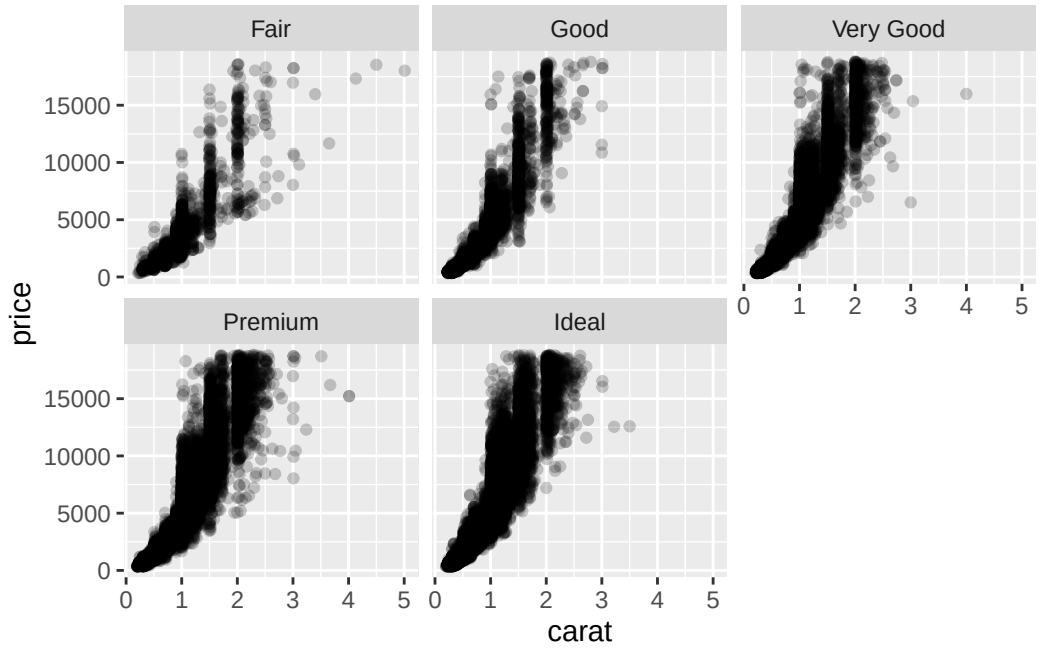



```
plot_tmp + facet_wrap(facets = vars(cyl), ncol = 2)
```

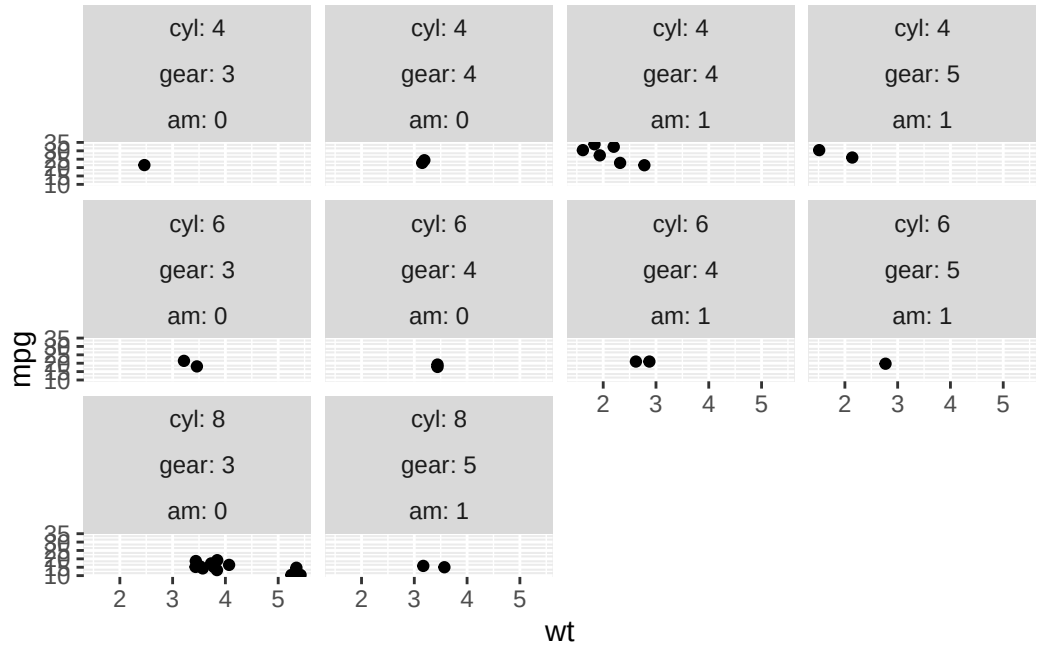



```
ggplot(diamonds, aes(carat, price))+
  geom_point(alpha = .2)+
```

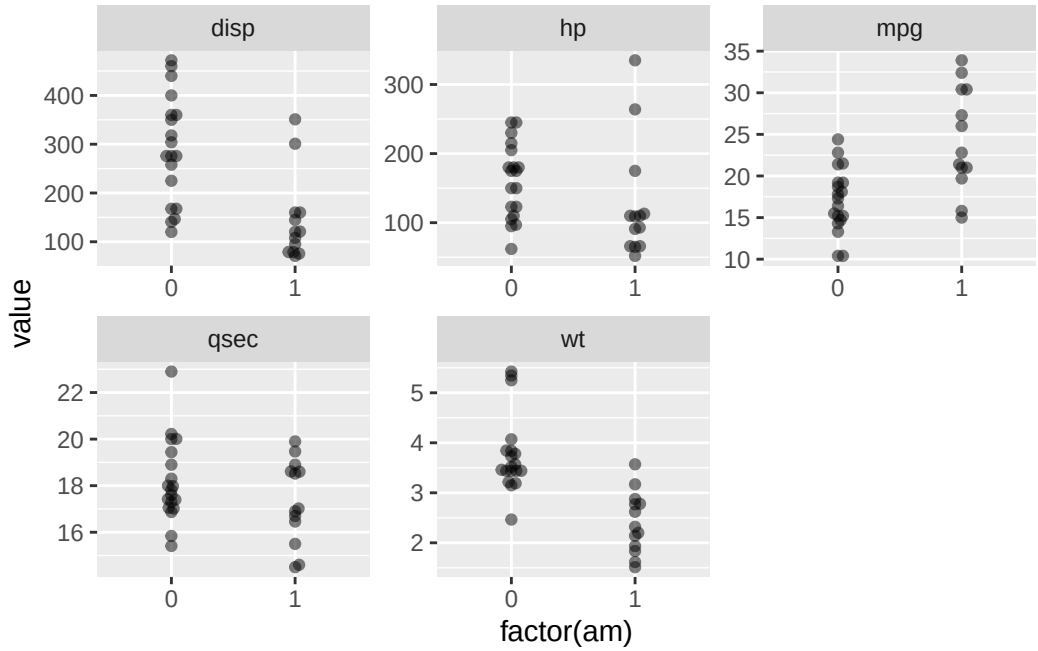
```
facet_wrap(facets = vars(cut))
```




```
# empty combination is dropped
plot_tmp + facet_wrap(facets = vars(cyl, gear, am), labeller = label_both)
```

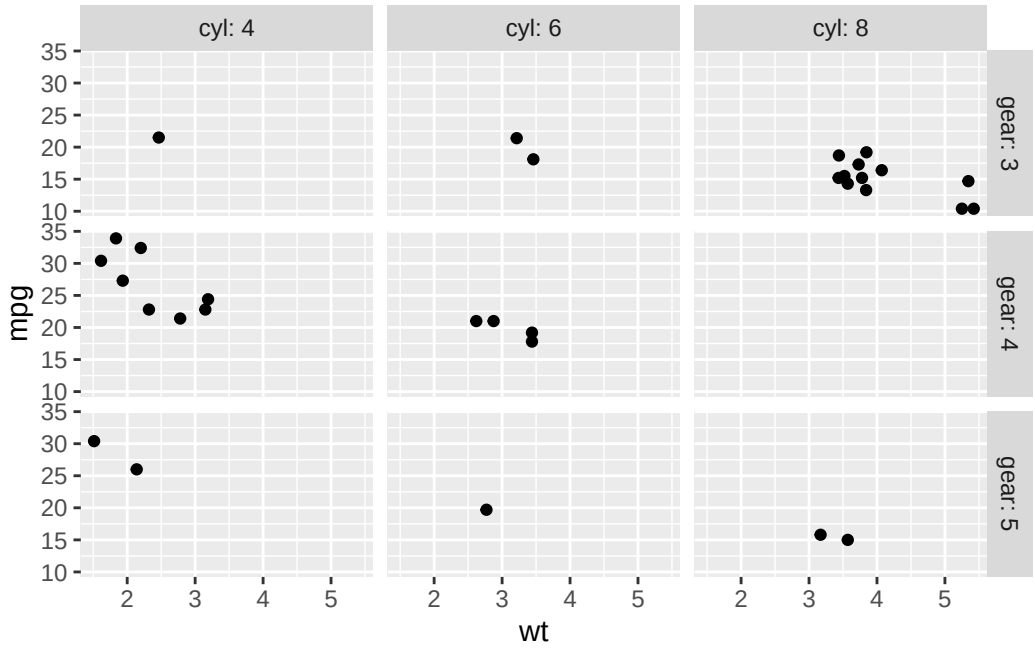


```r
# combining variables
mtcars |>
  pivot_longer(cols = c(wt, mpg, hp, disp, qsec)) |> # view()
  ggplot(aes(x = factor(am), y = value)) +
  geom_beeswarm(alpha = .5, cex = 2) +
  facet_wrap(facets = vars(name), scales = "free")
```

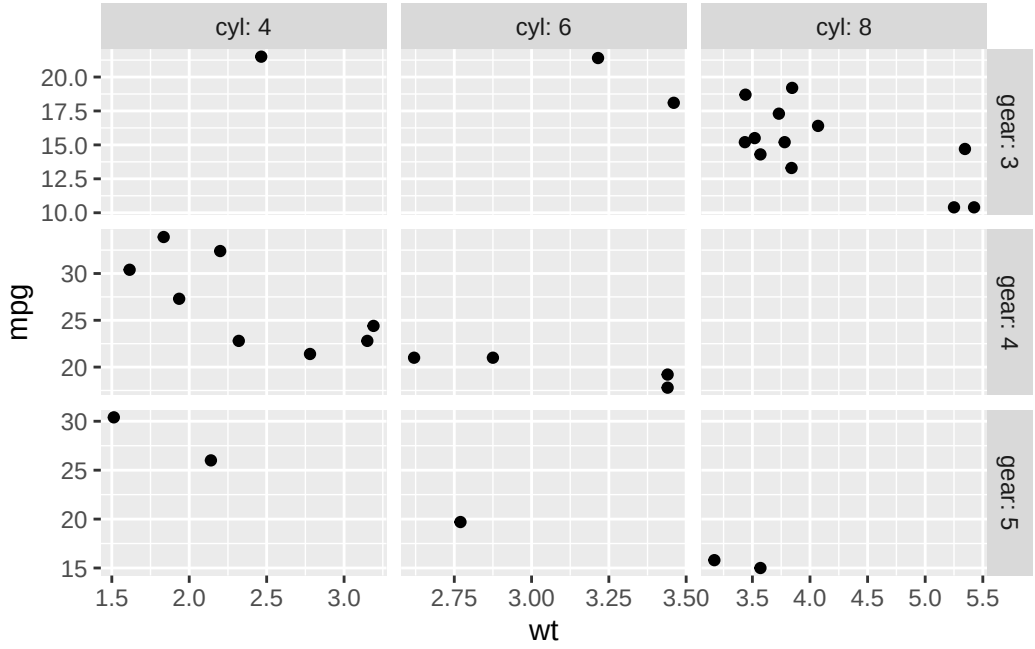


7.12.3 Controlling scales in facets (default: scales = "fixed")

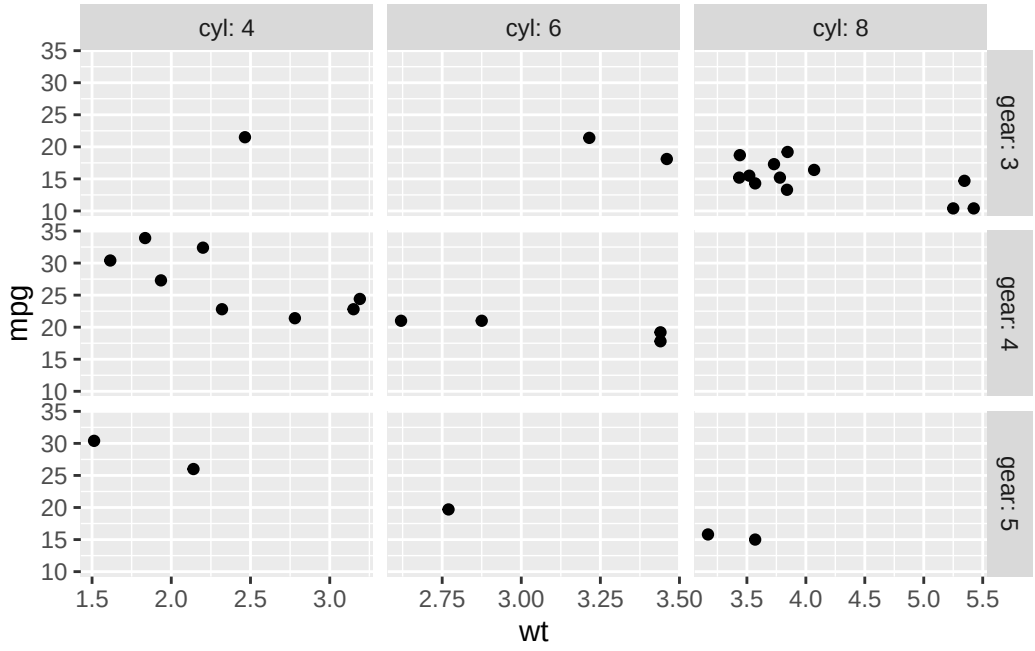
```r
plot_tmp + facet_grid(
  rows = vars(gear), cols = vars(cyl),
  labeller = label_both, scales = "fixed"
)
```



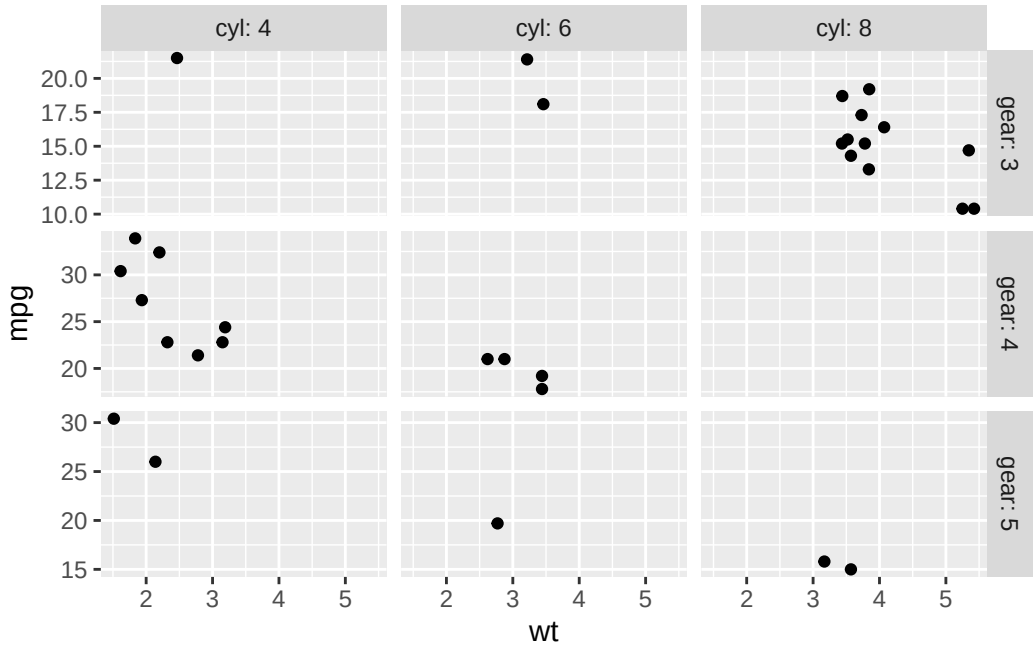

```
plot_tmp + facet_grid(
  rows = vars(gear), cols = vars(cyl),
  labeller = label_both, scales = "free"
)
```



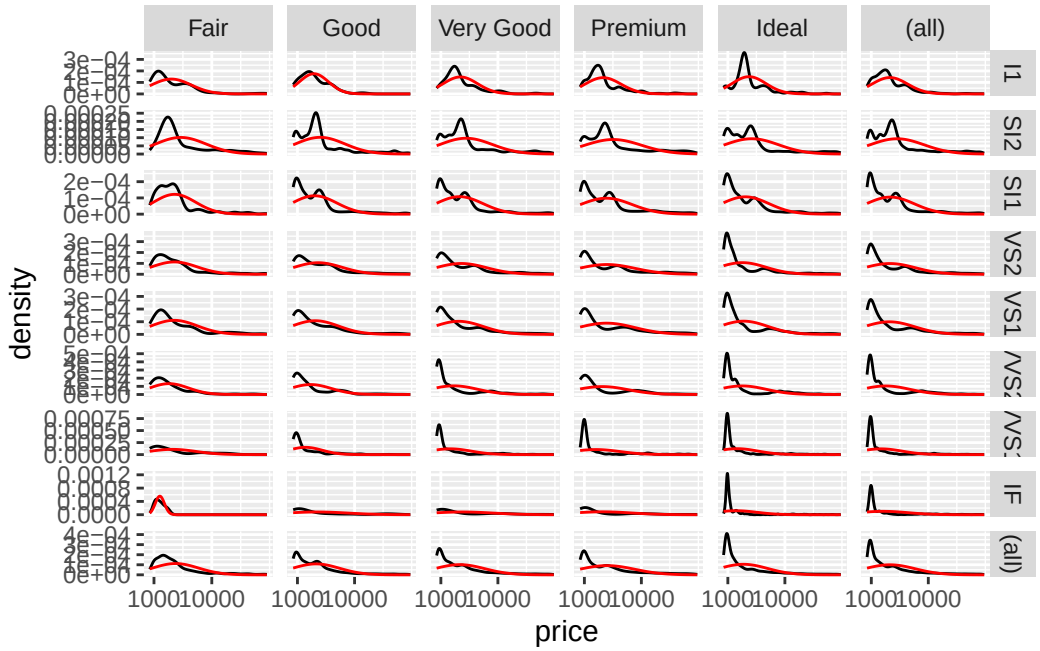
```
plot_tmp + facet_grid(
  rows = vars(gear), cols = vars(cyl),
  labeller = label_both, scales = "free_x"
)
```




```
plot_tmp + facet_grid(
  rows = vars(gear), cols = vars(cyl),
  labeller = label_both, scales = "free_y"
)
```




```
ggplot(diamonds, aes(price)) +
  geom_density() +
  stat_theodensity(distri = "norm", linewidth = 0.5, color = "red") +
  scale_x_continuous(breaks = 10^(3:5)) +
  facet_grid(rows = vars(clarity), cols = vars(cut),
             scales = "free",
             margins = TRUE)
```


Fair
Good
Very Good
Premium
Ideal
(all)
I1
SI2
SI1
VS2
VS1
VS2
VS1
IF
(all)
density
price
1000
10000

7.13 Exercise 2

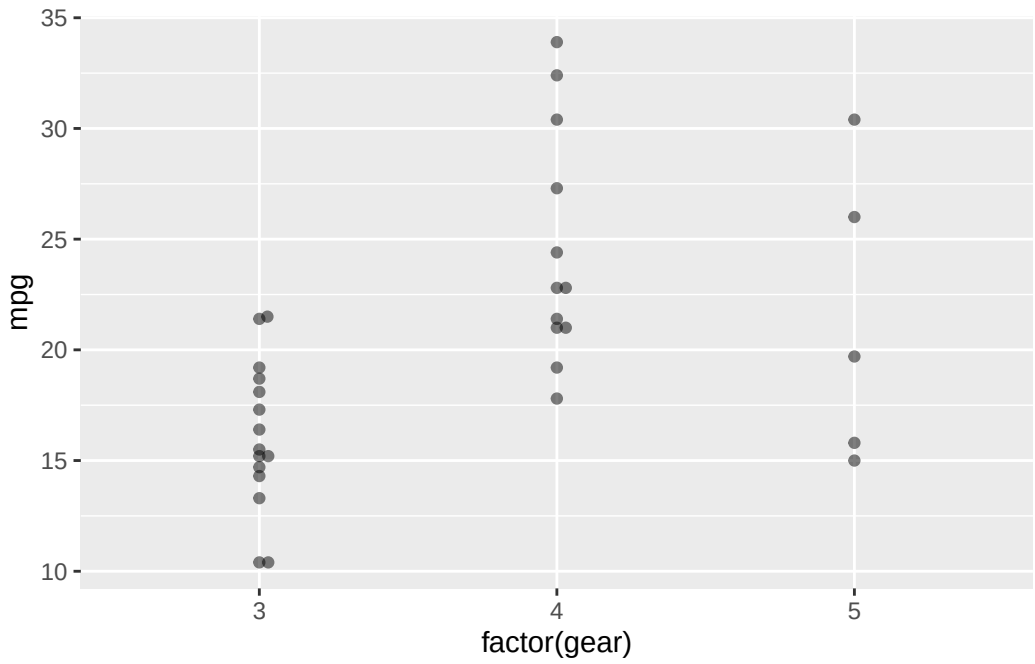
- distribution of body mass by species and sex (density, histogram)
- without / with facetting

7.14 Showing summaries

While plotting underlying rawdata is pretty informative, adding summary statistics guides the viewer. Error bars help to evaluate differences visible, but need to be labelled!!

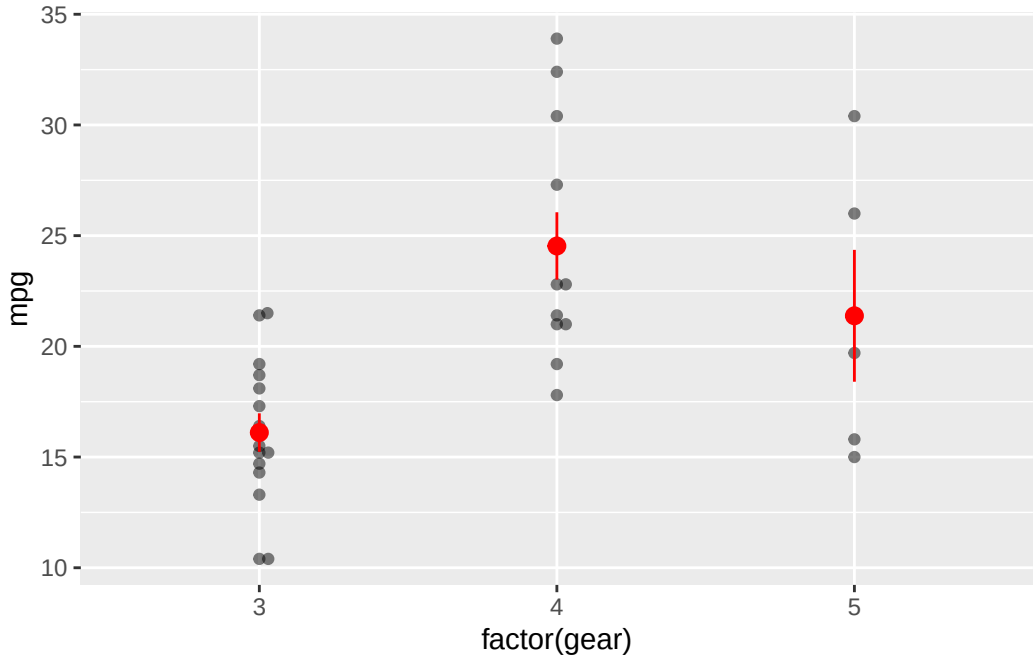
Functions for summary statistics (mean_se, mean_cl_normal, mean_cl_boot etc.) are build on top of Hmisc functions. So this package is needed but not automatically installed with ggplot2.

```
(plottemp <- ggplot(mtcars, aes(factor(gear), mpg)) +
  geom_beeswarm(alpha = .5))
```

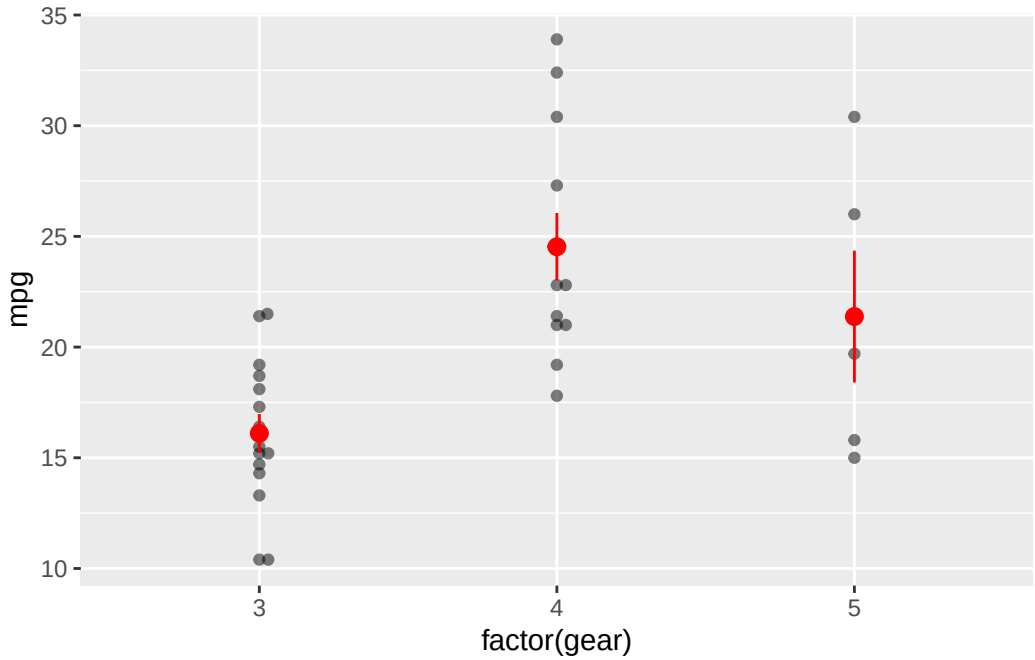



```
plottemp + stat_summary(color = "red")
```

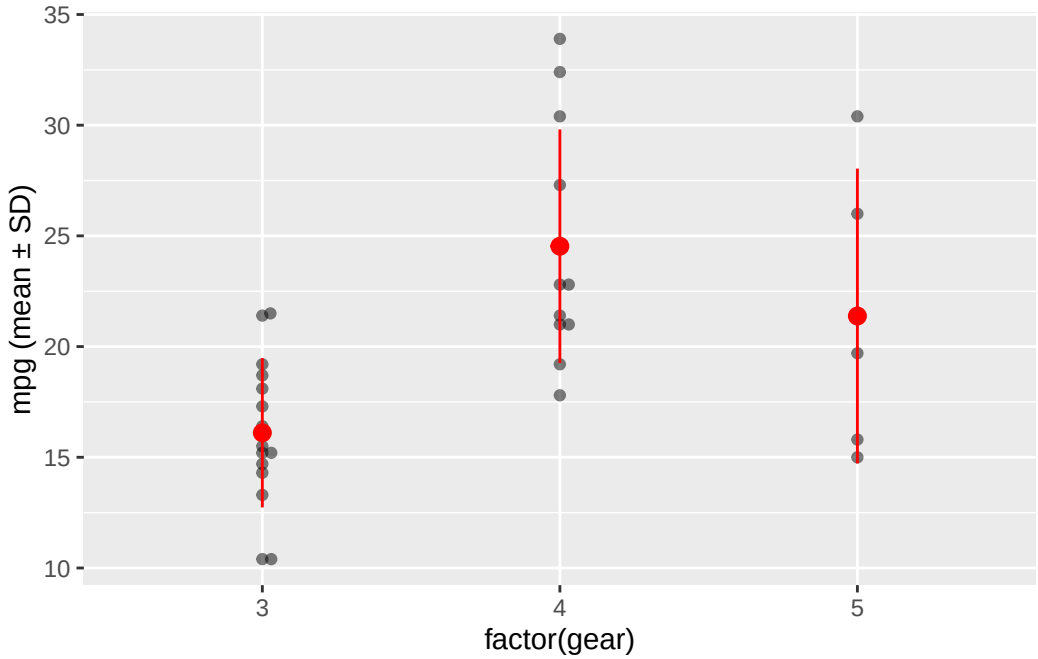
```
No summary function supplied, defaulting to `mean_se()`
```



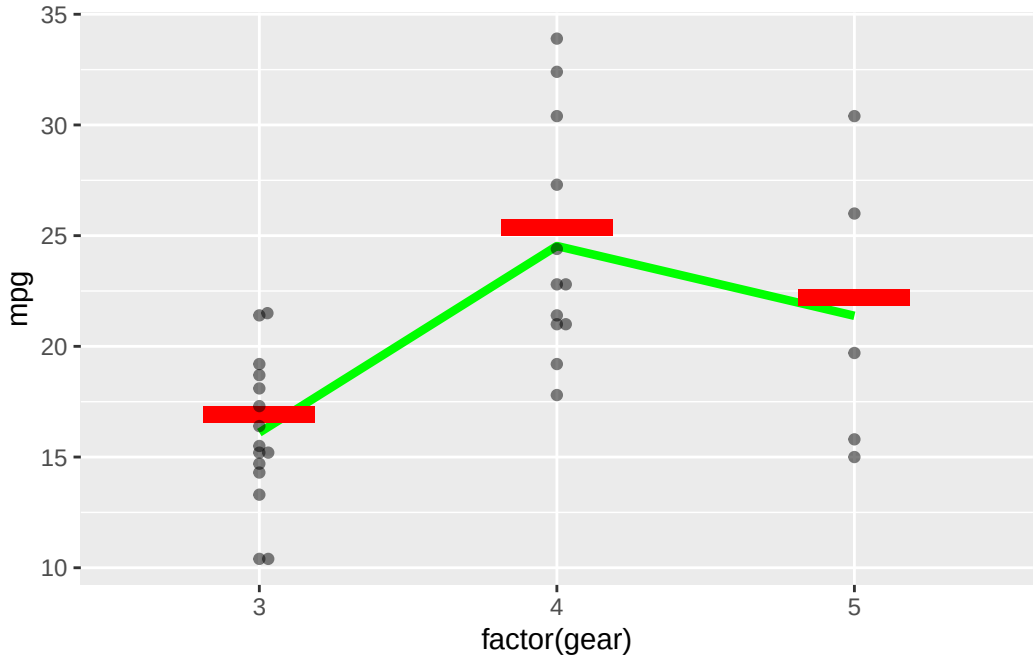

```
plottemp + stat_summary(
  fun.data = "mean_se",
  color = "red"
) # y-label still uninformative!
```



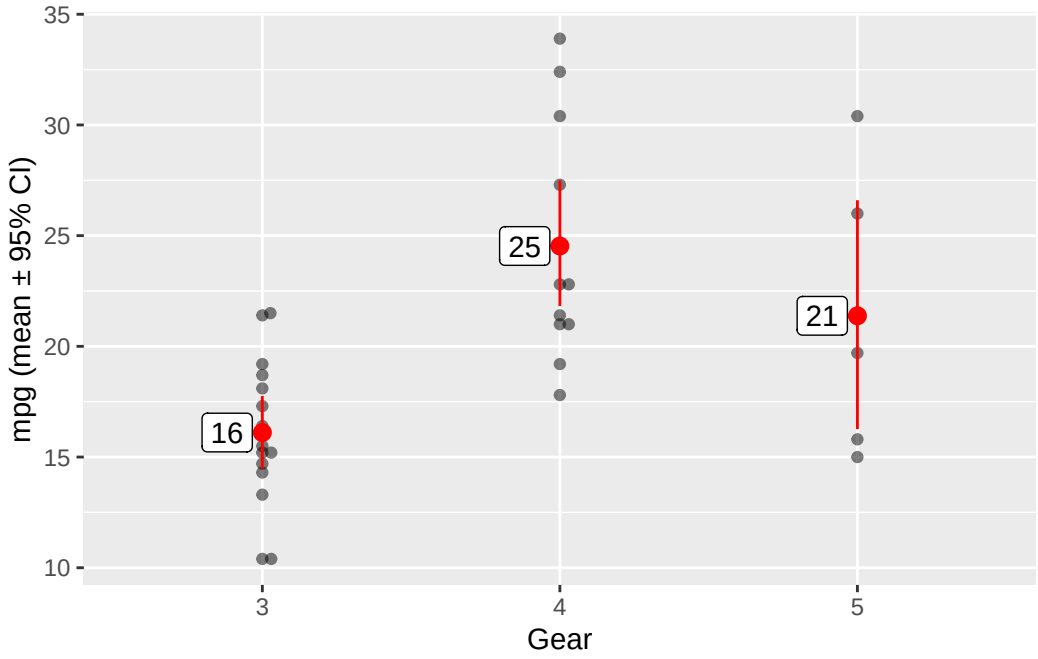
```
plottemp + stat_summary(
  fun.data = "mean_sdl",
  fun.args = list(mult = 1),
  color = "red"
) +
  ylab("mpg (mean \u00b1 SD)")
```



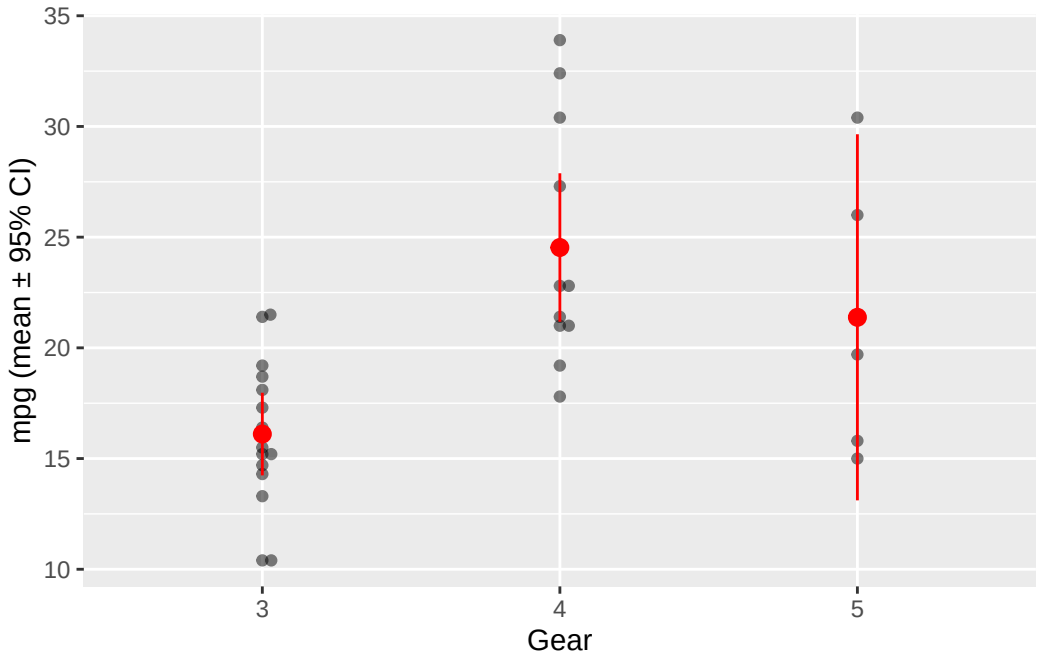

```
ggplot(mtcars, aes(factor(gear), mpg)) +
  stat_summary(
    geom = "line", aes(group = "all"),
    linewidth = 1.5,
    fun = "mean", color = "green"
  ) +
  stat_summary(
    geom = "point", shape = "-",
    size = 30,
    fun = "mean", color = "red"
  ) +
  geom_beeswarm(alpha = .5)
```



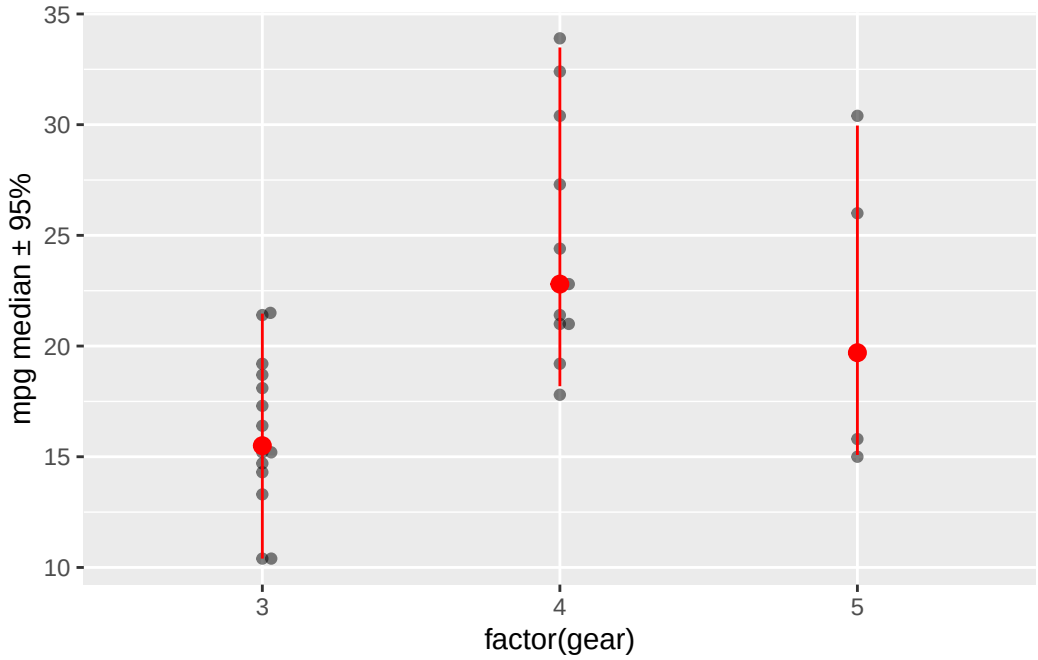

```
means <- mtcars |>
  group_by(gear) |>
  summarise(mean = round(mean(mpg), 3), sd = sd(mpg))
plottemp + stat_summary(
  fun.data = "mean_cl_boot",
  fun.args = list(B = 10^4),
  color = "red"
) +
  geom_label(
    data = means,
    aes(factor(gear), mean, label = round(mean)),
    hjust = 1.2
  ) +
  ylab("mpg (mean \u00b1 95% CI)") +
  xlab("Gear")
```



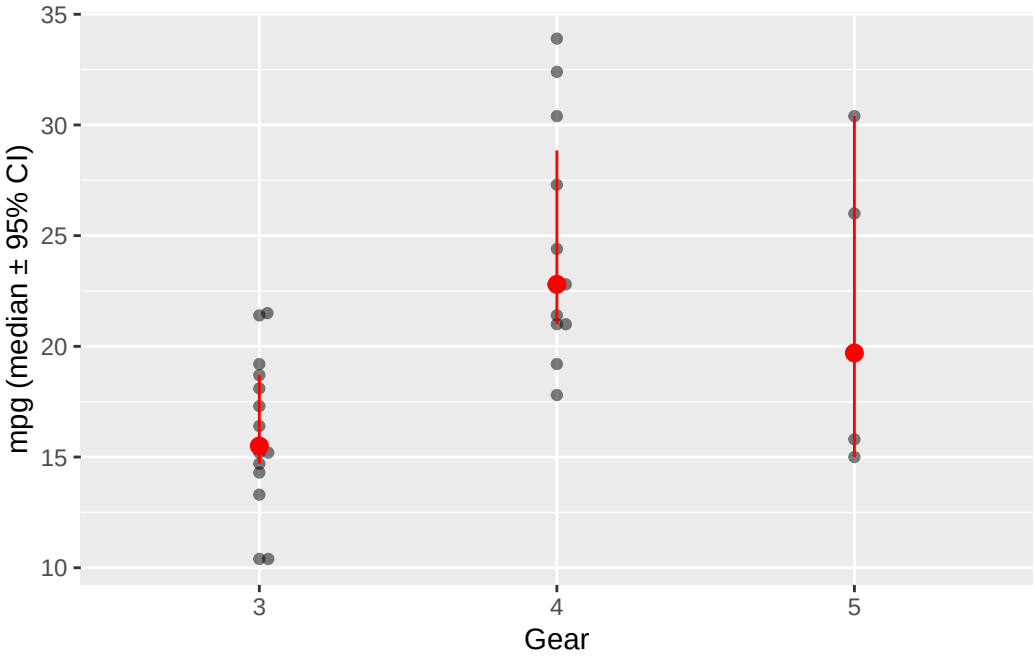

```
plottemp + stat_summary(fun.data = "mean_cl_normal", color = "red") +
  ylab("mpg (mean \u00b1 95% CI)") +
  xlab("Gear")
```



```
plottemp + stat_summary(fun.data = "median_hilow", color = "red") +
  ylab("mpg median \u00b1 95%")
```



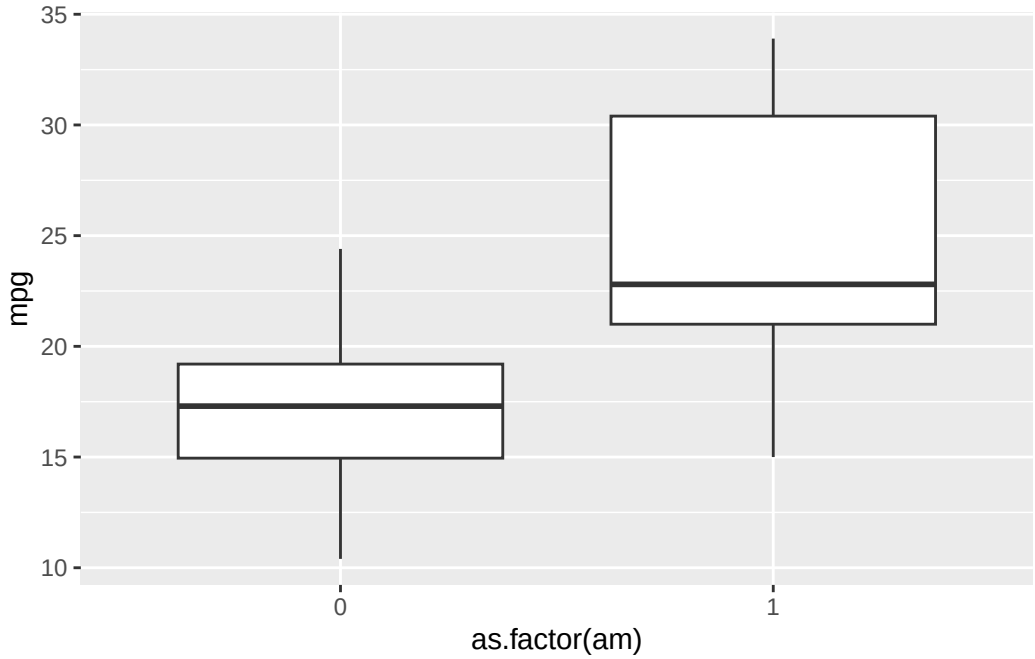

```
# geom_pointrange()
plottemp + stat_summary(fun.data = "median_cl_boot_gg", color = "red") +
  ylab("mpg (median \u00b1 95% CI)") +
  xlab("Gear")
```



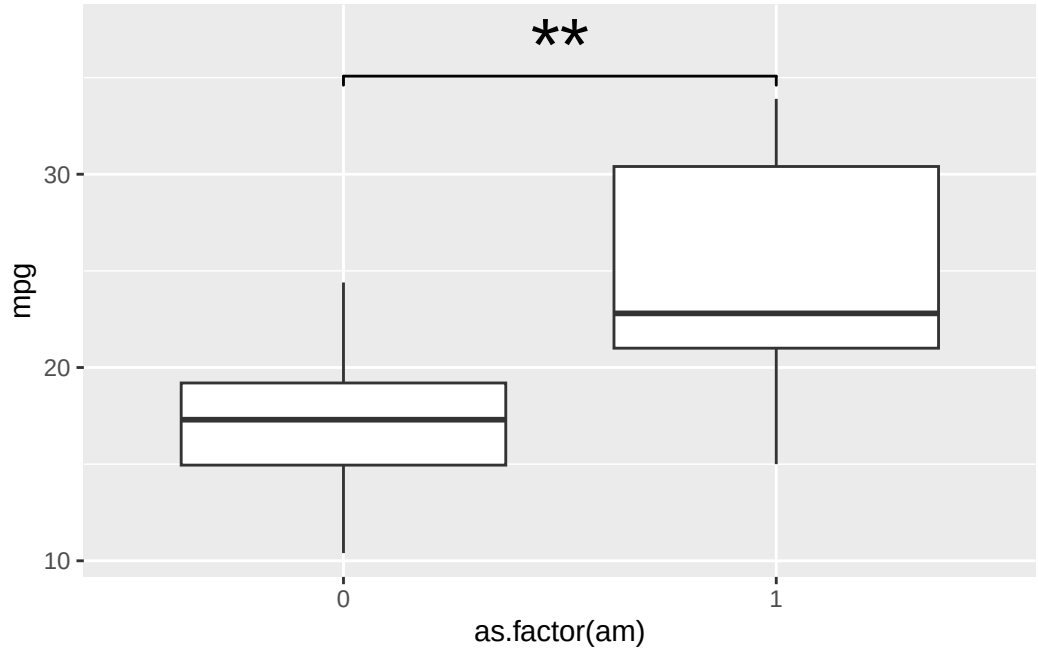
7.15 Indicating significances

Package ggsign makes it easier to add significance brackets (no more photoshopping), it either computes p-values or takes them from your testing (and this is what you should always be doing!).

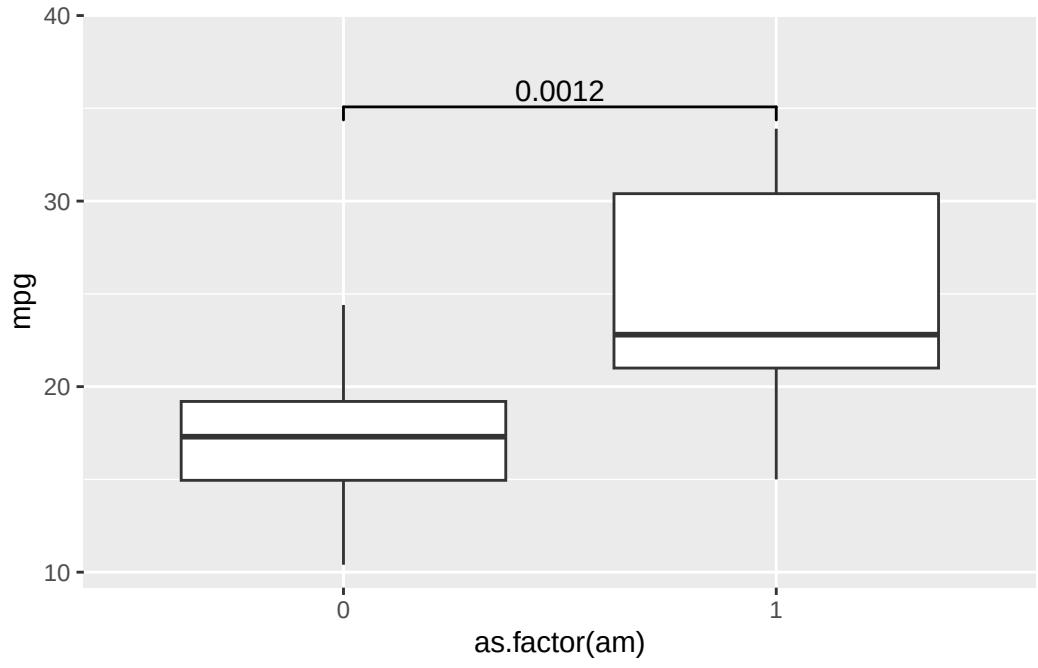
```
# ggsign ####
p <- round(
  wilcox.test(mtcars$mpg ~ mtcars$am, exact = FALSE)$p.value,
  5
)
(plottemp <- ggplot(mtcars, aes(as.factor(am), mpg)) +
  geom_boxplot())
```



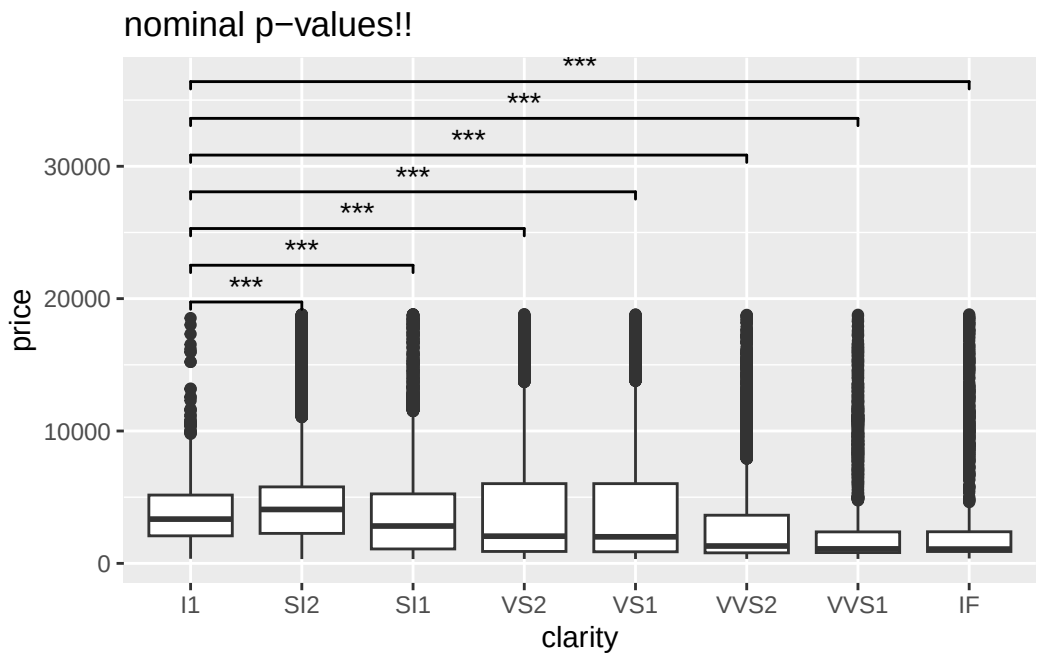

```
plottemp +
  geom_signif(
    comparisons = list(c(1, 2)),
    # aes(y = 0),
    textsize = rel(10), vjust = .0,
    # y_position = max(mtcars$mpg+3),
    # annotations = paste0("p = ", p),
    annotations = markSign(p),
    # annotations = p,
    tip_length = .02
  ) +
  scale_y_continuous(expand = expansion(mult = c(0.05, .15)))
```




```
plottemp + geom_signif(
  comparisons = list(1:2)
) +
  scale_y_continuous(expand = expansion(mult = c(0.05, .2)))
```

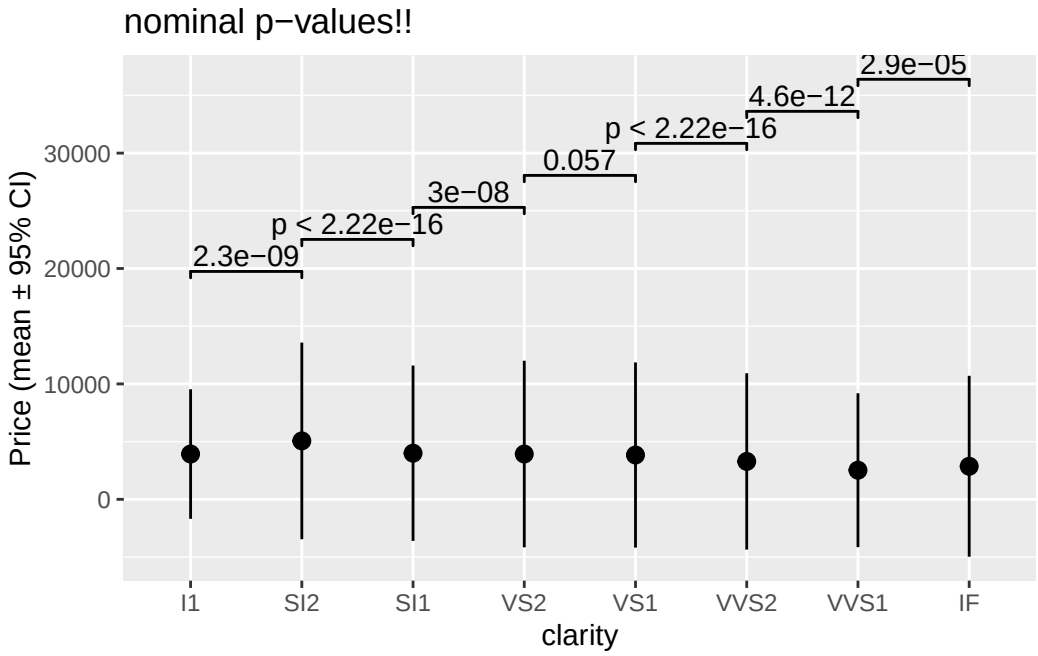


```r
ggplot(diamonds, aes(clarity, price)) +
  geom_boxplot() +
  # stat_summary(fun.data = mean_sdl) +
  geom_signif(
    comparisons = list(
      c(1, 2), c(1, 3), c(1, 4), c(1, 5),
      c(1, 6), c(1, 7), c(1, 8)
    ),
    map_signif_level = TRUE,
    step_increase = 0.15
  ) +
  ggtitle("nominal p-values!!")
```

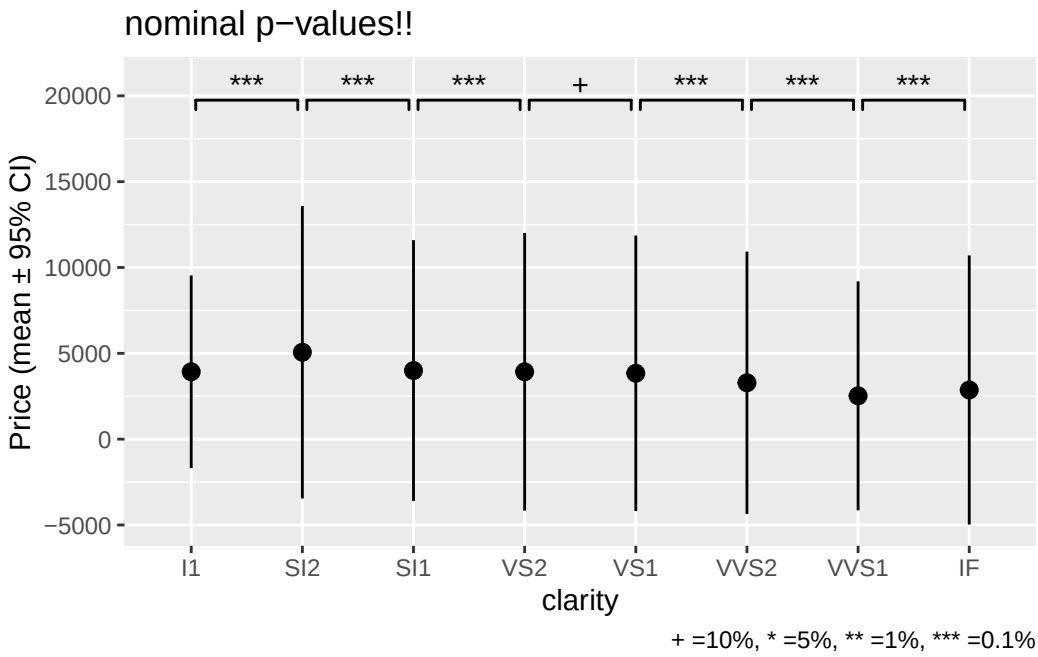



```r
ggplot(diamonds, aes(clarity, price)) +
  stat_summary(fun.data = mean_sdl) +
  ylab("Price (mean \u00b1 95% CI)") +
  geom_signif(
    comparisons = list(
      c(1, 2), c(2, 3), c(3, 4), c(4, 5),
      c(5, 6), c(6, 7), c(7, 8)
    ),
    # map_signif_level = TRUE,
    step_increase = 0.15
```

```
  ) +
  ggtitle("nominal p-values!!")
```




```
ggplot(diamonds, aes(clarity, price)) +
  stat_summary(fun.data = mean_sdl) +
  ylab("Price (mean \u00b1 95% CI)") +
  geom_signif(
    comparisons = list(
      c(1, 2), c(2, 3), c(3, 4), c(4, 5),
      c(5, 6), c(6, 7), c(7, 8)
    ),
    map_signif_level = markSign, # instead of TRUE: alternative function
    extend_line = -.005
  ) +
  ggtitle("nominal p-values!!") +
  scale_y_continuous(expand = expansion(mult = c(0.05, .1))) +
  labs(caption = "+ \u226410%, * \u22645%, ** \u22641%, *** \u22640.1%")
```

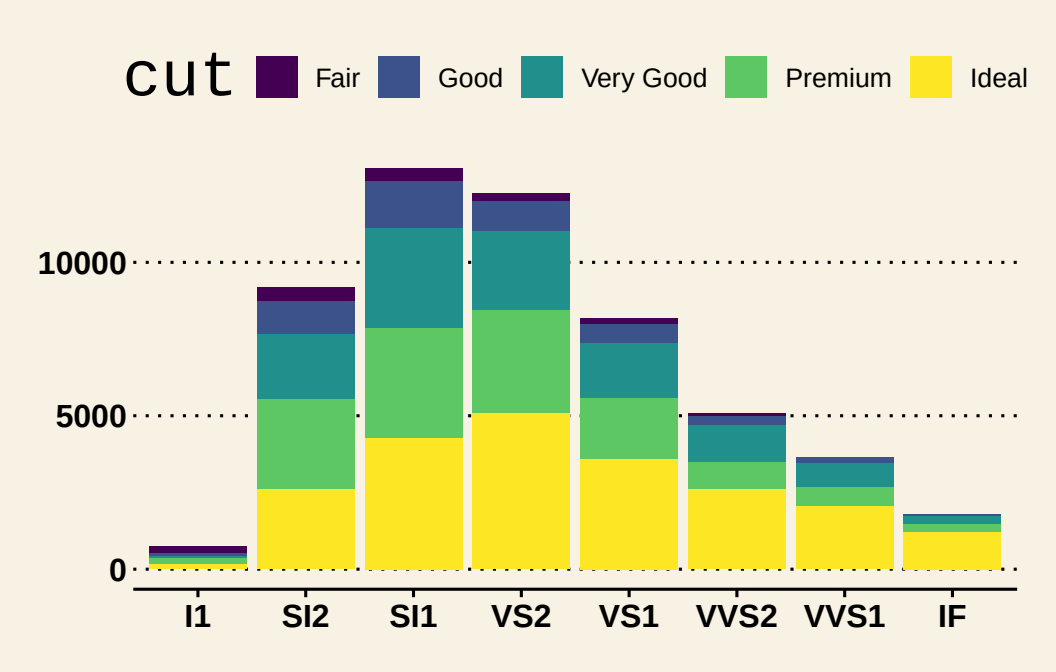


7.16 Theme definitions / changes

Themes define everything not-data-related in your figures, like margins, fonts, background color etc. There are many predefined themes, and all can be customized. You can change a theme for all plots to come (theme_update()) or just a single plot (+theme())

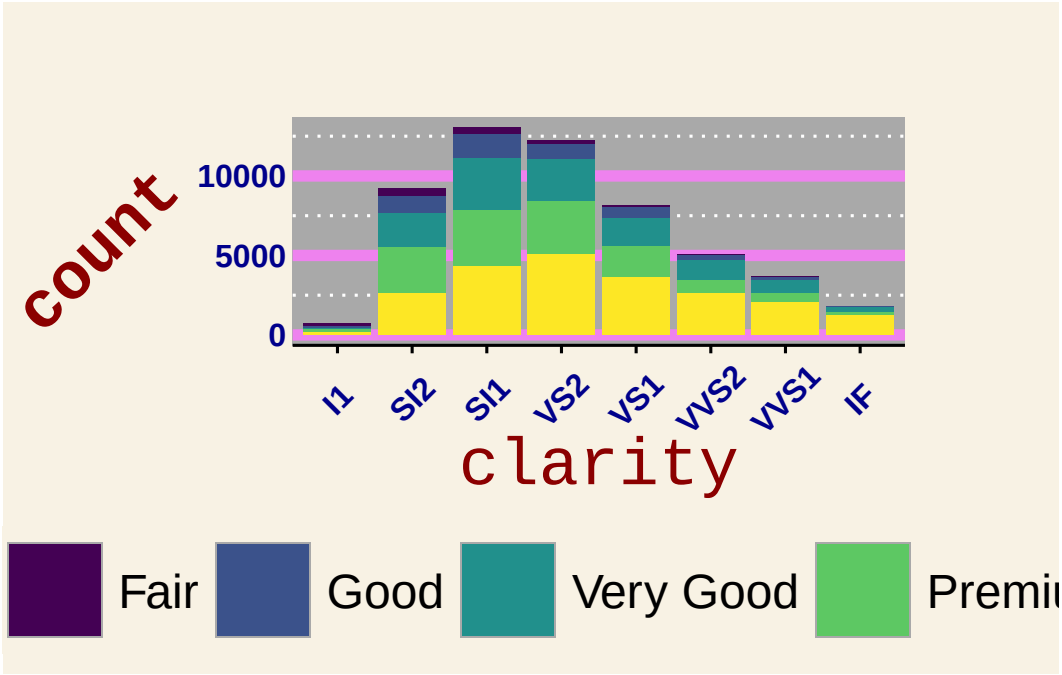
There is an informative blog post Styling plots that is a good starter.

```
old <- theme_set(theme_wsj())
ggplot(data = diamonds, aes(x = clarity, fill = cut)) +
  geom_bar()
```

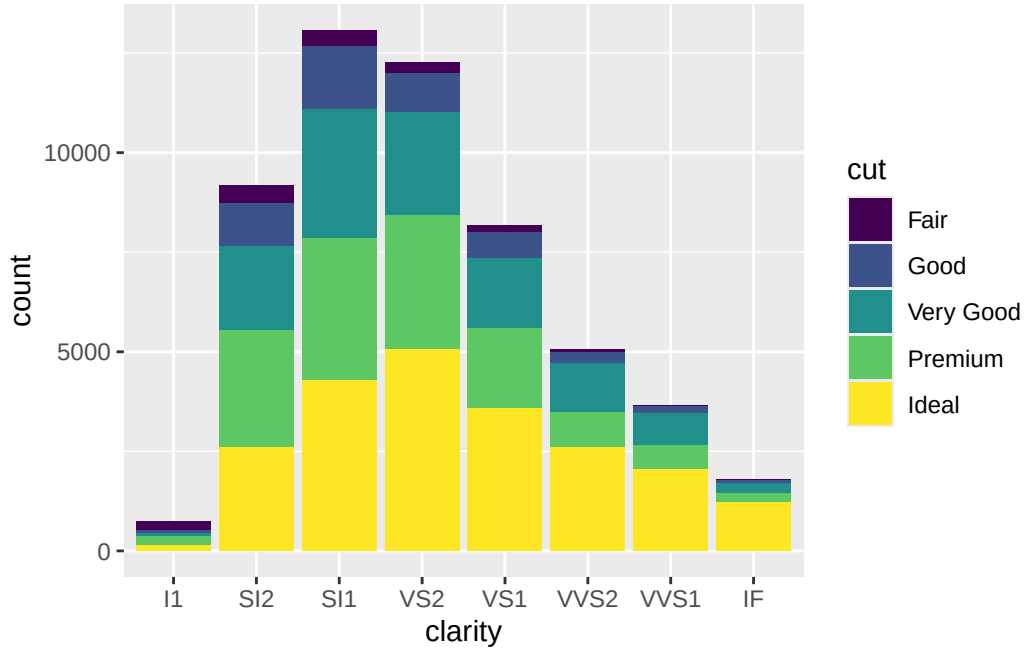



```
theme_update(
  legend.position = "bottom",
  axis.text = element_text(
    colour = "darkblue",
    size = 12
  ),
  axis.text.x = element_text(
    vjust = 0.5, angle = 45,
    family = "sans",
    face = "bold"
  ),
  axis.title = element_text(
    size = 25,
    color = "darkred"
  ),
  plot.margin = unit(c(3, 4, 0.5, 0.3), "lines"), # N,E,S,W
  axis.title.y = element_text(
    vjust = 0.4, angle = 45,
    face = "bold"
  ),
  legend.key.size = unit(2.5, "lines"),
  panel.background = element_rect(fill = "darkgrey"),
  panel.grid.minor = element_line(colour = "white"),
```

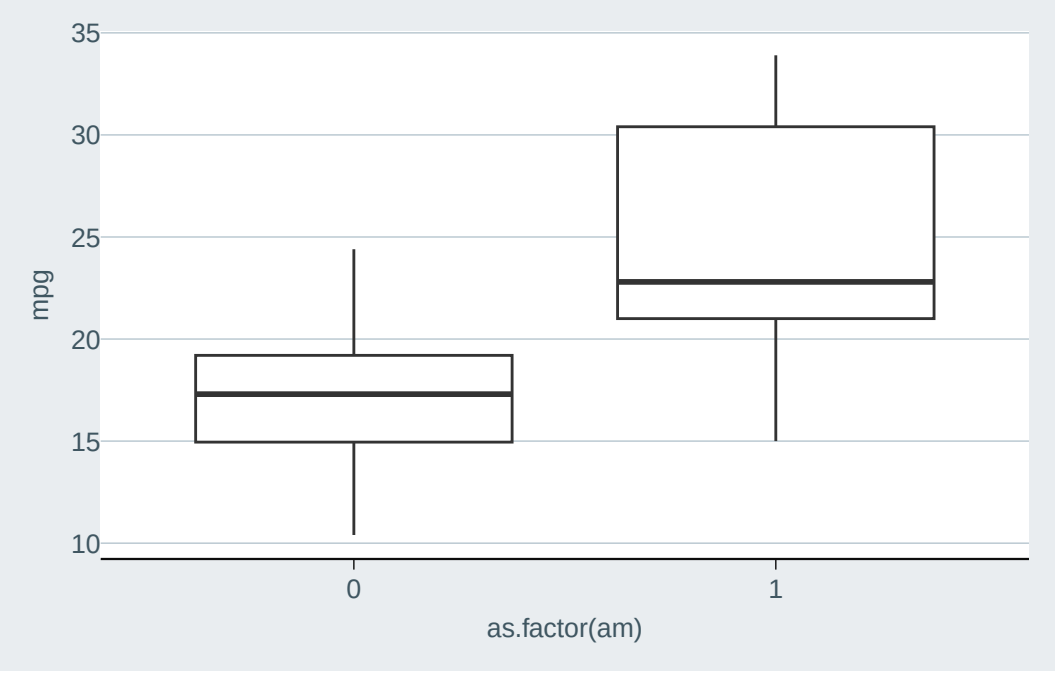
```
  panel.grid.major = element_line(
    linetype = 1,
    color = "violet", linewidth = 2
  ),
  legend.text = element_text(size = 18),
  legend.title = element_text(size = 30, color = "pink")
)
ggplot(data = diamonds, aes(x = clarity, fill = cut)) +
  geom_bar()
```




```
theme_set(theme_grey())
# theme_set(old)
ggplot(data = diamonds, aes(x = clarity, fill = cut)) +
  geom_bar()
```

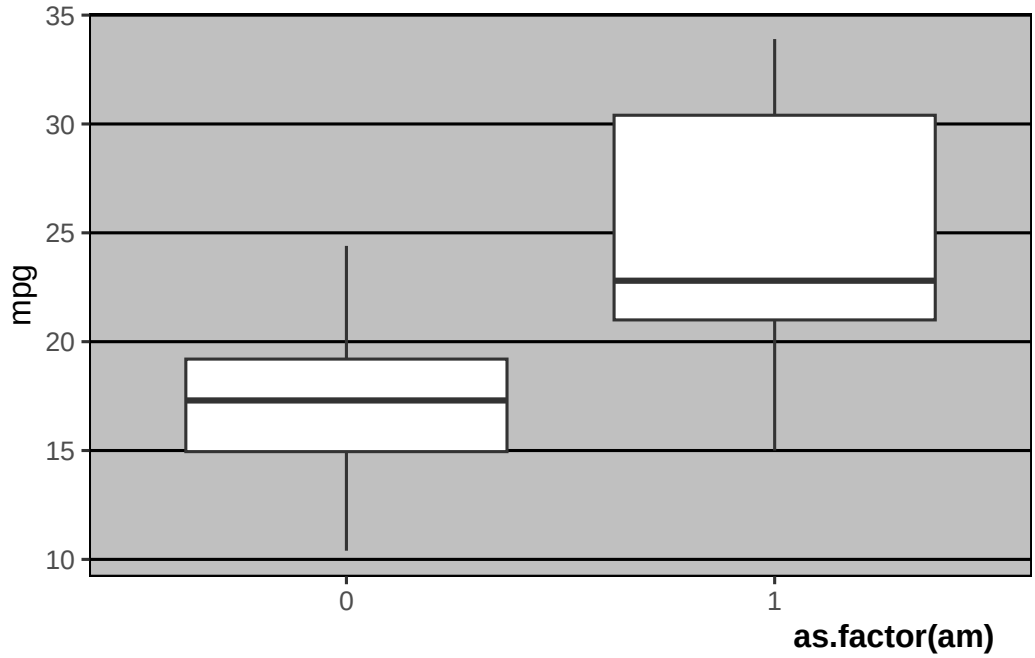



```
# ggthemes ####
plottemp + theme_economist()
```

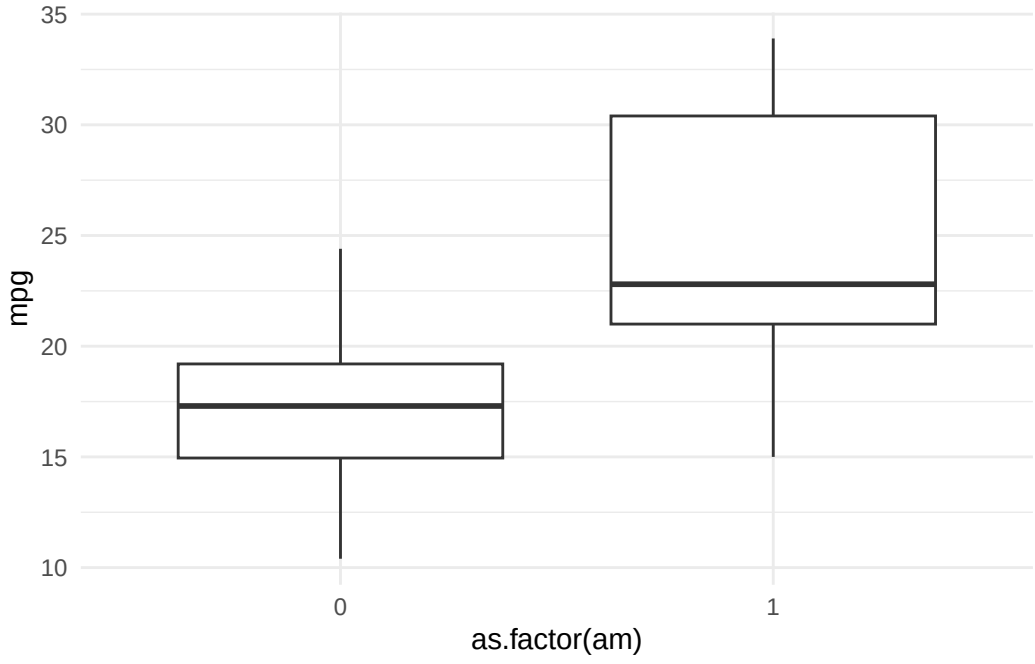



```
plottemp + theme_excel() +
```

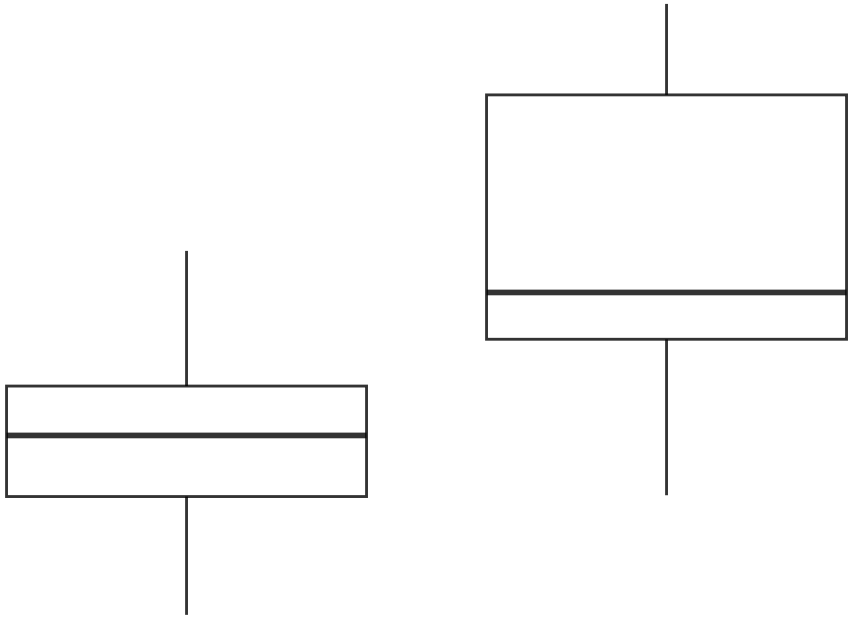
```
theme(axis.title.x = element_text(face = "bold", hjust = 0.95))
```



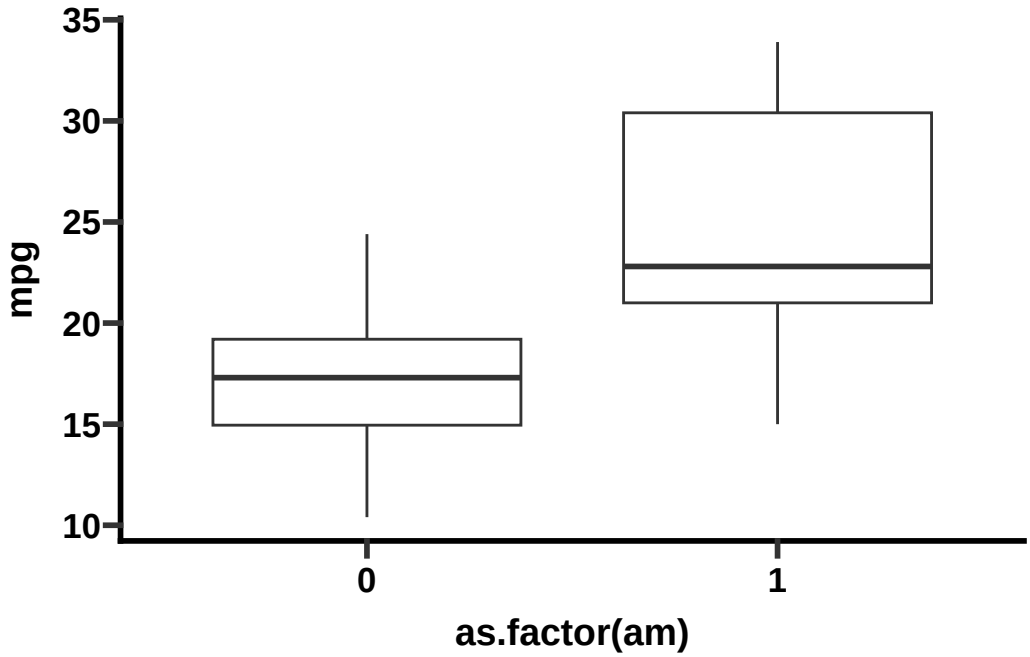

```
plottemp + theme_minimal()
```



```
plottemp + theme_void()
```



```
plottemp+theme_prism()
```



```
## install from Github
# devtools::install_github("jespermaag/gganatogram")
# gganatogram::gganatogram(
#   organism = "human", fill = "color",
#   data = tibble(organ = c("nerve", "brain"),
#  type = c("nervous system", "nervous system"),
#  colour = c("purple", "orange"),
#  value = c(1, 1))) +
#   theme_void()
# plottemp+theme_base()
# plottemp+theme_bw() +
#   theme(panel.grid.major.x = element_blank())

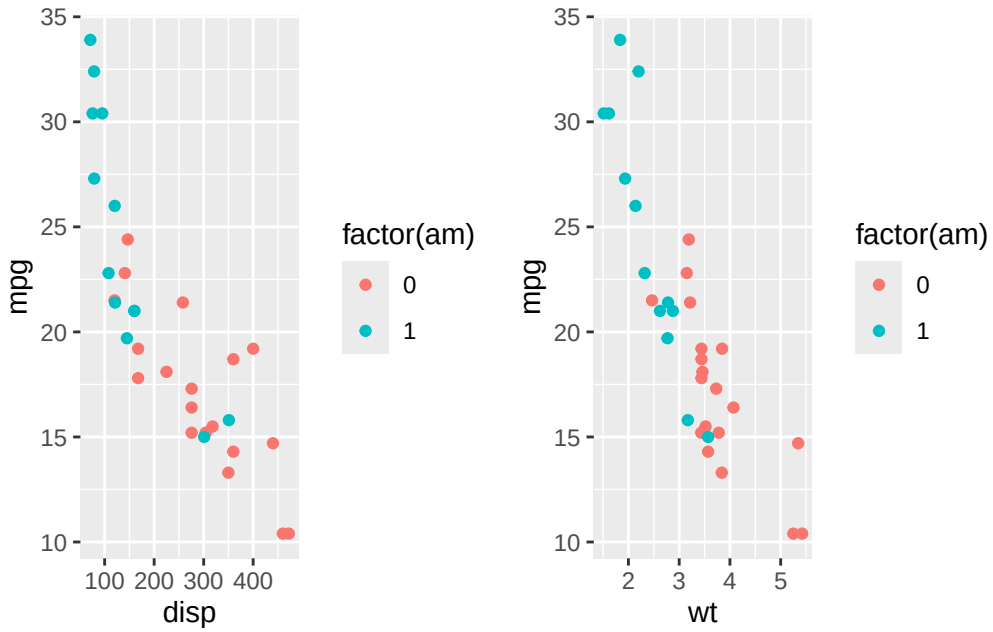
# https://www.data-imaginist.com/2019/a-flurry-of-facets/

# https://github.com/thomasp85/gganimate
```

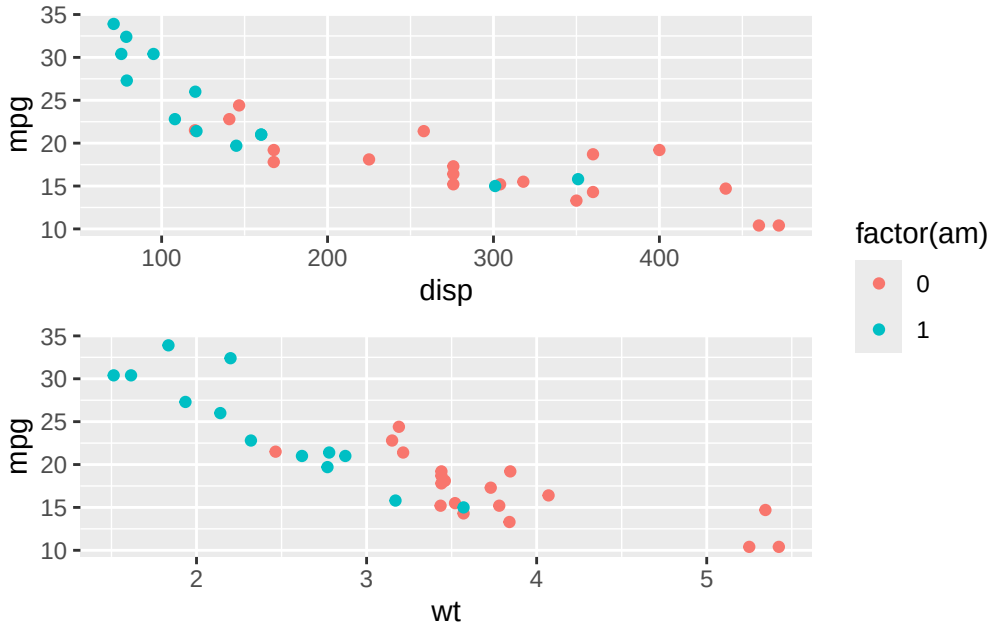
7.17 Combining figures with patchwork

More details on github

```
p1 <- ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(am)))
p2 <- ggplot(mtcars) +
  geom_point(aes(wt, mpg, color = factor(am)))
p1 | p2
```




```
(p1 / p2) +
  plot_layout(guides = "collect")
```

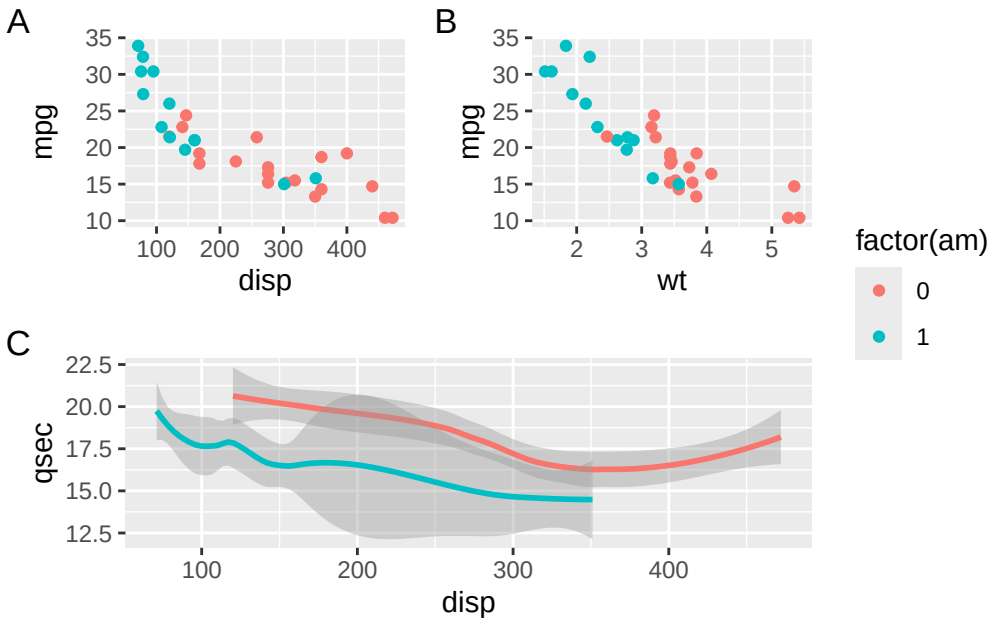



```
p3 <- ggplot(mtcars) +
```

```
  geom_smooth(aes(disp, qsec, color = factor(am)))
# p4 <- ggplot(mtcars) +
#   geom_bar(aes(factor(carb), fill = factor(am)))

(p1 | p2) /
  (p3 + guides(color = "none")) +
  plot_annotation(tag_levels = "A") +
  plot_layout(guides = "collect")
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

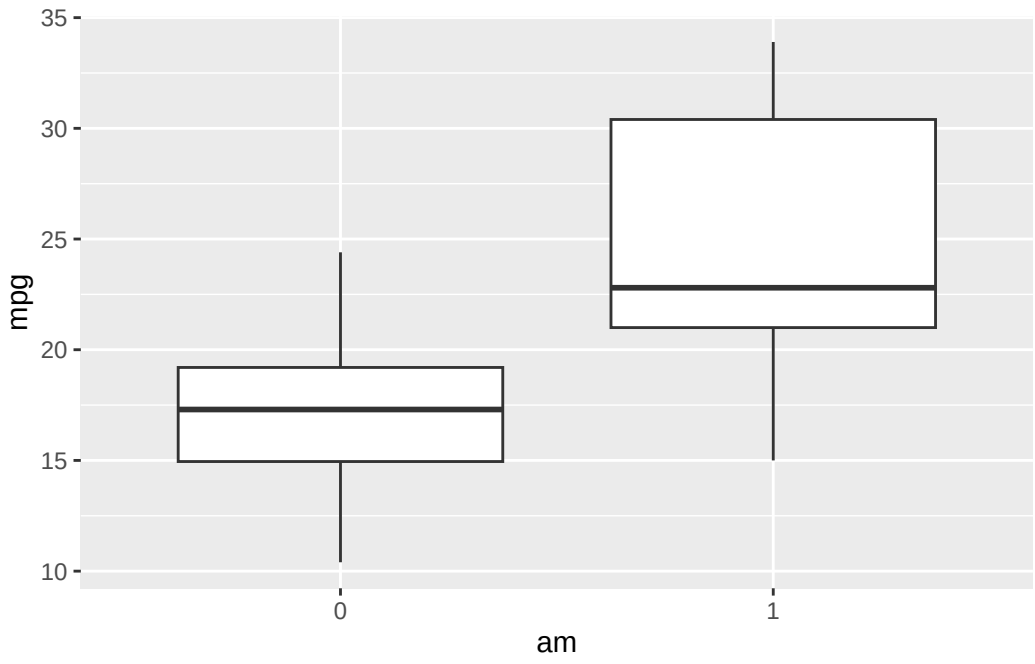


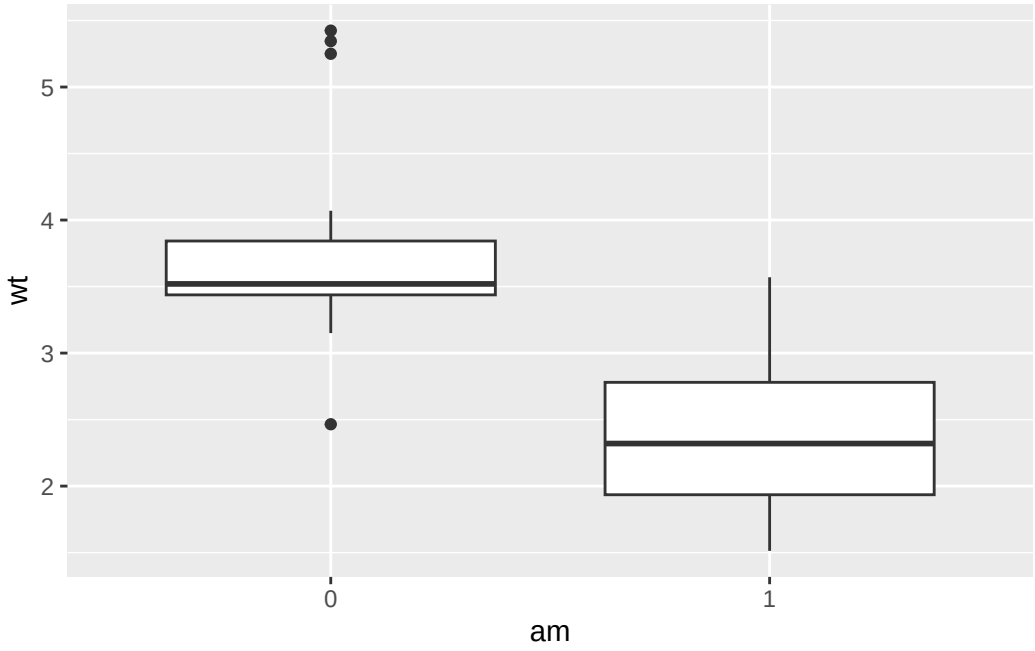
7.18 ggplots in loops

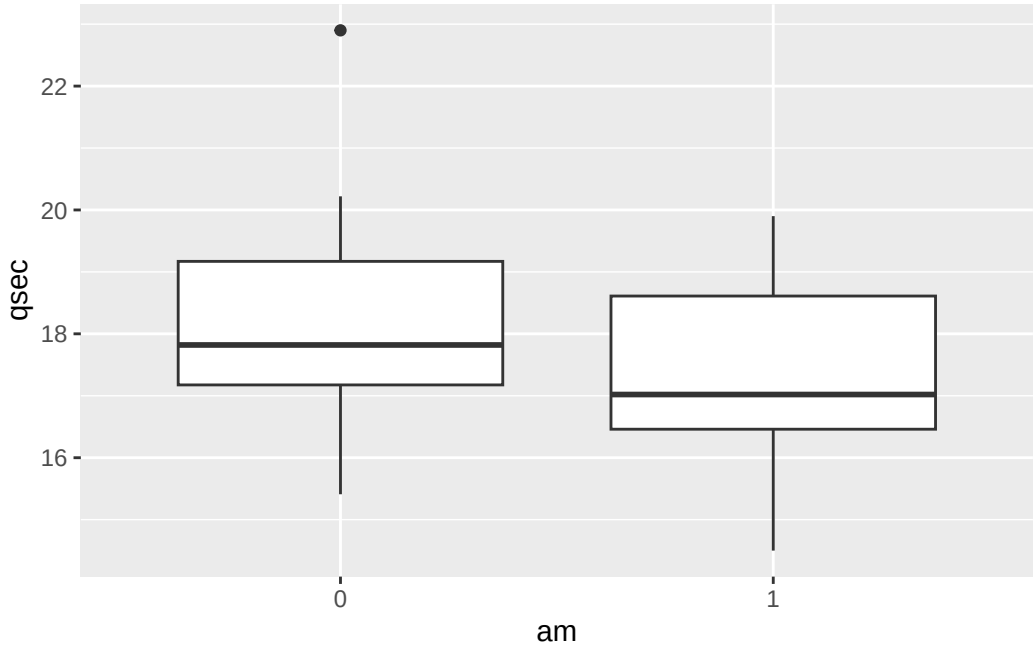
inside aes(), ggplot expects variables referring to columns in the data. In a loop, the loop index is often a character representation of a column name. So inside aes(), there is a variable containing a variable name. To make that work, we use .data as a reference to the data ggplot is working on, and the loop index inside double square brackets as its index:

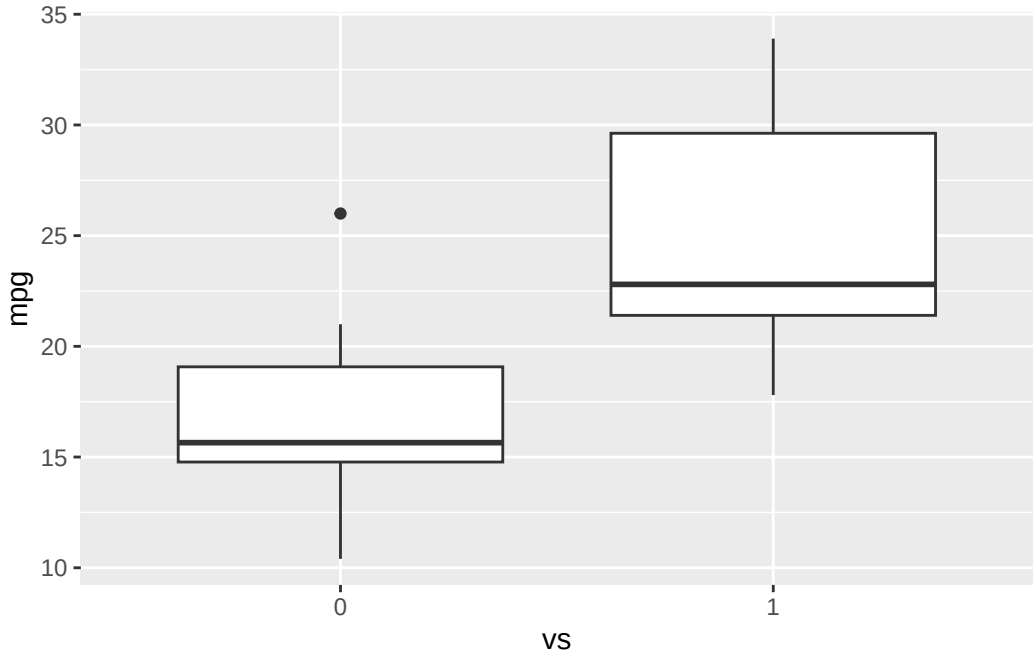
```
for (group_i in c("am", "vs")) {
  for (var_i in c("mpg", "wt", "qsec")) {
    plot_temp <-
```

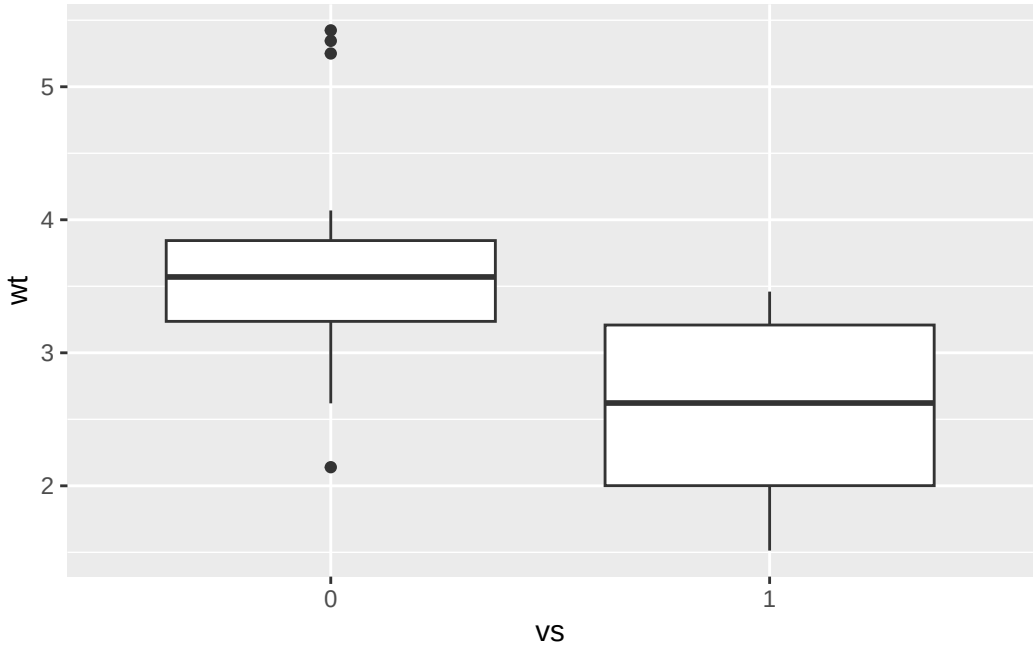
```
      mutate(mtcars,
        am = factor(am),
        vs = factor(vs)
      ) |>
      ggplot(aes(x = .data[[group_i]], y = .data[[var_i]])) +
      geom_boxplot()
    print(plot_temp)
  }
}
```

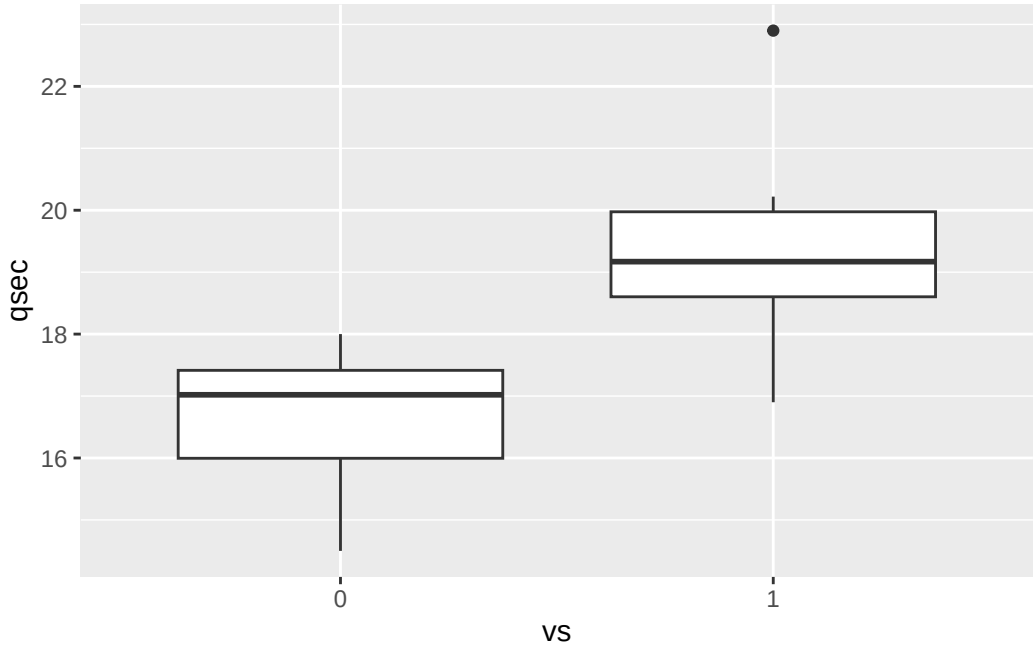



wt
5
4
3
2
0
1
am


qsec
22
20
18
16
0
1
am


35
30
25
20
15
10
mpg
0
1
vs


5
4
3
2
wt
0
1
vs

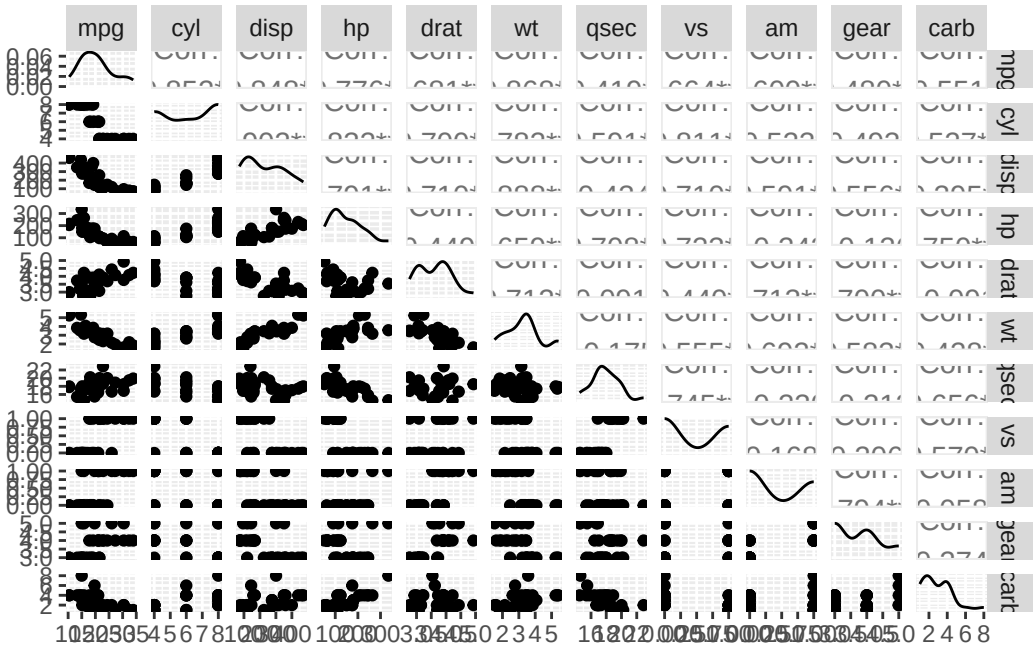


The function aes_string(), which expected a character representation instead of a variable name, is deprecated, but may still be suggested by chat-bots

more facets...

animations...

```
GGally::ggpairs(mtcars)
```



Colorblinder

8 Grouping of variables by type / distribution / use

```
pacman::p_load(conflicted, wrappedtools, tidyverse, here, ggh4x)
conflicts_prefer(dplyr::filter)
rawdata <- readRDS(here("data/rawdata.rds"))
```

8.1 Test for Normal distribution

8.1.1 Testing a single variable

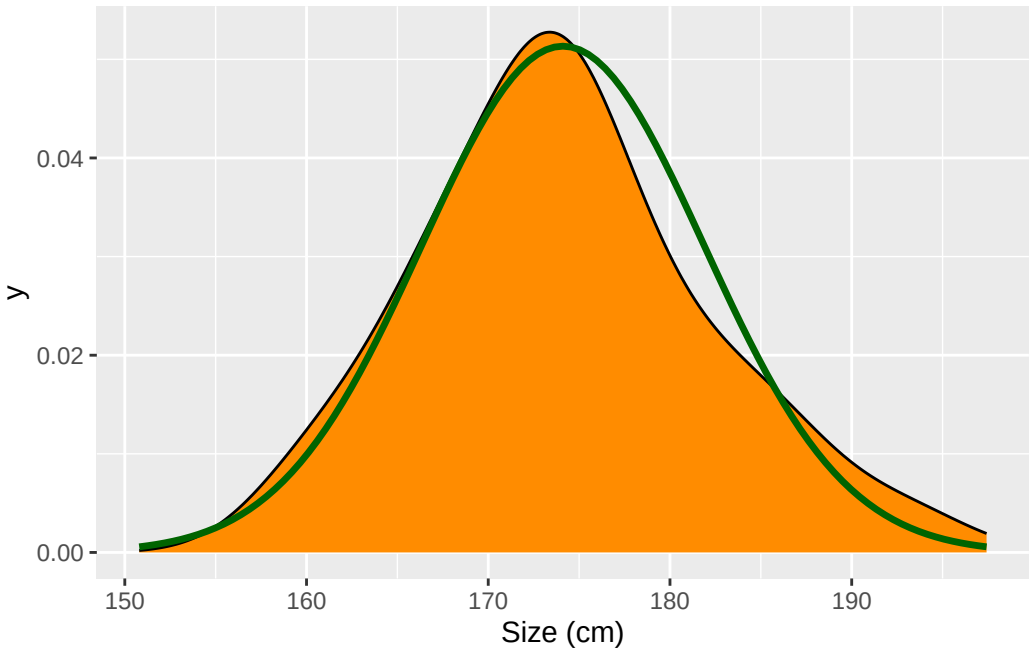
Before computing some test-statistics, a graphical exploration should be done by e.g. density plots.

There are a number of tests for Normal distribution, all testing the Null hypothesis of data coming from a population with Normal distribution. So small p-values lead to rejection of the Null and indicate deviation from normality. Kolmogorov-Smirnov-test (for larger sample sizes) and Shapiro-Wilk-test (for smaller samples) will be used as examples.

Other tests would be e.g. Anderson-Darling, and the Cramer-von Mises test, see package `nortest`.

```
ggplot(rawdata, aes(`Size (cm)`)) +
  geom_density(fill = "darkorange") + # the actual density plot
  stat_function( #theoretical distribution as comparison
    fun = dnorm,
    args = list(
      mean = mean(rawdata$`Size (cm)`),
      sd = sd(rawdata$`Size (cm)`)
    ),
    color = "darkgreen",
    linewidth = 1.2
  ) +
  scale_x_continuous( # cosmetics on axis
    limits = c(
      mean(rawdata$`Size (cm)`) - 3 * sd(rawdata$`Size (cm)`),
```

```
      mean(rawdata$`Size (cm)`) + 3 * sd(rawdata$`Size (cm)`)
    )
  )
```




```
ks.test(
  x = rawdata$`Size (cm)`,
  "pnorm",
  mean = mean(rawdata$`Size (cm)`, # essential!
    na.rm = TRUE
  ),
  sd = sd(rawdata$`Size (cm)`,
    na.rm = TRUE
  )
)
```

```
Warning in ks.test.default(x = rawdata$`Size (cm)`, "pnorm", mean =
mean(rawdata$`Size (cm)`, : ties should not be present for the one-sample
Kolmogorov-Smirnov test
```

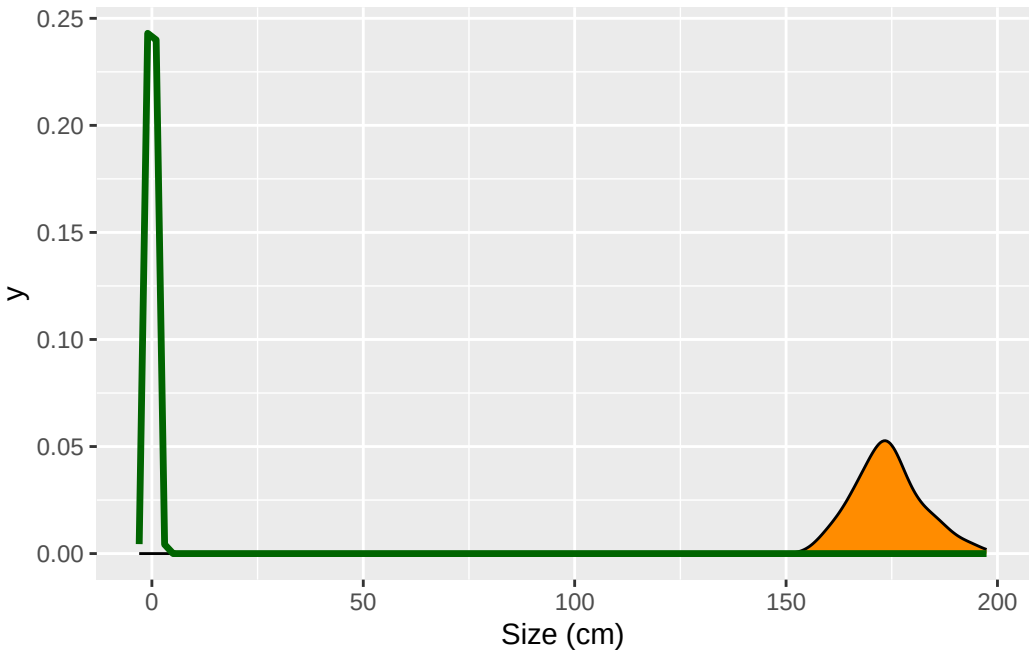
```
    Asymptotic one-sample Kolmogorov-Smirnov test

data:  rawdata$`Size (cm)`
```

```
D = 0.13284, p-value = 0.7064
alternative hypothesis: two-sided
```

If calling ks.test with defaults: this is what you'd be testing:

```
ggplot(rawdata, aes(`Size (cm)`)) +
  geom_density(fill = "darkorange") +
  stat_function(
    fun = dnorm,
    args = list(
      mean = 0,#mean(rawdata$`Size (cm)`),
      sd = 1#sd(rawdata$`Size (cm)`)
    ),
    color = "darkgreen",
    linewidth = 1.2
  ) +
  scale_x_continuous(
    limits = c(
      -3,#mean(rawdata$`Size (cm)`) - 3 * sd(rawdata$`Size (cm)`),
      mean(rawdata$`Size (cm)`) + 3 * sd(rawdata$`Size (cm)`)
    )
  )
```




```
ksnormal(rawdata$`Size (cm)`, lillie = FALSE)
```

```
p_Normal_KS
  0.7063825
```

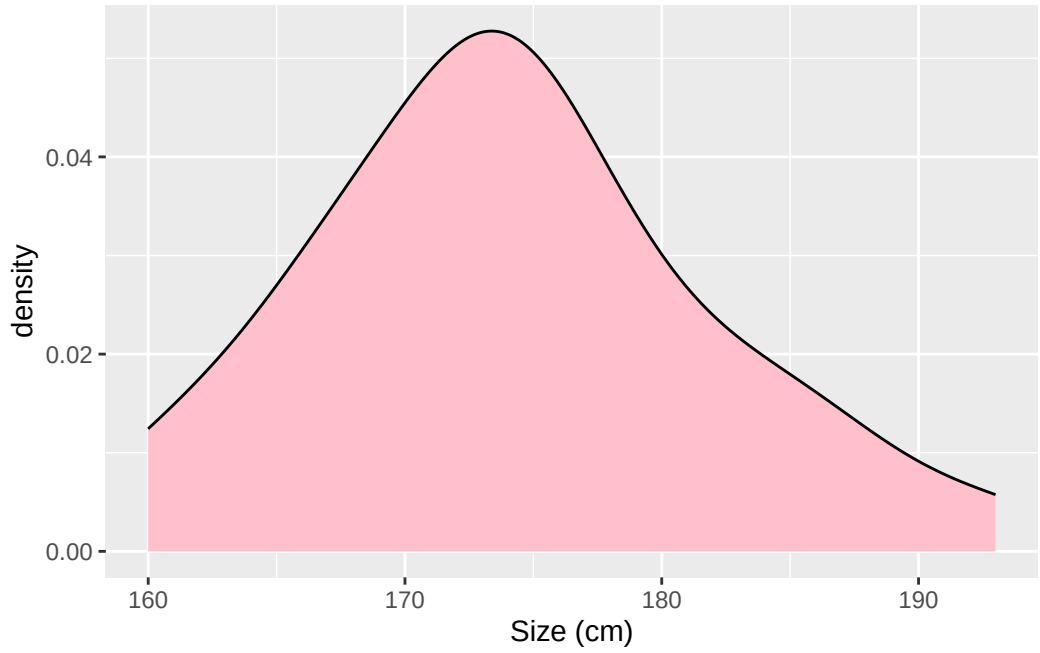
```
shapiro.test(rawdata$`Size (cm)`)
```

```

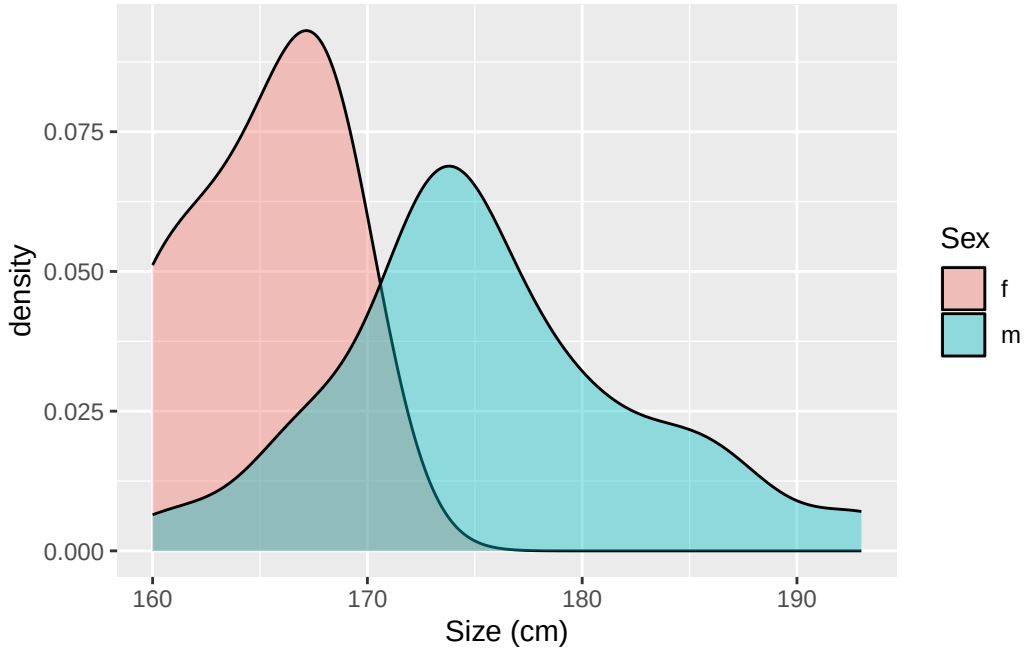
    Shapiro-Wilk normality test

data:  rawdata$`Size (cm)`
W = 0.9766, p-value = 0.7627
```

```
ggplot(rawdata, aes(x = `Size (cm)`)) +
  geom_density(fill = "pink")
```

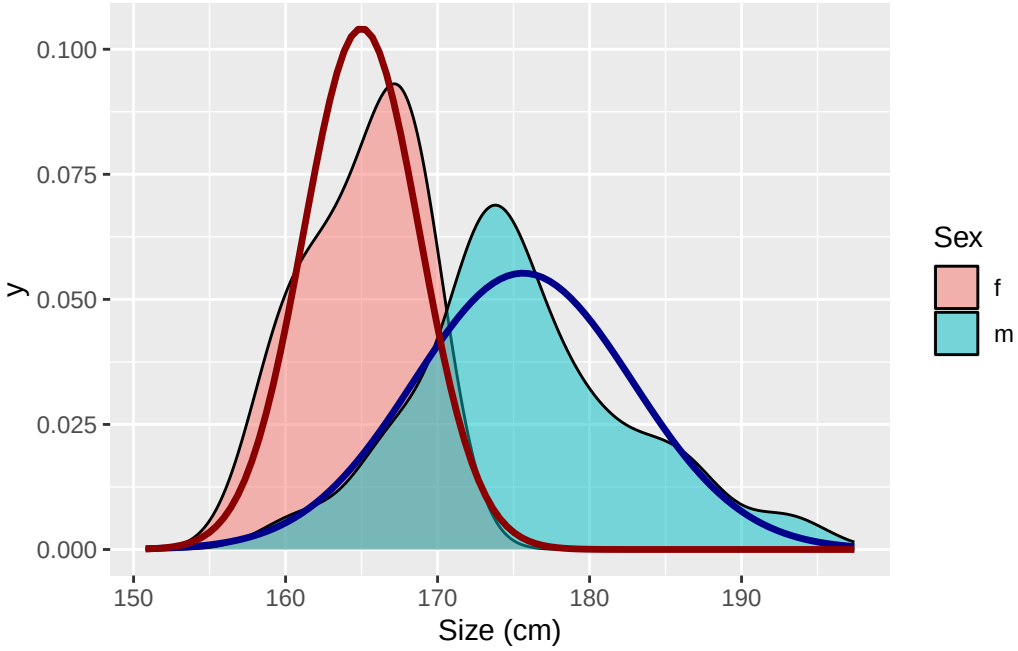



```
ggplot(rawdata, aes(x = `Size (cm)`, fill = Sex)) +
  geom_density(alpha = .4)
```

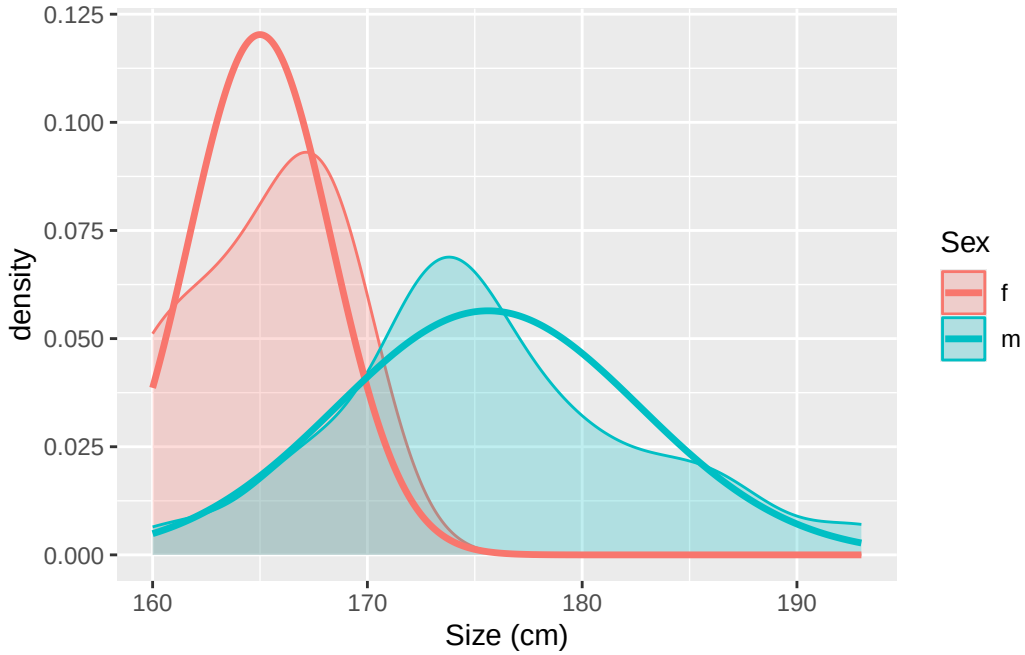



```
ggplot(rawdata, aes(`Size (cm)`)) +
  geom_density(aes(fill = Sex), alpha = .5) +
  stat_function(
    fun = dnorm,
    args = list(
      mean = mean(rawdata |>
                    filter(Sex == "m") |>
                    pull(`Size (cm)`)),
      sd = sd(rawdata |>
                filter(Sex == "m") |>
                pull(`Size (cm)`))
    ),
    color = "darkblue",
    linewidth = 1.2
  ) +
  stat_function(
    fun = dnorm,
    args = list(
      mean = mean(rawdata |>
                    filter(Sex == "f") |>
                    pull(`Size (cm)`)),
      sd = sd(rawdata |>
                filter(Sex == "f") |>
```

```
              pull(`Size (cm)`))
    ),
    color = "darkred",
    linewidth = 1.2
  ) +
  scale_x_continuous(
    limits = c(
      mean(rawdata$`Size (cm)`) - 3 * sd(rawdata$`Size (cm)`),
      mean(rawdata$`Size (cm)`) + 3 * sd(rawdata$`Size (cm)`)
    )
  )
```




```
# More efficient: ggh4x::stat_theodensity() fits mean/sd per group
# automatically (via fitdistrplus), so no manual filter()/pull() per group
# and no hard-coded +/-3 SD limits needed.
pacman::p_load(ggh4x)
ggplot(rawdata, aes(`Size (cm)`, colour = Sex, fill = Sex)) +
  geom_density(alpha = .25) +
  stat_theodensity(distri = "norm", linewidth = 1.2)
```



If severe group difference can be expected (case/control, sex …), exploration and analyses should be done in subgroups.

```
rawdata |>
  filter(Sex == "m") |>
  pull(`Size (cm)`) |>
  ksnormal()
```

```
p_Normal_Lilliefors
          0.1180981
```

```
rawdata |>
  group_by(Sex) |>
  summarise(
    n = n(),
    p_KS = ksnormal(`Size (cm)`, lillie = FALSE),
    p_KSlilli = ksnormal(`Size (cm)`),
    `pGauss (Shapiro)` = shapiro.test(`Size (cm)`) |>
      pluck("p.value")
  )
```

```
# A tibble: 2 x 5
  Sex       n  p_KS p_KSlilli `pGauss (Shapiro)`
```

```
  <chr> <int> <dbl>     <dbl>              <dbl>
1 f         4 0.905     0.905              0.272
2 m        24 0.575     0.118              0.638
```

8.1.2 Testing several variables

To explore larger data sets, it may be useful to test all numerical variables for normality, this can be done in a loop or with the across-function. As a start for the loop-solution we can get the names and positions for all (or selected) numerical variables with the ColSeeker-function from wrappedtools.

```
numvars <-
  ColSeeker(
    data = rawdata, # can be omitted, as it is the default
    varclass = c("numeric", "integer")
  )
numvars$index
```

```
 [1]  1  2  3  4  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
```

```
head(numvars$names)
```

```
[1] "Randomcode"     "Included"       "Finalized"      "Testmedication"
[5] "Size (cm)"      "Weight (kg)"
```

```
numvars$count
```

```
[1] 23
```

Loops can be created with either a numeric counter-like index or content-based index.

Loop Version 1:

```
## result table v1, pre-filled
resulttable1 <- tibble(
  Variables = numvars$names,
  pKS = NA_real_,
  pSh = NA_real_
)
## loop version 1
for (var_i in seq_len(numvars$count)) {
  resulttable1$pKS[var_i] <-
```

```
    ksnormal(rawdata[[numvars$names[var_i]]])
  resulttable1$pSh[var_i] <-
    shapiro.test(rawdata |>
      pull(numvars$names[var_i])) |>
    pluck("p.value")
}
head(resulttable1)
```

```
# A tibble: 6 x 3
  Variables           pKS      pSh
  <chr>             <dbl>    <dbl>
1 Randomcode     9.74e- 1 3.10e- 1
2 Included       4.88e-24 2.25e-11
3 Finalized      1.25e-21 1.86e- 9
4 Testmedication 6.96e- 9 4.31e- 7
5 Size (cm)      2.33e- 1 7.63e- 1
6 Weight (kg)    2.98e- 1 8.96e- 2
```

```
resulttable1 |>
  mutate(
    pKS = formatP(pKS, ndigits = 5),
    pSh = formatP(pSh, mark = TRUE)
  )
```

```
# A tibble: 23 x 3
   Variables      pKS     pSh
   <chr>          <chr>   <chr>
 1 Randomcode     0.97369 0.310 n.s.
 2 Included       0.00001 0.001 ***
 3 Finalized      0.00001 0.001 ***
 4 Testmedication 0.00001 0.001 ***
 5 Size (cm)      0.23305 0.763 n.s.
 6 Weight (kg)    0.29772 0.090 +
 7 sysBP V0       0.46905 0.256 n.s.
 8 diaBP V0       0.11863 0.095 +
 9 Lv Edv Mri     0.25374 0.167 n.s.
10 Lv Esv Mri     0.00288 0.001 ***
# i 13 more rows
```

```
resulttable1 |>
  mutate(across(
    .cols = starts_with("p"),
```

```
    # .fns = formatP
    # .fns = ~ formatP(.x, ndigits = 4, mark = TRUE)
    .fns = \(x) formatP(x,
      ndigits = 4,
      mark = TRUE
    )
  ))
```

```
# A tibble: 23 x 3
   Variables      pKS         pSh
   <chr>          <chr>       <chr>
 1 Randomcode     0.9737 n.s. 0.3095 n.s.
 2 Included       0.0001 ***  0.0001 ***
 3 Finalized      0.0001 ***  0.0001 ***
 4 Testmedication 0.0001 ***  0.0001 ***
 5 Size (cm)      0.2331 n.s. 0.7627 n.s.
 6 Weight (kg)    0.2977 n.s. 0.0896 +
 7 sysBP V0       0.4690 n.s. 0.2558 n.s.
 8 diaBP V0       0.1186 n.s. 0.0949 +
 9 Lv Edv Mri     0.2537 n.s. 0.1669 n.s.
10 Lv Esv Mri     0.0029 **   0.0004 ***
# i 13 more rows
```

Side note on anonymous functions:

`mean_function <- function(x) mean(x, na.rm = TRUE)`
would create a function `mean_function` with argument `x`.

`\()` is just a shorthand for the same thing
`\(x) mean(x, na.rm = TRUE)`
creates an unnamed / anonymous function. Anonymous functions let you define a quick, one-off transformation inline — without naming it. They're most useful inside across(), where you want to apply something to multiple columns at once.
`across()` iterates over selected columns and applies a function to each. When you just need mean or sum, you may pass the function directly. But when you need extra arguments, an anonymous function wraps it.
The `\(x)` form is now preferred over ~mean(.x,…) because it's plain R, works without loading purrr, and makes the argument explicit — which avoids confusion when functions have multiple arguments.

Loop Version 2:

```
## result table v2, just structure
resulttable2 <- tibble(
```

```
  Measures = NA_character_,
  pKS_Placebo = NA_character_,
  pKS_Verum = NA_character_,
  .rows = 0
)
for (var_i in numvars$names) {
  ks_tmp <-
    rawdata |>
    group_by(Testmedication) |>
    summarise(pKS = ksnormal(.data[[var_i]],
                             lillie = FALSE))
  resulttable2 <- add_row(resulttable2,
    Measures = var_i,
    pKS_Placebo = ks_tmp$pKS[1] |>
      formatP(mark = TRUE), # added rounding/formatting
    pKS_Verum = ks_tmp$pKS[2] |>
      formatP(mark = TRUE)
  )
}
head(resulttable2)
```

```
# A tibble: 6 x 3
  Measures       pKS_Placebo pKS_Verum
  <chr>          <chr>       <chr>
1 Randomcode     0.994 n.s.  0.996 n.s.
2 Included       0.001 ***   0.001 ***
3 Finalized      0.003 **    0.001 ***
4 Testmedication 0.001 ***   0.001 ***
5 Size (cm)      0.955 n.s.  0.964 n.s.
6 Weight (kg)    0.553 n.s.  0.793 n.s.
```

across() - Version:

`across()` in R (from `dplyr`) is a powerful tool that lets you apply the same operation to multiple columns in your data frame, similar to using loops or list comprehensions with Pandas DataFrames in Python. `across()` (sort of) loops over variables / columns and applies function(s), it can be used inside summarize, mutate, filter.

```
resulttable1a <-
  rawdata |>
  summarise(across(
    .cols = # where(is.numeric),
      all_of(numvars$names[-(1:4)]),
```

```
    .fns = list(
      pKS = \(x) ksnormal(x) |>
        formatP(mark = TRUE),
      pSh = \(x) shapiro.test(x) |>
        pluck("p.value") |>
        formatP(mark = TRUE)
    )
  )) |>
  # change output structure to long form
  pivot_longer(
    cols = everything(),
    names_to = c("Variable", ".value"),
    names_sep = "_"
  )
# pivot_longer(everything(),
#              names_to = c("Variable","test"), #.variable
#              names_sep = "_") |>
# pivot_wider(names_from = test, values_from = value)
head(resulttable1a)
```

```
# A tibble: 6 x 3
  Variable    pKS        pSh
  <chr>       <chr>      <chr>
1 Size (cm)   0.233 n.s. 0.763 n.s.
2 Weight (kg) 0.298 n.s. 0.090 +
3 sysBP V0    0.469 n.s. 0.256 n.s.
4 diaBP V0    0.119 n.s. 0.095 +
5 Lv Edv Mri  0.254 n.s. 0.167 n.s.
6 Lv Esv Mri  0.003 **   0.001 ***
```

```
head(resulttable1)
```

```
# A tibble: 6 x 3
  Variables           pKS      pSh
  <chr>             <dbl>    <dbl>
1 Randomcode     9.74e- 1 3.10e- 1
2 Included       4.88e-24 2.25e-11
3 Finalized      1.25e-21 1.86e- 9
4 Testmedication 6.96e- 9 4.31e- 7
5 Size (cm)      2.33e- 1 7.63e- 1
6 Weight (kg)    2.98e- 1 8.96e- 2
```

```r
resulttable2a <-
  rawdata |>
  mutate(Testmedication = factor(Testmedication,
    levels = c(0, 1),
    labels = c("Placebo", "Verum")
  )) |>
  # mutate(Testmedication = case_match(Testmedication,
  #                                    0~"Placebo",
  #                                    1~"Verum")) |>
  group_by(Testmedication) |>
  summarise(across(all_of(numvars$names[-(1:4)]),
    .fns = \(x) ksnormal(x, lillie = FALSE) |> # single function
      formatP(mark = TRUE)
  )) |>
  pivot_longer( # cosmetic changes to put vars in rows
    cols = -Testmedication,
    names_to = "Measure"
  ) |>
  pivot_wider( # and grups in columns
    names_from = Testmedication,
    values_from = value
  )
head(resulttable2a)
```

```
# A tibble: 6 x 3
  Measure     Placebo    Verum
  <chr>       <chr>      <chr>
1 Size (cm)   0.955 n.s. 0.964 n.s.
2 Weight (kg) 0.553 n.s. 0.793 n.s.
3 sysBP V0    0.555 n.s. 0.774 n.s.
4 diaBP V0    0.647 n.s. 0.942 n.s.
5 Lv Edv Mri  1.000 n.s. 0.880 n.s.
6 Lv Esv Mri  0.450 n.s. 0.236 n.s.
```

```r
head(resulttable2)
```

```
# A tibble: 6 x 3
  Measures       pKS_Placebo pKS_Verum
  <chr>          <chr>       <chr>
1 Randomcode     0.994 n.s.  0.996 n.s.
2 Included       0.001 ***   0.001 ***
3 Finalized      0.003 **    0.001 ***
```

```
4 Testmedication 0.001 ***   0.001 ***
5 Size (cm)      0.955 n.s.  0.964 n.s.
6 Weight (kg)    0.553 n.s.  0.793 n.s.
```

```
resulttable3 <-
  rawdata |>
  group_by(Testmedication) |>
  summarise(across(all_of(numvars$names[-(1:4)]),
    .fns = list( # list of functions
      Mean = \(x) mean(x, na.rm = TRUE) |>
        roundR(5),
      Median = \(x) median(x, na.rm = TRUE) |>
        roundR(5),
      pKS = \(x) ksnormal(x) |>
        formatP(mark = TRUE),
      pSh = \(x) shapiro.test(x) |>
        pluck("p.value") |>
        formatP(mark = TRUE)
    )
  )) |>
  pivot_longer(-Testmedication,
    names_to = c("Variable", "test"), # Variable,.value
    names_sep = "_"
  ) |>
  pivot_wider(names_from = test, values_from = value) |>
  arrange(Variable)
head(resulttable3)
```

```
# A tibble: 6 x 6
  Testmedication Variable     Mean   Median pKS        pSh
           <dbl> <chr>        <chr>  <chr>  <chr>      <chr>
1              0 Age          60.429 64.000 0.057 +    0.425 n.s.
2              1 Age          60.429 60.000 0.604 n.s. 0.282 n.s.
3              0 BMI          30.244 29.246 0.680 n.s. 0.430 n.s.
4              1 BMI          28.016 27.484 0.880 n.s. 0.862 n.s.
5              0 Ferritin Lab 258.21 220.00 0.112 n.s. 0.176 n.s.
6              1 Ferritin Lab 305.23 222.00 0.001 ***  0.003 **
```

```
rm(numvars)
```

8.2 Exercise: Distribution of penguin measures

- Using the penguin data, testing for which numerical measures a test for a ***gaussian*** distribution is meaningful
- For those measures, test Normality in the total sample as well as for subgroups defined by species and sex

    1. single measure, all penguins, plot and test
    2. single measure, by species/sex, plot and test
    3. all measures within species, plot and test
    4. all measures within species and sex, plot and test

8.3 Picking column names and positions

Based on data inspection, testing, and background knowledge, variables can be sorted into scale levels:

```
gaussvars <- ColSeeker(
  data = rawdata, # can be omitted, as it is the default
  namepattern = c("si", "we", "BMI", "BP", "mri"),
  casesensitive = FALSE
)

ordvars <- ColSeeker(
  data = rawdata,
  namepattern = c("Age", "Lab")
)

factvars <- ColSeeker(
  data = rawdata,
  namepattern = c("Sex", "med", "NYHA"),
  returnclass = TRUE
)

rawdata <- mutate(
  rawdata,
  across(
    all_of(factvars$names),
    \(x) factor(x)
```

```
  )
)
```

To make data accessible for other scripts, data will be saved as a binary memory dump:

```
save(rawdata,
  list = ls(pattern = "vars"),
  file = here("data/bookdata1.RData")
)
```

9 Descriptive statistics

Descriptive statistics are used to summarize and organize data in a manner that is meaningful and useful. They provide simple summaries about the sample and the measures, such as mean, median, standard deviation, or frequencies. Furthermore, they allow for the presentation of quantitative descriptions in a manageable form, aiding in understanding the data distribution and central tendency. For group comparisons, they will inform about direction and magnitude of differences.

9.1 Typical descriptives

- *mean() / sd() / meansd()*
- *median() / quantile() / median_quart()*
- *table() / prop.table() / cat_desc_stats()*

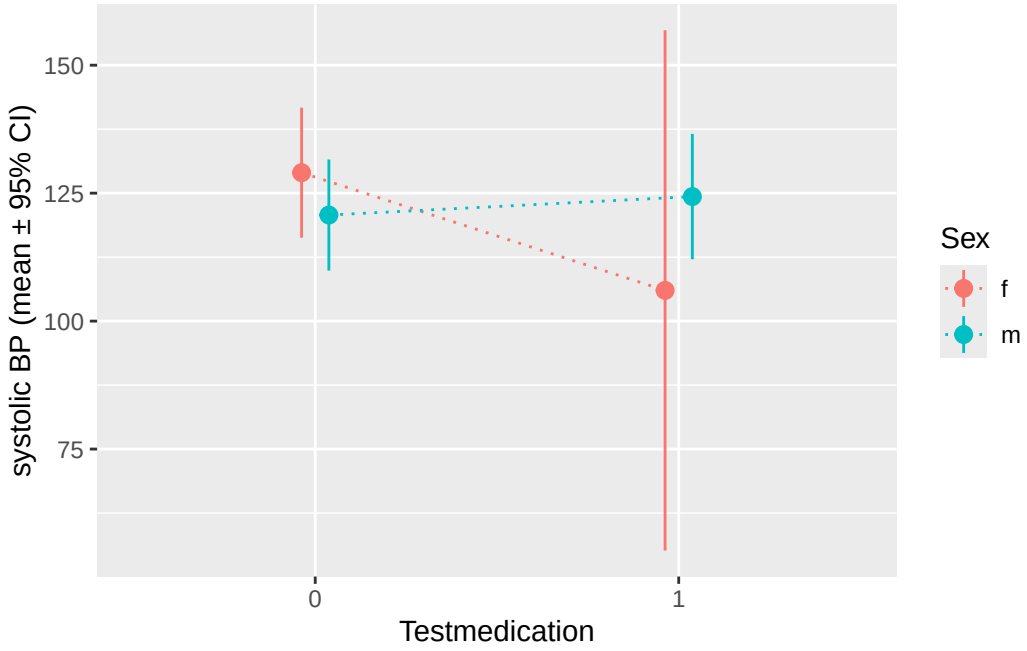
9.2 Reading in data

```
pacman::p_load(
  conflicted, tidyverse, wrappedtools, here,
  flextable
)
set_flextable_defaults(
  font.size = 9,
  padding.bottom = 1,
  padding.top = 3,
  padding.left = 3,
  padding.right = 4
)
load(here("data/bookdata1.RData"))
```

9.3 Graphical exploration should start before descriptive statistics

```
ggplot(rawdata, aes(Testmedication, `sysBP V0`, color = Sex)) +
  stat_summary(
    fun.data = mean_cl_normal,
    position =  position_dodge(width = .15)) +
  stat_summary(geom = "line", fun = \(x) mean(x, na.rm = TRUE),
               aes(group = Sex), linetype = 3,
               position =  position_dodge(width = .15)) +
  ylab("systolic BP (mean \u00b1 95% CI)")
```

```
Warning: Removed 1 row containing non-finite outside the scale range (`stat_summary()`).
Removed 1 row containing non-finite outside the scale range (`stat_summary()`).
```



9.4 Gaussian variables

9.5 A little theory

Sample size n: per variable, if there are NAs

Mean: central tendency, the expected *typical* value	$\frac{\sum x}{n}$
Variance: measure for variability/heterogeneity of data	$\frac{\sum (x-mean)^2}{n-1}$
Standard deviation SD: the *typical* weighted deviation from the mean	$\sqrt{Var}$

Standard error of the mean SEM: how reliable is the mean *estimate*, what would be the expected SD of means from repeated experiments?	$\frac{SD}{\sqrt{n}}$
Median: Split between lower/upper 50% of data	
Quartiles: Split at 25%/50%/75% of data (more general: **Quantiles**, e.g.**Percentiles**), used in boxplot	various computational approaches

9.5.1 Simple function calls

```
(mean_size <- mean(rawdata$`Size (cm)`))
```

```
[1] 174.1071
```

```
(sd_size <- sd(rawdata$`Size (cm)`))
```

```
[1] 7.771454
```

```
min(rawdata$`Size (cm)`)
```

```
[1] 160
```

```
SEM(rawdata$`Size (cm)`)
```

```
[1] 1.468667
```

9.5.2 Combined reporting

For publishable tables you should round the numbers to a reasonable number of digits. Function `roundR()` is more flexible than base `round()`, as it determines the number of digits necessary to obtain the desired precision. The `level` argument allows for rounding to a specific number of non-zero digits. The `.german` argument changes the decimal point to a comma.

```
round(mean_size, digits = 2)
```

```
[1] 174.11
```

```
roundR(mean_size, level = 2)
```

```
[1] "174"
```

Usually mean and sd are reported together, function `meansd()` computes, rounds, and pastes the statistics in one go, arguments allow for flexible reporting:

```
meansd(rawdata$`Size (cm)`,
  roundDig = 4,
  range = TRUE, add_n = TRUE
)
```

```
[1] "174.1 ± 7.8  [160.0 -> 193.0]  [n = 28]"
```

```
meansd(rawdata$`sysBP V0`,
  roundDig = 4,
  range = TRUE,
  add_n = TRUE, .german = TRUE
)
```

```
[1] "121,9 ± 16,9  [94,0 -> 152,0]  [n = 27]"
```

```
meanse(rawdata$`Size (cm)`, roundDig = 4)
```

```
[1] "174.1 ± 1.5"
```

9.6 Ordinal variables

Mean and SD describe symmetric distributions, but are not appropriate for ordinal data. For non-gaussian measurements, median and quartiles are more appropriate. When the distribution is known (e.g. Poisson for count data, other descriptives like lambda may be more informative).

```
median(rawdata$`Size (cm)`)
```

```
[1] 173.5
```

```
quantile(rawdata$`Size (cm)`, probs = c(.25, .75))
```

```
   25%    75%
168.00 178.25
```

```
median_quart(rawdata$`Size (cm)`)
```

```
[1] "174 (168/179)"
```

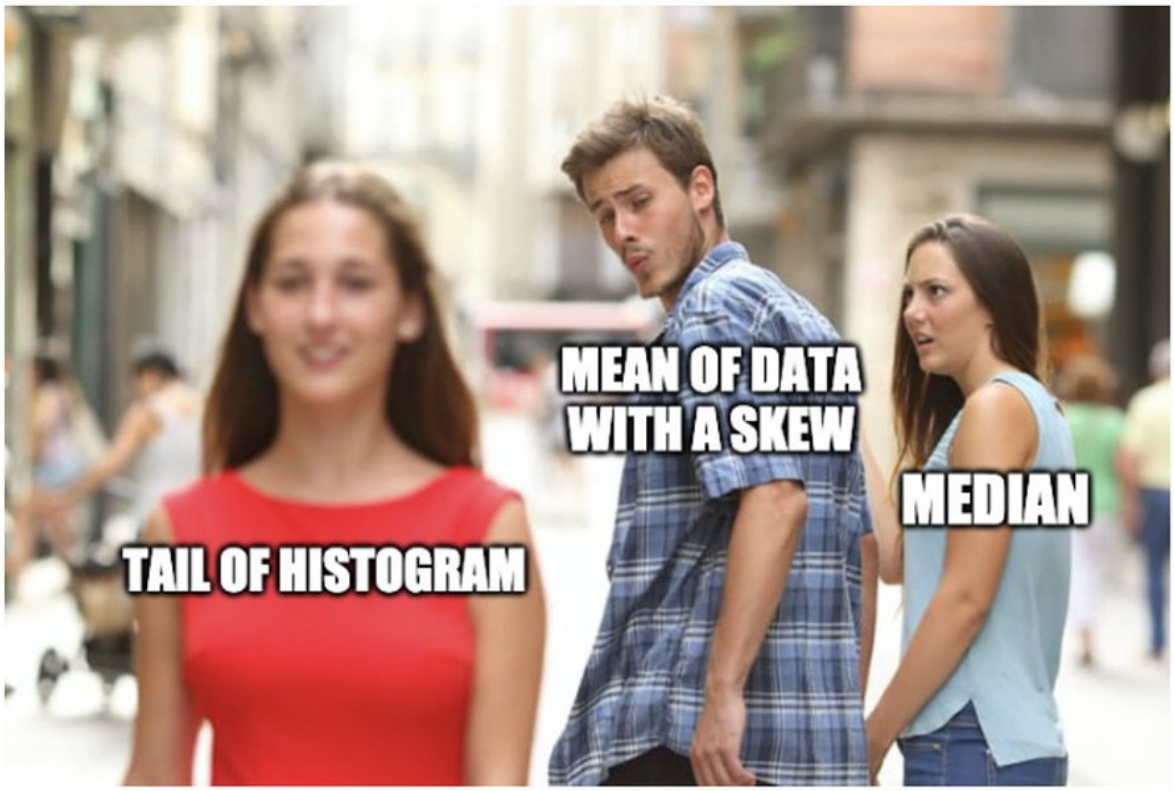
```
median_quart(rawdata$Age, range = T)
```

```
[1] "62 (53/67)  [43 -> 74]"
```

```
median_cl_boot(rawdata$`Size (cm)`)
```

```
# A tibble: 1 x 3
  Median CIlow CIhigh
   <dbl> <dbl>  <dbl>
1   174.  172.    176
```

Median and quartiles are sometimes the better choice even when assuming a Normal distribution, if there are outliers.


MEAN OF DATA
WITH A SKEW
MEDIAN
TAIL OF HISTOGRAM

9.7 Categorical variables

```r
table(rawdata$Sex, useNA = "a")
```

```
   f    m <NA>
   4   24    0
```

```r
sex_count <- table(rawdata$Sex, useNA = "ifany")
table(rawdata$`NYHA V2`, useNA = "always")
```

```
   0    1    2    3 <NA>
   1    6    2    2   17
```

```r
table(rawdata$`NYHA V2`, useNA = "i")
```

```
   0    1    2    3 <NA>
   1    6    2    2   17
```

```r
table(rawdata$`NYHA V2`, useNA = "no")
```

```
0 1 2 3
1 6 2 2
```

```r
randomize <- table(rawdata$Sex, rawdata$Testmedication)

prop.table(sex_count)
```

```
        f         m
0.1428571 0.8571429
```

```r
prop.table(randomize, margin = 2) * 100
```

```
           0        1
  f 14.28571 14.28571
  m 85.71429 85.71429
```

```
cat_desc_stats(rawdata$`NYHA V2`)
```

```
$level
# A tibble: 4 x 1
  value
  <chr>
1 0
2 1
3 2
4 3

$freq
# A tibble: 4 x 1
  desc
  <chr>
1 1 (9%)
2 6 (55%)
3 2 (18%)
4 2 (18%)
```

```
cat_desc_stats(rawdata$Sex, singleline = TRUE)
```

```
$level
[1] "f m"

$freq
# A tibble: 1 x 1
  desc
  <glue>
1 4 (14%) 24 (86%)
```

```
rawdata |>
  mutate(Testmedication = factor(Testmedication,
    levels = 0:1,
    labels = c(
      "Placebo",
      "Verum"
    )
  )) |>
  cat_desc_table(
    desc_vars = factvars$names
  ) |>
```

```
rename(`n (%)` = desc_all) |>
flextable() |>
align(i = ~ `n (%)` != " ", j = 1, align = "right") |>
width(j = c(1, 2), width = c(3, 4), unit = "cm") |>
bg(~ `n (%)` == " ", bg = "lightgrey")
```

Variable	n (%)
Testmedication	
Placebo	14 (50%)
Verum	14 (50%)
Sex	
f	4 (14.29%)
m	24 (85.71%)
NYHA V1	
0	2 (11.76%)
1	9 (52.94%)
2	3 (17.65%)
3	3 (17.65%)
NYHA V2	
0	1 (9.09%)
1	6 (54.55%)
2	2 (18.18%)
3	2 (18.18%)
NYHA V3	
1	6 (50%)
2	3 (25%)
3	3 (25%)

9.8 Summarize data

When creating tables with descriptive statistics, you usually report on more than just 1 variable in more than just 1 subgroup, and there may be more than 1 statistics to report. `Summarize()`, often in combination with `across()`, makes that task easier, usually in a pipeline. As the output will contain (n variables) * (n functions), wide to long pivoting often will be used. (More on this in the next chapter):

```
# we pipe the data
rawdata |>
  # into the summarize function
  summarize(across(all_of(gaussvars$names), # which variables to analyse?
    .fns = list( # which functions to apply?
      # \(x) defines an anonymous function; x is the argument (column values)
      n = \(x) n(),
      Mean = \(x) mean(x, na.rm = TRUE) |>
        roundR(textout = FALSE),
      Median = \(x) median(x, na.rm = TRUE) |>
        roundR(textout = FALSE),
      SD = \(x) sd(x, na.rm = TRUE) |>
        roundR(textout = FALSE)
    )
  )) |>
  # the wide table with n vars * n functions columns is reshaped
  pivot_longer(
    cols = everything(), # transform all columns into long form
    names_to = c("Variable", ".value"), # .value is extracting names for values
    names_sep = "_"
  ) |>
  # pipe into formatted table
  flextable() |>
  set_table_properties(width = 1, layout = "autofit")
```

Variable	n	Mean	Median	SD
Size (cm)	28	174	174	7.8
Weight (kg)	28	88	84	15.0
sysBP V0	28	122	126	17.0
diaBP V0	28	70	69	9.8
Lv Edv Mri	28	206	193	70.0

Variable	n	Mean	Median	SD
Lv Esv Mri	28	110	77	70.0
Lv Ef Mri	28	50	55	15.0
Lv Ef Biplan Mri	28	49	51	14.0
sysBP V2	28	119	120	13.0
diaBP V2	28	66	68	8.6
BMI	28	29	28	4.3

```
rawdata |>
  group_by(Testmedication, Sex) |> # describe subgroups
  summarise(
    WeightSummary = meansd(`Weight (kg)`, add_n = TRUE),
    .groups = "drop"
  ) |>
  flextable()
```

Testmedication	Sex	WeightSummary
0	f	86 ± 9 [n = 2]
0	m	91 ± 14 [n = 12]
1	f	66 ± 3 [n = 2]
1	m	90 ± 14 [n = 12]

```
rawdata |>
  group_by(Sex) |>
  summarise(across(gaussvars$names,
    .fns = \(x) meansd(x, range = TRUE,
                       add_n = T, rangesep = " / ")
  )) |>
  pivot_longer( # cosmetics
    cols = -Sex,
    names_to = "Measure"
  ) |>
  pivot_wider(names_from = Sex) |>
```

```
  flextable() |>
  set_table_properties(width = 1, layout = "autofit")
```

Measure	f	m
Size (cm)	165 ± 4 / [160 -> 168] / [n = 4]	176 ± 7 / [161 -> 193] / [n = 24]
Weight (kg)	76 ± 13 / [64 -> 93] / [n = 4]	90 ± 14 / [74 -> 120] / [n = 24]
sysBP V0	118 ± 14 / [102 -> 130] / [n = 4]	123 ± 18 / [94 -> 152] / [n = 23]
diaBP V0	71 ± 13 / [62 -> 90] / [n = 4]	69 ± 9 / [45 -> 90] / [n = 23]
Lv Edv Mri	235 ± 72 / [184 -> 286] / [n = 2]	203 ± 71 / [118 -> 379] / [n = 19]
Lv Esv Mri	158 ± 56 / [119 -> 198] / [n = 2]	105 ± 71 / [50 -> 294] / [n = 19]
Lv Ef Mri	33 ± 3 / [31 -> 35] / [n = 2]	52 ± 15 / [17 -> 74] / [n = 19]
Lv Ef Biplan Mri	26 ± 11 / [19 -> 34] / [n = 2]	51 ± 13 / [21 -> 69] / [n = 19]
sysBP V2	114 ± 18 / [96 -> 132] / [n = 3]	120 ± 12 / [100 -> 145] / [n = 21]
diaBP V2	58 ± 12 / [51 -> 72] / [n = 3]	67 ± 8 / [55 -> 80] / [n = 21]
BMI	28 ± 5 / [24 -> 35] / [n = 4]	29 ± 4 / [23 -> 41] / [n = 24]

```
rawdata |>
  group_by(Sex) |>
  summarise(across(gaussvars$names,
    .fns = list(
      n = \(x) n(),
      Mean = \(x) mean(x, na.rm = TRUE) |>
        roundR(textout = F),
      Median = \(x) median(x, na.rm = TRUE) |>
        roundR(textout = F),
      SD = \(x) sd(x, na.rm = TRUE) |>
        roundR(textout = F)
    )
  )) |>
  pivot_longer(
    cols = -Sex,
    names_to = c("Variable", "stat"),
    names_sep = "_"
  ) |>
  pivot_wider(
    names_from = c(stat, Sex),
    # names_sep = " ",
```

```
    names_glue = "{stat} ({Sex})",
    values_from = value
  ) |>
  flextable() |>
  set_table_properties(width = 1, layout = "autofit")
```

Variable	n (f)	Mean (f)	Median (f)	SD (f)	n (m)	Mean (m)	Median (m)	SD (m)
Size (cm)	4	165	166	3.8	24	176	174	7.2
Weight (kg)	4	76	74	13.0	24	90	86	14.0
sysBP V0	4	118	119	14.0	24	123	126	18.0
diaBP V0	4	71	66	13.0	24	69	70	9.5
Lv Edv Mri	4	235	235	72.0	24	203	193	71.0
Lv Esv Mri	4	158	158	56.0	24	105	71	71.0
Lv Ef Mri	4	33	33	2.8	24	52	56	15.0
Lv Ef Biplan Mri	4	26	26	11.0	24	51	54	13.0
sysBP V2	4	114	114	18.0	24	120	120	12.0
diaBP V2	4	58	52	12.0	24	67	68	7.8
BMI	4	28	27	4.7	24	29	29	4.4

```
compare2numvars(
  data = rawdata,
  dep_vars = c(
    "Size (cm)", "Weight (kg)",
    "sysBP V0", "diaBP V0"
  ),
  indep_var = "Sex",
  gaussian = TRUE
) |>
  rename(overall = desc_all) |>
  rename_with(.fn = \(x) str_remove(x, "Sex ")) |>
  rename_with(.cols = "p", .fn = \(x) paste(x, "- value")) |>
  flextable() |>
  set_table_properties(width = 1, layout = "autofit")
```

Variable	overall	f	m	p - value
Size (cm)	174 ± 8	165 ± 4	176 ± 7	0.009
Weight (kg)	88 ± 15	76 ± 13	90 ± 14	0.072
sysBP V0	122 ± 17	118 ± 14	123 ± 18	0.587
diaBP V0	70 ± 10	71 ± 13	69 ± 9	0.780

```
compare2qualvars(
  data = rawdata,
  dep_vars = factvars$names[-2],
  indep_var = factvars$names[2]
) |>
  flextable() |>
  bg(~ p != " ", bg = "lightgrey") |>
  set_table_properties(width = 1, layout = "autofit")
```

Variable	desc_all	Sex f	Sex m	p
Testmedication				1.000
0	14 (50%)	2 (50%)	12 (50%)	
1	14 (50%)	2 (50%)	12 (50%)	
NYHA V1				0.096
0	2 (11.76%)	1 (33.33%)	1 (7.14%)	
1	9 (52.94%)	0 (0%)	9 (64.29%)	
2	3 (17.65%)	1 (33.33%)	2 (14.29%)	
3	3 (17.65%)	1 (33.33%)	2 (14.29%)	
NYHA V2				1.000
0	1 (9.09%)	0 (0%)	1 (12.5%)	
1	6 (54.55%)	2 (66.67%)	4 (50%)	
2	2 (18.18%)	1 (33.33%)	1 (12.5%)	
3	2 (18.18%)	0 (0%)	2 (25%)	
NYHA V3				0.092
1	6 (50%)	0 (0%)	6 (66.67%)	

Variable	desc_all	Sex f	Sex m	p
2	3 (25%)	1 (33.33%)	2 (22.22%)	
3	3 (25%)	2 (66.67%)	1 (11.11%)	

9.9 Exercise

Analyze the penguins data using summarize, functions can be any of mean/sd/median/meansd/median_quart. Result tables should be pivoted if reasonable.

Describe species and sex with appropriate statistics as well.

- 1 function / 1 variable / no groups
- 2 functions / 1 variable / no groups
- 2 functions / 1 variable / subgroup species
- 1 function / 4 variables / no groups
- 1 function / 4 variables / subgroup species
- 2 functions / 4 variables / no subgroups
- 2 functions / 4 variables / subgroup species

10 Summarize / across

```
pacman::p_load(
  conflicted, tidyverse, wrappedtools,
  flextable
)
```

1 variable / 1 function / no groups

```
summarise(
  .data = mtcars,
  MeanSD = meansd(mpg)
) |>
  flextable()
```

MeanSD
20 ± 6

1 variable / 1 function / subgroups

```
mtcars |>
  group_by(am) |>
  summarise(
    MeanSD = meansd(mpg),
    .groups = "drop"
  ) |>
  flextable()
```

am	MeanSD
0	17 ± 4
1	24 ± 6

```
# groups in columns
mtcars |>
  group_by(am) |>
```

```
  summarise(
    MeanSD = meansd(mpg),
    .groups = "drop"
  ) |>
  pivot_wider(
    names_from = am,
    values_from = MeanSD,
    names_glue = "am ({.value}): {am}"
  ) |>
  flextable() |>
  separate_header(split = ": ")
```

am (MeanSD)	
0	1
17 ± 4	24 ± 6

1 variable / 2 functions / no groups

```
summarise(
  .data = mtcars,
  `MeanSD mpg` = meansd(mpg, roundDig = 3),
  `MedianQuartiles mpg` = median_quart(mpg, roundDig = 3)
) |>
  flextable() |>
  set_table_properties(width = .7, layout = "autofit")
```

MeanSD mpg	MedianQuartiles mpg
20.1 ± 6.0	19.2 (15.3/22.8)

1 variable / 2 functions / subgroups

```
mtcars |>
  group_by(am) |>
  summarise(
    `MeanSD mpg` = meansd(mpg),
    `MedianQuartiles mpg` = median_quart(mpg),
    .groups = "drop"
  ) |>
  flextable()
```

am	MeanSD mpg	Median-Quartiles mpg
0	17 ± 4	17 (15/19)
1	24 ± 6	23 (21/30)

```
# groups in columns
mtcars |>
  group_by(am) |>
  summarise(
    `MeanSD mpg` = meansd(mpg),
    `MedianQuartiles mpg` = median_quart(mpg),
    .groups = "drop"
  ) |>
  pivot_longer(
    cols = starts_with("M"),
    names_to = "Statistics",
    values_to = "estimate"
  ) |>
  pivot_wider(
    names_from = am,
    values_from = estimate,
    names_glue = "am: {am}"
  ) |>
  flextable()
```

Statistics	am: 0	am: 1
MeanSD mpg	17 ± 4	24 ± 6
Median-Quartiles mpg	17 (15/19)	23 (21/30)

2 variables / 1 function / no groups

```
# no function arguments
mtcars |>
  summarise(across(
    .cols = c(mpg, disp),
```

```
    .fns = meansd
  )) |>
  flextable() |>
  set_table_properties(width = .7, layout = "autofit")
```

mpg	disp
20 ± 6	231 ± 124

```
# with function arguments
mtcars |>
  summarise(across(
    .cols = c(mpg, disp),
    .fns = \(x) meansd(x, add_n = TRUE, range = TRUE)
  )) |>
  flextable() |>
  set_table_properties(width = .7, layout = "autofit")
```

mpg			disp		
20 ± 6	[10 -> 34]	[n = 32]	231 ± 124	[71 -> 472]	[n = 32]

```
# Variables in rows
mtcars |>
  summarise(across(c(mpg, disp),
    .fns = \(x) meansd(x, add_n = TRUE)
  )) |>
  pivot_longer(
    cols = everything(),
    names_to = "Variable",
    values_to = "MeanSD"
  ) |>
  flextable() |>
  set_table_properties(width = .7, layout = "autofit")
```

Variable	MeanSD
mpg	20 ± 6 [n = 32]
disp	231 ± 124 [n = 32]

2 variables / 1 function / subgroups

```
mtcars |>
  group_by(am) |>
  summarise(
    across(
      .cols = c(mpg, disp),
      .fns = \(x) meansd(x, add_n = TRUE)
    ),
    .groups = "drop"
  ) |>
  flextable() |>
  set_table_properties(width = .7, layout = "autofit")
```

am	mpg	disp
0	17 ± 4 [n = 19]	290 ± 110 [n = 19]
1	24 ± 6 [n = 13]	144 ± 87 [n = 13]

```
# Variables in rows
mtcars |>
  group_by(am) |>
  summarise(
    across(
      c(mpg, disp),
      \(x) meansd(x, add_n = TRUE)
    ),
    .groups = "drop"
  ) |>
  pivot_longer(
    cols = -am,
    names_to = "Variable",
    values_to = "MeanSD"
  ) |>
  pivot_wider(
    names_from = am,
    values_from = MeanSD,
    names_prefix = "am: "
  ) |>
  flextable() |>
  set_table_properties(width = .7, layout = "autofit")
```

Variable	am: 0	am: 1
mpg	17 ± 4 [n = 19]	24 ± 6 [n = 13]
disp	290 ± 110 [n = 19]	144 ± 87 [n = 13]

2 variables / 2 function / no groups

```
# with/without function arguments
mtcars |>
  summarise(across(
    c(mpg, disp),
    .fns = list(
      MeanSD = \(x) meansd(x,
        add_n = TRUE
      ),
      MedianQuart = median_quart
    ),
    .names = "{.col}: {.fn}"
  )) |>
  flextable() |>
  set_table_properties(width = 1, layout = "autofit")
```

mpg: MeanSD	mpg: MedianQuart	disp: MeanSD	disp: MedianQuart
20 ± 6 [n = 32]	19 (15/23)	231 ± 124 [n = 32]	196 (121/337)

```
# Variables in rows
mtcars |>
  summarise(across(
    c(mpg, disp),
    .fns = list(
      MeanSD = \(x) meansd(x,
        add_n = TRUE
      ),
      MedianQuart = median_quart
    )
  )) |>
  pivot_longer(
    cols = everything(),
    names_to = c("Variable", ".value"),
    names_sep = "_"
  ) |>
```

```
  flextable() |>
  set_table_properties(width = 1, layout = "autofit")
```

Variable	MeanSD	MedianQuart
mpg	20 ± 6 [n = 32]	19 (15/23)
disp	231 ± 124 [n = 32]	196 (121/337)

2 variables / 2 function / subgroups

```
# with/without function arguments
mtcars |>
  group_by(am) |>
  summarise(
    across(
      c(mpg, disp),
      .fns = list(
        MeanSD = \(x) meansd(x,
          add_n = TRUE
        ),
        MedianQuart = median_quart
      ),
      .names = "{.col}_{.fn}"), # default, but good to know!
    .groups = "drop"
  ) |>
  flextable() |>
  set_table_properties(width = 1, layout = "autofit")
```

am	mpg_MeanSD	mpg_MedianQuart	disp_MeanSD	disp_MedianQuart
0	17 ± 4 [n = 19]	17 (15/19)	290 ± 110 [n = 19]	276 (177/360)
1	24 ± 6 [n = 13]	23 (21/30)	144 ± 87 [n = 13]	120 (79/160)

```
# Variables in rows, groups in columns
mtcars |>
  group_by(am) |>
  summarise(
    across(
      c(mpg, disp),
      .fns = list(
        MeanSD = \(x) meansd(x,
```

```
          add_n = TRUE
        ),
        MedianQuart = median_quart
      )
    ),
    .groups = "drop"
  ) |>
  pivot_longer(
    cols = -am,
    names_to = c("Variable", ".value"),
    names_sep = "_"
  ) |>
  pivot_wider(
    names_from = am,
    values_from = starts_with("M"),
    names_glue = "am: {am}_{.value}",
    names_vary = "slowest"
  ) |>
  flextable() |>
  separate_header(split = "[:_]") |> # cosmetics for header
  vline(j = 3) |> # make group-split more obvious
  set_table_properties(width = 1, layout = "autofit")
```

	am			
	0		1	
Variable	MeanSD	MedianQuart	MeanSD	MedianQuart
mpg	17 ± 4 [n = 19]	17 (15/19)	24 ± 6 [n = 13]	23 (21/30)
disp	290 ± 110 [n = 19]	276 (177/360)	144 ± 87 [n = 13]	120 (79/160)

```
# pivoting to have variables in rows V2

# with/without function arguments
result_long <-
  mtcars |>
  group_by(am) |>
  summarise(across(
    c(mpg, disp),
    .fns = list(
      MeanSD = \(x) meansd(x,
        add_n = TRUE
```

```
      ),
      MedianQuart = median_quart
    )
  )) |>
  pivot_longer(
    cols = -c(am),
    names_to = c("Variable", ".value"),
    names_sep = "_",
    values_to = "Value"
  )
result_long |>
  flextable() |>
  merge_v(j = 1) |> # remove double entries
  hline(i = 2) |> # make group-split obvious
  set_table_properties(width = 1, layout = "autofit")
```

am	Variable	MeanSD	MedianQuart
0	mpg	17 ± 4 [n = 19]	17 (15/19)
	disp	290 ± 110 [n = 19]	276 (177/360)
1	mpg	24 ± 6 [n = 13]	23 (21/30)
	disp	144 ± 87 [n = 13]	120 (79/160)

```
result <-
  result_long |>
  pivot_wider(
    names_from = am,
    names_prefix = "am:",
    names_sep = " ",
    values_from = c(MeanSD, MedianQuart)
  )
result |>
  flextable() |>
  separate_header(split = "[ ]") |>
  set_table_properties(width = 1, layout = "autofit")
```

	MeanSD		MedianQuart	
Variable	am:0	am:1	am:0	am:1
mpg	17 ± 4 [n = 19]	24 ± 6 [n = 13]	17 (15/19)	23 (21/30)

	MeanSD		MedianQuart	
Variable	am:0	am:1	am:0	am:1
disp	290 ± 110 [n = 19]	144 ± 87 [n = 13]	276 (177/360)	120 (79/160)

11 Simple test statistics

11.1 Tests require hypotheses



11.1.1 Null hypothesis ?

- Working hypothesis: This is what you expect!
  E.g. treatment is lowering blood pressure more than placebo, transgenic animals become obese, bio reactor A is more efficient than B, concentration of substance is correlated with speed of reaction …
- Null hypothesis: This is what you test!
  No difference / relation, BP under therapy = BP under placebo

4 possibilities:

- Null hypothesis correct, test false positive (case A): alpha-error
- Null hypothesis correct, test correct negative (case B)
- Null hypothesis false, test false negative (case C): beta-error
- Null hypothesis false, test correct positive (case D)

Significance: NOT probability of case A, but probability of your data given the NULL hypothesis, calculated from your data, conventionally <0.05

Power: Probability of case D, *estimated* based on assumptions about effects and sample size, *calculation* would require knowledge of true difference, conventionally set at 0.80; this translates into a ***20% risk of false negative results***!

```
pacman::p_load(
  conflicted, tidyverse, wrappedtools, here,
  coin, ggsignif, patchwork, ggbeeswarm,
  flextable
)
# conflicted)
# conflict_prefer("filter", "dplyr")
load(here("data/bookdata1.RData"))
```

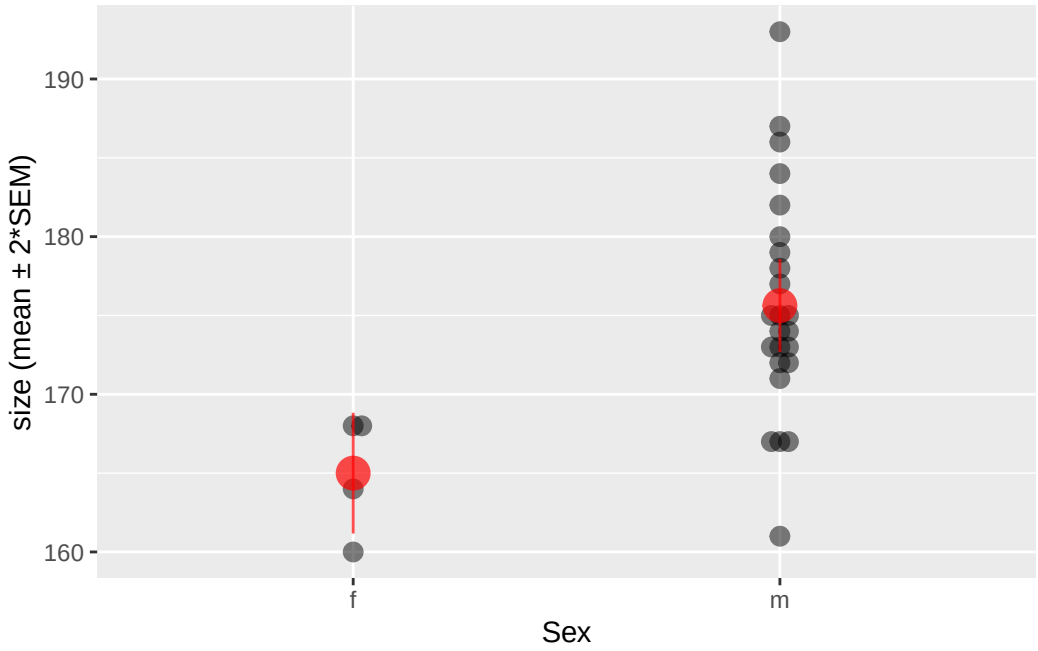
11.2 Quantitative measures with Gaussian distribution

t-test

- Assumptions: Continuous data with Normal distribution
- 1 or 2 (independent or dependent) samples with/without equal variances
- how big is the mean difference relative to uncertainty?
  t = ($mean_1$ - $mean_2$)/SEM

- t follows a t-distribution, allows estimation of probability of t under the NULL hypothesis

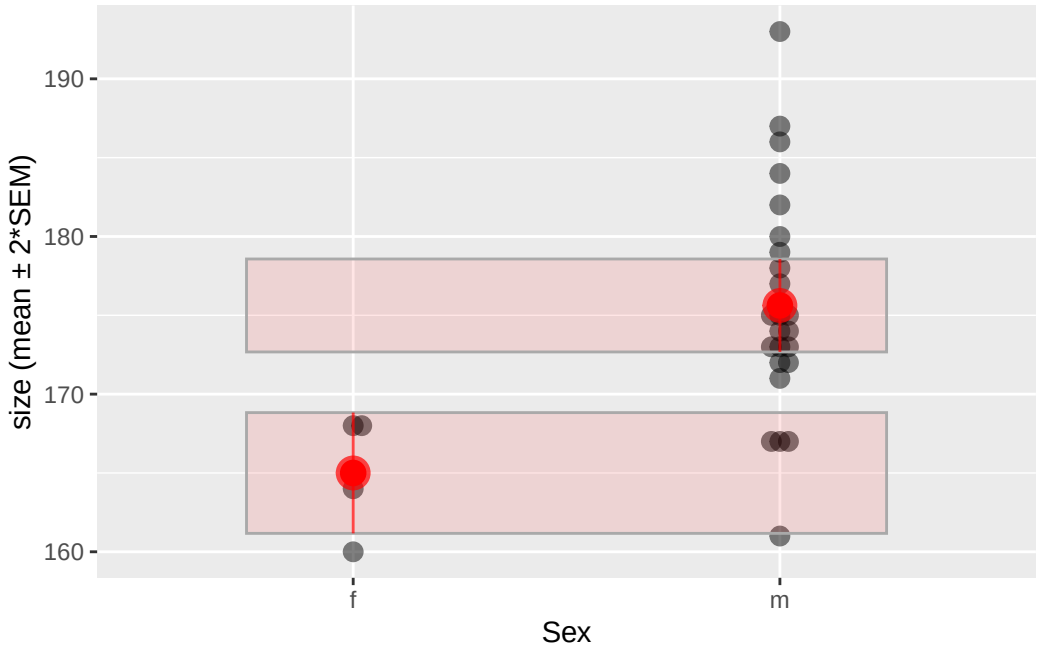
```
ggplot(rawdata, aes(x = Sex, y = `Size (cm)`)) +
  geom_beeswarm(size = 3, alpha = .5) +
  stat_summary(
    color = "red", size = 1.2, alpha = .7,
    fun.data = "mean_se", fun.args = list(mult = 2)
  ) +
  ylab("size (mean \u00B1 2*SEM)")
```




```
# emphasis on the non-overlapping CIs:
size_summary <- rawdata |>
  group_by(Sex) |>
  summarise(
    mean_size = mean(`Size (cm)`, na.rm = T),
    se_size = sd(`Size (cm)`, na.rm = T) / sqrt(n()),
    .groups = "drop"
  ) |>
  mutate(
    lower = mean_size - 2 * se_size,
    upper = mean_size + 2 * se_size)

ggplot(rawdata, aes(x = Sex, y = `Size (cm)`)) +
  geom_beeswarm(size = 3, alpha = .5) +
```

```
  geom_rect(
    data = size_summary |>
      mutate(x_pos = as.numeric(factor(Sex))),
    aes(xmin = 0.75, xmax = 2.25,
        ymin = lower, ymax = upper),
    alpha = .1, fill = "red", color = "darkgrey",
    inherit.aes = FALSE
  ) +
  geom_point(
    data = size_summary,
    aes(x = Sex, y = mean_size),
    color = "red", size = 4,
    inherit.aes = FALSE
  ) +
  stat_summary(
    color = "red", size = 1.2, alpha = .7,
    fun.data = "mean_se", fun.args = list(mult = 2)
  ) +
  ylab("size (mean ± 2*SEM)")
```




```
rawdata |>
  group_by(Sex) |>
  summarise(
```

```
    MeanSE = meanse(`Size (cm)`),
    SD = sd(`Size (cm)`)
  )
```

```
# A tibble: 2 x 3
  Sex   MeanSE     SD
  <fct> <chr>   <dbl>
1 f     165 ± 2  3.83
2 m     176 ± 1  7.22
```

```
t.test(
  x = rawdata$`Size (cm)`[which(rawdata$Sex == "f")],
  y = rawdata$`Size (cm)`[which(rawdata$Sex == "m")]
)
```

```
    Welch Two Sample t-test

data:  rawdata$`Size (cm)`[which(rawdata$Sex == "f")] and rawdata$`Size (cm)`[which(rawdata$S
t = -4.3967, df = 7.2767, p-value = 0.002887
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -16.295575  -4.954425
sample estimates:
mean of x mean of y
  165.000   175.625
```

```
rawdata$`Size (cm)`[which(rawdata$Sex == "f")]
```

```
[1] 164 168 168 160
```

```
rawdata |> filter(Sex == "f") |> pull(`Size (cm)`)
```

```
[1] 164 168 168 160
```

```
tOut <- t.test(rawdata$`Size (cm)` ~ rawdata$Sex)
tOut$p.value
```

```
[1] 0.00288704
```

```
# equal variances assumption?
vartestOut <- var.test(rawdata$`Size (cm)` ~ rawdata$Sex)
vartestOut
```

```
    F test to compare two variances

data:  rawdata$`Size (cm)` by rawdata$Sex
F = 0.2812, num df = 3, denom df = 23, p-value = 0.3232
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.07497668 3.97434769
sample estimates:
ratio of variances
          0.281199
```

```
# car::leveneTest(rawdata$`Size (cm)`~rawdata$Sex)
# manual entry
t.test(rawdata$`Size (cm)` ~ rawdata$Sex,
  var.equal = TRUE
)
```

```
    Two Sample t-test

data:  rawdata$`Size (cm)` by rawdata$Sex
t = -2.8446, df = 26, p-value = 0.008552
alternative hypothesis: true difference in means between group f and group m is not equal to
95 percent confidence interval:
 -18.302594  -2.947406
sample estimates:
mean in group f mean in group m
        165.000         175.625
```

```
t.test(rawdata$`Size (cm)` ~ rawdata$Sex,
  var.equal = vartestOut$p.value > .05
)
```

```
    Two Sample t-test

data:  rawdata$`Size (cm)` by rawdata$Sex
t = -2.8446, df = 26, p-value = 0.008552
alternative hypothesis: true difference in means between group f and group m is not equal to
95 percent confidence interval:
 -18.302594  -2.947406
```

```
sample estimates:
mean in group f mean in group m
        165.000         175.625
```

```
# picked from test
t.test(rawdata$`Size (cm)` ~ rawdata$Sex,
  var.equal = var.test(
    rawdata$`Size (cm)` ~ rawdata$Sex
  )$p.value > .05
)
```

```
    Two Sample t-test

data:  rawdata$`Size (cm)` by rawdata$Sex
t = -2.8446, df = 26, p-value = 0.008552
alternative hypothesis: true difference in means between group f and group m is not equal to
95 percent confidence interval:
 -18.302594  -2.947406
sample estimates:
mean in group f mean in group m
        165.000         175.625
```

```
# combined function
t_var_test(
  data = rawdata,
  formula = "`Size (cm)`~Sex",
  cutoff = .1
)
```

```
    Two Sample t-test

data:  Size (cm) by Sex
t = -2.8446, df = 26, p-value = 0.008552
alternative hypothesis: true difference in means between group f and group m is not equal to
95 percent confidence interval:
 -18.302594  -2.947406
sample estimates:
mean in group f mean in group m
        165.000         175.625
```

```r
# pairwise t-test, within subject
print(c(
  mean(rawdata$`sysBP V0`, na.rm = T),
  mean(rawdata$`sysBP V2`, na.rm = T)
))
```

```
[1] 121.8519 119.4583
```

```r
t.test(x = rawdata$`sysBP V0`,
       y = rawdata$`sysBP V2`,
       alternative = "greater", # x>y
       paired = TRUE
)
```

```

    Paired t-test

data:  rawdata$`sysBP V0` and rawdata$`sysBP V2`
t = 0.88151, df = 23, p-value = 0.1936
alternative hypothesis: true mean difference is greater than 0
95 percent confidence interval:
 -2.793386       Inf
sample estimates:
mean difference
       2.958333
```

```r
t.test(rawdata$`sysBP V0`,
  rawdata$`sysBP V2`,
  # alternative = "greater", # x>y
  paired = T
)$p.value / 2
```

```
[1] 0.1935805
```

```r
# 1-sample test
t.test(rawdata$`Size (cm)`, mu = 173)
```

```

    One Sample t-test

data:  rawdata$`Size (cm)`
t = 0.75384, df = 27, p-value = 0.4575
```

```
alternative hypothesis: true mean is not equal to 173
95 percent confidence interval:
 171.0937 177.1206
sample estimates:
mean of x
 174.1071
```

```
groupvars <- ColSeeker(namepattern = c("Sex", "Test"))
```

```
compare2numvars(
  data = rawdata, dep_vars = gaussvars$names,
  indep_var = "Sex", gaussian = T,
  round_desc = 3, n = T, mark = T
) |>
  flextable() |>
  set_table_properties(width = 1, layout = "autofit")
```

Variable	n	desc_all	n g1	Sex f	n g2	Sex m	p
Size (cm)	28	174 ± 8	4	165 ± 4	24	176 ± 7	0.009 **
Weight (kg)	28	88.4 ± 14.6	4	76.2 ± 13.1	24	90.4 ± 14.1	0.072 +
sysBP V0	27	122 ± 17	4	118 ± 14	23	123 ± 18	0.587 n.s.
diaBP V0	27	69.7 ± 9.8	4	71.0 ± 12.8	23	69.5 ± 9.5	0.780 n.s.
Lv Edv Mri	21	206 ± 70	2	235 ± 72	19	203 ± 71	0.559 n.s.
Lv Esv Mri	21	110 ± 70	2	158 ± 56	19	105 ± 71	0.322 n.s.
Lv Ef Mri	21	49.8 ± 14.9	2	33.0 ± 2.8	19	51.6 ± 14.6	0.095 +
Lv Ef Biplan Mri	21	48.7 ± 14.5	2	26.5 ± 10.6	19	51.0 ± 12.9	0.018 *
sysBP V2	24	119 ± 13	3	114 ± 18	21	120 ± 12	0.438 n.s.
diaBP V2	24	66.1 ± 8.6	3	58.3 ± 11.8	21	67.2 ± 7.8	0.097 +
BMI	28	29.1 ± 4.3	4	28.0 ± 4.7	24	29.3 ± 4.4	0.585 n.s.

```
compare2numvars(
  data = rawdata, dep_vars = gaussvars$names,
  indep_var = "Testmedication", gaussian = T,
  round_desc = 4
) |>
  flextable() |>
  set_table_properties(width = 1, layout = "autofit")
```

Variable	desc_all	Testmedication 0	Testmedication 1	p
Size (cm)	174.1 ± 7.8	173.4 ± 6.4	174.8 ± 9.2	0.653
Weight (kg)	88.36 ± 14.59	90.50 ± 13.51	86.21 ± 15.81	0.448
sysBP V0	121.9 ± 16.9	122.0 ± 15.1	121.7 ± 19.0	0.966
diaBP V0	69.70 ± 9.75	69.85 ± 7.12	69.57 ± 11.97	0.943
Lv Edv Mri	206.4 ± 70.3	242.9 ± 76.9	173.2 ± 45.0	0.019
Lv Esv Mri	110.5 ± 70.3	137.8 ± 87.0	85.6 ± 40.5	0.108
Lv Ef Mri	49.81 ± 14.95	47.60 ± 18.33	51.82 ± 11.63	0.532
Lv Ef Biplan Mri	48.67 ± 14.47	47.44 ± 17.06	49.58 ± 12.92	0.747
sysBP V2	119.5 ± 12.7	122.5 ± 11.2	116.9 ± 13.7	0.297
diaBP V2	66.08 ± 8.63	66.00 ± 8.10	66.15 ± 9.39	0.966
BMI	29.13 ± 4.35	30.24 ± 5.25	28.02 ± 3.00	0.180

```
for (group_i in seq_len(groupvars$count)) {
  resulttmp <-
    compare2numvars(
      data = rawdata,
      dep_vars = gaussvars$names,
      indep_var = groupvars$names[group_i], gaussian = T
    )
  # print(resulttmp)
  flextable(resulttmp) |>
    set_table_properties(width = 1, layout = "autofit") |>
    flex2rmd() #|> print()

  cat("\\newpage\n\n")
}
```

Variable	desc_all	Testmedication 0	Testmedication 1	p
Size (cm)	174 ± 8	173 ± 6	175 ± 9	0.653
Weight (kg)	88 ± 15	90 ± 14	86 ± 16	0.448
sysBP V0	122 ± 17	122 ± 15	122 ± 19	0.966
diaBP V0	70 ± 10	70 ± 7	70 ± 12	0.943
Lv Edv Mri	206 ± 70	243 ± 77	173 ± 45	0.019

Variable	desc_all	Testmedication 0	Testmedication 1	p
Lv Esv Mri	110 ± 70	138 ± 87	86 ± 41	0.108
Lv Ef Mri	50 ± 15	48 ± 18	52 ± 12	0.532
Lv Ef Biplan Mri	49 ± 14	47 ± 17	50 ± 13	0.747
sysBP V2	119 ± 13	122 ± 11	117 ± 14	0.297
diaBP V2	66 ± 9	66 ± 8	66 ± 9	0.966
BMI	29 ± 4	30 ± 5	28 ± 3	0.180

Variable	desc_all	Sex f	Sex m	p
Size (cm)	174 ± 8	165 ± 4	176 ± 7	0.009
Weight (kg)	88 ± 15	76 ± 13	90 ± 14	0.072
sysBP V0	122 ± 17	118 ± 14	123 ± 18	0.587
diaBP V0	70 ± 10	71 ± 13	69 ± 9	0.780
Lv Edv Mri	206 ± 70	235 ± 72	203 ± 71	0.559
Lv Esv Mri	110 ± 70	158 ± 56	105 ± 71	0.322
Lv Ef Mri	50 ± 15	33 ± 3	52 ± 15	0.095
Lv Ef Biplan Mri	49 ± 14	26 ± 11	51 ± 13	0.018
sysBP V2	119 ± 13	114 ± 18	120 ± 12	0.438
diaBP V2	66 ± 9	58 ± 12	67 ± 8	0.097
BMI	29 ± 4	28 ± 5	29 ± 4	0.585

11.3 Ordinal data

Wilcoxon-test / Mann-Whitney U test

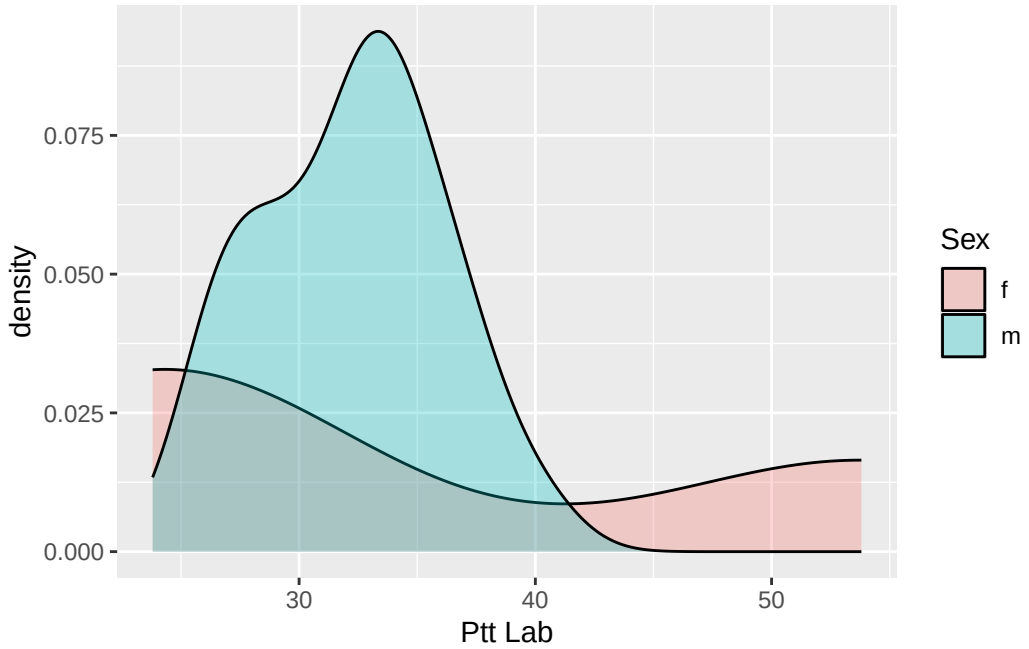
- nonparametric, no distribution is assumed
- based on rank-transformed data
- insensitive to extreme values

```
ordvars$names
```

```
[1] "Ptt Lab"         "Ferritin Lab"    "Iron Lab"        "Transferrin Lab"
[5] "Age"
```

```
ggplot(rawdata, aes(`Ptt Lab`, fill = Sex)) +
  geom_density(alpha = .3)
```

```
Warning: Removed 1 row containing non-finite outside the scale range
(`stat_density()`).
```



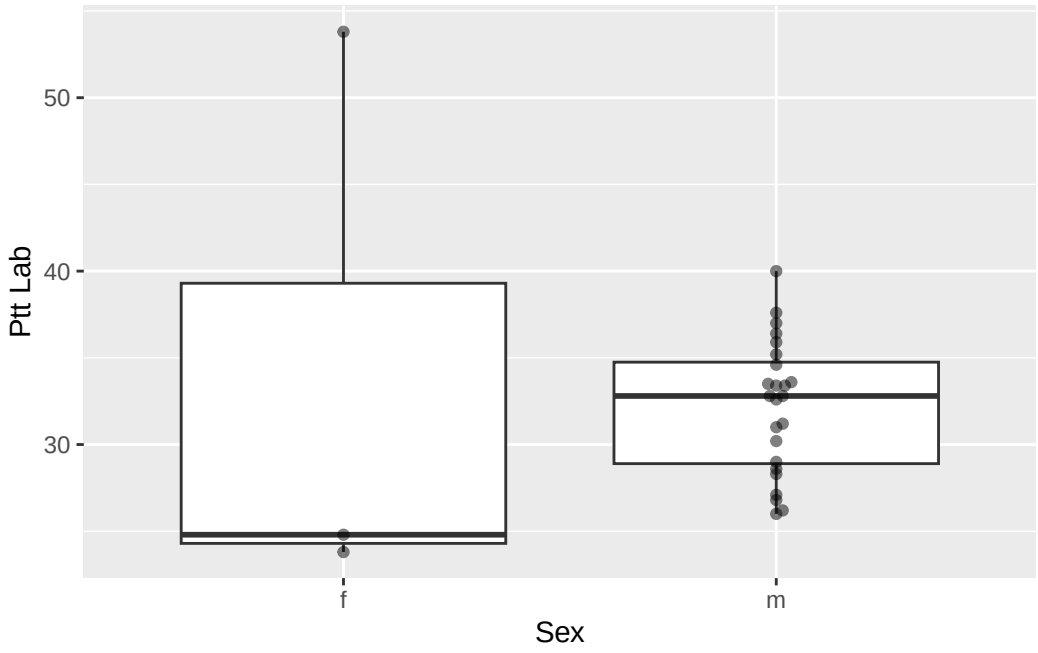
```r
by(
  data = rawdata[[ordvars$index[1]]],
  INDICES = rawdata$Sex, FUN = median_quart
)
```

```
rawdata$Sex: f
[1] "25 (24/49)"
------------------------------------------------------------
rawdata$Sex: m
[1] "33 (29/35)"
```

```r
ggplot(rawdata, aes(Sex, `Ptt Lab`)) +
  geom_boxplot() +
  geom_beeswarm(alpha = .5)
```

```
Warning: Removed 1 row containing non-finite outside the scale range
(`stat_boxplot()`).
```

```
Warning: Removed 1 row containing missing values or values outside the scale range
(`geom_point()`).
```



```
(uOut <- wilcox.test(
  rawdata[[ordvars$names[1]]] ~ rawdata$Sex,
  exact = F
))
```

```
    Wilcoxon rank sum test with continuity correction

data:  rawdata[[ordvars$names[1]]] by rawdata$Sex
W = 24, p-value = 0.3748
alternative hypothesis: true location shift is not equal to 0
```

```
uOut$p.value
```

```
[1] 0.3748016
```

```
#  coin::wilcox_test
(uOut2 <- wilcox_test(`Ptt Lab` ~ Sex,
  data = rawdata
))
```

```
    Asymptotic Wilcoxon-Mann-Whitney Test

data:  Ptt Lab by Sex (f, m)
Z = -0.9261, p-value = 0.3544
alternative hypothesis: true mu is not equal to 0
```

```
pvalue(uOut2) # no list-object, but methods to extract infos like p
```

```
[1] 0.3543925
```

```
wilcox.test(`Ptt Lab` ~ Sex,
  exact = F, correct = F,
  data = rawdata
)
```

```
    Wilcoxon rank sum test

data:  Ptt Lab by Sex
W = 24, p-value = 0.3544
alternative hypothesis: true location shift is not equal to 0
```

```
wilcox.test(
  x = rawdata$`sysBP V0`, y = rawdata$`sysBP V2`,
  exact = FALSE,
  correct = TRUE, paired = TRUE
)
```

```
    Wilcoxon signed rank test with continuity correction

data:  rawdata$`sysBP V0` and rawdata$`sysBP V2`
V = 143, p-value = 0.3478
alternative hypothesis: true location shift is not equal to 0
```

```
compare2numvars(
  data = rawdata, dep_vars = ordvars$names, n = F,
  range = T, add_n = T,
  indep_var = "Sex", gaussian = F
) |>
  flextable() |>
  set_table_properties(width = 1, layout = "autofit")
```

Variable	desc_all	Sex f	Sex m	p
Ptt Lab	33 (28/35) [24 -> 54] [n = 27]	25 (24/49) [24 -> 54] [n = 3]	33 (29/35) [26 -> 40] [n = 24]	0.38
Ferritin Lab	222 (162/339) [20 -> 1182] [n = 27]	138 (90/591) [81 -> 681] [n = 3]	224 (172/316) [20 -> 1182] [n = 24]	0.63
Iron Lab	80 (61/102) [31 -> 191] [n = 27]	95 (92/103) [91 -> 105] [n = 3]	75 (60/99) [31 -> 191] [n = 24]	0.16
Transferrin Lab	261 (233/276) [194 -> 343] [n = 27]	260 (227/299) [221 -> 307] [n = 3]	262 (235/276) [194 -> 343] [n = 24]	0.98
Age	62 (53/67) [43 -> 74] [n = 28]	66 (58/69) [53 -> 69] [n = 4]	60 (53/66) [43 -> 74] [n = 24]	0.25

11.4 Categorial data

```
factvars$names
```

```
[1] "Testmedication" "Sex"            "NYHA V1"        "NYHA V2"
[5] "NYHA V3"
```

```
(crosstab <- table(rawdata$Sex, rawdata$Testmedication))
```

```
    0  1
  f  2  2
  m 12 12
```

```
chisq.test(crosstab, simulate.p.value = T, B = 10^5) # empirical p-value
```

```

    Pearson's Chi-squared test with simulated p-value (based on 1e+05
    replicates)

data:  crosstab
X-squared = 0, df = NA, p-value = 1
```

```
chisq.test(table(rawdata$Sex, rawdata$`NYHA V1`)) # based on table
```

```
Warning in chisq.test(table(rawdata$Sex, rawdata$`NYHA V1`)): Chi-squared
approximation may be incorrect
```

```

    Pearson's Chi-squared test

data:  table(rawdata$Sex, rawdata$`NYHA V1`)
X-squared = 4.3849, df = 3, p-value = 0.2228
```

```
chisq.test(
  x = rawdata$Sex, y = rawdata$`NYHA V1`,
  simulate.p.value = T, B = 10^5
) # based on rawdata
```

```

    Pearson's Chi-squared test with simulated p-value (based on 1e+05
    replicates)

data:  rawdata$Sex and rawdata$`NYHA V1`
X-squared = 4.3849, df = NA, p-value = 0.1757
```

```
fisher_out <- fisher.test(
  table(rawdata$Sex, rawdata$`NYHA V1`)
)
fisher_out$p.value
```

```
[1] 0.09558824
```

```
(crosstab1 <- table(
  rawdata$Sex,
  rawdata$`Weight (kg)` <=
    median(rawdata$`Weight (kg)`)
))
```

```
   FALSE TRUE
  f     1    3
  m    13   11
```

```
(tabletestOut <- chisq.test(crosstab1,
  simulate.p.value = T,
  B = 10^5
))
```

```
    Pearson's Chi-squared test with simulated p-value (based on 1e+05
    replicates)

data:  crosstab1
X-squared = 1.1667, df = NA, p-value = 0.5953
```

```
tabletestOut$p.value
```

```
[1] 0.595274
```

```
tabletestOut$expected
```

```
   FALSE TRUE
  f     2    2
  m    12   12
```

```
tabletestOut$observed
```

```
   FALSE TRUE
  f     1    3
  m    13   11
```

```
tabletestOut$statistic
```

```
X-squared
 1.166667
```

```
# if minimum(expected<5) then Fishers exact test
if (min(tabletestOut$expected) < 5) {
  tabletestOut <- fisher.test(crosstab1)
}
tabletestOut$p.value
```

```
[1] 0.5955556
```

```
# report_cat
groupvar <- "Testmedication"

rawdata |>
  mutate(Testmedication = factor(Testmedication,
    levels = 0:1,
    labels = c("Placebo", "Verum")
  )) |>
  compare2qualvars(,
    dep_vars = factvars$names[-1],
    indep_var = groupvar, spacer = " "
  ) |>
  rename_with(\(x) str_remove(x, "Testmedication ")) |>
  rename(`Total sample` = desc_all) |>
  flextable() |>
  align(~ p == " ", j = 1, align = "center") |>
  bg(~ p != " ", bg = "lightgrey")
```

Variable	Total sample	Placebo	Verum	p
Sex				1.000
f	4 (14.29%)	2 (14.29%)	2 (14.29%)	
m	24 (85.71%)	12 (85.71%)	12 (85.71%)	
NYHA V1				0.727

Variable	Total sample	Placebo	Verum	p
0	2 (11.76%)	0 (0%)	2 (20%)	
1	9 (52.94%)	4 (57.14%)	5 (50%)	
2	3 (17.65%)	1 (14.29%)	2 (20%)	
3	3 (17.65%)	2 (28.57%)	1 (10%)	
NYHA V2				0.828
0	1 (9.09%)	1 (20%)	0 (0%)	
1	6 (54.55%)	2 (40%)	4 (66.67%)	
2	2 (18.18%)	1 (20%)	1 (16.67%)	
3	2 (18.18%)	1 (20%)	1 (16.67%)	
NYHA V3				0.771
1	6 (50%)	2 (40%)	4 (57.14%)	
2	3 (25%)	1 (20%)	2 (28.57%)	
3	3 (25%)	2 (40%)	1 (14.29%)	

11.5 Exercise

Compare the 4 measurements in the penguin data between the sex as well as between 2 species (e.g Adelie vs. Gentoo). Treat the measures first as gaussian, then as ordinal.

Use the basic functions first for at least one phenotype before trying out compare2numvars().

Look at the categorical data year, island, species and sex and run some tests on their independence.

Advanced: Visualize significances, possibly in a loop.

12 Covariance / Correlation

```
pacman::p_load(
  conflicted, tidyverse, wrappedtools, here,
  coin, ggsignif, patchwork, ggbeeswarm,
  flextable, corrr, correlation, corrplot
)
load(here("data/bookdata1.RData"))
```

12.1 Covariance

Covariance is a measure of the relationship between two random variables. The sign of the covariance shows the tendency in the linear relationship between the variables. The covariance between two variables is computed as follows:

```
cov(rawdata$`Weight (kg)`, rawdata$`Size (cm)`)
```

```
[1] 51.55291
```

```
cov(rawdata |> select(contains("BP")),
  use = "pairwise.complete.obs"
)
```

```
            sysBP V0  diaBP V0  sysBP V2    diaBP V2
sysBP V0 285.4387464  48.76211 102.23551  -0.5144928
diaBP V0  48.7621083  95.06268 -38.07065  28.7065217
sysBP V2 102.2355072 -38.07065 160.78080 -10.6050725
diaBP V2  -0.5144928  28.70652 -10.60507  74.5144928
```

12.2 Correlation

Correlation is a measure of the strength and direction of the linear relationship between two variables. Other than the covariance, it is standardized, so it is not affected by the scale of the variables. The correlation between two variables is computed as follows:

```
cor(rawdata$`Weight (kg)`, rawdata$`Size (cm)`)
```

```
[1] 0.454551
```

```
cor(rawdata |> select(contains("Mri")),
  use = "pairwise.complete.obs"
)
```

```
                Lv Edv Mri Lv Esv Mri  Lv Ef Mri Lv Ef Biplan Mri
Lv Edv Mri       1.0000000  0.9180259 -0.6653199       -0.6022713
Lv Esv Mri       0.9180259  1.0000000 -0.8913242       -0.8375638
Lv Ef Mri       -0.6653199 -0.8913242  1.0000000        0.9544592
Lv Ef Biplan Mri -0.6022713 -0.8375638  0.9544592        1.0000000
```

```
cor(rawdata |> select(contains("Mri")),
  use = "pairwise.complete.obs",
  method = "spearman"
)
```

```
                Lv Edv Mri Lv Esv Mri  Lv Ef Mri Lv Ef Biplan Mri
Lv Edv Mri       1.0000000  0.8530566 -0.4765356       -0.3944301
Lv Esv Mri       0.8530566  1.0000000 -0.7950419       -0.7357709
Lv Ef Mri       -0.4765356 -0.7950419  1.0000000        0.9614422
Lv Ef Biplan Mri -0.3944301 -0.7357709  0.9614422        1.0000000
```

```
corrr::correlate(rawdata |> select(contains("Mri")))
```

```
Correlation computed with
* Method: 'pearson'
* Missing treated using: 'pairwise.complete.obs'
```

```
# A tibble: 4 x 5
  term             `Lv Edv Mri` `Lv Esv Mri` `Lv Ef Mri` `Lv Ef Biplan Mri`
  <chr>                   <dbl>        <dbl>       <dbl>              <dbl>
1 Lv Edv Mri             NA            0.918      -0.665             -0.602
2 Lv Esv Mri              0.918       NA          -0.891             -0.838
3 Lv Ef Mri              -0.665       -0.891      NA                  0.954
4 Lv Ef Biplan Mri       -0.602       -0.838       0.954             NA
```

Significance of correlations can be tested:

```
cor.test(rawdata$`Weight (kg)`, rawdata$`Size (cm)`)
```

```
    Pearson's product-moment correlation

data:  rawdata$`Weight (kg)` and rawdata$`Size (cm)`
t = 2.6021, df = 26, p-value = 0.0151
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.09811224 0.70762684
sample estimates:
     cor
0.454551
```

```
wrappedtools::cortestR(rawdata |> select(contains("BP")))
```

```
           sysBP V0   diaBP V0   sysBP V2 diaBP V2
sysBP V0          1
diaBP V0  0.296n.s.          1
sysBP V2     0.455* -0.294n.s.          1
diaBP V2 -0.003n.s.  0.326n.s. -0.097n.s.        1
```

```
wrappedtools::cortestR(rawdata |> select(contains("BP")),
  split = TRUE
)
```

```
$corout
         sysBP V0 diaBP V0 sysBP V2 diaBP V2
sysBP V0        1
diaBP V0    0.296        1
sysBP V2    0.455   -0.294        1
diaBP V2   -0.003    0.326   -0.097        1

$pout
         sysBP V0 diaBP V0 sysBP V2 diaBP V2
sysBP V0      ***
diaBP V0     n.s.      ***
sysBP V2        *     n.s.      ***
diaBP V2     n.s.     n.s.     n.s.      ***
```

```
cor_out <-
  wrappedtools::cortestR(rawdata |> select(contains("BP")),
    split = TRUE, sign_symbol = FALSE
  )
cor_out
```

```
$corout
         sysBP V0 diaBP V0 sysBP V2 diaBP V2
sysBP V0        1
diaBP V0    0.296        1
sysBP V2    0.455   -0.294        1
diaBP V2   -0.003    0.326   -0.097        1

$pout
         sysBP V0 diaBP V0 sysBP V2 diaBP V2
sysBP V0    0.001
diaBP V0    0.134    0.001
sysBP V2    0.025    0.163    0.001
diaBP V2    0.988    0.121    0.652    0.001
```

```
correlation::correlation(rawdata |> select(contains("Mri")))
```

```
# Correlation Matrix (pearson-method)

Parameter1 |       Parameter2 |     r |         95% CI |     t | df |         p
-------------------------------------------------------------------------------
Lv Edv Mri |       Lv Esv Mri |  0.92 | [ 0.81,  0.97] | 10.09 | 19 | < .001***
Lv Edv Mri |        Lv Ef Mri | -0.67 | [-0.85, -0.33] | -3.88 | 19 | 0.002**
Lv Edv Mri | Lv Ef Biplan Mri | -0.60 | [-0.82, -0.22] | -3.20 | 18 | 0.005**
Lv Esv Mri |        Lv Ef Mri | -0.89 | [-0.96, -0.75] | -8.57 | 19 | < .001***
Lv Esv Mri | Lv Ef Biplan Mri | -0.84 | [-0.93, -0.63] | -6.50 | 18 | < .001***
Lv Ef Mri  | Lv Ef Biplan Mri |  0.95 | [ 0.89,  0.98] | 13.57 | 18 | < .001***

p-value adjustment method: Holm (1979)
Observations: 20-21
```

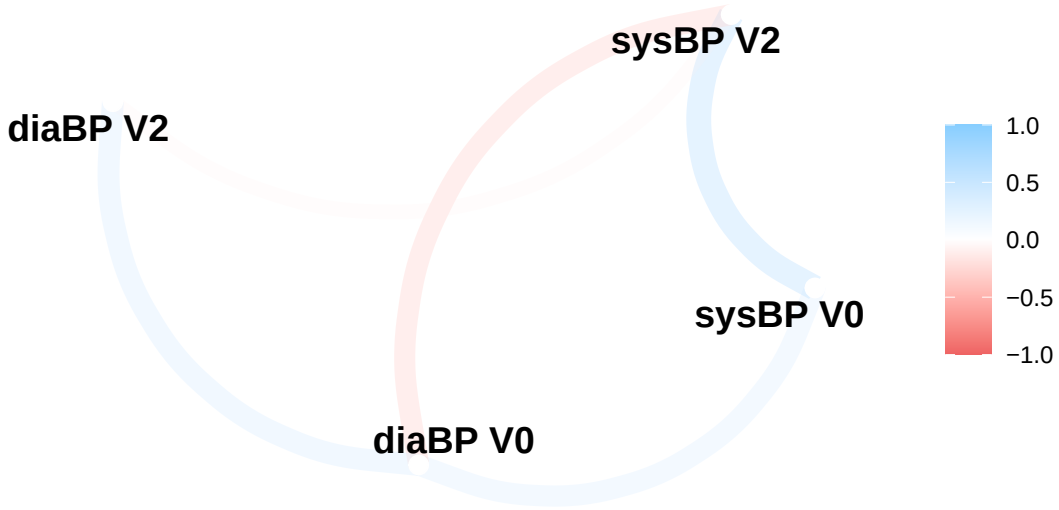
12.3 Vizualizations

```
corrr::correlate(rawdata |> select(contains("BP"))) |>
  corrr::network_plot(min_cor = .05)
```

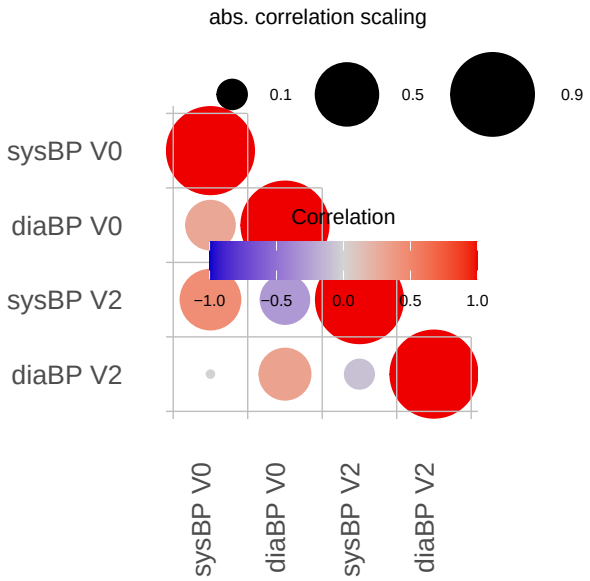
```
Correlation computed with
* Method: 'pearson'
* Missing treated using: 'pairwise.complete.obs'
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
```

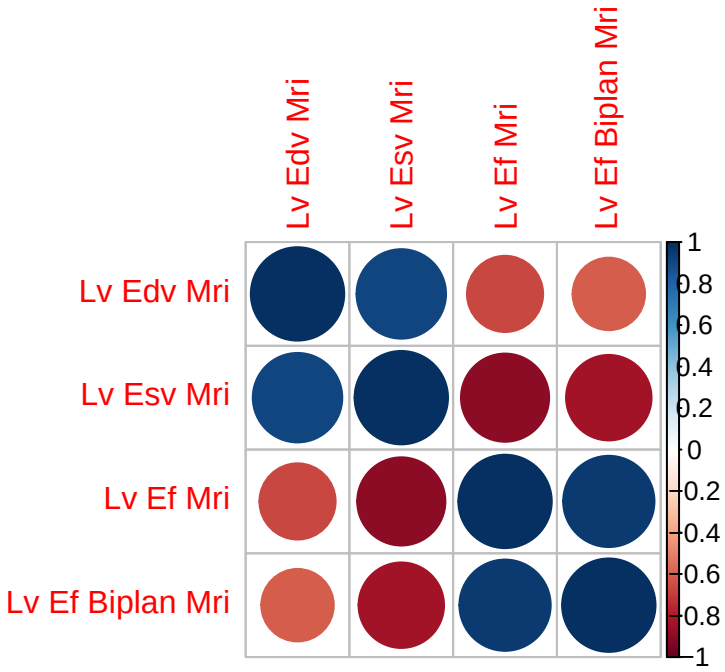
```
i Please use `linewidth` instead.
i The deprecated feature was likely used in the corrr package.
  Please report the issue at <https://github.com/tidymodels/corrr/issues>.
```



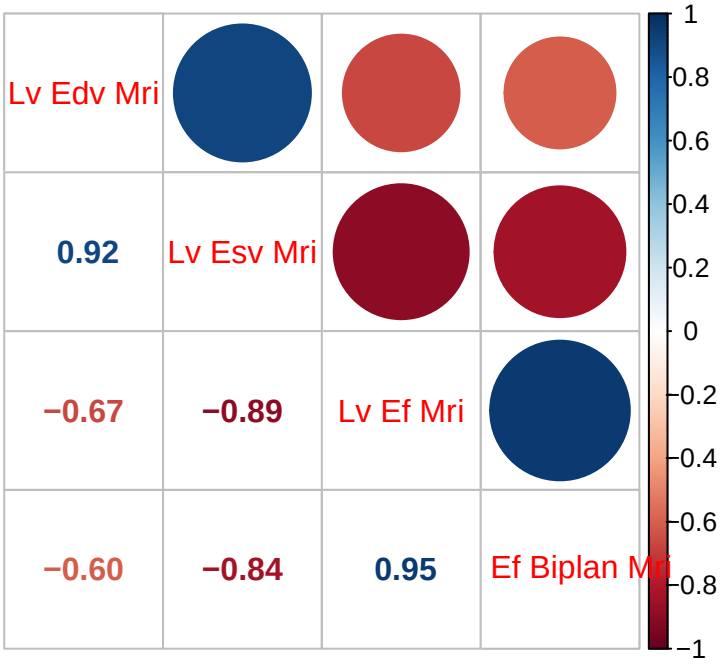

```
wrappedtools::ggcormat(
  cor_mat = cor_out$corout,
  maxpoint = 15
)
```




```
corrplot::corrplot(cor(rawdata |> select(contains("Mri")),
                       use = "pairwise.complete.obs"
))
```



```
corrplot::corrplot.mixed(cor(rawdata |> select(contains("Mri")),
                         use = "pairwise.complete.obs"
))
```




```
GGally::ggpairs(rawdata |> select(contains("BP")))
```

```
Warning: Removed 1 row containing non-finite outside the scale range
(`stat_density()`).
```

```
Warning: Removing 1 row that contained a missing value
```

```
Warning: Removed 4 rows containing missing values
Removed 4 rows containing missing values
```

```
Warning: Removed 1 row containing missing values or values outside the scale range
(`geom_point()`).
```

```
Warning: Removed 1 row containing non-finite outside the scale range
(`stat_density()`).
```

```
Warning: Removed 4 rows containing missing values
Removed 4 rows containing missing values
```

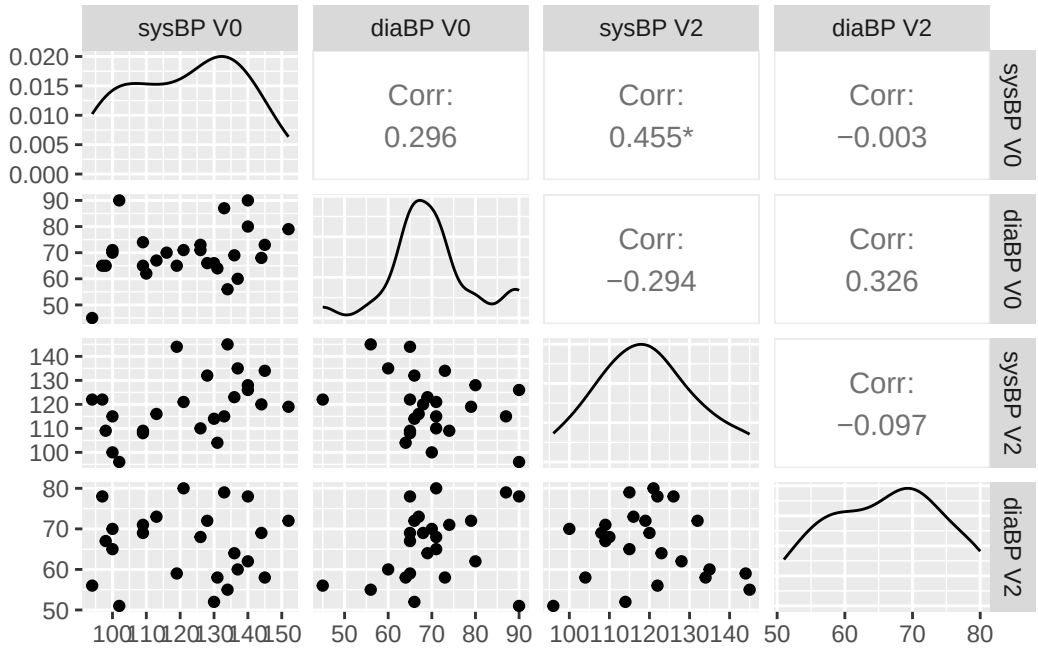
```
Warning: Removed 4 rows containing missing values or values outside the scale range
(`geom_point()`).
Removed 4 rows containing missing values or values outside the scale range
(`geom_point()`).

Warning: Removed 4 rows containing non-finite outside the scale range
(`stat_density()`).

Warning: Removed 4 rows containing missing values

Warning: Removed 4 rows containing missing values or values outside the scale range
(`geom_point()`).
Removed 4 rows containing missing values or values outside the scale range
(`geom_point()`).
Removed 4 rows containing missing values or values outside the scale range
(`geom_point()`).

Warning: Removed 4 rows containing non-finite outside the scale range
(`stat_density()`).
```



13 Intro to linear models

In this chapter, linear models (including linear regression and ANOVA) will be introduced. Output is not optimized for print, but rather for interactive use.

13.1 Setup

All packages necessary will be invoked by p_load. Packages with only a single function call or potential for name conflicts can be unloaded, this way we still checked for their existence and installed them if need be.

```
pacman::p_load(
  conflicted, tidyverse, wrappedtools, here,
  car, nlme, broom, flextable,
  multcomp,  foreign, DescTools,
  ggbeeswarm,
  lme4, nlme, merTools,
  easystats, patchwork
)
# pacman::p_unload(DescTools, foreign)
# conflict_scout()
conflicts_prefer(
  dplyr::select,
  dplyr::filter,
  modelbased::standardize
)
```

13.2 Import / Preparation

Data are read from an SPSS file. Numeric column Passage is mutated into a factor as Passage_F, this is necessary for group comparisons in ANOVA. The call to here() expands the path to a file from the project directory to the full system path.

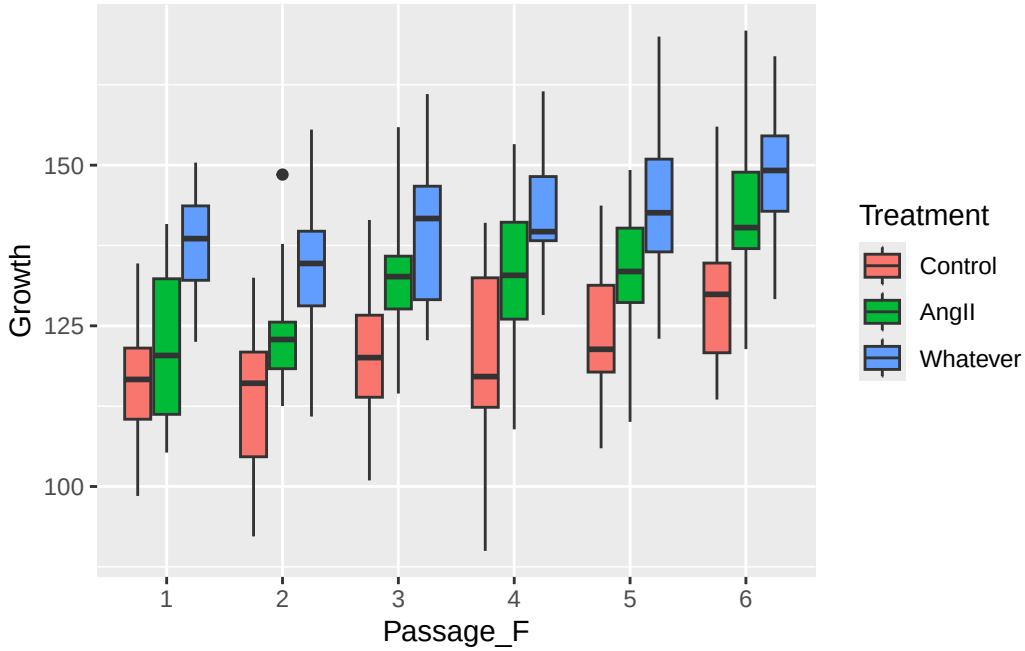
```
rawdata <- foreign::read.spss( # the acual import
  file = here("data/Zellbeads.sav"),
  use.value.labels = T, to.data.frame = T
) |>
  as_tibble() |> # adjustment of structure: data-frame to tibble
  dplyr::select(-ZahlZellen) |> # unselecting unused cols
  rename(Growth = Wachstum, Treatment = Bedingung) |> # cosmetic changes
  mutate(  # adjustment of content :
    Passage_F = factor(Passage),  # class change
    Treatment = fct_recode(Treatment, # factor-level change
      Control = "Kontrolle"
    )
  )
```

```
re-encoding from CP1252
```

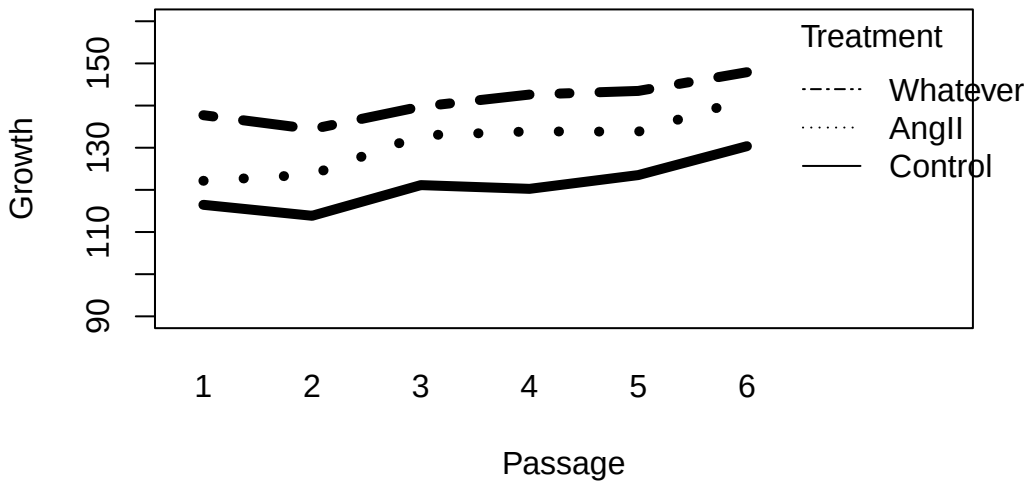
13.3 Graphical exploration

First impression of the data will be attempted by grouped boxplot, followed by interaction plots, both as basic and ggplot with variations.

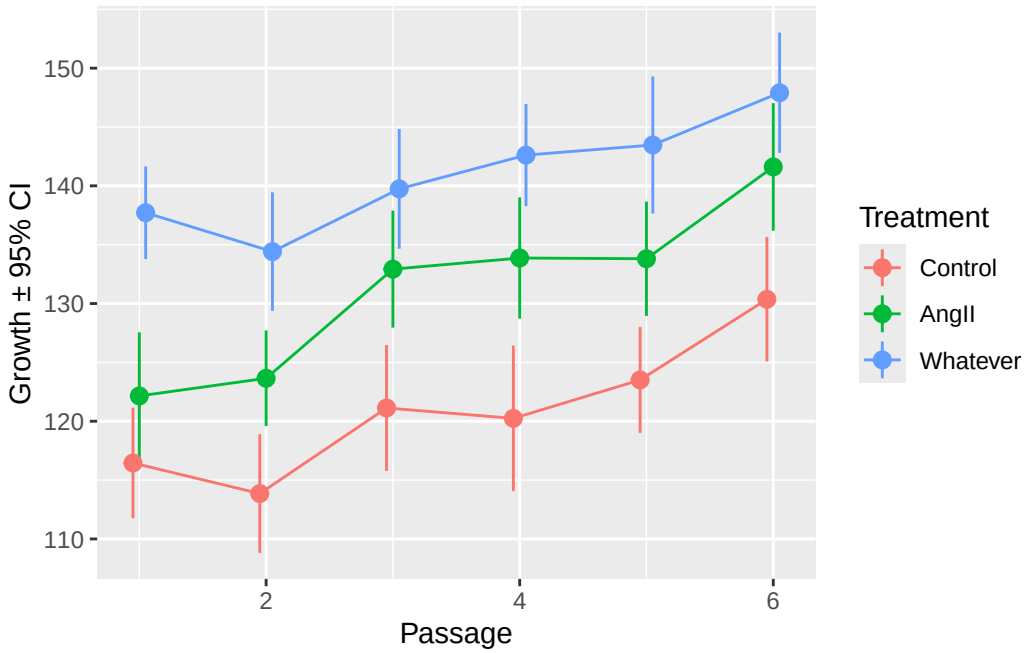
```
ggplot(rawdata, aes(Passage_F, Growth, fill = Treatment)) +
  geom_boxplot(coef = 3)
```




```
with(rawdata, interaction.plot(
  x.factor = Passage, trace.factor = Treatment, response = Growth,
  ylim = c(90, 160), lty = c(1, 3, 12), lwd = 5,
  ylab = "Growth", xlab = "Passage",
  trace.label = "Treatment"
))
```

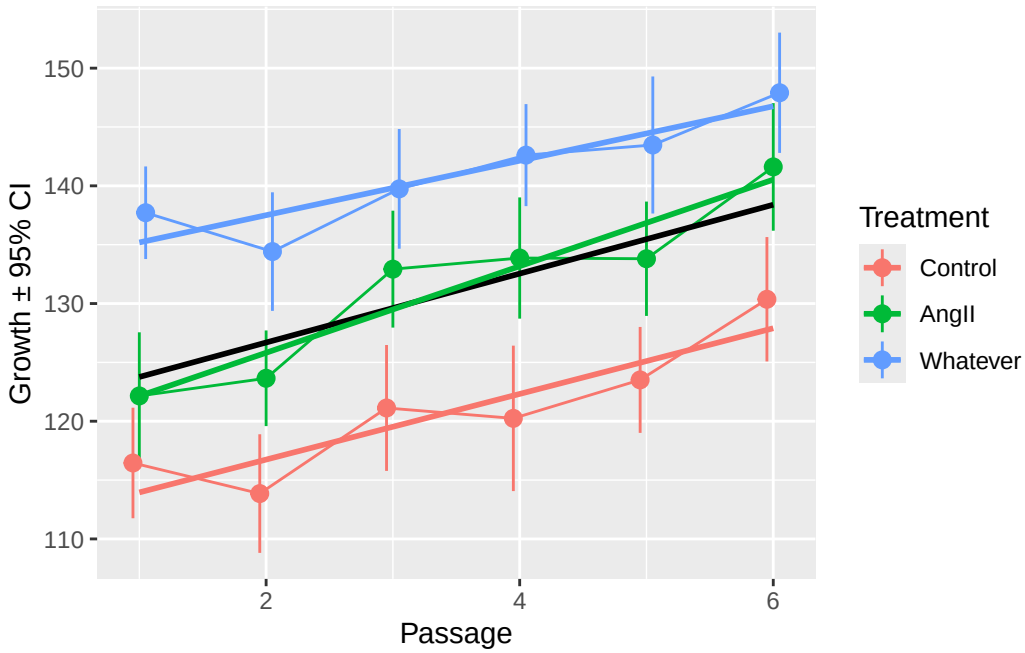



```
p1 <- ggplot(rawdata, aes(x = Passage, y = Growth)) +
  stat_summary(
    geom = "line", fun = "mean",
    aes(color = Treatment),
    position = position_dodge(width = .15)
  ) +
  stat_summary(aes(color = Treatment),
    position = position_dodge(width = .15),
    fun.data = "mean_cl_normal"
  ) +
  ylab("Growth \u00b1 95% CI")
p1
```



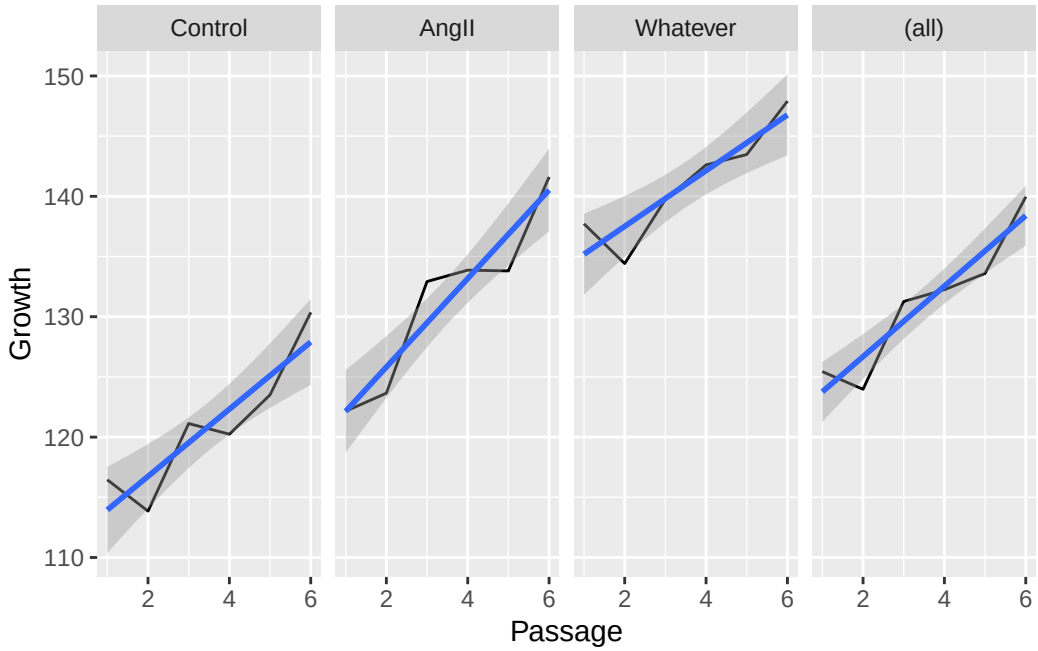

```
p1 + geom_smooth(method = "lm", color = "black", se = F) +
  geom_smooth(method = "lm", aes(color = Treatment), se = F)
```

```
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
```




```
ggplot(rawdata, aes(x = Passage, y = Growth)) +
  stat_summary(geom = "line", fun = "mean") +
  geom_smooth(method = "lm") +
  facet_grid(cols = vars(Treatment), margins = T)
```

```
`geom_smooth()` using formula = 'y ~ x'
```



13.4 Linear Models

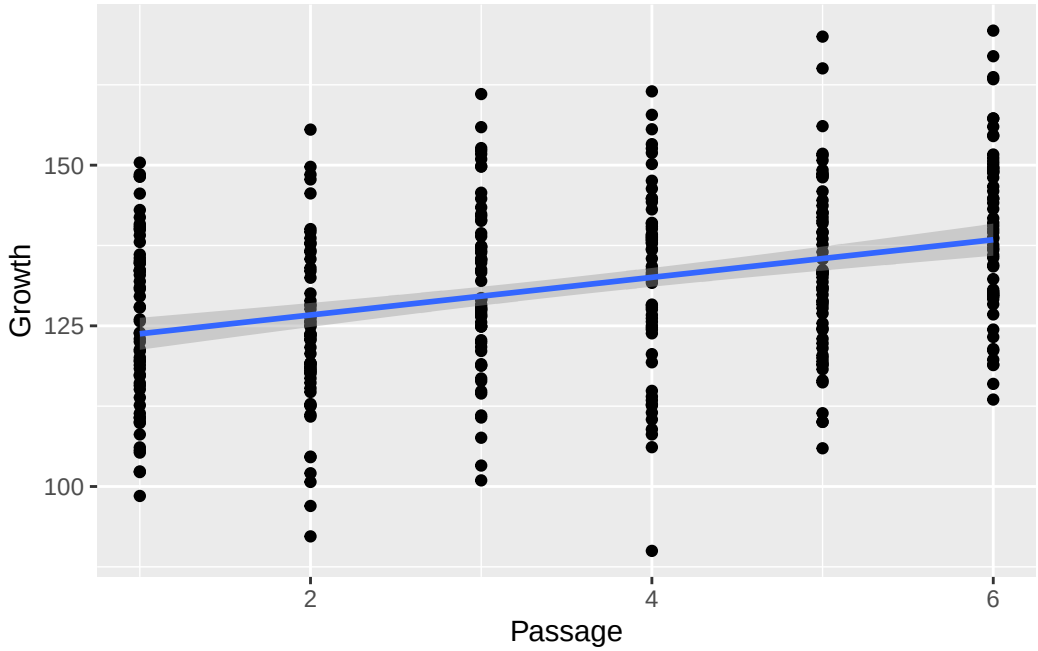
13.4.1 Linear regression

We will analyse the relation between dependent variable (DV) Growth and CONTINUOUS NUMERICAL independent variable (IV) Passage. This is traditionally called regression, here it is rather a regression-like linear model.

13.4.1.1 Graphical exploration

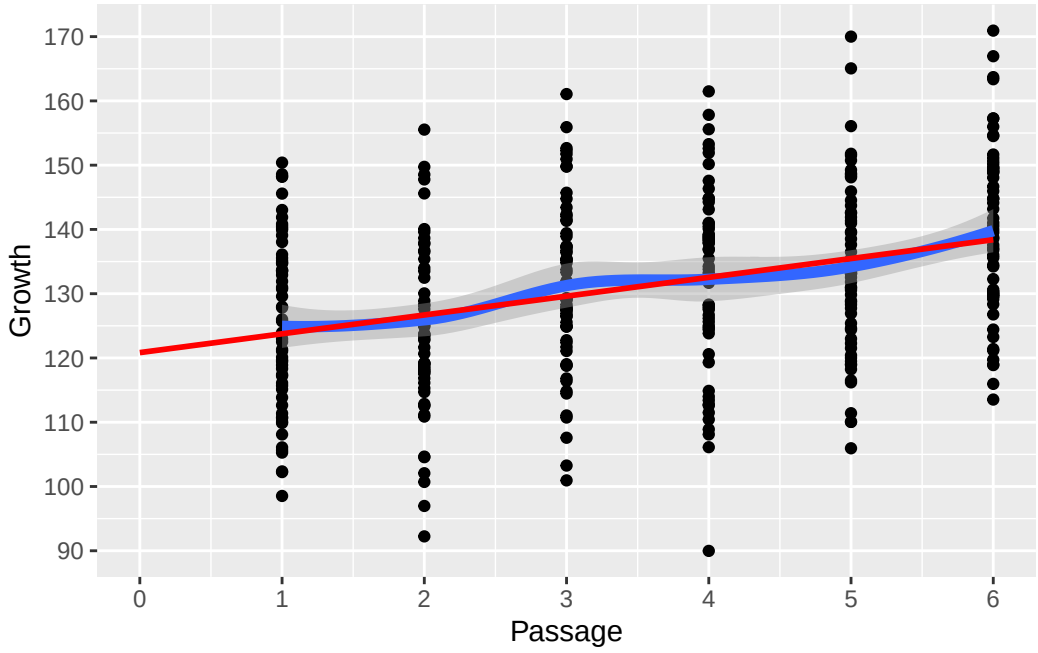
```
ggplot(rawdata, aes(Passage, Growth)) +
  geom_point() +
  geom_smooth(method = lm)
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
ggplot(rawdata, aes(Passage, Growth)) +
  geom_point() +
  scale_x_continuous(breaks = seq(0, 10, 1), limits = c(0, 6)) +
  scale_y_continuous(breaks = seq(0,1000,10)) +
  geom_smooth(linewidth = 2) +
  geom_smooth(method = lm, se = F, color = "red", fullrange = TRUE)
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
```



13.4.1.2 Modelling

This takes 2 steps, building the model and computing p-values.

```
# model
(regressionOut <- lm(Growth ~ Passage, data = rawdata))
```

```
Call:
lm(formula = Growth ~ Passage, data = rawdata)

Coefficients:
(Intercept)      Passage
    120.834        2.927
```

```
regressionOut$coefficients
```

```
(Intercept)     Passage
 120.833844    2.927085
```

```
# model and p.value for slope, not recommended
tidy(regressionOut)
```

```
# A tibble: 2 x 5
  term        estimate std.error statistic   p.value
  <chr>          <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)   121.       1.63      74.2  1.77e-219
2 Passage         2.93     0.418      7.00 1.26e- 11
```

```
# computation of SSQs and p-values, use this!
(anova_out <- anova(regressionOut))
```

```
Analysis of Variance Table

Response: Growth
           Df Sum Sq Mean Sq F value    Pr(>F)
Passage     1   8996  8996.2  49.022 1.257e-11 ***
Residuals 358  65698   183.5
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
anova_out$`Pr(>F)` #|> na.omit()
```

```
[1] 1.257266e-11           NA
```

```
tidy(anova_out)
```

```
# A tibble: 2 x 6
  term         df  sumsq meansq statistic   p.value
  <chr>     <int>  <dbl>  <dbl>     <dbl>     <dbl>
1 Passage       1  8996.  8996.      49.0  1.26e-11
2 Residuals   358 65698.   184.      NA   NA
```

```
summary(regressionOut)
```

```

Call:
lm(formula = Growth ~ Passage, data = rawdata)

Residuals:
    Min      1Q  Median      3Q     Max
-42.561  -8.646  -0.556   9.811  34.533

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept) 120.8338     1.6281  74.217  < 2e-16 ***
Passage       2.9271     0.4181   7.002 1.26e-11 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 13.55 on 358 degrees of freedom
Multiple R-squared:  0.1204,    Adjusted R-squared:  0.118
F-statistic: 49.02 on 1 and 358 DF,  p-value: 1.257e-11
```

```
# str(regressionOut)
```

13.4.1.3 Adjusting

To take out the variance due to Passage effects (passage-adjusted growth), we can use the residuals and shift them to the original mean:

```
rawdata <-
  mutate(rawdata,
    growthAdj = regressionOut$residuals + mean(Growth)
  )

summarise(
  rawdata,
  across(
    contains("growth"),
    \(x) meansd(x, roundDig = 4)
  )
)
```

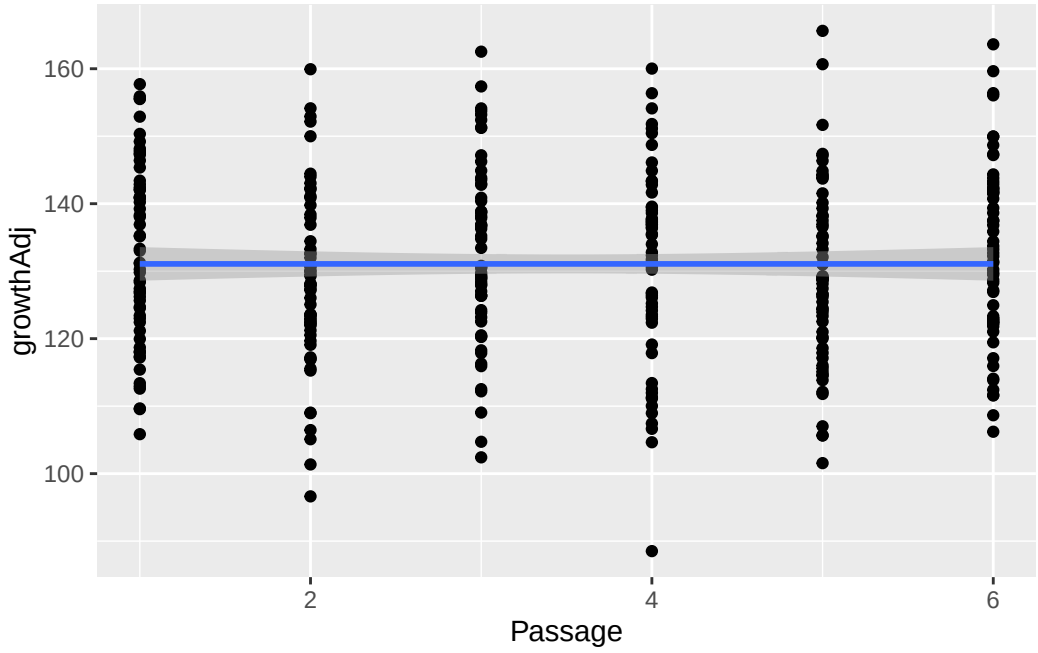
```
# A tibble: 1 x 2
  Growth       growthAdj
  <chr>        <chr>
1 131.1 ± 14.4 131.1 ± 13.5
```

```
summarise(
  rawdata,
  across(
    contains("growth"),
    \(x) meanse(x, roundDig = 4)
  )
)
```

```
# A tibble: 1 x 2
  Growth      growthAdj
  <chr>       <chr>
1 131.1 ± 0.8 131.1 ± 0.7
```

```
ggplot(rawdata, aes(Passage, growthAdj)) +
  geom_point() +
  geom_smooth(method = "lm")
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
lm(growthAdj ~ Passage, data = rawdata) |> tidy()
```

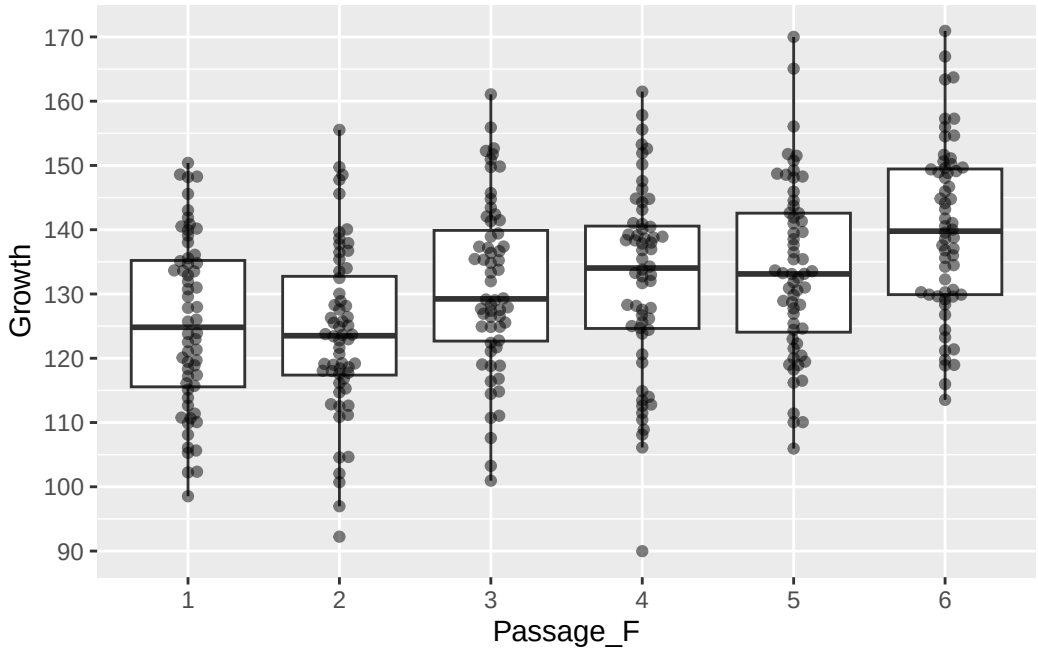
```
# A tibble: 2 x 5
  term        estimate std.error statistic    p.value
  <chr>          <dbl>     <dbl>     <dbl>      <dbl>
1 (Intercept) 1.31e+ 2     1.63   8.05e+ 1 2.04 e-231
2 Passage     6.80e-15     0.418  1.63e-14 1.000e+  0
```

13.4.2 ANOVA

In the linear regression, we had Passage as a continuous IV, estimating a global 'universal' effect supposed to be constant. Now we look at Passage_F and thus model a discrete IV, allowing for specific effects, and thereby comparing means between groups. So this is an ANOVA-like linear model.

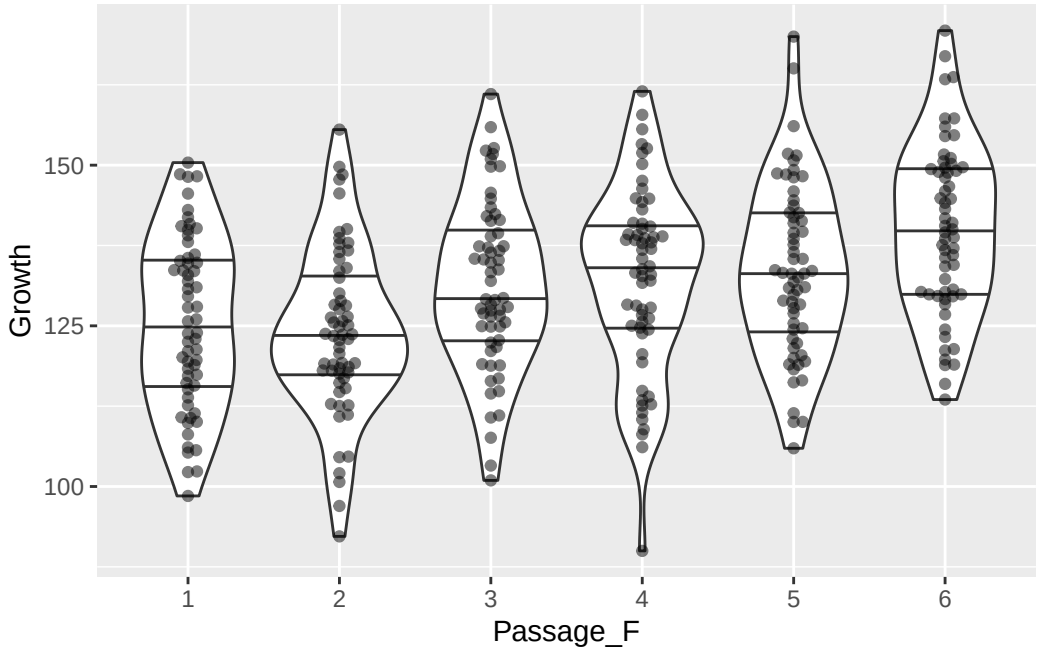
13.4.2.1 Graphical exploration

```
ggplot(rawdata, aes(x = Passage_F, y = Growth)) +
  geom_boxplot(outlier.alpha = 0) +
  geom_beeswarm(alpha = .5) +
  scale_y_continuous(breaks = seq(0, 1000, 10))
```




```
ggplot(rawdata, aes(x = Passage_F, y = Growth)) +
  geom_violin(draw_quantiles = c(.25, .5, .75)) +
  geom_beeswarm(alpha = .5)
```

```
Warning: The `draw_quantiles` argument of `geom_violin()` is deprecated as of ggplot2
4.0.0.
i Please use the `quantiles.linetype` argument instead.
```



13.4.2.2 Modelling

```
(AnovaOut <- lm(Growth ~ Passage_F, data = rawdata))
```

```
Call:
lm(formula = Growth ~ Passage_F, data = rawdata)

Coefficients:
(Intercept)   Passage_F2   Passage_F3   Passage_F4   Passage_F5   Passage_F6
    125.440       -1.467        5.824        6.801        8.156       14.520
```

```
tidy(AnovaOut)
```

```
# A tibble: 6 x 5
  term        estimate std.error statistic   p.value
  <chr>          <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)   125.        1.74    71.9   2.20e-213
2 Passage_F2     -1.47      2.47    -0.595 5.52e-  1
3 Passage_F3      5.82      2.47     2.36  1.87e-  2
4 Passage_F4      6.80      2.47     2.76  6.11e-  3
5 Passage_F5      8.16      2.47     3.31  1.04e-  3
6 Passage_F6     14.5       2.47     5.89  9.03e-  9
```

```
# summary(AnovaOut)
(t <- anova(AnovaOut))
```

```
Analysis of Variance Table

Response: Growth
           Df Sum Sq Mean Sq F value    Pr(>F)
Passage_F   5  10134 2026.71  11.113 5.852e-10 ***
Residuals 354  64561  182.38
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
t$`Pr(>F)`
```

```
[1] 5.851856e-10           NA
```

```
tidy(t)
```

```
# A tibble: 2 x 6
  term         df  sumsq meansq statistic   p.value
  <chr>     <int>  <dbl>  <dbl>     <dbl>     <dbl>
1 Passage_F     5 10134.  2027.      11.1  5.85e-10
2 Residuals   354 64561.   182.      NA   NA
```

13.4.2.3 Model comparison ANOVA-like against regression-like

```
anova(regressionOut, AnovaOut)
```

```
Analysis of Variance Table

Model 1: Growth ~ Passage
Model 2: Growth ~ Passage_F
  Res.Df   RSS Df Sum of Sq      F Pr(>F)
1    358 65698
2    354 64561  4    1137.4 1.5591 0.1847
```

13.4.2.4 Post-hoc analyses

The p-value from our model only tests the global Null hypothesis of no differences between any group (all means are the same / all groups come from the same population). Post-hoc tests are used to figure out which groups are different. Those tests need to take multiple testing into account. Try to limit selection of tests!

```
# possible in a loop, but nominal p
t.test(rawdata$Growth[which(rawdata$Passage == 1)],
  rawdata$Growth[which(rawdata$Passage == 2)],
  var.equal = T
)
```

```
    Two Sample t-test

data:  rawdata$Growth[which(rawdata$Passage == 1)] and rawdata$Growth[which(rawdata$Passage =
t = 0.60679, df = 118, p-value = 0.5452
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -3.321297  6.255936
sample estimates:
mean of x mean of y
 125.4396  123.9723
```

```
# all pairwise group combinations
pt_out <- pairwise.t.test(
  x = rawdata$Growth,
  g = rawdata$Passage_F,
  p.adjust.method = "none"
)
pt_out
```

```
    Pairwise comparisons using t tests with pooled SD

data:  rawdata$Growth and rawdata$Passage_F

  1       2       3       4       5
2 0.55215 -       -       -       -
3 0.01871 0.00331 -       -       -
4 0.00611 0.00088 0.69214 -       -
```

```
5 0.00104 0.00011 0.34487 0.58296 -
6 9e-09   3e-10   0.00048 0.00189 0.01025

P value adjustment method: none
```

```
pairwise.t.test(
  x = rawdata$Growth, g = rawdata$Passage_F,
  p.adjust.method = "fdr"
)
```

```
    Pairwise comparisons using t tests with pooled SD

data:  rawdata$Growth and rawdata$Passage_F

  1       2       3       4       5
2 0.62460 -       -       -       -
3 0.02552 0.00621 -       -       -
4 0.01018 0.00259 0.69214 -       -
5 0.00259 0.00057 0.43109 0.62460 -
6 6.8e-08 4.5e-09 0.00178 0.00405 0.01538

P value adjustment method: fdr
```

```
pairwise.t.test(
  x = rawdata$Growth, g = rawdata$Passage_F,
  p.adjust.method = "bonferroni"
)
```

```
    Pairwise comparisons using t tests with pooled SD

data:  rawdata$Growth and rawdata$Passage_F

  1       2       3      4      5
2 1.0000  -       -      -      -
3 0.2807  0.0497  -      -      -
4 0.0917  0.0133  1.0000 -      -
5 0.0155  0.0017  1.0000 1.0000 -
6 1.4e-07 4.5e-09 0.0071 0.0283 0.1538

P value adjustment method: bonferroni
```

```
# comparison against reference group 1
pt_out$p.value[, 1]
```

```
           2            3            4            5            6
5.521460e-01 1.871115e-02 6.110172e-03 1.036173e-03 9.031123e-09
```

```
# comparison against reference group 6
pt_out$p.value[5, ]
```

```
           1            2            3            4            5
9.031123e-09 3.001066e-10 4.757018e-04 1.889098e-03 1.025037e-02
```

```
# comparison for selection
c(
  pt_out$p.value[1, 1], pt_out$p.value[3, 2],
  pt_out$p.value[5, 1]
)
```

```
[1] 5.521460e-01 8.842382e-04 9.031123e-09
```

```
# comparison against next level
diag(pt_out$p.value)
```

```
[1] 0.55214600 0.00331248 0.69214393 0.58295615 0.01025037
```

```
# adjusting for multiple testing for selected comparisons
p.adjust(diag(pt_out$p.value), method = "fdr")
```

```
[1] 0.69214393 0.01656240 0.69214393 0.69214393 0.02562592
```

```
formatP(p.adjust(pt_out$p.value[, 1], method = "fdr"))
```

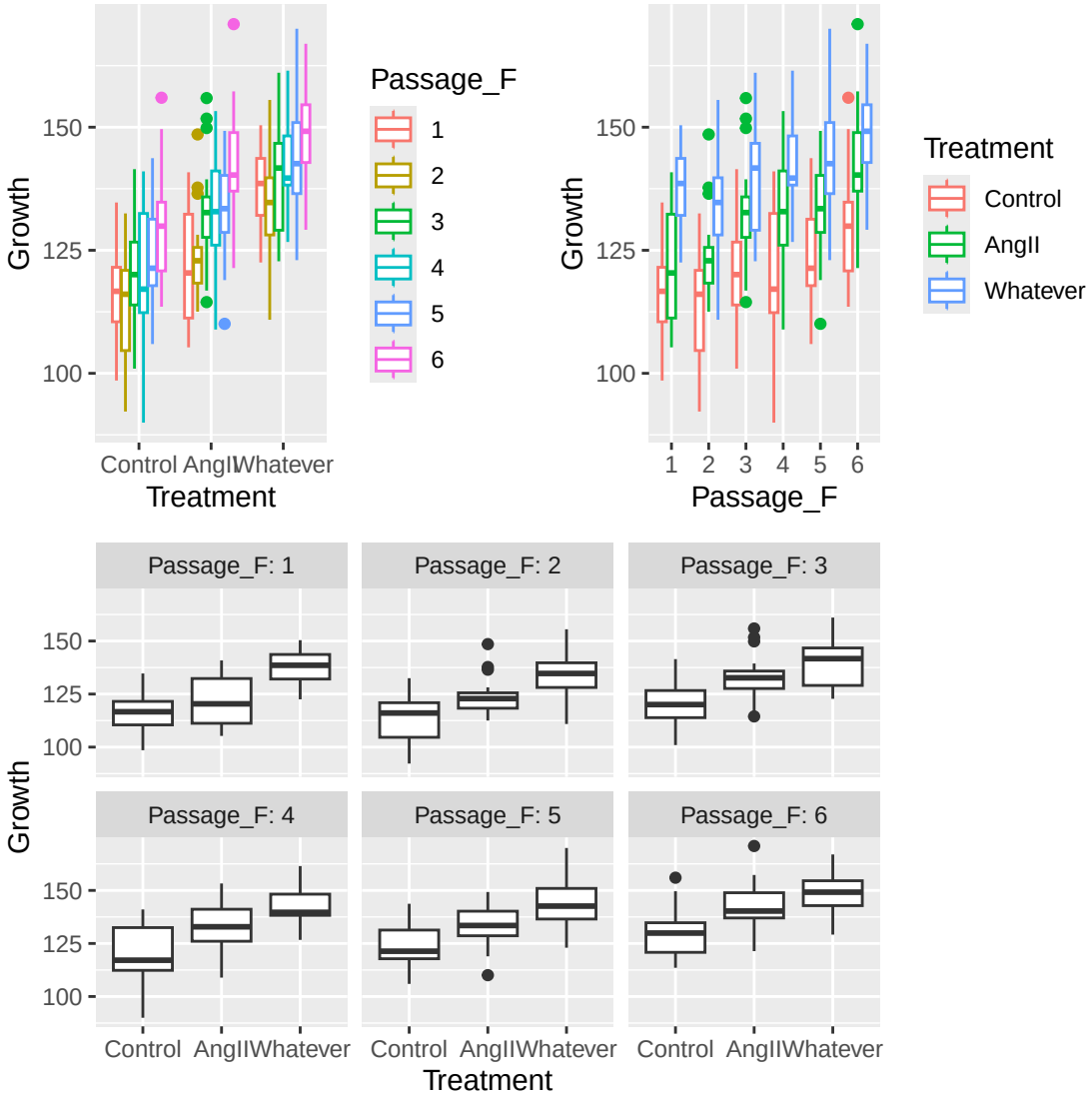
```
[1] "0.552" "0.023" "0.010" "0.003" "0.001"
```

13.4.3 LM with continuous AND categorical IV

Traditionally you may think of *regression* **OR** *ANOVA*, but they are no different and can be combined. This is called a general linear model. Multivariable models may contain interactions between independent variables V IV1*IV2.

13.4.3.1 Graphical exploration

```
p0 <- ggplot(rawdata, aes(Treatment, Growth)) +
  geom_boxplot()
p1 <- ggplot(rawdata, aes(Treatment, Growth, color = Passage_F)) +
  geom_boxplot()
p2 <- ggplot(rawdata, aes(color = Treatment, Growth, x = Passage_F)) +
  geom_boxplot()
p3 <- ggplot(rawdata, aes(Treatment, Growth)) +
  geom_boxplot() +
  facet_wrap(facets = vars(Passage_F), labeller = "label_both")
# from patchwork
(p1 + p2) / p3
```



13.4.3.2 Modelling

Models with (*) and without (+) interaction are build and tested.

```
lmOut_interaction <- lm(Growth ~ Passage * Treatment, data = rawdata)
Anova(lmOut_interaction, type = 3)
```

```
Anova Table (Type III tests)
```

```
Response: Growth
                  Sum Sq  Df   F value    Pr(>F)
(Intercept)       285160   1 2448.5613 < 2.2e-16 ***
Passage             2723   1   23.3855 1.981e-06 ***
Treatment           5635   2   24.1924 1.419e-10 ***
Passage:Treatment    335   2    1.4376    0.2389
Residuals          41227 354
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
#
lmOut_additive <- lm(Growth ~ Passage + Treatment, data = rawdata)
Anova_out <- Anova(lmOut_additive, type = 2)
Anova_out$`Pr(>F)`
```

```
[1] 7.051564e-17 4.006053e-36           NA
```

```
tidy(Anova_out)
```

```
# A tibble: 3 x 5
  term       sumsq    df statistic   p.value
  <chr>      <dbl> <dbl>     <dbl>     <dbl>
1 Passage    8996.     1      77.1  7.05e-17
2 Treatment 24137.     2     103.   4.01e-36
3 Residuals 41562.   356      NA   NA
```

```
lmOut_univ <- lm(growthAdj ~ Treatment, data = rawdata)
anova(lmOut_univ) |> tidy()
```

```
# A tibble: 2 x 6
  term         df  sumsq meansq statistic   p.value
  <chr>     <int>  <dbl>  <dbl>     <dbl>     <dbl>
1 Treatment     2 24137. 12068.      104.  3.19e-36
2 Residuals   357 41562.   116.       NA  NA
```

```
# for comparison, here is the univariable model
lmOut_uni <- lm(Growth ~ Treatment, data = rawdata)
a_uni <- anova(lmOut_uni)
a_uni$`Pr(>F)`
```

```
[1] 5.549803e-31           NA
```

13.4.3.3 Post-hoc analyses

For multivariable models, pairwise.t.test() is not appropriate, Dunnet or Tukey tests (depending on hypothesis) are typical solutions.

```
glht_out <-
  summary(glht(
    model = lmOut_additive,
    linfct = mcp(Treatment = "Dunnett")
  ))
glht_out$test$pvalues
```

```
[1] 1.316836e-12 5.551115e-16
attr(,"error")
[1] 1e-15
```

```
tidy(glht_out) |>
  select(-null.value)
```

```
# A tibble: 2 x 6
  term      contrast           estimate std.error statistic adj.p.value
  <chr>     <chr>                 <dbl>     <dbl>     <dbl>       <dbl>
1 Treatment AngII - Control        10.4      1.39      7.46    1.32e-12
2 Treatment Whatever - Control     20.1      1.39     14.4     5.55e-16
```

```
summary(glht(
  model = lmOut_additive,
  linfct = mcp(Treatment = "Tukey")
))
```

```
     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: Tukey Contrasts


Fit: lm(formula = Growth ~ Passage + Treatment, data = rawdata)

Linear Hypotheses:
                         Estimate Std. Error t value Pr(>|t|)
AngII - Control == 0       10.409      1.395   7.462   <1e-10 ***
Whatever - Control == 0    20.052      1.395  14.375   <1e-10 ***
```

```
Whatever - AngII == 0      9.643      1.395   6.913   <1e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

```
DescTools::DunnettTest(Growth ~ Passage_F, data = rawdata)
```

```
  Dunnett's test for comparing several treatments with a control :
    95% family-wise confidence level

$`1`
         diff     lwr.ci    upr.ci    pval
2-1 -1.467320 -7.6899251  4.755285  0.9648
3-1  5.824059 -0.3985468 12.046664  0.0750 .
4-1  6.801105  0.5784996 13.023710  0.0262 *
5-1  8.156143  1.9335375 14.378748  0.0048 **
6-1 14.520106  8.2975011 20.742712 2.9e-08 ***

---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
DescTools::DunnettTest(Growth ~ Treatment, data = rawdata)
```

```
  Dunnett's test for comparing several treatments with a control :
    95% family-wise confidence level

$Control
                     diff    lwr.ci   upr.ci   pval
AngII-Control    10.40895  6.996811 13.82109  1e-10 ***
Whatever-Control 20.05199 16.639849 23.46413 <2e-16 ***

---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
pairwise.t.test(rawdata$Growth, rawdata$Treatment, p.adjust.method = "n")
```

```
    Pairwise comparisons using t tests with pooled SD

data:  rawdata$Growth and rawdata$Treatment
```

```
         Control AngII
AngII    5.1e-11 -
Whatever < 2e-16 1.0e-09

P value adjustment method: none
```

```
# mean(rawdata$Growth[which(rawdata$Passage == 1 &
# rawdata$Treatment == 'Control')])
anova_out$"Pr(>F)"
```

```
[1] 1.257266e-11           NA
```

```
# aOut$`Sum Sq`
summary(lmOut_additive)
```

```

Call:
lm(formula = Growth ~ Passage + Treatment, data = rawdata)

Residuals:
    Min      1Q  Median      3Q     Max
-32.407  -7.793  -0.281   7.255  32.283

Coefficients:
                  Estimate Std. Error t value Pr(>|t|)
(Intercept)       110.6802     1.5280  72.432  < 2e-16 ***
Passage             2.9271     0.3334   8.778  < 2e-16 ***
TreatmentAngII     10.4089     1.3949   7.462 6.59e-13 ***
TreatmentWhatever  20.0520     1.3949  14.375  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 10.8 on 356 degrees of freedom
Multiple R-squared:  0.4436,    Adjusted R-squared:  0.4389
F-statistic:  94.6 on 3 and 356 DF,  p-value: < 2.2e-16
```

```
(result <- tibble(
  predictor = rownames(anova_out),
  p = formatP(anova_out$"Pr(>F)", ndigits = 5)
))
```

```
# A tibble: 2 x 2
  predictor p
  <chr>     <chr>
1 Passage   "0.00001"
2 Residuals ""
```

```
broom::tidy(anova_out)
```

```
# A tibble: 2 x 6
  term         df  sumsq meansq statistic   p.value
  <chr>     <int>  <dbl>  <dbl>     <dbl>     <dbl>
1 Passage       1  8996.  8996.      49.0  1.26e-11
2 Residuals   358 65698.   184.      NA    NA
```

13.5 compare_n_numvars() as reporting option

```
raw_out <- compare_n_numvars(rawdata,
  dep_vars = c("Growth", "growthAdj"),
  indep_var = "Treatment",
  gaussian = TRUE, add_n = TRUE
)
```

```
raw_out$results |>
  select(Variable, `p(ANOVA)` = multivar_p, contains("fn")) |>
  rename_with(\(x) str_replace_all(
    x,
    c(
      " fn" = "",
      "Treatment " = ""
    )
  )) |>
  flextable() |>
  add_footer_lines("Symbols b,c indicate significance in post-hoc test vs. group 2 or 3") |>
  set_table_properties(width = 1, layout = "autofit")
```

Variable	p(ANOVA)	Control	AngII	Whatever
Growth	0.001	121 ± 12 [n = 120] bc	131 ± 12 [n = 120] c	141 ± 11 [n = 120]
growthAdj	0.001	121 ± 11 [n = 120] bc	131 ± 11 [n = 120] c	141 ± 11 [n = 120]

Variable	p(ANOVA)	Control	AngII	Whatever

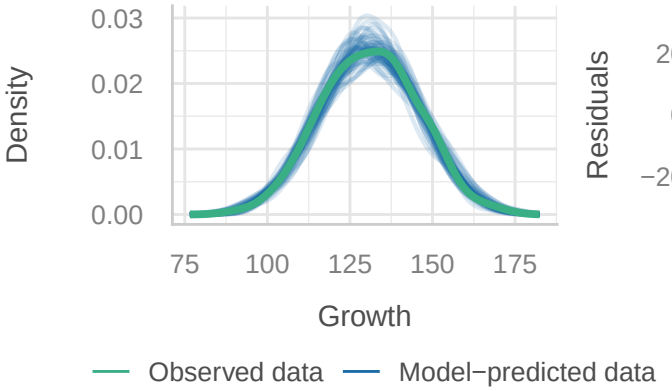
Symbols b,c indicate significance in post-hoc test vs. group 2 or 3

13.6 Model exploration with package performance

```
# x11() #interactive only!
# quartz() for mac
# from package performance
check_model(lmOut_additive)
```

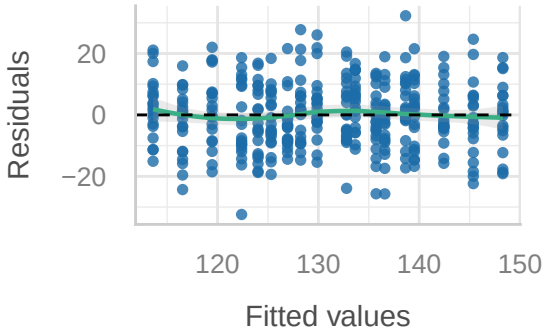
Posterior Predictive Check

Model−predicted lines should resemble observed data line



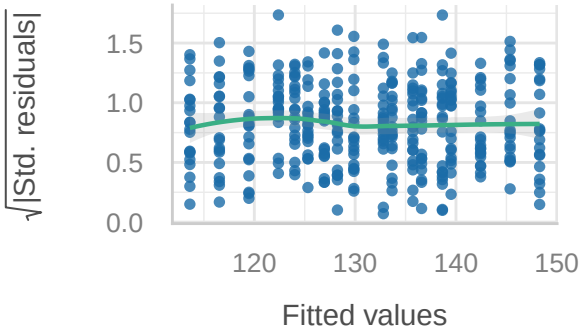
Linearity

Red data line should be flat and horizont



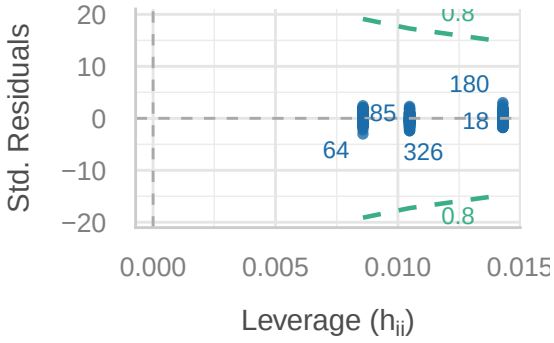
Homogeneity of Variance

Reference line should be flat and horizontal



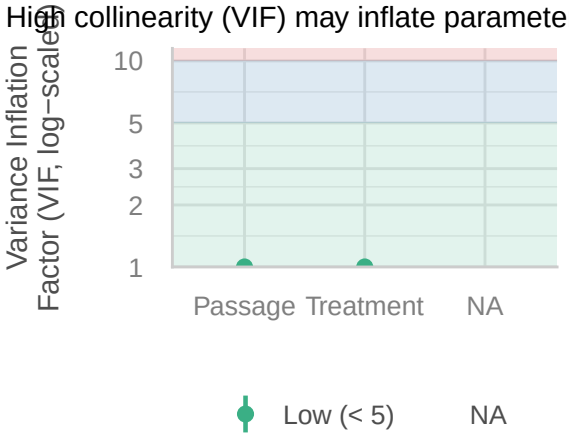
Influential Observations

Points should be inside the contour lines



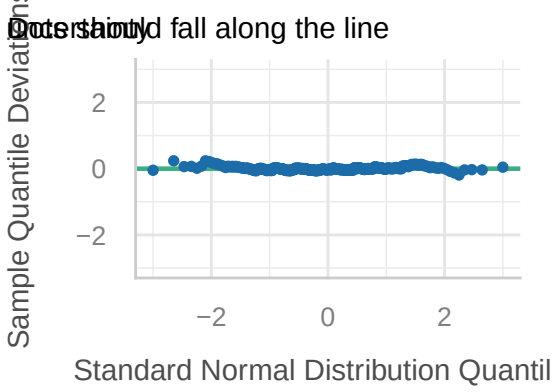
Collinearity

High collinearity (VIF) may inflate parameter uncertainty



Normality of Residuals

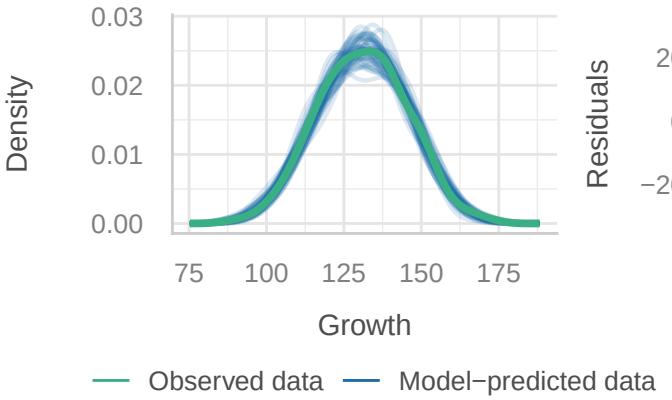
Dots should fall along the line



```
check_model(lmOut_interaction)
```

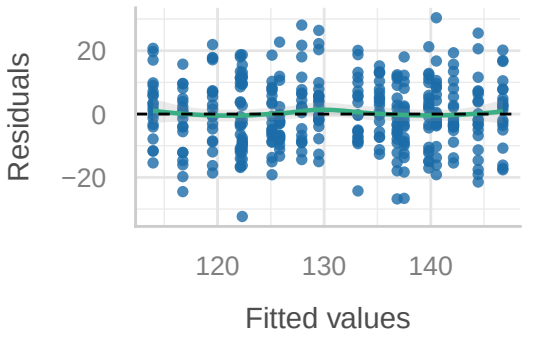
Posterior Predictive Check

Model−predicted lines should resemble observed data line



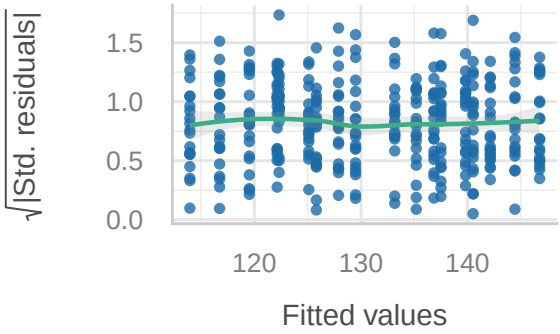
Linearity

Reference line should be flat and horizont



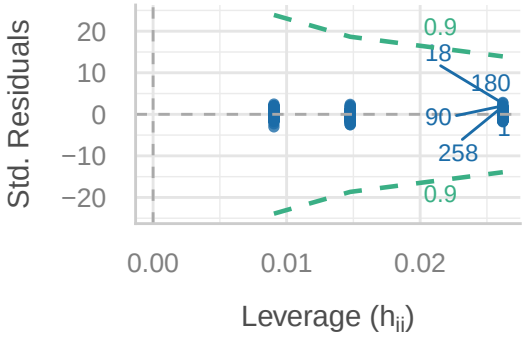
Homogeneity of Variance

Reference line should be flat and horizontal



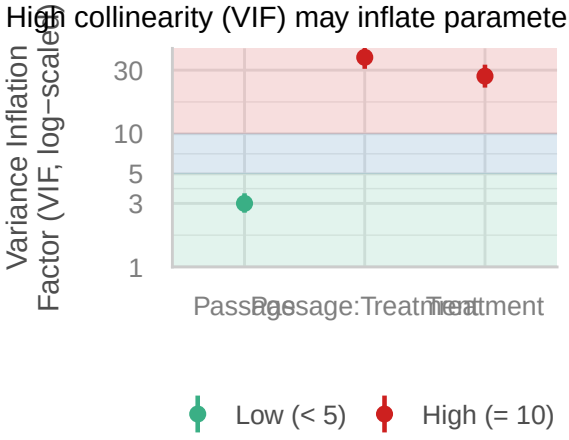
Influential Observations

Points should be inside the contour lines



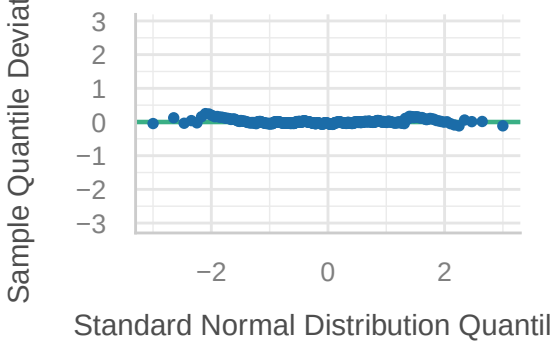
Collinearity

High collinearity (VIF) may inflate parameter uncertainty



Normality of Residuals

Dots should fall along the line



```
# dev.off()
```

13.7 Exercise

In the penguin data as well as in the Adelies only, model relationship between

1. regression-like

- DV flipper length and IV body weight
- DV bill depth and IV body weight

2. anova-like

- DV flipper length and IV species
- DV flipper length and IV species and sex
- DV bill depth and IV species and sex

3. combined

- DV flipper length and IV species and sex and body weight

When appropriate, do post-hoc analyses as well.

Vizualize results indicating significances.

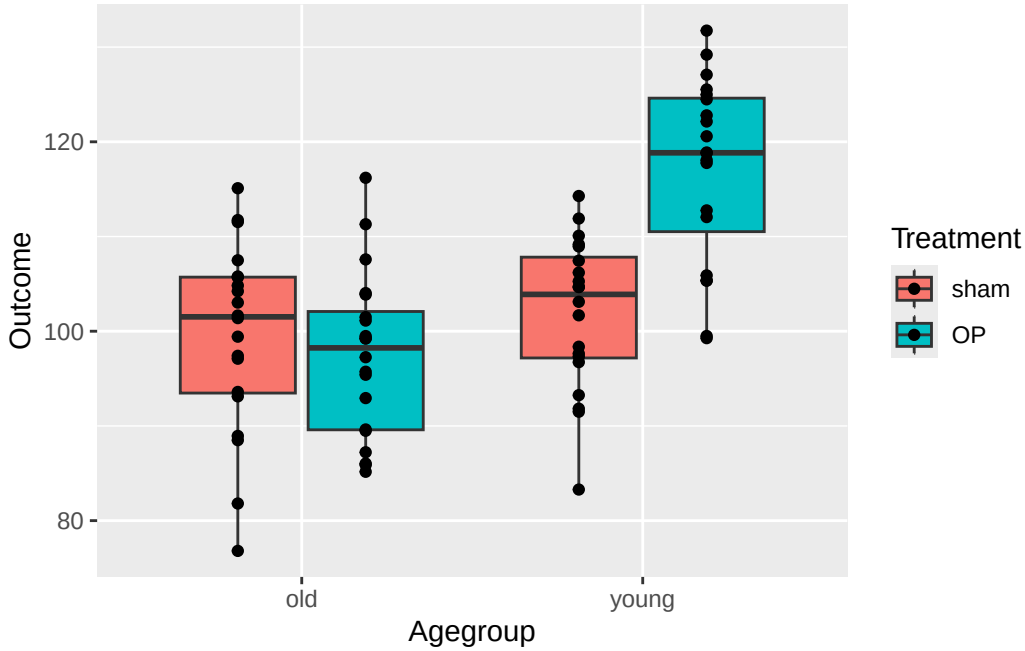
14 Interaction in linear models

```
pacman::p_load(conflicted, tidyverse, wrappedtools,
               car, multcomp, broom)
conflicts_prefer(dplyr::select, dplyr::filter)
```

For a better understanding, data with defined effect size and interactions will be simulated and analyzed.

14.1 No age effect, no treatment effect, interaction treatment with agegroup

```
set.seed(101)
rawdata <- tibble(
  Agegroup = factor(
    rep(c("young", "old"), each = 40),
    levels = c("old", "young")
  ),
  Treatment = factor(
    rep(c("sham", "OP"), 40),
    levels = c("sham", "OP")
  )
) |>
  mutate(Outcome = rnorm(n = 80, mean = 100, sd = 10) +
    ((Treatment == "OP") *
      (Agegroup == "young")) * 20)
ggplot(rawdata, aes(
  x = Agegroup, y = Outcome,
  fill = Treatment
)) +
  geom_boxplot() +
  geom_point(position = position_dodge(width = .75))
```




```
lmout <- lm(Outcome ~ Agegroup * Treatment,
  data = rawdata
)
tidy(lmout) |>
  select(1:2)
```

```
# A tibble: 4 x 2
  term                        estimate
  <chr>                          <dbl>
1 (Intercept)                    99.5
2 Agegroupyoung                   2.41
3 TreatmentOP                    -2.04
4 Agegroupyoung:TreatmentOP      17.3
```

```
anova(lmout) |> # this is WRONG!!!
  tidy() |>
  slice(1:3) |>
  mutate(p.value = formatP(p.value, ndigits = 3, mark = TRUE))
```

```
# A tibble: 3 x 6
  term                  df sumsq meansq statistic p.value
  <chr>              <int> <dbl>  <dbl>     <dbl> <chr>
1 Agegroup               1 2443.  2443.      29.2 0.001 ***
```

```
2 Treatment              1  872.   872.      10.4 0.002 **
3 Agegroup:Treatment     1 1494.  1494.      17.9 0.001 ***
```

```
Anova(lmout, type = 3) |>
  tidy() |>
  slice(1:4) |>
  mutate(p.value = formatP(p.value, ndigits = 3, mark = TRUE))
```

```
# A tibble: 4 x 5
  term                  sumsq    df statistic p.value
  <chr>                 <dbl> <dbl>     <dbl> <chr>
1 (Intercept)        197833.      1  2368.    0.001 ***
2 Agegroup               58.0     1     0.695 0.407 n.s.
3 Treatment              41.7     1     0.499 0.482 n.s.
4 Agegroup:Treatment   1494.      1    17.9   0.001 ***
```

```
summary(glht(
  model = lmout,
  linfct = mcp(Treatment = "Tukey")
))
```

```
Warning in mcp2matrix(model, linfct = linfct): covariate interactions found --
default contrast might be inappropriate
```

```
     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: Tukey Contrasts


Fit: lm(formula = Outcome ~ Agegroup * Treatment, data = rawdata)

Linear Hypotheses:
               Estimate Std. Error t value Pr(>|t|)
OP - sham == 0   -2.042      2.890  -0.707    0.482
(Adjusted p values reported -- single-step method)
```

```
summary(glht(
  model = lmout,
  linfct = mcp(Agegroup = "Tukey")
))
```

```
Warning in mcp2matrix(model, linfct = linfct): covariate interactions found --
default contrast might be inappropriate


     Simultaneous Tests for General Linear Hypotheses

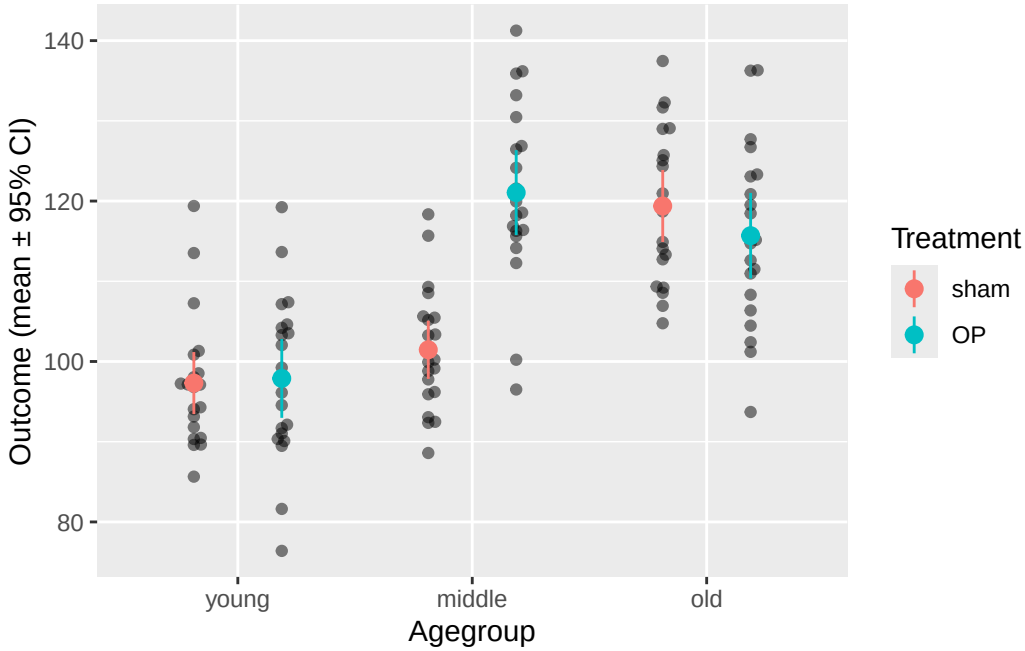
Multiple Comparisons of Means: Tukey Contrasts


Fit: lm(formula = Outcome ~ Agegroup * Treatment, data = rawdata)

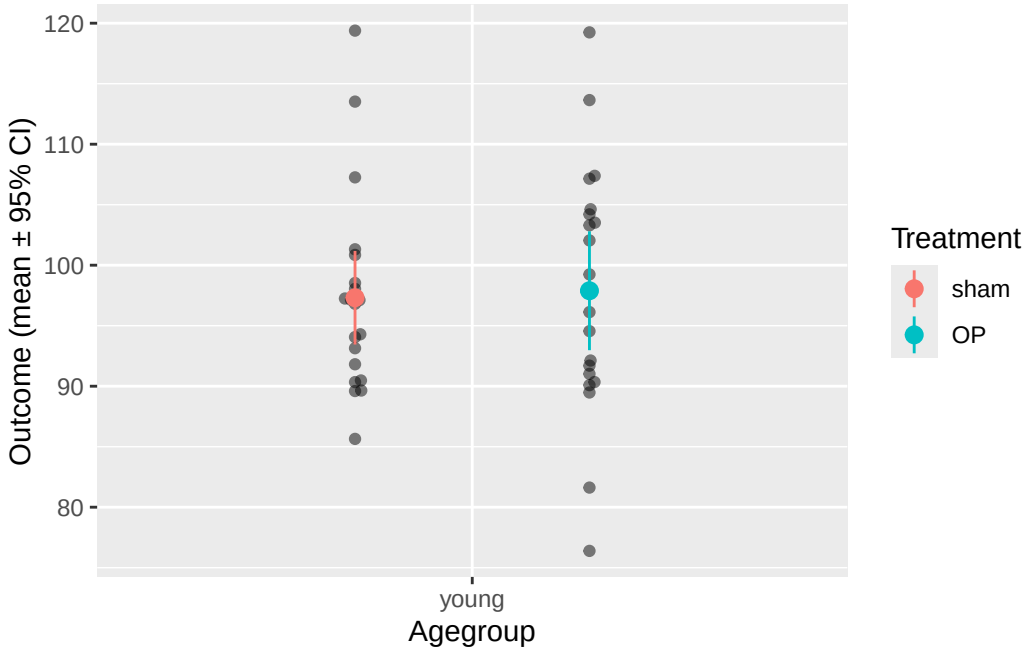
Linear Hypotheses:
                 Estimate Std. Error t value Pr(>|t|)
young - old == 0    2.409      2.890   0.834    0.407
(Adjusted p values reported -- single-step method)
```

14.2 Age effect, no treatment effect, interaction treatment with agegroup

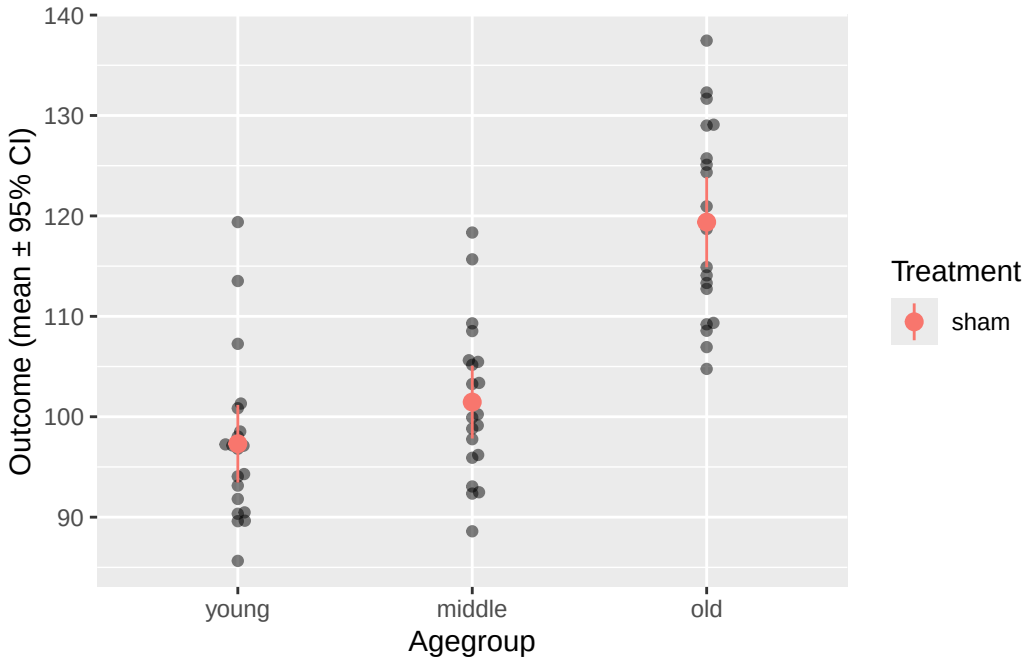
```
set.seed(1010)
rawdata <- tibble(
  Agegroup = factor(
    rep(c("young", "middle", "old"), each = 40),
    levels = c("young", "middle", "old")
  ),
  Treatment = factor(
    rep(c("sham", "OP"), 60),
    levels = c("sham", "OP")
  ),
  Outcome = rnorm(120, 100, 10) +
    (Treatment == "OP") *
      (Agegroup == "middle") * 20 +
    (Agegroup == "old") * 20
)
ggplot(rawdata, aes(
  x = Agegroup, y = Outcome,
  fill = Treatment
)) +
  # geom_boxplot()+
  ggbeeswarm::geom_beeswarm(dodge.width = .75, alpha = .5) +
  stat_summary(aes(color = Treatment),
    fun.data = mean_cl_normal,
    position = position_dodge(width = .75)
  ) +
  ylab("Outcome (mean \u00b1 95% CI)")
```




```
rawdata |>
  filter(Agegroup == "young") |>
  ggplot(aes(
    x = Agegroup, y = Outcome,
    fill = Treatment
  )) +
  ggbeeswarm::geom_beeswarm(dodge.width = .75, alpha = .5) +
  stat_summary(aes(color = Treatment),
    fun.data = mean_cl_normal,
    position = position_dodge(width = .75)
  ) +
  ylab("Outcome (mean \u00b1 95% CI)")
```




```
rawdata |>
  filter(Treatment == "sham") |>
  ggplot(aes(
    x = Agegroup, y = Outcome,
    fill = Treatment
  )) +
  # geom_boxplot()+
  ggbeeswarm::geom_beeswarm(dodge.width = .75, alpha = .5) +
  stat_summary(aes(color = Treatment),
    fun.data = mean_cl_normal,
    position = position_dodge(width = .75)
  ) +
  ylab("Outcome (mean \u00b1 95% CI)")
```




```
lmout <- lm(Outcome ~ Agegroup * Treatment,
  data = rawdata
)
tidy(lmout) |>
  select(1:2)
```

```
# A tibble: 6 x 2
  term                        estimate
  <chr>                          <dbl>
1 (Intercept)                   97.3
2 Agegroupmiddle                 4.15
3 Agegroupold                   22.1
4 TreatmentOP                    0.578
5 Agegroupmiddle:TreatmentOP    19.0
6 Agegroupold:TreatmentOP       -4.28
```

```
anova(lmout) |> # this is WRONG!!!
  tidy() |>
  slice(1:3) |>
  mutate(p.value = formatP(p.value, ndigits = 3, mark = TRUE))
```

```
# A tibble: 3 x 6
  term                  df sumsq meansq statistic p.value
```

```
  <chr>              <int> <dbl>  <dbl>     <dbl> <chr>
1 Agegroup               2 8310.  4155.     42.4  0.001 ***
2 Treatment              1  904.   904.      9.22 0.003 **
3 Agegroup:Treatment     2 3074.  1537.     15.7  0.001 ***
```

```
Anova(lmout, type = 3) |>
  tidy() |>
  slice(1:4) |>
  mutate(p.value = formatP(p.value, ndigits = 3, mark = TRUE))
```

```
# A tibble: 4 x 5
  term                  sumsq    df statistic p.value
  <chr>                 <dbl> <dbl>     <dbl> <chr>
1 (Intercept)        189389.       1 1931.     0.001 ***
2 Agegroup             5504.       2   28.1    0.001 ***
3 Treatment               3.35     1    0.0341 0.854 n.s.
4 Agegroup:Treatment   3074.       2   15.7    0.001 ***
```

```
summary(glht(
  model = lmout,
  linfct = mcp(Treatment = "Tukey")
))
```

```
Warning in mcp2matrix(model, linfct = linfct): covariate interactions found --
default contrast might be inappropriate
```

```
     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: Tukey Contrasts


Fit: lm(formula = Outcome ~ Agegroup * Treatment, data = rawdata)

Linear Hypotheses:
               Estimate Std. Error t value Pr(>|t|)
OP - sham == 0   0.5784     3.1315   0.185    0.854
(Adjusted p values reported -- single-step method)
```

```
summary(glht(
  model = lmout,
  linfct = mcp(Agegroup = "Tukey")
))
```

```
Warning in mcp2matrix(model, linfct = linfct): covariate interactions found --
default contrast might be inappropriate


     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: Tukey Contrasts

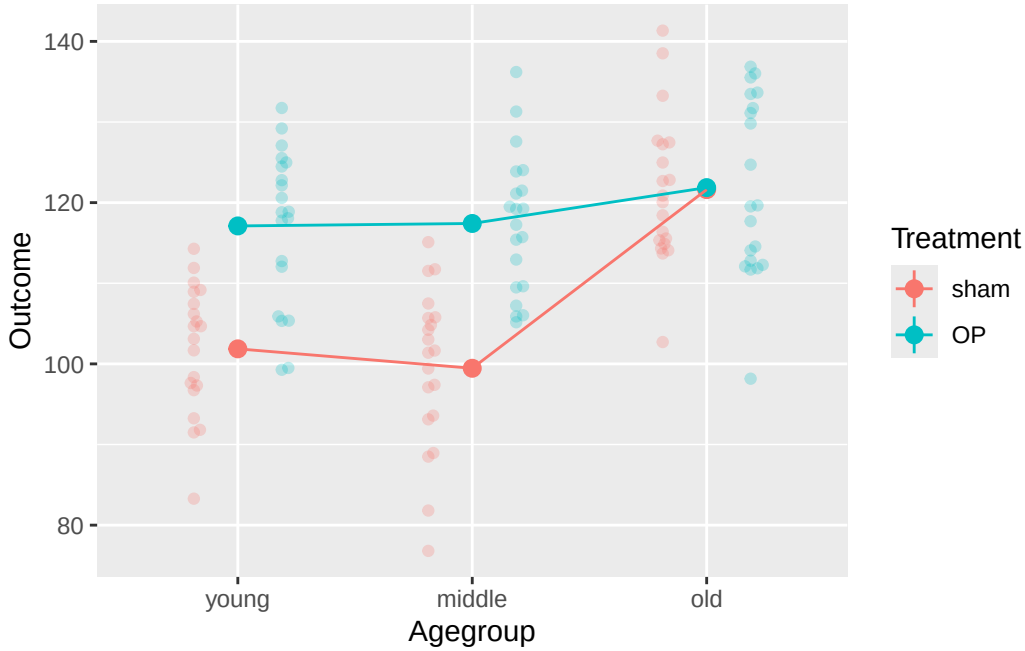

Fit: lm(formula = Outcome ~ Agegroup * Treatment, data = rawdata)

Linear Hypotheses:
                     Estimate Std. Error t value Pr(>|t|)
middle - young == 0    4.149      3.131   1.325    0.384
old - young == 0      22.073      3.131   7.049   <1e-04 ***
old - middle == 0     17.924      3.131   5.724   <1e-04 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

14.3 Age effect, treatment effect, interaction treatment with agegroup

```
set.seed(101)
rawdata <- tibble(
  Agegroup = factor(
    rep(c("young", "middle", "old"), each = 40),
    levels = c("young", "middle", "old")
  ),
  Treatment = factor(
    rep(c("sham", "OP"), 60),
    levels = c("sham", "OP")
  )
) |>
  mutate(Outcome = rnorm(120, 100, 10) +
    (Treatment == "OP") *
      # (Agegroup %in% c('young','middle'))*
      (Agegroup != "old") * 20 +
    (Agegroup == "old") * 20)
ggplot(rawdata, aes(
  x = Agegroup, y = Outcome,
  color = Treatment
)) +
  # geom_boxplot()+
  ggbeeswarm::geom_beeswarm(dodge.width = .75, alpha = .25) +
  stat_summary(
    fun = mean,
    aes(group = Treatment)
  ) +
  # position = position_dodge(width = .75))+
  stat_summary(
    fun = mean, geom = "line",
    aes(group = Treatment)
  )
```

```
Warning: Removed 6 rows containing missing values or values outside the scale range
(`geom_segment()`).
```




```
suppressWarnings(
  ggplot(rawdata, aes(x = as.numeric(Agegroup), y = Outcome, fill = Treatment)) +
    ggbeeswarm::geom_beeswarm(aes(shape = Treatment),
      alpha = .5, dodge.width = .15
    ) +
    geom_smooth() +
    scale_x_continuous("Agegroup",
      breaks = 1:3,
      labels = c("young", "middle", "old")
    )
)
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 0.99
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 2.01
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 2.0996e-16
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 4.0401

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
0.99

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
2.01

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 2.0996e-16

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 4.0401

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 0.99

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 2.01

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 2.0996e-16

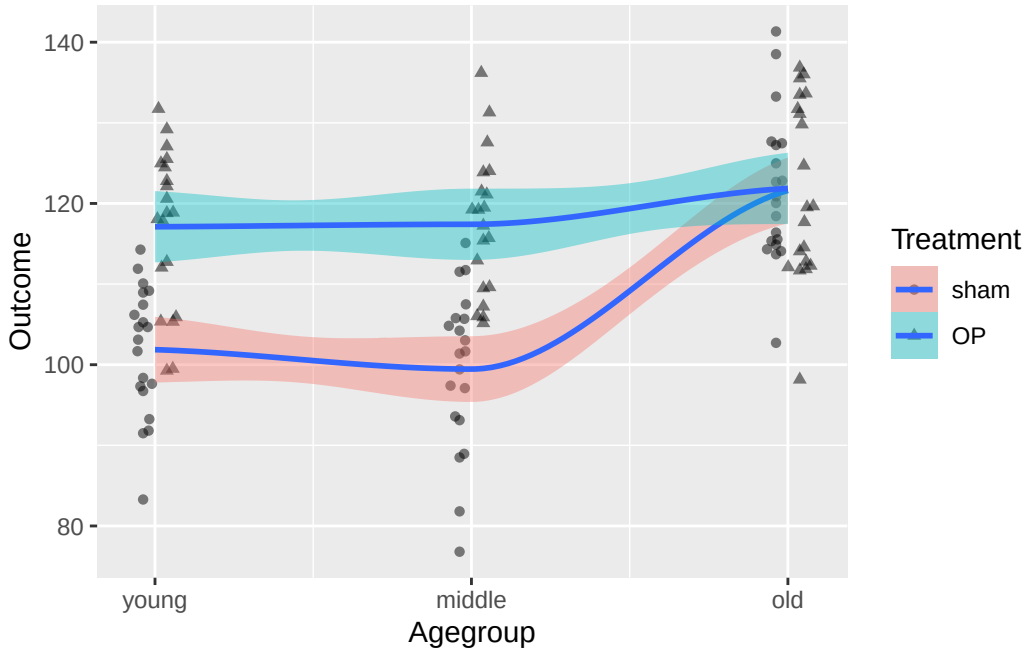
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 4.0401

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
0.99
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
2.01
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 2.0996e-16
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 4.0401
```




```
lmout <- lm(Outcome ~ Agegroup * Treatment,
  data = rawdata
)
tidy(lmout) |>
  select(1:2)
```

```
# A tibble: 6 x 2
  term                        estimate
  <chr>                          <dbl>
1 (Intercept)                   102.
2 Agegroupmiddle                 -2.41
3 Agegroupold                    19.8
4 TreatmentOP                    15.2
5 Agegroupmiddle:TreatmentOP      2.71
6 Agegroupold:TreatmentOP       -15.0
```

```
anova(lmout) |> # this is WRONG!!!
  tidy() |>
  slice(1:3) |>
  mutate(p.value = formatP(p.value, ndigits = 3, mark = TRUE))
```

```
# A tibble: 3 x 6
  term                  df sumsq meansq statistic p.value
  <chr>              <int> <dbl>  <dbl>     <dbl> <chr>
1 Agegroup               2 4377.  2188.      24.3 0.001 ***
2 Treatment              1 3730.  3730.      41.4 0.001 ***
3 Agegroup:Treatment     2 1819.   910.      10.1 0.001 ***
```

```
Anova(lmout, type = 3) |>
  tidy() |>
  slice(1:4) |>
  mutate(p.value = formatP(p.value, ndigits = 3, mark = TRUE))
```

```
# A tibble: 4 x 5
  term                 sumsq    df statistic p.value
  <chr>                <dbl> <dbl>     <dbl> <chr>
1 (Intercept)        207533.     1    2301.  0.001 ***
2 Agegroup             5913.     2      32.8 0.001 ***
3 Treatment            2324.     1      25.8 0.001 ***
4 Agegroup:Treatment   1819.     2      10.1 0.001 ***
```

```
summary(glht(
  model = lmout,
  linfct = mcp(Treatment = "Tukey")
))
```

```
Warning in mcp2matrix(model, linfct = linfct): covariate interactions found --
default contrast might be inappropriate
```

```
     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: Tukey Contrasts


Fit: lm(formula = Outcome ~ Agegroup * Treatment, data = rawdata)

Linear Hypotheses:
               Estimate Std. Error t value Pr(>|t|)
OP - sham == 0   15.245      3.003   5.076 1.52e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

```r
summary(glht(
  model = lmout,
  linfct = mcp(Agegroup = "Tukey")
))
```

```
Warning in mcp2matrix(model, linfct = linfct): covariate interactions found --
default contrast might be inappropriate


     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: Tukey Contrasts


Fit: lm(formula = Outcome ~ Agegroup * Treatment, data = rawdata)

Linear Hypotheses:
                     Estimate Std. Error t value Pr(>|t|)
middle - young == 0   -2.409      3.003  -0.802    0.702
old - young == 0      19.750      3.003   6.576   <1e-05 ***
old - middle == 0     22.159      3.003   7.378   <1e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

Ignoring the interaction in the model is no solution. Effect sizes will be wrongly estimated:

```r
# falsch!!!
lmout_add <- lm(Outcome ~ Agegroup + Treatment,
  data = rawdata
)
tidy(lmout_add) |>
  select(1:2)
```

```
# A tibble: 4 x 2
  term           estimate
  <chr>             <dbl>
1 (Intercept)      104.
2 Agegroupmiddle    -1.05
3 Agegroupold       12.3
4 TreatmentOP       11.2
```

```r
anova(lmout) |> # this is WRONG!!!
  tidy() |>
  slice(1:2) |>
  mutate(p.value = formatP(p.value, ndigits = 3, mark = TRUE))
```

```
# A tibble: 2 x 6
  term         df sumsq meansq statistic p.value
  <chr>     <int> <dbl>  <dbl>     <dbl> <chr>
1 Agegroup      2 4377.  2188.      24.3 0.001 ***
2 Treatment     1 3730.  3730.      41.4 0.001 ***
```

```r
Anova(lmout, type = 2) |>
  tidy() |>
  slice(1:2) |>
  mutate(p.value = formatP(p.value, ndigits = 3, mark = TRUE))
```

```
# A tibble: 2 x 5
  term      sumsq    df statistic p.value
  <chr>     <dbl> <dbl>     <dbl> <chr>
1 Agegroup  4377.     2      24.3 0.001 ***
2 Treatment 3730.     1      41.4 0.001 ***
```

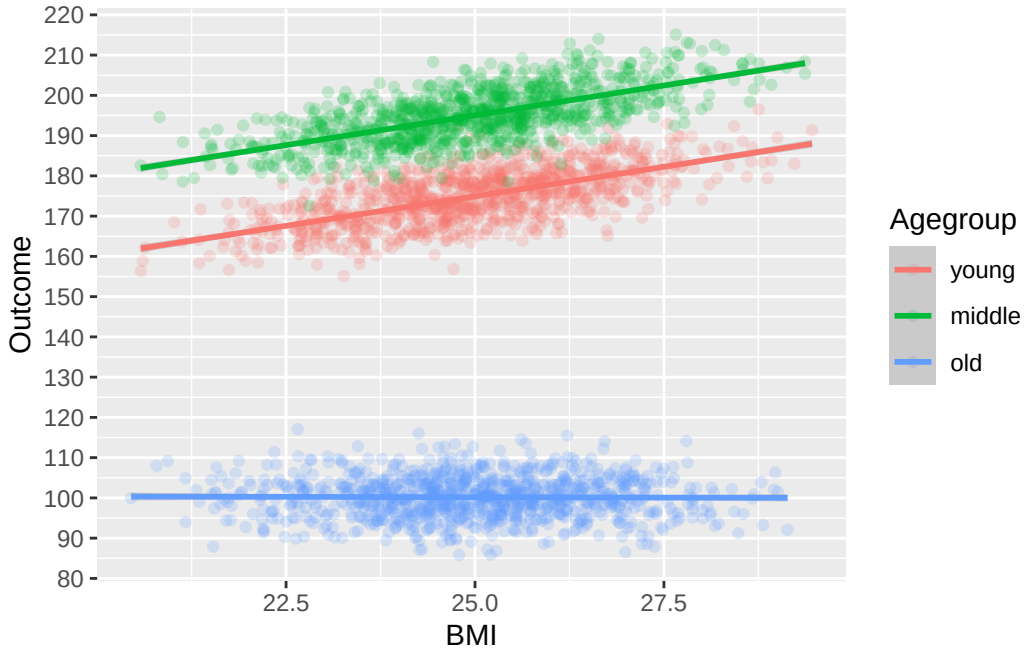
```r
lmout_add <- lm(Outcome ~ Agegroup + Treatment,
  data = rawdata |> filter(Agegroup != "old")
)
tidy(lmout_add) |>
  select(1:2)
```

```
# A tibble: 3 x 2
  term           estimate
  <chr>             <dbl>
1 (Intercept)      101.
2 Agegroupmiddle    -1.05
3 TreatmentOP       16.6
```

14.4 Interaction in regression-like lm

```
set.seed(1111)
rawdata <- tibble(
  Agegroup = factor(
    rep(c("young", "middle", "old"), each = 1000),
    levels = c("young", "middle", "old")
  ),
  BMI = rnorm(3000,25,1.5)
) |>
  mutate(Outcome =
           case_when(
             Agegroup == "young" ~
               100 + BMI * 3,
             Agegroup == "middle" ~
               100 + 20 + BMI * 3,
             Agegroup == "old" ~
               100 + BMI * 0) + #(3 + (-2))) +
           rnorm(3000,0,5))
ggplot(rawdata, aes(
  x = BMI, y = Outcome,
  color = Agegroup)) +
  geom_point(alpha = .2) +
  geom_smooth(method = "lm") +
  # geom_smooth(method = "lm", color = "black") +
  scale_y_continuous(breaks = seq(0,1000,10))
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
lmout <- lm(Outcome ~ Agegroup * BMI,
  data = rawdata
)
tidy(lmout) |>
  mutate(p.value = formatP(p.value, mark = TRUE)) |>
  select(1:2, 5)
```

```
# A tibble: 6 x 3
  term                estimate p.value
  <chr>                  <dbl> <chr>
1 (Intercept)          102.     0.001 ***
2 Agegroupmiddle        19.3    0.001 ***
3 Agegroupold           -0.559  0.882 n.s.
4 BMI                    2.93   0.001 ***
5 Agegroupmiddle:BMI     0.0333 0.827 n.s.
6 Agegroupold:BMI       -2.97   0.001 ***
```

```
Anova(lmout, type = 3) |>
  tidy() |>
  slice(1:4) |>
  mutate(p.value = formatP(p.value, ndigits = 3, mark = TRUE))
```

```
# A tibble: 4 x 5
```

```
  term          sumsq    df statistic p.value
  <chr>         <dbl> <dbl>     <dbl> <chr>
1 (Intercept)  36552.     1    1428.  0.001 ***
2 Agegroup       898.     2      17.5 0.001 ***
3 BMI          19006.     1     742.  0.001 ***
4 Agegroup:BMI 13367.     2     261.  0.001 ***
```

```
summary(glht(
  model = lmout,
  linfct = mcp(Agegroup = "Tukey")
))
```

```
Warning in mcp2matrix(model, linfct = linfct): covariate interactions found --
default contrast might be inappropriate
```

```
     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: Tukey Contrasts


Fit: lm(formula = Outcome ~ Agegroup * BMI, data = rawdata)

Linear Hypotheses:
                     Estimate Std. Error t value Pr(>|t|)
middle - young == 0  19.2941     3.8219   5.048 1.25e-06 ***
old - young == 0     -0.5586     3.7772  -0.148    0.988
old - middle == 0   -19.8528     3.7950  -5.231  < 1e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

14.5 How to specify interaction in multivariable models

```
# all possible interactions
(lm_out <- lm(mpg ~ wt * gear * factor(am) * cyl,
  data = mtcars
))
```

```

Call:
lm(formula = mpg ~ wt * gear * factor(am) * cyl, data = mtcars)

Coefficients:
            (Intercept)                      wt                    gear
                456.687                -158.453                -146.900
           factor(am)1                     cyl                 wt:gear
               -180.550                 -16.129                  53.938
        wt:factor(am)1       gear:factor(am)1                  wt:cyl
                 63.239                 102.915                  10.048
            gear:cyl         factor(am)1:cyl      wt:gear:factor(am)1
                 5.764                 -33.954                 -37.005
          wt:gear:cyl       wt:factor(am)1:cyl     gear:factor(am)1:cyl
                -3.593                   8.905                   3.456
wt:gear:factor(am)1:cyl
                    NA
```

```
(lm_out <- lm(mpg ~ (wt * factor(gear) * factor(am) * factor(cyl)),
  data = mtcars
))
```

```

Call:
lm(formula = mpg ~ (wt * factor(gear) * factor(am) * factor(cyl)),
    data = mtcars)

Coefficients:
                            (Intercept)
                                 39.880
                                     wt
                                 -7.456
                          factor(gear)4
                               -143.080
                          factor(gear)5
```

```
                      -149.462
                   factor(am)1
                       150.599
                  factor(cyl)6
                        24.824
                  factor(cyl)8
                       -14.821
              wt:factor(gear)4
                        47.456
              wt:factor(gear)5
                        49.599
                wt:factor(am)1
                       -49.160
    factor(gear)4:factor(am)1
                            NA
    factor(gear)5:factor(am)1
                            NA
             wt:factor(cyl)6
                        -6.013
             wt:factor(cyl)8
                         5.018
   factor(gear)4:factor(cyl)6
                       -72.234
   factor(gear)5:factor(cyl)6
                       -31.058
   factor(gear)4:factor(cyl)8
                            NA
   factor(gear)5:factor(cyl)8
                        -4.057
     factor(am)1:factor(cyl)6
                        21.011
     factor(am)1:factor(cyl)8
                            NA
 wt:factor(gear)4:factor(am)1
                            NA
 wt:factor(gear)5:factor(am)1
                            NA
wt:factor(gear)4:factor(cyl)6
                        15.173
wt:factor(gear)5:factor(cyl)6
                            NA
wt:factor(gear)4:factor(cyl)8
                            NA
```

```
            wt:factor(gear)5:factor(cyl)8
                                       NA
              wt:factor(am)1:factor(cyl)6
                                       NA
              wt:factor(am)1:factor(cyl)8
                                       NA
   factor(gear)4:factor(am)1:factor(cyl)6
                                       NA
   factor(gear)5:factor(am)1:factor(cyl)6
                                       NA
   factor(gear)4:factor(am)1:factor(cyl)8
                                       NA
   factor(gear)5:factor(am)1:factor(cyl)8
                                       NA
wt:factor(gear)4:factor(am)1:factor(cyl)6
                                       NA
wt:factor(gear)5:factor(am)1:factor(cyl)6
                                       NA
wt:factor(gear)4:factor(am)1:factor(cyl)8
                                       NA
wt:factor(gear)5:factor(am)1:factor(cyl)8
                                       NA
```

```
# only two-way interactions
(lm_out <- lm(mpg ~ (wt + gear + factor(am) + cyl)^2,
  data = mtcars
))
```

```

Call:
lm(formula = mpg ~ (wt + gear + factor(am) + cyl)^2, data = mtcars)

Coefficients:
     (Intercept)                wt              gear       factor(am)1
        57.58968         -17.65458          -7.80045          13.91506
             cyl           wt:gear    wt:factor(am)1            wt:cyl
         3.13308           4.90387         -11.51010           0.06975
gear:factor(am)1          gear:cyl   factor(am)1:cyl
         2.23288          -1.43230           2.15419
```

```
# some selected interactions
(lm_out <- lm(mpg ~ wt * (gear + factor(am) + cyl),
  data = mtcars
))
```

```
Call:
lm(formula = mpg ~ wt * (gear + factor(am) + cyl), data = mtcars)

Coefficients:
   (Intercept)              wt            gear     factor(am)1             cyl
      50.53760        -8.43482        -5.80119        21.62110        -
1.04500
       wt:gear  wt:factor(am)1          wt:cyl
       2.03447        -7.59198        -0.00499
```

```
(lm_out <- lm(mpg ~ wt + gear + factor(am) + cyl +
                wt * cyl + wt * gear,
  data = mtcars
))
```

```
Call:
lm(formula = mpg ~ wt + gear + factor(am) + cyl + wt * cyl +
    wt * gear, data = mtcars)

Coefficients:
(Intercept)           wt         gear  factor(am)1          cyl       wt:cyl
    51.9639      -7.5257       0.6564      -0.5099      -3.6140       0.7489
    wt:gear
    -0.2980
```

15 Logistic regression

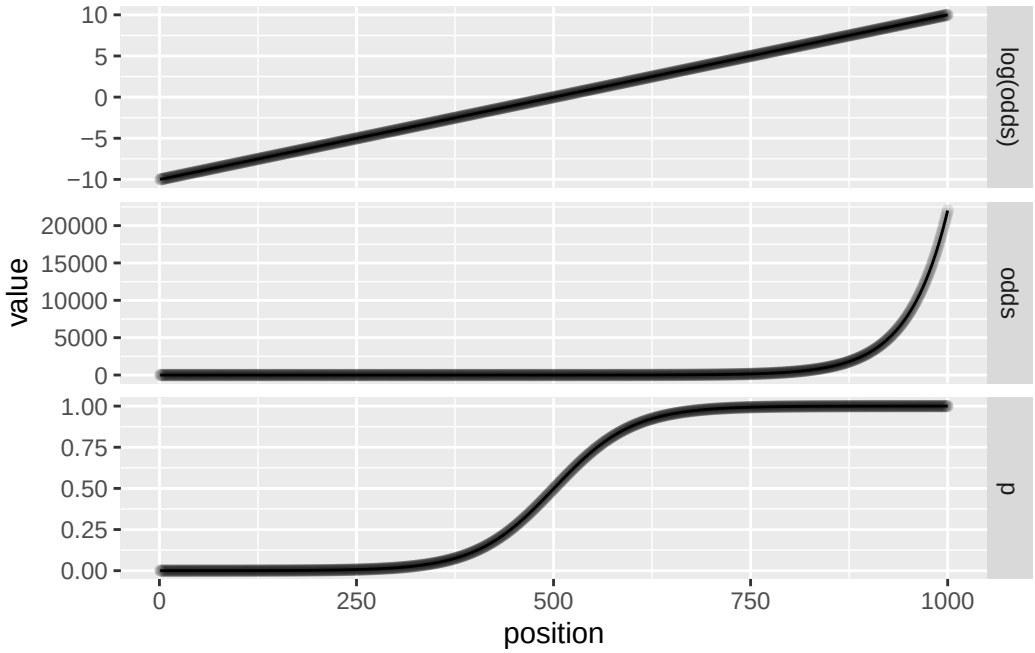
Generalized linear models will be introduced using logistic regression as an example. The data are coming from a simulation on variables influencing success or failure in a python exam where the outcome follows a binomial distribution.

```
pacman::p_load(
  conflicted, tidyverse, wrappedtools, here, flextable,
  car, ggbeeswarm,
  pROC, rpart, rpart.plot
)
conflicts_prefer(dplyr::filter)
```

```
[conflicted] Will prefer dplyr::filter over any other package.
```

15.1 Odds vs. probability

Probabilities are usually expressed xx%, 20% risk for rain means that on 20 out of 100 days like this it rains. Odds are expressed as xx:yy, 20:80 means that on 20 out of 100 days like this it rains, while on 80 days it does not. The odds are calculated as p/(1-p), where p is the probability. The log odds are calculated as log(odds).
For many traits it is reasonable to assume a sigmoidal relationship between a risk (like LDL cholesterol) and the probability of an outcome like myocardial infarction. Those probabilities are restrained between 0 and 1. The odds are restrained at 0 but have not upper bound. The log odds are not restrained and are linearly related to the risk increase. While this may cause mental knots, it is useful mathematically and forms the basis for logistic regression, a generalized linear model for outcomes with binomial distribution.



15.2 Data preparation

Before the analysis, the data set is cleaned and prepared for the analysis. Variables with 0 / 1 are coded as factor No / Yes, variable practice_problems_solved is lumped into three categories.

```
# rawdata <- infert |>
#   as_tibble() |>
#   select(-contains("stratum"))
rawdata <-
  read_csv(here("data/python_learning_exam_performance.csv"))
```

```
Rows: 3000 Columns: 15
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (2): country, prior_programming_experience
dbl (13): student_id, age, weeks_in_course, hours_spent_learning_per_week, p...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

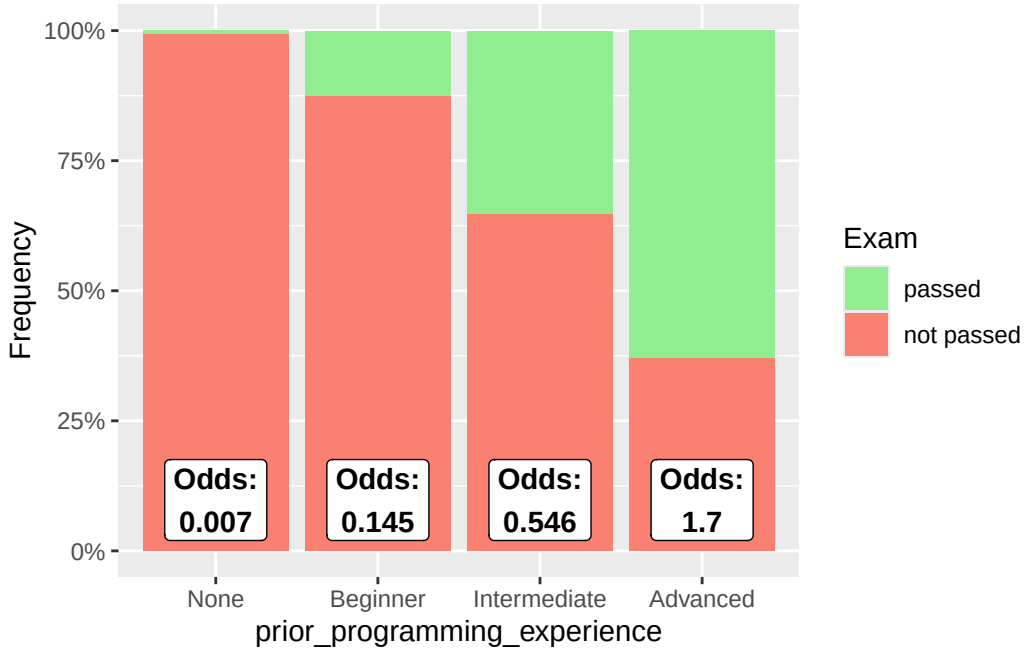
```
head(rawdata)
```

```
# A tibble: 6 x 15
  student_id   age country  prior_programming_experience weeks_in_course
       <dbl> <dbl> <chr>    <chr>                                   <dbl>
1          1    54 Pakistan Beginner                                   12
2          2    44 Nigeria  None                                       15
3          3    30 India    Intermediate                                5
4          4    23 Nigeria  Beginner                                    4
5          5    36 UK       None                                        3
6          6    54 Brazil   Intermediate                               10
# i 10 more variables: hours_spent_learning_per_week <dbl>,
#   practice_problems_solved <dbl>, projects_completed <dbl>,
#   tutorial_videos_watched <dbl>, uses_kaggle <dbl>,
#   participates_in_discussion_forums <dbl>, debugging_sessions_per_week <dbl>,
#   self_reported_confidence_python <dbl>, final_exam_score <dbl>,
#   passed_exam <dbl>
```

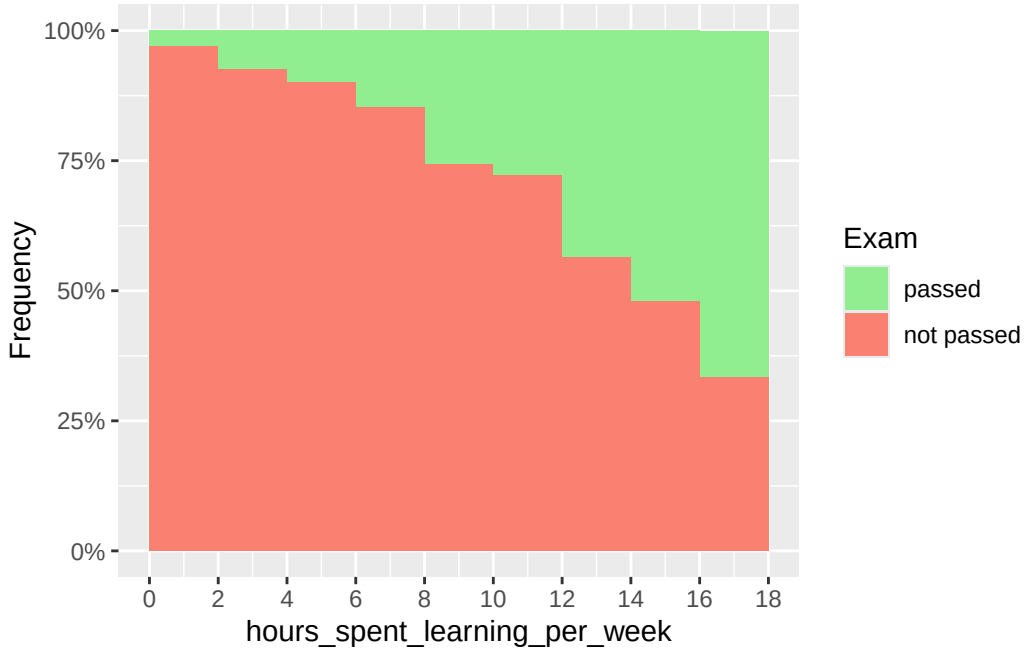
```
rawdata <-
  mutate(rawdata,
         across(where(is.character), factor),
         prior_programming_experience =
           factor(prior_programming_experience,
                  levels = c("None", "Beginner", "Intermediate", "Advanced")),
         across(where( \(x) is.numeric(x) & all(x %in% 0:1)),
                \(x) factor(x, levels = c(0,1), labels = c("No", "Yes"))),
         practice_problems_solved =
           cut(practice_problems_solved,
               breaks = c(0, 55, 65, Inf),
               labels = c("few", "average", "many")),
  )
```

15.3 Exploration

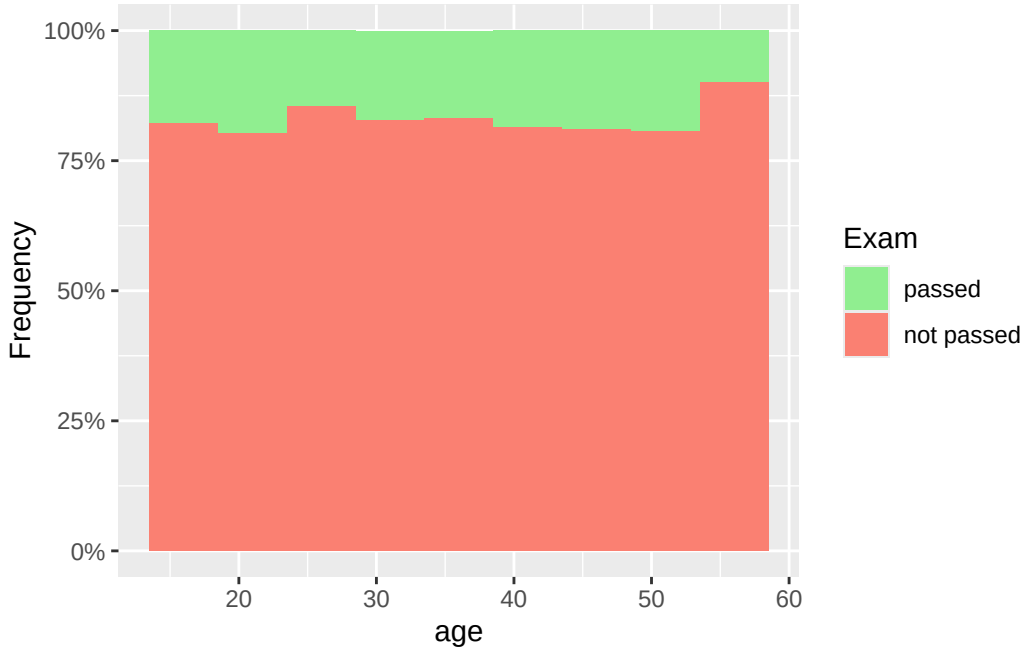
```
plot_data <- rawdata |>
  count(prior_programming_experience, passed_exam) |>
  group_by(prior_programming_experience) |>
  mutate(
    proportion = n / sum(n),
    odds = proportion[passed_exam == "Yes"] /
      proportion[passed_exam == "No"],
    odds_label = paste0("Odds:\n", round(odds, 3))
  ) |>
  ungroup()
rawdata |>
  ggplot(aes(prior_programming_experience,
             fill = fct_rev(passed_exam))) +
  geom_bar(position = "fill") +
  geom_label(
    data = plot_data |> distinct(prior_programming_experience, odds_label),
    aes(x = prior_programming_experience, y = 0.02, label = odds_label),
    inherit.aes = FALSE,
    vjust = 0,
    fontface = "bold"
  ) +
  scale_y_continuous("Frequency", labels = scales::percent) +
  scale_fill_manual("Exam",
                    values = c("lightgreen", "salmon"),
                    labels = c("passed", "not passed"))
```




```
rawdata |>
  ggplot(aes(hours_spent_learning_per_week,
             fill = fct_rev(passed_exam))) +
  geom_histogram(position = "fill", binwidth = 2,
                 center = 1, closed = "left") +
  scale_y_continuous("Frequency", labels = scales::percent) +
  scale_x_continuous(breaks = seq(0,20,2)) +
  scale_fill_manual("Exam",
                    values = c("lightgreen", "salmon"),
                    labels = c("passed", "not passed"))
```




```
rawdata |>
  ggplot(aes(age, fill = fct_rev(passed_exam))) +
  geom_histogram(position = "fill", binwidth = 5,
                 center = 1, closed = "left") +
  scale_y_continuous("Frequency", labels = scales::percent) +
  # scale_x_continuous(breaks = seq(0,20,2)) +
  scale_fill_manual("Exam",
                    values = c("lightgreen", "salmon"),
                    labels = c("passed", "not passed"))
```



The probability of passing the exam is going up with more experience and more learning hours, as reflected in the odds of passing. Age does not seem to have a significant effect.

15.4 Build model

The model is built using `glm()` and the output is extracted and transformed. The model is tested using `Anova()` and `summary()`. The results are then prepared for plotting.

15.5 Univariable model

```
logreg_out_uni <-
  glm(
     passed_exam ~ prior_programming_experience,
    family = binomial(), data = rawdata
  )
logreg_out_uni
```

```
Call:  glm(formula = passed_exam ~ prior_programming_experience, family = binomial(),
    data = rawdata)
```

```
Coefficients:
                              (Intercept)
                                   -5.015
    prior_programming_experienceBeginner
                                    3.085
prior_programming_experienceIntermediate
                                    4.411
    prior_programming_experienceAdvanced
                                    5.546

Degrees of Freedom: 2999 Total (i.e. Null);  2996 Residual
Null Deviance:      2804
Residual Deviance: 2050     AIC: 2058
```

```
# extract/transform model parameters
(ORs <- exp(logreg_out_uni$coefficients))
```

```
                              (Intercept)
                             6.635071e-03
    prior_programming_experienceBeginner
                             2.186442e+01
prior_programming_experienceIntermediate
                             8.234146e+01
    prior_programming_experienceAdvanced
                             2.562143e+02
```

```
(CIs <- exp(confint(logreg_out_uni)))
```

```
Waiting for profiling to be done...
```

```
                                                2.5 %       97.5 %
(Intercept)                                2.845442e-03   0.01287032
prior_programming_experienceBeginner       1.096095e+01  51.86378396
prior_programming_experienceIntermediate 4.152565e+01 194.63673770
prior_programming_experienceAdvanced       1.257998e+02 616.53031911
```

```
counts_none <- rawdata |>
  filter(prior_programming_experience == "None") |>
  group_by(passed_exam) |>
  count()
counts_beginner <- rawdata |>
```

```
  filter(prior_programming_experience == "Beginner") |>
  group_by(passed_exam) |>
  count()
odds_none <-
  counts_none$n[2] / counts_none$n[1]
odds_beginner <-
  counts_beginner$n[2] / counts_beginner$n[1]
or_beginner <-
  odds_beginner / odds_none
cat("OR Beginner from model:", ORs[2], "directly:", or_beginner,"\n")
```

```
OR Beginner from model: 21.86442 directly: 21.86442
```

```
# test model
(Anova_out_uni <- Anova(logreg_out_uni, type = 2) |>
    broom::tidy() |>
    mutate(p.value = formatP(p.value, ndigits = 5)))
```

```
# A tibble: 1 x 4
  term                          statistic    df p.value
  <chr>                             <dbl> <dbl> <chr>
1 prior_programming_experience      754.     3 0.00001
```

```
## test each OR
#(sum_out <- summary(logreg_out))

sum_out <- broom::tidy(logreg_out_uni)
```

15.6 Multivariable model

```
pred_vars <- ColSeeker(namepattern = ".",
                       exclude = c("_id", "exam",
                                   "kaggle", "country"))
glm_formula <- reformulate(pred_vars$names, response = "passed_exam")
logreg_out <-
  glm(
    # case ~ `age [pentayears]` + education + parity_grp +
    #   induced_f + spontaneous_f,
    glm_formula, family = binomial(), data = rawdata
  )
logreg_out
```

```
Call:  glm(formula = glm_formula, family = binomial(), data = rawdata)

Coefficients:
                              (Intercept)
                                -17.273442
                                      age
                                  0.005711
    prior_programming_experienceBeginner
                                  4.025041
prior_programming_experienceIntermediate
                                  6.305788
    prior_programming_experienceAdvanced
                                  8.323812
                           weeks_in_course
                                 -0.003777
          hours_spent_learning_per_week
                                  0.498719
        practice_problems_solvedaverage
                                  0.251013
           practice_problems_solvedmany
                                  1.202738
                      projects_completed
                                  0.966498
                tutorial_videos_watched
                                  0.010889
    participates_in_discussion_forumsYes
                                  0.448354
             debugging_sessions_per_week
                                  0.210403
         self_reported_confidence_python
                                  0.380086

Degrees of Freedom: 2999 Total (i.e. Null);  2986 Residual
Null Deviance:      2804
Residual Deviance: 1229     AIC: 1257
```

```
# extract/transform model parameters
(ORs <- exp(logreg_out$coefficients))
```

```
                              (Intercept)
                                3.149484e-08
                                      age
```

```
                            1.005727e+00
    prior_programming_experienceBeginner
                            5.598262e+01
prior_programming_experienceIntermediate
                            5.477328e+02
    prior_programming_experienceAdvanced
                            4.120838e+03
                           weeks_in_course
                            9.962299e-01
          hours_spent_learning_per_week
                            1.646611e+00
         practice_problems_solvedaverage
                            1.285326e+00
            practice_problems_solvedmany
                            3.329221e+00
                       projects_completed
                            2.628723e+00
                  tutorial_videos_watched
                            1.010948e+00
    participates_in_discussion_forumsYes
                            1.565734e+00
             debugging_sessions_per_week
                            1.234175e+00
         self_reported_confidence_python
                            1.462410e+00
```

```
(CIs <- exp(confint(logreg_out)))
```

```
Waiting for profiling to be done...
```

```
                                            2.5 %       97.5 %
(Intercept)                          4.199248e-09 2.078012e-07
age                                  9.930644e-01 1.018590e+00
prior_programming_experienceBeginner 2.534626e+01 1.456518e+02
prior_programming_experienceIntermediate 2.372509e+02 1.483541e+03
prior_programming_experienceAdvanced 1.627747e+03 1.207575e+04
weeks_in_course                      9.632138e-01 1.030334e+00
hours_spent_learning_per_week        1.549488e+00 1.755507e+00
practice_problems_solvedaverage      9.129149e-01 1.817073e+00
practice_problems_solvedmany         2.220513e+00 5.029093e+00
projects_completed                   2.333932e+00 2.978338e+00
tutorial_videos_watched              9.875305e-01 1.034965e+00
```

```
participates_in_discussion_forumsYes     1.173755e+00 2.095184e+00
debugging_sessions_per_week              1.157360e+00 1.317647e+00
self_reported_confidence_python          1.381704e+00 1.551571e+00
```

```
# test model
(Anova_out <- Anova(logreg_out, type = 2) |>
  broom::tidy() |>
  mutate(p.value = formatP(p.value, ndigits = 5)))
```

```
# A tibble: 10 x 4
   term                                statistic    df p.value
   <chr>                                   <dbl> <dbl> <chr>
 1 age                                     0.780     1 0.37710
 2 prior_programming_experience         1005.        3 0.00001
 3 weeks_in_course                         0.0484    1 0.82588
 4 hours_spent_learning_per_week         364.        1 0.00001
 5 practice_problems_solved               38.5       2 0.00001
 6 projects_completed                    336.        1 0.00001
 7 tutorial_videos_watched                 0.829     1 0.36242
 8 participates_in_discussion_forums       9.34      1 0.00225
 9 debugging_sessions_per_week            42.3       1 0.00001
10 self_reported_confidence_python       210.        1 0.00001
```

```
## test each OR
#(sum_out <- summary(logreg_out))
```

```
sum_out <- broom::tidy(logreg_out)
```

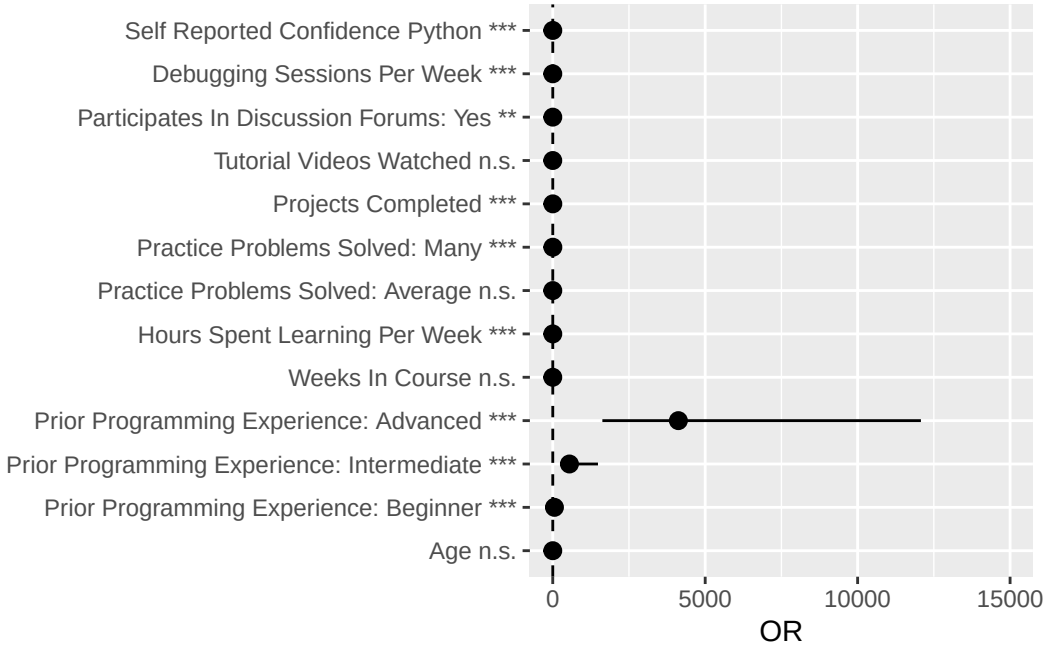
15.7 Create structure for ggplot

```
OR_plotdata <- tibble(
  Predictor = names(ORs)[-1] |>
    # make names nicer
    str_replace_all("_", " ") |>
    str_replace_all(c(
      "([[:lower:]])([[:upper:]])" = "\\1: \\2",
      "`" = "",
      "(.)(yes)" = "\\1: \\2",
      "(solved)(.+)" = "\\1: \\2"
    )) |>
    str_to_title() |>
```

```
    fct_inorder(),
  OR = ORs[-1],
  CI_low = CIs[-1, 1],
  CI_high = CIs[-1, 2],
  p = sum_out$p.value[-1],
  Significance = markSign(p),
  Label = paste(Predictor, Significance) |>
    fct_inorder()
)
```

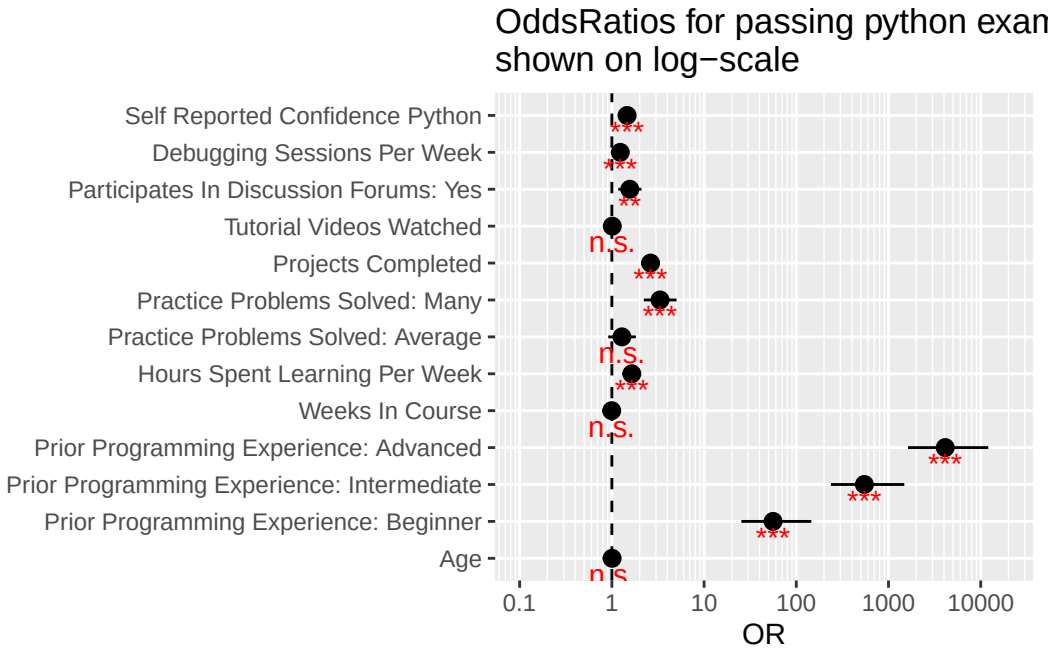
15.8 Create forest plot

```
  ggplot(OR_plotdata, aes(x = Label, y = OR)) +
  geom_pointrange(aes(ymin = CI_low, ymax = CI_high)) +
  geom_hline(yintercept = 1, linewidth = .5, linetype = 2) +
  coord_flip() +
  scale_y_continuous(limits = c(0.1, 1.5 * 10^4)) +
  xlab(NULL)
```




```
ggplot(OR_plotdata, aes(x = Predictor, y = OR)) +
  geom_pointrange(aes(ymin = CI_low, ymax = CI_high)) +
  geom_hline(yintercept = 1, linewidth = .5, linetype = 2) +
```

```
  coord_flip() +
  scale_y_log10(
    breaks = logrange_1,
    minor_breaks = logrange_123456789,
    limits = c(0.1, 2*10^4),
    labels = prettyNum) +
  geom_text(aes(label = Significance), vjust = 1.25, color = "red") +
  ggtitle("OddsRatios for passing python exam\nshown on log-scale") +
  xlab(NULL)
```



15.9 Create predictions

```
rawdata$pGLM <-
  predict(logreg_out, type = "response") # predict probability 0-1

rawdata |>
  group_by(passed_exam, pGLM >= .5) |>
  count()

# A tibble: 4 x 3
# Groups:   passed_exam, pGLM >= 0.5 [4]
  passed_exam `pGLM >= 0.5`     n
```

```
  <fct>       <lgl>         <int>
1 No          FALSE          2364
2 No          TRUE            104
3 Yes         FALSE           163
4 Yes         TRUE            369
```

```
rawdata |>
  group_by(passed_exam, pGLM >= .25) |>
  count()
```

```
# A tibble: 4 x 3
# Groups:   passed_exam, pGLM >= 0.25 [4]
  passed_exam `pGLM >= 0.25`     n
  <fct>       <lgl>          <int>
1 No          FALSE           2220
2 No          TRUE             248
3 Yes         FALSE             84
4 Yes         TRUE             448
```

```
# run ROC for cutoff
roc_out <- roc(
  response = rawdata$passed_exam,
  predictor = rawdata$pGLM
)
```

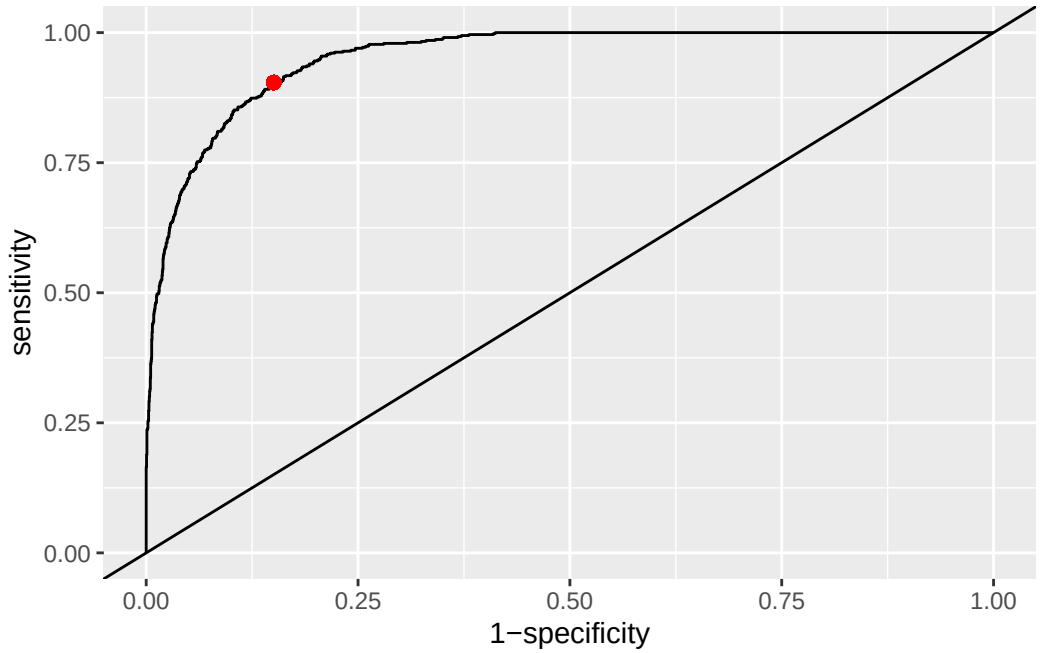
```
Setting levels: control = No, case = Yes
```

```
Setting direction: controls < cases
```

```
youden <- pROC::coords(roc_out,
  x = "best",
  best.method = "youden"
)
youden
```

```
  threshold specificity sensitivity
1  0.157032   0.8492707   0.9041353
```

```
ggroc(roc_out, legacy.axes = T) +
  geom_abline(slope = 1, intercept = 0) +
  geom_point(
    x = 1 - youden$specificity,
    y = youden$sensitivity, color = "red", size = 2
  )
```

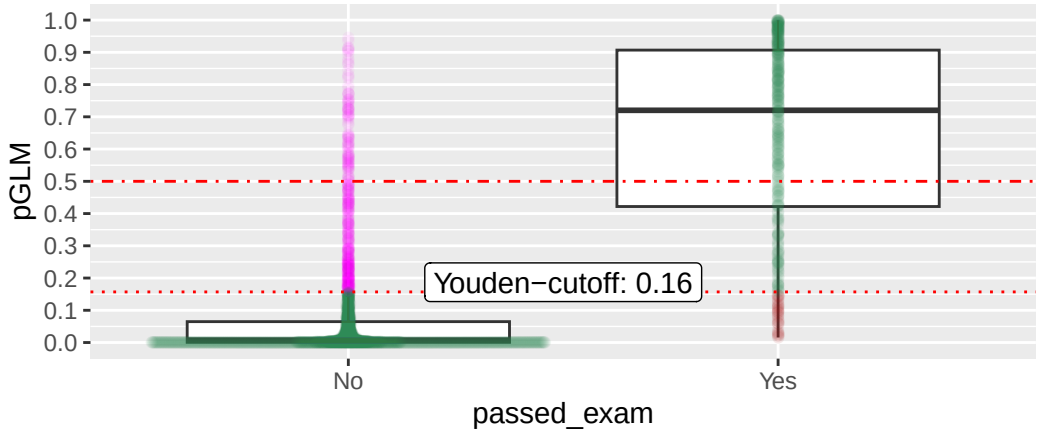



```
# plot predictions
tempdata <-
  rawdata |>
  mutate(
    `prediction` =
     case_when(
       passed_exam == "Yes" &
         pGLM < youden$threshold ~
         "false negative",
       passed_exam == "No" &
         pGLM >= youden$threshold
       ~ "false positive",
       passed_exam == "Yes" & pGLM >= youden$threshold ~
         "correct positive",
       .default = "correct negative"
     )
  )
prediction_counts <-
  tempdata |>
  group_by(prediction) |>
  count()

rawdata |>
```

```
mutate(
  `prediction quality` =
    case_when(
      passed_exam == "Yes" &
        pGLM < youden$threshold ~
        "false negative",
      passed_exam == "No" &
        pGLM >= youden$threshold
      ~ "false positive",
      .default = "correct"
    )
) |>
ggplot(aes(passed_exam, pGLM)) +
geom_boxplot(outlier.alpha = 0) +
scale_y_continuous(breaks = seq(0, 1, .1)) +
geom_beeswarm(
  alpha = .05, cex = .075,
  aes(color = `prediction quality`)
) +
scale_color_manual(values = c("seagreen", "firebrick", "magenta"),
                   ) +
guides(
  color = guide_legend(override.aes = list(alpha = .8))) +
geom_hline(
  yintercept = c(youden$threshold, .5),
  color = "red",
  linetype = 3:4
) +
annotate(
  geom = "label",
  x = 1.5, y = youden$threshold,
  label = paste(
    "Youden-cutoff:",
    roundR(youden$threshold)
  ),
  hjust = .5, vjust = 0.25
) +
theme(legend.position = "bottom") +
labs(
  caption =
    paste(prediction_counts$prediction,
      prediction_counts$n,
      collapse = "\n"
```

```
    )
)
```





correct negative 2096
correct positive 481
false negative 51
false positive 372

```
ORreport <-
  tibble(
    Predictor = names(ORs),
    OR = ORs,
    OR_low = CIs[, 1],
    OR_high = CIs[, 2]
  ) |>
  rowwise() |>
  mutate(
    across(
      -Predictor,
      \(x) roundR(x, level = 3)
    ),
    `OR(CI)` = paste0(OR, " (", OR_low, " / ", OR_high, ")")
  ) |>
  ungroup()

ORreport |>
  slice(-1) |>
  flextable::flextable() |>
```

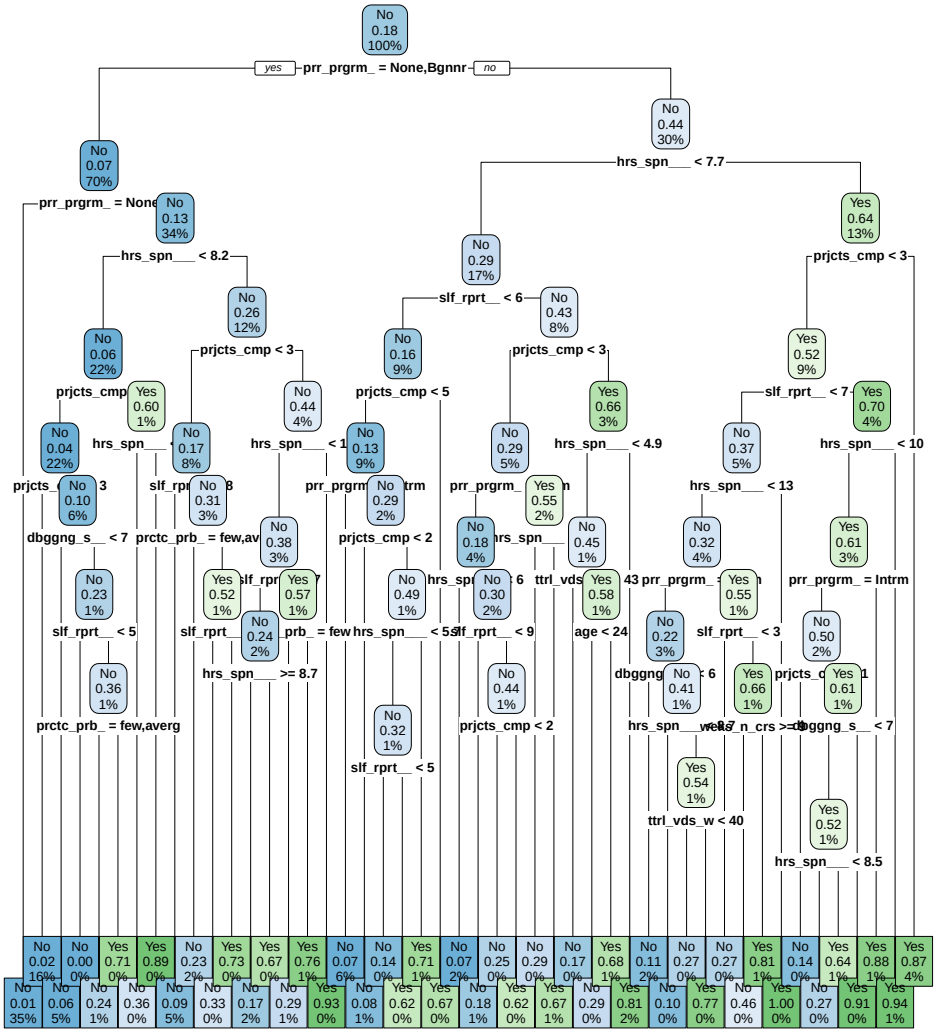
```
  flextable::set_table_properties(width = 1, layout = "autofit")
```

Predictor	OR	OR_low	OR_high	OR(CI)
age	1.01	0.993	1.02	1.01 (0.993 / 1.02)
prior_programming_experienceBeginner	56.0	25.3	146	56.0 (25.3 / 146)
prior_programming_experienceIntermediate	548	237	1484	548 (237 / 1484)
prior_programming_experienceAdvanced	4121	1628	12076	4121 (1628 / 12076)
weeks_in_course	0.996	0.963	1.03	0.996 (0.963 / 1.03)
hours_spent_learning_per_week	1.65	1.55	1.76	1.65 (1.55 / 1.76)
practice_problems_solvedaverage	1.29	0.913	1.82	1.29 (0.913 / 1.82)
practice_problems_solvedmany	3.33	2.22	5.03	3.33 (2.22 / 5.03)
projects_completed	2.63	2.33	2.98	2.63 (2.33 / 2.98)
tutorial_videos_watched	1.01	0.988	1.03	1.01 (0.988 / 1.03)
participates_in_discussion_forumsYes	1.57	1.17	2.10	1.57 (1.17 / 2.10)
debugging_sessions_per_week	1.23	1.16	1.32	1.23 (1.16 / 1.32)
self_reported_confidence_python	1.46	1.38	1.55	1.46 (1.38 / 1.55)

```
cat(" \n\n")
```

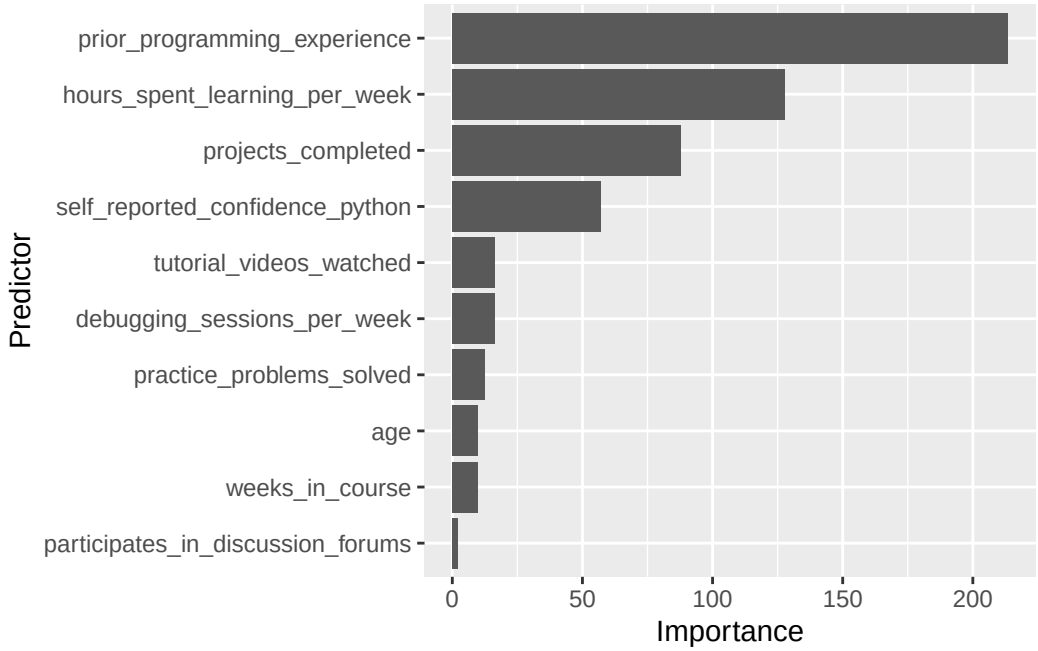

15.10 Regression tree as alternative to glm

```
regtree_out <- rpart(glm_formula,
  minsplit = 20, cp = .001,
  data = rawdata
)
rpart.plot(regtree_out, type = 2, tweak = 2.0,
           varlen = 10, faclen = 5, leaf.round = 0)
```




```
importance <-
  as_tibble(regtree_out$variable.importance,
    rownames = "Predictor"
  ) |>
  dplyr::rename("Importance" = 2) |>
  mutate(Predictor = fct_reorder(
    .f = Predictor,
    .x = Importance,
    .fun = min
  )) |>
```

```
  arrange(desc(Importance))
importance |>
  ggplot(aes(Predictor, Importance)) +
  geom_col() +
  coord_flip()
```




```
rawdata$pRT <- predict(regtree_out)[, 2]

# pROC
roc_out_rt <- roc(
  response = rawdata$passed_exam,
  predictor = rawdata$pRT
)
```

```
Setting levels: control = No, case = Yes
```

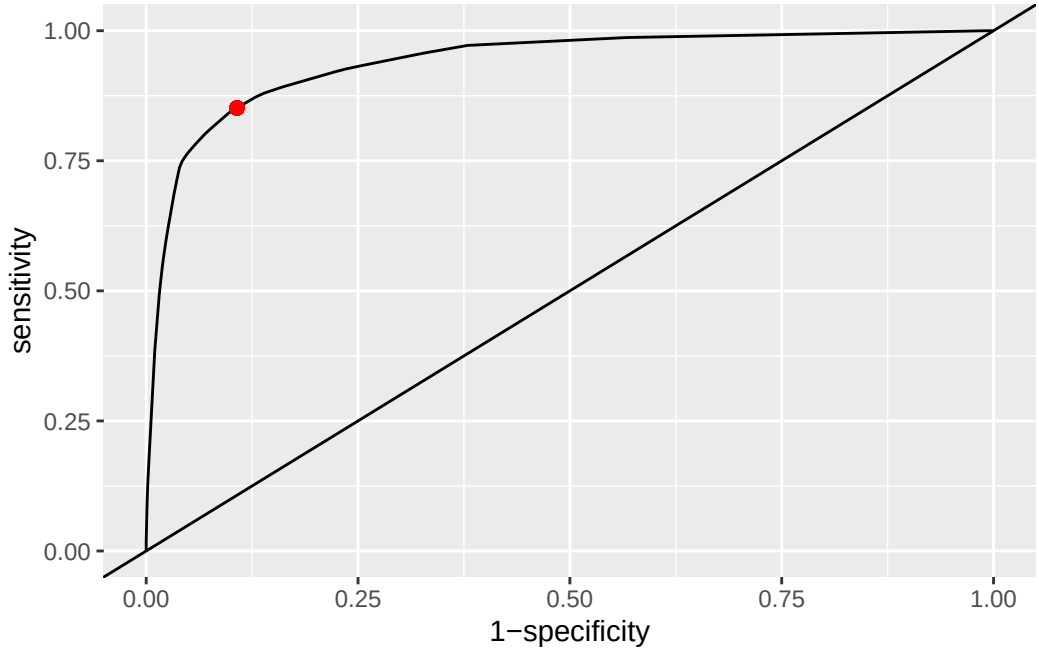
```
Setting direction: controls < cases
```

```
youden <- pROC::coords(roc_out_rt,
  x = "best",
  best.method = "youden")

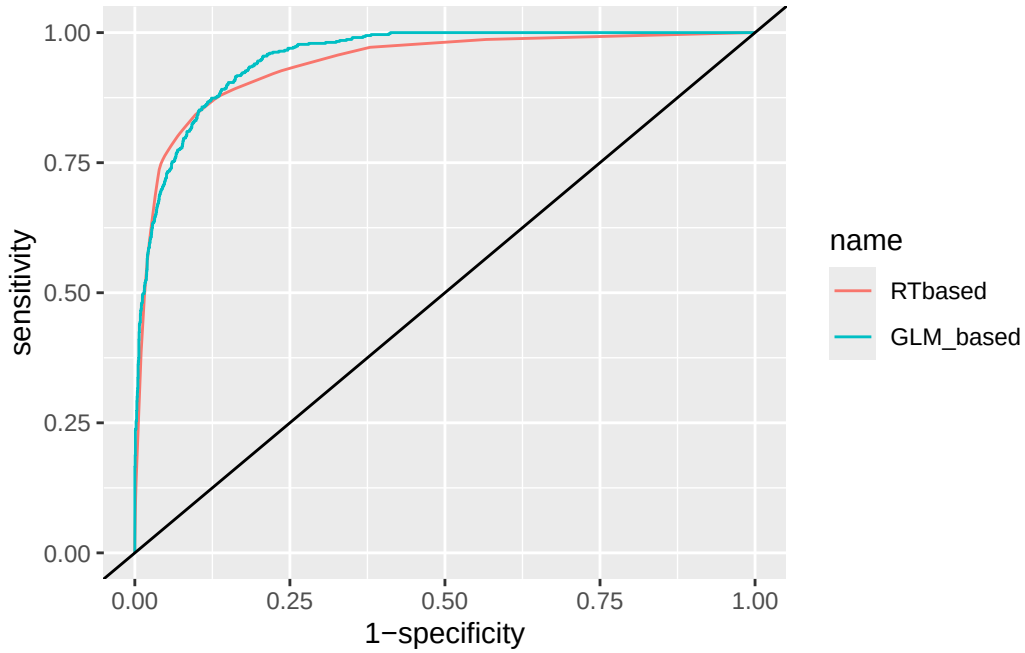
youden
```

```
  threshold specificity sensitivity
1 0.1741914   0.8930308   0.8515038
```

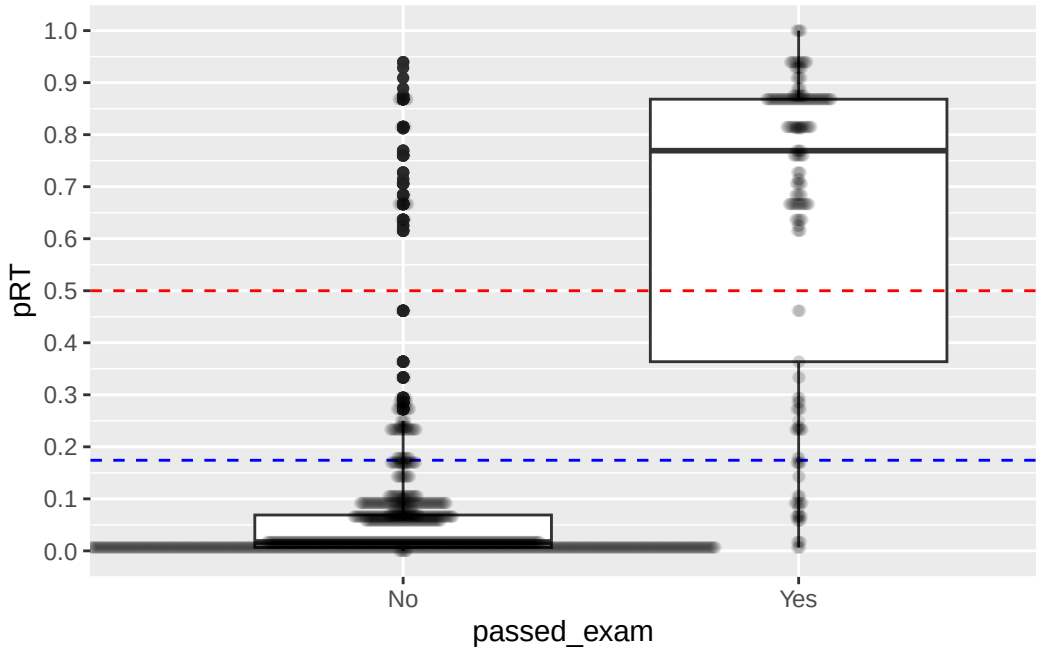
```
ggroc(roc_out_rt, legacy.axes = T) +
  geom_abline(slope = 1, intercept = 0) +
  geom_point(
    x = 1 - youden$specificity,
    y = youden$sensitivity, color = "red", size = 2
  )
```




```
ggroc(list(RTbased = roc_out_rt, GLM_based = roc_out), legacy.axes = T) +
  geom_abline(slope = 1, intercept = 0)
```




```
ggplot(rawdata, aes(x = passed_exam, y = pRT)) +
  geom_boxplot(coef = 3) +
  scale_y_continuous(breaks = seq(from = 0, to = 1, by = .1)) +
  geom_hline(
    yintercept = c(.5, youden$threshold),
    color = c("red", "blue"), linetype = 2
  ) +
  ggbeeswarm::geom_beeswarm(alpha = .05, cex = 0.075)
```


1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
pRT
No
Yes
passed_exam

16 Linear Mixed Models

Linear Mixed-Effects Models (LMMs) are statistical tools used to analyze data where observations are grouped or repeated, allowing researchers to model both **fixed effects** (effects for which an estimate/prediction and a p-value are of interest, which represent population-level averages, like treatment type) and **random effects** (effects for which usually no prediction or p-value is required, which account for variation and non-independence among the groups, like individual subjects, families, or experimental batches). In biology, LMMs are essential for analyzing longitudinal studies (e.g., tracking an animal's growth over time), repeated measure experiments (e.g., measuring gene expression in the same tissue across different conditions, or exploring experimental variation between technical replicates), and ecological studies (e.g., accounting for variation between different field sites or populations).

```
pacman::p_load(
  conflicted, tidyverse, wrappedtools, here,
  car, broom, multcomp, DescTools, haven,
  ggbeeswarm,
  lme4, nlme, merTools,
  easystats, patchwork
)

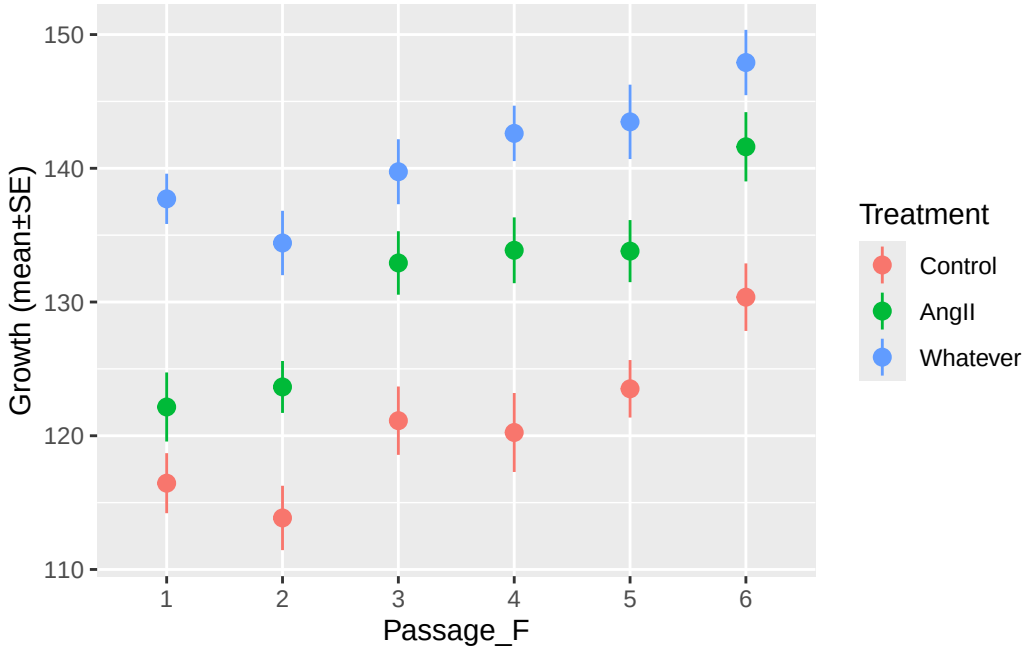
conflicts_prefer(
  modelbased::standardize,
  dplyr::filter,
  dplyr::select,
  dplyr::recode_values
)
```

16.1 Import / Preparation

The data are the same as for the lm analyses. This time, passage effect is modeled as a random effect, as we are not interested in the effect of each passage, but want to account for the variation between passages. Future experiments are not expected to have the exact same passage differences. Passage numbers are arbitrary anyway. The numeric column Passage is mutated into a factor as Passage_F, this is necessary for group comparisons in ANOVA.

```
rawdata <- read_sav(file = here("data/Zellbeads.sav")) |>
  as_factor() |>
  dplyr::select(-ZahlZellen) |>
  rename(Growth = Wachstum, Treatment = Bedingung) |>
  mutate(
    Passage_F = factor(Passage),
    Treatment = fct_recode(Treatment,
      Control = "Kontrolle"
    )
  )

ggplot(rawdata, aes(Passage_F, Growth, color = Treatment)) +
  stat_summary(fun.data = mean_se) +
  ylab("Growth (mean\u00b1SE)")
```



16.2 Random intercept model

16.2.1 Package nlme

```
lme_out <- nlme::lme(Growth ~ Treatment,
  data = rawdata,
  random = ~ 1 | Passage_F
) # RIntercept
lme_out
```

```
Linear mixed-effects model fit by REML
  Data: rawdata
  Log-restricted-likelihood: -1367.644
  Fixed: Growth ~ Treatment
      (Intercept)    TreatmentAngII TreatmentWhatever
        120.92499          10.40895          20.05199

Random effects:
 Formula: ~1 | Passage_F
        (Intercept) Residual
StdDev:    5.644869 10.71643

Number of Observations: 360
Number of Groups: 6
```

```
Anova(lme_out)
```

```
Analysis of Deviance Table (Type II tests)

Response: Growth
           Chisq Df Pr(>Chisq)
Treatment 210.17  2  < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

16.2.2 Package lme4

```
lmer_out <- lme4::lmer(Growth ~ Treatment + (1 | Passage_F),
  data = rawdata
)

lmer_out
```

```
Linear mixed model fit by REML ['lmerMod']
Formula: Growth ~ Treatment + (1 | Passage_F)
   Data: rawdata
REML criterion at convergence: 2735.287
Random effects:
 Groups    Name        Std.Dev.
 Passage_F (Intercept)  5.645
 Residual              10.716
Number of obs: 360, groups:  Passage_F, 6
Fixed Effects:
      (Intercept)     TreatmentAngII  TreatmentWhatever
           120.92              10.41              20.05
```

```
Anova(lmer_out)
```

```
Analysis of Deviance Table (Type II Wald chisquare tests)

Response: Growth
           Chisq Df Pr(>Chisq)
Treatment 210.17  2  < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

16.2.3 Visualization of fixed and random effects

Parameters can be conveniently extracted with `model_parameters()`. The intercept is the mean of the reference group (Control), the slopes are the differences to the reference group. The random effects are deviations from the overall intercept for each passage. The overall intercept and slope are shown as dashed line, the passage-specific intercepts with identical slopes as colored lines.

```
(lmer_out_param <- model_parameters(lmer_out, group_level = T))
```

```
# Fixed Effects

Parameter             | Coefficient |   SE |           95% CI | t(355) |      p
-------------------------------------------------------------------------------
(Intercept)           |      120.92 | 2.50 | [116.00, 125.85] |  48.30 | < .001
Treatment [AngII]     |       10.41 | 1.38 | [  7.69,  13.13] |   7.52 | < .001
Treatment [Whatever]  |       20.05 | 1.38 | [ 17.33,  22.77] |  14.49 | < .001

# Random Effects: Passage_F

Parameter        | Coefficient |   SE |         95% CI
-------------------------------------------------------
(Intercept) [1]  |       -5.32 | 1.34 | [-7.95, -2.69]
(Intercept) [2]  |       -6.70 | 1.34 | [-9.34, -4.07]
(Intercept) [3]  |        0.17 | 1.34 | [-2.46,  2.81]
(Intercept) [4]  |        1.10 | 1.34 | [-1.54,  3.73]
(Intercept) [5]  |        2.37 | 1.34 | [-0.26,  5.01]
(Intercept) [6]  |        8.38 | 1.34 | [ 5.74, 11.01]


Uncertainty intervals (equal-tailed) and p-values (two-tailed) computed
  using a Wald t-distribution approximation.
```

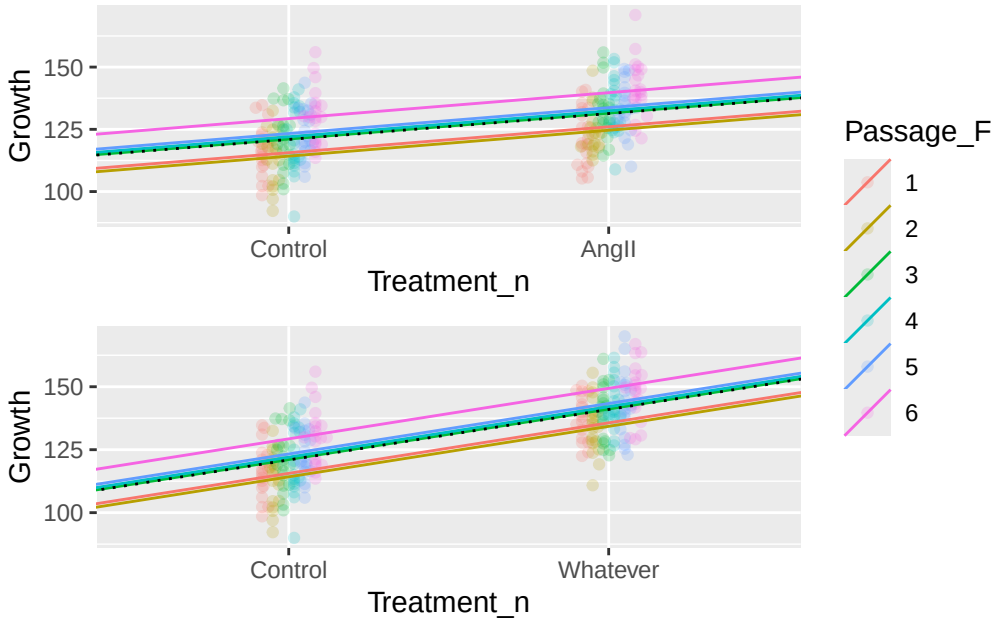
```
intercept_all <- lmer_out_param$Coefficient[1]
slope_Ang <- lmer_out_param$Coefficient[2]
slope_What <- lmer_out_param$Coefficient[3]
plotdata <- tibble(
  Passage_F = lmer_out_param$Level[-(1:3)],
  intercept = lmer_out_param$Coefficient[-(1:3)] +
    intercept_all
)
```

```
p1 <- rawdata |>
  filter(Treatment != "Whatever") |>
  mutate(Treatment_n = recode_values(Treatment, "Control" ~ 0,
    default = 1
  )) |>
  ggplot(aes(Treatment_n, Growth, color = Passage_F)) +
  geom_beeswarm(alpha = .2, dodge.width = .2) +
  geom_abline(
    data = plotdata,
    aes(
      color = Passage_F,
      intercept = intercept,
      slope = slope_Ang
    )
  ) +
  geom_abline(
    color = "black", linetype = 3,
    aes(
      intercept = intercept_all,
      slope = slope_Ang
    )
  ) +
  scale_x_continuous(
    breaks = 0:1,
    limits = c(-.5, 1.5),
    labels = c("Control", "AngII"),
    minor_breaks = NULL
  )

p2 <- rawdata |>
  filter(Treatment != "AngII") |>
  mutate(Treatment_n = recode_values(Treatment, "Control" ~ 0,
    default = 1
  )) |>
  ggplot(aes(Treatment_n, Growth, color = Passage_F)) +
  geom_beeswarm(alpha = .2, dodge.width = .2) +
  geom_abline(
    data = plotdata,
    aes(
      color = Passage_F,
      intercept = intercept,
      slope = slope_What
```

```
    )
  ) +
  geom_abline(
    color = "black", linetype = 3,
    aes(
      intercept = intercept_all,
      slope = slope_What
    )
  ) +
  scale_x_continuous(
    breaks = 0:1,
    limits = c(-.5, 1.5),
    labels = c("Control", "Whatever"),
    minor_breaks = NULL
  )

p1 / p2 + plot_layout(guides = "collect")
```



The regression lines for the passages are parallel but shifted, reflecting the random intercept model.

16.3 Random slope model

```
lmer_out2 <- lme4::lmer(
  Growth ~ Treatment +
    (1 + Treatment | Passage_F),
  data = rawdata,
  REML = FALSE
)
```

```
boundary (singular) fit: see help('isSingular')
```

```
lmer_out2
```

```
Linear mixed model fit by maximum likelihood  ['lmerMod']
Formula: Growth ~ Treatment + (1 + Treatment | Passage_F)
   Data: rawdata
      AIC       BIC    logLik -2*log(L)  df.resid
 2760.793  2799.654 -1370.397  2740.793       350
Random effects:
 Groups    Name              Std.Dev. Corr
 Passage_F (Intercept)        4.9984
           TreatmentAngII     1.3240   1.00
           TreatmentWhatever  0.9348  -1.00 -1.00
 Residual                     10.6434
Number of obs: 360, groups:  Passage_F, 6
Fixed Effects:
      (Intercept)     TreatmentAngII  TreatmentWhatever
           120.92              10.41              20.05
optimizer (nloptwrap) convergence code: 0 (OK) ; 0 optimizer warnings; 1 lme4 warnings
```

```
Anova(lmer_out2)
```

```
Analysis of Deviance Table (Type II Wald chisquare tests)

Response: Growth
           Chisq Df Pr(>Chisq)
Treatment 202.85  2  < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

16.3.1 Visualization of fixed and random effects

```
(lmer_out_param2 <- model_parameters(lmer_out2, group_level = T))

# Fixed Effects

Parameter            | Coefficient |   SE |          95% CI | t(350) |      p
-------------------------------------------------------------------------------
(Intercept)          |      120.92 | 2.26 | [116.48, 125.37] |  53.50 | < .001
Treatment [AngII]    |       10.41 | 1.48 | [  7.50,  13.31] |   7.05 | < .001
Treatment [Whatever] |       20.05 | 1.43 | [ 17.25,  22.86] |  14.06 | < .001

# Random Effects: Passage_F

Parameter             | Coefficient |   SE |         95% CI
-----------------------------------------------------------
(Intercept) [1]       |       -5.37 | 1.27 | [-7.87, -2.88]
(Intercept) [2]       |       -6.34 | 1.27 | [-8.84, -3.85]
(Intercept) [3]       |        0.35 | 1.27 | [-2.15,  2.84]
(Intercept) [4]       |        1.10 | 1.27 | [-1.39,  3.60]
(Intercept) [5]       |        2.22 | 1.27 | [-0.28,  4.72]
(Intercept) [6]       |        8.05 | 1.27 | [ 5.55, 10.55]
TreatmentAngII [1]    |       -1.42 | 0.34 | [-2.08, -0.76]
TreatmentAngII [2]    |       -1.68 | 0.34 | [-2.34, -1.02]
TreatmentAngII [3]    |        0.09 | 0.34 | [-0.57,  0.75]
TreatmentAngII [4]    |        0.29 | 0.34 | [-0.37,  0.95]
TreatmentAngII [5]    |        0.59 | 0.34 | [-0.07,  1.25]
TreatmentAngII [6]    |        2.13 | 0.34 | [ 1.47,  2.79]
TreatmentWhatever [1] |        1.00 | 0.24 | [ 0.54,  1.47]
TreatmentWhatever [2] |        1.19 | 0.24 | [ 0.72,  1.65]
TreatmentWhatever [3] |       -0.06 | 0.24 | [-0.53,  0.40]
TreatmentWhatever [4] |       -0.21 | 0.24 | [-0.67,  0.26]
TreatmentWhatever [5] |       -0.42 | 0.24 | [-0.88,  0.05]
TreatmentWhatever [6] |       -1.51 | 0.24 | [-1.97, -1.04]


Uncertainty intervals (equal-tailed) and p-values (two-tailed) computed
  using a Wald t-distribution approximation.
```

```
intercept_all <- lmer_out_param2$Coefficient[1]
slope_Ang <- lmer_out_param2$Coefficient[2]
slope_What <- lmer_out_param2$Coefficient[3]
```

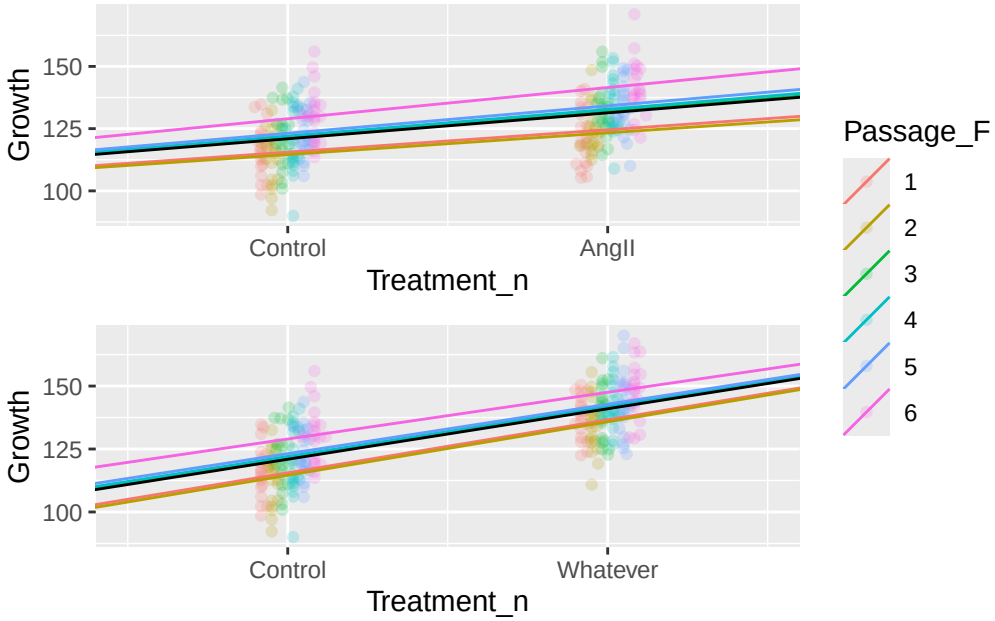
```r
interceptdata <-
  tibble(
    Passage_F = lmer_out_param2$Level[4:9],
    intercept = lmer_out_param2$Coefficient[4:9] + intercept_all,
    slopeAng = lmer_out_param2$Coefficient[10:15] +
      slope_Ang,
    slopeWhat = lmer_out_param2$Coefficient[16:21] +
      slope_What
  )

p1 <- rawdata |>
  filter(Treatment != "Whatever") |>
  mutate(Treatment_n = recode_values(Treatment, "Control" ~ 0,
    default = 1
  )) |>
  ggplot(aes(Treatment_n, Growth, color = Passage_F)) +
  geom_beeswarm(alpha = .2, dodge.width = .2) +
  geom_abline(
    data = interceptdata,
    aes(
      color = Passage_F,
      intercept = intercept,
      slope = slopeAng
    )
  ) +
  geom_abline(
    color = "black",
    aes(
      intercept = intercept_all,
      slope = slope_Ang
    )
  ) +
  scale_x_continuous(
    breaks = 0:1,
    limits = c(-.5, 1.5),
    labels = c("Control", "AngII")
  )

p2 <- rawdata |>
  filter(Treatment != "AngII") |>
  mutate(Treatment_n = recode_values(Treatment, "Control" ~ 0,
    default = 1
  )) |>
```

```
  ggplot(aes(Treatment_n, Growth, color = Passage_F)) +
  geom_beeswarm(alpha = .2, dodge.width = .2) +
  geom_abline(
    data = interceptdata,
    aes(
      color = Passage_F,
      intercept = intercept,
      slope = slopeWhat
    )
  ) +
  geom_abline(
    color = "black",
    aes(
      intercept = intercept_all,
      slope = slope_What
    )
  ) +
  scale_x_continuous(
    breaks = 0:1,
    limits = c(-.5, 1.5),
    labels = c("Control", "Whatever")
  )

p1 / p2 + plot_layout(guides = "collect")
```



This time, the Passage-related lines have individual intercepts AND slopes, indicating that the treatment effects are not exactly identical within the passages.

16.4 Exercise:

With the penguin data, build random intercept and random slope models with bill depth as dependent and weight as independent variable. Treat species (and possibly sex) as the random effects, in which intercept / slope are nested.

Visualize the random effects.

17 Machine Learning with R: Basic concepts

17.1 Literature links

Feature engineering

Tidy Modeling with R

Tidy Models Cheat Sheets

17.2 structured vs. unstructured data

17.3 supervised vs. unsupervised methods

17.4 Resampling methods

- Creation of new samples based on one observed sample.
- Helps in creating new synthetic datasets for training machine learning models and to estimate properties of a dataset when the dataset is unknown, difficult to estimate, or when the sample size of the dataset is small.
- Resampling Methods
  - Permutation tests (also re-randomization tests)
  - Bootstrapping
  - Jackknife (Leave one out validation)
  - Cross validation

17.4.1 Permutation tests

- Permutation reshuffles the observed cases, sampling without replacement, e.g. Random combination of 2 classes (like disease / risk)
- Original:

Class 1	Class 2
Affected	Exposed
Affected	Exposed
Affected	-
Control	-
Control	-

- Permutation 1

Class 1	Class 2
Affected	-
Affected	Exposed
Affected	-
Control	-
Control	Exposed

- repeated x times
- count, how frequent results as "extreme" as the observation occur by coincidence

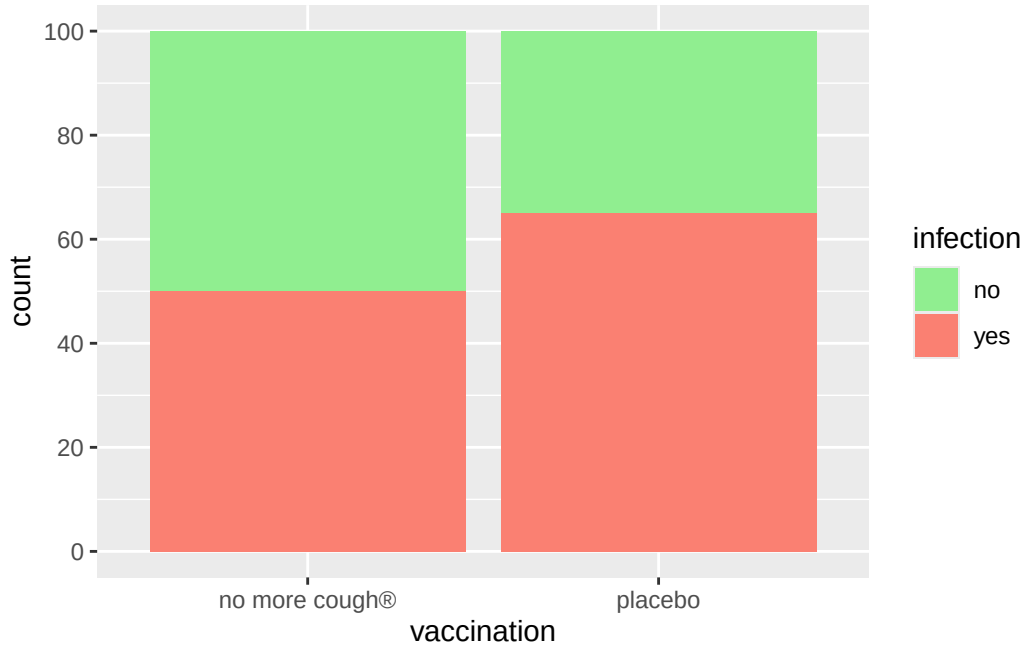
```
pacman::p_load(
  conflicted, wrappedtools, tidyverse, ggbeeswarm,
  broom, boot, Hmisc
)

conflicts_prefer(
  dplyr::filter,
  dplyr::select
)
```

Simulation of a weak effect with vaccination reducing cough incidence from 70% to 55%:

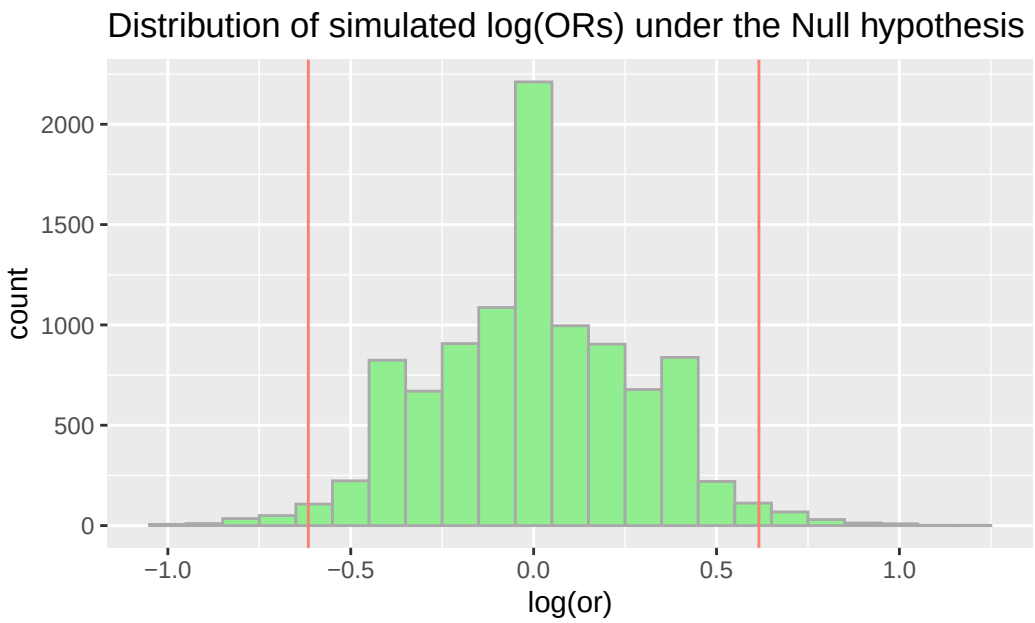
```
set.seed(16)
vaccination <-
  tibble(
    vaccination = rep(c("no more cough\u00ae", "placebo"), each = 100),
```

```
    infection = c(
      sample(c("yes", "no"),
        size = 100,
        replace = TRUE, prob = c(.55, .45)
      ),
      sample(c("yes", "no"),
        size = 100,
        replace = TRUE, prob = c(.7, .3)
      )
    )
  )
vaccination |>
  ggplot(aes(vaccination, fill = infection)) +
  geom_bar() +
  scale_fill_manual(values = c("lightgreen", "salmon")) +
  scale_y_continuous(breaks = seq(0, 1000, 20))
```




```
observed_or <- fisher.test(
  vaccination$vaccination,
  vaccination$infection
)$estimate
estimated_p <- fisher.test(vaccination$vaccination,
  vaccination$infection,
```

```
  simulate.p.value = FALSE
)$p.value
extremes_log_or <- c(-log(observed_or), log(observed_or))
n_reps <- 10^4
simulated_or <- tibble(
  run_i = 1:n_reps,
  or = NA_real_
)
for (shuffle_i in seq_len(n_reps)) {
  shuffled_infection <- sample(vaccination$infection)
  simulated_or$or[shuffle_i] <-
    fisher.test(
      vaccination$vaccination,
      shuffled_infection
    )$estimate
}
ggplot(simulated_or, aes(x = log(or))) +
  geom_histogram(
    binwidth = .1, center = 0,
    fill = "lightgreen", color = "darkgrey"
  ) +
  geom_vline(xintercept = extremes_log_or, color = "salmon") +
  labs(
    title = "Distribution of simulated log(ORs) under the Null hypothesis",
    caption = "Vertical lines indicate the observed log(OR) and its negative counterpart"
  )
```

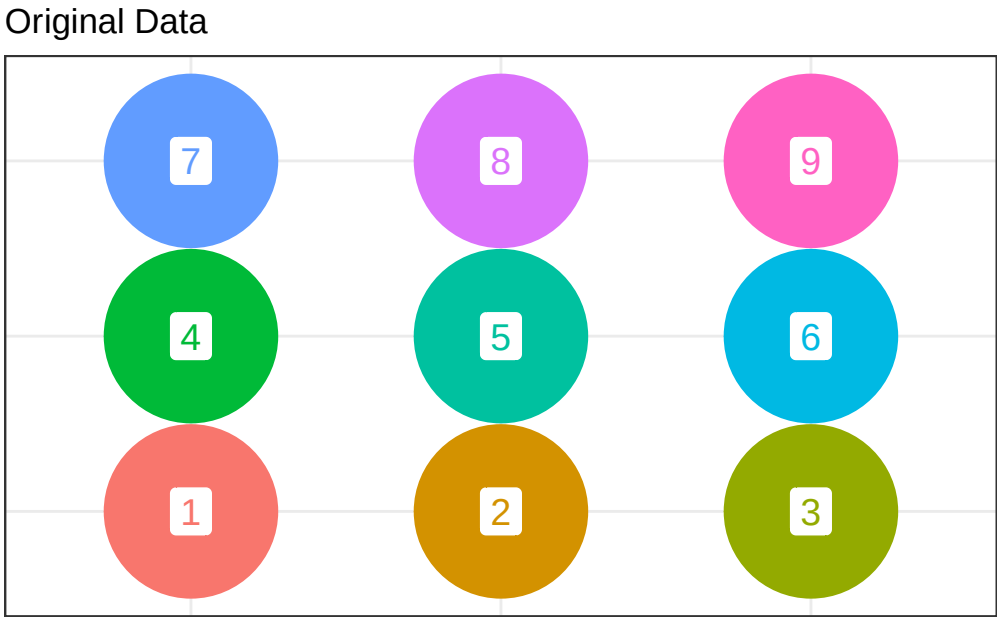


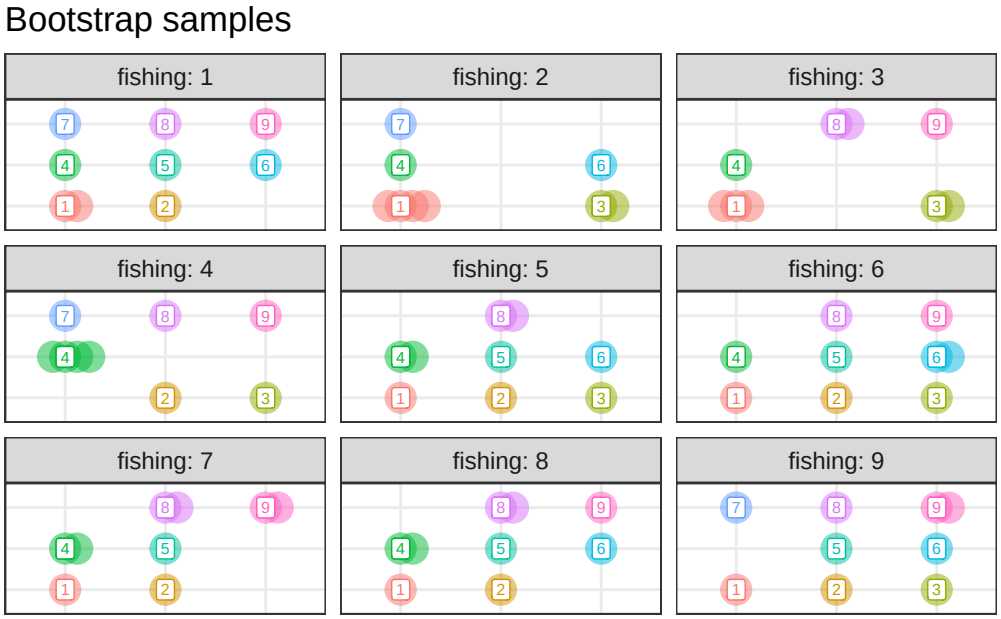

```
extremes_count <-
  table(between(
    log(simulated_or$or),
    extremes_log_or[1], extremes_log_or[2]
  ))
```

The conventional p-value is 0.04493, while shuffling the data resulted in 0.0223% of the simulated ORs being as extreme as the observed OR.

17.4.2 Bootstrapping

- Bootstrapping selects from the population of observed cases, sampling with replacement.
- Bootstrap is a powerful statistical tool used to quantify the uncertainty of a given model.

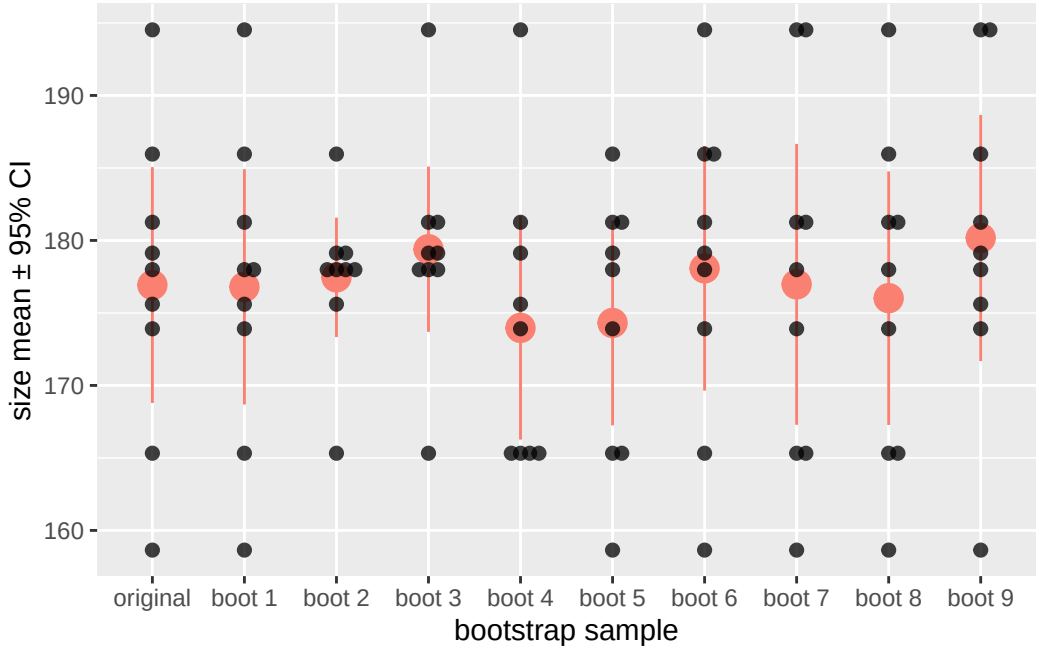




Based on the bootstrap samples, we can visualize variation in outcome **size**:

```
desc_boot <-
  boot_data |>
```

```
  # filter(fishing != 0) |>
  ggplot(aes(x = factor(fishing), y = size)) +
  stat_summary(
    color = "salmon", fun.data = mean_cl_normal,
    size = 1
  ) +
  geom_beeswarm(alpha = .75, size = 2) +
  ylab("size mean \u00b1 95% CI") +
  scale_x_discrete("bootstrap sample",
    labels = c("original", paste("boot", 1:9))
  )
desc_boot
```



A large number of bootstrap (>1000) samples is usually required to obtain stable results. We simulate 50 cases with true mean of 170 and true SD of 10:

```
n_subjects <- 50
real_mean <- 170
real_sd <- 10
n_boot <- 10^4
set.seed(12345)
experimental_data <-
  rnorm(n = n_subjects, mean = real_mean, sd = real_sd)
```

Conventional estimates from sample:

```
meansd(experimental_data, roundDig = 4)
```

```
[1] "171.8 ± 11.0"
```

```
meanse(experimental_data, roundDig = 4)
```

```
[1] "171.8 ± 1.6"
```

```
smean.cl.normal(experimental_data)
```

```
    Mean    Lower    Upper
171.7957 168.6792 174.9121
```

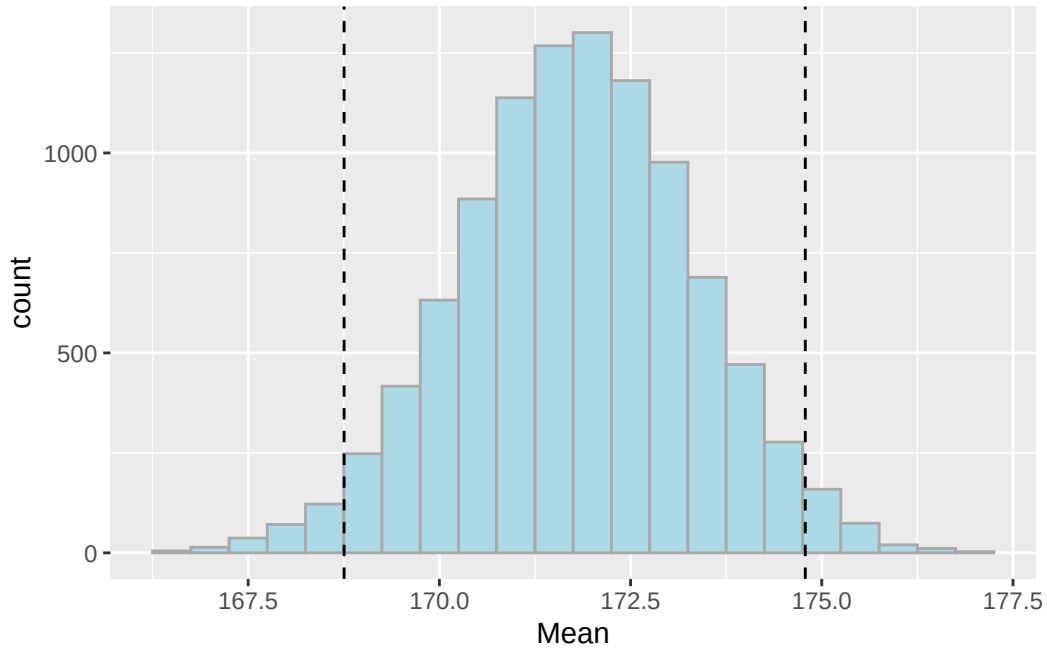
Bootstrap estimates:

```
smean.cl.boot(experimental_data, B = n_boot)
```

```
    Mean    Lower    Upper
171.7957 168.7258 174.8658
```

```
# looped bootstraping
boot_means <- tibble(Mean = NA_real_, .rows = 0)
for (run_i in seq_len(n_boot)) {
  boot_means <-
    add_row(boot_means,
      Mean = mean(sample(
        x = experimental_data,
        size = n_subjects,
        replace = TRUE
      ))
    )
}

cl_boot <- quantile(boot_means$Mean, probs = c(.025, .975))
ggplot(boot_means, aes(Mean)) +
  geom_histogram(binwidth = .5, fill = "lightblue", color = "darkgrey") +
  geom_vline(xintercept = cl_boot, linetype = 2)
```




```r
# package boot
# Custom function to calculate the mean for each resample
boot_mean <- function(d, i) {
  d[i] |> mean()
}
# Sample data
# Execute n_boot bootstrap replicates
boot_results <- boot(
  data = experimental_data,
  statistic = boot_mean,
  R = n_boot
)

# View the result
boot_results
```

```
ORDINARY NONPARAMETRIC BOOTSTRAP


Call:
boot(data = experimental_data, statistic = boot_mean, R = n_boot)
```

```
Bootstrap Statistics :
    original      bias    std. error
t1* 171.7957 -0.01456677    1.534077
```

```r
# Get multiple types of CIs
boot_ci <- boot.ci(boot_results, type = "basic")

print(boot_ci)
```

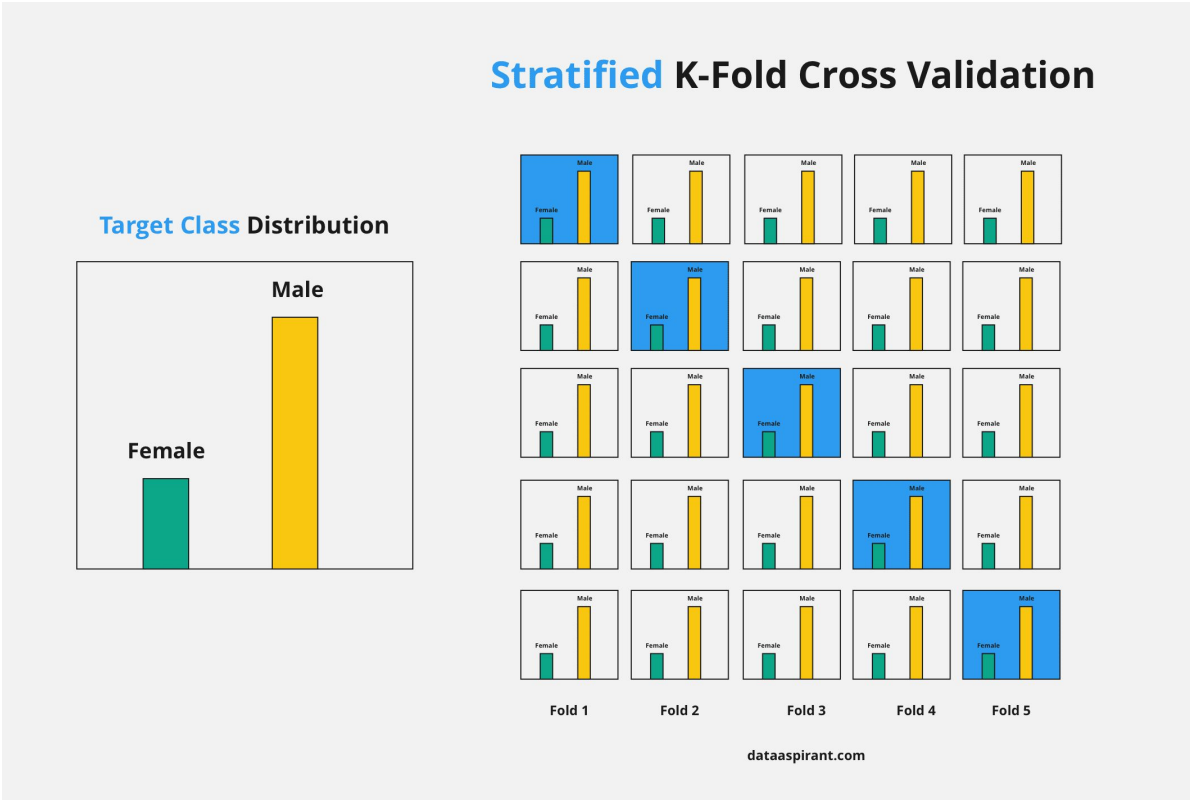
```
BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 10000 bootstrap replicates

CALL :
boot.ci(boot.out = boot_results, type = "basic")

Intervals :
Level      Basic
95%   (168.8, 174.9 )
Calculations and Intervals on Original Scale
```

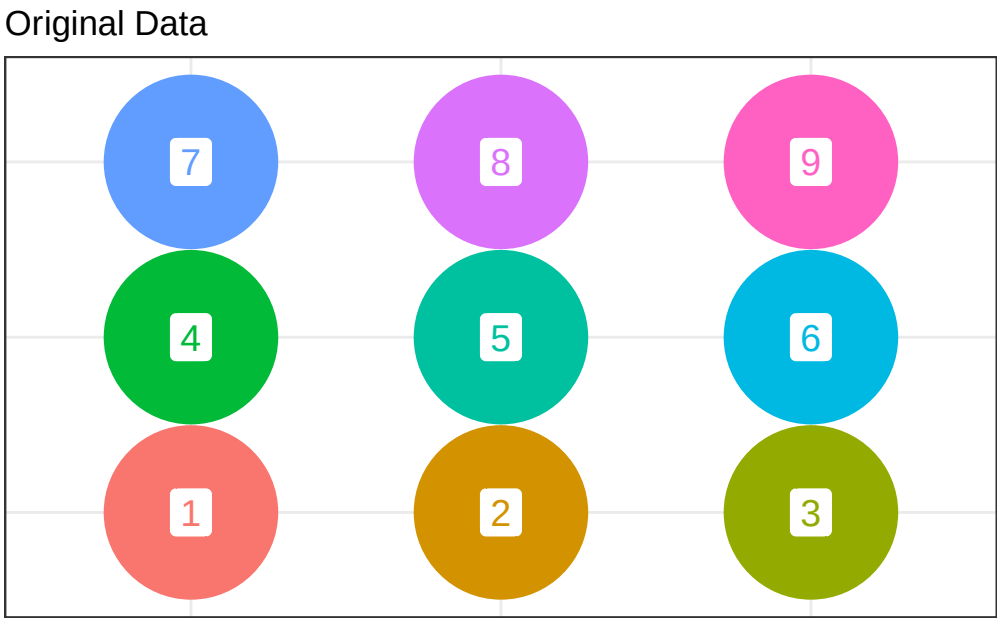
17.4.3 Cross validation

- Cross-validation is a resampling method used to evaluate the performance of a model on unseen data.
- It involves partitioning the data into `k` subsets, training the model on one subset (training set) made of k-1 subsets, and evaluating it on the left-out subset (testing set).
- This process is repeated `k` times, with each subset serving as the testing set once, and the results are averaged to provide an overall performance estimate.
- Cross-validation is often repeated multiple times to reduce the variance of the performance estimates and to get a more reliable estimate of the model's performance. 10 repeated 5-fold cross-validations would provide 50 estimates of the model's performance, capturing average as well as possible extreme results.


Stratified K-Fold Cross Validation
Target Class Distribution
Male
Female
Male
Female
Fold 1
Fold 2
Fold 3
Fold 4
Fold 5
dataaspirant.com

17.4.4 Jacknife

- Jackknife (or **L**eave **O**ne **O**ut) is resampling without replacement, where one observation is left out at a time to create new samples. It is used to estimate the bias and variance of a statistic, as well as to construct confidence intervals.
- Creates n samples of size n-1



LOO samples



18 Markov Chain Monte Carlo: Metropolis algorithm simulation

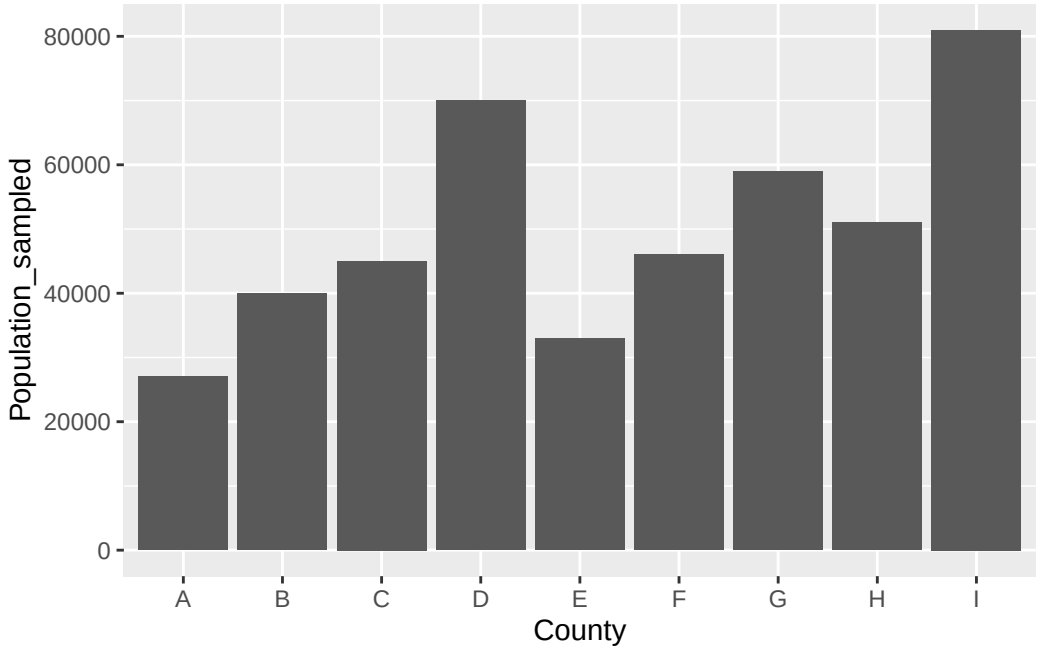
based on Markov Chain Monte Carlo | Columbia Public Health with slightly changed assumptions (10 counties, pre-defined populations either systematically chosen or randomized) and these rules:

1. Flip a coin. Heads to move east, tails to move west. If in borderline county, wrap around (alternative: move towards center?)
2. If the district indicated by the coin (east or west) has more voters than the present district, move there.
3. If the district indicated by the coin has fewer likely voters, make the decision based on a probability calculation:
4. calculate the probability of moving as the ratio of the number of likely voters in the proposed district, to the number of voters in the current district:
5. Pr[move] = voters in indicated district/voters in present district
6. Take a random sample between 0 and 1.
7. If the value of the random sample is between 0 and the probability of moving, move. Otherwise, stay put.

```
pacman::p_load(
  wrappedtools, tidyverse, ggrepel, ggforce,
  ggnewscale, ggtext, tictoc
)

set.seed(1012)
counties <- tibble(
  County = LETTERS[1:9],
  Population_defined = seq(
    from = 10^4,
    to = 9 * 10^4,
    by = 10^4
  ),
```

```
  Population_sampled = runif(
    n = 9,
    min = 10^4,
    max = 9 * 10^4
  ) |>
    roundR(
      level = 2,
      textout = F,
      smooth = T
    )
)
pop_selected <- "Population_sampled"
ggplot(counties, aes(County, .data[[pop_selected]])) +
  geom_col()
```



18.1 Rule definition

```
move_selection <- function(.counties = counties,
                           current_county,
                           which_population = pop_selected) {
  coinresult <- sample(
    x = c(1, -1),
```

```
    size = 1
  )
  # if(current_county == 1) {
  #   coinresult <- 1
  # }
  # if(current_county == nrow(.counties)) {
  #   coinresult <- -1
  # }
  next_county <- current_county + coinresult
  if (next_county == 0) {
    next_county <- nrow(.counties)
  }
  if (next_county > nrow(counties)) {
    next_county <- 1
  }
  population_ratio <- .counties[[next_county, which_population]] /
    .counties[[current_county, which_population]]
  if (runif(n = 1, 0, 1) > population_ratio) {
    next_county <- current_county
  }
  return(next_county)
}

n_moves <- 10^5
n_burnin <- 10^3
start_county <- 5
```

18.2 Data structures for simulation

```
moves <- tibble(
  move = seq_len(n_moves),
  position = NA_integer_
)
moves$position[1] <- start_county
```

18.3 Simulation

```
set.seed(1210)
tictoc::tic("here we go...")
for (step_i in 2:n_moves) {
```

```
  moves$position[step_i] <-
    move_selection(current_county = moves$position[step_i - 1])
}
tictoc::toc()
```

```
here we go...: 24.31 sec elapsed
```

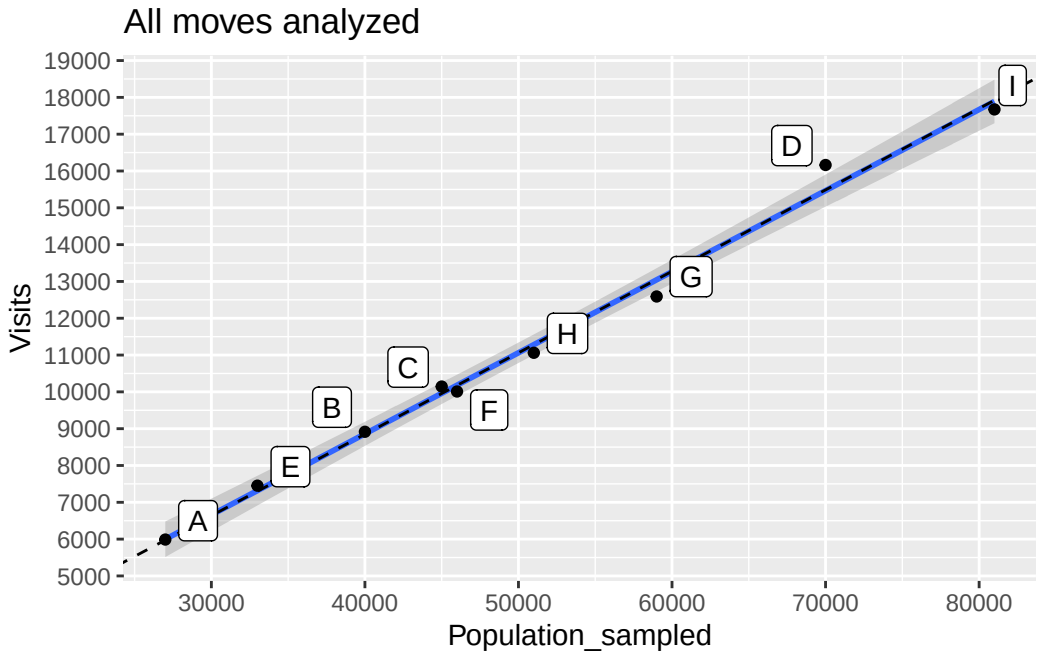
18.4 Results

```
visits <- moves |>
  group_by(position) |>
  summarise(Visits = n()) |>
  ungroup() |>
  mutate(County = LETTERS[position]) |>
  select(-position) |>
  full_join(counties)
```

```
Joining with `by = join_by(County)`
```

```
ggplot(visits, aes(.data[[pop_selected]], Visits)) +
  geom_smooth(method = "lm") +
  geom_abline(
    intercept = 0,
    slope = n_moves / sum(counties[[pop_selected]]),
    linetype = 2
  ) +
  geom_point() +
  geom_label_repel(aes(label = County), nudge_x = 0, nudge_y = 100) +
  scale_shape_manual(values = LETTERS, guide = NULL) +
  scale_x_continuous(breaks = seq(0, 10^5, 10^4)) +
  scale_y_continuous(breaks = seq(0, 10^5, 10^3)) +
  ggtitle("All moves analyzed")
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
visits <- moves |>
  filter(move > n_burnin) |>
  group_by(position) |>
  summarise(Visits = n()) |>
  ungroup() |>
  mutate(County = LETTERS[position]) |>
  select(-position) |>
  full_join(counties)
```

```
Joining with `by = join_by(County)`
```

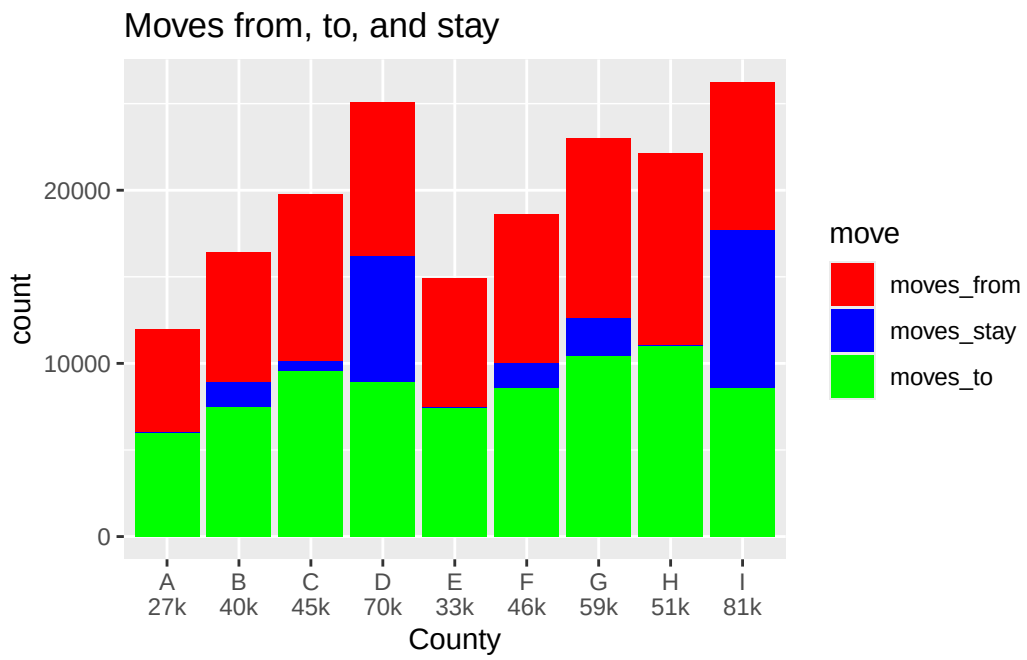
```
moves_from_to <- expand.grid(1:9, 1:9) |>
  as_tibble() |>
  rename(
    from = Var1,
    to = Var2
  ) |>
  filter(abs(from - to) < 2 | abs(from - to) == 8) |>
  mutate(
    count = 0,
    start = LETTERS[from],
    stop = LETTERS[to]
  )
```

```
for (move_i in seq_len(nrow(moves_from_to))) {
  moves_from_to$count[move_i] <-
    sum(moves$position[-nrow(moves)] == moves_from_to$from[move_i] &
      moves$position[-1] == moves_from_to$to[move_i])
}
moves_to <- moves_from_to |>
  filter(to != from) |>
  group_by(to) |>
  summarise(moves_to = sum(count)) |>
  mutate(County = LETTERS[to])
moves_from <- moves_from_to |>
  filter(to != from) |>
  group_by(from) |>
  summarise(moves_from = sum(count)) |>
  mutate(County = LETTERS[from])
moves_stay <- moves_from_to |>
  filter(to == from) |>
  group_by(from) |>
  summarise(moves_stay = sum(count)) |>
  mutate(County = LETTERS[from])
moves_from_to_stay <-
  full_join(moves_from, moves_to) |>
  full_join(moves_stay) |>
  full_join(counties |> select(-Population_defined)) |>
  select(-from, -to) |>
  pivot_longer(
    cols = c(moves_from, moves_to, moves_stay),
    names_to = "move",
    values_to = "count"
  ) |>
  mutate(County = paste0(
    County, "\n",
    round(Population_sampled / 1000),
    "k"
  ))
```

```
Joining with `by = join_by(County)`
Joining with `by = join_by(from, County)`
Joining with `by = join_by(County)`
```

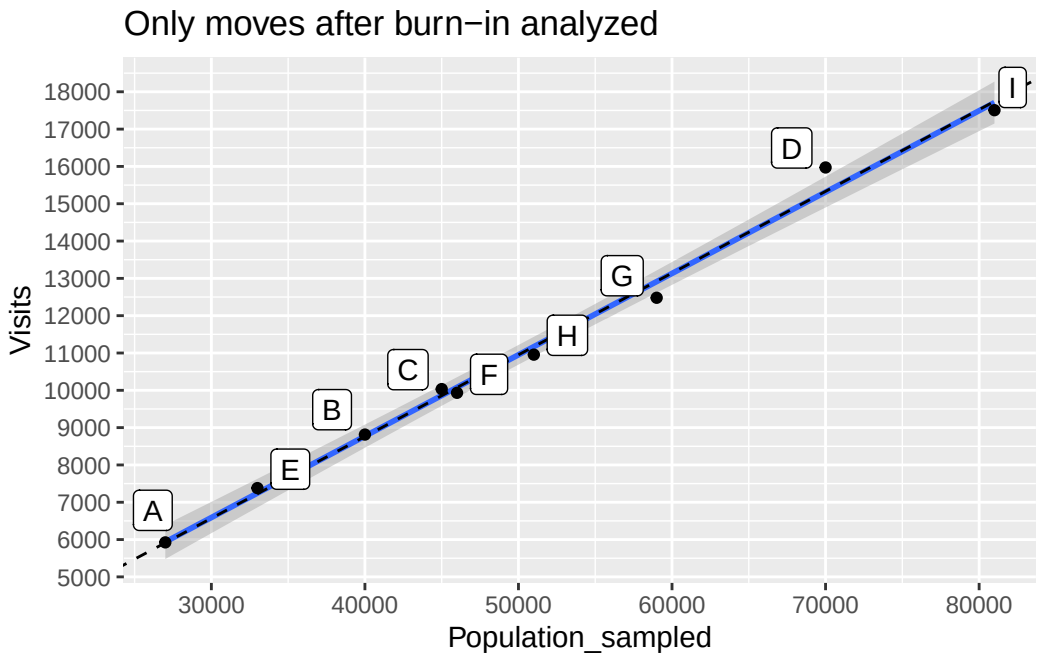
```
ggplot(moves_from_to_stay, aes(County, count, fill = move)) +
  geom_col() +
```

```
  scale_fill_manual(values = c("red", "blue", "green")) +
  ggtitle("Moves from, to, and stay")
```




```
ggplot(visits, aes(.data[[pop_selected]], Visits)) +
  geom_smooth(method = "lm") +
  geom_abline(
    intercept = 0,
    slope = (n_moves - n_burnin) / sum(counties[[pop_selected]]),
    linetype = 2
  ) +
  geom_point() +
  geom_label_repel(aes(label = County), nudge_x = 0, nudge_y = 100) +
  scale_shape_manual(values = LETTERS, guide = NULL) +
  scale_x_continuous(breaks = seq(0, 10^5, 10^4)) +
  scale_y_continuous(breaks = seq(0, 10^5, 10^3)) +
  ggtitle("Only moves after burn-in analyzed")
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
# circle for county plot
# Define circle aesthetics
theta <- seq(0, 2 * pi, length.out = 10)[c(4:9, 1:3)] |>
  rev()
# Create sequence for angles (0 to 2*pi) with 5 equally spaced points
radius <- 1 # Set radius of the circle

# Create data frame with circle coordinates and labels
circle_data <- tibble(
  angle = theta,
  x = radius * cos(theta),
  y = radius * sin(theta),
  label = LETTERS[1:9] # Assign letters A to E as labels
) |>
  full_join(visits, by = c("label" = "County")) |>
  mutate(plotlabel = paste0(
    label, "\n",
    round(Population_sampled / 1000),
    "k"
  ))
arrow_data <-
  moves_from_to |>
  full_join(circle_data |>
```

```
    select(x:label, angle), by = c("start" = "label")) |>
  rename(from_x = "x", from_y = "y") |>
  full_join(circle_data |>
    select(x:label), by = c("stop" = "label")) |>
  rename(to_x = "x", to_y = "y") |>
  mutate(
    x_end = from_x + count * 10 / n_moves * cos(angle),
    y_end = from_y + count * 10 / n_moves * sin(angle),
    count = case_when((from < to & !(from == 1 & to == 9)) | (from == 9 & to == 1) ~ -count,
      .default = count
    )
  )

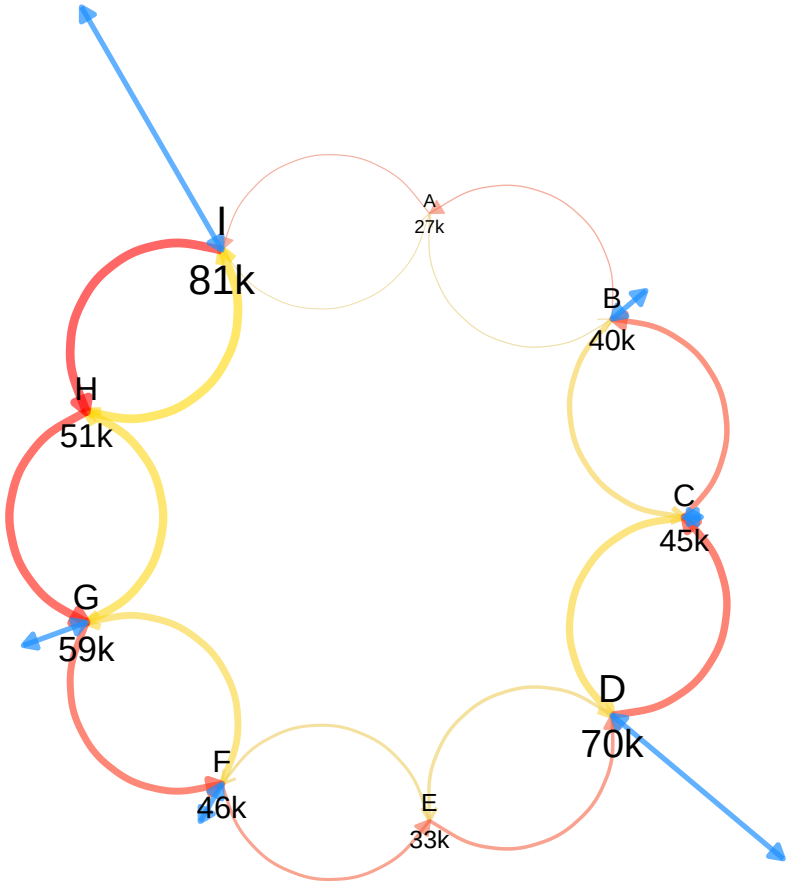

# Create ggplot with circle and labels
# ggplot(circle_data, aes(x = x, y = y)) +
#   # geom_point(aes(size = Visits), shape = 1) +  # Increase point size for better visibili
#   geom_circle(aes(r = 1,x0 = 0,y0 = 0),
#               color = "darkorange2")+
#   geom_curve(data = arrow_data |>
#                filter(from < to),
#              aes(x = from_x,y = from_y,
#                  xend = to_x,yend = to_y,
#                  linewidth = count),
#              arrow = arrow(length = unit(0.1,"inches")),
#              curvature = -1.0,
#              color = "darkolivegreen")+
#   geom_curve(data = arrow_data |>
#                filter(from > to),
#              aes(x = from_x,y = from_y,
#                  xend = to_x,yend = to_y,
#                  linewidth = count),
#              arrow = arrow(length = unit(0.1,"inches")),
#              angle=90,
#              curvature=-.75,
#              color="dodgerblue")+
#   geom_text(aes(label = plotlabel, size=.data[[pop_selected]]),
#             hjust = 0.5, vjust = 0.5) +  # Adjust text position slightly
#   scale_size_continuous(range = c(3,7)) +  # Set size of labels
#   scale_linewidth_continuous(range=c(.5,3)) +  # Set size of labels
#   coord_fixed(xlim = c(-radius - radius/5, radius + radius/5), ylim = c(-radius - radius/5
#   labs(title = "County population and move count", x = "", y = "",
#        caption = "inner arrows: moves to left neighbor,\nouter arrows: moves to right neig
```

```
#   guides(size="none", linewidth="none")+
#   theme_void()  # Remove background gridlines


# Create ggplot with circle and labels
ggplot(circle_data, aes(x = x, y = y)) +
  # geom_point(aes(size = Visits), shape = 1) +  # Increase point size for better visibility
  # geom_circle(aes(r = 1,x0 = 0,y0 = 0),
  #             color = "darkorange2")+
  geom_curve(
    data = arrow_data |>
      filter((from < to & !(to == 9 & from == 1)) | (to == 1 & from == 9)),
    aes(
      x = from_x, y = from_y,
      xend = to_x, yend = to_y,
      color = count, linewidth = abs(count)
    ),
    arrow = arrow(length = unit(0.1, "inches")),
    curvature = .75,
    alpha = .6
  ) + # , linewidth=1.5,)+
  scale_colour_gradient2(low = "gold", mid = "grey", high = "red") +
  # scale_color_gradient("move count", low = "gold",high = "gold4") +
  geom_curve(
    data = arrow_data |>
      filter(from > to & !(from == 9 & to == 1) | (to == 9 & from == 1)),
    aes(
      x = from_x, y = from_y,
      xend = to_x, yend = to_y,
      color = count, linewidth = abs(count)
    ),
    arrow = arrow(
      length = unit(0.1, "inches"),
      type = "closed"
    ),
    # angle = 90,#linewidth = 1.5,
    curvature = .75,
    alpha = .6
  ) +
  geom_segment(
    data = arrow_data |>
      filter(from == to, count > 0),
    aes(
```

```
    x = from_x, y = from_y,
    xend = x_end,
    yend = y_end
  ),
  arrow = arrow(
    length = unit(0.1, "inches"),
    ends = "both",
    type = "closed"
  ),
  color = "dodgerblue", linewidth = 1.2, alpha = .7
) +
geom_text(aes(label = plotlabel, size = .data[[pop_selected]]),
  hjust = 0.5, vjust = 0.5
) + # Adjust text position slightly
scale_size_continuous(range = c(3, 7)) + # Set size of labels
scale_linewidth_continuous(range = c(.25, 2)) + # Set size of labels
coord_fixed(
  xlim = c(-radius * 1.75, radius * 1.75),
  ylim = c(-radius * 1.75, radius * 1.75)
) + # Set axis limits slightly bigger than radius
labs(
  title = "County population and move count", x = "", y = "",
  caption = "inner golden arrows: moves to right neighbor (clockwise),\nouter redish arrows
) + # Add title and remove axis labels
guides(size = "none", linewidth = "none", color = "none") +
theme_void() # Remove background gridlines
```

County population and move count



inner golden arrows: moves to right neighbor (clockwise),
outer redish arrows: moves to left neighbor (counter−clockwise)
straight blue arrows: stay put

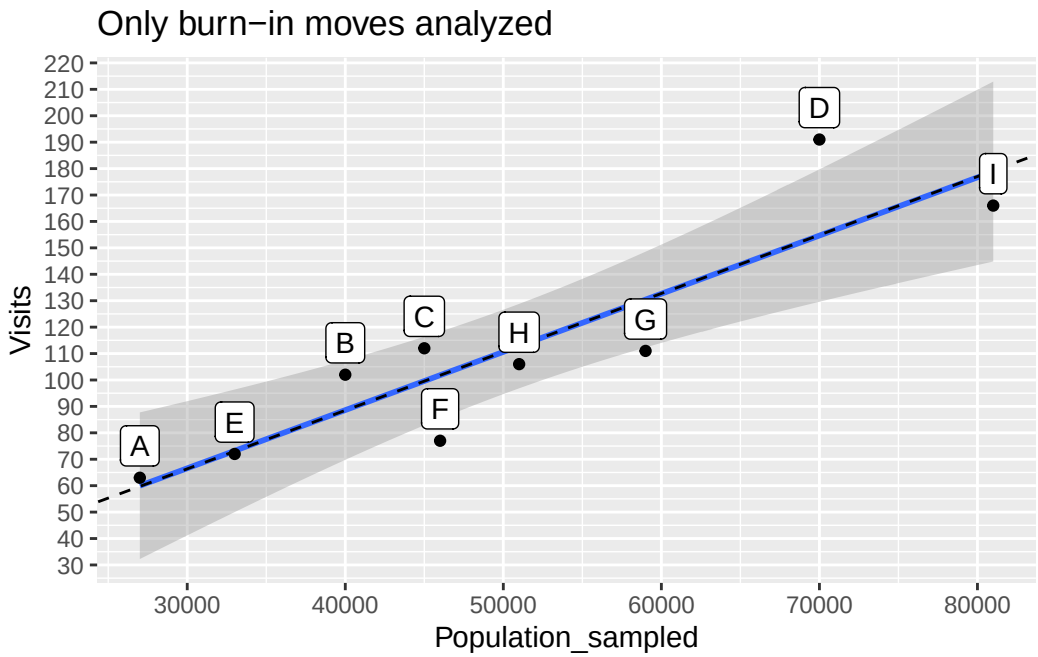
```
visits <- moves |>
```

```
  filter(move <= n_burnin) |>
  group_by(position) |>
  summarise(Visits = n()) |>
  ungroup() |>
  mutate(County = LETTERS[position]) |>
  select(-position) |>
  full_join(counties)
```

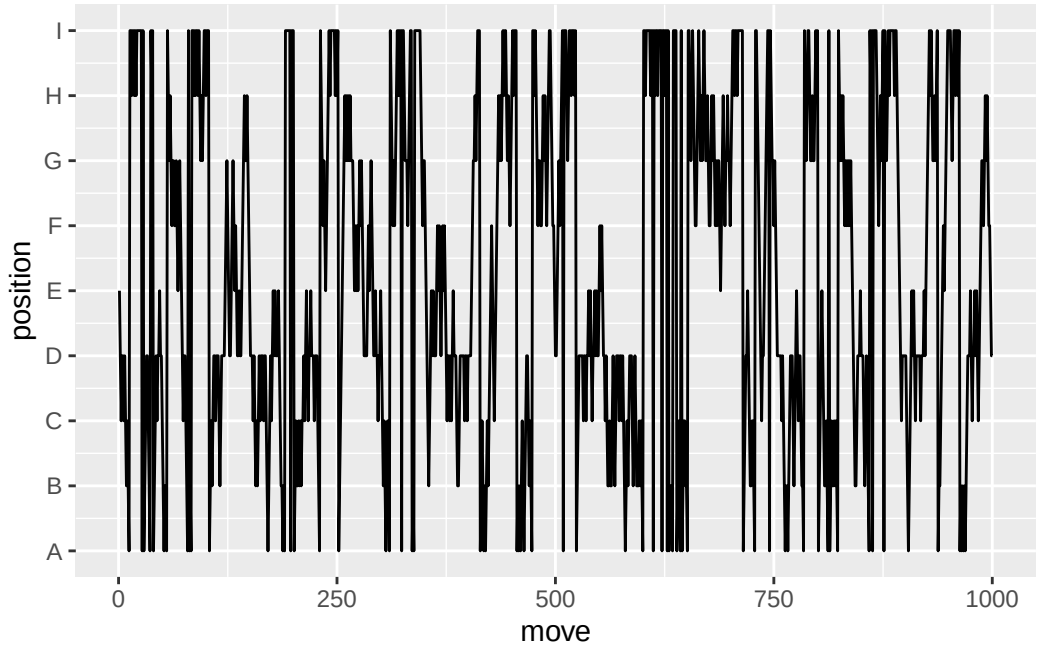
```
Joining with `by = join_by(County)`
```

```
ggplot(visits, aes(.data[[pop_selected]], Visits)) +
  geom_smooth(method = "lm") +
  geom_abline(
    intercept = 0,
    slope = n_burnin / sum(counties[[pop_selected]]),
    linetype = 2
  ) +
  geom_point() +
  geom_label_repel(aes(label = County), nudge_x = 0, nudge_y = 10) +
  scale_shape_manual(values = LETTERS, guide = NULL) +
  scale_x_continuous(breaks = seq(0, 10^5, 10^4)) +
  scale_y_continuous(breaks = seq(0, 10^5, 10^1)) +
  ggtitle("Only burn-in moves analyzed")
```

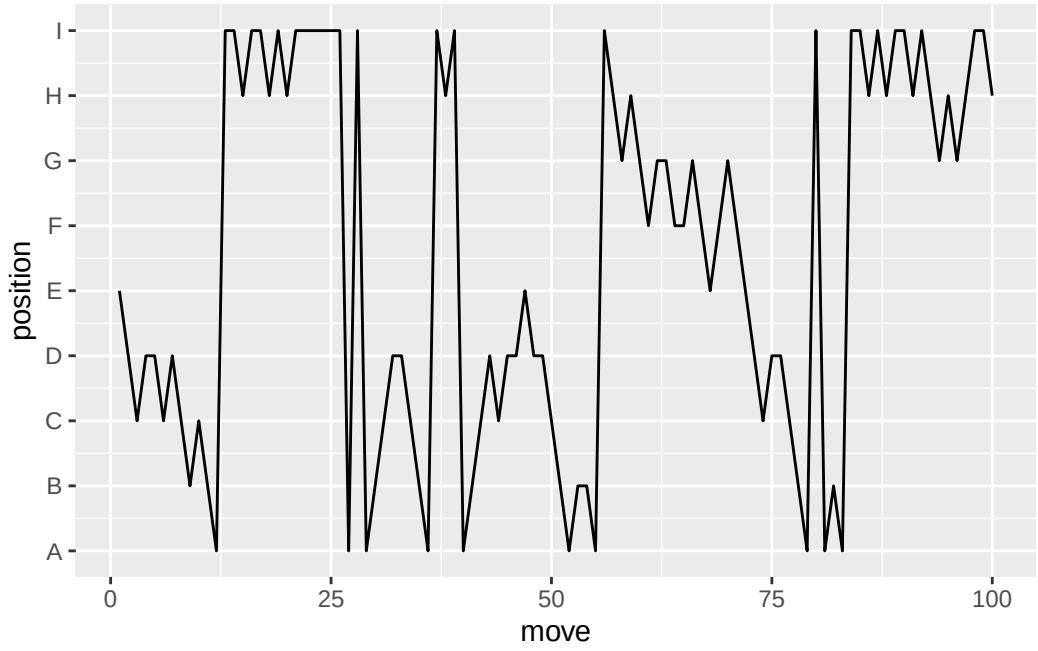
```
`geom_smooth()` using formula = 'y ~ x'
```



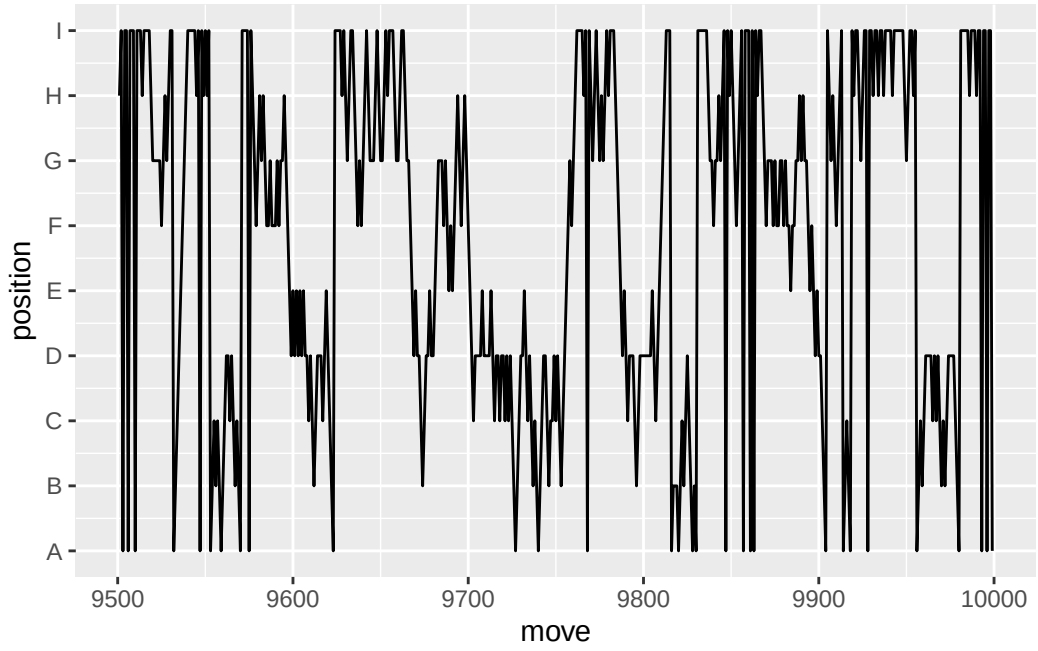

```
moves |>
  filter(move <= n_burnin) |>
  ggplot(aes(move, position)) +
  # geom_point()+
  geom_line() +
  scale_y_continuous(
    breaks = 1:9,
    labels = LETTERS[1:9]
  )
```




```
moves |>
  filter(move <= 100) |>
  ggplot(aes(move, position)) +
  # geom_point()+
  geom_line() +
  scale_y_continuous(
    breaks = 1:9,
    labels = LETTERS[1:9]
  )
```

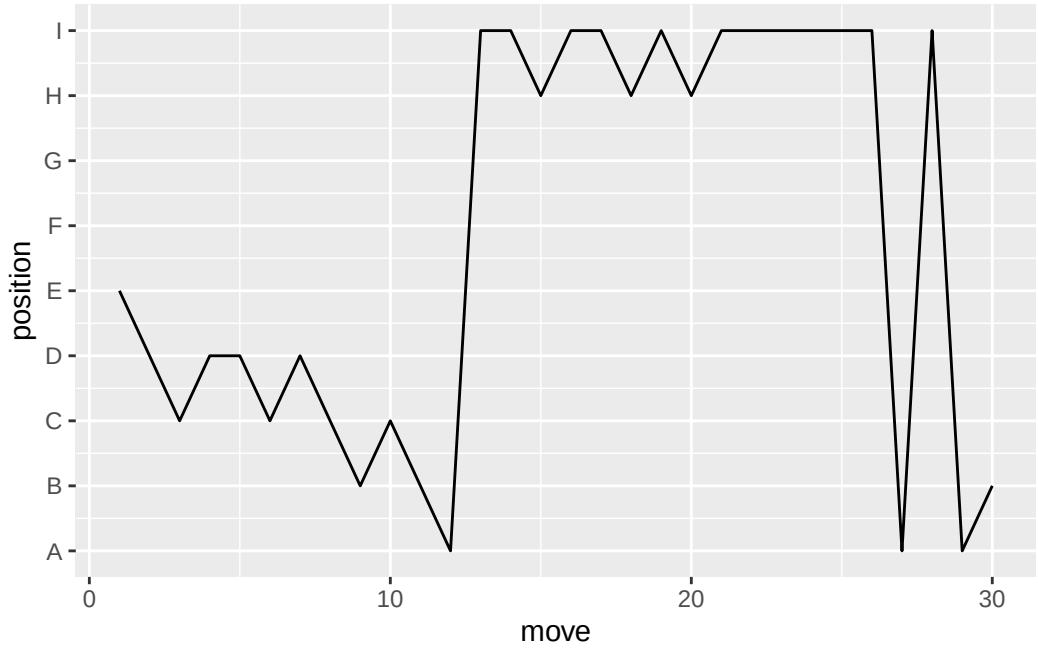



```
moves |>
  filter(move > 9500, move < 10000) |>
  ggplot(aes(move, position)) +
  # geom_point()+
  geom_line() +
  scale_y_continuous(
    breaks = 1:9,
    labels = LETTERS[1:9]
  )
```

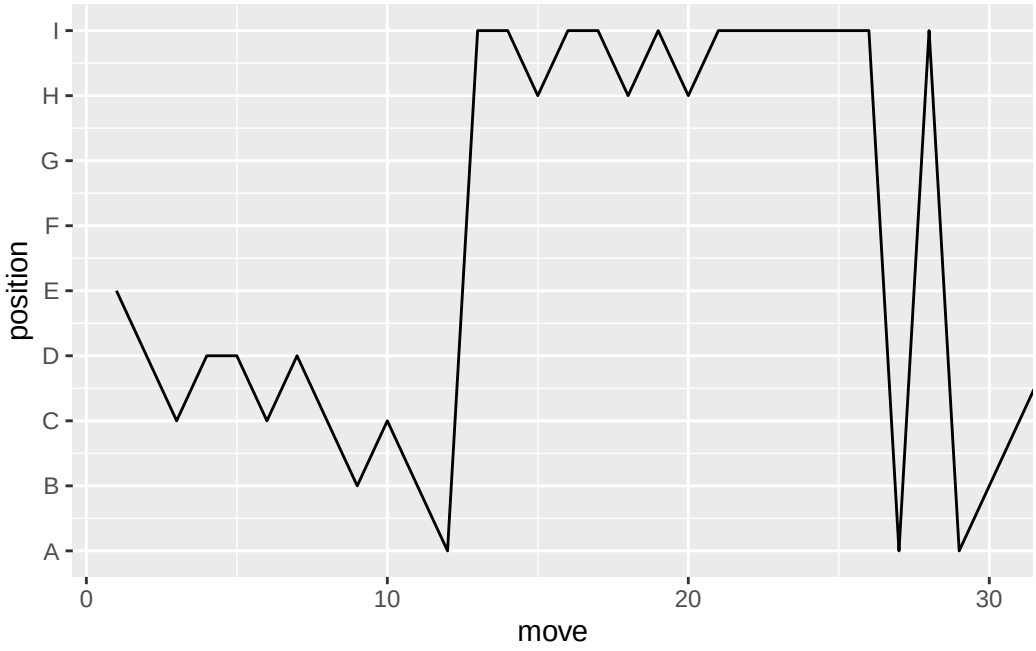



```
moves |>
  # filter(move <= n_burnin) |>
  ggplot(aes(move, position)) +
  # geom_point()+
  geom_line() +
  scale_x_continuous(limits = c(1, 30)) +
  scale_y_continuous(
    breaks = 1:9,
    labels = LETTERS[1:9]
  )
```

```
Warning: Removed 99970 rows containing missing values or values outside the scale range
(`geom_line()`).
```




```
moves |>
  # filter(move <= n_burnin) |>
  ggplot(aes(move, position)) +
  # geom_point()+
  geom_line() +
  # scale_x_continuous(limits = c(1,30))+
  scale_y_continuous(
    breaks = 1:9,
    labels = LETTERS[1:9]
  ) +
  coord_cartesian(xlim = c(1, 30))
```


I
H
G
F
E
D
C
B
A
position
0
10
20
30
move

19 k nearest neighbors knn

```
pacman::p_load(
  conflicted,
  tidyverse,
  wrappedtools, # just tools
  palmerpenguins, # data
  ggforce, # for cluster plots, hulls, zoom etc
  ggbeeswarm,
  flextable,
  caret, # Classification and Regression Training Framework
  tidymodels, # alternative to caret
  gmodels, # tools for model fitting
  easystats,
  yardstick,
  kknn
) # model performance

# conflict_scout()
conflicts_prefer(
  dplyr::slice,
  dplyr::filter,
  palmerpenguins::penguins
)
```

19.1 Data preparation

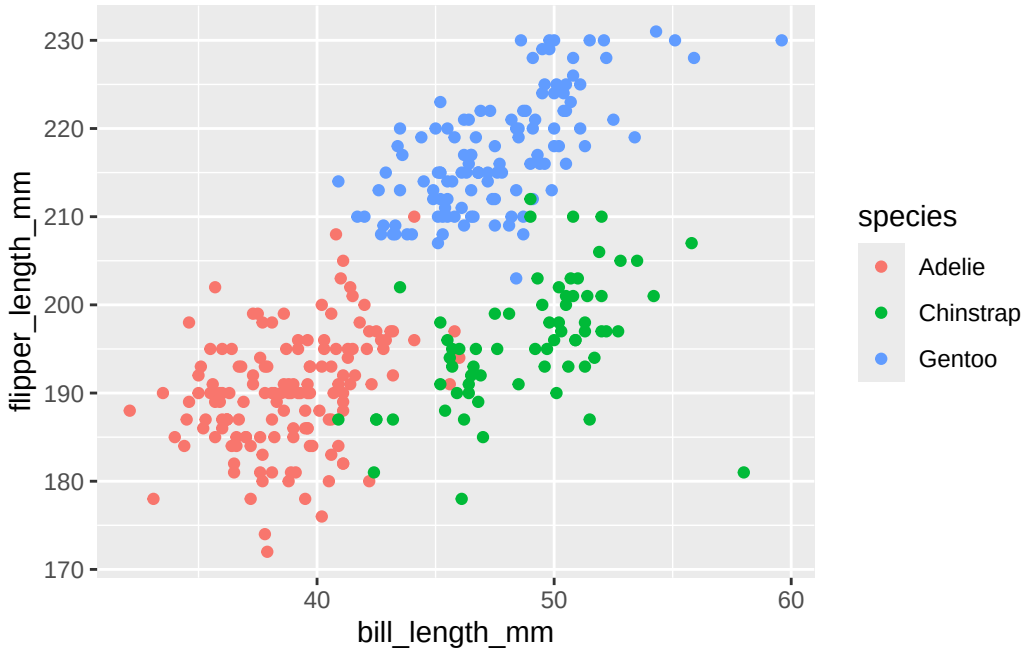
In the penguin data set, the four numerical variables related to anatomy are `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, and `body_mass_g`. We will use the first two variables to demonstrate the use of knn.

```
rawdata <- penguins |>
  drop_na()
predvars <- ColSeeker(rawdata, "length")
```

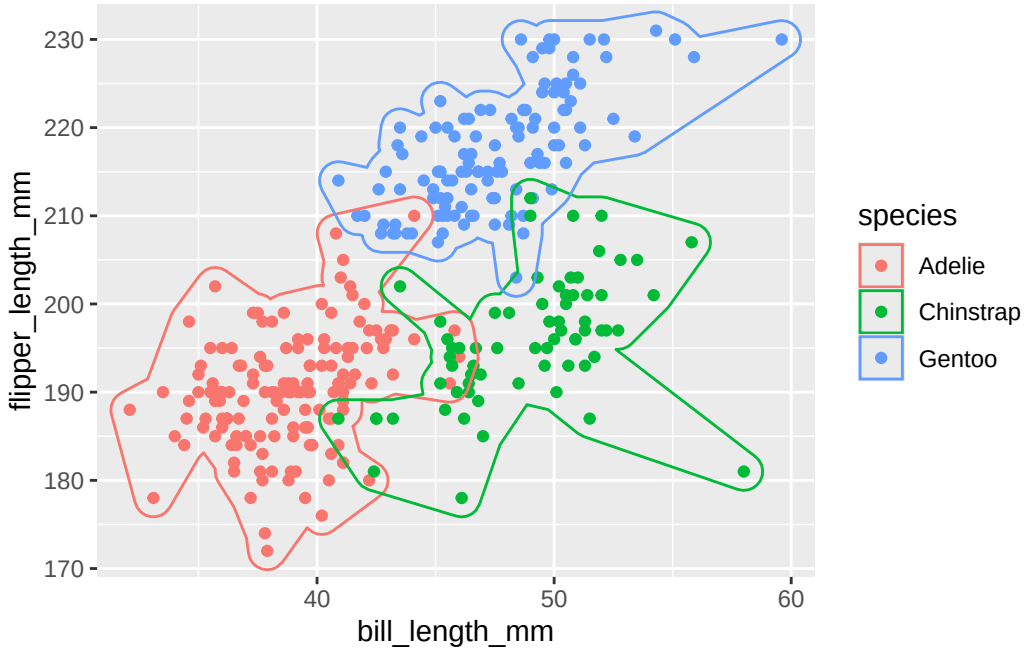
19.2 Exploratory plots

Package `ggforce` provides functions to plot convex hulls, ellipses, and zoomed facets. This is helpful in exploring group separation across 2 dimensions.

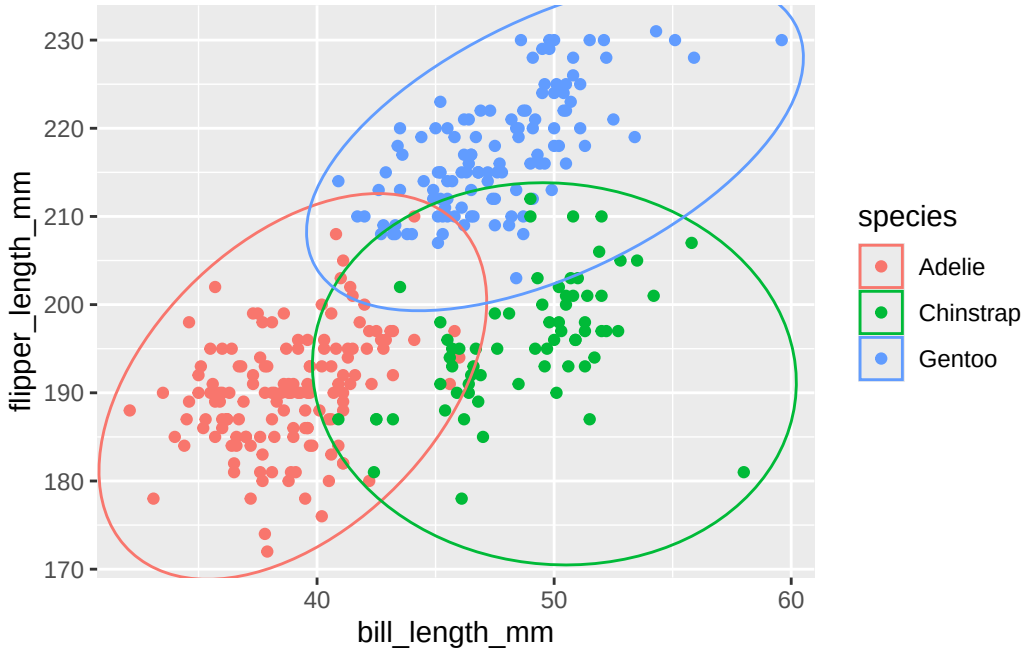
```
rawplot <-
  ggplot(
    rawdata,
    aes(.data[[predvars$names[1]]],
      .data[[predvars$names[2]]],
      color = species
    )
  ) +
  geom_point()
rawplot
```




```
rawplot +
  geom_mark_hull(expand = unit(2.5, "mm"))
```




```
rawplot +
  geom_mark_ellipse(expand = unit(2.5, "mm"))
```



19.3 Scaling

To avoid the effect of different scales / ranges of the variables, it is desirable to scale the variables. Depending on the distribution of the independent / predictor variables, there are a number of approaches to make center and spread/range of the data comparable. Here use functions from the `recipes` package from tidymodels. To compare the scaled data to the original, we'll keep the original data in the data frame. The 'real' scaling will happen later after splitting into training and test data.

19.3.1 Center + scale (z-score normalization)

```
# Build recipe for scaling
model_formula <-
  paste("~", paste(predvars$names, collapse = "+")) |>
  as.formula()
rec_normalized <- recipe(model_formula, data = rawdata) |>
  step_normalize(all_predictors())

# Fit recipe and apply to data
penguins_normalized <- rec_normalized |>
  prep() |>
  bake(new_data = rawdata) |>
  rename_with(\(x) paste0(x, "_norm"))
```

19.3.2 Range transformation

```
rec_ranged <- recipe(model_formula, data = rawdata) |>
  step_range(all_predictors(), min = 0, max = 1)

penguins_ranged <- rec_ranged |>
  prep() |>
  bake(new_data = rawdata) |>
  rename_with(\(x) paste0(x, "_range"))
```

19.3.3 Percentile transformation

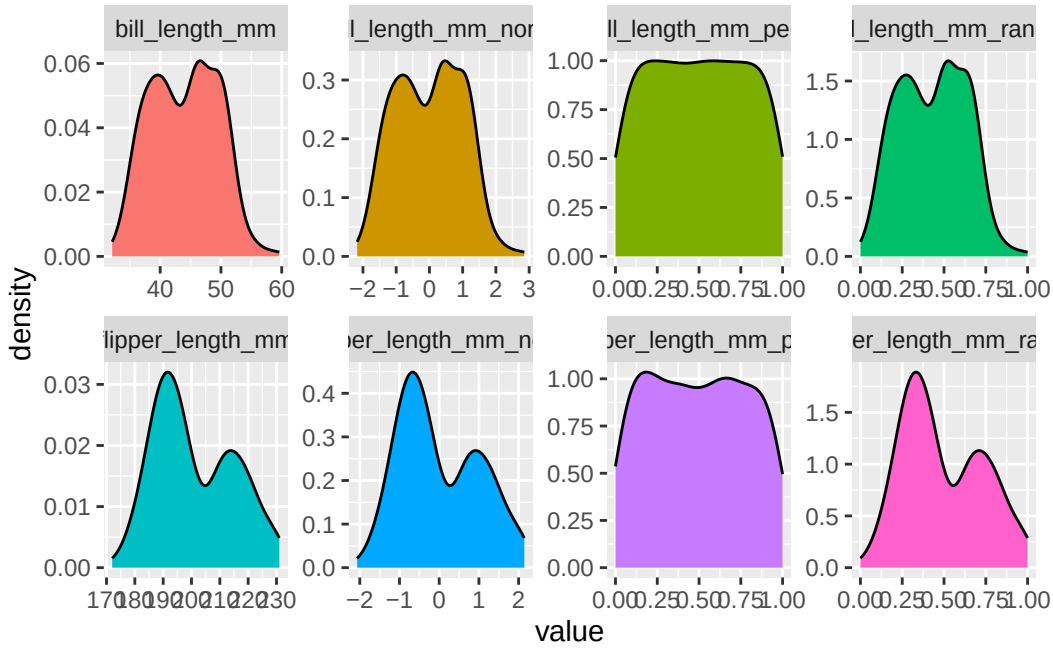
```
rec_quantiled <- recipe(model_formula, data = rawdata) |>
  step_percentile(all_predictors())

penguins_quantiled <- rec_quantiled |>
```

```
  prep() |>
  bake(new_data = rawdata) |>
  rename_with(\(x) paste0(x, "_perc"))
```

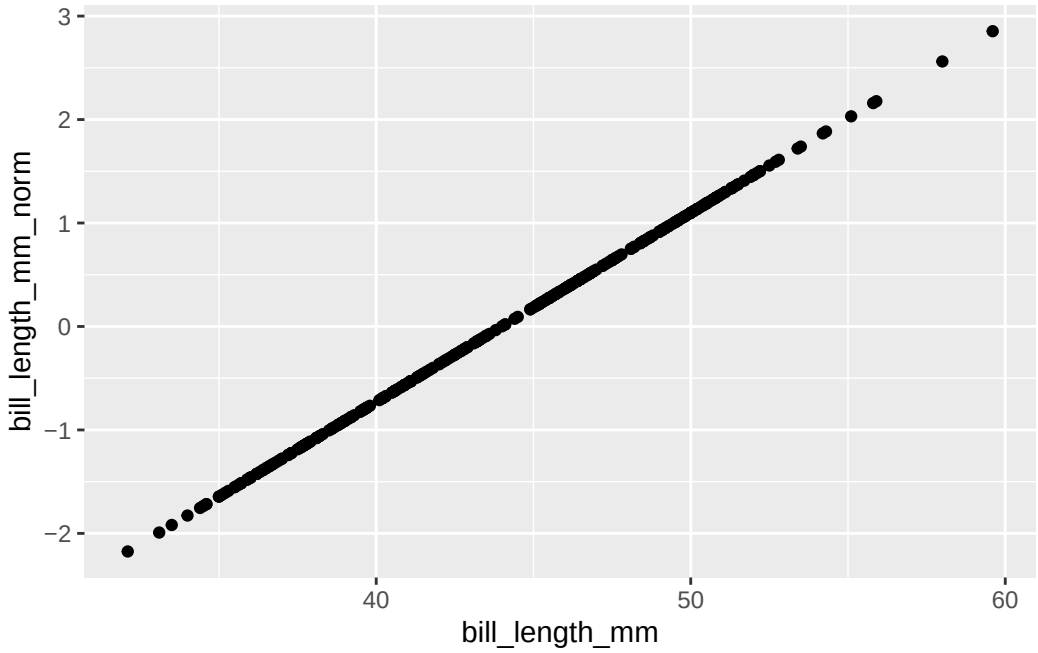
19.3.4 Visual comparison of the scaled data

```
rawdata <-
  bind_cols(
    rawdata,
    penguins_normalized,
    penguins_ranged,
    penguins_quantiled
  )
rawdata |>
  select(contains("length")) |>
  pivot_longer(everything()) |>
  ggplot(aes(value, fill = name)) +
  geom_density() +
  facet_wrap(facets = vars(name), scales = "free", nrow = 2) +
  guides(fill = "none")
```

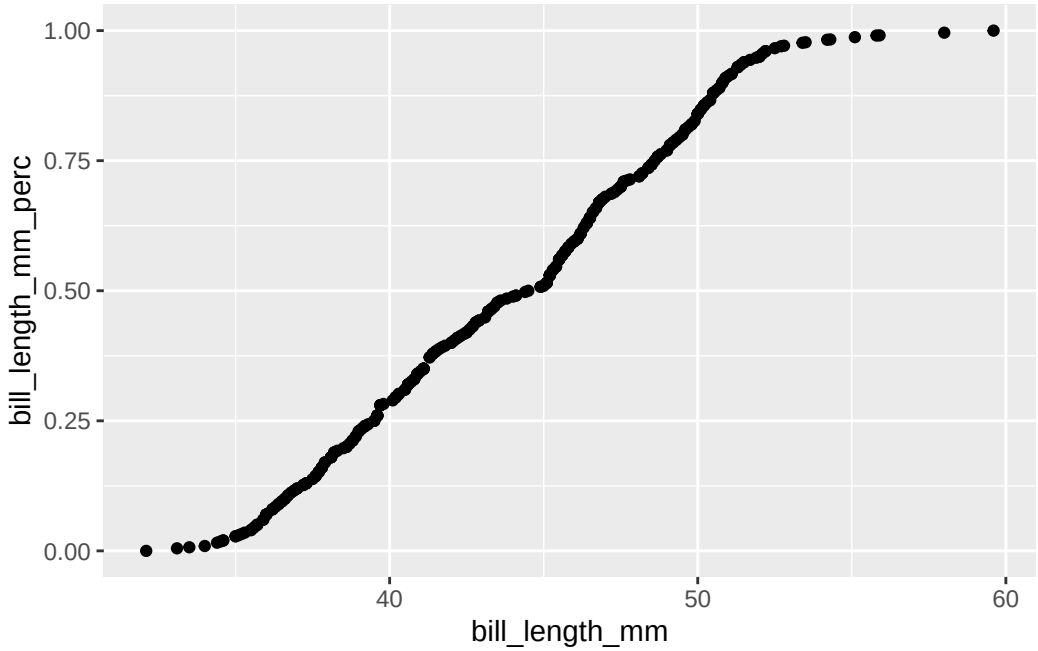



```
rawdata |>
  ggplot(aes(bill_length_mm, bill_length_mm_norm)) +
```

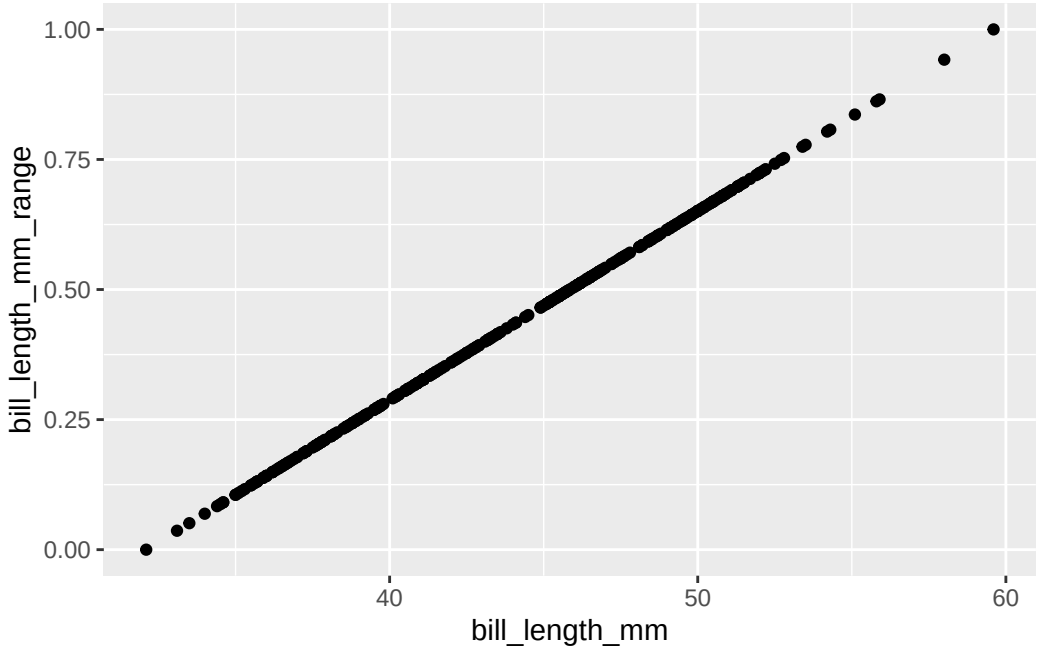
```
  geom_point()
```




```
rawdata |>
  ggplot(aes(bill_length_mm, bill_length_mm_perc)) +
  geom_point()
```




```
rawdata |>
  ggplot(aes(bill_length_mm, bill_length_mm_range)) +
  geom_point()
```



19.4 Modelling

19.4.1 Definition of predictor variables (IV)

```
predvars <- ColSeeker(namepattern = "_",
                      exclude = "_.+_.+_")
```

19.4.2 Data splitting

```
set.seed(202606)
data_split <- initial_split(
  rawdata,
  prop = 2 / 3,
  strata = species
)
traindata <- training(data_split)
testdata <- testing(data_split)
```

19.4.3 Model fitting

For the training data, a model is created and used to (re-)predict the species of the test data. The `k` parameter specifies the number of neighbors to consider. Two calls to predict return classes as well as probabilities of the species.

19.4.3.1 Model specification and workflow

```
knn_recipe <- recipe(rawdata) |>
  update_role(species, new_role = "outcome") |>
  update_role(all_of(predvars$names), new_role = "predictor") |>
  step_normalize()
```

If you want to see / analyze / visualize the preprocessd data, you can bake the data like this:

```
# View preprocessed training data (new_data = NULL returns the training set)
rec_normalized |>
  prep() |>
  bake(new_data = NULL) |>
  summarise(across(everything(),
                   \(x) meansd(x, roundDig = 5)))
```

```
# A tibble: 1 x 2
  bill_length_mm  flipper_length_mm
  <chr>           <chr>
1 0.0000 ± 1.0000 0.0000 ± 1.0000
```

```
# Apply the same learned normalization to test data
rec_normalized |>
  prep() |>
  bake(new_data = testdata) |>
  summarise(across(everything(),
                   \(x) meansd(x, roundDig = 5)))
```

```
# A tibble: 1 x 2
  bill_length_mm  flipper_length_mm
  <chr>           <chr>
1 0.0261 ± 1.0337 -0.0027 ± 1.0627
```

```
knn_spec <- nearest_neighbor(neighbors = 5) |>
  set_engine("kknn") |>
  set_mode("classification")

knn_workflow <- workflow() |>
  add_model(knn_spec) |>
  add_recipe(knn_recipe)
```

19.4.3.2 Fit the model to the training data

```
knn_fit <- knn_workflow |>
  fit(data = traindata)
```

19.4.3.3 Predict classes

```
predictions <- knn_fit |>
  predict(new_data = testdata)
prob_preds <- knn_fit |>
  predict(new_data = testdata, type = "prob") |>
  rename_with(\(x) str_remove(x, "^.pred_"))
# Combine with original data to check accuracy
test_results <- testdata |>
  select(species) |>
  bind_cols(predictions, prob_preds)
```

19.4.4 Model evaluation

```
CrossTable(test_results$.pred_class,
  test_results$species,
  prop.chisq = FALSE, prop.t = FALSE,
  format = "SAS"
)
```

```
   Cell Contents
|-------------------------|
|                       N |
|           N / Row Total |
|           N / Col Total |
|-------------------------|


Total Observations in Table:  112


                         | test_results$species
test_results$.pred_class |    Adelie | Chinstrap |    Gentoo | Row Total |
-------------------------|-----------|-----------|-----------|-----------|
                  Adelie |        49 |         3 |         0 |        52 |
                         |     0.942 |     0.058 |     0.000 |     0.464 |
                         |     1.000 |     0.130 |     0.000 |           |
-------------------------|-----------|-----------|-----------|-----------|
               Chinstrap |         0 |        20 |         0 |        20 |
                         |     0.000 |     1.000 |     0.000 |     0.179 |
                         |     0.000 |     0.870 |     0.000 |           |
-------------------------|-----------|-----------|-----------|-----------|
                  Gentoo |         0 |         0 |        40 |        40 |
                         |     0.000 |     0.000 |     1.000 |     0.357 |
                         |     0.000 |     0.000 |     1.000 |           |
-------------------------|-----------|-----------|-----------|-----------|
            Column Total |        49 |        23 |        40 |       112 |
                         |     0.438 |     0.205 |     0.357 |           |
-------------------------|-----------|-----------|-----------|-----------|
```

```
caret::confusionMatrix(test_results$.pred_class,
                       test_results$species
)
```

```
Confusion Matrix and Statistics

           Reference
Prediction  Adelie Chinstrap Gentoo
  Adelie        49         3      0
  Chinstrap      0        20      0
  Gentoo         0         0     40

Overall Statistics

               Accuracy : 0.9732
                 95% CI : (0.9237, 0.9944)
    No Information Rate : 0.4375
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.9577

 Mcnemar's Test P-Value : NA

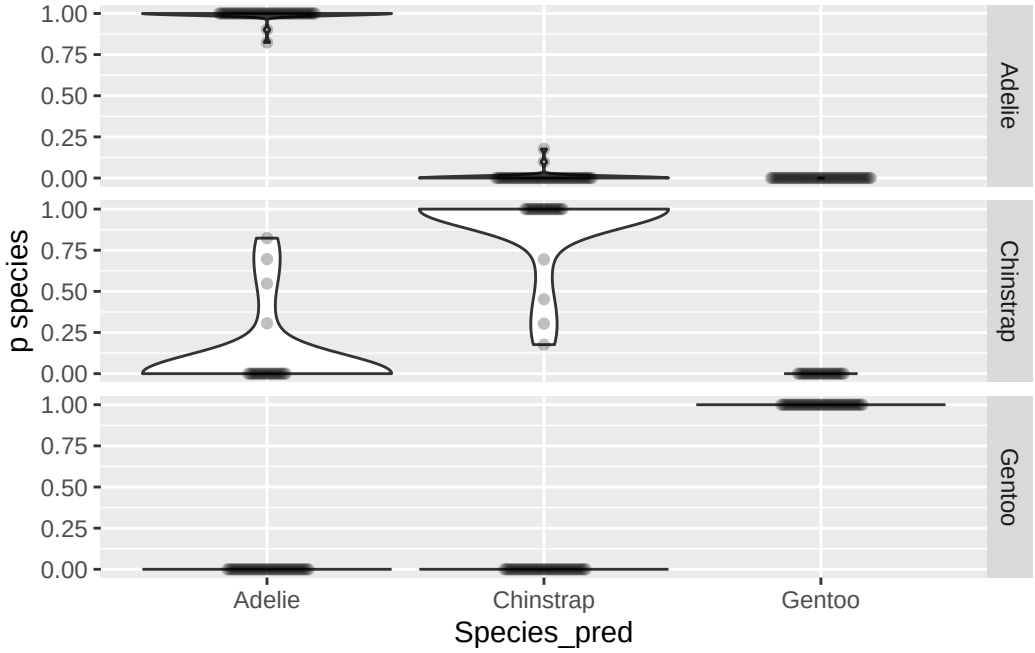
Statistics by Class:

                     Class: Adelie Class: Chinstrap Class: Gentoo
Sensitivity                 1.0000           0.8696        1.0000
Specificity                 0.9524           1.0000        1.0000
Pos Pred Value              0.9423           1.0000        1.0000
Neg Pred Value              1.0000           0.9674        1.0000
Prevalence                  0.4375           0.2054        0.3571
Detection Rate              0.4375           0.1786        0.3571
Detection Prevalence        0.4643           0.1786        0.3571
Balanced Accuracy           0.9762           0.9348        1.0000
```

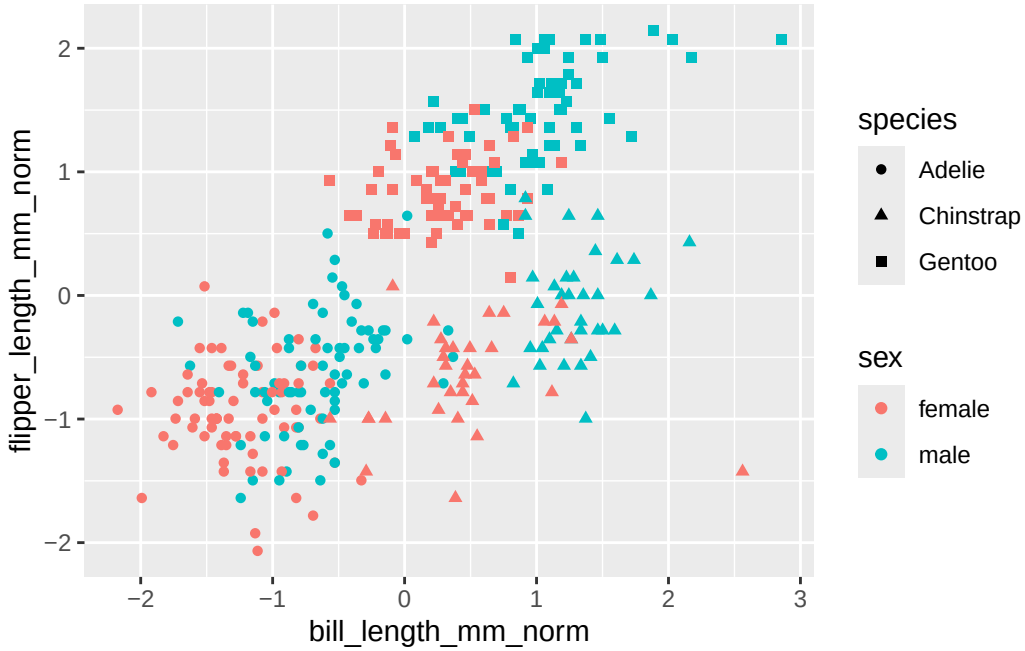
```
yardstick::accuracy(data = test_results, truth = species, estimate = .pred_class)
```

```
# A tibble: 1 x 3
  .metric  .estimator .estimate
  <chr>    <chr>          <dbl>
1 accuracy multiclass     0.973
```

```
test_results |>
  pivot_longer(c(Adelie, Chinstrap, Gentoo),
    values_to = "p species",
    names_to = "Species_pred"
  ) |>
  ggplot(aes(Species_pred, `p species`)) +
  geom_violin() +
  geom_beeswarm(cex = .25, alpha = .25) +
  facet_grid(rows = vars(species))
```




```
ggplot(rawdata, aes(bill_length_mm_norm, flipper_length_mm_norm, color = sex, shape = species
  geom_point()
```



19.5 More complex model

Now we'll try to predict the sex based on body measures and species. Species will be automatically recoded using one-hot encoding.

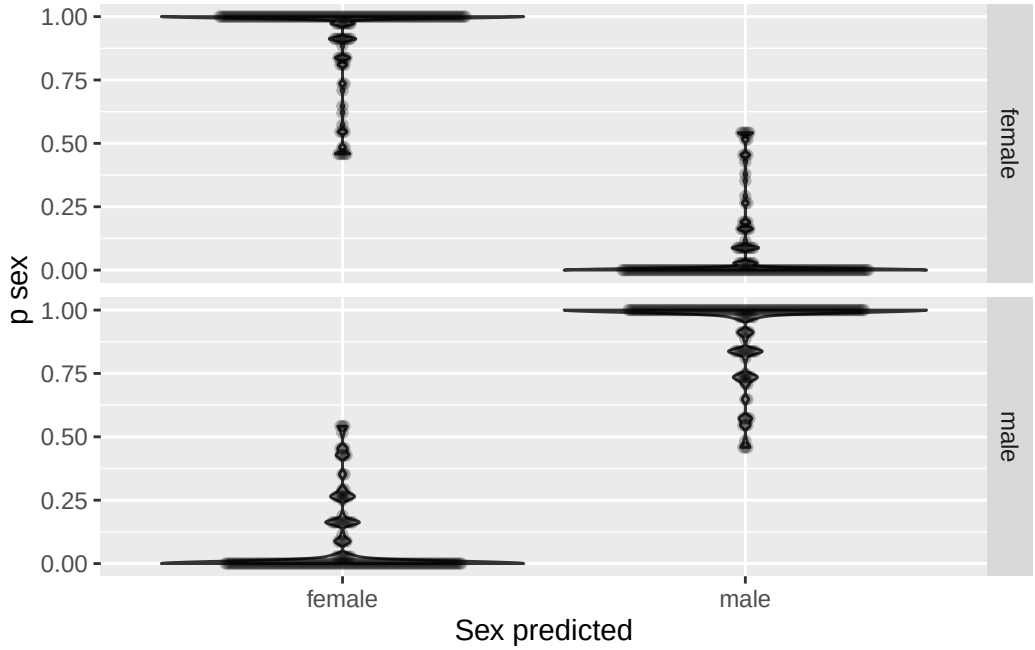
```
model_formula <-
  paste(" sex ~", paste(c(predvars$names, "species"), collapse = "+")) |>
  as.formula()
penguins_preped <- recipe(model_formula, data = rawdata) |>
  step_normalize(all_numeric_predictors()) |>
  step_dummy(all_nominal_predictors(), one_hot = TRUE) |>
  prep() |>
  bake(new_data = rawdata)
knn_spec <- nearest_neighbor(neighbors = 5) |>
  set_engine("kknn") |>
  set_mode("classification")

knn_workflow2 <- workflow() |>
  add_model(knn_spec) |>
  add_formula(sex ~ .)

# Fit the model to the training data
```

```
knn_fit2 <- knn_workflow2 |>
  fit(data = penguins_preped)
# Predict classes
predictions <- knn_fit2 |>
  predict(new_data = penguins_preped)
prob_preds <- knn_fit2 |>
  predict(new_data = penguins_preped, type = "prob") |>
  rename_with(\(x) str_remove(x, "^.pred_"))
# Combine with original data to check accuracy
all_results <- penguins_preped |>
  select(sex) |>
  bind_cols(predictions, prob_preds)

all_results |>
  pivot_longer(c(female, male),
    values_to = "p sex",
    names_to = "Sex predicted"
  ) |>
  ggplot(aes(`Sex predicted`, `p sex`)) +
  geom_violin() +
  geom_beeswarm(cex = .25, alpha = .25) +
  facet_grid(rows = vars(sex))
```



```r
yardstick::accuracy(
  data = all_results, truth = sex,
  estimate = .pred_class
)
```

```
# A tibble: 1 x 3
  .metric  .estimator .estimate
  <chr>    <chr>          <dbl>
1 accuracy binary         0.967
```

```r
yardstick::sensitivity(
  data = all_results, truth = sex,
  estimate = .pred_class,
  event_level = "second"
)
```

```
# A tibble: 1 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 sensitivity binary         0.976
```

```r
yardstick::specificity(
  data = all_results, truth = sex,
  estimate = .pred_class,
  event_level = "second"
)
```

```
# A tibble: 1 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 specificity binary         0.958
```

```r
yardstick::ppv(
  data = all_results, truth = sex,
  estimate = .pred_class,
  event_level = "second"
)
```

```
# A tibble: 1 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 ppv     binary         0.959
```

```r
CrossTable(all_results$.pred_class,
  all_results$sex,
  prop.chisq = F, prop.t = F,
  format = "SPSS"
)
```

```
   Cell Contents
|-------------------------|
|                   Count |
|             Row Percent |
|          Column Percent |
|-------------------------|

Total Observations in Table:  333

                         | all_results$sex
all_results$.pred_class  |   female  |     male  | Row Total |
-------------------------|-----------|-----------|-----------|
                 female  |      158  |        4  |      162  |
                         |   97.531% |    2.469% |   48.649% |
                         |   95.758% |    2.381% |           |
-------------------------|-----------|-----------|-----------|
                   male  |        7  |      164  |      171  |
                         |    4.094% |   95.906% |   51.351% |
                         |    4.242% |   97.619% |           |
-------------------------|-----------|-----------|-----------|
            Column Total |      165  |      168  |      333  |
                         |   49.550% |   50.450% |           |
-------------------------|-----------|-----------|-----------|
```

```r
caret::confusionMatrix(all_results$.pred_class, all_results$sex,
  positive = "male"
)
```

```
Confusion Matrix and Statistics

          Reference
Prediction female male
    female    158    4
```

```
   male        7  164

               Accuracy : 0.967
                 95% CI : (0.9417, 0.9834)
    No Information Rate : 0.5045
    P-Value [Acc > NIR] : <2e-16

                  Kappa : 0.9339

 Mcnemar's Test P-Value : 0.5465

            Sensitivity : 0.9762
            Specificity : 0.9576
         Pos Pred Value : 0.9591
         Neg Pred Value : 0.9753
             Prevalence : 0.5045
         Detection Rate : 0.4925
   Detection Prevalence : 0.5135
      Balanced Accuracy : 0.9669

       'Positive' Class : male
```

20 Regression and classification trees

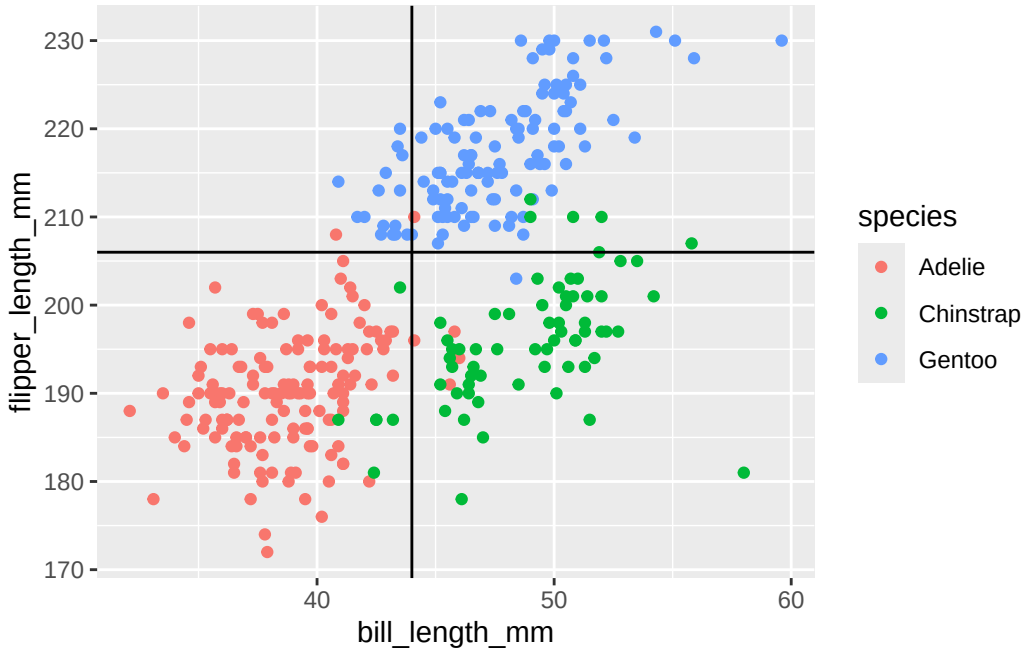
```
pacman::p_load(
  conflicted,
  tidyverse,
  wrappedtools,
  palmerpenguins,
  rpart, rpart.plot,
  randomForest,
  caret, gmodels,
  tidymodels,
  future.mirai,
  flextable,
  ggbeeswarm,
  tictoc
)

# conflict_scout()
conflicts_prefer(
  dplyr::slice,
  dplyr::filter,
  palmerpenguins::penguins
)
rawdata <- penguins |>
  na.omit()
rawdata <- mutate(rawdata,
  ID = paste("P", 1:nrow(rawdata))
) |>
  select(ID, everything())
```

20.1 Graphical exploration

When looking at the 2 measures `bill_length_mm` and `flipper_length_mm`, we can see that the 3 species of penguins are separated to a large extend, with the Gentoos having longer flippers than Adelies and Chinstraps, and Adelies and Chinstraps separated by smaller / longer bills. We can define a rough classification rule based on that just by eyeballing the data.

```
predvars <- ColSeeker(namepattern = "length")
rawplot <-
  ggplot(
    rawdata,
    aes(.data[[predvars$names[1]]],
      .data[[predvars$names[2]]],
      color = species
    )
  ) +
  geom_point()
rawplot +
  geom_hline(yintercept = 206) +
  geom_vline(xintercept = 44)
```



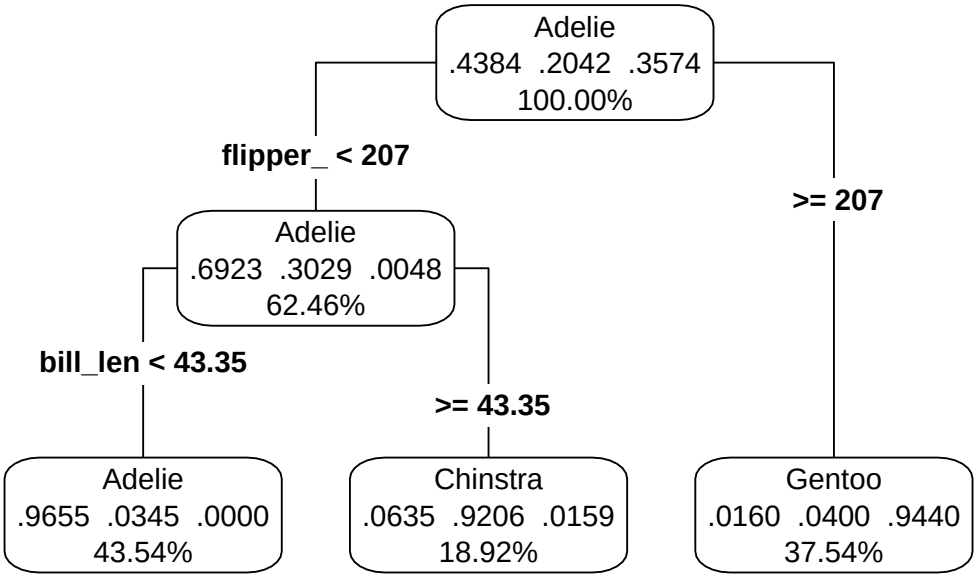
20.2 Modelling

A regression tree is following the same logic as the classification rule we defined above, but in a more systematic way. It is a recursive partitioning algorithm that splits the data into subsets that are as homogeneous as possible with respect to the target variable. The algorithm is greedy, meaning that it makes the best split at each step, without considering the impact of the split on future steps. This can lead to over-fitting, so we need to be careful with the depth of the tree. As each split is testing each variable separately, there is no need for scaling. Accordingly,

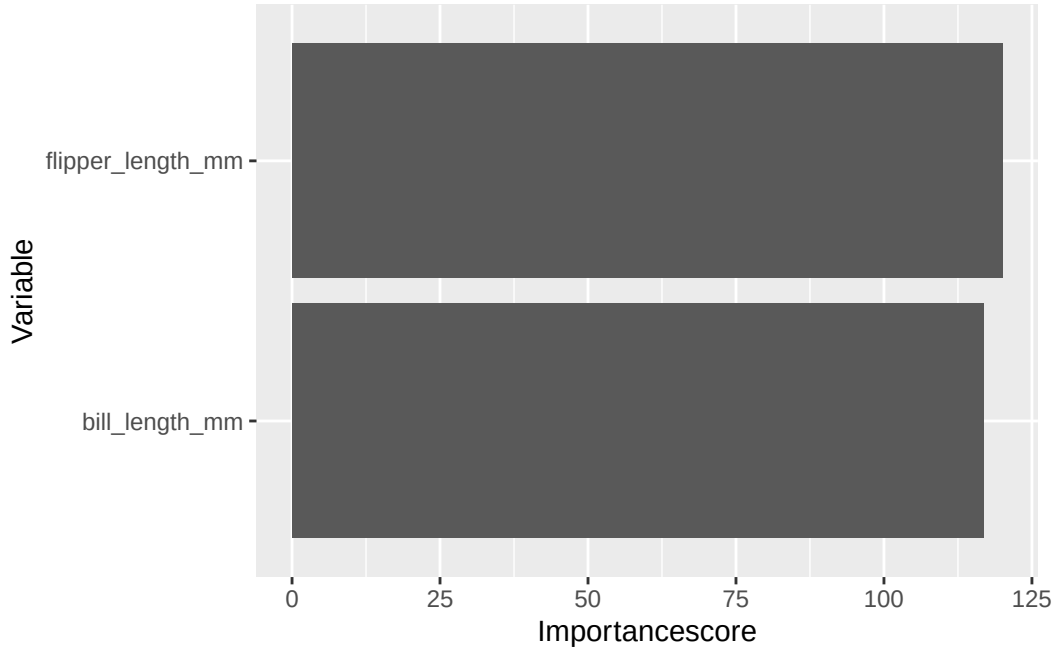
the results are easily comprehensible as they relate to the actual measurements. Beside the classification rules, the output contains information on variable importance as well.

20.2.1 Direct call to rpart

```
rpart_formula <- paste("species",
  paste(predvars$names,
    collapse = "+"
  ),
  sep = "~"
) |>
  as.formula()
rpart_out <- rpart(
  formula = rpart_formula,
  data = rawdata
)
prp(rpart_out,
  type = 4,
  extra = 104,
  digits = 4,
  fallen.leaves = TRUE
)
```



```
importance <- tibble(
  Variable = names(rpart_out$variable.importance),
  Importancescore = rpart_out$variable.importance
)
ggplot(importance, aes(x = Variable, y = Importancescore)) +
  geom_col() +
  coord_flip()
```



20.2.2 Tidy-models approach

To make model more interesting, we add 2 more variables. We'll switch from the direct call to rpart to the tidy-models approach, as this will allow tuning and better validation strategies. Data is split into training and test sets using cross-validation. As a 1st step predictors are defined:

```
predvars <- ColSeeker(namepattern = c("_"))
```

Validation is defined as 5 times repeated 5-fold cross-validation, stratified by the target variable to ensure that each fold has a similar distribution of classes. Repeating the cross-validation process multiple times helps to get a more reliable estimate of the model's performance and reduces the variance of the performance estimates.

```
set.seed(2026)
tree_folds <- vfold_cv(rawdata, v = 5, strata = species, repeats = 5)
```

Data preparation includes one-hot encoding of the categorical predictor.

```
tree_recipe <- recipe(rawdata) |>
  update_role(species, new_role = "outcome") |>
  update_role(all_of(predvars$names), new_role = "predictor") |>
  step_dummy(all_nominal_predictors(), one_hot = TRUE)
```

Tuning parameters are hard-coded in a grid of parameters to search over. The `expand.grid()` function is used to create a grid of all combinations of the specified parameters.

```
tree_grid <- expand.grid(
  tree_depth = seq(2, 6, by = 2),
  cost_complexity = c(0.01, 0.001),
  min_n = seq(5, 20, by = 5)
) |>
  as_tibble()
```

The `tune()` function is used to indicate which parameters we want to tune.

```
tree_spec <- decision_tree(
  cost_complexity = tune(),
  tree_depth = tune(),
  min_n = tune()
) |>
  set_engine("rpart", model = TRUE) |>
  set_mode("classification")
tree_workflow <- workflow() |>
  add_recipe(tree_recipe) |>
  add_model(tree_spec)
```

For the model fitting, parallel computing is used to speed up the tuning process, as it can be computationally intensive. The `plan(mirai_multisession)` function from the `future.mirai` package is used to set up a parallel processing plan that utilizes multiple CPU cores. The `availableCores()` function is used to determine the number of available cores, and we subtract 2 from that number to leave some resources free for other tasks. After the tuning process is complete, we switch back to sequential processing with `plan(sequential)`.

```
# Fit the model to the data
plan(mirai_multisession, workers = availableCores() - 2) # Use all but 2 cores for parallel
tictoc::tic("tuning timer")
tree_results <- tree_workflow |>
  tune_grid(
```

```
    resamples = tree_folds,
    grid = tree_grid,
    metrics = metric_set(roc_auc, accuracy, kap)
  )
tictoc::toc()
```

```
tuning timer: 8.01 sec elapsed
```

```
plan(sequential)
```

See the top performing models

```
tree_results |>
  show_best(metric = "kap") |>
  select(-.config, -.estimator) |>
  mutate(across(where(is.numeric), \(x) roundR(x, 5, drop0 = TRUE))) |>
  flextable()
```

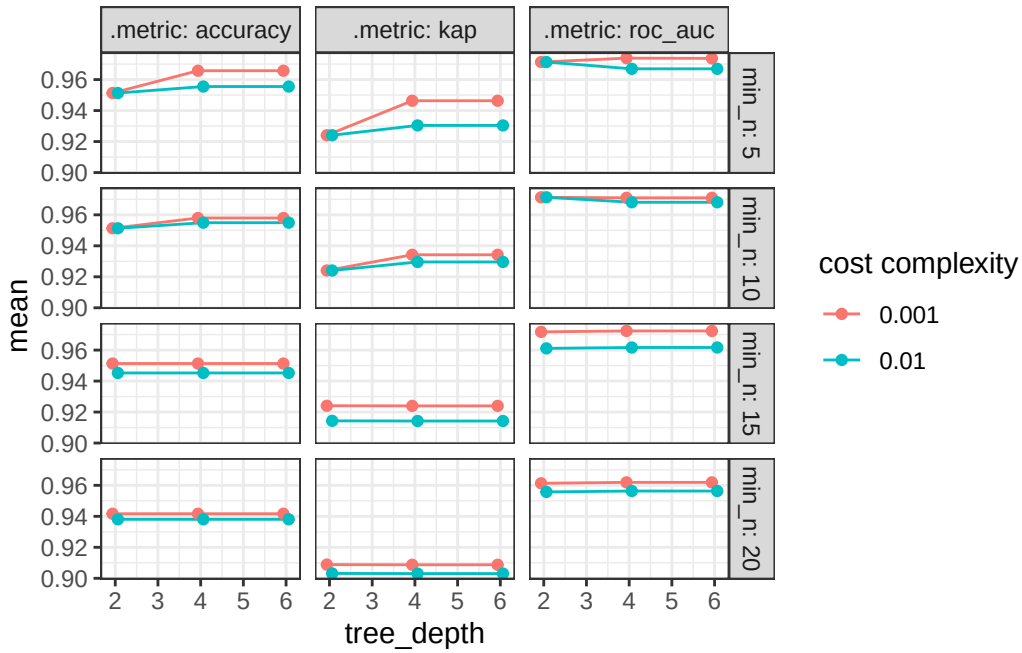
cost_complexity	tree_depth	min_n	.metric	mean	n	std_err
0.001	4	5	kap	0.94634	25	0.0057499
0.001	6	5	kap	0.94634	25	0.0057499
0.001	4	10	kap	0.93421	25	0.0069109
0.001	6	10	kap	0.93421	25	0.0069109
0.01	4	5	kap	0.93041	25	0.0069047

```
# Select the best parameters based on your preferred metric
tree_best <- tree_results |>
  select_best(metric = "accuracy")
```

20.2.3 Visualize the tuning results

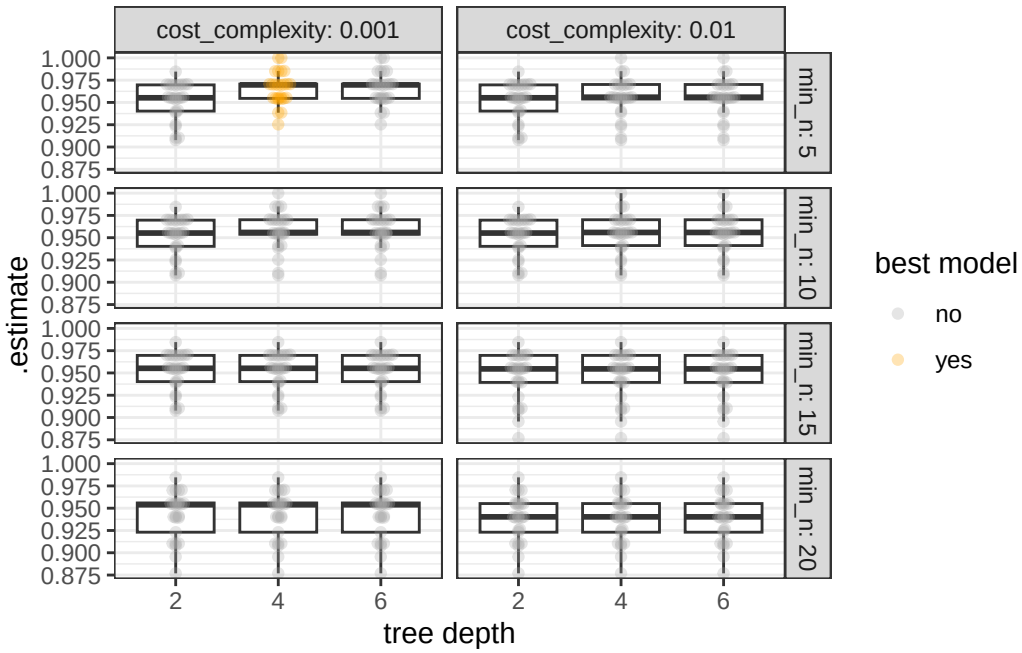
```
tree_results |>
  collect_metrics() |>
  ggplot(aes(x = tree_depth, y = mean, color = as.factor(cost_complexity))) +
  geom_line(position = position_dodge(.25)) +
  geom_point(position = position_dodge(.25)) +
  scale_color_discrete("cost complexity") +
  facet_grid(
```

```
    cols = vars(.metric), rows = vars(min_n),
    labeller = "label_both"
  ) +
  theme_bw()
```




```
tree_results |>
  collect_metrics(summarize = FALSE) |>
  filter(.metric == "accuracy") |>
  ggplot(aes(x = factor(tree_depth), y = .estimate)) +
  geom_boxplot(outlier.alpha = 0) +
  geom_beeswarm(
    alpha = 0.3,
    aes(color = .config == tree_best$.config)
  ) +
  facet_grid(
    rows = vars(min_n), cols = vars(cost_complexity), # scales = "free",
    labeller = "label_both"
  ) +
  scale_color_manual(
    "best model",
    values = c("darkgrey", "orange"),
    labels = c("no", "yes")
  ) +
```

```
  xlab("tree depth") +
  theme_bw()
```



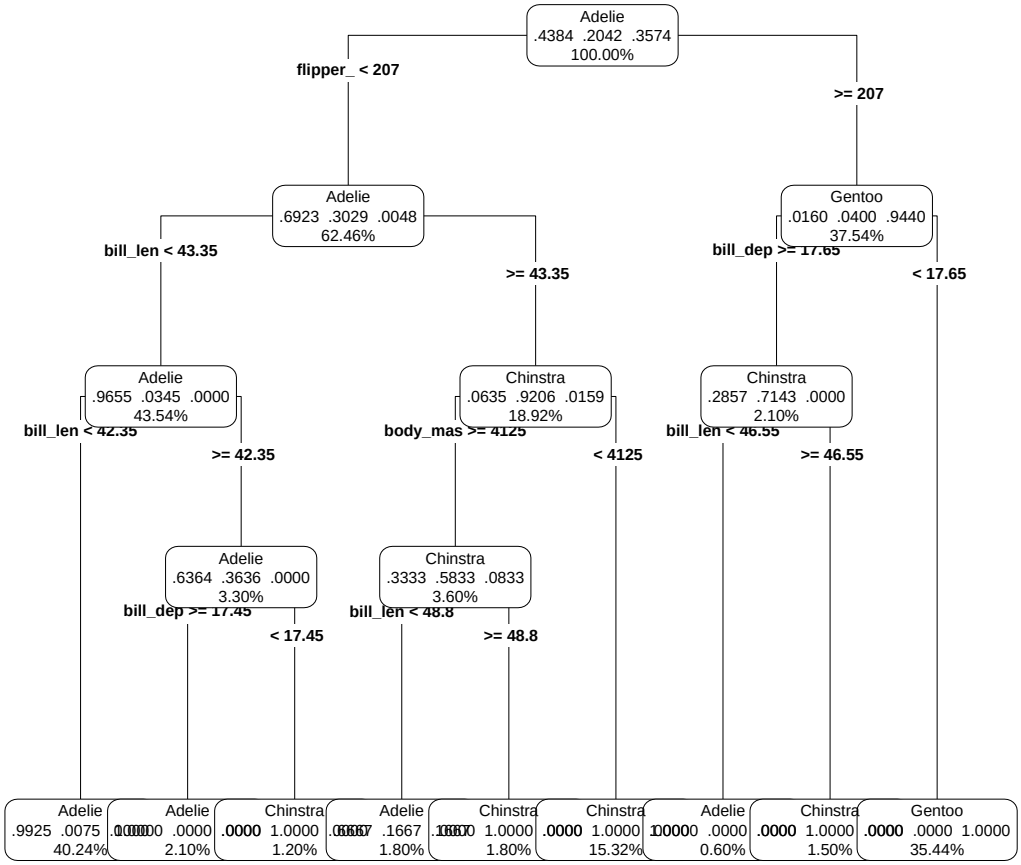
Update the workflow with those best parameters

```
final_workflow <- tree_workflow |>
  finalize_workflow(tree_best)

# Fit to the entire dataset (or use test split)
final_fit <- final_workflow |>
  fit(data = rawdata)

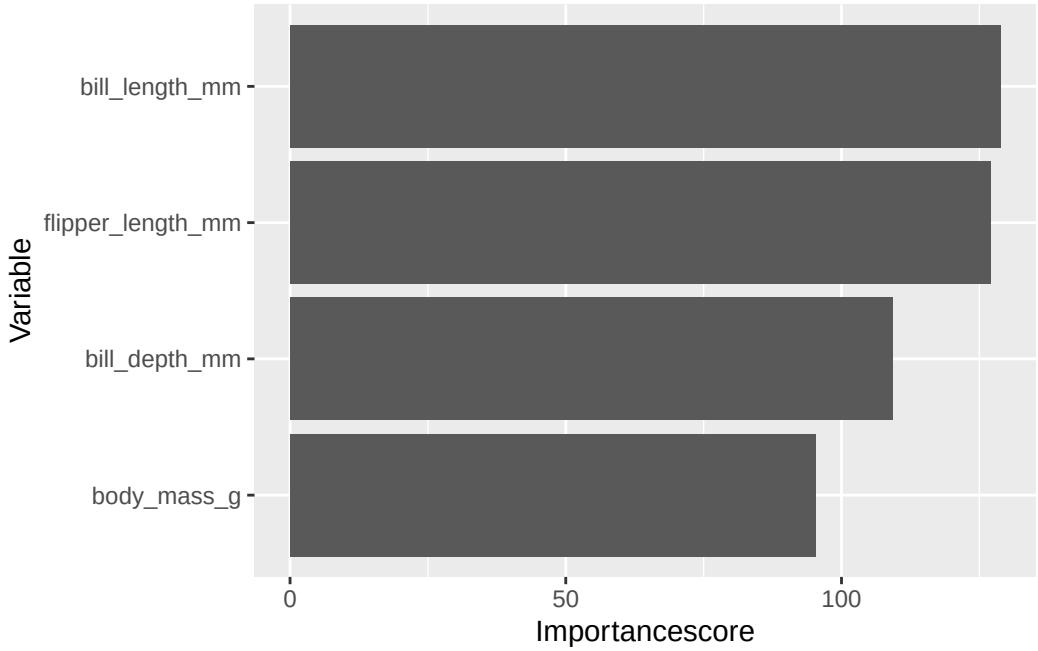
test_predicted <- final_fit |>
  augment(new_data = rawdata) |>
  rename_with(\(x) str_remove(x, "^.pred_"), starts_with(".pred_"))

final_out = extract_fit_engine(final_fit)
prp(final_out,
    type = 4,
    extra = 104,
    digits = 4,
    cex = .5,
    fallen.leaves = TRUE
)
```




```
importance <- tibble(
  Variable = names(final_out$variable.importance),
  Importancescore = final_out$variable.importance
) |>
  mutate(Variable = fct_reorder(Variable, Importancescore))

ggplot(importance, aes(x = Variable, y = Importancescore)) +
  geom_col() +
  coord_flip()
```



20.3 Model evaluation

class-based metrics:

Metric	Function	Meaning
Accuracy	accuracy()	The percentage of total predictions that were correct. Simple, but can be misleading if classes are imbalanced
Kappa	kap()	Similar to accuracy, but accounts for the possibility of the agreement occurring by chance
Sensitivity (Recall)	sens()	Out of all actual positives, how many did the model find? (True Positive Rate)
Specificity	spec()	Out of all actual negatives, how many did the model correctly identify? (True Negative Rate)

Metric	Function	Meaning
Precision (PPV)	precision()	Out of all rows the model claimed were positive, how many actually were?
F-Measure	f_meas()	The harmonic mean of Precision and Recall. Great for finding a balance between the two

probability-based metrics:

Metric	Function	Meaning
ROC AUC	roc_auc()	Measures the model's ability to distinguish between classes across all possible thresholds
PR AUC	pr_auc()	Area under the Precision-Recall curve. Preferred over ROC AUC when the "Positive" class is very rare
Log Loss	mn_log_loss()	Measures how "close" the predicted probability is to the actual binary label (0 or 1). Heavily penalizes being "confidently wrong"
Brier Score	brier_class()	The mean squared error of the probability predictions. A lower score is better

```
CrossTable(test_predicted$class,
  test_predicted$species,
  prop.chisq = F, prop.t = F,
  format = "SPSS"
)
```

```
   Cell Contents
|-------------------------|
|                   Count |
|             Row Percent |
```

```
|          Column Percent |
|-------------------------|

Total Observations in Table:  333

                     | test_predicted$species
test_predicted$class |    Adelie  | Chinstrap  |    Gentoo  | Row Total |
---------------------|------------|------------|------------|-----------|
              Adelie |       146  |         2  |         1  |       149 |
                     |   97.987%  |     1.342% |     0.671% |   44.745% |
                     |  100.000%  |     2.941% |     0.840% |           |
---------------------|------------|------------|------------|-----------|
           Chinstrap |         0  |        66  |         0  |        66 |
                     |    0.000%  |   100.000% |     0.000% |   19.820% |
                     |    0.000%  |    97.059% |     0.000% |           |
---------------------|------------|------------|------------|-----------|
              Gentoo |         0  |         0  |       118  |       118 |
                     |    0.000%  |     0.000% |   100.000% |   35.435% |
                     |    0.000%  |     0.000% |    99.160% |           |
---------------------|------------|------------|------------|-----------|
        Column Total |       146  |        68  |       119  |       333 |
                     |   43.844%  |   20.420%  |   35.736%  |           |
---------------------|------------|------------|------------|-----------|
```

```
confusionMatrix(
  test_predicted$class,
  test_predicted$species
)
```

```
Confusion Matrix and Statistics

           Reference
Prediction  Adelie Chinstrap Gentoo
  Adelie       146         2      1
  Chinstrap      0        66      0
  Gentoo         0         0    118

Overall Statistics

               Accuracy : 0.991
                 95% CI : (0.9739, 0.9981)
```

```
    No Information Rate : 0.4384
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.9859

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: Adelie Class: Chinstrap Class: Gentoo
Sensitivity                 1.0000           0.9706        0.9916
Specificity                 0.9840           1.0000        1.0000
Pos Pred Value              0.9799           1.0000        1.0000
Neg Pred Value              1.0000           0.9925        0.9953
Prevalence                  0.4384           0.2042        0.3574
Detection Rate              0.4384           0.1982        0.3544
Detection Prevalence        0.4474           0.1982        0.3544
Balanced Accuracy           0.9920           0.9853        0.9958
```

```
tidy_metrics <- metric_set(accuracy, kap, f_meas, roc_auc, mn_log_loss)
tidy_metrics(test_predicted,
  truth = species, estimate = class,
  Adelie, Chinstrap, Gentoo
) |>
  flextable()
```

.metric	.estimator	.estimate
accuracy	multiclass	0.99099099
kap	multiclass	0.98585086
f_meas	macro	0.99022858
roc_auc	hand_till	0.99748624
mn_log_loss	multiclass	0.03333177

21 Random forests

```
pacman::p_load(
  conflicted,
  tidyverse,
  ggbeeswarm,
  wrappedtools, # just tools
  palmerpenguins, # data
  caret, # Classification and Regression Training
  gmodels, # tools for model fitting
  easystats,
  yardstick, # model performance
  xgboost,
  tidymodels,
  ranger,
  vip,
  flextable,
  future.mirai
) # XGBoost
conflicts_prefer(
  dplyr::slice,
  dplyr::filter,
  dplyr::lag,
  palmerpenguins::penguins
)

rawdata <- penguins |>
  na.omit() |>
  mutate(year = factor(year))
```

Random forrests are an ensemble method that builds multiple decision trees and averages their predictions. This reduces the risk of overfitting and increases the accuracy of the model. The random part comes from the fact that each tree is built on a random subset of the data and a random subset of the variables. The number of trees and the number of variables to consider at each split are hyper-parameters that need to be tuned. (more on tuning in chapter tidymodels)

21.1 Modelling

```
set.seed(2026)
rf_folds <- vfold_cv(rawdata, v = 5, strata = species, repeats = 5)
predvars <- ColSeeker(namepattern = c("_"))
rf_recipe <- recipe(rawdata) |>
  update_role(species, new_role = "outcome") |>
  update_role(all_of(predvars$names), new_role = "predictor")

rf_grid <- expand.grid(mtry = 2:4) |>
  as_tibble()
rf_spec <- rand_forest(
  trees = 1000,
  mtry = tune()
) |>
  set_engine("ranger", importance = "impurity") |>
  set_mode("classification")
rf_workflow <- workflow() |>
  add_recipe(rf_recipe) |>
  add_model(rf_spec)

# Fit the model to the data
plan(mirai_multisession, workers = availableCores() - 2) # Use all but 2 cores for parallel
rf_results <- rf_workflow |>
  tune_grid(
    resamples = rf_folds,
    grid = rf_grid,
    metrics = metric_set(roc_auc, accuracy, kap)
  )
plan(sequential)
```

21.2 See the top performing models

```
rf_results |>
show_best(metric = "accuracy") |>
  select(-.config, -.estimator) |>
  mutate(across(where(is.numeric), \(x) roundR(x, 5, drop0 = TRUE))) |>
  flextable()
```

mtry	.metric	mean	n	std_err
2	accuracy	0.96932	25	0.0043984
3	accuracy	0.96446	25	0.0046367
4	accuracy	0.96264	25	0.0045825

Select the best parameters based on your preferred metric.

```
rf_best <- rf_results |>
  select_best(metric = "accuracy")
```

Visualize the tuning results

```
rf_results |>
  collect_metrics() |>
  ggplot(aes(x = mtry, y = mean)) +
  geom_line() +
  geom_point() +
  facet_grid(rows = vars(.metric)) +
  theme_bw()

rf_results |>
  collect_metrics(summarize = FALSE) |>
  filter(.metric == "accuracy") |>
  ggplot(aes(x = factor(mtry), y = .estimate)) +
  geom_boxplot(outlier.alpha = 0) +
  geom_beeswarm(
    alpha = 0.3,
    aes(color = .config == rf_best$.config)
  ) +
  scale_color_manual(
    "best model",
    values = c("darkgrey", "orange"),
    labels = c("no", "yes")
  ) +
  theme_bw()
```

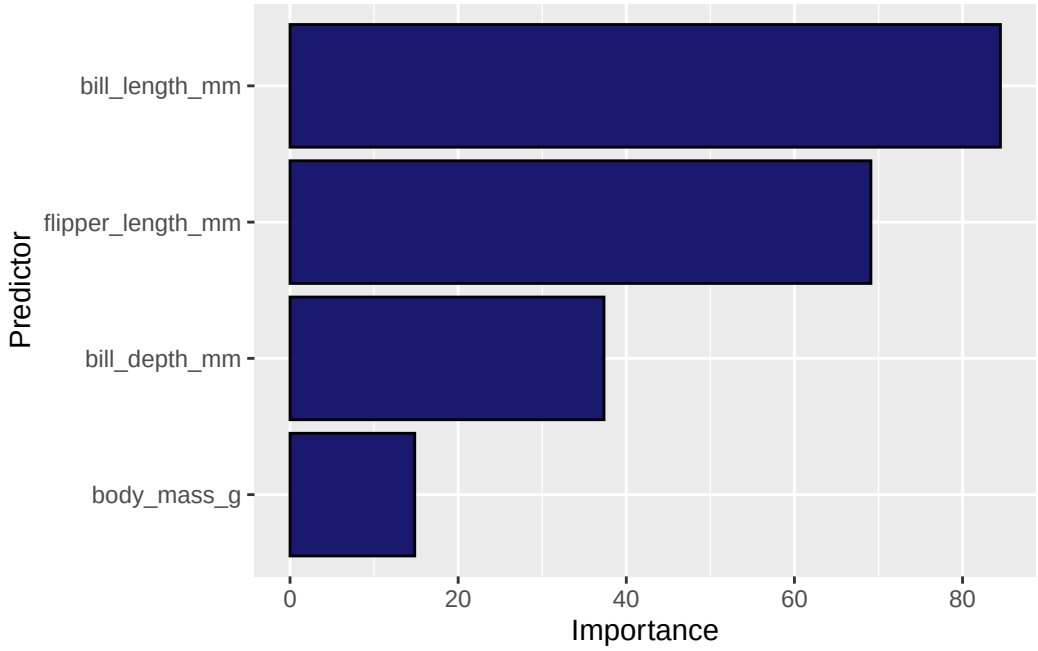
Fit to the entire dataset (or use test split / new data)

```
# Update the workflow with those best parameters
final_workflow <- rf_workflow |>
  finalize_workflow(rf_best)
```

```r
final_fit <- final_workflow |>
  fit(data = rawdata)
```

21.3 Variable importance

```r
final_out = extract_fit_engine(final_fit)
tibble(
  Predictor = names(importance(final_out)),
  Importance = importance(final_out)) |>
  mutate(Predictor = fct_reorder(.f = Predictor,
                                 .x = Importance)) |>
  ggplot(aes(Predictor, Importance)) +
  geom_col(fill = "midnightblue", color = "black") +
  coord_flip()
```




```r
# # Create an explainer for the workflow
# explainer <- explain_tidymodels(
#   model = final_fit,
#   y = rawdata$species,
#   data = rawdata,
#   verbose = TRUE
```

```
# )
#
# # Compute loss-based variable importance
# VI <- model_parts(explainer)
# plot(VI)

# final_fit |>
#   extract_fit_parsnip() |>
#   vip::vip(geom = "col", aesthetics = list(fill = "midnightblue"))
test_predicted <- final_fit |>
  augment(new_data = rawdata) |>
  rename_with(\(x) str_remove(x, "^.pred_"), starts_with(".pred_"))
```

21.4 Model evaluation

```
tidy_metrics <- metric_set(accuracy, kap, f_meas, roc_auc, mn_log_loss)
tidy_metrics(test_predicted,
  truth = species, estimate = class,
  Adelie, Chinstrap, Gentoo
) |>
  flextable()
```

.metric	.estimator	.estimate
accuracy	multiclass	0.99099099
kap	multiclass	0.98590297
f_meas	macro	0.98926430
roc_auc	hand_till	0.99989927
mn_log_loss	multiclass	0.04290948

```
CrossTable(test_predicted$class, test_predicted$species)
```

```
   Cell Contents
|-------------------------|
|                       N |
| Chi-square contribution |
|           N / Row Total |
```

```
|           N / Col Total |
|         N / Table Total |
|-------------------------|


Total Observations in Table:  333


                     | test_predicted$species
test_predicted$class |    Adelie | Chinstrap |    Gentoo | Row Total |
---------------------|-----------|-----------|-----------|-----------|
              Adelie |       144 |         1 |         0 |       145 |
                     |   101.747 |    27.643 |    51.817 |           |
                     |     0.993 |     0.007 |     0.000 |     0.435 |
                     |     0.986 |     0.015 |     0.000 |           |
                     |     0.432 |     0.003 |     0.000 |           |
---------------------|-----------|-----------|-----------|-----------|
           Chinstrap |         2 |        67 |         0 |        69 |
                     |    26.384 |   198.683 |    24.658 |           |
                     |     0.029 |     0.971 |     0.000 |     0.207 |
                     |     0.014 |     0.985 |     0.000 |           |
                     |     0.006 |     0.201 |     0.000 |           |
---------------------|-----------|-----------|-----------|-----------|
              Gentoo |         0 |         0 |       119 |       119 |
                     |    52.174 |    24.300 |   137.526 |           |
                     |     0.000 |     0.000 |     1.000 |     0.357 |
                     |     0.000 |     0.000 |     1.000 |           |
                     |     0.000 |     0.000 |     0.357 |           |
---------------------|-----------|-----------|-----------|-----------|
        Column Total |       146 |        68 |       119 |       333 |
                     |     0.438 |     0.204 |     0.357 |           |
---------------------|-----------|-----------|-----------|-----------|
```

22 Boosted regression trees

```
pacman::p_load(
  conflicted,
  tidyverse,
  ggbeeswarm,
  wrappedtools, # just tools
  palmerpenguins, # data
  caret, # Classification and Regression Training
  gmodels, # tools for model fitting
  easystats,
  yardstick, # model performance
  xgboost,
  tidymodels,
  vip,
  flextable,
  future.mirai
) # XGBoost
conflicts_prefer(
  dplyr::slice,
  dplyr::filter,
  dplyr::lag,
  palmerpenguins::penguins
)

rawdata <- penguins |>
  na.omit() |>
  mutate(year = factor(year))
```

22.1 Direct modelling

22.1.1 Preprocessing

one_hot encoding transforms factors in dummy columns for EVERY level, recommended for tree-based models as they can handle the resulting collinearity, and it is more intuitive to

interpret the importance of each level. The `recipe` package provides a convenient way to do this. The resulting columns are added to the original data frame, and the original factor columns can be removed if desired. For linear models, one-hot encoding is not recommended as it can lead to multicollinearity issues.

```
rawdata <-
  recipe(species ~ ., data = rawdata) |>
  step_dummy(all_nominal_predictors(), one_hot = TRUE) |>
  prep() |>
  bake(new_data = rawdata)
cn()
```

```
 [1] "bill_length_mm"    "bill_depth_mm"     "flipper_length_mm"
 [4] "body_mass_g"       "species"           "island_Biscoe"
 [7] "island_Dream"      "island_Torgersen"  "sex_female"
[10] "sex_male"          "year_X2007"        "year_X2008"
[13] "year_X2009"
```

```
predvars <- ColSeeker(
  rawdata,
  c(
    "_.+_", "sex.+",
    "island.+", "year_"
  )
)
predvars$names
```

```
 [1] "bill_length_mm"    "bill_depth_mm"     "flipper_length_mm"
 [4] "body_mass_g"       "island_Biscoe"     "island_Dream"
 [7] "island_Torgersen"  "sex_female"        "sex_male"
[10] "year_X2007"        "year_X2008"        "year_X2009"
```

```
boostdata <- rawdata |>
  select(predvars$names) |>
  as.matrix()
```

22.1.2 Modelling

```
xgb_out <- xgboost(
  x = boostdata,
  y = rawdata$species,
  nrounds = 100,
```

```
  objective = "multi:softprob"
) # requests class probabilities
prediction <- predict(xgb_out, boostdata) |>
  as_tibble() |>
  mutate(
    predicted = # find highest prob
      case_when(
        Adelie >= Chinstrap &
          Adelie >= Gentoo ~ "Adelie",
        Gentoo >= Chinstrap ~ "Gentoo",
        !is.na(Chinstrap) ~ "Chinstrap"
      ) |>
        factor()
  )

slice_sample(prediction, n = 10)
```

```
# A tibble: 10 x 4
     Adelie Chinstrap   Gentoo predicted
      <dbl>     <dbl>    <dbl> <fct>
 1 0.00146   0.998    0.000421 Chinstrap
 2 0.999     0.000539 0.000192 Adelie
 3 0.000624  0.00115  0.998    Gentoo
 4 0.998     0.00150  0.000702 Adelie
 5 0.999     0.000857 0.000402 Adelie
 6 0.00125   0.000944 0.998    Gentoo
 7 0.998     0.00150  0.000702 Adelie
 8 0.000584  0.00108  0.998    Gentoo
 9 0.996     0.00374  0.000651 Adelie
10 0.000299  0.999    0.000556 Chinstrap
```

22.2 Tidy model approach

```
predvars <- ColSeeker(namepattern = c("_mm", "_g", "island", "sex"))

set.seed(2026)
xgb_folds <- vfold_cv(rawdata, v = 5, strata = species, repeats = 5)

xgb_recipe <- recipe(rawdata) |>
  update_role(species, new_role = "outcome") |>
  update_role(all_of(predvars$names), new_role = "predictor")
```

```
xgb_grid <- grid_space_filling(
  tree_depth(range = c(5, 10)), # Searches between 2 and 10
  trees(range = c(100, 200)),
  learn_rate(),
  loss_reduction(),
  min_n(range = c(10, 20)),
  finalize(mtry(), rawdata),
  size = 20
)
xgb_spec <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  min_n = tune(),
  loss_reduction = tune(),
  mtry = tune(),
  learn_rate = tune()
) |>
  set_engine("xgboost") |>
  set_mode("classification")
xgb_workflow <- workflow() |>
  add_recipe(xgb_recipe) |>
  add_model(xgb_spec)
```

22.2.1 Fit the model to the data

```
plan(mirai_multisession, workers = availableCores() - 2) # Use all but 2 cores for parallel p
xgb_results <- xgb_workflow |>
  tune_grid(
    resamples = xgb_folds,
    grid = xgb_grid,
    metrics = metric_set(roc_auc, accuracy)
  )
plan(sequential)
```

22.2.2 See the top performing models

```
xgb_results |>
  show_best(metric = "accuracy") |>
  select(-.metric, -.estimator, -n, -std_err, -.config) |>
  rowwise() |> mutate(across(everything(), roundR)) |> ungroup() |>
```

```
  flextable() |>
  set_table_properties(width = 1, layout = "autofit")
```

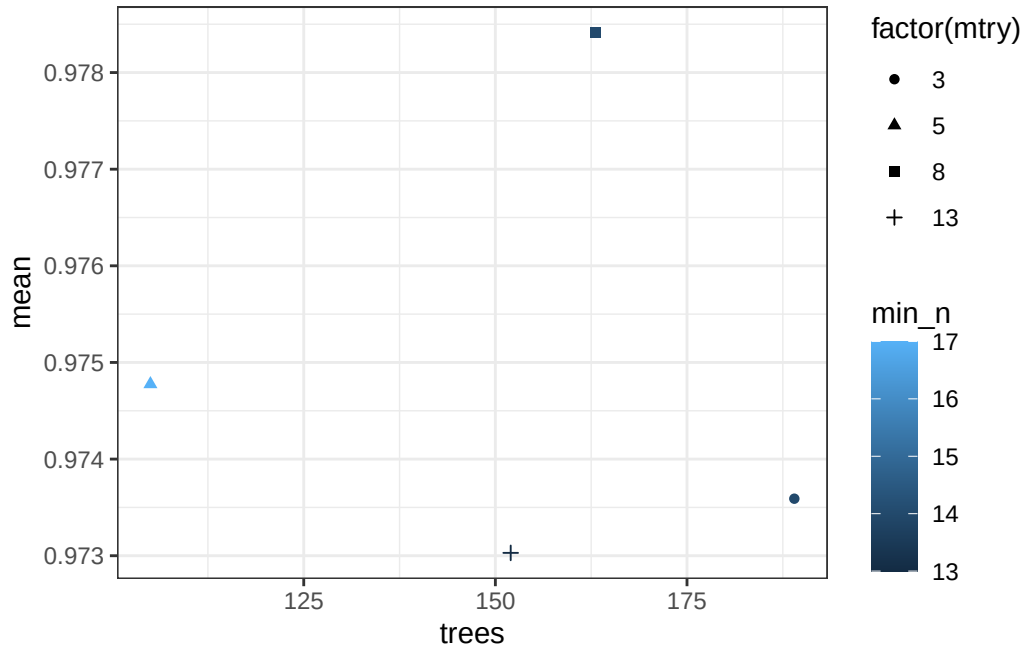
mtry	trees	min_n	tree_depth	learn_rate	loss_reduction	mean
8.0	163	14	10	0.10	0.0000017	0.98
5.0	105	17	6.0	0.034	0.000000026	0.97
3.0	189	14	6.0	0.0038	0.0000000065	0.97
13	152	13	5.0	0.011	0.0018	0.97
6.0	110	16	8.0	0.0013	32	0.76

22.2.3 Select the best parameters based on your preferred metric

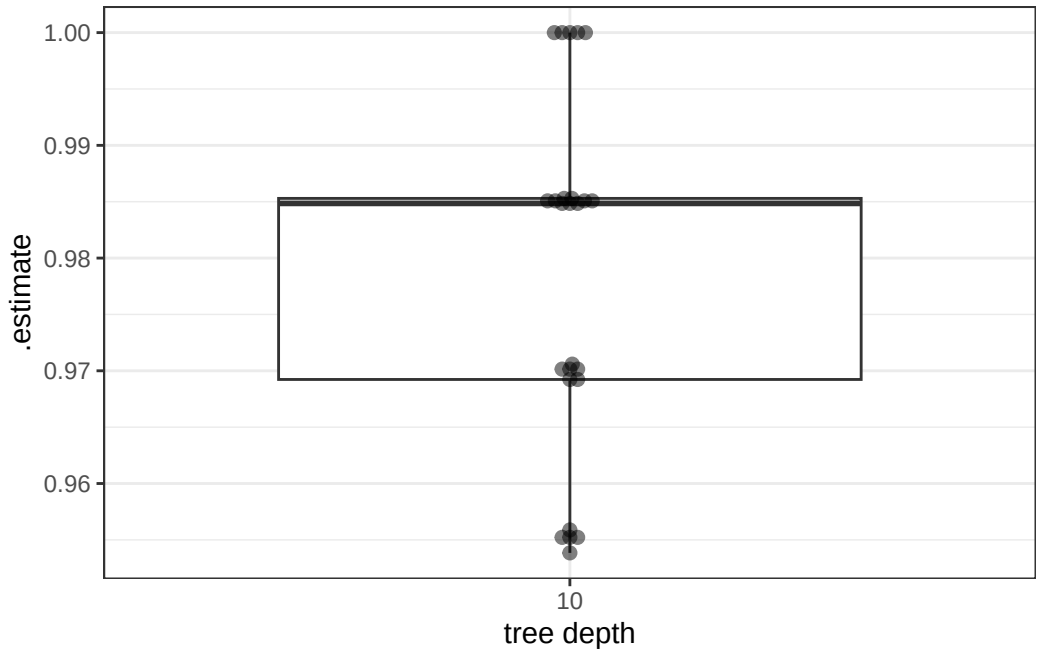
```
xgb_best <- xgb_results |>
  select_best(metric = "accuracy")
```

22.2.4 Visualize the tuning results

```
xgb_results |>
  collect_metrics() |>
  filter(
    .metric == "accuracy",
    mean >= median(mean)
  ) |>
  arrange(desc(mean)) |>
  ggplot(aes(
    x = trees, y = mean,
    color = min_n, shape = factor(mtry)
  )) +
  #  geom_line(position = position_dodge(.25)) +
  geom_point(position = position_dodge(.25)) +
  theme_bw()
```




```
xgb_results |>
  collect_metrics(summarize = FALSE) |>
  filter(
    .metric == "accuracy",
    mtry == xgb_best$mtry,
    trees == xgb_best$trees,
    min_n == xgb_best$min_n,
    tree_depth == xgb_best$tree_depth
  ) |>
  ggplot(aes(x = factor(tree_depth), y = .estimate)) +
  geom_boxplot(outlier.alpha = 0) +
  geom_beeswarm(alpha = 0.5, size = 2) +
  xlab("tree depth") +
  theme_bw()
```




```
# Update the workflow with those best parameters
final_workflow <- xgb_workflow |>
  finalize_workflow(xgb_best)

# Fit to the entire dataset (or use test split)
final_fit <- final_workflow |>
  fit(data = rawdata)

test_predicted <- final_fit |>
  augment(new_data = rawdata) |>
  rename_with(\(x) str_remove(x, "^.pred_"), starts_with(".pred_"))
```

22.2.5 Model evaluation

```
yardstick::conf_mat(test_predicted, truth = species, estimate = class)
```

```
           Truth
Prediction  Adelie Chinstrap Gentoo
  Adelie       145         1      0
  Chinstrap      0        67      0
  Gentoo         1         0    119
```

```
caret::confusionMatrix(test_predicted$species,
                       test_predicted$class)
```

```
Confusion Matrix and Statistics

          Reference
Prediction  Adelie Chinstrap Gentoo
  Adelie       145         0      1
  Chinstrap      1        67      0
  Gentoo         0         0    119

Overall Statistics

               Accuracy : 0.994
                 95% CI : (0.9785, 0.9993)
    No Information Rate : 0.4384
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.9906

 Mcnemar's Test P-Value : NA

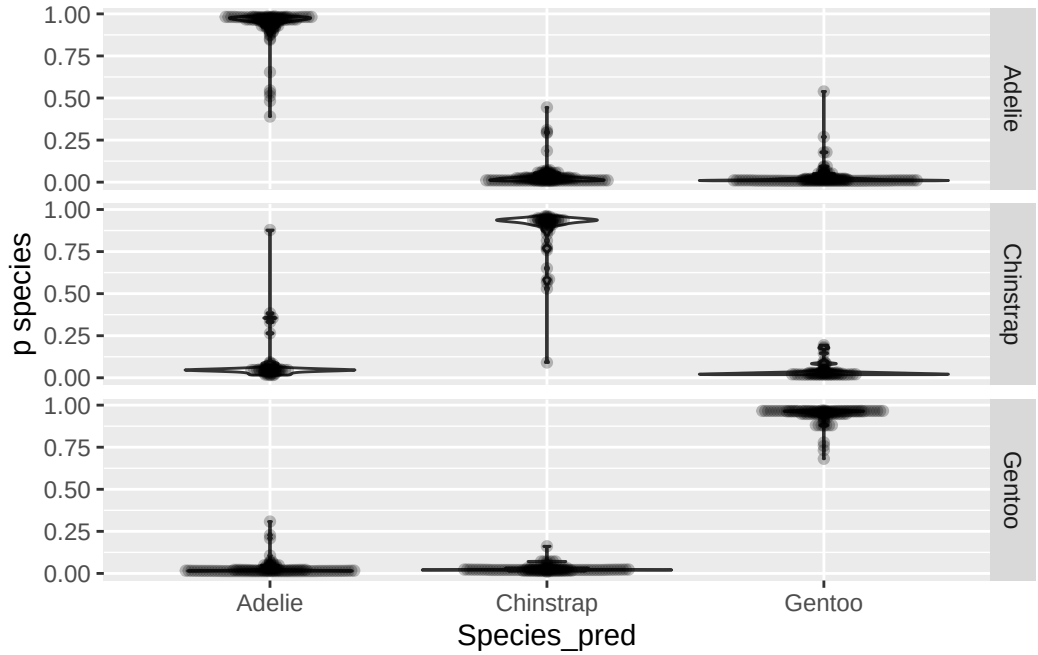
Statistics by Class:

                     Class: Adelie Class: Chinstrap Class: Gentoo
Sensitivity                 0.9932           1.0000        0.9917
Specificity                 0.9947           0.9962        1.0000
Pos Pred Value              0.9932           0.9853        1.0000
Neg Pred Value              0.9947           1.0000        0.9953
Prevalence                  0.4384           0.2012        0.3604
Detection Rate              0.4354           0.2012        0.3574
Detection Prevalence        0.4384           0.2042        0.3574
Balanced Accuracy           0.9939           0.9981        0.9958
```

```
tidy_metrics <- metric_set(accuracy, kap, f_meas, roc_auc, mn_log_loss)
tidy_metrics(test_predicted,
  truth = species, estimate = class,
  Adelie, Chinstrap, Gentoo
) |>
  flextable()
```

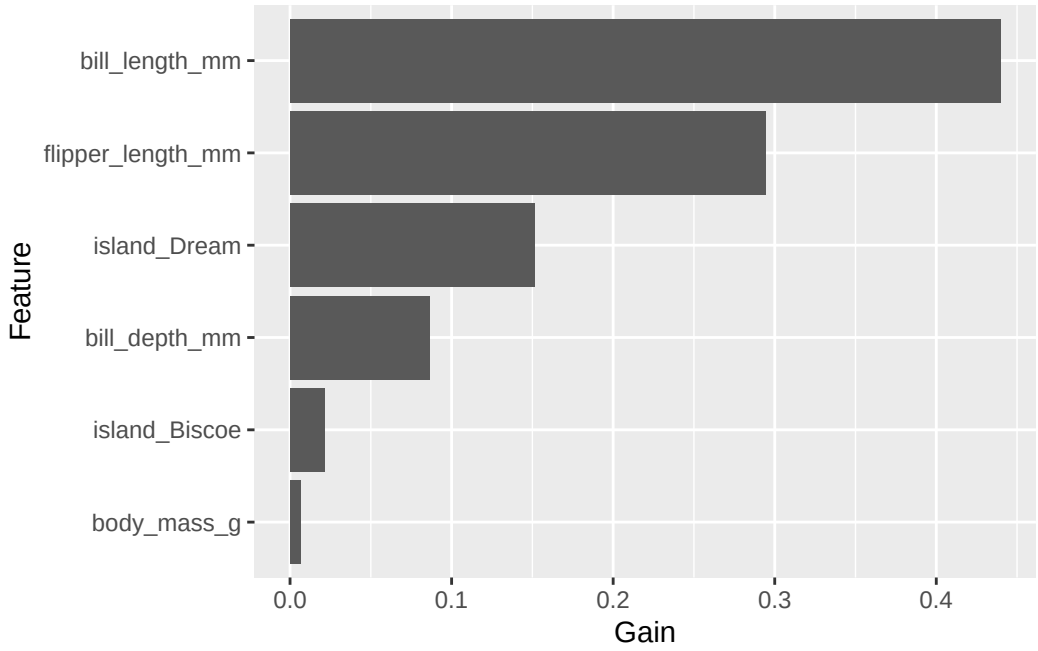
.metric	.estimator	.estimate
accuracy	multiclass	0.99399399
kap	multiclass	0.99058484
f_meas	macro	0.99385306
roc_auc	hand_till	0.99974438
mn_log_loss	multiclass	0.08180201

```
test_predicted |>
  pivot_longer(c(Adelie, Chinstrap, Gentoo),
    values_to = "p species",
    names_to = "Species_pred"
  ) |>
  ggplot(aes(Species_pred, `p species`)) +
  geom_violin() +
  geom_beeswarm(cex = .35, alpha = .25) +
  facet_grid(rows = vars(species))
```




```
# final_fit |>
#   extract_fit_engine() |>
#   vip(geom = "col")
```

```r
final_fit |>
  extract_fit_engine() |>
  xgb.importance() |>
  as_tibble() |>
  ggplot(aes(reorder(Feature, Gain), Gain)) +
  geom_col() +
  coord_flip() +
  labs(x = "Feature", y = "Gain")
```



23 Linear discriminant analysis LDA

LDAs are used to find a linear combination of features that characterizes or separates two or more classes of objects or events. The resulting combination may be used as a linear classifier, or, more commonly, for dimensionality reduction before later classification. LDA is comparable to PCA, but instead of finding the component axes that maximize the variance of our data (PCA), we are interested in the axes that maximize the separation between multiple classes (LDA).

```
pacman::p_load(
  conflicted,
  tidyverse,
  wrappedtools,
  here,
  palmerpenguins,
  ggfortify, GGally,
  MASS,
  caret,
  tidymodels, MASS, discrim
)

# conflict_scout()
conflicts_prefer(
  dplyr::select,
  dplyr::filter,
  palmerpenguins::penguins
)
rawdata <- penguins |>
  na.omit()

predvars <- ColSeeker(namepattern = c("_mm", "_g"))
norm_rec <-
  recipe(rawdata) |>
  update_role(species, new_role = "outcome") |>
  update_role(all_of(predvars$names), new_role = "predictor") |>
  step_normalize(all_numeric_predictors())
lda_data <-
```

```
  norm_rec |>
  prep() |>
  bake(new_data = rawdata)

lda_formula <- paste("species",
  paste(predvars$names, collapse = "+"),
  sep = "~"
) |>
  as.formula()
```

Direct call from MASS package

```
lda_out <- lda(lda_formula, data = lda_data)
lda_out$prior
```

```
   Adelie Chinstrap    Gentoo
0.4384384 0.2042042 0.3573574
```

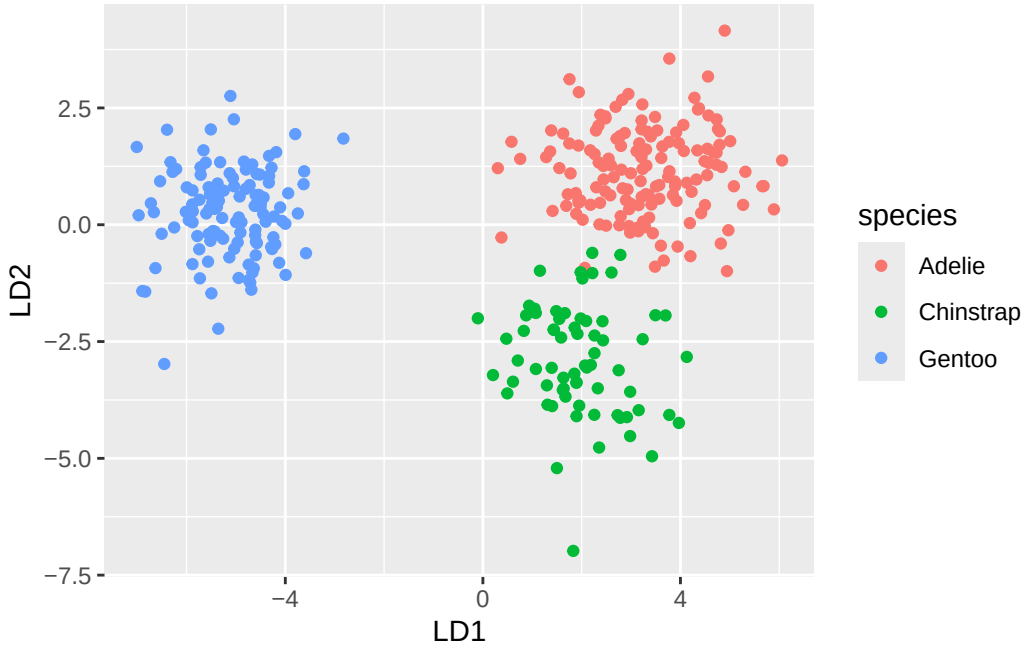
```
lda_out$svd^2 / sum(lda_out$svd^2) # explained variance
```

```
[1] 0.8654754 0.1345246
```

```
lda_out$scaling
```

```
                         LD1         LD2
bill_length_mm    -0.4699047 -2.27825597
bill_depth_mm      2.0512477 -0.02052478
flipper_length_mm -1.1850728  0.19966192
body_mass_g       -1.0849272  1.35726341
```

```
lda_pred <- predict(lda_out)
lda_plotdata <-
  lda_pred$x |>
  as_tibble() |>
  cbind(rawdata |> select(species))
lda_plotdata |>
  ggplot(aes(LD1, LD2, color = species)) +
  geom_point()
```




```
confusionMatrix(lda_pred$class, rawdata$species)
```

```
Confusion Matrix and Statistics

           Reference
Prediction  Adelie Chinstrap Gentoo
  Adelie       145         3      0
  Chinstrap      1        65      0
  Gentoo         0         0    119

Overall Statistics

               Accuracy : 0.988
                 95% CI : (0.9695, 0.9967)
    No Information Rate : 0.4384
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.9811

 Mcnemar's Test P-Value : NA

Statistics by Class:
```

```
                     Class: Adelie Class: Chinstrap Class: Gentoo
Sensitivity                 0.9932           0.9559        1.0000
Specificity                 0.9840           0.9962        1.0000
Pos Pred Value              0.9797           0.9848        1.0000
Neg Pred Value              0.9946           0.9888        1.0000
Prevalence                  0.4384           0.2042        0.3574
Detection Rate              0.4354           0.1952        0.3574
Detection Prevalence        0.4444           0.1982        0.3574
Balanced Accuracy           0.9886           0.9761        1.0000
```

tidymodels approach

```
lda_spec <- discrim_linear() |>
  set_mode("classification") |>
  set_engine("MASS")
lda_workflow <- workflow() |>
  add_recipe(norm_rec) |>
  add_model(lda_spec)

lda_fit <-
  lda_workflow |>
  fit(data = rawdata)

lda_coefficients <- lda_fit |>
  extract_fit_engine() |>
  # For MASS engine, coefficients are in the 'scaling' element
  getElement("scaling")

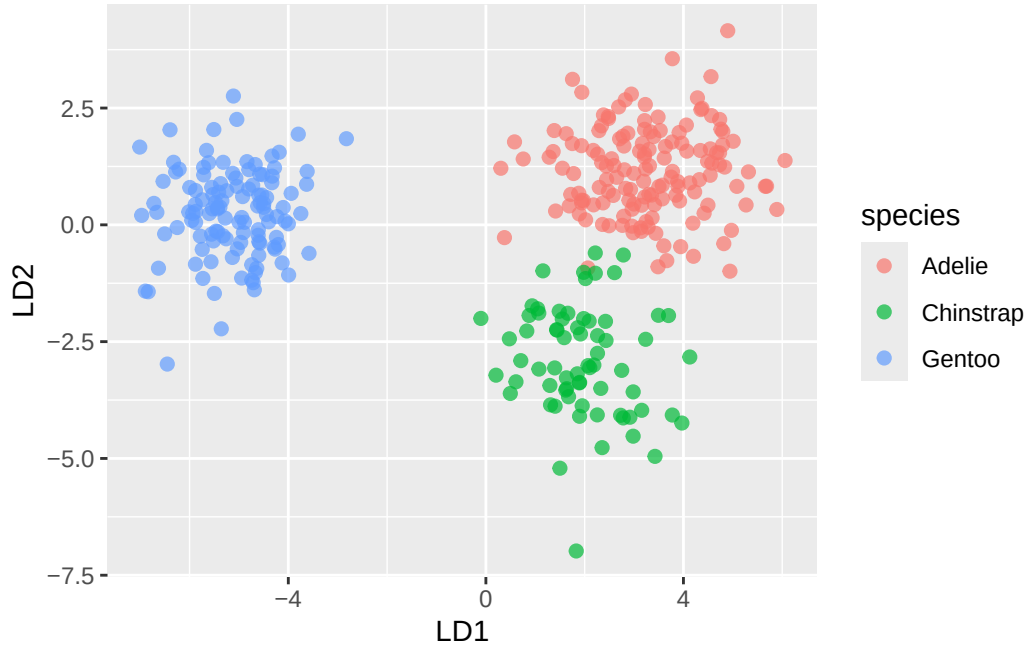
print(lda_coefficients)
```

```
                        LD1         LD2
bill_length_mm    -0.4699047 -2.27825597
bill_depth_mm      2.0512477 -0.02052478
flipper_length_mm -1.1850728  0.19966192
body_mass_g       -1.0849272  1.35726341
```

```
lda_data <-
  bind_cols(
    lda_data,
    lda_fit |>
      extract_fit_engine() |>
      predict(newdata = lda_data) |>
      pluck("x") |>
```

```
      as_tibble()
  )

ggplot(lda_data, aes(x = LD1, y = LD2, color = species)) +
  geom_point(alpha = 0.7, size = 2)
```




```
# data(bordeaux)
bordeaux <- readxl::read_excel(here("data/bordeaux.xlsx")) |>
  mutate(quality = factor(quality,
    levels = c("bad", "medium", "good")
  )) |>
  select(-year)
lda_rec <-
  recipe(bordeaux) |>
  update_role(everything(), new_role = "predictor") |>
  update_role(quality, new_role = "outcome") |>
  step_normalize(all_numeric_predictors())
lda_spec <- discrim_linear() |>
  set_mode("classification") |>
  set_engine("MASS")
lda_workflow <- workflow() |>
  add_recipe(lda_rec) |>
  add_model(lda_spec)
```

```r
lda_fit <-
  lda_workflow |>
  fit(data = bordeaux)

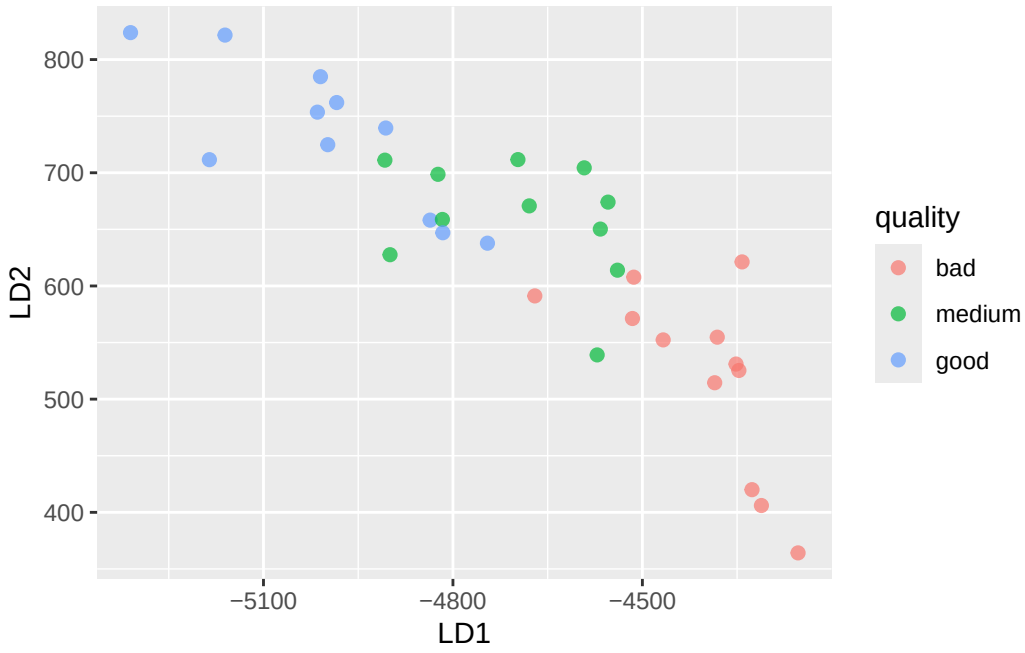
lda_coefficients <- lda_fit |>
  extract_fit_engine() |>
  # For MASS engine, coefficients are in the 'scaling' element
  getElement("scaling")

print(lda_coefficients)
```

```
                  LD1          LD2
temperature -1.2093914  0.006529859
sun         -0.8577274  0.674810955
heat         0.2709930 -1.278475787
rain         0.5361312 -0.564364371
```

```r
bordeaux <-
  bind_cols(
    bordeaux,
    lda_fit |>
      extract_fit_engine() |>
      predict(newdata = bordeaux) |>
      pluck("x") |>
      as_tibble()
  )

ggplot(bordeaux, aes(x = LD1, y = LD2, color = quality)) +
  geom_point(alpha = 0.7, size = 2)
```


800
700
600
500
400
LD2
−5100
−4800
−4500
LD1
quality
bad
medium
good

24 tidymodels framework

In the **tidymodels** framework, the `parsnip` package provides a unified interface for model specification. This abstraction allows researchers to switch between radically different mathematical engines without altering their data processing pipeline.

Extensive documentation is available here: Tidy Modelling with R

```
pacman::p_load(
  conflicted,
  tidyverse,
  palmerpenguins,
  tidymodels,
  kernlab, # SVM engines
  rpart, # Decision Trees
  klaR, # Naive Bayes
  nnet, # Neural Networks and Multinomial Regression
  ranger, # Fast Random Forest
  xgboost, # Gradient Boosting
  kknn, # Weighted KNN
  discrim, # For LDA and Naive Bayes engines
  C50,
  glmnet,
  ggbeeswarm,
  ggforce,
  workflowsets,
  vip,
  future.mirai, mirai
)

conflicts_prefer(
  dplyr::filter,
  dplyr::slice,
  palmerpenguins::penguins,
  vip::vi
)
```

```
rawdata <- penguins |>
  drop_na()
```

Building the tidymodel framework is done in several steps:

24.1 Step 1: Pre-processing

A base_recipe is defined to handle missing data, on-hot-encoding, and zero-variance. This is extended in norm_recipe with normalization.

```
base_recipe <- recipe(species ~ ., data = rawdata) |>
  # 1. Fix missing values (use with care!!)
  step_impute_median(all_numeric_predictors()) |>
  step_impute_mode(all_nominal_predictors()) |>
  # 2. Turn words into numbers
  # This performs one-hot encoding, converting a single nominal column into multiple binary
  step_dummy(all_nominal_predictors(), one_hot = TRUE) |>
  # 3. Remove Zero Variance predictors
  # It identifies columns that contain only a single unique value. Such variables are comput
  step_zv(all_predictors())

norm_recipe <-
  base_recipe |>
  # 4. Centering and Scaling transformation
  # For every numeric predictor, it subtracts the mean and divides by the standard deviation
  # -all_outcomes() ensures the target variable (species) remains a factor and is not mathem
  step_normalize(all_predictors())
```

24.2 Step 2: Defining the model using Parsnip package

Here we will explore multiple machine learning approaches to implement in the work. This demonstrates the modularity of tidymodels and how practical it is to use it to test different ML models to decide the best model for your data.

24.2.1 1. Multinomial Regression (`multinom_reg`)

This is an extension of logistic regression used for multi-class classification. It estimates the probability of a sample belonging to each category. It is computationally efficient and provides interpretable log-odds.

```
multinom_spec <- multinom_reg(penalty = tune(), mixture = tune()) |>
  set_engine("glmnet", family = "multinomial", nlambda = 100) |>
  set_mode("classification")
```

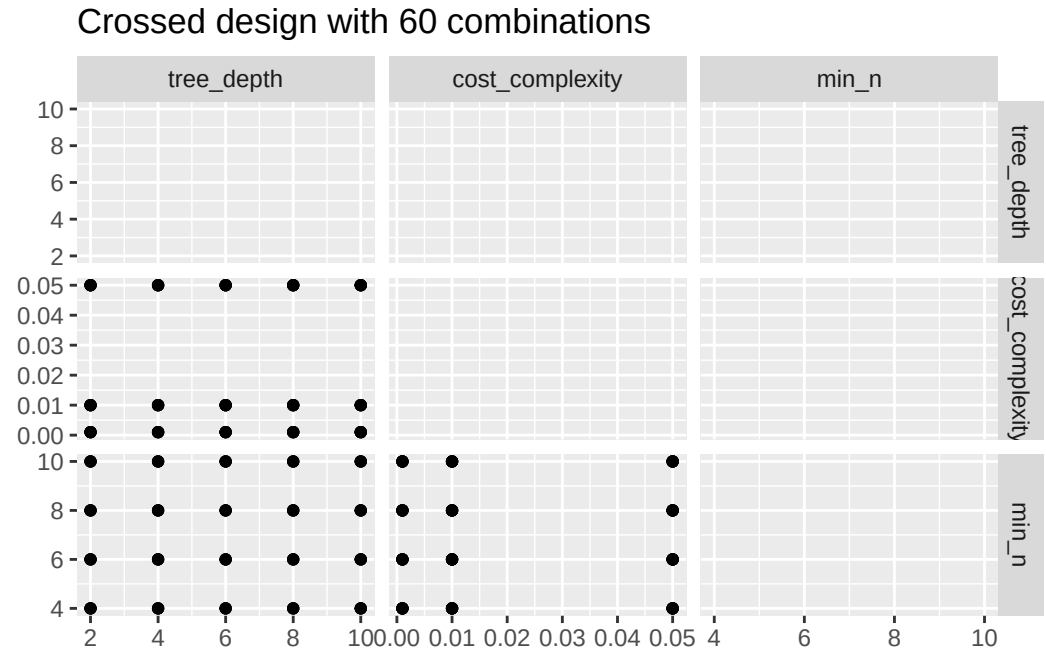
- `penalty`: Represents the total amount of regularization. It penalizes large coefficients to prevent overfitting.
- `mixture`: A value between 0 and 1 that determines the proportion of L1 (Lasso) vs L2 (Ridge) regularization.
- `tune()`: This acts as a placeholder, signaling to the tune package that these values will be determined later via cross-validation or a grid search to find the optimal balance between bias and variance. Downstream when we set grid to eg. 3, the model will test 3 random values for the parameters of the model, giving us 3 accuracy values.
- `family = "multinomial"`: While glmnet usually infers this from the mode, explicitly passing it to the engine ensures the underlying glmnet::glmnet() function uses the grouped-penalty approach, which is more stable for multi-class biological datasets.
- `nlambda = 100`: This instructs the engine to evaluate a path of 100 penalty values. This provides a denser "regularization path," making the subsequent tuning process more granular and likely to find the global optimum for the F1 score or Accuracy.

24.2.2 2. Decision Trees (`decision_tree`)

Decision trees use recursive partitioning to split data into increasingly homogeneous subsets (nodes).

Here a predefined set of tuning parameters is hard coded.

```
tree_grid <- expand.grid(
  tree_depth = seq(2, 10, by = 2),
  cost_complexity = c(0.05, 0.01, 0.001),
  min_n = seq(4, 10, by = 2)
) |>
  as_tibble()
tree_grid |>
  ggplot(aes(x = .panel_x, y = .panel_y)) +
  geom_point() +
  geom_blank() +
  facet_matrix(vars(tree_depth, cost_complexity, min_n),
    layer.upper = FALSE, layer.diag = 2
  ) +
  labs(title = str_glue("Crossed design with {nrow(tree_grid)} combinations"))
```




```
tree_spec <- decision_tree(
  cost_complexity = tune(),
  tree_depth = tune(),
  min_n = tune()
) |>
  set_engine("rpart", model = TRUE) |>
  set_mode("classification")
```

- `cost_complexity`: A pruning parameter that controls the trade-off between model complexity and its fit to the training data. Larger values penalize more complex trees, effectively "snipping" off branches that do not significantly reduce the overall impurity.
- `tree_depth`: Limits the maximum number of nodes from the root to the furthest leaf. A high depth risks overfitting by capturing noise in biological samples, while a low depth may underfit by missing critical diagnostic thresholds.
- `min_n`: the minimum number of data points required in a node to attempt a further split.

24.2.3 3. Random Forests (`rand_forest`)

An ensemble method that builds multiple decision trees using **bagging** (Bootstrap Aggregating) and feature randomness.

```
rf_grid <- expand.grid(mtry = 2:4) |>
  as_tibble()
rf_spec <- rand_forest(
  trees = 1000,
  mtry = tune()
) |>
  set_engine("ranger", importance = "impurity") |>
  set_mode("classification")
```

- `trees`: Specifies the number of trees in the ensemble.
- `mtry` = tune(): It defines the number of predictors that will be randomly sampled at each split in each tree. Tuning this prevents a few dominant predictors (like a very large body mass) from overshadowing other subtle but biologically relevant features.
- `importance` = "impurity": This calculates Variable Importance Scores based on the Gini impurity. It measures how much each variable (e.g., flipper length) contributes to making the nodes "pure" across all 1000 trees.

24.2.4 4. K-Nearest Neighbors (`nearest_neighbor`)

A non-parametric, instance-based learning algorithm. It classifies a point based on the majority vote of its k neighbors in the feature space. **Note:** Features must be normalized/scaled for KNN to function correctly.

```
knn_spec <- nearest_neighbor(
  neighbors = tune(),
  weight_func = "triangular"
) |>
  set_engine("kknn") |>
  set_mode("classification")
```

- **`neighbors = tune()`**: The number of nearest data points to consider.
- **`weight_func = "triangular"`**: This defines the kernel function used to weight the neighbors. In a "triangular" scheme, neighbors that are geographically closer to the query point in the feature space exert a stronger influence on the final classification than those further away. This often yields better boundary definitions than simple majority voting.
- **`weight_func = tune()`**: Instead of assuming a triangular kernel, tuning allows the grid search to test "rectangular" (simple voting), "gaussian", or "optimal" kernels to see which best captures the biological clusters.

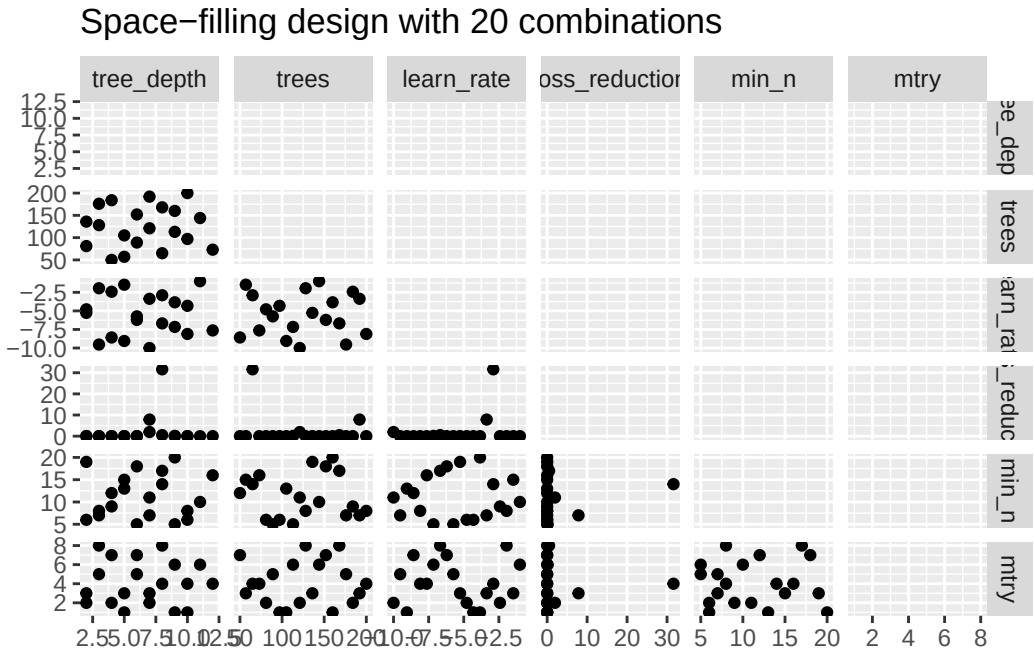
Statistical Note: k-NN is susceptible to the "Curse of Dimensionality." In datasets with hundreds of features (like transcriptomics), the "distance" between points becomes less meaningful. In such cases, dimensionality reduction (e.g., PCA) should be added to the `recipe` before the `knn_spec`.

24.2.5 5. Boosted Trees (`boost_tree` via XGBoost)

Unlike Random Forests, Gradient Boosting builds trees **sequentially**. Each new tree attempts to correct the errors (residuals) of the previous ones.

Tuning parameters are defined by specifying ranges and the number of steps. This gives greater control as compared to just a call to `tune()`.

```
xgb_grid <- grid_space_filling(
  tree_depth(range = c(2, 12)), # Searches between 2 and 10
  trees(range = c(50, 200)),
  learn_rate(),
  loss_reduction(),
  min_n(range = c(5, 20)),
  # finalize(sample_size(), rawdata),
  finalize(mtry(), rawdata),
  size = 20
)
xgb_grid |>
  mutate(learn_rate = log10(learn_rate)) |>
  ggplot(aes(x = .panel_x, y = .panel_y)) +
  geom_point() +
  geom_blank() +
  facet_matrix(vars(everything()),
    layer.upper = FALSE, layer.diag = 2
  ) +
  labs(title = str_glue("Space-filling design with {nrow(xgb_grid)} combinations"))
```




```
xgb_spec <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  min_n = tune(),
  loss_reduction = tune(),
  mtry = tune(),
  # sample_size = tune(),
  learn_rate = tune()
) |>
  set_engine("xgboost") |>
  set_mode("classification")
```

- **`trees = tune()`**: Specifies the number of boosting iterations.
- **`learn_rate = tune()`**: Scales the contribution of each new tree. A smaller learning rate makes the model more robust to overfitting but requires more **`trees`** to converge.
- **`loss_reduction`**: Specifies the minimum loss reduction required to make a further partition on a leaf node. It acts as a conservative guard against capturing noise.

24.2.6 6. C5.0 Decision Trees (`boost_tree` via C5.0)

This defines a **Boosted Decision Tree specification** using the **C5.0** engine. C5.0 is an evolution of the landmark C4.5 algorithm, specifically designed to be faster, more memory-

efficient, and more accurate through a process called **boosting**.

```
c50_spec <- boost_tree(trees = 10) |>
  set_engine("C5.0") |>
  set_mode("classification")
```

24.2.7 7. Linear Discriminant Analysis (`discrim_linear`)

LDA seeks to find a linear combination of features that maximizes the separation between classes. It assumes that predictors follow a multivariate normal distribution with a common covariance matrix. It uses **Bayes' Theorem** to estimate the probability that a given observation belongs to a specific penguin species.

```
lda_spec <- discrim_linear() |>
  set_engine("MASS") |>
  set_mode("classification")

# slda_spec <- discrim_linear(penalty = tune()) |>
#   set_engine("mda") |>
#   set_mode("classification")
```

- **penalty**: If you were working with genomic data where the number of predictors exceeds the number of samples, the standard `MASS` engine would fail because the covariance matrix becomes non-invertible. Switching to a penalized engine (like `mda` or `klaR`) allows for **Shrinkage Discriminant Analysis**, which regularizes the covariance matrix to ensure numerical stability.

24.2.8 8. Support Vector Machines (`svm_poly` / `svm_rbf`)

SVMs find the **hyperplane** that maximizes the "margin" between classes. Using the "kernel trick," they can project data into higher dimensions to separate classes that are not linearly separable in their original state.

```
svm_spec <- svm_poly(cost = tune(), degree = tune()) |>
  set_engine("kernlab") |>
  set_mode("classification")
```

- **`cost`**: This is the regularization parameter. It controls the trade-off between maximizing the margin and minimizing training errors. A **high cost** penalizes misclassifications heavily, leading to a complex boundary that may overfit. A **low cost** allows for a "softer" margin, which can improve generalization in noisy biological data.

- **`degree`:** This defines the exponent of the polynomial. A degree of 1 is equivalent to a linear kernel; a degree of 2 or 3 allows for quadratic or cubic curves. Higher degrees can capture very complex patterns but are computationally expensive and prone to overfitting.

24.2.9 9. Naive Bayes (`naive_Bayes`)

Based on **Bayes' Theorem**, this model assumes that all predictors are independent of one another given the class label (the "naive" assumption). It is remarkably fast and handles categorical data well.

```
nb_spec <- naive_Bayes(smoothness = tune()) |>
  set_engine("klaR", usekernel = TRUE) |>
  # set_engine("naivebayes") |>
  set_mode("classification")
```

- **`smoothness = tune()`**: This parameter controls the flexibility of the probability estimates. In biological datasets where some measurements might be rare or unique, a higher smoothness value prevents the model from assigning a "zero probability" to a species just because a specific value hasn't been seen in the training data.
- **`usekernel = TRUE`**: This tells the engine to use **Kernel Density Estimation (KDE)** instead of assuming a perfect Gaussian (normal) distribution for numeric features. This is significantly more flexible for biological data that might be skewed or bimodal.
- `naivebayes` engine is a lightweight alternative to `klaR` that is often faster for large-scale screenings where complex kernel density estimation is not required.

Note: Naive Bayes is particularly susceptible to highly correlated features. In your `base_recipe`, if variables like `body_mass_g` and `flipper_length_mm` are almost identical in their distribution, the model may "double-count" their evidence, leading to overconfident (and potentially incorrect) probability estimates.

24.2.10 10. Multilayer Perceptron / Neural Networks (`mlp`)

The foundation of deep learning. It consists of an input layer, one or more hidden layers, and an output layer. It uses backpropagation to adjust weights and learn complex hierarchical patterns.

```
mlp_spec <- mlp(
  hidden_units = tune(),
  penalty = tune(),
  epochs = 500
```

```
) |>
  set_engine("nnet", MaxNWts = 2000) |>
  set_mode("classification")
```

- **`hidden_units = tune()`**: This defines the number of nodes in the hidden layer. Tuning this is critical; too few units will fail to capture the complexity of the biological data (**underfitting**), while too many can lead to the model memorizing the training samples (**overfitting**).
- **`penalty = tune()`**: This introduces **weight decay**. It penalizes large weights in the network, forcing the model to remain simpler and more robust to noise in the physical measurements.
- **`epochs = 500`**: An epoch represents one full pass of the entire training dataset through the network. 500 iterations allow the weight optimization (backpropagation) sufficient time to converge on a solution that minimizes the loss function.
- **`MaxNWts = 2000`**: Biological datasets with many predictors can quickly exceed the default weight limit in `nnet`. Increasing this ensures the model can actually initialize if the feature space grows.

24.2.11 11. Logistic Regression with Elastic Net (`logistic_reg`)

This model combines **Lasso** ($L1$) and **Ridge** ($L2$) regularization. It is ideal for gene expression data where you have more variables than samples ($p > n$).

Note: For multi-class (penguins), multinom_reg is used, but logistic_reg is standard for binary outcomes.

```
elastic_spec <- logistic_reg(penalty = tune(), mixture = tune()) |>
  set_engine("glmnet")
```

24.3 Step 3: Competitive Model Evaluation (Using `workflow_set`)

As different models have different requirements for preprocessing, a base recipe and an extended recipe with normalization have been prepared. `workflow_set` allows you to define multiple workflows with different combinations of recipes and models. This is particularly useful for comparing models that require different preprocessing steps (e.g., KNN needs normalization, while Random Forest does not).

For some models the tuning grid had been specified by either vectors of values or range definitions, they are added as options.

```
model_set_base <-
  workflow_set(
    preproc = list(base = base_recipe),
    models = list(
      dec_tree = tree_spec,
      rf = rf_spec,
      xgb = xgb_spec,
      c50 = c50_spec
    )
  ) |>
  option_add(grid = tree_grid, id = "base_dec_tree") |>
  option_add(grid = xgb_grid, id = "base_xgb") |>
  option_add(grid = rf_grid, id = "base_rf")
model_set_norm <-
  workflow_set(
    preproc = list(norm = norm_recipe),
    models = list(
      multinom = multinom_spec,
      knn = knn_spec,
      lda = lda_spec,
      svm = svm_spec,
      nb = nb_spec,
      mlp = mlp_spec
    )
  )
model_set <- bind_rows(model_set_base, model_set_norm)
```

24.4 Step 4: Cross-Validation

Once you have your `workflow_set`, you don't use `fit()` like you do for a single model. You usually use **Resampling** (like Cross-Validation) to see which model is the most suitable before you pick a winner.

```
# Create Cross-Validation folds
set.seed(123)
folds <- 5
penguin_folds <- vfold_cv(rawdata, v = folds, repeats = 10)

# Simplify the metrics to avoid naming conflicts
penguin_metrics <- metric_set(accuracy, kap)

# Run all models in the set across all folds
```

```
plan(mirai_multisession, workers = max(availableCores() - 2, folds))
models_test_results <-
  model_set |>
  workflow_map(
    "tune_grid",
    resamples = penguin_folds,
    metrics = penguin_metrics,
    verbose = TRUE,
    seed = 321
  )
```

```
i  1 of 10 tuning:     base_dec_tree
```

```
v  1 of 10 tuning:     base_dec_tree (15.9s)
```

```
i  2 of 10 tuning:     base_rf
```

```
v  2 of 10 tuning:     base_rf (2.5s)
```

```
i  3 of 10 tuning:     base_xgb
```

```
v  3 of 10 tuning:     base_xgb (11.6s)
```

```
i   No tuning parameters. `fit_resamples()` will be attempted
```

```
i  4 of 10 resampling: base_c50
```

```
v  4 of 10 resampling: base_c50 (1.2s)
```

```
i  5 of 10 tuning:     norm_multinom
```

```
v  5 of 10 tuning:     norm_multinom (3.8s)
```

```
i  6 of 10 tuning:     norm_knn
```

```
v  6 of 10 tuning:     norm_knn (2s)
```

```
i   No tuning parameters. `fit_resamples()` will be attempted
```

```
i  7 of 10 resampling: norm_lda

v  7 of 10 resampling: norm_lda (1.9s)

i  8 of 10 tuning:     norm_svm

v  8 of 10 tuning:     norm_svm (5.6s)

i  9 of 10 tuning:     norm_nb

v  9 of 10 tuning:     norm_nb (6.4s)

i 10 of 10 tuning:     norm_mlp

v 10 of 10 tuning:     norm_mlp (5.1s)
```

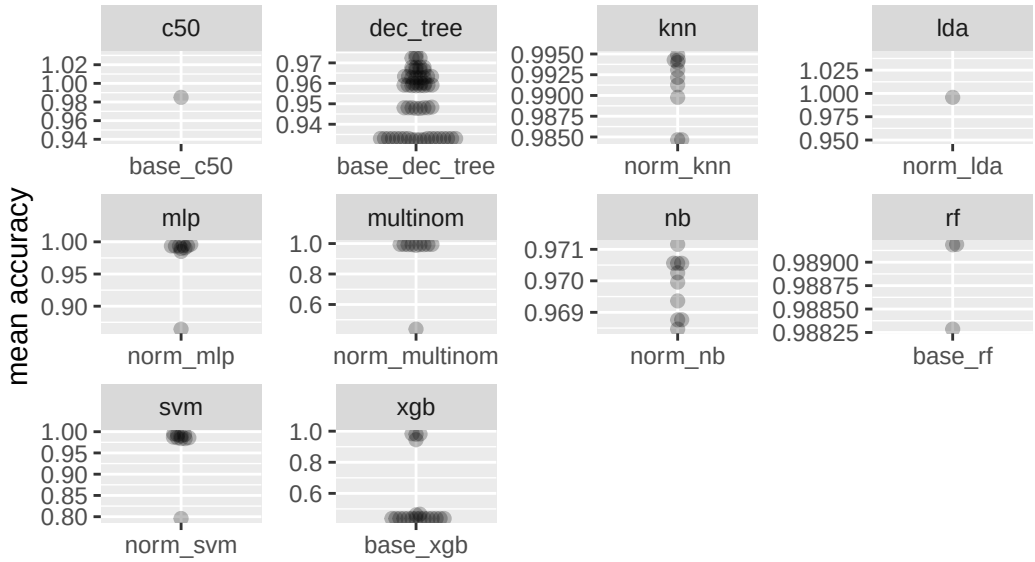
```
plan(sequential)
```

24.5 Step 5: Ranking the models

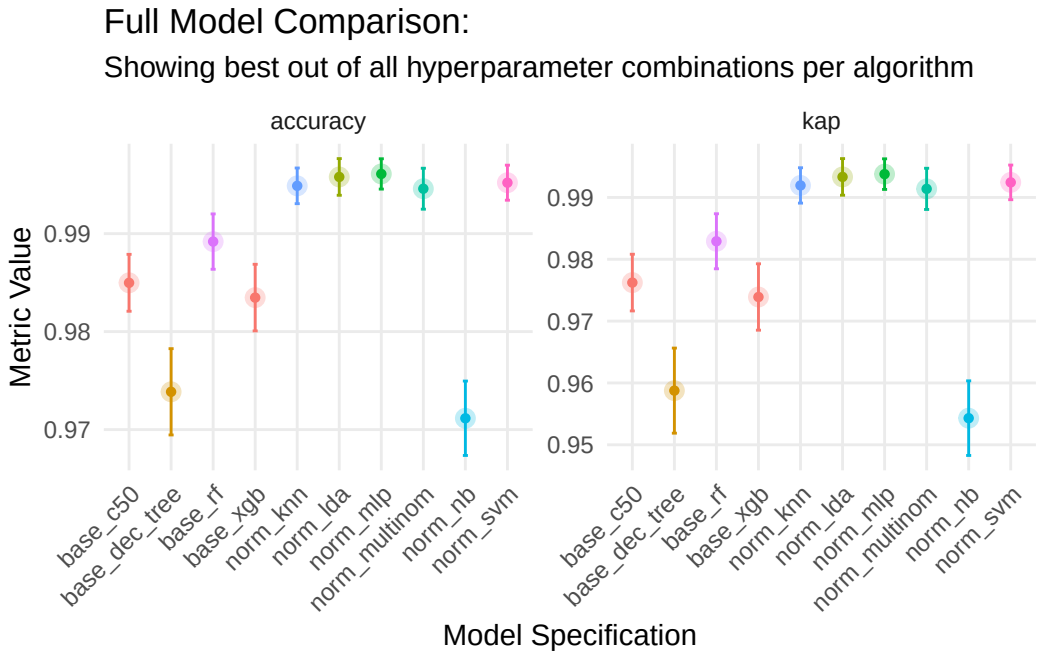
```
# Extract the metrics
# If some models failed, this will show you only the ones that succeeded
model_metrics <- models_test_results |>
  filter(sapply(result, function(x) inherits(x, "tune_results"))) |>
  collect_metrics()

# Visualize the ranking of all models

model_metrics |>
  filter(.metric == "accuracy") |>
  mutate(model = str_remove(wflow_id, "^(base|norm|reduced)_")) |>
  arrange(model) |>
  ggplot(aes(x = wflow_id, y = mean)) +
  geom_beeswarm(size = 2, alpha = 0.25, cex = 3) +
  facet_wrap(facets = vars(model), scales = "free") +
  scale_x_discrete(name = NULL) +
  labs(
    title = "Full Model Comparison: All Grid Search Results",
    subtitle = "Showing all hyperparameter combinations per algorithm",
    y = "mean accuracy"
  )
```




```r
autoplot(models_test_results, select_best = TRUE) +
  # Set select_best = FALSE to reveal all grid combinations
  # Map wflow_id to X to group triplicates together
  aes(x = wflow_id) +
  # Increase point size for better visibility
  geom_point(size = 3, alpha = 0.25) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, vjust = 1, hjust = 1),
    legend.position = "none",
    panel.grid.minor = element_blank()
  ) +
  labs(
    title = "Full Model Comparison: ",
    subtitle = "Showing best out of all hyperparameter combinations per algorithm",
    x = "Model Specification",
    y = "Metric Value"
  )
```



What the Output Table Means

When this code works, you will see a table (a `tibble`) with several columns. Here is how to read them:

Column	What it tells you
`wflow_id`	Which model/recipe combo this is (e.g., `base_rf` for Random Forest).
`.metric`	What is being measured (`accuracy` or `roc_auc`).
`mean`	This is the average score from your repeated 5-fold cross-validation. Closer to 1.0 is better.
`n`	How many times the model was tested (should match your folds * times, e.g., 5*10).
`std_err`	The "Wiggle Room." If this is high, the model is inconsistent. You want this to be low.

Note: C50 and lda models show only 1 value for accuracy, as compared to the multiple values for other models.

This is because:

C50: there are no parameters that were set to tune(), since trees were set to 10.

lda: LDA is a "closed-form" mathematical formula. It doesn't have parameters to tune. It just calculates the distance between group means; therefore, the grid simply collapses into 1.

24.6 Step 6: Best model selection

```
# View the top models
model_metrics_accuracy <- model_metrics |>
  filter(.metric == "accuracy") |>
  arrange(desc(mean))

# 1. Identify the best model type
best_results <- models_test_results |>
  rank_results(rank_metric = "kap", select_best = TRUE)

# 2. Extract the best workflow configuration for the top model
best_workflow_fit <- models_test_results |>
  extract_workflow_set_result("norm_multinom") |> # Replace with your winner ID
  select_best(metric = "kap")
best_workflow_fit_xgb <- models_test_results |>
  extract_workflow_set_result("base_xgb") |> # Replace with your winner ID
  select_best(metric = "kap")
best_workflow_fit_rf <- models_test_results |>
  extract_workflow_set_result("base_rf") |> # Replace with your winner ID
  select_best(metric = "accuracy")

# 3. Finalize the workflow
final_workflow <- models_test_results |>
  extract_workflow("base_rf") |>
  finalize_workflow(best_workflow_fit_rf)
```

- **`finalize_workflow()`**: It maps these values onto the model specification.
- **Note:** After this line, the `final_workflow` is no longer tunable. It is now a **fully defined model** ready to be used for prediction.

Why is this necessary?

In a bioinformatic pipeline, you cannot simply use the results from the `tune_grid()` directly for prediction. The tuning process happens across multiple "folds" (subsets) of your data. To get the most robust version of your chosen model, you must:

1. **Identify** the best settings from the folds.
2. **Apply** those settings to a fresh workflow (Finalize).

3. **Train** that finalized workflow on the **entire** `rawdata` (Last Fit).

25 Principal Components Analysis

```
pacman::p_load(
  conflicted,
  tidyverse,
  wrappedtools,
  palmerpenguins,
  ggfortify, GGally,
  tidytext,
  FactoMineR,
  ggrepel, ggforce,
  boot,
  tidymodels,
  corrplot
)

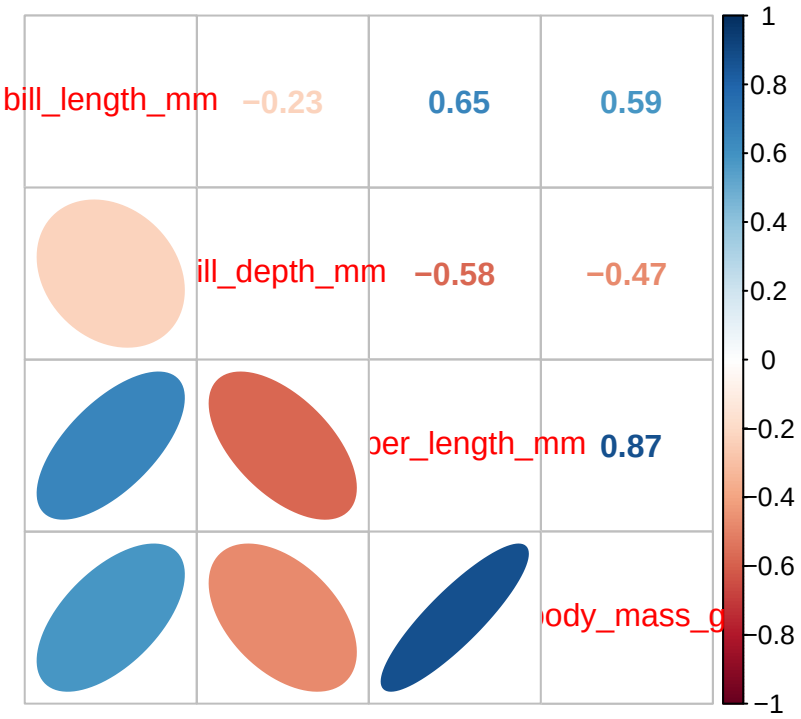
# conflict_scout()
conflicts_prefer(
  dplyr::slice,
  dplyr::filter,
  stats::screeplot,
  stats::biplot,
  palmerpenguins::penguins
)
rawdata <- penguins |>
  na.omit()
rawdata <- mutate(rawdata,
  ID = paste("P", 1:nrow(rawdata))
) |>
  select(ID, everything())
predvars <- ColSeeker(namepattern = c("_mm", "_g"))
```

25.1 Exploration of correlations between predictor variables

```
corout <- rawdata |>
  select(predvars$names) |>
```

```
  cor()

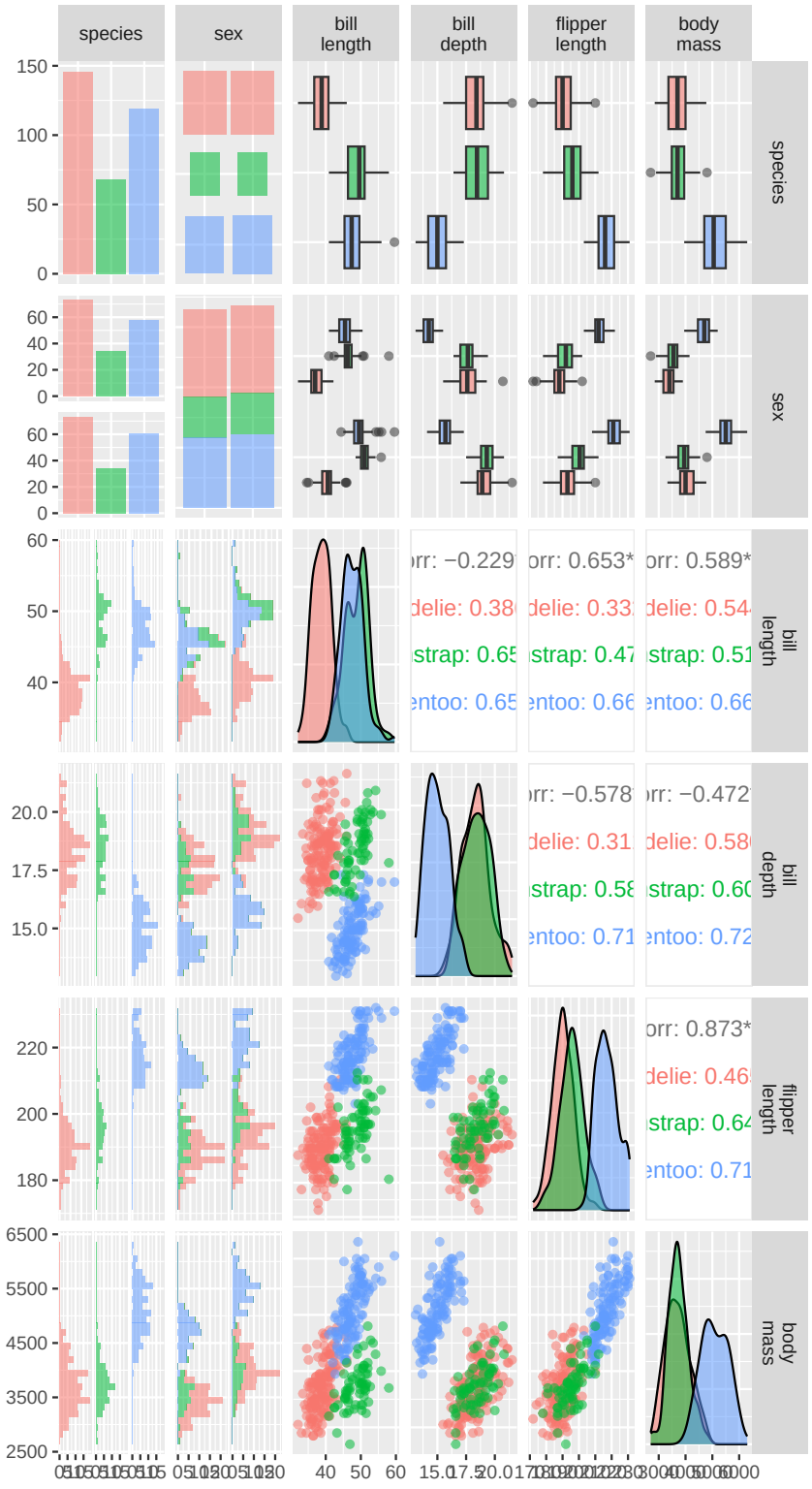
corrplot(corout,
  type = "upper",
  method = "number", tl.pos = "d"
)
corrplot(corout,
  add = TRUE, type = "lower",
  method = "ellipse",
  diag = FALSE, tl.pos = "n", cl.pos = "n"
)
```




```
ggpairs(
  rawdata |> select(
    species, sex,
```

```
    predvars$names
  ) |>
    rename_with(\(x) str_replace(x, "(.+)_(.+)_.+", "\\1 \\2") |>
      str_wrap(8)),
  aes(color = species, alpha = .5)
)
```

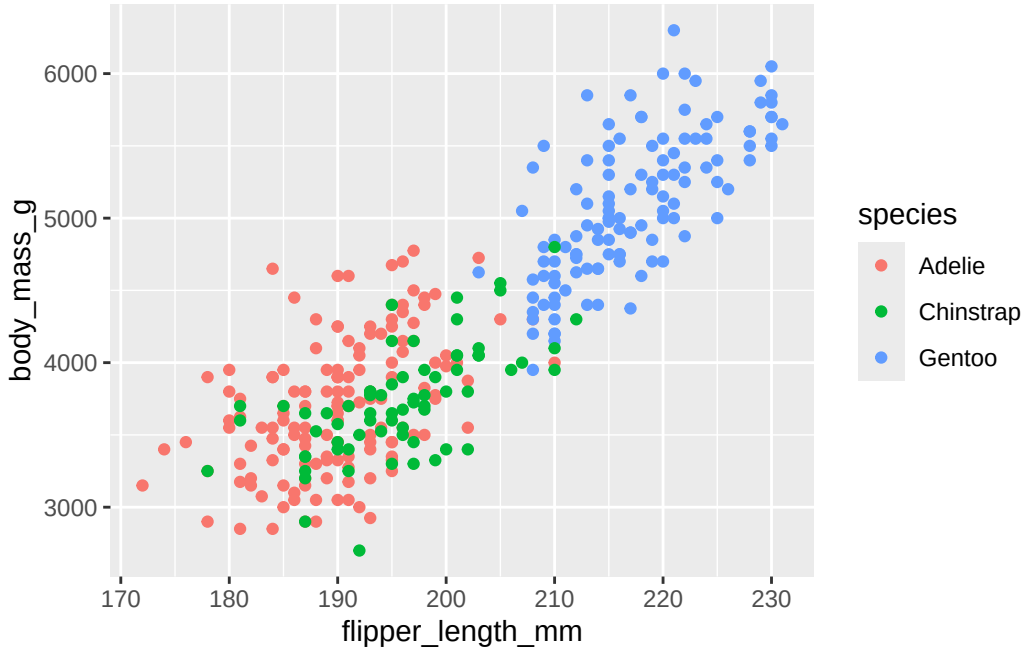
```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```


species
sex
bill length
bill depth
flipper length
body mass
150
100
50
0
60
40
20
0
60
50
40
20.0
17.5
15.0
220
200
180
6500
5500
4500
3500
2500
40
50
60
15.0
17.5
20.0
orr: −0.229
orr: 0.653*
orr: 0.589*
delie: 0.38
delie: 0.33
delie: 0.54
strap: 0.65
strap: 0.47
strap: 0.51
entoo: 0.65
entoo: 0.66
entoo: 0.66
orr: −0.578
orr: −0.472
delie: 0.31
delie: 0.58
strap: 0.58
strap: 0.60
entoo: 0.71
entoo: 0.72
orr: 0.873*
delie: 0.46
strap: 0.64
entoo: 0.71

Video on PCA

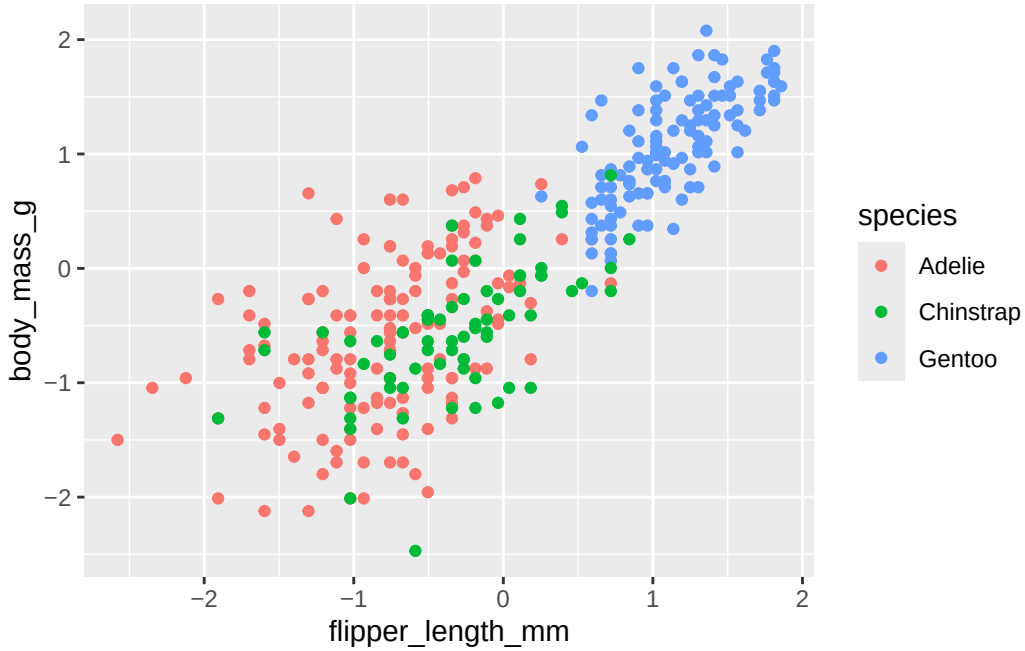
25.2 Two variable example

```
predvars2 <- ColSeeker(namepattern = c("body", "flipper"))
ggplot(rawdata, aes(.data[[predvars2$names[1]]], .data[[predvars2$names[2]]])) +
  geom_point(aes(color = species))
```

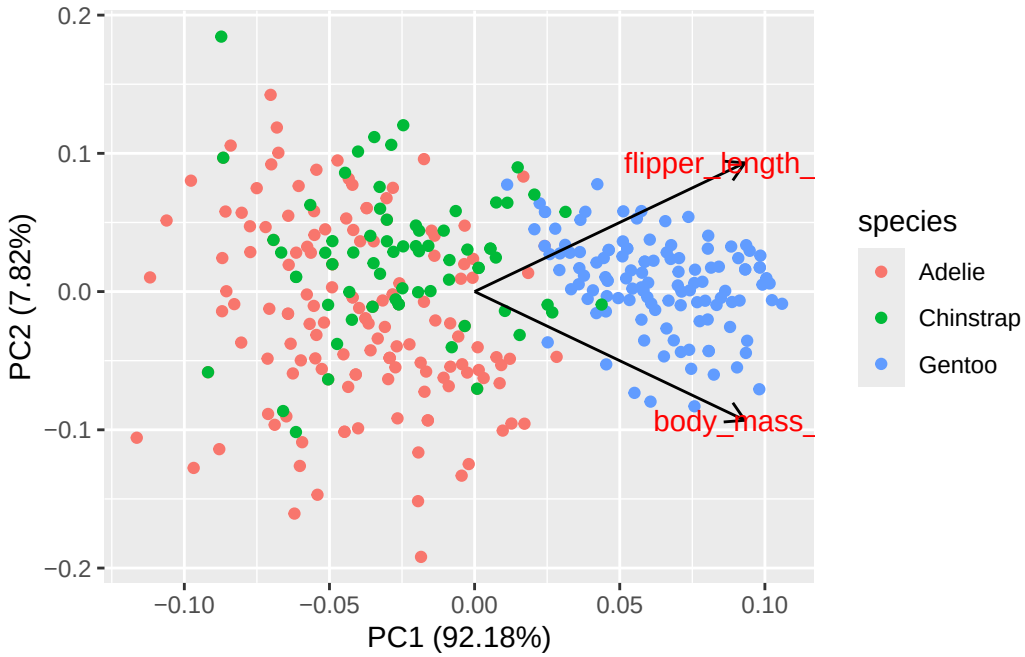



```
var2_rec <-
  recipe(rawdata) |>
  update_role(all_of(predvars2$names), new_role = "predictor") |>
  step_YeoJohnson(all_numeric_predictors()) |>
  step_normalize(all_numeric_predictors())
v2data <-
  var2_rec |>
  prep() |>
  bake(rawdata)

ggplot(v2data, aes(.data[[predvars2$names[1]]], .data[[predvars2$names[2]]])) +
  geom_point(aes(color = species))
```



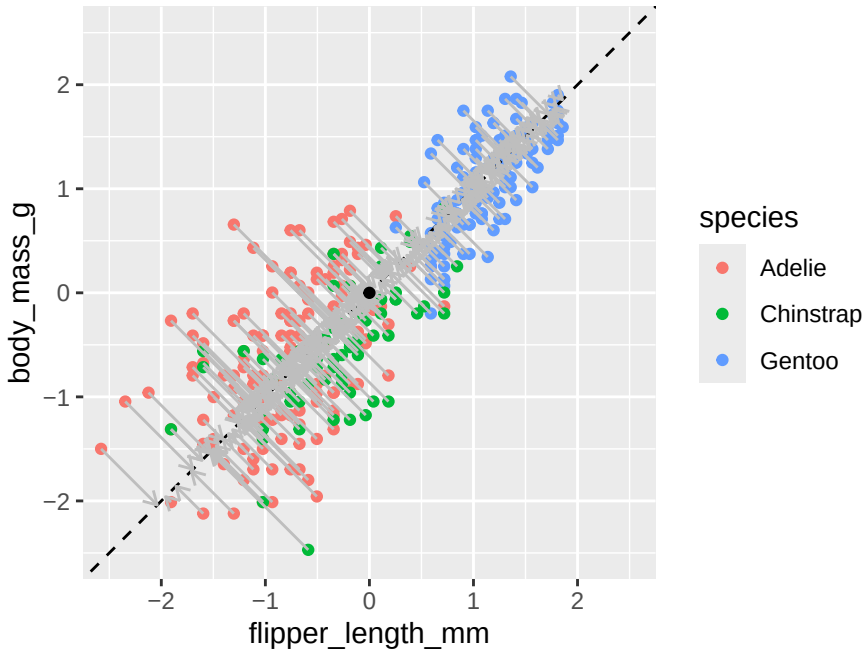

```
pca2_out <- prcomp(
  v2data |>
    select(predvars2$names),
  center = F, scale. = F
)
autoplot(pca2_out,
  data = v2data, colour = "species",
  loadings = TRUE, loadings.colour = "black",
  loadings.label = TRUE, loadings.label.size = 4
)
```




```
# Extract PC1 loadings
pc1_loadings <- pca2_out$rotation[, 1]

# Calculate axis endpoint
axis_end <- c(pc1_loadings[1] * 2, pc1_loadings[2] * 2)
slope_pc1 <- pc1_loadings[2] / pc1_loadings[1]
# Create the plot
ggplot(v2data, aes(
  .data[[predvars2$names[1]]],
  .data[[predvars2$names[2]]]
)) +
  geom_point(aes(color = species)) +
  geom_abline(
    intercept = 0, slope = slope_pc1,
    color = "black", linetype = "dashed"
  ) +
  geom_segment(
    aes(
      xend = (.data[[predvars2$names[1]]] +
        .data[[predvars2$names[2]]]) / 2,
      yend = (.data[[predvars2$names[1]]] +
        .data[[predvars2$names[2]]]) / 2
    ),
```

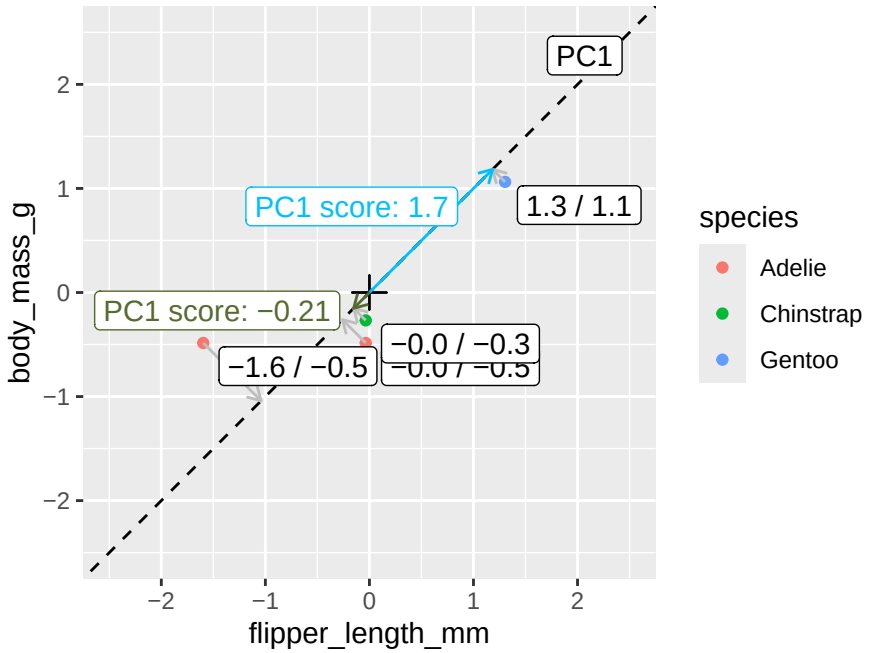
```
    color = "grey",
    arrow = arrow(length = unit(0.2, "cm"))
  ) +
  annotate(geom = "point", x =0, y = 0) +
  coord_fixed(ratio = 1, xlim = c(-2.5, 2.5), ylim = c(-2.5, 2.5))
```




```
pc1plot <-
  ggplot(v2data |> slice(1, 101, 201, 301), aes(
    .data[[predvars2$names[1]]],
    .data[[predvars2$names[2]]]
  )) +
  geom_point(x = 0, y = 0, shape = 3, size = 4) +
  geom_point(aes(color = species)) +
  geom_abline(
    intercept = 0, slope = slope_pc1,
    color = "black", linetype = "dashed"
  ) +
  annotate("label",
    x = 2.5, y = 2.5, label = paste("PC1"),
    hjust = 1.1, vjust = 1.1, color = "black"
  ) +
  geom_segment(
    aes(
```

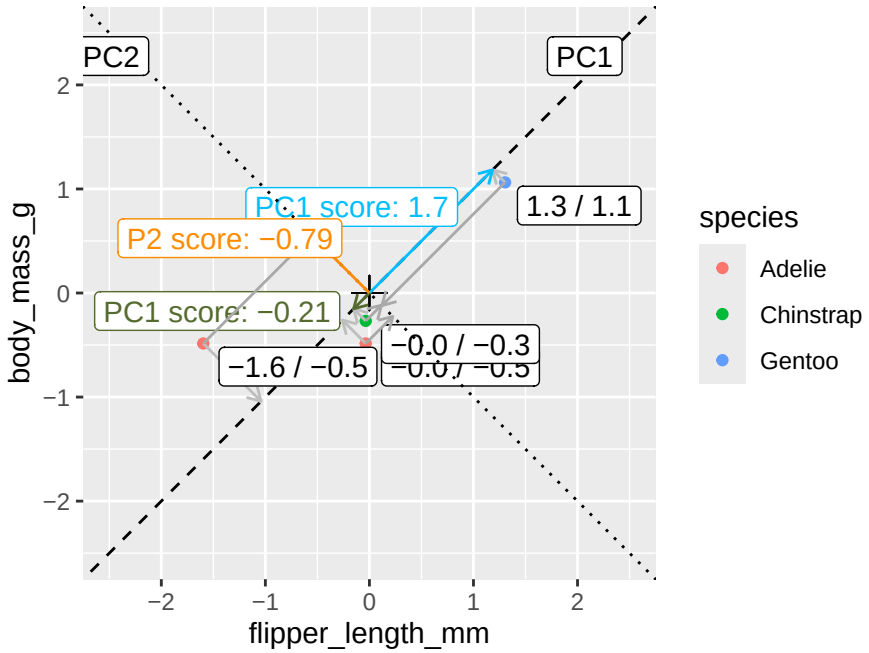
```
    xend = (.data[[predvars2$names[1]]] +
      .data[[predvars2$names[2]]]) / 2,
    yend = (.data[[predvars2$names[1]]] +
      .data[[predvars2$names[2]]]) / 2
  ),
  color = "grey",
  arrow = arrow(length = unit(0.2, "cm"))
) +
geom_label(
  aes(label = paste(
    roundR(.data[[predvars2$names[1]]]), "/",
    roundR(.data[[predvars2$names[2]]])
  )),
  hjust = -.1, vjust = 1.1
) +
annotate("segment",
  x = 0, y = 0,
  xend = (v2data[[predvars2$names[1]]][201] +
    v2data[[predvars2$names[2]]][201]) / 2,
  yend = (v2data[[predvars2$names[1]]][201] +
    v2data[[predvars2$names[2]]][201]) / 2,
  arrow = arrow(length = unit(0.2, "cm")), color = "deepskyblue"
) +
annotate("label",
  x = 1.05, y = 1.05, label = paste(
    "PC1 score:",
    roundR(pca2_out$x[201, 1])
  ),
  hjust = 1.1, vjust = 1.1, color = "deepskyblue"
) +
annotate("segment",
  x = 0, y = 0,
  xend = (v2data[[predvars2$names[1]]][301] +
    v2data[[predvars2$names[2]]][301]) / 2,
  yend = (v2data[[predvars2$names[1]]][301] +
    v2data[[predvars2$names[2]]][301]) / 2,
  arrow = arrow(length = unit(0.2, "cm")), color = "darkolivegreen"
) +
annotate("label",
  x = -0.05, y = 0.05, label = paste(
    "PC1 score:",
    roundR(pca2_out$x[301, 1])
  ),
```

```
    hjust = 1.1, vjust = 1.1, color = "darkolivegreen"
  ) +
  coord_fixed(ratio = 1, xlim = c(-2.5, 2.5), ylim = c(-2.5, 2.5))
pc1plot
```




```
pc1plot +
  geom_abline(
    intercept = 0, slope = -slope_pc1,
    color = "black", linetype = "dotted"
  ) +
  annotate("label",
    x = -2.05, y = 2.5, label = paste("PC2"),
    hjust = 1.1, vjust = 1.1, color = "black"
  ) +
  geom_segment(
    aes(
      xend = (.data[[predvars2$names[1]]] -
        .data[[predvars2$names[2]]]) / 2,
      yend = (.data[[predvars2$names[1]]] -
        .data[[predvars2$names[2]]]) / -2
    ),
    color = "darkgrey",
    arrow = arrow(length = unit(0.2, "cm"))
```

```
  ) +
  annotate("segment",
    x = 0, y = 0,
    xend = (v2data[[predvars2$names[1]]][1] -
      v2data[[predvars2$names[2]]][1]) / 2,
    yend = (v2data[[predvars2$names[1]]][1] -
      v2data[[predvars2$names[2]]][1]) / -2,
    arrow = arrow(length = unit(0.2, "cm")), color = "darkorange"
  ) +
  annotate("label",
    x = -0.05, y = 0.75, label = paste(
      "P2 score:",
      roundR(pca2_out$x[1, 2])
    ),
    hjust = 1.1, vjust = 1.1, color = "darkorange"
  )
```




```
pca2_out$x[c(1, 101, 201, 301), ]
```

```
            PC1        PC2
[1,] -1.4719176 -0.7863345
[2,] -0.3676815  0.3179016
[3,]  1.6738703  0.1701917
```

```
[4,] -0.2143560  0.1645761
```

```
v2data[c(1, 101, 201, 301), ]
```

```
# A tibble: 4 x 9
  ID    species   island    bill_length_mm bill_depth_mm flipper_length_mm
  <chr> <fct>     <fct>              <dbl>         <dbl>             <dbl>
1 P 1   Adelie    Torgersen           39.1          18.7           -1.60
2 P 101 Adelie    Biscoe              38.6          17.2           -0.0352
3 P 201 Gentoo    Biscoe              45            15.4            1.30
4 P 301 Chinstrap Dream               47.5          16.8           -0.0352
# i 3 more variables: body_mass_g <dbl>, sex <fct>, year <int>
```

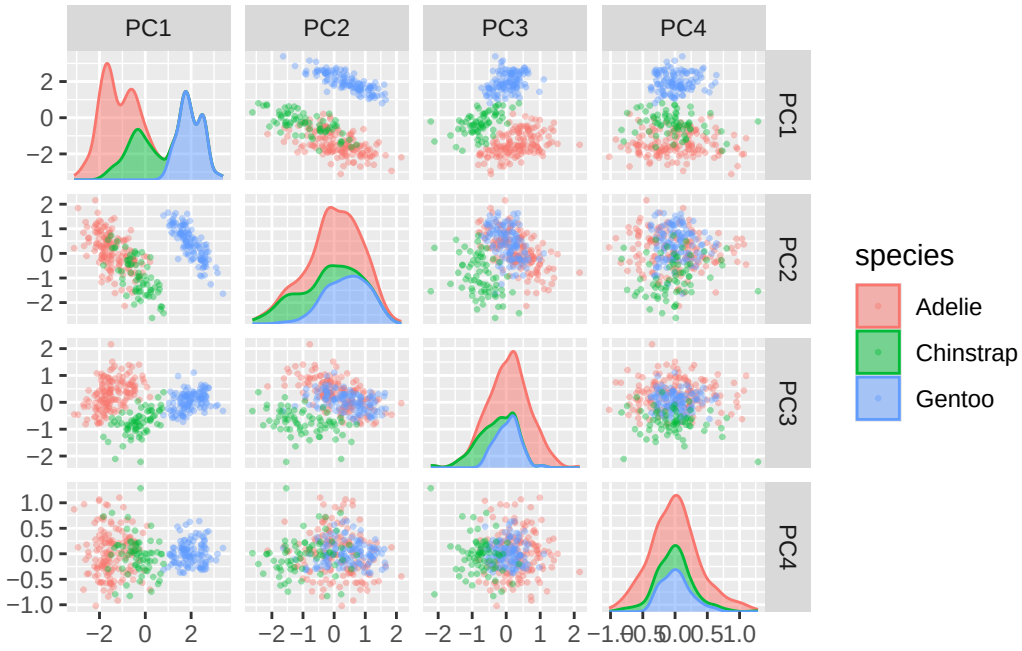
25.3 tidymodels approach

```
pca_rec <-
  recipe(rawdata) |>
  update_role(all_of(predvars$names), new_role = "predictor") |>
  step_YeoJohnson(all_numeric_predictors()) |>
  step_normalize(all_numeric_predictors())
pca_rec_trained <-
  pca_rec |>
  prep()

pca_preped <-
  pca_rec_trained |>
  step_pca(all_numeric_predictors(), id = "pca_step") |>
  prep()

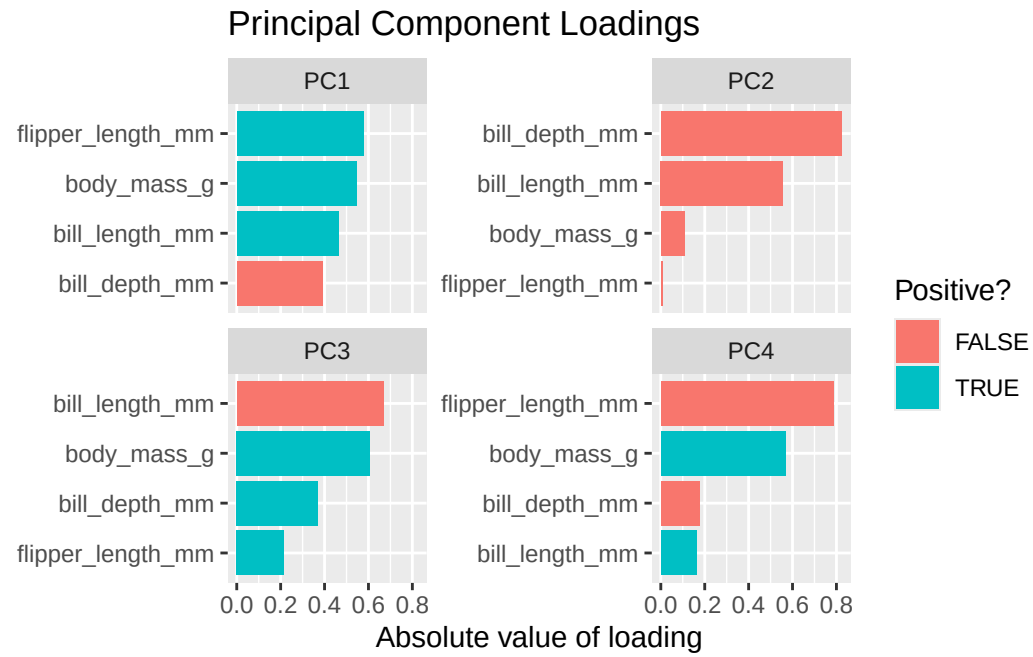
pca_baked <-
  pca_preped |>
  bake(new_data = rawdata)

pca_baked |>
  ggplot(aes(x = .panel_x, y = .panel_y, color = species, fill = species)) +
  geom_point(alpha = 0.4, size = 0.5) +
  geom_autodensity(alpha = .5) +
  facet_matrix(vars(starts_with("PC")), layer.diag = 2)
```

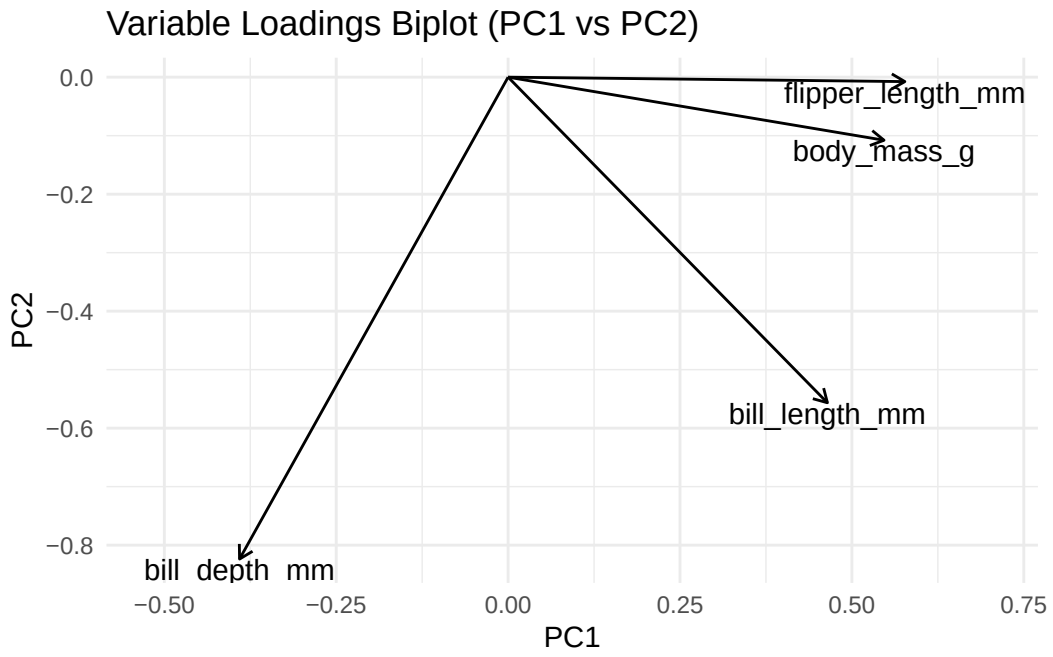



```
pca_loadings <-
  pca_preped |>
  tidy(id = "pca_step")

pca_loadings |>
  group_by(component) |>
  slice_max(abs(value), n = 10) |> # Get top 10 influencers per PC
  ungroup() |>
  ggplot(aes(
    x = abs(value), y = reorder_within(terms, abs(value), component),
    fill = value > 0
  )) +
  geom_col() +
  scale_y_reordered() + # Works with reorder_within for clean facets
  facet_wrap(vars(component), scales = "free_y") +
  labs(
    x = "Absolute value of loading",
    y = NULL,
    fill = "Positive?",
    title = "Principal Component Loadings"
  )
```




```
pca_loadings |>
  filter(component %in% c("PC1", "PC2")) |>
  pivot_wider(names_from = component, values_from = value) |>
  ggplot(aes(x = PC1, y = PC2)) +
  geom_segment(aes(xend = PC1, yend = PC2),
    x = 0, y = 0, arrow = arrow(length = unit(0.2, "cm"))
  ) +
  geom_text(aes(label = terms), check_overlap = TRUE, vjust = 1, hjust = .5) +
  theme_minimal() +
  labs(title = "Variable Loadings Biplot (PC1 vs PC2)")+
  scale_x_continuous(expand = expansion(.2))
```

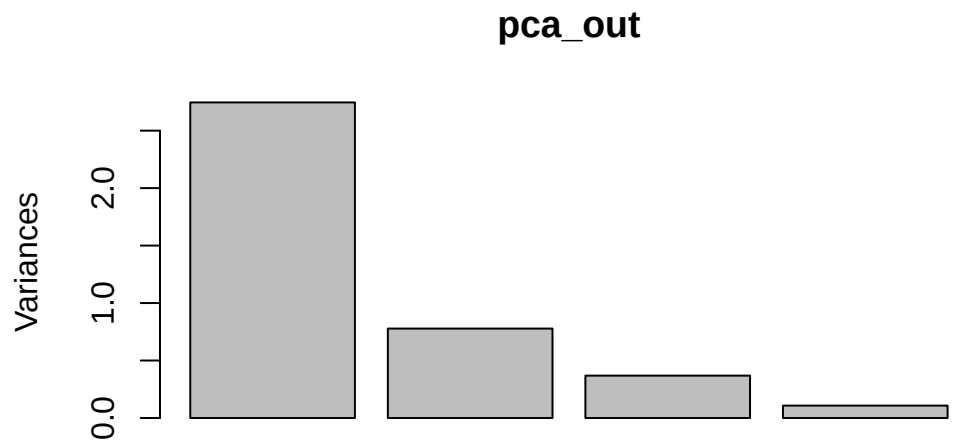


25.4 base approach

```
pca_out <- prcomp(rawdata |>
                    select(predvars$names),
                  center = T,scale. = T)
summary(pca_out)
```

```
Importance of components:
                          PC1    PC2     PC3     PC4
Standard deviation     1.6569 0.8821 0.60716 0.32846
Proportion of Variance 0.6863 0.1945 0.09216 0.02697
Cumulative Proportion  0.6863 0.8809 0.97303 1.00000
```

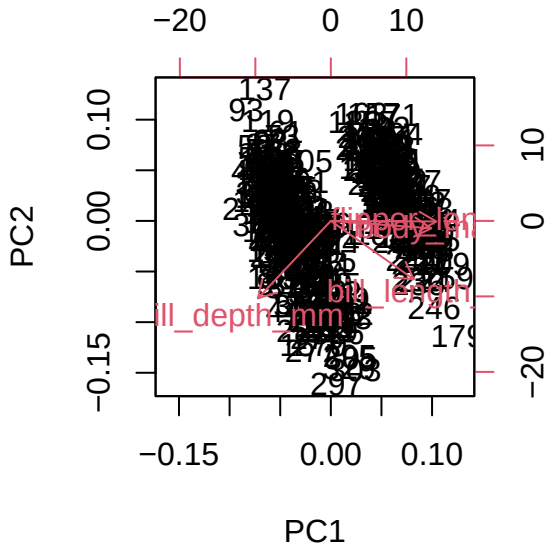
```
screeplot(pca_out,npcs = 4)
```



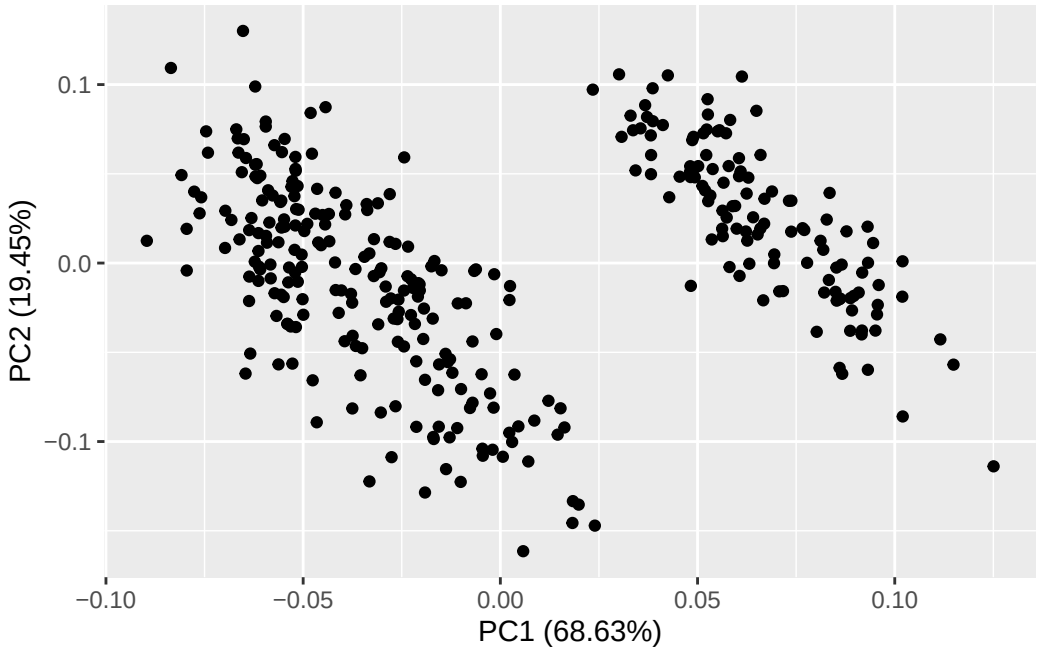

```
pca_out$rotation
```

```
                       PC1         PC2        PC3        PC4
bill_length_mm     0.4537532 -0.60019490 -0.6424951  0.1451695
bill_depth_mm     -0.3990472 -0.79616951  0.4258004 -0.1599044
flipper_length_mm  0.5768250 -0.00578817  0.2360952 -0.7819837
body_mass_g        0.5496747 -0.07646366  0.5917374  0.5846861
```

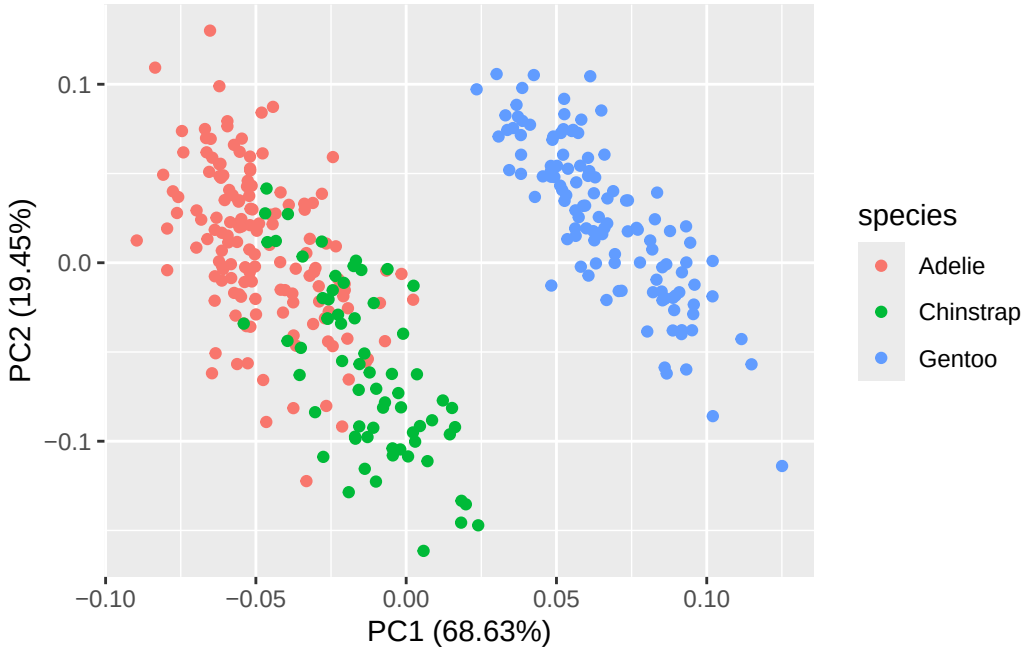
```
biplot(pca_out)
```



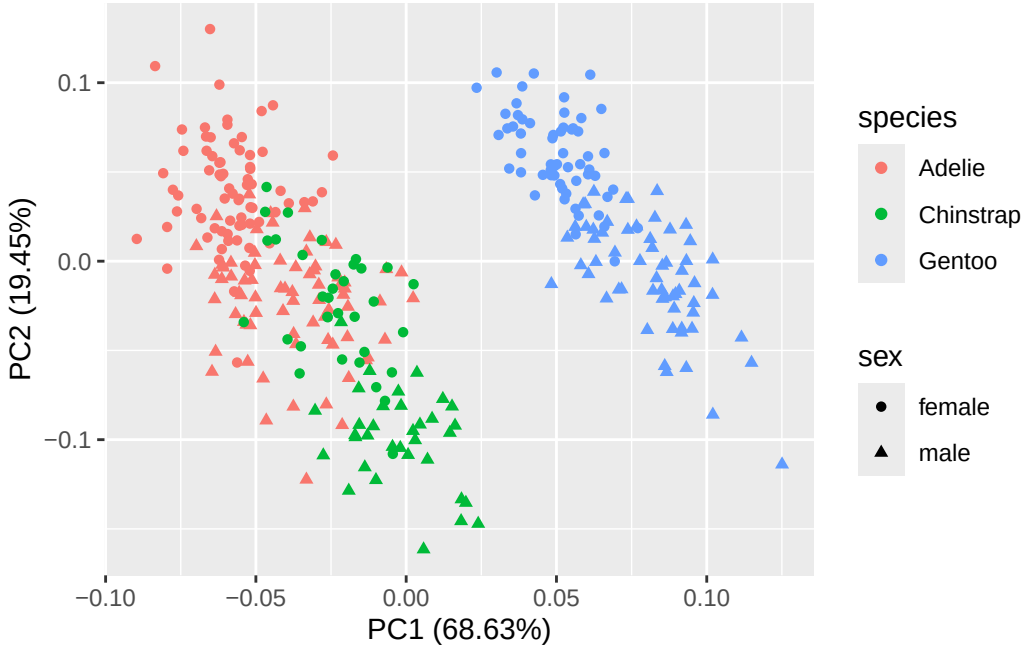

```
autoplot(pca_out)
```




```
autoplot(pca_out, data = rawdata, colour = 'species')
```

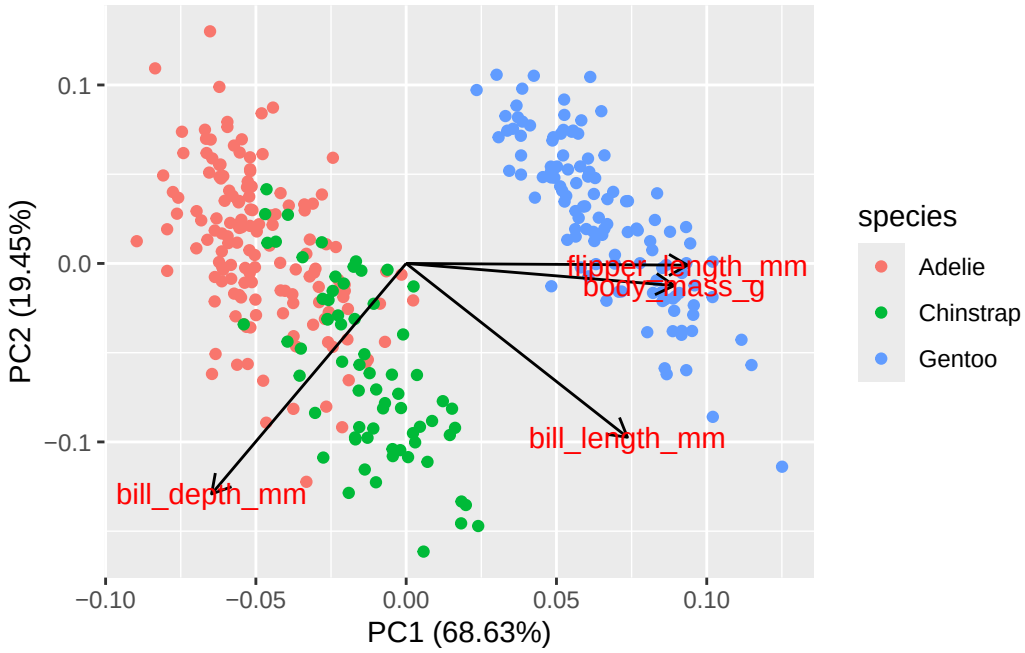



```
autoplot(pca_out, data = rawdata,
         colour = 'species', shape = "sex")
```

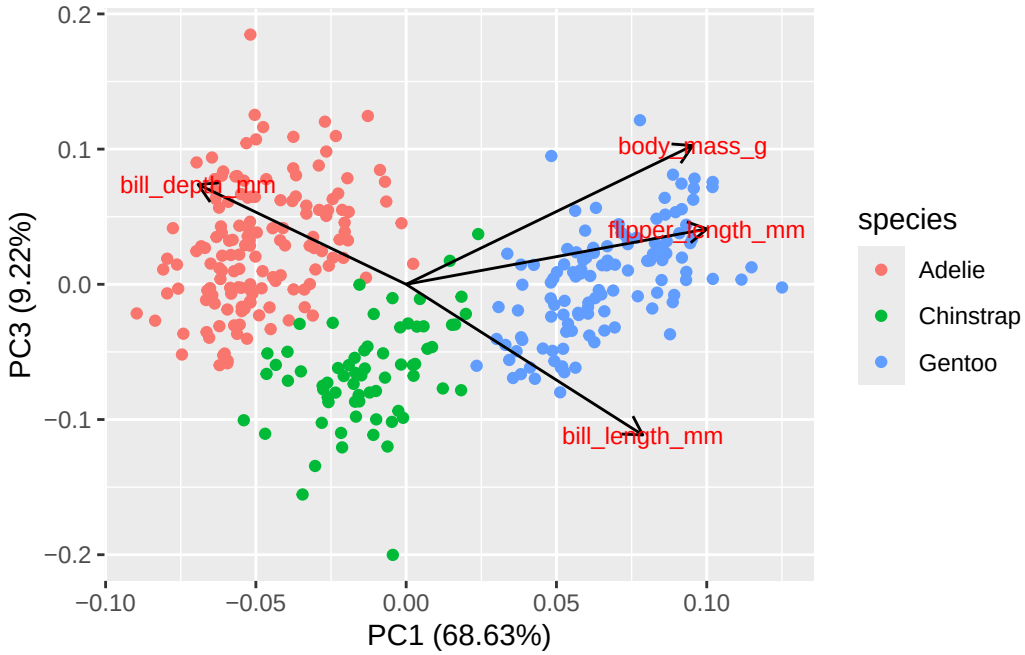



```
autoplot(pca_out, data = rawdata, colour = 'species',
```

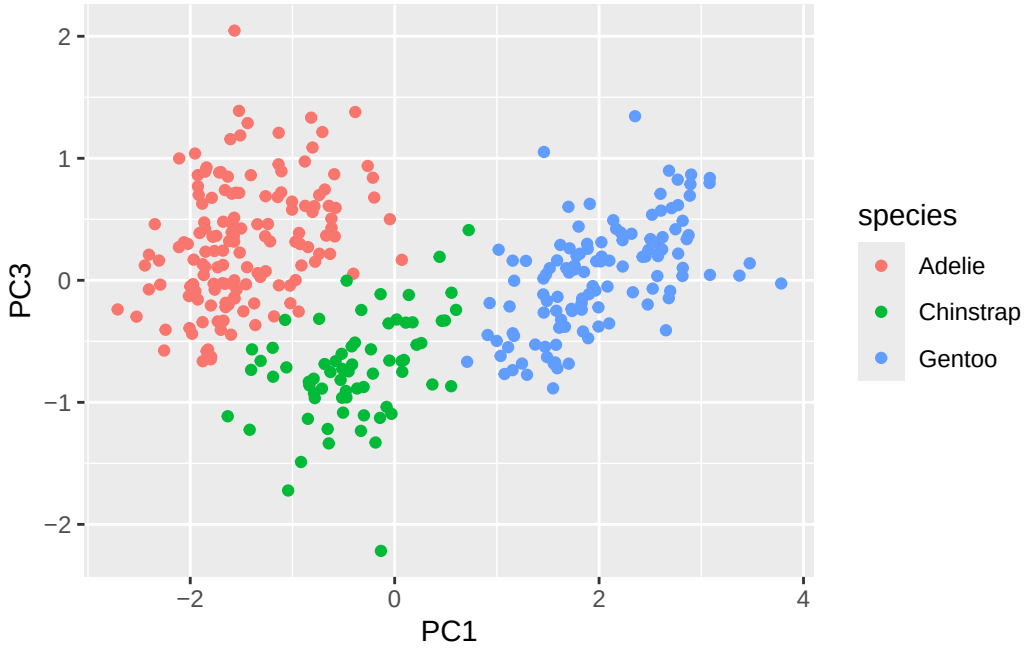
```
        loadings = TRUE, loadings.colour = 'black',
        loadings.label = TRUE, loadings.label.size = 4)
```



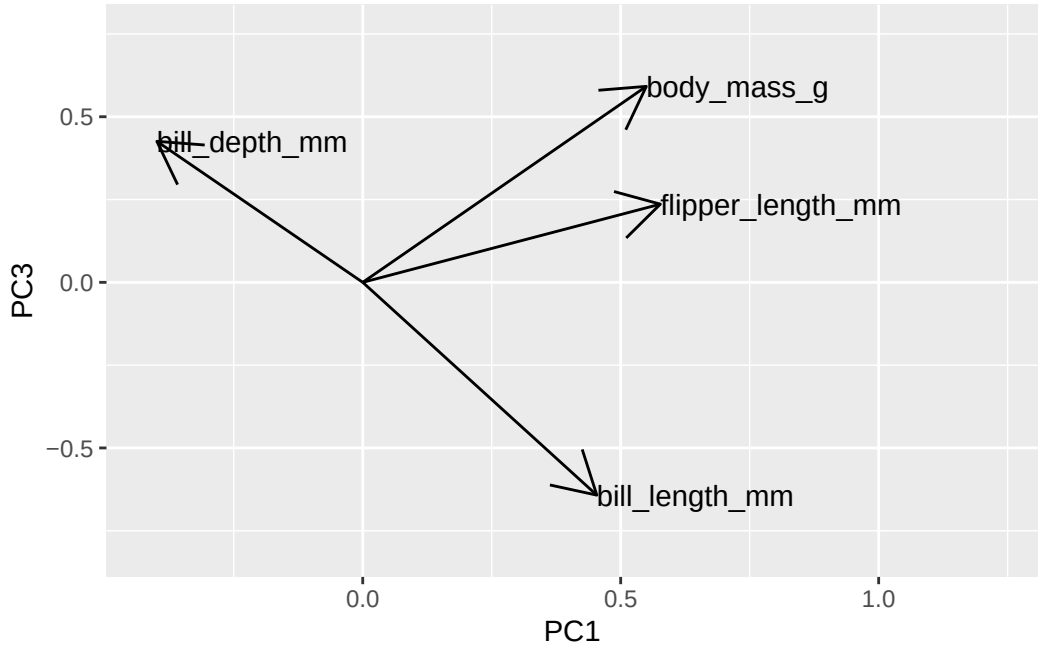

```
#contribution of variables to component
autoplot(pca_out, data=rawdata, colour='species',
         loadings = TRUE, loadings.colour = 'black',
         loadings.label = TRUE, loadings.label.size = 3,
         x=1,y=3)
```




```
pca_out$x |>
  as_tibble() |>
  bind_cols(rawdata) |>
  ggplot(aes(PC1,PC3,color = species))+
  geom_point()
```

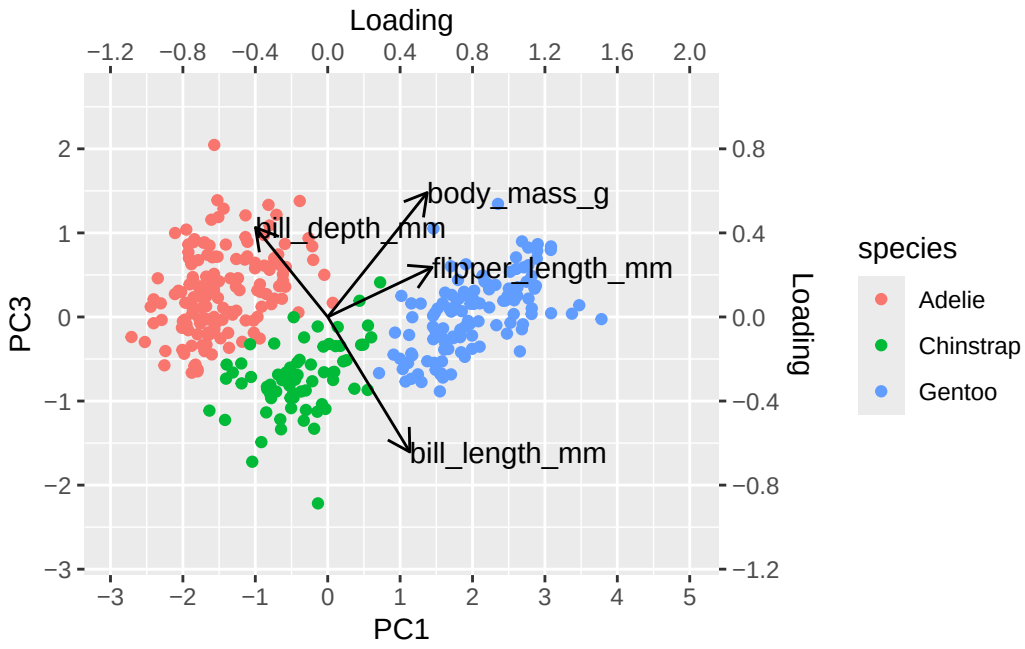



```
pca_out$rotation |>
  as_tibble(rownames = "Variable") |>
  ggplot(aes(PC1,PC3))+
  # geom_label_repel(aes(label=Variable))+
  geom_text(aes(label = Variable),
            hjust = 0)+
  geom_segment(aes(xend = 0, yend = 0),
               arrow = arrow(end = 'first'))+
  scale_y_continuous(expand = expansion(.2))+
  scale_x_continuous(expand = expansion(
    mult = c(.1,.75)))
```

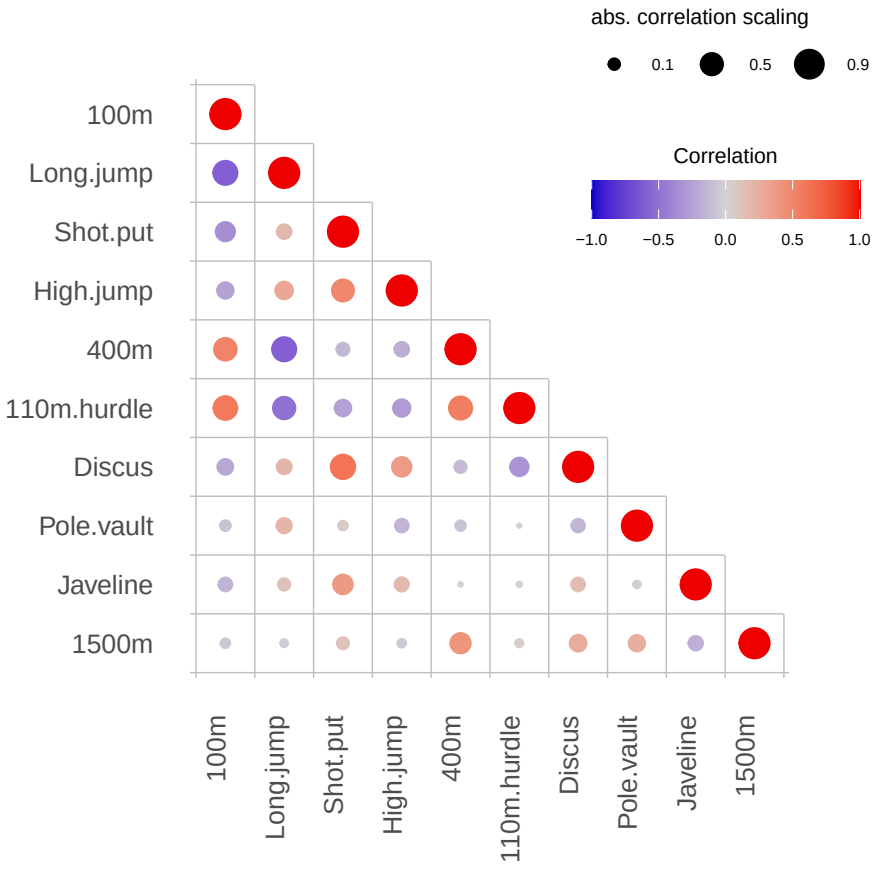



```
pca_loadings <- (pca_out$rotation*2.5) |>
  as_tibble(rownames = "Variable")
pca_out$x |>
  as_tibble() |>
  cbind(rawdata) |>
  ggplot(aes(PC1,PC3,color = species))+
  geom_point()+
  geom_text(
    data=pca_loadings,
    color = "black",
    aes(label = Variable),
            hjust = 0)+
  geom_segment(aes(xend = 0, yend = 0),
               data = pca_loadings,
               color = "black",
               arrow = arrow(end = 'first',
                             length = unit(.05,
                                           'npc')))+
  scale_y_continuous(expand = expansion(.2),
                     breaks = seq(-10,10,1),
                     sec.axis = sec_axis(
                       ~(./2.5),
                       name = "Loading",
```

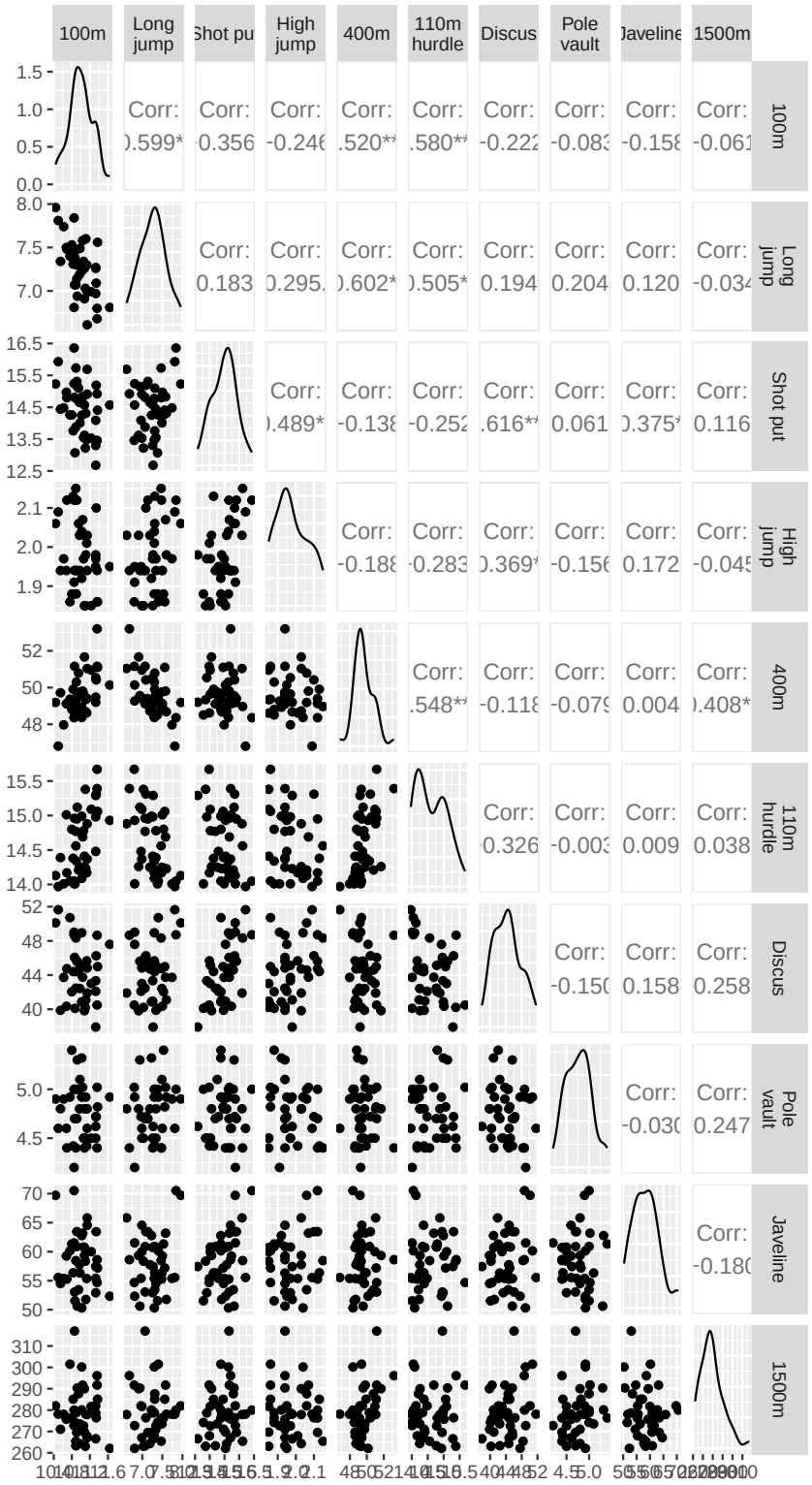
```
                      breaks = seq(-3,10,1)/2.5))+
  scale_x_continuous(expand = expansion(
    mult = c(.1,.25)),
    breaks = seq(-10,10,1),
    sec.axis = sec_axis(
      ~(./2.5), name = "Loading",
      breaks = seq(-10,10,1)/2.5))
```




```
# decathlon
data("decathlon")
cortestR(decathlon |> select(1:10),
  split = T
) |>
  pluck("corout") |>
  ggcormat(maxpoint = 5)
```




```
ggpairs(decathlon |> select(1:10) |>
  rename_with(\(x) str_replace(x, "\\.", " ") |>
    str_wrap(8)))
```


100m
Long jump
Shot put
High jump
400m
110m hurdle
Discus
Pole vault
Javeline
1500m

```r
pca_out_deca <- prcomp(decathlon |> select(1:10),
  center = T, scale. = T
)
summary(pca_out_deca)
```

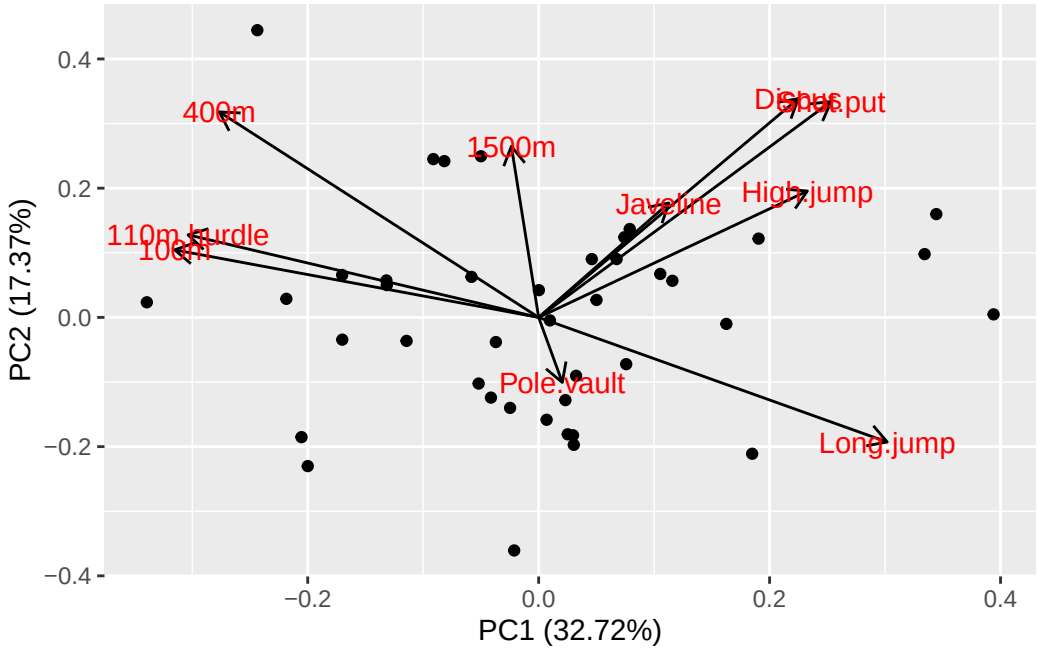
```
Importance of components:
                          PC1    PC2    PC3    PC4     PC5     PC6     PC7
Standard deviation     1.8088 1.3180 1.1853 1.0280 0.82751 0.77412 0.67174
Proportion of Variance 0.3272 0.1737 0.1405 0.1057 0.06848 0.05993 0.04512
Cumulative Proportion  0.3272 0.5009 0.6414 0.7471 0.81556 0.87548 0.92061
                           PC8     PC9    PC10
Standard deviation     0.62998 0.46348 0.42688
Proportion of Variance 0.03969 0.02148 0.01822
Cumulative Proportion  0.96030 0.98178 1.00000
```

```r
pca_out_deca$rotation |>
  as_tibble(rownames = "Exercise") |>
  mutate(across(
    -Exercise,
    ~ case_when(
      abs(.) < .25 ~ 0,
      TRUE ~ .
    )
  )) |>
  select(1:6)
```

```
# A tibble: 10 x 6
   Exercise       PC1    PC2    PC3    PC4    PC5
   <chr>        <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
 1 100m        -0.428  0      0      0      0.365
 2 Long.jump    0.410 -0.262  0      0      0
 3 Shot.put     0.344  0.454  0      0      0
 4 High.jump    0.316  0.266  0      0      0.671
 5 400m        -0.376  0.432  0      0      0
 6 110m.hurdle -0.413  0      0     -0.283  0
 7 Discus       0.305  0.460  0      0.253  0
 8 Pole.vault   0      0      0.584 -0.536  0.399
 9 Javeline     0      0     -0.329 -0.693 -0.369
10 1500m        0      0.360  0.660  0      0
```

```r
autoplot(pca_out_deca,
  data = decathlon,
```

```
  loadings = TRUE, loadings.colour = "black",
  loadings.label = TRUE, loadings.label.size = 4
)
```



26 Cluster analysis

26.1 kmeans

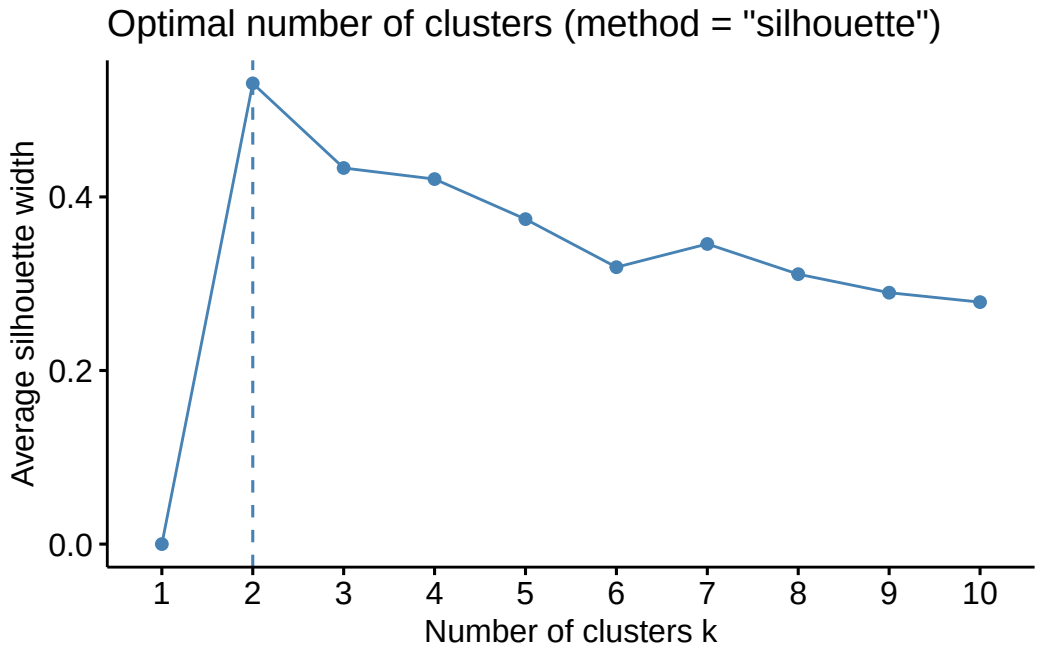
```
pacman::p_load(
  conflicted, tidyverse,
  wrappedtools,
  palmerpenguins,
  ggfortify, GGally, ggdendro, ggpubr,
  factoextra,
  caret, clue,
  dendextend, circlize,
  easystats, NbClust, mclust,
  tidymodels, tidyclust
)

# conflict_scout()
conflicts_prefer(
  dplyr::slice,
  dplyr::filter,
  palmerpenguins::penguins
)
rawdata <- penguins |>
  na.omit()

predvars <- ColSeeker(namepattern = c("_mm", "_g"))

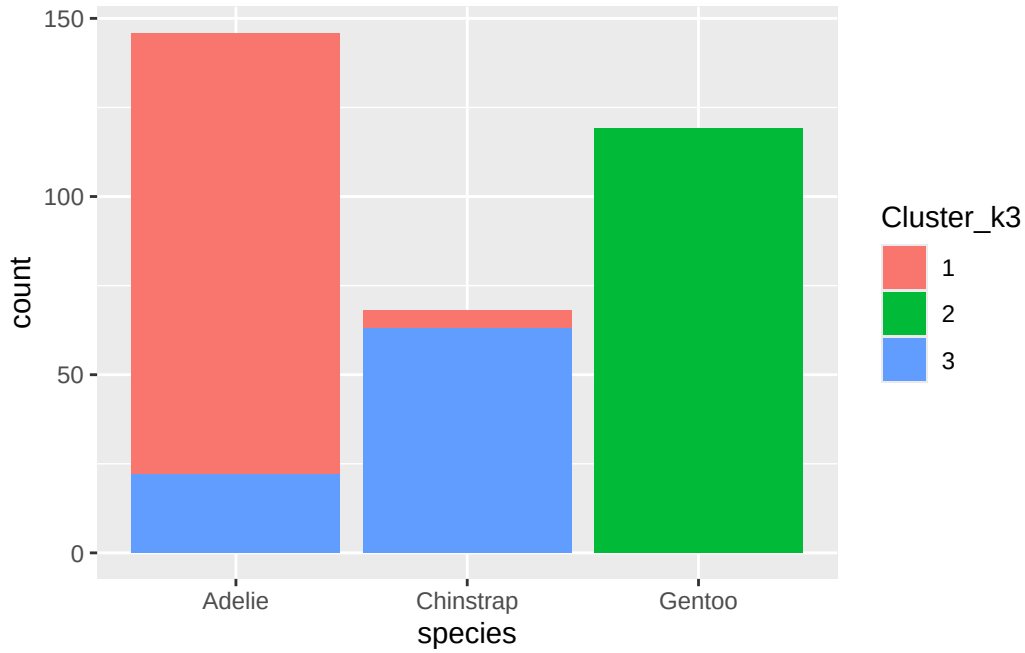
norm_rec <-
  recipe(rawdata) |>
  update_role(all_of(predvars$names), new_role = "predictor") |>
  step_normalize(all_numeric_predictors())
rawdata <-
  norm_rec |>
  prep() |>
  bake(new_data = rawdata)
fviz_nbclust(rawdata |> select(predvars$names),
```

```
  FUNcluster = kmeans
)
```

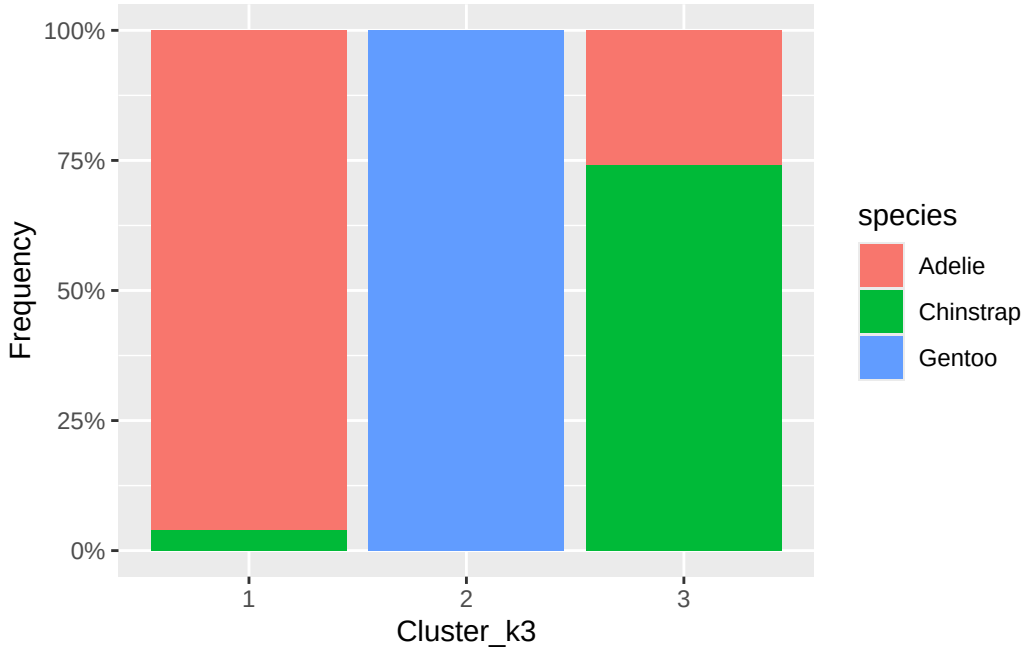



```
kmeans_out <- kmeans(rawdata |> select(predvars$names),
  centers = 3,iter.max = 100
)

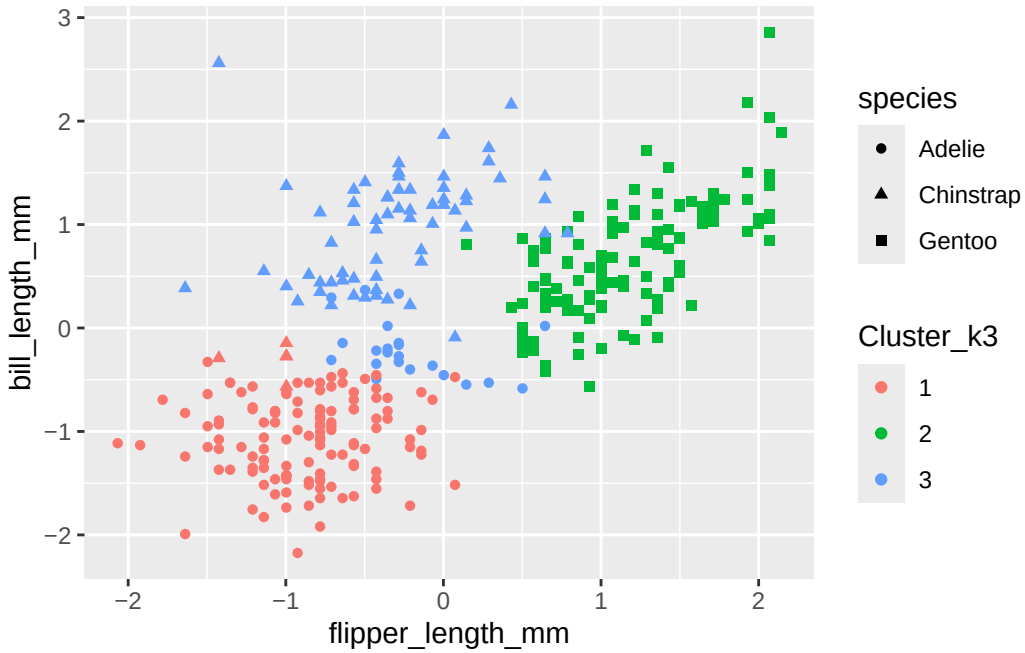
rawdata <-
  mutate(rawdata, Cluster_k3 = kmeans_out$cluster |> as.factor())
rawdata |>
  ggplot(aes(species, fill = Cluster_k3)) +
  geom_bar()
```



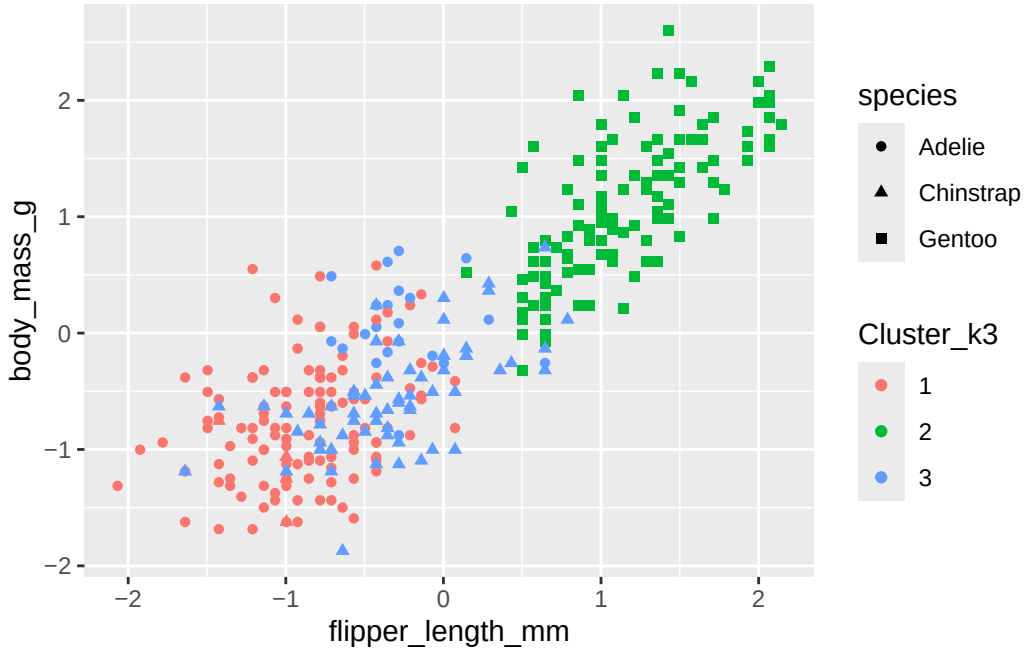

```
rawdata |>
  ggplot(aes(fill = species, x = Cluster_k3)) +
  geom_bar(position = "fill") +
  scale_y_continuous(name = "Frequency", labels = scales::percent)
```



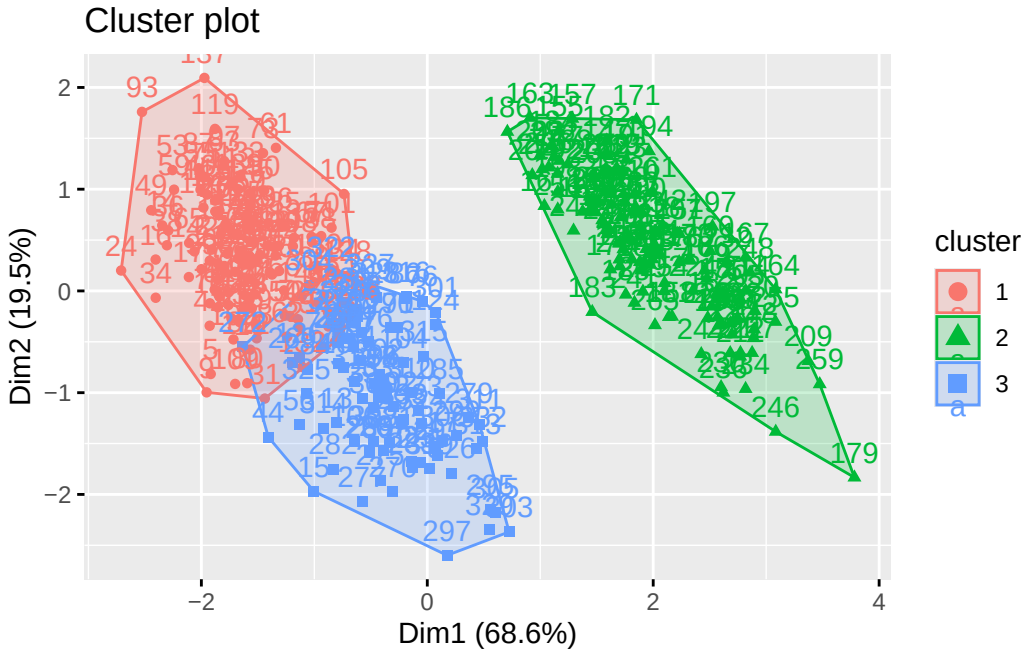
```
rawdata |>
  ggplot(aes(flipper_length_mm, bill_length_mm,
    shape = species, color = Cluster_k3
  )) +
  geom_point()
```




```
rawdata |>
  ggplot(aes(flipper_length_mm, body_mass_g,
    shape = species, color = Cluster_k3
  )) +
  geom_point()
```




```
fviz_cluster(kmeans_out, rawdata |> select(predvars$names))
```

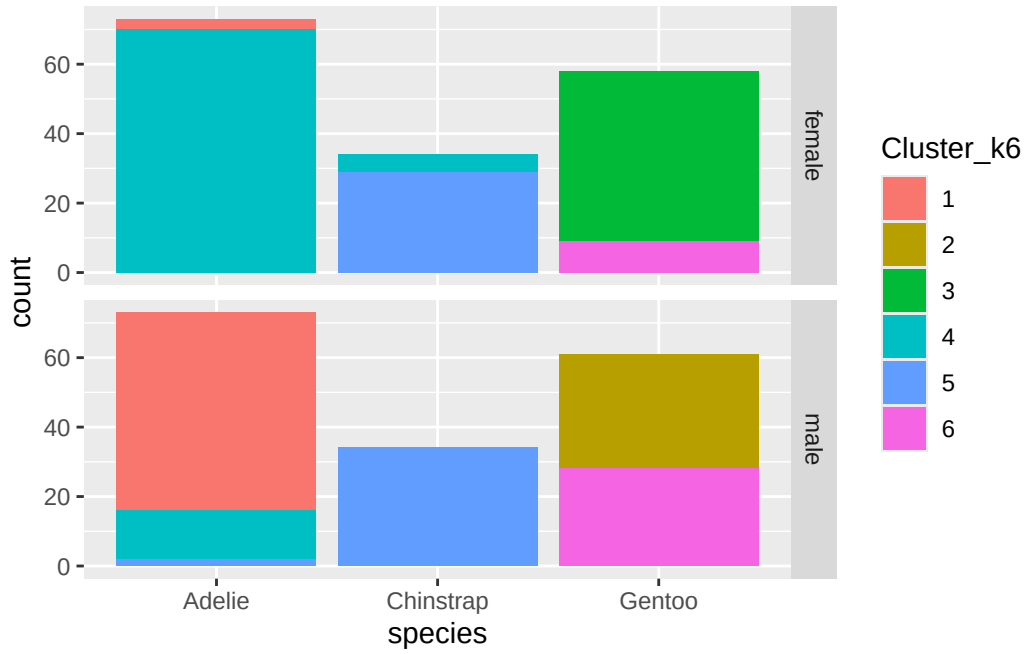



```
# predict(kmeans_out)
sample(clue::cl_predict(kmeans_out), 10)
```

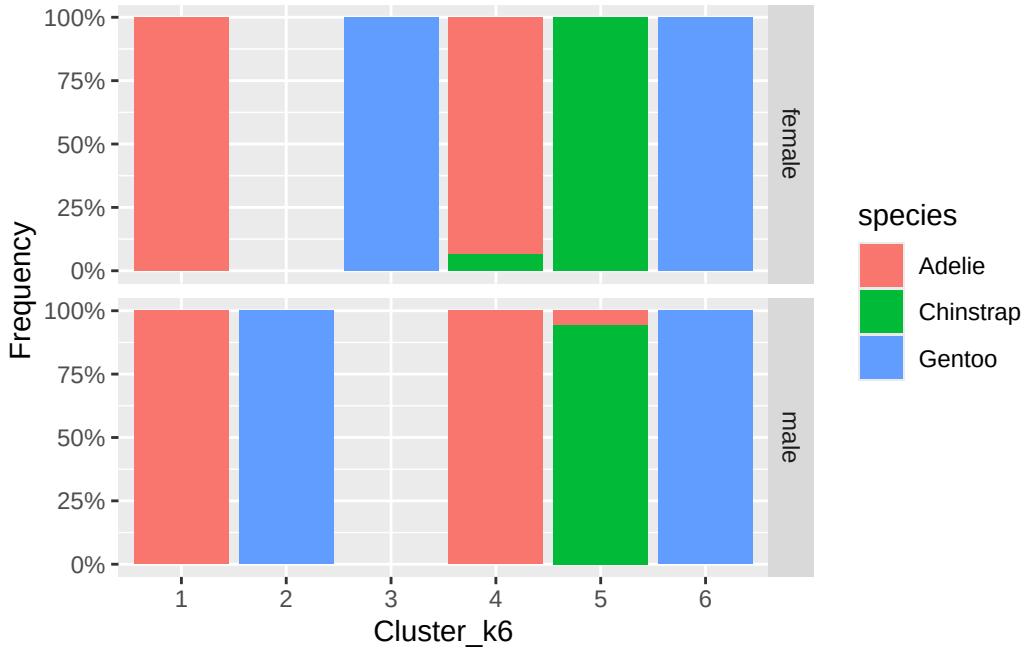
```
 [1] 1 3 2 1 1 1 1 1 2 1
```

```
kmeans_out6 <- kmeans(rawdata |> select(predvars$names),
  centers = 6, iter.max = 100
)

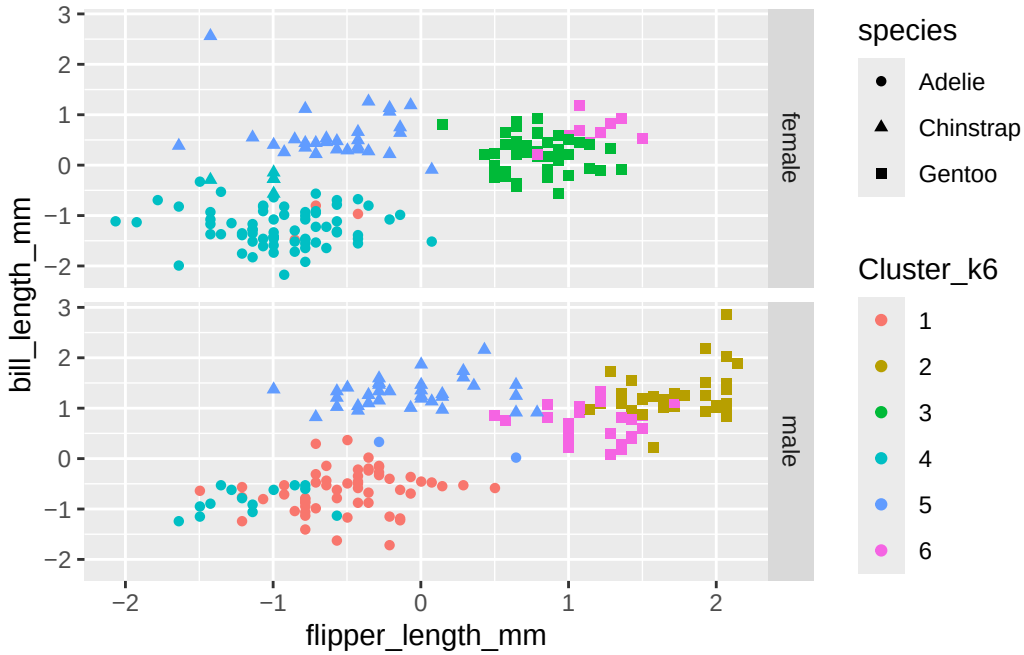
rawdata <-
  mutate(rawdata, Cluster_k6 = kmeans_out6$cluster |> as.factor())
rawdata |>
  ggplot(aes(species, fill = Cluster_k6)) +
  geom_bar() +
  facet_grid(rows = vars(sex))
```




```
rawdata |>
  ggplot(aes(fill = species, x = Cluster_k6)) +
  geom_bar(position = "fill") +
  scale_y_continuous(name = "Frequency", labels = scales::percent) +
  facet_grid(rows = vars(sex))
```




```
rawdata |>
  ggplot(aes(flipper_length_mm, bill_length_mm,
    shape = species, color = Cluster_k6
  )) +
  geom_point() +
  facet_grid(rows = vars(sex))
```



26.2 HClust

```
penguins_scaled <-
  rawdata |>
  select(predvars$names)
# Compute Distance Matrix
# Hierarchical clustering starts by calculating the distance between every
#  pair of observations. The most common distance metric is Euclidean distance.

distance_matrix <- dist(penguins_scaled, method = "euclidean")

as.matrix(distance_matrix)[1:5, 1:5] # small portion of the matrix
```

```
          1         2         3         4         5
1 0.0000000 0.7564881 1.2481347 1.0757725 1.1661972
2 0.7564881 0.0000000 0.9965556 1.2772797 1.6607532
3 1.2481347 0.9965556 0.0000000 0.9753012 1.4665233
4 1.0757725 1.2772797 0.9753012 0.0000000 0.8771267
5 1.1661972 1.6607532 1.4665233 0.8771267 0.0000000
```

```
summary(distance_matrix)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.1089  1.5057  2.5986  2.5571  3.4774  7.2635
```

```r
# Perform Hierarchical Clustering
#    `hclust()` from base R.
#    `method` parameter specifies the agglomeration method (linkage method).
#    Common methods: "ward.D2", "complete", "average", "single".
#    "ward.D2" is often preferred as it tends to produce more balanced clusters.

hc_result <- hclust(distance_matrix, method = "ward.D2")

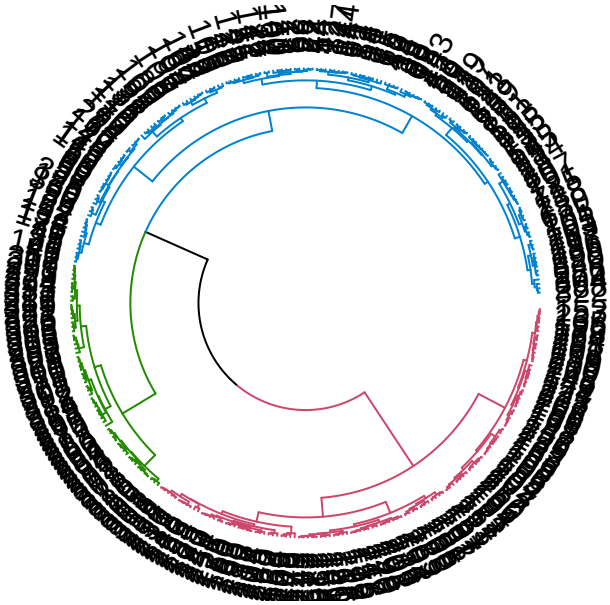
hc_result
```

```

Call:
hclust(d = distance_matrix, method = "ward.D2")

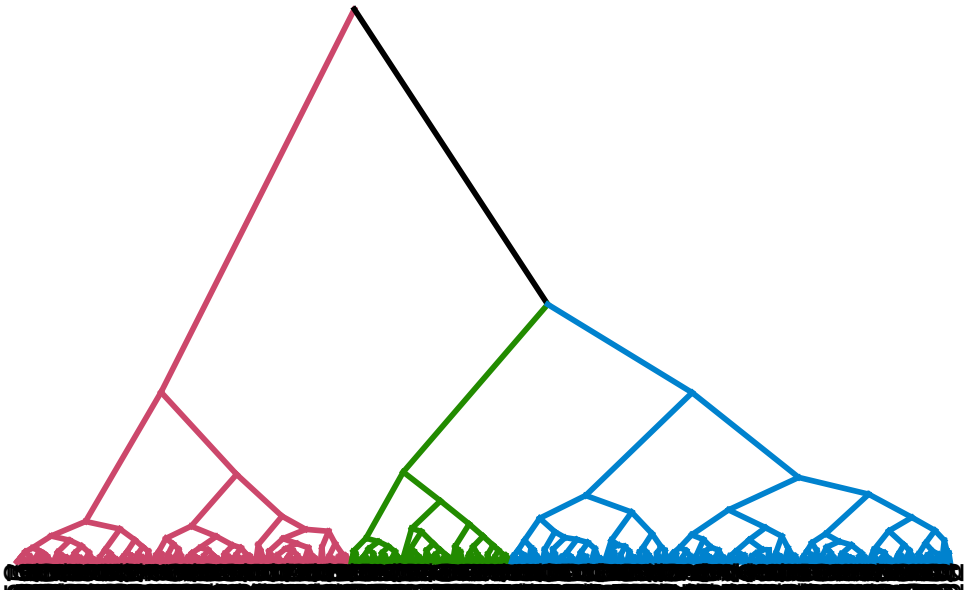
Cluster method   : ward.D2
Distance         : euclidean
Number of objects: 333
```

```r
dend <- as.dendrogram(hc_result)
dend <- color_branches(dend, k = 3)
dendextend::circlize_dendrogram(dend,
  facing = "outside"
)
```



```
dendextend::as.ggdend(dend, type = "triangle")
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
i The deprecated feature was likely used in the dendextend package.
  Please report the issue at <https://github.com/talgalili/dendextend/issues>.
```



```
# Cut the Dendrogram and Extract Clusters
#    To form distinct clusters, you "cut" the dendrogram at a certain height
#    or specify the desired number of clusters (k).

num_clusters <- 3 # Let's aim for 3 clusters, similar to the 3 penguin species

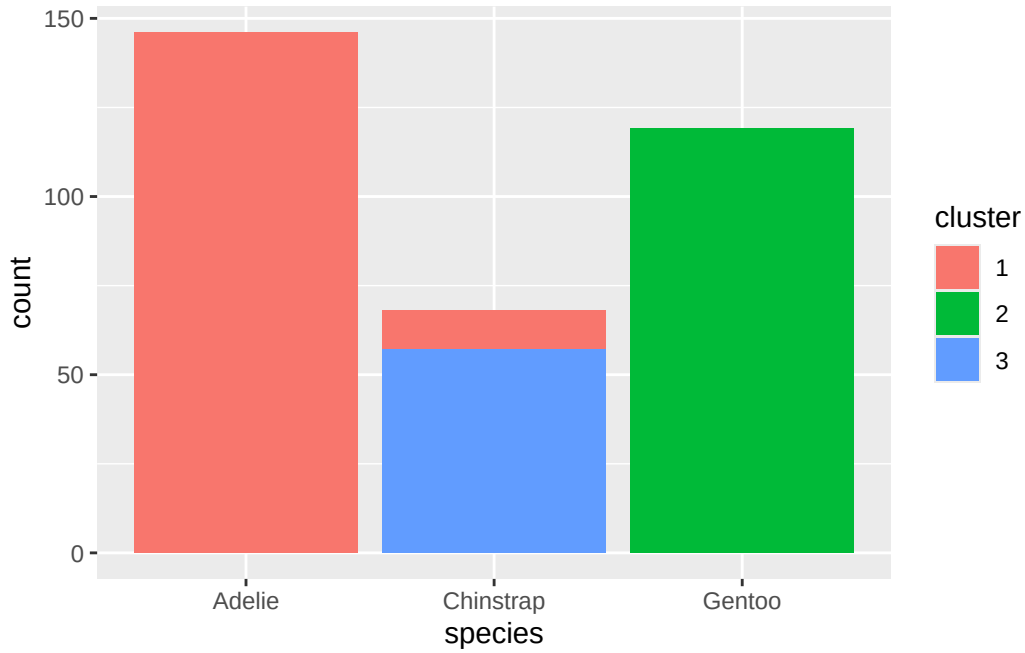
# Cut tree into k groups
clusters <- cutree(hc_result, k = num_clusters)

sample(clusters, 10)
```

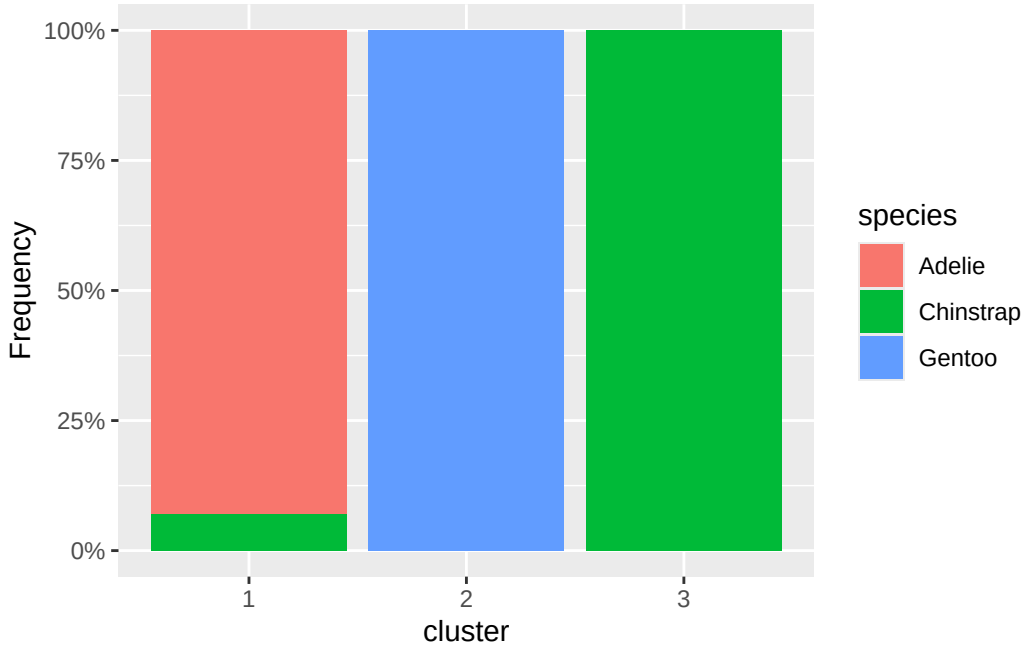
```
 [1] 2 1 1 2 1 3 1 1 1 1
```

```
# Add cluster assignments back to the original (non-scaled) data for easier interpretation
rawdata <- rawdata |>
  mutate(cluster = as.factor(clusters))

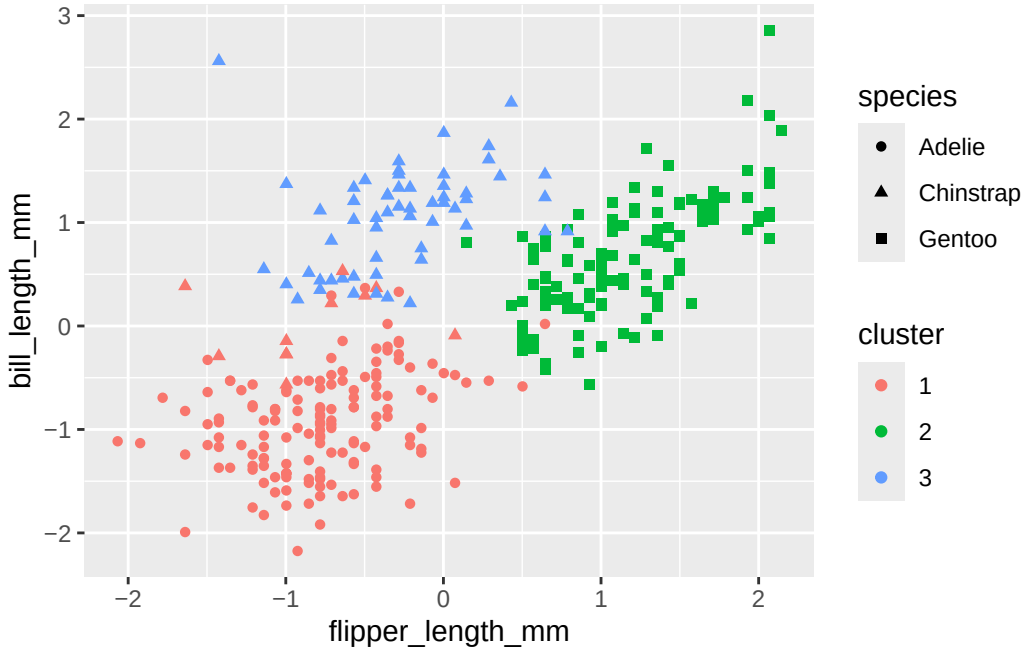
rawdata |>
  ggplot(aes(species, fill = cluster)) +
  geom_bar()
```




```
rawdata |>
  ggplot(aes(fill = species, x = cluster)) +
  geom_bar(position = "fill") +
  scale_y_continuous(name = "Frequency", labels = scales::percent)
```



```
rawdata |>
  ggplot(aes(flipper_length_mm, bill_length_mm,
    shape = species, color = cluster
  )) +
  geom_point()
```




```
confusionMatrix(rawdata$cluster, reference = rawdata$species |>
  as.numeric() |> factor(
    levels = c(1, 3, 2),
    labels = c(1, 2, 3)
  ))
```

```
Confusion Matrix and Statistics

          Reference
Prediction   1   2   3
         1 146   0  11
         2   0 119   0
         3   0   0  57

Overall Statistics

               Accuracy : 0.967
```

```
                 95% CI : (0.9417, 0.9834)
    No Information Rate : 0.4384
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.9476

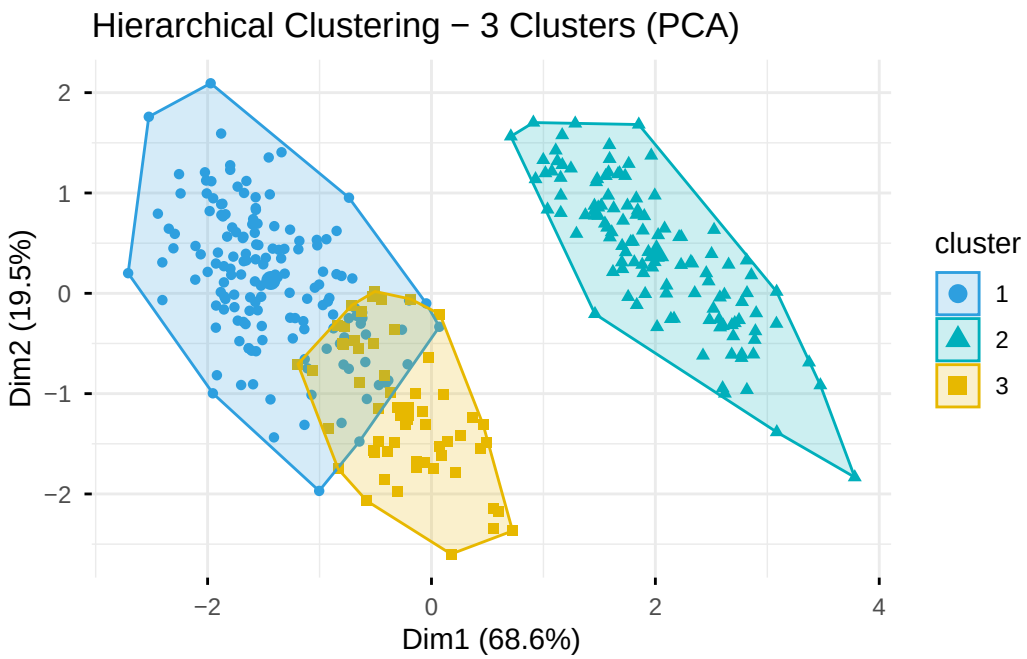
 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: 1 Class: 2 Class: 3
Sensitivity            1.0000   1.0000   0.8382
Specificity            0.9412   1.0000   1.0000
Pos Pred Value         0.9299   1.0000   1.0000
Neg Pred Value         1.0000   1.0000   0.9601
Prevalence             0.4384   0.3574   0.2042
Detection Rate         0.4384   0.3574   0.1712
Detection Prevalence   0.4715   0.3574   0.1712
Balanced Accuracy      0.9706   1.0000   0.9191
```

```r
# Visualize the Clusters on a Scatter Plot using factoextra
#    `fviz_cluster()` helps to visualize the clusters formed on a scatter plot
#    (using PCA for dimensionality reduction if data has > 2 dimensions).

fviz_cluster(list(data = penguins_scaled, cluster = clusters),
  geom = "point",
  ellipse.type = "convex", # Draw convex hulls around clusters
  palette = c("#2E9FDF", "#00AFBB", "#E7B800"), # Use same colors as dendrogram
  main = paste0("Hierarchical Clustering - ", num_clusters, " Clusters (PCA)"),
  # xlab = "Principal Component 1",
  # ylab = "Principal Component 2",
  ggtheme = theme_minimal()
)
```



26.3 Unified from easystats

```
easycluster1 <- cluster_analysis(penguins_scaled,
  n = 3,
  standardize = FALSE
)
print(easycluster1)
```

```
# Clustering Solution

The 3 clusters accounted for 64.31% of the total variance of the original data.

Cluster | n_Obs | Sum_Squares | bill_length_mm | bill_depth_mm
-------------------------------------------------------------
1       |   213 |      403.28 |          -0.37 |          0.61
2       |    62 |       25.49 |           0.28 |         -1.45
3       |    58 |       45.24 |           1.07 |         -0.68

Cluster | flipper_length_mm | body_mass_g
-----------------------------------------
1       |             -0.65 |       -0.62
```

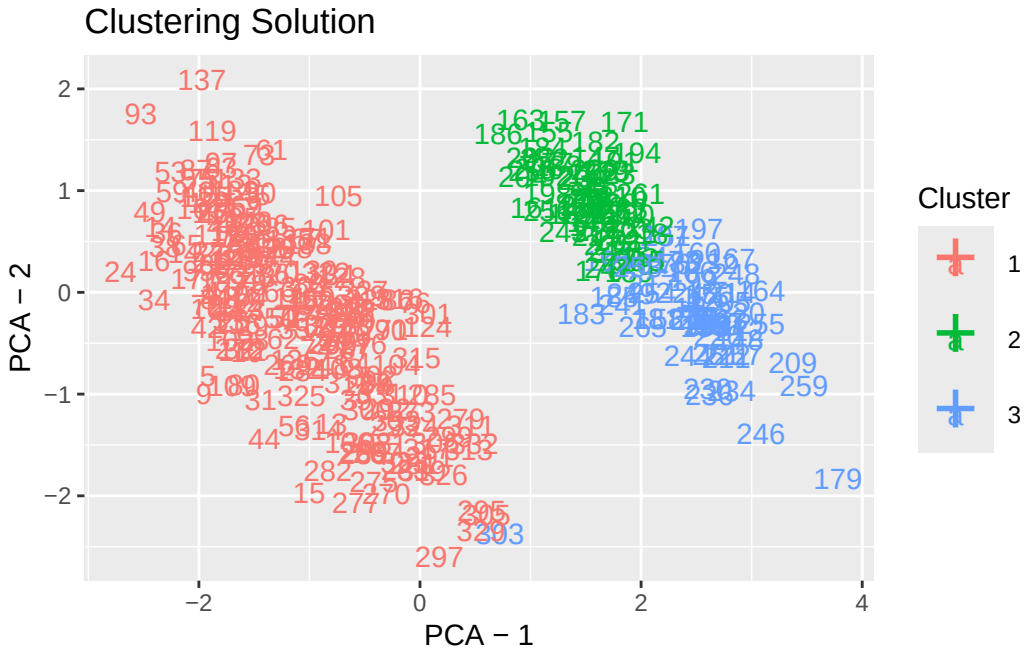
```
2       |              0.85 |        0.61
3       |              1.48 |        1.61

# Indices of model performance

Sum_Squares_Total | Sum_Squares_Between | Sum_Squares_Within |    R2
-----------------------------------------------------------------------
1328              |             853.985 |            474.015 | 0.643

# You can access the predicted clusters via `predict()`.
```

```
plot(easycluster1)
```




```
easycluster2 <- cluster_analysis(penguins_scaled,
  standardize = FALSE,
  method = "hclust",
  n = 6
)
print(easycluster2)
```

```
# Clustering Solution

The 6 clusters accounted for 83.82% of the total variance of the original data.
```

```
Cluster | n_Obs | Sum_Squares | bill_length_mm | bill_depth_mm
-------------------------------------------------------------
1       |    97 |       65.06 |          -1.11 |          0.29
2       |    54 |       39.63 |          -0.58 |          1.10
3       |    51 |       39.55 |           0.87 |          0.55
4       |    49 |       16.73 |           0.21 |         -1.56
5       |    70 |       49.43 |           0.96 |         -0.78
6       |    12 |        4.50 |           1.40 |          1.36

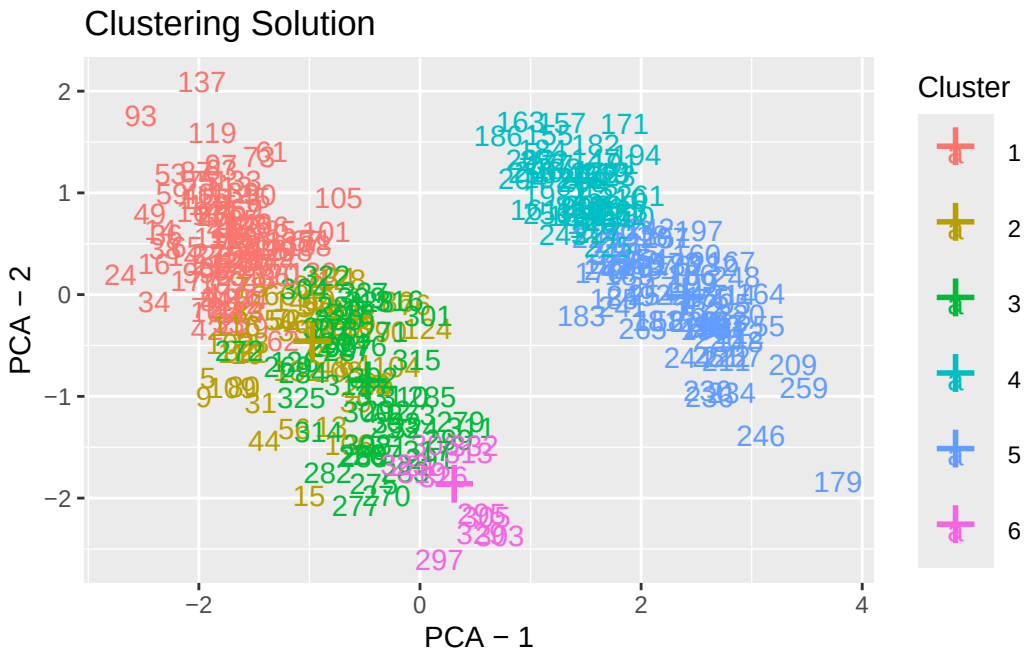
Cluster | flipper_length_mm | body_mass_g
---------------------------------------
1       |             -0.99 |       -0.96
2       |             -0.43 |       -0.05
3       |             -0.47 |       -0.69
4       |              0.77 |        0.53
5       |              1.43 |        1.50
6       |              0.37 |        0.01

# Indices of model performance

Sum_Squares_Total | Sum_Squares_Between | Sum_Squares_Within |    R2
-----------------------------------------------------------------------
1328              |            1113.100 |            214.900 | 0.838

# You can access the predicted clusters via `predict()`.
```

```
plot(easycluster2)
```




```
# 1. Model Specification
kmeans_spec <- k_means(num_clusters = 3) |>
  set_engine("stats")

# 2. Preprocessing (Scaling is CRITICAL for K-means)
# done above
# 3. Workflow
kmeans_wflow <- workflow() |>
  add_model(kmeans_spec) |>
  add_formula(~.)

# 4. Fit
kmeans_fit <- kmeans_wflow |> fit(data = rawdata |>
  select(all_of(predvars$names)))
```